MOUNTAIN AI ▪ ТОМ I

ЛЕКЦИИ ПО МАШИННОМУ ОБУЧЕНИЮ

Обучение с *подкреплением*

От бандитов до RLHF

К. К. ПЧЕЛИН

На основе лекций ▪ Москва, 2026

github.com/pyshka501/rl-textbook ▪ t.me/mountainai_info

ПЕРВОЕ ИЗДАНИЕ

Книга свёрстана с помощью LaTeX.

Copyright © 2026 На основе лекций К. К. Пчелина
Лицензия: Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0)

Печать на продажу, коммерческое издание и производные произведения этой лицензией не разрешены. Для таких целей требуется письменное разрешение автора.

Предисловие

Эта книга выросла из убеждения, что две великие традиции обучения с подкреплением — классическая математическая теория и современная практика выравнивания больших языковых моделей — заслуживают быть изложенными как единая, связная история. В последние годы идеи, десятилетиями существовавшие лишь на страницах теории управления и динамического программирования, вырвались в центр разработки систем искусственного интеллекта, лежа в основе обучения систем, которые пишут, рассуждают и ведут диалог. Именно этому пути посвящена настоящая книга.

Замечание об источниках вдохновения и оригинальности. Каждый учебник опирается на труды предшественников, и эта книга не исключение. Мы хотим отдать должное двум книгам, которые сформировали облик всей области и, неизбежно, нашу собственную точку зрения, — одновременно разъяснив, чем настоящая книга от них отличается.

Первая — *Reinforcement Learning: An Introduction* Ричарда С. Саттона и Эндрю Г. Барто, стандартный справочник по классическому RL. Саттон и Барто заложили педагогическую последовательность — от бандитов через МДП к методам временных разностей — которой с тех пор следует практически каждый курс. Наше изложение в частях I и II строится по той же логической дуге, но отличается акцентами и подачей: мы приводим более строгий анализ сходимости с учётом недавних результатов, включаем обобщённое оценивание преимущества (GAE) и PPO в основное повествование, а не в качестве дополнений, и мотивируем каждую классическую идею с прицелом на её будущую роль в обучении языковых моделей. Эти решения отражают уроки, извлечённые при разработке и преподавании университетских курсов по RL, а также при применении этих алгоритмов в производственных условиях.

Вторая — *The RLHF Book* Натана Ламберта, наиболее полное из написанных практических руководств по обучению с подкреплением на основе обратной связи от людей для языковых моделей. Ламберт показал, что алгоритмические идеи, лежащие в основе ChatGPT, Claude и их преемников, поддаются точному и доступному изложению. Части III и IV нашей книги охватывают пересекающуюся тематику — моделирование вознаграждения, оптимизацию предпочтений, PPO для LLM — но расширяют обсуждение в направлениях, обусловленных нашим собственным исследовательским и инженерным опытом: мы даём единое формальное описание, связывающее DPO, IPO и KTO через RL-цель с KL-ограничением, и рассматриваем ORPO как самостоятельный подход без эталонной модели; посвящаем отдельные главы GRPO и обучению с подкреплением на верифицируемых вознаграждениях (RLVR) — темам, появившимся после первого издания книги Ламберта; а также включаем обширную главу о многошаговых агентных системах, которая перебрасывает мост между теорией RL и нарождающейся практикой языковых агентов, использующих инструменты.

Более широко: эта книга — не синтез двух упомянутых работ, а попытка рассказать *новую* историю: единое непрерывное повествование, которое начинается с принципа оптимальности Беллмана и заканчивается обучающими пайплайнами, лежащими в основе современных моделей-рассуждения (reasoning models). Эта книга опирается на практический опыт масштабного обучения с RLHF, отладки моделей вознаграждения и практических трудностей, которые не может передать ни один сугубо теоретический текст. Там, где нам удаётся сделать этот мост ощущающимся естественным, — заслуга принадлежит сообществу, чьи открытые исследования сделали это возможным. Там, где мы не дотягиваем, — ответственность полностью лежит на нас.

Цели этой книги. Центральная задача настоящего тома — выстроить единый, непрерывный мост между математическими основами, заложенными Беллманом, Саттоном и Барто, и практическими методами, которые сегодня определяют обучение и выравнивание больших языковых моделей. Путь от принципа оптимальности Беллмана до работающего RLHF-пайплайна короче, чем может показаться, но требует подлинного владения обоими мирами. Практик, знающий только инженерные рецепты, упускает теоретические гарантии и диагностическую интуицию, которую даёт теория. Теоретик, не соприкоснувшийся с реальностью обучения языковых моделей — нестабильностью моделей вознаграждения, чувствительностью гиперпараметров PPO, сложностью формализации предпочтений людей — плохо подготовлен к тому, чтобы вносить вклад в наиболее важное приложение RL на сегодняшний день.

Части I и II данной книги (охватывающие бандитов, МДП, динамическое программирование, методы Монте-Карло, обучение на основе временных разностей, аппроксимацию функций, градиенты стратегии и методы «актор-критик») написаны под влиянием строгого изложения Саттона и Барто и могут быть прочитаны полностью самостоятельно как законченный учебник по классическому RL. Части III и IV опираются на этот фундамент и охватывают моделирование вознаграждения, RLHF, прямую оптимизацию предпочтений (DPO), групповую оптимизацию относительной стратегии (GRPO), мультиагентные системы и открытые направления исследований, существенно используя практические идеи Ламберта.

Для кого эта книга. Мы писали наш труд, имея в виду три пересекающиеся аудитории.

Аспиранты в области машинного обучения и ИИ, желающие получить математически честное изложение обучения с подкреплением, которое не останавливается на классических результатах, а доводит их до границы выравнивания языковых моделей.

Инженеры и специалисты-практики в области МО, знакомые с RLHF в контексте языковых моделей и желающие понять теоретический механизм, лежащий в основе реализации, чтобы уверенно отлаживать, расширять и улучшать его.

Исследователи, переходящие от классического обучения с учителем или без учителя к обучению с подкреплением, включая тех, чьи основные интересы лежат в области выравнивания, безопасности или оценки, — и кому нужна прочная база в базовых концепциях перед погружением в исследовательскую литературу.

Книга не рассчитана на полных новичков в машинном обучении: предполагается некоторое предварительное знакомство с идеями этой области. Однако относительно обучения с подкреплением книга является самодостаточной: никаких предварительных знаний в области RL не требуется.

Необходимые знания. Предполагается, что читатель уверенно владеет базовой теорией вероятностей (случайные величины, математическое ожидание, условная вероятность, теорема Байеса), линейной алгеброй (векторы, матрицы, собственные значения) и одно- и многомерным математическим анализом (производные, градиенты, правило цепочки). Знакомство с ключевыми концепциями обучения с учителем — функциями потерь, градиентным спуском, нейронными сетями, переобучением — сделает части III и IV значительно более доступными, хотя мы напоминаем ключевые идеи по мере необходимости. Знание Python полезно для реализационно-ориентированного материала в приложениях, но не является обязательным для понимания теоретического изложения.

Как читать эту книгу. Четыре части книги образуют естественную последовательность, однако читатели с разной подготовкой могут начинать с разных мест.

Часть I (Основы) развивает классическую теорию с нуля: многорукие бандиты, марковские процессы принятия решений, динамическое программирование, методы Монте-Карло и обучение на основе временных разностей. Читателю без предварительного знакомства с RL следует прорабатывать эти главы по порядку.

Часть II (Аппроксимация функций и глубокое RL) переходит к задачам, где табличные методы уже неприменимы. Она охватывает линейную и нелинейную аппроксимацию функции ценности, методы градиента стратегии и семейство «актор-критик», завершая изложение алгоритмом PPO. Вместе с частью I этот материал представляет собой законченное введение в классическое и глубокое RL.

Часть III (RL для языковых моделей, главы 10–14) применяет всё изложенное в частях I и II к современной задаче обучения и выравнивания больших языковых моделей. Читатели, уже владеющие классическим RL, могут переходить непосредственно сюда. В этих главах рассматриваются формирование вознаграждения и модели вознаграждения (глава 10), пайплайн обучения языковых моделей от предобучения до SFT (глава 11), прямая оптимизация предпочтений и её варианты (глава 12), групповая оптимизация относительной стратегии и онлайн-RL с верифицируемыми вознаграждениями (глава 13), а также инженерная реализация RLHF в промышленном масштабе (глава 14).

Часть IV (Горизонты, главы 15–17) исследует фронтир области: рассуждение, использование инструментов и дообучение с подкреплением (глава 15); мультиагентные системы (глава 16); обзор открытых проблем и новых направлений (глава 17).

Каждая глава завершается набором упражнений, диапазон которых охватывает как простые проверочные задания, так и более сложные доказательства и задачи проектирования. Мы рекомендуем выполнять хотя бы несколько упражнений к каждой главе: обучение с подкреплением — это, что вполне символично, предмет, который лучше всего понимается через практику.

Сопроводительный код и ноутбуки. К книге прилагается открытый репозиторий на GitHub, содержащий практические Jupyter-ноутбуки — по одному на каждую главу. В этих ноутбуках представлены работающие реализации всех основных алгоритмов, обсуждаемых в тексте, — от ε-жадных бандитов и Q-обучения до DQN, PPO, DPO и GRPO, — а также визуализации и управляемые эксперименты. Все ноутбуки рассчитаны на работу на бесплатных платформах, таких как Google Colab или Kaggle, и не требуют специализированного оборудования. Репозиторий также включает подробные решения упражнений

с полными доказательствами и выводами, а также инфраструктуру для переводов на другие языки силами сообщества.

Сопроводительный репозиторий доступен по адресу:

https://github.com/pyshka501/rl-textbook

Мы рекомендуем работать с ноутбуками параллельно с изучением соответствующих глав. Сочетание математической строгости текста и практического экспериментирования с кодом является, по нашему опыту, наиболее эффективным способом выработать подлинное мастерство в области обучения с подкреплением.

Мы надеемся, что эта книга послужит вам хорошим проводником — где бы вы ни находились на пути от классической теории к современной практике выравнивания.

Москва, 2026

Оглавление

Предисловие iv

Обозначения xvii

I Основы 1

1 Введение в RL 2
1.1 Что такое обучение с подкреплением? . . . 2
1.2 Ключевые проблемы обучения с подкреплением . . . 2
1.3 Краткая история обучения с подкреплением . . . 3
1.4 Обучение с подкреплением и другие парадигмы машинного обучения . . . 3
1.5 Применения обучения с подкреплением . . . 4
1.6 Интерфейс агент–среда . . . 5
1.7 Обзор книги . . . 5
Упражнения . . . 5

2 Многорукие бандиты 7
2.1 Постановка задачи . . . 7
2.2 Ценности действий, оптимальные действия и сожаление . . . 7
2.3 Жадные и ε-жадные стратегии . . . 8
2.4 Выборочные средние и инкрементные обновления . . . 9
2.5 Верхние доверительные границы (UCB) . . . 10
2.6 Выборка Томпсона . . . 12
2.7 Нижняя оценка Лаи–Роббинса . . . 14
2.8 Контекстные бандиты . . . 15
2.9 Эмпирическое сравнение . . . 17
2.10 Резюме . . . 18
Упражнения . . . 19

3 MDP и уравнение Беллмана 21
3.1 Марковское свойство . . . 21
3.2 Формальное определение MDP . . . 22
3.3 Эпизоды, доходы и коэффициент дисконтирования . . . 23
3.4 Политики . . . 24
3.5 Функции ценности . . . 25
3.6 Уравнение Беллмана . . . 26
3.7 Оптимальные функции ценности и уравнение оптимальности Беллмана . . . 28
3.8 Оператор Беллмана . . . 28
3.9 Числовой пример: MDP студента . . . 29
3.10 Резюме . . . 31
Упражнения . . . 31

4 Динамическое программирование 33
4.1 Оценка политики (предсказание) . . . 33
4.2 Улучшение политики . . . 35
4.3 Итерация политики . . . 36
4.4 Итерация ценности . . . 37
4.5 Обобщённая итерация политики . . . 39

4.6 Асинхронное динамическое программирование . . . 39
4.7 Эффективность и ограничения динамического программирования . . . 40
4.8 Числовой пример: аренда автомобилей Джека . . . 40
4.9 Резюме . . . 41
Упражнения . . . 41

5 Методы Монте-Карло 43
5.1 Предсказание методом Монте-Карло . . . 43
5.2 Разобранный пример: Блэкджек . . . 46
5.3 Оценка функций ценности действий методом Монте-Карло . . . 47
5.4 Управление методом Монте-Карло с исследующими стартами . . . 48
5.5 Управление методом Монте-Карло на текущей стратегии . . . 48
5.6 Метод Монте-Карло на другой стратегии с взвешиванием по важности . . . 50
5.7 Управление методом Монте-Карло на другой стратегии . . . 52
5.8 Сводка и перспективы . . . 52
Упражнения . . . 52

6 Обучение с временной разностью 54
6.1 Предсказание TD(0) . . . 54
6.1.1 От метода Монте-Карло к временной разности . . . 54
6.1.2 Ошибка TD . . . 55
6.1.3 Свойство нулевого среднего ошибки TD . . . 55
6.1.4 Алгоритм TD(0) . . . 55
6.1.5 Сходимость табличного TD(0) . . . 56
6.1.6 Разобранный пример: случайное блуждание . . . 56
6.2 Преимущества методов TD . . . 56
6.3 SARSA: управление TD на текущей стратегии . . . 57
6.3.1 От предсказания к управлению . . . 57
6.3.2 Обновление SARSA . . . 57
6.3.3 Сходимость SARSA . . . 58
6.3.4 Разобранный пример: хождение по скалам . . . 58
6.4 Q-learning: управление TD на другой стратегии . . . 58
6.4.1 Обновление Q-learning . . . 58
6.4.2 Сходимость Q-learning . . . 59
6.4.3 Смещение максимизации . . . 59
6.5 Expected SARSA . . . 59
6.6 Double Q-learning . . . 60
6.7 Сравнение: SARSA, Q-learning и Expected SARSA . . . 61
6.8 n-шаговые методы TD . . . 61
6.8.1 n-шаговая отдача . . . 61
6.8.2 Компромисс смещение–дисперсия в n-шаговом TD . . . 62
6.8.3 n-шаговый SARSA . . . 62
6.9 TD(λ) и следы приемлемости . . . 62
6.9.1 λ-отдача . . . 62
6.9.2 Взгляд вперёд на TD(λ) . . . 63
6.9.3 Следы приемлемости . . . 63
6.9.4 Взгляд назад на TD(λ): онлайновый алгоритм . . . 64
6.9.5 Эквивалентность взглядов вперёд и назад . . . 64
6.9.6 Практические соображения . . . 65
6.9.7 Разобранный пример: случайное блуждание с TD(λ) . . . 65
6.10 Обобщённая оценка преимущества (GAE) . . . 65
6.10.1 Функция преимущества . . . 65
6.10.2 n-шаговые оценки преимущества . . . 66
6.10.3 GAE: преимущество с весами по λ . . . 66
6.10.4 Частные случаи и компромисс смещение–дисперсия . . . 67
6.10.5 Связь с TD(λ) . . . 67
6.11 Анализ смещения, дисперсии и сходимости . . . 67
6.11.1 Формальный анализ смещения . . . 67
6.11.2 Анализ дисперсии . . . 67
6.11.3 Разложение среднеквадратичной ошибки . . . 68

6.11.4 Сравнительная таблица . . . 68
6.11.5 Смертельная триада . . . 68
6.11.6 Сходимость при расписании Роббинса–Монро . . . 68
6.12 Итоги . . . 69
Упражнения . . . 69

II Аппроксимация функций и глубокое RL 72

7 Аппроксимация функций ценности 73
7.1 Почему табличные методы терпят неудачу . . . 74
7.1.1 Проклятие размерности . . . 74
7.1.2 Аргумент обобщения . . . 74
7.2 Линейная аппроксимация функций ценности . . . 74
7.2.1 Признаковые векторы и линейный анзац . . . 74
7.2.2 Тайловое кодирование . . . 75
7.2.3 Линейное TD(0) . . . 75
7.3 Проецированное уравнение Беллмана . . . 76
7.3.1 Геометрия аппроксимации функций . . . 76
7.3.2 Существование неподвижной точки TD . . . 77
7.3.3 Ошибка приближения в неподвижной точке TD . . . 77
7.4 Методы полуградиента . . . 77
7.4.1 Истинный градиентный спуск для предсказания . . . 77
7.4.2 Почему TD нарушает истинный градиентный спуск . . . 78
7.4.3 Алгоритм полуградиентного TD . . . 78
7.4.4 Сходимость полуградиентного TD . . . 78
7.5 Смертельная триада . . . 79
7.5.1 Три ингредиента нестабильности . . . 79
7.5.2 Контрпример Бэрда . . . 80
7.6 Deep Q-Networks . . . 80
7.6.1 От линейного к нелинейному приближению . . . 80
7.6.2 Буфер повторного воспроизведения опыта . . . 80
7.6.3 Целевые сети . . . 81
7.6.4 Алгоритм DQN . . . 81
7.6.5 Архитектура и результаты на Atari . . . 81
7.7 Расширения DQN . . . 82
7.7.1 Double DQN . . . 82
7.7.2 Дуэльный DQN . . . 82
7.7.3 Приоритетное повторное воспроизведение опыта . . . 83
7.8 Анализ сходимости . . . 83
7.8.1 Что можно доказать . . . 83
7.8.2 Методы градиентного TD . . . 83
7.8.3 Нелинейный случай . . . 83
7.9 Приложения и практические соображения . . . 84
7.9.1 Инженерия признаков против сквозного обучения . . . 84
7.9.2 Чувствительность к гиперпараметрам . . . 84
7.9.3 Практические рекомендации . . . 84
7.9.4 Исторические замечания . . . 85
Упражнения . . . 85

8 Методы градиента стратегии 87
8.1 Зачем оптимизировать стратегию напрямую? . . . 87
8.1.1 Ограничения подхода, основанного на функции ценности . . . 87
8.1.2 Стохастические стратегии иногда необходимы . . . 88
8.1.3 Целевая функция . . . 88
8.2 Теорема о градиенте стратегии . . . 88
8.2.1 Трюк логарифмического отношения . . . 88
8.2.2 Лемма о причинности и отдача с текущего момента . . . 89
8.2.3 Теорема о градиенте стратегии: компактная форма . . . 90
8.3 REINFORCE . . . 90

8.3.1 Алгоритм . . . 90
8.3.2 Разобранный пример . . . 91
8.4 Снижение дисперсии: базовые линии . . . 92
8.4.1 Теорема о базовой линии . . . 92
8.4.2 Оптимальная базовая линия . . . 92
8.4.3 Базовая линия на основе функции ценности состояния . . . 92
8.5 REINFORCE с обученной базовой линией . . . 93
8.6 RLOO: REINFORCE Leave-One-Out . . . 93
8.6.1 Мотивация: градиент стратегии в контексте языковых моделей . . . 93
8.6.2 Оценщик RLOO . . . 93
8.6.3 Несмещённость RLOO . . . 94
8.6.4 Снижение дисперсии в RLOO . . . 95
8.6.5 Связь с RLHF . . . 95
8.7 Натуральный градиент стратегии . . . 95
8.7.1 Проблема обычного градиента . . . 95
8.7.2 Матрица информации Фишера . . . 95
8.7.3 Практические соображения . . . 96
8.8 Анализ сходимости . . . 96
8.8.1 Сходимость REINFORCE . . . 96
8.8.2 Скорость сходимости . . . 97
8.8.3 Компромисс между смещением и дисперсией . . . 97
8.9 Сравнение оценщиков градиента стратегии . . . 97
Упражнения . . . 98

9 Методы актор-критик и PPO 100
9.1 Функция преимущества . . . 100
9.1.1 Определение и основные свойства . . . 100
9.1.2 Почему преимущество снижает дисперсию . . . 101
9.2 Архитектура актор-критик . . . 101
9.2.1 Двухкомпонентный дизайн . . . 101
9.2.2 Раздельные или совместные сети . . . 102
9.3 Ошибка TD как оценка преимущества . . . 102
9.3.1 Одношаговая оценка . . . 102
9.3.2 Одношаговый алгоритм актор-критик . . . 103
9.4 Обобщённая оценка преимущества . . . 103
9.4.1 Многошаговые отдачи и n-шаговое преимущество . . . 103
9.4.2 Определение и вывод GAE . . . 104
9.4.3 Параметр λ: компромисс между смещением и дисперсией . . . 105
9.4.4 Цель GAE для критика . . . 105
9.5 A2C и A3C . . . 105
9.5.1 Параллелизм как инструмент снижения дисперсии . . . 105
9.5.2 A2C: синхронный вариант . . . 106
9.6 Методы доверительной области . . . 106
9.6.1 Повторное использование данных с другой политики через взвешивание по важности 106
9.6.2 Опасность больших обновлений . . . 107
9.6.3 Trust Region Policy Optimization (TRPO) . . . 107
9.7 Proximal Policy Optimization . . . 107
9.7.1 От доверительных областей к отсечению . . . 107
9.7.2 Анализ случаев механизма отсечения . . . 108
9.7.3 Полная целевая функция PPO . . . 109
9.7.4 Алгоритм PPO . . . 109
9.8 PPO для языковых моделей . . . 110
9.8.1 Языковые модели как стратегии ОП . . . 110
9.8.2 Моделирование вознаграждения и штраф KL . . . 110
9.8.3 Четырёхмодельная архитектура RLHF . . . 111
9.8.4 Практические соображения . . . 111
9.9 Сходимость и практические соображения . . . 112
9.9.1 Свойства сходимости . . . 112
9.9.2 Сравнение алгоритмов градиента стратегии . . . 112
9.9.3 Чувствительность к гиперпараметрам и советы по реализации . . . 113

Упражнения . . . 113

III RL для языковых моделей 116

10 Формирование и модели вознаграждения 117
10.1 Формирование вознаграждения на основе потенциала . . . 117
10.1.1 Проблема разреженного вознаграждения . . . 117
10.1.2 Теорема о формировании . . . 118
10.1.3 Проектирование потенциальных функций: примеры . . . 119
10.2 От ручного проектирования к обученным вознаграждениям . . . 119
10.2.1 Пределы ручного проектирования вознаграждения . . . 119
10.2.2 Взлом вознаграждения в языковых моделях . . . 120
10.3 Модель Bradley–Terry . . . 120
10.3.1 Данные о предпочтениях . . . 120
10.3.2 Модель предпочтений Bradley–Terry . . . 121
10.3.3 Связь с рейтингами Эло . . . 121
10.4 Модели вознаграждения по предпочтениям . . . 122
10.4.1 Оценка максимального правдоподобия . . . 122
10.4.2 Выбор архитектуры . . . 122
10.4.3 Практическая процедура обучения . . . 123
10.5 Модели вознаграждения по исходам (ORM) . . . 123
10.5.1 Разреженные терминальные вознаграждения . . . 123
10.5.2 Когда использовать ORM . . . 123
10.6 Модели вознаграждения по процессу (PRM) . . . 124
10.6.1 Надзор на уровне шагов . . . 124
10.6.2 Обучение моделей вознаграждения по процессу . . . 124
10.6.3 Использование PRM для поиска во время вывода . . . 125
10.6.4 ORM против PRM: сравнение . . . 125
10.7 Цели обучения и практические соображения . . . 125
10.7.1 Варианты функции потерь с запасом . . . 125
10.7.2 Сглаживание меток . . . 126
10.7.3 Качество данных и протоколы аннотирования . . . 126
10.7.4 Метрики оценки . . . 126
10.8 Переоптимизация вознаграждения . . . 127
10.8.1 Закон Гудхарта в обучении с подкреплением . . . 127
10.8.2 Штраф KL как регуляризация . . . 127
10.9 Резюме и связи . . . 128
Упражнения . . . 129

11 Языковые модели и постобучение 131
11.1 Архитектура Трансформера . . . 131
11.1.1 Само-внимание . . . 131
11.1.2 Позиционное кодирование . . . 132
11.1.3 Блоки прямой связи и нормализация слоёв . . . 132
11.2 Авторегрессионное языковое моделирование . . . 133
11.2.1 Цель обучения: кросс-энтропийная потеря . . . 133
11.2.2 Законы масштабирования . . . 134
11.3 Генерация языка как марковский процесс принятия решений . . . 134
11.4 Предобучение . . . 135
11.4.1 Данные . . . 135
11.4.2 Потеря предобучения и эмерджентные способности . . . 136
11.4.3 Базовые модели и их ограничения . . . 136
11.5 Supervised Fine-Tuning . . . 136
11.5.1 Формат данных и шаблоны инструкций . . . 137
11.5.2 Функция потерь SFT . . . 137
11.5.3 Датасеты и примеры . . . 137
11.5.4 Смещение воздействия и потолок SFT . . . 137
11.6 Пайплайн постобучения . . . 137
11.6.1 Взаимодействие этапов . . . 138

11.6.2 Что оптимизирует каждый этап . . . 138
11.7 Синтетические данные для постобучения . . . 138
11.7.1 Почему синтетические данные необходимы . . . 138
11.7.2 Self-Instruct . . . 139
11.7.3 Дистилляция из моделей-учителей . . . 139
11.7.4 Evol-Instruct и нарастание сложности . . . 139
11.7.5 Генерация на основе персон . . . 140
11.7.6 Синтез данных предпочтений . . . 140
11.7.7 Фильтрация данных и контроль качества . . . 140
11.7.8 Коллапс модели и меры противодействия . . . 141
11.7.9 Ключевые синтетические датасеты . . . 141
11.8 Оценка языковых моделей . . . 142
11.8.1 Автоматизированные бенчмарки . . . 142
11.8.2 Оценка людьми . . . 142
11.8.3 LLM как судья . . . 142
11.8.4 Оценка модели вознаграждения . . . 143
11.9 Резюме . . . 143
Упражнения . . . 143

12 DPO и методы выравнивания **146**
12.1 От RLHF к прямому выравниванию . . . 146
12.1.1 Стоимость модели вознаграждения . . . 146
12.1.2 Простота, память и стабильность . . . 147
12.2 Вывод DPO . . . 147
12.2.1 KL-ограниченная цель . . . 147
12.2.2 Параметризация оптимальной стратегии . . . 148
12.2.3 Подстановка в модель Брэдли — Терри . . . 148
12.2.4 Функция потерь DPO . . . 148
12.3 Алгоритм DPO . . . 149
12.3.1 Процедура обучения . . . 149
12.3.2 Требования к данным . . . 150
12.3.3 Детали реализации . . . 150
12.3.4 Сравнение пайплайнов DPO и PPO: диаграмма . . . 150
12.4 Варианты DPO . . . 150
12.4.1 IPO: оптимизация идентичных предпочтений . . . 150
12.4.2 KTO: оптимизация Канемана — Тверски . . . 151
12.4.3 cDPO: консервативный DPO . . . 152
12.4.4 ORPO: оптимизация отношения шансов . . . 152
12.5 Ограничения прямого выравнивания . . . 152
12.5.1 Проблема сдвига распределения . . . 152
12.5.2 Отсутствие онлайн-исследования . . . 153
12.5.3 Поиск моды против покрытия мод . . . 153
12.5.4 Когда DPO не работает . . . 153
12.6 Online RL в Production-системах . . . 153
12.6.1 Почему Online RL важен . . . 153
12.6.2 On-Policy и Off-Policy Соображения . . . 154
12.6.3 Смягчение катастрофического забывания . . . 154
12.6.4 Ограничения безопасности в онлайн-RL . . . 155
12.6.5 PPO против DPO в онлайн-режиме . . . 155
12.6.6 Практические паттерны развёртывания . . . 155
12.7 Исчерпывающее сравнение методов выравнивания . . . 156
12.7.1 Обзор методов . . . 156
12.7.2 Выбор метода выравнивания по цели . . . 156
12.8 Итоги . . . 158
Упражнения . . . 159

13 GRPO и онлайн-RL **161**
13.1 Мотивация: за пределами PPO и DPO . . . 161
13.1.1 Стоимость критика в PPO . . . 161
13.1.2 Проблема сдвига распределения в DPO . . . 162
13.1.3 GRPO: лучшее из двух миров . . . 162
13.2 Цель GRPO . . . 162
13.2.1 Постановка задачи и обозначения . . . 163
13.2.2 Group-Relative Advantage . . . 163
13.2.3 Отсечённая суррогатная цель . . . 163
13.2.4 Штраф KL . . . 163
13.2.5 Полная цель GRPO . . . 164
13.3 Алгоритм GRPO . . . 164
13.3.1 Ключевые гиперпараметры . . . 164
13.3.2 Проработанный пример . . . 165
13.4 Теоретический анализ . . . 166
13.4.1 Связь с REINFORCE и Baseline . . . 166
13.4.2 Сравнение с RLOO . . . 166
13.4.3 Снижение дисперсии при групповой нормализации . . . 166
13.4.4 Влияние размера группы G . . . 167
13.5 RLVR: обучение с подкреплением с проверяемыми вознаграждениями . . . 167
13.5.1 Определение и мотивация . . . 167
13.5.2 Формальная постановка . . . 168
13.5.3 Связь с GRPO . . . 168
13.5.4 Области применения RLVR . . . 169
13.6 Rejection Sampling и best-of-N . . . 169
13.6.1 Алгоритм . . . 169
13.6.2 Rejection Sampling Fine-Tuning . . . 169
13.6.3 Сравнение с GRPO . . . 169
13.7 Онлайн-обучение RL в масштабе . . . 170
13.7.1 Двухфазная архитектура . . . 170
13.7.2 Обработка завершений переменной длины . . . 170
13.7.3 Формирование батча . . . 170
13.7.4 Синхронный и асинхронный дизайн . . . 170
13.7.5 Воспроизведение опыта для GRPO . . . 171
13.7.6 Мониторинг и диагностика . . . 171
13.8 Сравнение: PPO, GRPO и DPO . . . 171
13.8.1 Когда применять каждый метод . . . 171
13.9 Итоги . . . 172
Упражнения . . . 173

14 Практический RLHF **174**
14.1 Пайплайн RLHF End-to-End . . . 174
14.2 Сбор данных о предпочтениях . . . 175
14.2.1 Форматы аннотирования . . . 175
14.2.2 Согласованность аннотаторов . . . 175
14.2.3 Качество данных против их количества . . . 176
14.2.4 Распространённые ошибки . . . 176
14.3 Обучение моделей вознаграждения на практике . . . 176
14.3.1 Выбор архитектуры . . . 176
14.3.2 Устойчивость обучения и переобучение . . . 177
14.3.3 Калибровка и метрики оценки . . . 177
14.3.4 Когда переобучать . . . 177
14.4 PPO для языковых моделей: детали реализации . . . 177
14.4.1 Четырёхмодельная схема . . . 177
14.4.2 Реализация штрафа KL . . . 178
14.4.3 Оценка преимущества с помощью GAE . . . 178
14.4.4 Формирование мини-батча и накопление градиентов . . . 178
14.4.5 Обучение со смешанной точностью . . . 179
14.4.6 Гиперпараметры . . . 179
14.5 Отладка RLHF . . . 179

14.5.1 Взлом вознаграждения . . . 179
14.5.2 Взрыв дивергенции KL . . . 180
14.5.3 Коллапс моды . . . 180
14.5.4 Нестабильность обучения . . . 180
14.5.5 Диагностические метрики для мониторинга . . . 180
14.6 Масштабирование RLHF . . . 181
14.6.1 Генерация против обучения . . . 181
14.6.2 Стратегии параллелизма . . . 181
14.6.3 Оценки вычислительного бюджета . . . 181
14.7 Ограничения безопасности и элайнмента . . . 181
14.7.1 Красная команда . . . 182
14.7.2 Интеграция Constitutional AI . . . 182
14.7.3 Полезность против безвредности . . . 182
14.7.4 Когда останавливать обучение . . . 182
14.8 Кейсы . . . 182
14.8.1 InstructGPT (Ouyang et al., 2022) . . . 182
14.8.2 Llama 2 (Touvron et al., 2023) . . . 183
14.8.3 Подход Anthropic (Bai et al., 2022) . . . 183
14.8.4 Подход DeepSeek с GRPO (DeepSeek-AI, 2024) . . . 183
14.9 Резюме . . . 183
Упражнения . . . 184

IV Фронтир 186

15 Рассуждения и дообучение с подкреплением 187
15.1 От RLHF к обучению с подкреплением с верифицируемыми вознаграждениями . . . 187
15.1.1 Идея верификации . . . 187
15.1.2 Верифицируемые области . . . 188
15.1.3 Формальное сравнение с целью RLHF . . . 188
15.2 Рассуждения типа chain-of-thought . . . 189
15.2.1 Основная идея . . . 189
15.2.2 Почему более длинное рассуждение помогает . . . 190
15.2.3 Токены рассуждений и формат . . . 190
15.2.4 Связь с вычислениями на этапе вывода . . . 190
15.3 Обучение моделей рассуждений . . . 190
15.3.1 Двухэтапный рецепт . . . 190
15.3.2 Фильтрация по сложности . . . 191
15.3.3 Удаление штрафа KL . . . 191
15.3.4 Награды за формат и штрафы за длину . . . 192
15.3.5 Нормализация функции потерь и смещение GRPO . . . 192
15.3.6 Самообучаемое рассуждение и дистилляция . . . 192
15.4 Масштабирование на этапе вывода . . . 192
15.4.1 Гипотеза о вычислениях на этапе вывода . . . 192
15.4.2 Голосование большинства . . . 193
15.4.3 Сэмплирование с направлением по ценности . . . 193
15.4.4 Вычислительно-оптимальный вывод . . . 194
15.5 Использование инструментов и вызов функций . . . 194
15.5.1 Мотивация . . . 194
15.5.2 Формат вызова функций . . . 194
15.5.3 Обучение использованию инструментов . . . 195
15.5.4 SWiRL: пошаговый RL для использования инструментов . . . 195
15.6 Многошаговые агенты и агентное RL . . . 195
15.6.1 Цикл агента . . . 195
15.6.2 Задачи и среды агентного RL . . . 196
15.6.3 Отнесение заслуг в многошаговых траекториях . . . 196
15.6.4 Исследование в агентном RL . . . 197
15.7 Практические соображения . . . 197
15.7.1 Выбор алгоритма . . . 197
15.7.2 Онлайн- и офлайн-RL . . . 198

15.7.3 Инфраструктура в масштабе . . . 198
15.7.4 Инструменты с открытым исходным кодом . . . 198
15.8 Итоги и открытые вопросы . . . 199
Упражнения . . . 199

16 Мультиагентные системы и RL **201**
16.1 Мультиагентные марковские игры . . . 201
16.1.1 От MDP к марковским играм . . . 201
16.1.2 Равновесие Нэша . . . 202
16.1.3 Частичная наблюдаемость и Dec-POMDP . . . 203
16.2 Кооперативный MARL . . . 203
16.2.1 Кооперативная обстановка . . . 203
16.2.2 Централизованное обучение с децентрализованным исполнением . . . 203
16.2.3 QMIX . . . 203
16.2.4 MAPPO . . . 204
16.3 Конкурентные и смешанные обстановки . . . 205
16.3.1 Игры с нулевой суммой и самоигра . . . 205
16.3.2 AlphaGo и AlphaZero . . . 205
16.3.3 Нестабильность конкурентного обучения и нетранзитивные стратегии . . . 205
16.4 Коммуникация и возникающее поведение . . . 206
16.4.1 Обученные протоколы коммуникации . . . 206
16.4.2 Возникающая коммуникация . . . 206
16.4.3 Социальные дилеммы в многоагентном обучении . . . 206
16.5 Мультиагентный RL для LLM . . . 206
16.5.1 Дебаты . . . 207
16.5.2 Многоагентная генерация кода . . . 207
16.5.3 LLM-агенты, сотрудничающие над задачами . . . 207
16.5.4 Agentic Workflows со специализированными агентами . . . 208
16.6 Проблемы мультиагентного RL . . . 208
16.6.1 Нестационарность . . . 208
16.6.2 Отнесение заслуг в командах . . . 208
16.6.3 Масштабируемость . . . 209
16.6.4 Формирование вознаграждения для кооперации . . . 209
16.6.5 Эффективность по образцам . . . 209
16.7 Итоги . . . 209
Упражнения . . . 210

17 Открытые задачи **212**
17.1 Эффективность по образцам . . . 212
17.1.1 Модельно-ориентированный RL и модели мира . . . 212
17.1.2 Перенос знаний и few-shot RL . . . 213
17.2 Спецификация вознаграждения и выравнивание . . . 213
17.2.1 Масштабируемый надзор . . . 213
17.2.2 Фундаментальный вызов . . . 213
17.3 Безопасность и устойчивость . . . 214
17.3.1 Безопасное исследование . . . 214
17.3.2 Распределительная устойчивость и состязательные атаки . . . 214
17.3.3 Формальная верификация . . . 214
17.4 Офлайн и пакетный RL . . . 214
17.4.1 Задача сдвига распределения . . . 214
17.4.2 Моделирование последовательностей и Decision Transformer . . . 215
17.4.3 Переход от офлайн к онлайн . . . 215
17.5 Мультимодальный и воплощённый RL . . . 215
17.5.1 Перенос из симуляции в реальность . . . 215
17.5.2 Vision-Language-Action Models . . . 215
17.5.3 Непрерывное управление at scale . . . 215
17.6 RL и базовые модели . . . 216
17.6.1 RL для языка и рассуждений . . . 216
17.6.2 RL для визуальной генерации . . . 216
17.6.3 RL для научных открытий . . . 216

17.7 Теоретические рубежи . . . 216
17.7.1 Сожаление и сложность выборки для глубокого RL . . . 216
17.7.2 Сходимость нелинейной аппроксимации функций . . . 217
17.7.3 Статистическая сложность RLHF . . . 217
17.7.4 Почему RL работает для LLM? . . . 217
17.8 Путь вперёд . . . 217

A Математические основы **219**
A.1 Основы линейной алгебры . . . 219
A.2 Математический анализ и оптимизация . . . 220
A.3 Теория вероятностей . . . 221
A.4 Методы выборки . . . 222
A.5 Теория информации . . . 223
A.6 Операторы и сжимающие отображения . . . 223

Послесловие **225**

Предметный указатель **226**

Обозначения

В следующих таблицах собраны символы, используемые на протяжении всей книги единообразно. Если символ несёт значение, специфическое для отдельной главы, эта глава вводит его явно; приведённые здесь записи охватывают глобально повторяющиеся соглашения.

Общая математика

Символ	Значение
$\mathbb{R}$, $\mathbb{R}^n$	Вещественные числа; n-мерное вещественное векторное пространство
$\mathbb{N}$	Натуральные числа $\{0, 1, 2, \ldots\}$
$\mathbb{Z}$	Целые числа
$[n]$	Индексное множество $\{1, 2, \ldots, n\}$
$\lvert S\rvert$	Мощность множества S
$\langle x, y\rangle$	Скалярное произведение векторов x и y
$\lVert x\rVert$, $\lVert x\rVert_p$	Евклидова норма (2-норма); ℓ^p-норма
$\lVert A\rVert_F$	Норма Фробениуса матрицы A
$A^\top$	Транспонирование матрицы A
A^{-1}	Обратная матрица к квадратной матрице A
$\nabla_\theta f$	Градиент f по θ
$\nabla^2_\theta f$	Гессиан f по θ
$\partial_x f$	Частная производная f по x
$f \circ g$	Композиция функций: $(f \circ g)(x) = f(g(x))$
$\propto$	Пропорционально (с точностью до нормировочной константы)
$\mathbb{1}\{A\}$	Индикаторная функция: 1, если событие A произошло, иначе 0
$\log$	Натуральный логарифм (основание e)
$\log_2$	Двоичный логарифм (основание 2)
$\exp(x)$	Показательная функция e^x
$[x]_+$	Положительная часть: $\max(x, 0)$
$\lfloor x\rfloor$, $\lceil x\rceil$	Функции пола и потолка

Теория вероятностей и статистика

Символ	**Значение**
$\mathbb{P}(A)$	Вероятность события A
$\mathbb{P}(A \mid B)$	Условная вероятность A при условии B
$\mathbb{E}[X]$	Математическое ожидание случайной величины X
$\mathbb{E}[X \mid Y]$	Условное математическое ожидание X при условии Y
$\mathrm{Var}[X]$	Дисперсия X: $\mathbb{E}[X^2] - (\mathbb{E}[X])^2$
$\mathrm{Cov}[X, Y]$	Ковариация X и Y
$p(x)$	Функция плотности вероятности или функция масс вероятности
$p(x \mid y)$	Условная плотность/масса x при условии y
$X \sim p$	X распределена по закону p
$X \perp Y$	X и Y независимы
$\mathcal{N}(\mu, \sigma^2)$	Нормальное распределение со средним μ и дисперсией σ^2
$\mathrm{Bernoulli}(p)$	Распределение Бернулли с вероятностью успеха p
$\mathrm{Cat}(\mathbf{p})$	Категориальное распределение с вектором вероятностей $\mathbf{p}$
$D_{\mathrm{KL}}(p \parallel q)$	Дивергенция Кульбака-Лейблера от q к p
$H(p)$	Энтропия Шеннона распределения p
$H(p, q)$	Перекрёстная энтропия между p и q
$I(X; Y)$	Взаимная информация между X и Y
$\sigma(x)$	Сигмоидная функция $1/(1 + e^{-x})$
$\mathrm{softmax}(\mathbf{z})_i$	$e^{z_i} / \sum_j e^{z_j}$

Основные обозначения обучения с подкреплением

Символ	Значение
$\mathcal{S}$	Пространство состояний
$\mathcal{A}$	Пространство действий
$\mathcal{R}$	Пространство вознаграждений (также используется для сожаления)
$\mathcal{M}$	Кортеж МДП ($cS, \mathcal{A}, P, R, \gamma$)
$P(s' \mid s, a)$	Функция вероятности переходов
$R(s, a)$, $r(s, a)$	Ожидаемое вознаграждение; сигнал вознаграждения
s_t, a_t, r_t	Состояние, действие, вознаграждение на шаге t
τ	Траектория (эпизод): $(s_0, a_0, r_0, s_1, \ldots)$
T	Горизонт (длина эпизода); может быть ∞
$\gamma \in [0, 1)$	Коэффициент дисконтирования
G_t	Доход с момента t: $\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$
$\pi(a \mid s)$	Стохастическая стратегия
$\pi(s)$	Детерминированная стратегия (отображает s в действие)
π^*	Оптимальная стратегия
$V^\pi(s)$	Функция ценности состояния при стратегии π
$Q^\pi(s, a)$	Функция ценности действия (Q-функция) при стратегии π
$V^*(s)$, $Q^*(s, a)$	Оптимальные функции ценности
$A^\pi(s, a)$	Функция преимущества $Q^\pi(s, a) - V^\pi(s)$
δ_t	Ошибка временной разности на шаге t
α	Скорость обучения / размер шага
ϵ	Коэффициент исследования (например, ϵ-жадный)
λ	Скорость затухания следов приемлемости (TD(λ))
n	n-шаговый просмотр вперёд
θ, ϕ, ψ	Векторы параметров (стратегии, функции ценности, модели вознаграждения)
π_θ	Стратегия, параметризованная θ
V_ϕ, Q_ϕ	Функция ценности, параметризованная ϕ
$\mathcal{T}^\pi$	Оператор Беллмана для стратегии π
$\mathcal{T}^*$	Оператор оптимальности Беллмана
$d^\pi(s)$	Стационарное распределение состояний при стратегии π
$J(\theta)$	Целевая функция стратегии (ожидаемый доход)
$\hat{A}_t$	Оценка преимущества на шаге t
$\hat{g}$	Стохастическая оценка градиента стратегии
L^{CLIP}	Усечённая суррогатная целевая функция PPO
$\rho_t(\theta)$	Отношение вероятностей $\pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$

Обозначения для языковых моделей

Символ	Значение
$\mathcal{V}$	Словарь (множество токенов)
x	Промпт / входная последовательность
y	Ответ / выходная последовательность
y_t	t-й токен ответа y
$y_{<t}$	Токены ответа до позиции t
$\|y\|$	Длина ответа y (в токенах)
eos	Токен конца последовательности
$\pi_\theta(y \mid x)$	Стратегия языковой модели (авторегрессионная)
$\pi_{\text{ref}}(y \mid x)$	Эталонная (референсная) стратегия
$r_\psi(x, y)$	Модель вознаграждения с параметрами ψ
β	Коэффициент KL-штрафа
$\mathcal{D}_{\text{pref}}$	Набор данных о предпочтениях людей
y^+, y^-	Предпочтительный и отвергнутый ответы
$p^*(y^+ \succ y^-)$	Вероятность предпочтения людей
$Z(x)$	Нормировочная функция (статсумма) для оптимальной стратегии LM
ℓ_{DPO}	Функция потерь DPO
g_i	i-е сгенерированное завершение (GRPO)
$\hat{r}_i$	Нормализованное вознаграждение для группы i (GRPO)

Распространённые сокращения

Сокращение	Расшифровка
RL	Reinforcement Learning (обучение с подкреплением)
MDP	Markov Decision Process (марковский процесс принятия решений)
POMDP	Partially Observable MDP (частично наблюдаемый МДП)
TD	Temporal Difference (временная разность)
MC	Monte Carlo (метод Монте-Карло)
DP	Dynamic Programming (динамическое программирование)
GPI	Generalised Policy Iteration (обобщённая итерация стратегии)
VFA	Value Function Approximation (аппроксимация функции ценности)
DQN	Deep Q-Network
DDPG	Deep Deterministic Policy Gradient
A2C/A3C	Advantage Actor-Critic / Asynchronous A3C
PPO	Proximal Policy Optimisation
TRPO	Trust Region Policy Optimisation
GAE	Generalised Advantage Estimation
SGD	Stochastic Gradient Descent (стохастический градиентный спуск)
SFT	Supervised Fine-Tuning (дообучение с учителем)
RLHF	Reinforcement Learning from Human Feedback
DPO	Direct Preference Optimisation
GRPO	Group Relative Policy Optimisation
RLVR	RL with Verifiable Rewards (RL с верифицируемыми вознаграждениями)
CoT	Chain-of-Thought (цепочка рассуждений)
LLM	Large Language Model (большая языковая модель)
KL	Kullback-Leibler (дивергенция Кульбака-Лейблера)
i.i.d.	Independent and Identically Distributed (независимые одинаково распределённые)
w.r.t.	With Respect To (по отношению к)

Часть I

Основы

Глава 1

Введение в обучение с подкреплением

Агент учится не потому, что ему говорят правильное действие, а открывая — через пробы и взаимодействие — какие действия приносят наибольшее долгосрочное вознаграждение.

Обучение с подкреплением (RL) является разделом машинного обучения, посвящённым *последовательному принятию решений в условиях неопределённости*. В отличие от обучения с учителем, где заранее предоставляется фиксированный набор размеченных примеров, агент RL должен взаимодействовать со *средой*, выбирать *действия*, наблюдать их последствия и использовать получаемые *вознаграждения* для улучшения своего поведения со временем.

Данная глава знакомит с ключевыми идеями, мотивацией и проблемами обучения с подкреплением, а также помещает его в контекст более широкого пространства парадигм машинного обучения.

1.1 Что такое обучение с подкреплением?

Определение 1.1 (Обучение с подкреплением)**. Обучение с подкреплением** — это вычислительный подход к обучению на основе взаимодействия, при котором *агент* совершает действия в *среде*, получает *вознаграждения* и стремится максимизировать суммарное вознаграждение на горизонте взаимодействия.

Определяющая черта RL состоит в том, что агент никогда не получает явных указаний о том, какое действие является «правильным». Вместо этого он получает лишь *оценочную обратную связь*: скалярный сигнал, указывающий, насколько хорошим (или плохим) оказался результат его поведения. Задача агента, таким образом, заключается не в имитации заданной разметки, а в *открытии стратегии* — называемой *политикой* — которая максимизирует долгосрочную эффективность.

Чтобы формализовать качество поведения агента, введём понятие *дохода* (также называемого *дисконтированным суммарным вознаграждением*), агрегирующего будущие вознаграждения начиная с шага t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{1.1}$$

где $\gamma \in [0, 1)$ — *коэффициент дисконтирования*, а R_{t+k+1} — вознаграждение, полученное через $k+1$ шаг после момента t. Следуя стандартному соглашению, на шаге t агент выбирает действие A_t, после чего среда возвращает вознаграждение R_{t+1} и следующее состояние S_{t+1}.

Если политика агента обозначена π, стандартная цель RL такова:

$$J(\pi) = \mathbb{E}_\pi[G_0]. \tag{1.2}$$

Иными словами, мы ищем политику π, максимизирующую ожидаемый доход (1.2).

Замечание 1.1. В *эпизодических задачах* сумма в (1.1) обрывается в терминальном состоянии. В *непрерывных задачах* горизонт бесконечен, и коэффициент дисконтирования $\gamma < 1$ обеспечивает конечность суммы.

1.2 Ключевые проблемы обучения с подкреплением

Даже в простейших постановках RL возникает ряд фундаментальных проблем:

- **Отложенное вознаграждение.** Последствия действия могут не проявиться вплоть до многих шагов спустя. Ход в шахматах, кажущийся безобидным в дебюте, может определить исход партии пятьюдесятью ходами позже.
- **Исследование против эксплуатации.** Агент должен находить баланс между *исследованием* — пробой незнакомых действий с целью открыть потенциально лучшие стратегии — и *эксплуатацией* — повторением действий, которые уже кажутся выгодными. Это противоречие является центральной темой главы 2.
- **Сдвиг распределения, вызванный политикой.** В отличие от обучения с учителем, где обучающее распределение обычно фиксировано, в RL собственная политика агента определяет, какие состояния и переходы он будет наблюдать далее. Изменение политики меняет данные.
- **Коррелированные данные.** Последовательные наблюдения вдоль траектории временно коррелированы и не являются независимо взятыми, что усложняет применение стандартных гарантий статистического обучения.

Замечание 1.2. Именно зависимость данных от текущей политики делает RL существенно более чувствительным к ошибкам на ранних этапах обучения. Плохая начальная стратегия может не допустить агента в области пространства состояний-действий, где могло бы быть обнаружено более качественное поведение.

1.3 Краткая история обучения с подкреплением

Интеллектуальные корни RL лежат на пересечении нескольких областей:

Психология (1900-е–1950-е) *Закон эффекта* Эдварда Торндайка (1911) гласил, что действия, за которыми следуют удовлетворяющие последствия, имеют тенденцию повторяться. Эта идея обучения по сигналу вознаграждения является концептуальным предшественником RL.

Оптимальное управление (1950-е) Ричард Беллман сформулировал теорию *динамического программирования* и ввёл *уравнение Беллмана*, которое по сей день остаётся основой алгоритмов RL.

Стохастическая аппроксимация Роббинс и Монро (1951) разработали итерационные методы нахождения корней стохастических уравнений, заложив математический фундамент для инкрементных правил обучения.

Временное разностное обучение (1980-е–1990-е) Алгоритм TD(λ) Саттона (1988) объединил идеи метода Монте-Карло и динамического программирования. Уоткинс ввёл Q-learning (1989), а TD-Gammon Тесауро (1992) продемонстрировал, что RL способен достигать уровня эксперта в нарды.

Глубокий RL (2013 — настоящее время) Сочетание глубоких нейронных сетей с RL привело к серии прорывов: DQN играет в игры Atari на уровне человека (Mnih et al., 2013, 2015), AlphaGo обыгрывает чемпиона мира по го (Silver et al., 2016, 2017), а алгоритмы управления с непрерывными действиями, такие как DDPG, SAC и PPO, обеспечивают манипуляции в робототехнике.

RL для языковых моделей (2020 — настоящее время) Применение методов RL — в особенности PPO-based RLHF — для выравнивания больших языковых моделей (Ouyang et al., 2022; OpenAI, 2023) сделало RL центральным инструментом современного ИИ. Методы DPO (Rafailov et al., 2023) и GRPO (Shao et al., 2024) продолжают расширять границу RL–LLM.

1.4 Обучение с подкреплением и другие парадигмы машинного обучения

Чтобы оценить отличительные особенности RL, полезно сравнить его с другими ключевыми парадигмами машинного обучения.

Парадигма	Данные	Цель	Ключевая проблема
Обучение с учителем	пары вход–метка (x, y)	минимизация ошибки предсказания	обобщение, шум меток, сдвиг распределения
Обучение без учителя	неразмеченные данные x	обнаружение структуры, плотности или представлений	выбор метрики качества; интерпретация
Обучение с имитацией	демонстрации эксперта	воспроизведение политики эксперта	сдвиг ковариат; накапливающиеся ошибки вдоль траекторий
Обучение с подкреплением	траектории взаимодействия со средой	максимизация ожидаемого дохода	исследование, отложенное вознаграждение, нестабильность обучения

Таблица 1.1: Сравнение обучения с подкреплением с другими основными парадигмами машинного обучения.

Наиболее существенное отличие состоит в том, что в RL обучающее распределение *не фиксировано*: собственная политика агента определяет, какие данные он будет собирать далее. При изменении политики меняется и распределение наблюдений.

С практической точки зрения RL зачастую более требователен, чем обучение с учителем, по нескольким причинам:

- Сбор данных дорогостоящ: он требует симуляции, взаимодействия с физической системой или большого числа эпизодов.
- Сигнал вознаграждения нередко зашумлён, разрежён или плохо сконструирован.
- Использование нейросетевых аппроксиматоров функций привносит нестабильности, требующие специализированных методов стабилизации (см. «смертельную триаду» в главе 7).

1.5 Применения обучения с подкреплением

Обучение с подкреплением было успешно применено в самых разнообразных областях:

Игры Настольные игры (го, шахматы, сёги посредством AlphaZero), видеоигры (Atari, StarCraft II, Dota 2), карточные игры (покер посредством Libratus, Pluribus).

Робототехника Локомоция, ловкие манипуляции и перенос из симуляции в реальность с использованием PPO, SAC и модельных методов.

Рекомендательные системы Оптимизация долгосрочной вовлечённости пользователей вместо мгновенного показателя кликабельности.

Автономное вождение Сквозное обучение политик вождения на основе симуляции и реального взаимодействия.

Исследование операций Управление запасами, динамическое ценообразование, распределение ресурсов и комбинаторная оптимизация.

Здравоохранение Персонализированные стратегии лечения, адаптивные клинические испытания и оптимизация дозирования препаратов.

Выравнивание языковых моделей RLHF для следования инструкциям, безопасности и полезности; DPO, GRPO и связанные методы для выравнивания LLM с человеческими предпочтениями. Данному применению посвящена Часть III этой книги.

1.6 Интерфейс агент–среда

На самом высоком уровне каждая задача RL включает два участника:

1. **Агент**: обучающийся и принимающий решения.
2. **Среда**: всё, с чем взаимодействует агент.

На каждом дискретном шаге $t = 0, 1, 2, \ldots$ повторяется следующий цикл:

1. Агент наблюдает текущее состояние S_t (или наблюдение O_t в частично наблюдаемых постановках).
2. Агент выбирает действие A_t в соответствии со своей политикой π.
3. Среда переходит в новое состояние S_{t+1} и выдаёт скалярное вознаграждение R_{t+1}.

Это взаимодействие порождает *траекторию*:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots \tag{1.3}$$

Формальная математическая рамка этого взаимодействия — *марковский процесс принятия решений* (MDP) — является предметом главы 3.

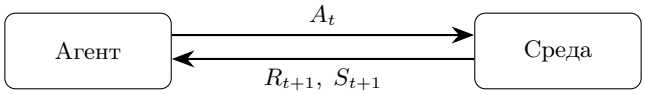


Рис. 1.1: Цикл взаимодействия агента и среды. На каждом шаге агент посылает действие A_t и получает вознаграждение R_{t+1} и следующее состояние S_{t+1}.

1.7 Обзор книги

Остальная часть книги организована в четыре части.

Часть I: Основы (главы 2–6). Мы начинаем с задачи многорукого бандита (глава 2), которая вводит компромисс между исследованием и эксплуатацией в чистейшем виде. Затем формализуем полную постановку RL через марковские процессы принятия решений (глава 3), разрабатываем точные методы решения на основе динамического программирования (глава 4) и вводим два семейства алгоритмов, основанных на выборке: методы Монте-Карло (глава 5) и временное разностное обучение (глава 6).

Часть II: Аппроксимация функций и глубокий RL (главы 7–9). Мы рассматриваем проблему масштабирования RL на большие пространства состояний и действий посредством аппроксимации функций (глава 7), методов градиента политики (глава 8) и архитектур актор-критик, венчающихся PPO (глава 9).

Часть III: RL для языковых моделей (главы 10–14). Мы применяем инструментарий RL к обучению и выравниванию больших языковых моделей: начинаем с формирования вознаграждения и моделей вознаграждения (глава 10), затем переходим к пайплайну дообучения LLM (глава 11), методам DPO и прямого выравнивания (глава 12), GRPO и онлайн-RL с верифицируемыми вознаграждениями (глава 13), а также практике RLHF (глава 14).

Часть IV: Направления исследований (главы 15–17). Исследуем рассуждение и дообучение с подкреплением (глава 15), мультиагентные системы (глава 16) и завершаем обзором открытых проблем и перспективных направлений исследований (глава 17).

Упражнения

Упражнение 1.1. Приведите три реальные задачи, которые естественно формулируются как задачи обучения с подкреплением. Для каждой опишите агента, среду, пространство действий и сигнал вознаграждения.

Упражнение 1.2. Рассмотрите задачу обучения робота ходьбе. Объясните, почему подход с обучением с учителем (например, обучение по заранее записанным траекториям ходящего эксперта) может оказаться неудачным и почему подход RL может быть более подходящим. Какую роль играет компромисс между исследованием и эксплуатацией?

Упражнение 1.3. Покажите, что при $\gamma \in [0, 1)$ и ограниченных вознаграждениях $|R_t| \leq R_{\max}$ для всех t, доход (1.1) удовлетворяет

$$|G_t| \leq \frac{R_{\max}}{1 - \gamma}.$$

Упражнение 1.4. Для каждой из следующих задач определите, какая парадигма машинного обучения (с учителем, без учителя, с имитацией или с подкреплением) является наиболее естественной, и обоснуйте свой ответ:

(a) Классификация электронных писем на спам и не-спам.

(b) Обучение чат-бота быть полезным и безвредным с использованием данных о человеческих предпочтениях.

(c) Обнаружение клиентских сегментов по истории покупок.

(d) Обучение беспилотного автомобиля навигации по записям водителей-экспертов.

Глава 2

Многорукие бандиты

Простейшая постановка, в которой дилемма исследования и эксплуатации уже проявляется во всей силе — это задача многорукого бандита.

В этой главе мы изучаем задачу *многорукого бандита* — задачу последовательного принятия решений без состояний, в которой агент многократно выбирает одно из k действий и наблюдает стохастическое вознаграждение. Несмотря на простоту, постановка бандита уже обнажает ключевое противоречие между сбором информации (исследованием) и максимизацией немедленного выигрыша (эксплуатацией). Мы начинаем с базовой формулировки, анализируем стратегии ε-жадности, а затем разрабатываем более обоснованные подходы: *верхние доверительные границы* (UCB), *выборку Томпсона* и информационно-теоретическую нижнюю оценку Лаи и Роббинса. Глава завершается обсуждением контекстных бандитов — мостом к полноценной задаче обучения с подкреплением — и эмпирическим сравнением изученных алгоритмов.

2.1 Постановка задачи

Определение 2.1 (k-рукий бандит)**.** **k-рукий бандит** — это задача последовательного принятия решений, в которой на каждом шаге $t = 1, 2, \ldots$ агент выбирает действие A_t из конечного множества

$$\mathcal{A} = \{1, 2, \ldots, k\},$$

и получает стохастическое вознаграждение $R_t \in \mathbb{R}$.

Замечание 2.1 (Соглашение об обозначениях)*.* В постановке бандита мы пишем R_t для вознаграждения, полученного на шаге t в ответ на действие A_t. Это отличается от соглашения MDP, принятого в главе 3, где вознаграждение обозначается R_{t+1} (получаемое после перехода из состояния S_t при действии A_t). Различие возникает потому, что у бандитов нет переходов между состояниями; более простая индексация R_t является стандартной в литературе по бандитам.

Для каждого действия $a \in \mathcal{A}$ существует распределение вознаграждений ν_a. Когда выбирается действие $A_t = a$, вознаграждение независимо извлекается из ν_a. В *стационарной* постановке эти распределения не меняются со временем.

В отличие от общей задачи RL, рассматриваемой далее в этой книге, постановка бандита предполагает:

- отсутствие понятия состояния;
- немедленные (не отложенные) вознаграждения;
- отсутствие переходной динамики — каждый раунд независим.

Тем не менее фундаментальная проблема уже присутствует: чтобы выяснить, какое действие является наилучшим, агент должен пробовать разные действия, однако для максимизации вознаграждения ему следует отдавать предпочтение уже кажущимся выгодными действиям.

2.2 Ценности действий, оптимальные действия и сожаление

Определение 2.2 (Ценность действия)**.** **Истинная ценность** (или *ожидаемое вознаграждение*) действия $a \in \mathcal{A}$ равна

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a]. \tag{2.1}$$

Если бы значения (2.1) были известны, оптимальной стратегией было бы просто всегда играть

$$a^* \in \operatorname*{argmax}_{a \in \mathcal{A}} \; q_*(a). \tag{2.2}$$

Пусть

$$q^* = \max_{a \in \mathcal{A}} \; q_*(a) \tag{2.3}$$

обозначает ценность наилучшего действия.

Поскольку ценности неизвестны, мы оцениваем эффективность путём сравнения с всезнающей стратегией.

Определение 2.3 (Псевдо-сожаление)**. Псевдо-сожаление** после T шагов равно

$$\mathcal{R}_T = T\, q^* - \sum_{t=1}^{T} q_*(A_t). \tag{2.4}$$

Его математическое ожидание равно

$$\mathbb{E}[\mathcal{R}_T] = T\, q^* - \sum_{t=1}^{T} \mathbb{E}[q_*(A_t)].$$

Интуитивно $\mathcal{R}_T$ измеряет, сколько вознаграждения алгоритм «оставил на столе» по сравнению с постоянным выбором оптимального рычага.

Определим *зазор субоптимальности* действия a как

$$\Delta_a = q^* - q_*(a) \geq 0, \tag{2.5}$$

и пусть $N_T(a) = \sum_{t=1}^{T} \mathbb{1}\{A_t = a\}$ считает число раз, когда действие a было выбрано.

Утверждение 2.1 (Разложение сожаления)**.** *В стационарной постановке бандита (где распределения вознаграждений не меняются со временем),*

$$\mathbb{E}[\mathcal{R}_T] = \sum_{a \in \mathcal{A}} \Delta_a \; \mathbb{E}[N_T(a)]. \tag{2.6}$$

Доказательство. Поскольку постановка стационарна, $q_*(a)$ постоянна во всех раундах. Так как $\sum_{a \in \mathcal{A}} N_T(a) = T$ и $\sum_{t=1}^{T} q_*(A_t) = \sum_{a \in \mathcal{A}} q_*(a)\, N_T(a)$ (группируя сумму по выбранным рычагам), имеем

$$\mathcal{R}_T = T\, q^* - \sum_{a \in \mathcal{A}} q_*(a)\, N_T(a) = \sum_{a \in \mathcal{A}} (q^* - q_*(a))\, N_T(a) = \sum_{a \in \mathcal{A}} \Delta_a\, N_T(a).$$

Взяв математические ожидания обеих частей, получаем (2.6). □

Формула (2.6) фундаментальна: минимизация сожаления эквивалентна ограничению числа раз, когда выбираются субоптимальные действия.

Замечание 2.2. Если $\mathbb{E}[\mathcal{R}_T] = O(\log T)$, то среднее сожаление за шаг убывает как $O(\log T/T) \to 0$, что означает, что агент выбирает субоптимальные действия с исчезающей частотой.

2.3 Жадные и ε-жадные стратегии

Поскольку истинные ценности действий неизвестны, агент должен оценивать их по данным.

Определение 2.4 (Оценка ценности действия)**. Оценка ценности действия** $Q_t(a)$ — это величина, вычисленная по всем вознаграждениям, наблюдённым для действия a до шага t.

Определение 2.5 (Жадное действие)**.** Действие a является **жадным** на шаге t, если

$$a \in \operatorname*{argmax}_{b \in \mathcal{A}} \; Q_t(b). \tag{2.7}$$

Простейшая стратегия — *чисто жадная*: всегда выбирать жадное действие. При всей интуитивной привлекательности она может катастрофически провалиться: ранние зашумлённые наблюдения могут заставить агента недооценить наилучший рычаг и никогда к нему не вернуться.

Пример 2.1. Рассмотрим 2-рукого бандита с $q_*(1) = 1$ и $q_*(2) = 0.9$. Если первое наблюдение рычага 1 окажется необычно низким, а рычага 2 — необычно высоким, жадный агент может зафиксироваться на рычаге 2 и никогда повторно не исследовать рычаг 1.

Чтобы избежать таких ловушек, введём случайное исследование.

Определение 2.6 (ε-жадная стратегия)**.** **ε-жадная стратегия** выбирает действия по правилу

$$A_t = \begin{cases} \operatorname*{argmax}_{a \in \mathcal{A}} Q_t(a), & \text{с вероятностью } 1-\varepsilon, \\ \text{равномерно случайное из } \mathcal{A}, & \text{с вероятностью } \varepsilon, \end{cases} \tag{2.8}$$

где $\varepsilon \in [0, 1]$.

Замечание 2.3. Если оптимальное действие правильно определено как жадное, оно выбирается с вероятностью не менее $1 - \varepsilon + \varepsilon/k$. Однако при фиксированном $\varepsilon > 0$ ожидаемое число случайных (потенциально субоптимальных) выборов линейно растёт с T, что даёт $\mathbb{E}[\mathcal{R}_T] = \Omega(T)$. То есть стратегии с фиксированным ε имеют *линейное* сожаление.

2.4 Выборочные средние и инкрементные обновления

Наиболее естественная оценка $q_*(a)$ — это *выборочное среднее*:

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i \, \mathbb{1}\{A_i = a\}}{N_t(a)}, \qquad N_t(a) > 0. \tag{2.9}$$

Для действий, ещё не опробованных, инициализируем $Q_1(a) = 0$.

Утверждение 2.2. *Если вознаграждения для действия a являются н.о.р. со средним $q_*(a)$ и $N_T(a) \to \infty$, то $Q_T(a) \to q_*(a)$ почти наверное по усиленному закону больших чисел.*

Вычислять (2.9) с нуля на каждом шаге расточительно. Вместо этого используем *инкрементное обновление*:

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{n+1}\big(R_{n+1}(a) - Q_n(a)\big), \tag{2.10}$$

где $n = N_t(a)$ и $R_{n+1}(a)$ — новейшее вознаграждение для действия a.

Замечание 2.4 (Общее правило обновления). Уравнение (2.10) является частным случаем

$$Q \leftarrow Q + \alpha\,(R - Q), \tag{2.11}$$

при $\alpha = 1/(n+1)$. Для *нестационарных* задач часто предпочтителен постоянный размер шага $\alpha \in (0, 1)$, дающий экспоненциально убывающие веса более старым наблюдениям:

$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i.$$

Теперь ε-жадного агента-бандита можно сформулировать кратко:

Алгоритм 1: ε-жадный агент-бандит

Input: Число рычагов k, уровень исследования ε, горизонт T

Инициализировать $Q(a) \leftarrow 0$, $N(a) \leftarrow 0$ для всех $a \in \mathcal{A}$;
for $t = 1, 2, \ldots, T$ **do**
  С вероятностью $1 - \varepsilon$: $A_t \leftarrow \operatorname{argmax}_a Q(a)$;
  С вероятностью ε: $A_t \leftarrow$ Равномерное($\mathcal{A}$);
  Наблюдать вознаграждение R_t;
  $N(A_t) \leftarrow N(A_t) + 1$;
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$;

2.5 Верхние доверительные границы (UCB)

Стратегия ε-жадности исследует *равномерно случайным образом*, не учитывая, какие рычаги более неопределённы. Более обоснованный подход — исследовать предпочтительно те действия, о которых мы знаем меньше всего. Такова идея *оптимизма перед лицом неопределённости*.

Принцип оптимизма

Интуиция за оптимизмом элегантна: когда мы не уверены в ценности рычага, следует *предполагать лучшее*. Если это оптимистичное допущение окажется верным, мы получим щедрое вознаграждение. Если оно ошибочно — рычаг не столь хорош, как надеялись — мы получили полезную информацию и в будущем назначим ему меньшую доверительную границу. В любом случае оптимизм продуктивен.

Более конкретно, предположим, что для каждого рычага a мы поддерживаем верхнюю доверительную границу $U_t(a)$ такую, что $q_*(a) \le U_t(a)$ выполняется с высокой вероятностью. Выбор рычага с наибольшим $U_t(a)$ одновременно эксплуатирует истинно хорошие рычаги и исследует рычаги, истинная ценность которых ещё достаточно неопределённа, чтобы их верхняя граница была высокой. По мере того как рычаг нажимается, накапливаются наблюдения и граница $U_t(a)$ сжимается вокруг эмпирического среднего $Q_t(a)$; рычаг, оказавшийся субоптимальным, в конечном счёте получит низкую верхнюю границу и будет отброшен.

Определение 2.7 (UCB1)**.** Алгоритм **UCB1** (Auer, Cesa-Bianchi и Fischer, 2002) выбирает действие

$$A_t = \operatorname*{argmax}_{a \in \mathcal{A}} \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right], \tag{2.12}$$

где $c > 0$ — параметр исследования (обычно $c = \sqrt{2}$).

Величина $Q_t(a) + c\sqrt{(\ln t)/N_t(a)}$ служит *верхней доверительной границей* для истинной ценности $q_*(a)$. Второе слагаемое — *бонус за исследование* — велико, когда действие a исследовалось редко, побуждая агента его исследовать. Множитель $\ln t$ в числителе растёт медленно, обеспечивая продолжение посещения неопределённых рычагов по мере роста горизонта, тогда как $N_t(a)$ в знаменателе обеспечивает уменьшение бонуса по мере накопления выборок рычага.

Вывод из неравенства Хёффдинга. Для ограниченных вознаграждений $R \in [0, 1]$ неравенство Хёффдинга даёт

$$\mathbb{P}\big(q_*(a) > Q_t(a) + u\big) \le e^{-2N_t(a)\,u^2}.$$

Приравнивая правую часть t^{-4} (чтобы вероятность сбоя была суммируемой) и решая относительно u, получаем

$$u = \sqrt{\frac{2\ln t}{N_t(a)}},$$

что мотивирует форму (2.12) при $c = \sqrt{2}$.

Числовой пример

Чтобы увидеть UCB1 в действии, рассмотрим простого 3-рукого бандита с неизвестными истинными ценностями. Инициализируем каждый рычаг одним принудительным нажатием (распространённая стратегия инициализации для избежания деления на ноль).

Пример 2.2 (Трассировка выбора рычагов UCB1)**.** Пусть $k = 3$ и после трёх принудительных начальных нажатий ($t = 1, 2, 3$) мы наблюдали вознаграждения $R_1 = 0.8$ от рычага 1, $R_2 = 0.4$ от рычага 2 и $R_3 = 0.6$ от рычага 3. Таким образом, $Q_3(1) = 0.8$, $Q_3(2) = 0.4$, $Q_3(3) = 0.6$, $N_3(a) = 1$ для всех a. Положим $c = \sqrt{2}$.

При $t = 4$ показатели UCB:

$$\begin{aligned}
U_4(1) &= 0.8 + \sqrt{\tfrac{2\ln 4}{1}} \approx 0.8 + 1.664 = 2.464,\\
U_4(2) &= 0.4 + \sqrt{\tfrac{2\ln 4}{1}} \approx 0.4 + 1.664 = 2.064,\\
U_4(3) &= 0.6 + \sqrt{\tfrac{2\ln 4}{1}} \approx 0.6 + 1.664 = 2.264.
\end{aligned}$$

Выбирается рычаг 1. Предположим, что наблюдается $R_4 = 0.3$. Теперь $Q(1) = (0.8 + 0.3)/2 = 0.55$ и $N(1) = 2$.

При $t = 5$:

$$U_5(1) = 0.55 + \sqrt{\tfrac{2\ln 5}{2}} \approx 0.55 + 1.270 = 1.820,$$
$$U_5(2) = 0.4 + \sqrt{\tfrac{2\ln 5}{1}} \approx 0.4 + 1.794 = 2.194,$$
$$U_5(3) = 0.6 + \sqrt{\tfrac{2\ln 5}{1}} \approx 0.6 + 1.794 = 2.394.$$

Выбирается рычаг 3. Ключевое наблюдение: рычаг 1, несмотря на наибольшее начальное вознаграждение, был «наказан» результатом $R_4 = 0.3$, а рычаги 2 и 3 — которые больше не нажимались — сохранили большие бонусы за исследование. Алгоритм продолжит циклиться, пока все рычаги не будут достаточно исследованы, чтобы бонусы уменьшились и стали доминировать эмпирические средние.

На рисунке ниже показано, как доверительные интервалы сужаются по мере нажатия рычага, а показатель UCB сходится к истинному значению.

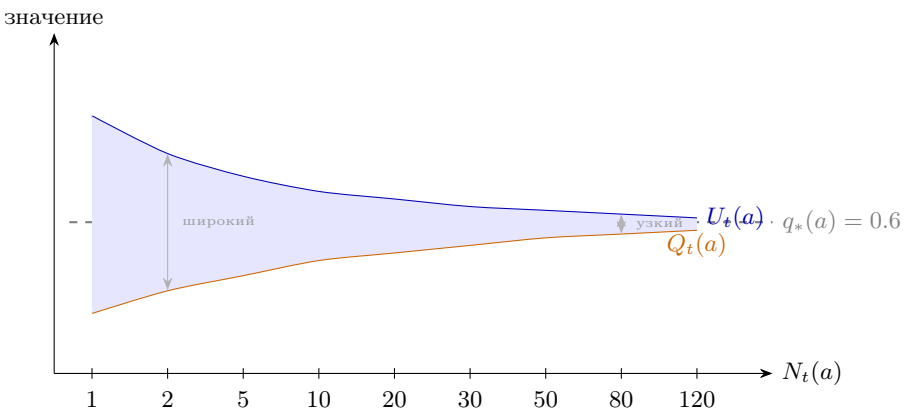


Рис. 2.1: Схема сужения доверительных границ UCB по мере нажатия рычага a. Голубая полоса представляет доверительный интервал $[Q_t(a), U_t(a)]$. По мере роста $N_t(a)$ бонус за исследование $c\sqrt{\ln t / N_t(a)}$ убывает и показатель UCB сходится к истинному значению $q_*(a)$ (пунктирная линия).

Оценка сожаления и набросок доказательства

Теорема 2.3 (Оценка сожаления для UCB1)**.** *Для k-рукого бандита с вознаграждениями в $[0, 1]$ алгоритм UCB1 с $c = \sqrt{2}$ удовлетворяет*

$$\mathbb{E}[\mathcal{R}_T] \le \sum_{a:\Delta_a>0} \left(\frac{8\ln T}{\Delta_a}\right) + \left(1 + \frac{\pi^2}{3}\right) \sum_{a=1}^{k} \Delta_a. \tag{2.13}$$

В частности, $\mathbb{E}[\mathcal{R}_T] = O\left(\sqrt{kT\ln T}\right)$ в наихудшем случае по зазорам Δ_a.

Доказательство теоремы 2.3 поучительно, поскольку иллюстрирует ключевую технику *подсчёта плохих событий*. Набросаем основной аргумент.

Набросок доказательства. Разложение сожаления (2.6) говорит нам, что $\mathbb{E}[\mathcal{R}_T] = \sum_{a:\Delta_a>0} \Delta_a \, \mathbb{E}[N_T(a)]$. Поэтому достаточно ограничить $\mathbb{E}[N_T(a)]$ для каждого субоптимального рычага a.

Зафиксируем субоптимальный рычаг a с зазором $\Delta_a > 0$ и положим $\ell = \lceil 8\ln T/\Delta_a^2 \rceil$. Рычаг a нажимается в момент t только если его показатель UCB превышает показатель оптимального рычага a^*:

$$Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \ge Q_t(a^*) + c\sqrt{\frac{\ln t}{N_t(a^*)}}.$$

Чтобы это произошло при $N_t(a) \ge \ell$, либо эмпирическое среднее a^* должно быть намного ниже истинного значения, либо эмпирическое среднее a — намного выше истинного значения. По неравенству Хёффдинга

(применённому с граничным суммированием по всем t) вероятность любого из этих событий после ℓ нажатий не превышает $2t^{-4}$, что суммируемо. Тщательное вычисление тогда даёт

$$\mathbb{E}[N_T(a)] \le \ell + \sum_{t=1}^{T} \mathbb{P}(\text{плохое событие в } t) \le \frac{8 \ln T}{\Delta_a^2} + 1 + \frac{\pi^2}{3}.$$

Умножение на Δ_a и суммирование по субоптимальным рычагам даёт (2.13). □

Замечание 2.5. Ключевое понимание состоит в том, что UCB достигает сожаления $O(\log T)$ — значительно лучше, чем линейное сожаление $\Omega(T)$ у ε-жадного с фиксированным ε. Более того, исследование направленное, а не равномерное: рычаги, которые более неопределённы или имеют более высокие эмпирические средние, получают больше внимания. Константа $\pi^2/3$ в (2.13) возникает следующим образом: вероятность сбоя по Хёффдингу за раунд равна $2t^{-4}$, но суммирование по всем возможным значениям двух размеров выборки (каждый до t) вносит множитель t^2, давая вклад за раунд $t^2 \cdot 2t^{-4} = 2t^{-2}$. Суммирование даёт $2\sum_{t=1}^{\infty} t^{-2} = 2 \cdot \pi^2/6 = \pi^2/3$.

2.6 Выборка Томпсона

Альтернативный подход к компромиссу между исследованием и эксплуатацией — поддерживать *байесовский апостериорный* подход над ценностями действий и использовать его для направления исследования. Вместо построения явной оптимистичной границы апостериорная неопределённость сама по себе выполняет эту работу.

Интуиция за апостериорной выборкой

Выборка Томпсона — введённая Уильямом Р. Томпсоном в 1933 году, задолго до появления термина «задача бандита» — основана на красиво простой идее: действуй так, как если бы мир был настолько хорош, насколько это правдоподобно. На каждом шаге мы извлекаем одну выборку из апостериорного распределения ценности каждого рычага, а затем жадно действуем с учётом этих выборок.

Почему это работает? Рассмотрим два рычага: один хорошо исследованный (узкое апостериорное распределение) и один плохо исследованный (широкое апостериорное распределение). Для хорошо исследованного рычага выборка почти наверное окажется близко к истинному среднему. Для плохо исследованного рычага выборка с ненулевой вероятностью может оказаться намного выше истинного среднего. Это означает, что неопределённые рычаги исследуются с большей вероятностью, естественным образом реализуя принцип оптимизма — но в случайном, байесовском, а не наихудшем, частотном смысле.

Принципиально, что вероятность выбора рычага a равна апостериорной вероятности того, что a является оптимальным:

$$\mathbb{P}(A_t = a) = \mathbb{P}\big(\tilde{q}(a) = \max_b \tilde{q}(b)\big) \approx \mathbb{P}\big(q_*(a) = \max_b q_*(b) \;\big|\; \text{данные}\big).$$

Это свойство *вероятностного сопоставления*: агент выбирает каждый рычаг пропорционально вероятности того, что он является оптимальным с учётом всего наблюдённого. По мере накопления свидетельств эти вероятности концентрируются на истинно лучшем рычаге.

Определение 2.8 (Выборка Томпсона)**. Выборка Томпсона** (Thompson, 1933) действует следующим образом на каждом шаге:

1. Для каждого рычага a извлекаем выборку $\tilde{q}(a)$ из апостериорного распределения $q_*(a)$.
2. Нажимаем рычаг с наибольшей выборкой: $A_t = \operatorname{argmax}_a \tilde{q}(a)$.
3. Обновляем апостериорное распределение по наблюдённому вознаграждению.

Бернуллиевы бандиты с бета-априорными распределениями. Рассмотрим случай, когда вознаграждения являются бернуллиевыми с неизвестной вероятностью успеха θ_a для каждого рычага. Если поместить априорное распределение $\mathrm{Beta}(\alpha_a, \beta_a)$ на θ_a, то после наблюдения s_a успехов и f_a неудач апостериорное распределение равно

$$\theta_a \mid \text{данные} \;\sim\; \mathrm{Beta}(\alpha_a + s_a,\ \beta_a + f_a).$$

Тогда выборка Томпсона извлекает $\tilde{\theta}_a \sim \mathrm{Beta}(\alpha_a + s_a, \beta_a + f_a)$ для каждого рычага и нажимает $A_t = \operatorname{argmax}_a \tilde{\theta}_a$.

Бета-распределение сопряжено с бернуллиевским правдоподобием — то есть обновления апостериорного распределения имеют простую замкнутую форму. Это делает алгоритм чрезвычайно эффективным: численное интегрирование не требуется, а каждое обновление сводится лишь к инкременту двух счётчиков.

Алгоритм 2: Выборка Томпсона для бернуллиевских бандитов

Input: Число рычагов k, параметры априорного распределения α_0, β_0 (напр. $\alpha_0 = \beta_0 = 1$)
Инициализировать $S(a) \leftarrow 0$, $F(a) \leftarrow 0$ для всех a;
for $t = 1, 2, \ldots, T$ **do**
  for $a = 1, \ldots, k$ **do**
    Извлечь $\tilde{\theta}_a \sim \mathrm{Beta}(\alpha_0 + S(a),\ \beta_0 + F(a))$;
  $A_t \leftarrow \mathrm{argmax}_a\ \tilde{\theta}_a$;
  Наблюдать вознаграждение $R_t \in \{0, 1\}$;
  if $R_t = 1$ **then**
    $S(A_t) \leftarrow S(A_t) + 1$
  else
    $F(A_t) \leftarrow F(A_t) + 1$

Пошаговый числовой пример

Пример 2.3 (Выборка Томпсона с тремя бернуллиевыми рычагами)**.** Рассмотрим 3-рукого бернуллиевского бандита с истинными вероятностями успеха $\theta_1 = 0.3$, $\theta_2 = 0.5$, $\theta_3 = 0.7$. Начинаем с равномерных априорных распределений $\mathrm{Beta}(1, 1)$ на каждом рычаге.

Раунд 1. Извлекаем по одной выборке из каждого априорного $\mathrm{Beta}(1, 1) = \mathrm{Uniform}(0, 1)$. Пусть выборки: $\tilde{\theta}_1 = 0.62$, $\tilde{\theta}_2 = 0.29$, $\tilde{\theta}_3 = 0.45$. Выбираем рычаг 1 (наибольшая выборка). Исход — неудача ($R = 0$), поэтому обновляем: апостериорное рычага 1 становится $\mathrm{Beta}(1, 2)$.

Раунд 2. Извлекаем из $\mathrm{Beta}(1, 2)$, $\mathrm{Beta}(1, 1)$, $\mathrm{Beta}(1, 1)$. $\mathrm{Beta}(1, 2)$ скошено к меньшим значениям (среднее $1/3$), поэтому рычаг 1 теперь менее вероятно победит. Пусть $\tilde{\theta}_1 = 0.21$, $\tilde{\theta}_2 = 0.74$, $\tilde{\theta}_3 = 0.38$. Выбирается рычаг 2. Пусть успех ($R = 1$): рычаг 2 становится $\mathrm{Beta}(2, 1)$.

Раунд 3. Рычаг 3 никогда не нажимался и сохраняет априорное $\mathrm{Beta}(1, 1)$. Рычаг 2 имеет апостериорное $\mathrm{Beta}(2, 1)$ (среднее $2/3$, скошено вверх). Пусть выборки: $\tilde{\theta}_1 = 0.18$, $\tilde{\theta}_2 = 0.55$, $\tilde{\theta}_3 = 0.82$, поэтому выбирается рычаг 3. Пусть успех: рычаг 3 становится $\mathrm{Beta}(2, 1)$.

После многих раундов рычаги с более высокими истинными вероятностями накапливают больше успехов, их бета-апостериорные сдвигаются вправо и их выборки стохастически больше. Апостериорные концентрируются, и рычаг 3 в конечном счёте выбирается почти исключительно.

Рисунок 2.2 иллюстрирует эволюцию бета-апостериорных для каждого рычага по мере накопления данных.

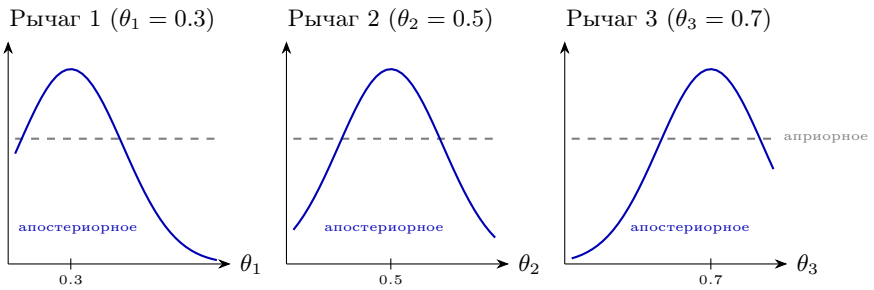


Рис. 2.2: Схематичная эволюция бета-апостериорных для 3-рукого бернуллиевского бандита. Пунктирные линии показывают плоское априорное $\mathrm{Beta}(1, 1)$; сплошные кривые — апостериорное после $T = 50$ раундов. Каждое апостериорное концентрируется вблизи истинной вероятности успеха своего рычага. Апостериорное рычага 3 (наилучшего) является наиболее высоким и узким, отражая наибольший объём накопленных свидетельств.

Вариант с гауссовскими бандитами

Выборка Томпсона естественно распространяется на гауссовские распределения вознаграждений. Пусть $R_t \mid A_t = a \sim \mathcal{N}(\mu_a, \sigma^2)$ с известной дисперсией σ^2 и неизвестным средним μ_a. Сопряжённым априорным является $\mu_a \sim \mathcal{N}(\mu_0, \sigma_0^2)$. После n наблюдений с выборочным средним $\bar{r}_a$ апостериорное равно

$$\mu_a \mid \text{данные} \ \sim\ \mathcal{N}\left(\frac{\sigma^{-2} \cdot n\, \bar{r}_a + \sigma_0^{-2} \mu_0}{\sigma^{-2} \cdot n + \sigma_0^{-2}},\ \frac{1}{\sigma^{-2} \cdot n + \sigma_0^{-2}} \right).$$

На каждом шаге извлекаем выборку $\tilde{\mu}_a$ из этого апостериорного для каждого рычага и выбираем $A_t = \operatorname{argmax}_a \tilde{\mu}_a$. Гауссовский случай представляет собой непрерывный аналог бета-бернуллиевской модели и удобен, когда вознаграждения не ограничены.

Выборка Томпсона против UCB

Оба алгоритма достигают сожаления $O(\log T)$, но различаются по характеру. UCB — *детерминированный* алгоритм (при данных наблюдениях он всегда делает один и тот же выбор), и его оптимизм явен через доверительный член. Выборка Томпсона — *случайный* алгоритм: даже если текущие выборки случайно не выбирают наилучший рычаг, вероятность этого убывает по мере накопления свидетельств.

На практике выборка Томпсона нередко достигает меньшего сожаления, чем UCB1, особенно на начальной фазе, когда априорные информативны. Её главное ограничение — необходимость задать априорное распределение и извлекать выборки из апостериорного, что может быть вычислительно дорогим для сложных моделей. UCB, напротив, не требует априорного и прост в реализации в любом контексте, где значимы эмпирические средние.

Замечание 2.6. Выборка Томпсона обладает рядом привлекательных свойств:

- Она естественно балансирует исследование и эксплуатацию: неопределённые рычаги исследуются, поскольку их апостериорное широко, что с ненулевой вероятностью порождает высокие выборки.
- Эмпирически она нередко соответствует или превосходит UCB.
- Она достигает байесовского сожаления $O(\log T)$ (Kaufmann, Korda и Munos, 2012) и частотного сожаления $O(\sqrt{kT \ln k})$ во многих постановках.
- Алгоритм модульный: изменение модели вознаграждения требует лишь замены правдоподобия и сопряжённого априорного, не затрагивая высокоуровневый цикл.

2.7 Нижняя оценка Лаи–Роббинса

Мы видели, что UCB1 и выборка Томпсона оба достигают сожаления $O(\log T)$. Возникает естественный вопрос: является ли $O(\log T)$ наилучшим возможным, или более умный алгоритм мог бы работать ещё лучше? Основополагающий результат Лаи и Роббинса (1985) отвечает на этот вопрос однозначно — ни один состоятельный алгоритм не может работать лучше, чем $\Omega(\log T)$.

Почему $\log T$ является фундаментальным пределом

Интуитивная причина барьера $\log T$ состоит в том, что любой алгоритм, в конечном счёте обучающийся тому, какой рычаг наилучший, должен *непрерывно проверять* гипотезу о субоптимальности данного рычага. Различение рычага a (со средним μ_a) от оптимального рычага (со средним $\mu^* > \mu_a$) является задачей статистической проверки. По стандартным информационно-теоретическим аргументам, если мы наблюдали только n выборок из рычага a, мы можем различить μ_a и μ^* лишь при $n \gtrsim 1/\mathrm{KL}(\nu_a \| \nu_{a^*})$, где дивергенция Куллбака–Лейблера показывает, насколько различно выглядят два распределения на одну выборку. Чтобы алгоритм был *состоятельным* — то есть в конечном счёте определял наилучший рычаг на каждом экземпляре задачи — он должен нажимать рычаг a не менее $\Omega(\log T/\mathrm{KL}(\nu_a \| \nu_{a^*}))$ раз. Умножение на зазор Δ_a даёт нижнюю оценку (2.14) ниже.

Определение 2.9 (Состоятельный алгоритм)**.** Алгоритм бандита является **состоятельным**, если для каждого экземпляра задачи и каждого субоптимального рычага a

$$\mathbb{E}[N_T(a)] = o(T^\alpha) \quad \text{для всех } \alpha \in (0, 1).$$

Иными словами, ожидаемое число нажатий любого субоптимального рычага растёт субполиномиально с T, поэтому доля времени, потраченная на него, стремится к нулю. Состоятельность — минимальное требование рациональности: состоятельный алгоритм в конечном счёте перестаёт тратить время на явно худшие действия. Заметим, что любой алгоритм с сожалением $O(\log T)$ (такой как UCB1 или выборка Томпсона) автоматически является состоятельным.

Теорема 2.4 (Лай и Роббинс, 1985)**.** *Для любого состоятельного алгоритма бандита и любого экземпляра задачи с зазорами субоптимальности $\Delta_a > 0$,*

$$\liminf_{T \to \infty} \frac{\mathbb{E}[\mathcal{R}_T]}{\ln T} \geq \sum_{a : \Delta_a > 0} \frac{\Delta_a}{\mathrm{KL}(\nu_a \| \nu_{a^*})}, \tag{2.14}$$

где $\mathrm{KL}(\nu_a \| \nu_{a^*})$ *обозначает дивергенцию Куллбака–Лейблера между распределениями вознаграждений рычага* a *и оптимального рычага* a^*.

Набросок информационно-теоретического аргумента

Доказательство теоремы 2.4 можно понять через аргумент *смены меры*. Предположим, что у нас есть задача бандита $\mathcal{P}$ и мы рассматриваем альтернативную задачу $\mathcal{P}'$, отличающуюся только распределением вознаграждений рычага a: в $\mathcal{P}'$ рычаг a фактически является наилучшим. Любой состоятельный алгоритм должен вести себя по-разному на $\mathcal{P}$ и $\mathcal{P}'$, нажимая рычаг a реже на $\mathcal{P}$ (где он субоптимален), чем на $\mathcal{P}'$ (где он оптимален). Дивергенция $\mathrm{KL}(\nu_a \| \nu_{a^*})$ управляет скоростью, с которой алгоритм может различить $\mathcal{P}$ и $\mathcal{P}'$ по выборкам из рычага a. Лемма об обработке данных затем устанавливает, что число нажатий рычага a на $\mathcal{P}$ должно расти не медленнее, чем $\ln T / \mathrm{KL}(\nu_a \| \nu_{a^*})$, чтобы алгоритм надёжно различал две задачи.

Числовой пример: гауссовский случай

Для гауссовских вознаграждений с единичной дисперсией $\mathrm{KL}(\nu_a \| \nu_{a^*}) = \Delta_a^2/2$, и оценка принимает вид

$$\liminf_{T\to\infty} \frac{\mathbb{E}[\mathcal{R}_T]}{\ln T} \geq \sum_{a:\Delta_a>0} \frac{2}{\Delta_a}.$$

Пример 2.4 (Гауссовская нижняя оценка, два рычага)**.** Рассмотрим 2-рукого гауссовского бандита с $q_*(1) = 0$ и $q_*(2) = \Delta > 0$ (единичные дисперсии). Нижняя оценка даёт

$$\liminf_{T\to\infty} \frac{\mathbb{E}[\mathcal{R}_T]}{\ln T} \geq \frac{2}{\Delta}.$$

Заметим, что при малом Δ — два рычага трудно различить — оценка велика: алгоритм должен нажимать субоптимальный рычаг много раз, чтобы убедиться, что он действительно не является наилучшим. Напротив, при большом Δ рычаги легко различить, и требуется лишь $O(1/\Delta)$ нажатий, что влечёт пропорционально меньшее сожаление за каждое принудительное исследование. Это формализует интуицию о том, что задачи бандита с почти равными ценностями рычагов внутренне сложнее тех, где ценности явно различаются.

Замечание 2.7. Оценка Лаи–Роббинса означает, что сожаление $\Omega(\log T)$ неизбежно для любого алгоритма, в конечном счёте обучающегося оптимальному действию. Оба алгоритма — UCB и выборка Томпсона — достигают сожаления порядка $O(\log T)$, поэтому они *асимптотически оптимальны* в смысле зависимости от конкретного экземпляра задачи. Алгоритм KL-UCB (Garivier и Cappé, 2011) и выборка Томпсона с сопряжёнными априорными известны тем, что достигают точной константы Лаи–Роббинса, а не только правильного порядка.

2.8 Контекстные бандиты

Рассматриваемая до сих пор постановка бандита является *бесконтекстной*: агент наблюдает только вознаграждения, но не какую-либо побочную информацию об окружении. Однако в большинстве реальных приложений перед каждым решением доступен богатый контекст. Рекомендательная система знает историю просмотров пользователя; клиническое испытание знает возраст и биомаркеры пациента; система размещения рекламы знает содержание просматриваемой веб-страницы. Игнорирование этого контекста приводит к единой политике для всех, тогда как его учёт позволяет персонализацию. Это мотивирует задачу *контекстного бандита*.

Определение 2.10 (Контекстный бандит)**.** На каждом шаге t:

1. Агенту предъявляется контекст $x_t \in \mathcal{X}$.
2. Агент выбирает $A_t \in \mathcal{A}$.
3. Наблюдается вознаграждение R_t, извлечённое из распределения, зависящего как от x_t, так и от A_t.

Цель — обучить политику $\pi : \mathcal{X} \to \mathcal{A}$, максимизирующую ожидаемое суммарное вознаграждение.

Ключевое отличие от стандартного бандита: оптимальное действие может меняться в зависимости от контекста. В клиническом испытании лечение, наилучшее для пожилых пациентов, может быть вредным для молодых. Хороший алгоритм контекстного бандита должен обучаться этой зависимости по данным.

LinUCB: верхние доверительные границы с линейными моделями

Широко используемым алгоритмом контекстного бандита является **LinUCB** (Li et al., 2010), предполагающий, что ожидаемое вознаграждение линейно по контексту:

$$\mathbb{E}[R_t \mid x_t, A_t = a] = x_t^\top \theta_a^*, \tag{2.15}$$

где $\theta_a^* \in \mathbb{R}^d$ — неизвестный вектор параметров для рычага a. По наблюдениям можно вычислить оценку наименьших квадратов $\hat{\theta}_a$, а гребневая регрессия предоставляет доверительный эллипсоид вокруг неё.

Определение 2.11 (Правило выбора LinUCB)**.** На каждом шаге t LinUCB выбирает

$$A_t = \operatorname*{argmax}_{a \in \mathcal{A}} \left[x_t^\top \hat{\theta}_a(t) + \alpha \sqrt{x_t^\top A_a(t)^{-1} x_t} \right], \tag{2.16}$$

где $A_a(t) = \lambda I + \sum_{s<t:\, A_s = a} x_s x_s^\top$ — регуляризованная матрица плана для рычага a, а $\alpha > 0$ — параметр исследования.

Член $\sqrt{x_t^\top A_a(t)^{-1} x_t}$ является *эллипсоидальным бонусом за исследование*: он велик, когда контекст x_t лежит в направлении, которое не было хорошо исследовано для рычага a, и убывает по мере наблюдения более разнообразных контекстов. LinUCB удовлетворяет оценке сожаления $O(d\sqrt{T \ln T})$ при выполнении допущения о линейности (2.15), где d — размерность контекста.

Практические применения

Контекстные бандиты естественно возникают в ряде важных областей.

Онлайн-реклама. Когда пользователь посещает веб-страницу, система должна решить, какую рекламу показать. Контекст x_t включает характеристики пользователя (демография, история просмотров) и характеристики страницы (тема, время суток). Каждый рычаг соответствует рекламному объявлению, а вознаграждение — индикатору клика. Поскольку предпочтения пользователей варьируются в широких пределах, персонализированная политика может значительно превзойти политику, показывающую одну и ту же рекламу всем. LinUCB применялся компанией Yahoo! для рекомендации новостных статей (Li et al., 2010), принеся существенный прирост показателя кликабельности по сравнению с бесконтекстными подходами.

Клинические испытания и персонализированная медицина. В адаптивных клинических испытаниях пациент поступает с вектором признаков x_t (возраст, вес, генетические маркеры) и врач должен назначить лечение из множества $\mathcal{A}$. Исход R_t — показатель здоровья, например уменьшение размера опухоли или артериального давления. Политика контекстного бандита обучается в ходе испытания тому, какое лечение наилучшее для какой подгруппы пациентов. По сравнению с классическим рандомизированным контролируемым испытанием (назначающим лечения без учёта признаков пациентов), контекстный бандит способен уменьшить число пациентов, получающих худшее лечение, по-прежнему собирая достаточно данных для получения обоснованных выводов.

Рекомендация новостей. Новостной сайт должен решить, какую статью показать пользователю на главной странице. Контекст кодирует историю пользователя и характеристики статьи; вознаграждение — время прочтения или вовлечённость. Компромисс между исследованием и эксплуатацией здесь особенно деликатен: показ только статей, которые пользователь уже предпочитает, жертвует шансом обнаружить новую тему, которая могла бы ему понравиться.

От контекстных бандитов к полноценному RL

Контекстные бандиты занимают естественное промежуточное положение между задачей бандита без состояний и полноценным обучением с подкреплением.

Бандиты без состояний Нет контекста; одно глобально оптимальное действие.

Контекстные бандиты Наблюдается контекст x_t; оптимальное действие зависит от x_t; но действия не влияют на будущие контексты.

Полноценный RL (MDP) Наблюдается состояние s_t; оптимальное действие зависит от s_t; и действия влияют на распределение будущих состояний.

Критическое различие состоит в том, что в постановке контекстного бандита действия не влияют на распределение будущих контекстов — нажатие рычага сейчас никак не сказывается на том, какой контекст появится следующим. В полноценном RL (изучаемом начиная с главы 3) действия формируют будущее, требуя от агента рассуждений о долгосрочных последствиях. Техники, разработанные для контекстных бандитов — в особенности бонус за исследование в стиле UCB и методы апостериорной выборки — вдохновили алгоритмы для исследования в RL, такие как UCRL (Jaksch, Ortner и Auer, 2010) и RLSVI (Osband et al., 2016).

2.9 Эмпирическое сравнение

Чтобы подкрепить теоретическое обсуждение практикой, рассмотрим стандартный *10-рукий гауссовский стенд*, введённый Саттоном и Барто:

Пример 2.5 (Гауссовский 10-рукий стенд)**.** Каждый рычаг $a = 1, \ldots, 10$ имеет истинную ценность $q_*(a) \sim \mathcal{N}(0, 1)$, взятую один раз и затем фиксированную. Вознаграждения генерируются как $R_t \mid A_t = a \sim \mathcal{N}(q_*(a), 1)$.

Результаты усредняются по многим независимо взятым экземплярам задачи для снижения влияния какой-либо конкретной конфигурации ценностей рычагов.

Кривые сожаления

Рисунок 2.3 показывает схематичные кривые суммарного сожаления для алгоритмов, изученных в этой главе. Стоит отметить ряд качественных особенностей.

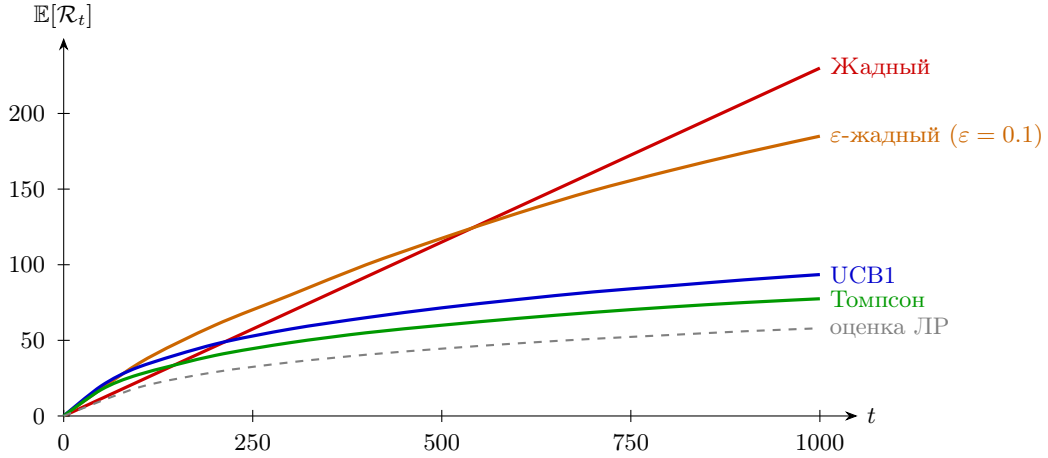


Рис. 2.3: Схематичное суммарное сожаление $\mathbb{E}[\mathcal{R}_t]$ в зависимости от шага t для алгоритмов, изученных в этой главе, на 10-руком гауссовском стенде. Чисто жадный алгоритм проявляет линейное сожаление, тогда как ε-жадный достигает меньшего, но всё ещё линейного сожаления. UCB1 и выборка Томпсона достигают логарифмического сожаления (вогнутые кривые), при этом выборка Томпсона чуть ближе к нижней оценке Лаи–Роббинса (ЛР). Фактическая эффективность зависит от конкретного экземпляра задачи и настроек параметров.

Чисто жадный. Поскольку чисто жадный алгоритм рано фиксируется на каком-то рычаге, его сожаление нарастает линейно. Наклон крут: алгоритм с существенной вероятностью застревает на субоптимальном рычаге на весь горизонт.

ε-жадный. При $\varepsilon = 0.1$ десять процентов всех шагов являются чисто случайными, давая алгоритму достаточно возможностей для обнаружения лучших рычагов. Однако эта случайность продолжается бесконечно, поэтому сожаление всё равно линейно растёт со скоростью $\varepsilon\,\bar{\Delta}/k$ (где $\bar{\Delta}$ — средний зазор субоптимальности), лишь с меньшим коэффициентом, чем у чисто жадного.

UCB1. Логарифмическое сожаление явно видно как изгиб кривой к горизонтали. Начальный переходный период несколько дорог, поскольку UCB должен опробовать каждый рычаг хотя бы раз, а начальные бонусы за исследование велики для всех рычагов. Как только каждый рычаг опробован, алгоритм переходит в почти жадный режим, исследуя только при наличии реальной неопределённости.

Выборка Томпсона. Выборка Томпсона обычно работает несколько лучше UCB1 по суммарному сожалению, иногда приближаясь к нижней оценке Лаи–Роббинса. Это преимущество наиболее выражено, когда априорное хорошо задано; неправильно заданные априорные могут ухудшить производительность.

Чувствительность к параметрам

Оба алгоритма — UCB1 и ε-жадный — обнажают параметр (c и ε соответственно), управляющий скоростью исследования.

- Для UCB1 слишком малое c вызывает недостаточное исследование: субоптимальные рычаги отбрасываются до того, как их ценности надёжно оценены. Слишком большое c вызывает избыточное исследование: агент тратит слишком много времени на непривлекательные рычаги. Теоретически обоснованный выбор $c = \sqrt{2}$ хорошо работает на многих эталонах, но практики порой настраивают c на отложенных данных.
- Для ε-жадного распространённая эвристика — убывающее $\varepsilon_t = \min(1, \varepsilon_0/t)$, достигающее сублинейного (хотя и не логарифмического) сожаления. Даже при убывании оптимальное расписание зависит от специфических констант задачи, неизвестных заранее.
- Выборка Томпсона сравнительно устойчива к ошибочно заданному априорному, когда оно неинформативно (например, $\mathrm{Beta}(1,1)$), и не обнажает явного параметра исследования. Это делает её более готовой к использованию «из коробки» на практике.

Таблица 2.1 обобщает ключевые свойства всех алгоритмов, рассмотренных в этой главе.

Таблица 2.1: Сравнение алгоритмов многорукого бандита.

Алгоритм	Сожаление	Парам.	Апр.	Примечания
Чисто жадный	$\Theta(T)$	—	Нет	Может застрять на субоптим. рычаге
ε-жадный (фикс.)	$\Theta(\varepsilon T)$	ε	Нет	Простой; линейное сожаление
ε-жадный (убыв.)	$O(\sqrt{T})$	распис.	Нет	Субопт.; зависит от расписания
UCB1	$O(\log T)$	c	Нет	Детерм.; оптим. в наихуд. случае
Выборка Томпсона	$O(\log T)$	—	Да	Случайный; лучший эмпирически

2.10 Резюме

Многорукий бандит — одна из наиболее тщательно изученных задач последовательного принятия решений, и не без оснований: она достаточно проста для строгого анализа, однако достаточно богата, чтобы уловить ключевое противоречие между исследованием и эксплуатацией, пронизывающее всё обучение с подкреплением.

Мы начали с базовой формулировки в разделе 2.1 и установили разложение сожаления $\mathbb{E}[\mathcal{R}_T] = \sum_a \Delta_a \, \mathbb{E}[N_T(a)]$ в разделе 2.2. Это тождество переформулирует задачу: контроль сожаления означает контроль частоты выбора субоптимальных рычагов, что требует от алгоритма обучения рангу рычагов по зашумлённым наблюдениям.

Семейство ε-жадных (раздел 2.3) даёт простую базовую линию. Резервируя фиксированную долю шагов для случайного исследования, алгоритм предотвращает зависание на плохом рычаге, но платит за это ценой постоянного случайного исследования, что даёт линейное сожаление. Инкрементное правило обновления (раздел 2.4) показывает, что точные оценки ценностей можно поддерживать за постоянное время на шаг, что важно с практической точки зрения.

Верхние доверительные границы (раздел 2.5) предлагают обоснованное решение. Добавляя бонус за исследование, пропорциональный $\sqrt{\ln t / N_t(a)}$, UCB1 направляет исследование к рычагам, которые либо истинно хороши, либо недостаточно опробованы. Алгоритм достигает сожаления $O(\log T)$, что теорема 2.3

уточняет через зависящую от экземпляра оценку. Доказательство оценки сожаления иллюстрирует широко применимую технику: разложить плохие события по рычагу и времени, ограничить вероятность каждого события с помощью концентрационных неравенств, затем сложить части.

Выборка Томпсона (раздел 2.6) принимает байесовскую точку зрения, поддерживая апостериорное распределение по ценности каждого рычага и извлекая из него выборки для принятия решений. В бета-бернуллиевской модели обновления апостериорного тривиально дешевы (инкремент двух счётчиков), а свойство вероятностного сопоставления обеспечивает калиброванность исследования по фактической апостериорной неопределённости. Выборка Томпсона соответствует гарантии сожаления UCB1 $O(\log T)$, при этом нередко превосходя его эмпирически, и чисто обобщается на гауссовских бандитов и другие модели вознаграждений из показательного семейства.

Нижняя оценка Лаи–Роббинса (раздел 2.7) замыкает круг. Ни один состоятельный алгоритм не может достичь сожаления, растущего медленнее $c \ln T$, где c — зависящая от экземпляра задачи константа, определяемая дивергенциями KL между распределениями рычагов. Оба алгоритма — UCB и выборка Томпсона — достигают этой скорости, делая их *асимптотически оптимальными*. Идея доказательства — что для различения субоптимального рычага от наилучшего требуется минимальное число наблюдений, определяемое теорией информации — является прообразом аргументов нижних оценок во всей теории обучения с подкреплением.

Контекстные бандиты (раздел 2.8) показывают, как рамка бандита масштабируется к реальной сложности. Обусловливая решения на наблюдаемом контексте, алгоритмы типа LinUCB достигают персонализированных политик, превосходящих любой бесконтекстный подход, ценой оценки параметров большей размерности. Применения охватывают онлайн-рекламу, клинические испытания и рекомендацию контента, а ключевые идеи — оптимизм в условиях неопределённости, апостериорная выборка — напрямую переносятся из бесконтекстной постановки.

Заглядывая вперёд: задача бандита — не просто упрощённая разминка; это строительный блок. Каждый алгоритм RL должен по существу решать локальную версию задачи исследования–эксплуатации в каждом состоянии. Инструменты, разработанные здесь — доверительные границы, байесовские апостериорные, анализ сожаления, нижние оценки через теорию информации — вновь появятся в анализе временного разностного обучения, методов градиента политики и современных стратегий исследования в глубоком RL в последующих главах.

Упражнения

Упражнение 2.1. Докажите, что ε-жадная стратегия с фиксированным $\varepsilon > 0$ и хотя бы одним строго субоптимальным действием удовлетворяет $\mathbb{E}[\mathcal{R}_T] = \Omega(T)$.

Упражнение 2.2. Предположим, что оптимальное действие имеет наибольшую текущую оценку. Найдите:

(a) вероятность выбора оптимального действия при ε-жадном алгоритме;

(b) вероятность выбора конкретного субоптимального действия;

(c) ожидаемое число исследовательских шагов за T раундов.

Упражнение 2.3. Пусть $R_1, \ldots, R_n$ — н.о.р. вознаграждения со средним $q_*(a)$ и дисперсией σ_a^2. Докажите, что выборочное среднее $Q_n(a) = \frac{1}{n}\sum_{i=1}^n R_i$ является несмещённым и имеет дисперсию σ_a^2/n.

Упражнение 2.4. Выведите явную форму обновления с постоянным размером шага:

$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i.$$

Обсудите, как веса прошлых наблюдений ведут себя при росте n.

Упражнение 2.5. Постройте конкретный пример двурукого бандита, в котором чисто жадный алгоритм, начиная с $Q_1(1) = Q_1(2) = 0$, с положительной вероятностью навсегда фиксируется на субоптимальном рычаге. Явно задайте распределения вознаграждений.

Упражнение 2.6. Покажите, что любой алгоритм, удовлетворяющий $\mathbb{P}(A_t = a) \geq \delta$ для всех a, всех t и некоторого $\delta > 0$, должен иметь $\mathbb{E}[\mathcal{R}_T] \geq cT$ для некоторой константы $c > 0$ (предполагая, что хотя бы один рычаг строго субоптимален).

Упражнение 2.7 (Бонус за исследование UCB)**.** Начиная с неравенства Хёффдинга для ограниченных случайных величин, выведите бонус за исследование UCB1 $\sqrt{2\ln t/N_t(a)}$. *Подсказка*: установите вероятность сбоя равной t^{-4} и примененитe граничное суммирование по рычагам.

Упражнение 2.8 (Симуляция выборки Томпсона)**.** Рассмотрим 3-рукого бернуллиевского бандита с вероятностями успеха $\theta_1 = 0.3$, $\theta_2 = 0.5$, $\theta_3 = 0.7$. Используя априорные $\mathrm{Beta}(1,1)$, промоделируйте выборку Томпсона за $T = 1000$ шагов. Постройте апостериорные распределения $\theta_1, \theta_2, \theta_3$ при $t = 10$, $t = 100$ и $t = 1000$. Пронаблюдайте, как апостериорные концентрируются вокруг истинных значений.

Упражнение 2.9. Для гауссовского бандита с единичной дисперсией дивергенция KL равна $\mathrm{KL}(\mathcal{N}(\mu_a, 1) \| \mathcal{N}(\mu^*, 1)) = \Delta_a^2/2$. Подставьте в оценку Лаи–Роббинса (2.14) и интерпретируйте: как сложность различения рычага a от оптимума зависит от Δ_a?

Упражнение 2.10 (Вывод LinUCB)**.** Предположим, что вознаграждения удовлетворяют $R_t = x_t^\top \theta^* + \eta_t$, где $\eta_t \sim \mathcal{N}(0, \sigma^2)$ и все рычаги разделяют единственный параметр $\theta^* \in \mathbb{R}^d$. Покажите, что оценка гребневой регрессии $\hat{\theta}_t = A_t^{-1} b_t$ (где $A_t = \lambda I + \sum_{s \le t} x_{A_s,s} x_{A_s,s}^\top$ и $b_t = \sum_{s \le t} R_s x_{A_s,s}$) является оценкой максимума апостериорного вероятности при гауссовском априорном $\theta^* \sim \mathcal{N}(0, \sigma^2 \lambda^{-1} I)$. Убедитесь, что бонус за исследование $\sqrt{x_t^\top A_t^{-1} x_t}$ является апостериорным стандартным отклонением $x_t^\top \theta^*$.

Глава 3

Марковские процессы принятия решений и уравнение Беллмана

Теория динамического программирования зиждется на единственном наблюдении: будущее независимо от прошлого при условии знания настоящего.

В главе 2 мы изучали задачу многорукого бандита, в которой действия не имеют последствий помимо немедленного вознаграждения. Реальное принятие решений богаче: усердная учёба сегодня влияет на результат завтрашнего экзамена, который в свою очередь влияет на карьерные перспективы в следующем году. Выборы агента формируют мир, в котором он затем существует. Чтобы строго рассуждать о подобных задачах, нам необходима математическая рамка, улавливающая взаимодействие состояний, действий, вознаграждений и переходов — рамка, достаточно ёмкая, чтобы кодировать шахматы, локомоцию роботов, диалоговые системы и выравнивание языковых моделей одинаково.

Этой рамкой является *марковский процесс принятия решений* (MDP). Введённый Ричардом Беллманом в 1950-х годах и формализованный Говардом (1960) и Путерманом (1994), MDP стал стандартной моделью последовательного принятия решений в условиях неопределённости. Практически каждый алгоритм, изучаемый в этой книге — динамическое программирование, методы Монте-Карло, временное разностное обучение и методы градиента политики — можно понять как стратегию решения или приближённого решения MDP.

Эта глава вводит MDP с нуля. Мы начинаем с марковского свойства, являющегося как математическим удобством, так и глубоким утверждением о структуре мира. Затем формально определяем MDP, вводим политики и функции ценности, выводим знаменитые уравнения Беллмана и изучаем их теоретические свойства. Глава завершается детальным числовым примером, связывающим все концепции воедино.

3.1 Марковское свойство

Мотивация

Представьте робота, перемещающегося в здании. В любой момент у робота есть положение, ориентация и скорость. Чтобы решить, куда двигаться дальше, нужно ли ему помнить каждый шаг с момента включения? Интуитивно — нет — знания текущего положения и скорости должно быть достаточно для планирования следующего хода. Эта интуиция и есть марковское свойство.

Формально рассмотрим последовательность случайных величин $S_0, A_0, R_1, S_1, A_1, \ldots$, представляющих состояние, действие и вознаграждение на каждом шаге. Полная история до момента t — кортеж

$$H_t = (S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{t-1}, A_{t-1}, R_t, S_t).$$

В принципе следующее состояние S_{t+1} может зависеть от всей этой истории. Марковское свойство утверждает, что важно только текущее состояние S_t.

Определение 3.1 (Марковское свойство)**.** Случайный процесс $(S_t)_{t\geq 0}$ удовлетворяет **марковскому свойству**, если для всех $t \geq 0$ и всех состояний $s, s' \in \mathcal{S}$ и действий $a \in \mathcal{A}$,

$$\mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a, S_{t-1}, A_{t-1}, \ldots, S_0, A_0) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a). \tag{3.1}$$

Если (3.1) выполняется, говорят, что текущее состояние S_t является **достаточной статистикой** для истории H_t.

Замечание 3.1. В постановке RL мы дополнительно требуем, чтобы совместное распределение следующего состояния *и* вознаграждения удовлетворяло марковскому свойству:

$$\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid H_t) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t).$$

Именно эта форма используется на протяжении всей книги.

Примеры

Пример 3.1 (Модель погоды)**.** Предположим, что завтрашняя погода зависит только от сегодняшней, но не от прошлой недели. Если мы кодируем состояние как текущую погоду (солнечно, облачно или дождливо), марковское свойство выполняется и динамика описывается матрицей переходов 3×3. Добавление большей детализации — температуры, влажности, давления — может дать ещё лучшую марковскую модель.

Пример 3.2 (Навигация робота)**.** Следующее положение мобильного робота зависит от текущего положения и поданной команды скорости. Если мы представляем состояние как (x, y, θ, v)—положение, курс, скорость — динамика примерно марковская. Принципиально, что чисто позиционное состояние (x, y) *не* было бы марковским, поскольку будущее движение зависит также от текущей скорости.

Когда выполняется марковское свойство?

Для многих естественных представлений состояния марковское свойство является хорошим приближением, а не точной истиной. Погодные паттерны имеют дальноиграющие зависимости; положение робота зависит от износа суставов. На практике мы выбираем представление состояния максимально информативным при условии вычислительной приемлемости.

Когда марковское свойство нарушается — то есть когда агент может наблюдать лишь *частичный* вид истинного состояния — мы входим в область *частично наблюдаемых MDP* (POMDP). POMDPs существенно сложнее решать, и мы кратко возвращаемся к ним в главе 16. Пока мы предполагаем, что марковское свойство выполняется точно.

Почему марковское свойство важно?

Марковское свойство имеет глубокое следствие: любое оптимальное правило принятия решений можно сформулировать как функцию только текущего состояния. Без него оптимальная политика могла бы нуждаться в обращении к постоянно растущей истории. Это сделало бы и анализ, и вычисления неприемлемыми. С ним мы можем записать краткие условия оптимальности — уравнения Беллмана — и разработать эффективные алгоритмы для нахождения или приближения оптимального поведения.

3.2 Формальное определение MDP

Определение 3.2 (Марковский процесс принятия решений)**. (Дисконтированный, конечный) марковский процесс принятия решений** — это кортеж

$$\mathcal{M} = (\mathcal{S},\, \mathcal{A},\, p,\, \gamma), \tag{3.2}$$

где:

$\mathcal{S}$ — конечное множество **состояний**;

$\mathcal{A}$ — конечное множество **действий** (пишем $\mathcal{A}(s)$ для действий, доступных в состоянии s);

$p(s', r \mid s, a)$ — **ядро перехода-вознаграждения** (модель среды): вероятность того, что выполнение действия a в состоянии s приводит к следующему состоянию s' и вознаграждению $r \in \mathcal{R} \subset \mathbb{R}$;

$\gamma \in [0, 1)$ — **коэффициент дисконтирования**.

Множество вознаграждений $\mathcal{R}$ считается конечным (или счётным) и ограниченным: существует $R_{\max} > 0$ такое, что $|r| \le R_{\max}$ для всех $r \in \mathcal{R}$. Ядро p удовлетворяет условию нормировки

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1 \qquad \forall s \in \mathcal{S},\ \forall a \in \mathcal{A}(s). \tag{3.3}$$

Функция p — единственный наиболее важный объект в MDP: она кодирует всё, что может делать среда. На каждом шаге t агент наблюдает $S_t = s$, выбирает $A_t = a$, и среда извлекает $(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot \mid s, a)$.

Производные величины

Работать напрямую с совместным ядром $p(s', r \mid s, a)$ иногда громоздко. Удобнее три производные величины.

Утверждение 3.1 (Производные величины)**.** *Для любых $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, $s' \in \mathcal{S}$:*

$$p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a), \tag{3.4}$$

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r\, p(s', r \mid s, a), \tag{3.5}$$

$$r(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r\, \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}, \quad p(s' \mid s, a) > 0. \tag{3.6}$$

Доказательство. **Уравнение** (3.4)**.** Маргинальная вероятность перехода в s' получается суммированием совместного распределения по всем значениям вознаграждения:

$$\begin{aligned} p(s' \mid s, a) &= \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a). \end{aligned}$$

Уравнение (3.5)**.** По закону полного математического ожидания,

$$\mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r\, \mathbb{P}(R_{t+1} = r \mid S_t = s, A_t = a).$$

Маргинализуя по s':

$$\mathbb{P}(R_{t+1} = r \mid S_t = s, A_t = a) = \sum_{s' \in \mathcal{S}} p(s', r \mid s, a).$$

Подстановка даёт (3.5).

Уравнение (3.6)**.** По определению условного математического ожидания,

$$\mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r\, \mathbb{P}(R_{t+1} = r \mid S_t = s, A_t = a, S_{t+1} = s').$$

Применяя теорему Байеса,

$$\begin{aligned} &\mathbb{P}(R_{t+1} = r \mid S_t = s, A_t = a, S_{t+1} = s') \\ &\quad = \frac{\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)}{\mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a)} = \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}. \end{aligned}$$

Подстановка даёт (3.6). □

3.3 Эпизоды, доходы и коэффициент дисконтирования

Эпизодические и непрерывные задачи

Задачи RL делятся на два вида. В **эпизодической задаче** взаимодействие агента и среды естественно разбивается на конечные отрезки, называемые *эпизодами*: партия в шахматы, захват объекта рукой робота, диалог с пользователем. Каждый эпизод заканчивается в **терминальном состоянии**, после чего среда сбрасывается. В **непрерывной задаче** взаимодействие продолжается бесконечно: торговый алгоритм, контроллер энергосети, рекомендательная система.

Доход

Какой бы ни была задача, цель агента — максимизировать суммарное собранное вознаграждение. Формализуем это с помощью *дохода*.

Определение 3.3 (Доход)**.** **Доход** G_t в момент t — это дисконтированная сумма будущих вознаграждений:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots. \tag{3.7}$$

Для эпизодической задачи, заканчивающейся на шаге T, вознаграждения после T равны нулю, поэтому (3.7) сводится к $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$.

Рекуррентное свойство и его доказательство

Доход удовлетворяет простой, но принципиальной рекурсии.

Теорема 3.2 (Рекуррентный доход)**.** *Для всех t,*

$$G_t = R_{t+1} + \gamma G_{t+1}. \tag{3.8}$$

Доказательство. Начиная с определения (3.7):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma\big(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots\big) = R_{t+1} + \gamma G_{t+1}. \qquad \square$$

Это однострочное доказательство обманчиво важно. Оно гласит, что ценность нахождения в состоянии прямо сейчас равна немедленному вознаграждению плюс дисконтированная ценность следующего состояния — основа каждого уравнения Беллмана и алгоритма временного разностного обучения.

Ограниченность дисконтированного дохода

Утверждение 3.3 (Ограниченный доход)**.** *Если $|R_t| \le R_{\max}$ для всех t и $\gamma \in [0, 1)$, то*

$$|G_t| \le \frac{R_{\max}}{1-\gamma}. \tag{3.9}$$

Доказательство. По неравенству треугольника и формуле суммы геометрического ряда:

$$|G_t| \le \sum_{k=0}^{\infty} \gamma^k |R_{t+k+1}| \le R_{\max} \sum_{k=0}^{\infty} \gamma^k = \frac{R_{\max}}{1-\gamma}. \qquad \square$$

Роль коэффициента дисконтирования

Параметр γ интерполирует между двумя крайностями. При $\gamma = 0$ агент *близорук*: он заботится только о немедленном вознаграждении R_{t+1}. При $\gamma \to 1$ агент *дальновиден*: вознаграждения в далёком будущем важны почти так же, как немедленные. На практике γ выбирается близко к 1 (например, 0.99) для задач, требующих долгосрочного планирования, и ближе к 0 для задач, где наиболее информативны недавние вознаграждения.

Дисконтирование имеет и финансовую интерпретацию: γ — коэффициент процента за шаг, поэтому γ^k — приведённая стоимость вознаграждения, полученного через k шагов.

Пример 3.3 (Вычисление дохода)**.** Предположим, агент наблюдает вознаграждения $R_1 = 1,\ R_2 = 2,\ R_3 = -1,\ R_4 = 3$, после чего следует терминальное состояние (все последующие вознаграждения равны нулю), коэффициент дисконтирования $\gamma = 0.9$.

Вычисляя в обратном порядке, начиная с $t = 4$:

$$\begin{aligned}
&G_4 = 0 \quad \text{(терминальное)},\\
&G_3 = R_4 + \gamma G_4 = 3 + 0.9 \times 0 = 3.000,\\
&G_2 = R_3 + \gamma G_3 = -1 + 0.9 \times 3 = -1 + 2.7 = 1.700,\\
&G_1 = R_2 + \gamma G_2 = 2 + 0.9 \times 1.7 = 2 + 1.53 = 3.530,\\
&G_0 = R_1 + \gamma G_1 = 1 + 0.9 \times 3.53 = 1 + 3.177 = 4.177.
\end{aligned}$$

Заметим, что вычисление в обратном порядке через рекурсию (3.8) эффективно: одного прохода по траектории достаточно.

3.4 Политики

Политика определяет поведение агента. Формально она отображает состояния в распределения над действиями.

Определение 3.4 (Политика)**. Стохастическая политика** π — это семейство распределений вероятностей над действиями, индексированных состояниями:

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s), \qquad s \in \mathcal{S},\ a \in \mathcal{A}(s). \tag{3.10}$$

Детерминированная политика — частный случай, в котором существует функция $\pi : \mathcal{S} \to \mathcal{A}$ такая, что $\pi(a \mid s) = \mathbb{1}\{a = \pi(s)\}$.

Замечание 3.2. Мы используем один и тот же символ π как для стохастического, так и для детерминированного случаев; контекст всегда прояснит, что имеется в виду. Все детерминированные политики являются стохастическими политиками (с точечными распределениями), но не наоборот.

Пример 3.4 (Политики в решётке 2×2)**.** Рассмотрим четырёхклеточную решётку с клетками $\{(0,0),(0,1),(1,0),(1,1)\}$ и действиями {Вверх, Вниз, Влево, Вправо}. Детерминированная политика может отображать $(0,0) \mapsto$ Вправо, $(0,1) \mapsto$ Вверх, $(1,0) \mapsto$ Вправо, $(1,1) \mapsto$ Вверх. Стохастическая политика может назначать равную вероятность 0.25 каждому действию в каждом состоянии — равномерное случайное блуждание.

При фиксированной политике π MDP сводится к марковскому процессу с вознаграждением: переходы между состояниями управляются

$$p_\pi(s' \mid s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)\, p(s' \mid s, a),$$

а ожидаемое вознаграждение из состояния s равно $r_\pi(s) = \sum_a \pi(a \mid s)\, r(s,a)$.

3.5 Функции ценности

Функции ценности измеряют, насколько хорошо находиться в данном состоянии или выполнять данное действие в данном состоянии при следовании политике π.

Определение 3.5 (Функции ценности состояния и действия)**.** Пусть π — политика. **Функция ценности состояния** $V_\pi : \mathcal{S} \to \mathbb{R}$ равна

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \tag{3.11}$$

а **функция ценности действия** $Q_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ равна

$$Q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s,\ A_t = a]. \tag{3.12}$$

Здесь $\mathbb{E}_\pi[\,\cdot\,]$ обозначает математическое ожидание по распределению над траекториями, порождаемому π и p.

Связь между V_π и Q_π

Утверждение 3.4. *Для любой политики π и состояния $s \in \mathcal{S}$,*

$$V_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)\, Q_\pi(s,a). \tag{3.13}$$

Доказательство. Обусловливая на действие, выбранное в момент t, и применяя закон полного математического ожидания:

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \sum_{a \in \mathcal{A}(s)} \mathbb{P}_\pi(A_t = a \mid S_t = s)\ \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\ &= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)\, Q_\pi(s,a). \end{aligned}$$

□

Интуитивно $V_\pi(s)$ — взвешенное по политике среднее ценностей действий: ожидаемый доход агента — это среднее того, что дало бы каждое действие, взвешенное по вероятности, которую политика назначает каждому действию.

Числовой пример: MDP с тремя состояниями

Пример 3.5 (Вычисление функции ценности)**.** Рассмотрим MDP на рис. 3.1 с тремя состояниями $\{s_1, s_2, s_3\}$, двумя действиями $\{a_1, a_2\}$, доступными только в s_1 (состояния s_2 и s_3 — поглощающие с нулевым вознаграждением), и коэффициентом дисконтирования $\gamma = 0.5$.

Поскольку s_2 и s_3 поглощающие с нулевым будущим вознаграждением, $V_\pi(s_2) = V_\pi(s_3) = 0$ для любой политики.

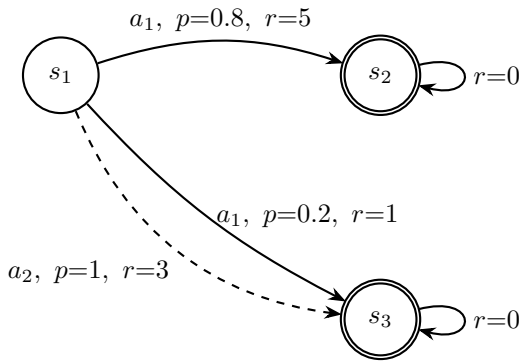


Рис. 3.1: MDP с тремя состояниями. Действие a_1 (сплошная линия) ведёт в s_2 с вероятностью 0.8 и вознаграждением 5, или в s_3 с вероятностью 0.2 и вознаграждением 1. Действие a_2 (пунктир) детерминированно ведёт в s_3 с вознаграждением 3. Состояния s_2 и s_3 — поглощающие.

Для s_1 ценности действий вычисляются непосредственно из определения (3.12):

$$\begin{aligned} Q_\pi(s_1, a_1) &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s_1, A_t = a_1] \\ &= 0.8 \cdot (5 + 0.5 \cdot 0) + 0.2 \cdot (1 + 0.5 \cdot 0) = 4.0 + 0.2 = 4.2, \\ Q_\pi(s_1, a_2) &= 1.0 \cdot (3 + 0.5 \cdot 0) = 3.0. \end{aligned}$$

Предположим, политика $\pi(a_1 \mid s_1) = 0.6$, $\pi(a_2 \mid s_1) = 0.4$. Тогда по (3.13):

$$V_\pi(s_1) = 0.6 \times 4.2 + 0.4 \times 3.0 = 2.52 + 1.20 = 3.72.$$

При жадной политике ($\pi(a_1 \mid s_1) = 1$) получаем $V_\pi(s_1) = 4.2$, что больше.

3.6 Уравнение Беллмана

Уравнение Беллмана — центральное тождество теории RL. Оно выражает функцию ценности как условие самосогласованности: ценность состояния должна равняться ожидаемому немедленному вознаграждению плюс дисконтированная ценность состояния-преемника.

Вывод для V_π

Теорема 3.5 (Уравнение ожиданий Беллмана для V_π)**.** *Для любой политики π и любого состояния $s \in \mathcal{S}$,*

$$V_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \big[r + \gamma V_\pi(s')\big]. \tag{3.14}$$

Доказательство. Начинаем с определения (3.11) и применяем рекуррентный доход (3.8):

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s].$$

По линейности математического ожидания:

$$V_\pi(s) = \mathbb{E}_\pi[R_{t+1} \mid S_t = s] + \gamma\ \mathbb{E}_\pi[G_{t+1} \mid S_t = s].$$

Применяем свойство башни к второму члену, обусловливая на S_{t+1}:

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \mathbb{E}_\pi\big[\mathbb{E}_\pi[G_{t+1} \mid S_{t+1}] \mid S_t = s\big] = \mathbb{E}_\pi[V_\pi(S_{t+1}) \mid S_t = s].$$

Следовательно, $V_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s]$. Явно раскрывая математическое ожидание по действиям и переходам:

$$\begin{aligned} V_\pi(s) &= \sum_a \pi(a \mid s)\ \mathbb{E}\big[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s, A_t = a\big] \\ &= \sum_a \pi(a \mid s) \sum_{s'} \sum_r p(s', r \mid s, a) \big[r + \gamma V_\pi(s')\big]. \end{aligned}$$

□

Уравнение Беллмана для Q_π

Утверждение 3.6 (Уравнение Беллмана для Q_π)**.**

$$Q_\pi(s,a) = \sum_{s'\in\mathcal{S}} \sum_{r\in\mathcal{R}} p(s',r \mid s,a)\Big[r + \gamma \sum_{a'\in\mathcal{A}(s')} \pi(a' \mid s')\, Q_\pi(s',a')\Big]. \tag{3.15}$$

Доказательство. Начиная с $Q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$ и применяя свойство башни, обусловленное на (S_{t+1}, R_{t+1}), затем используя $\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s'] = V_\pi(s') = \sum_{a'} \pi(a' \mid s') Q_\pi(s',a')$. □

Резервная диаграмма

Уравнение Беллмана имеет естественное графическое представление, известное как *резервная диаграмма*, показанная на рис. 3.2. Пустые кружки представляют состояния; закрашенные кружки — пары состояние-действие. Корень — состояние s; диаграмма разветвляется к действиям по π, затем далее к состояниям-преемникам по p.

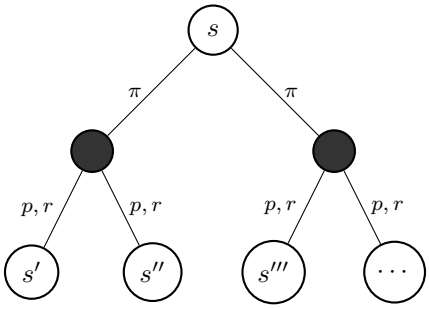


Рис. 3.2: Резервная диаграмма для уравнения Беллмана для $V_\pi(s)$. Пустые кружки — состояния; закрашенные кружки — пары состояние-действие. Рёбра с меткой π показывают выбор действия; рёбра с меткой p, r — переходы среды с соответствующим вознаграждением.

Матричная форма для конечных MDP

Когда $\mathcal{S}$ конечно и $n = |\mathcal{S}|$ состояний, уравнение (3.14) можно записать в матрично-векторной форме. Определим:

- $\mathbf{v}_\pi \in \mathbb{R}^n$ с $[\mathbf{v}_\pi]_s = V_\pi(s)$;
- $\mathbf{r}_\pi \in \mathbb{R}^n$ с $[\mathbf{r}_\pi]_s = r_\pi(s) = \sum_a \pi(a \mid s) r(s,a)$;
- $\mathbf{P}_\pi \in \mathbb{R}^{n\times n}$ с $[\mathbf{P}_\pi]_{s,s'} = \sum_a \pi(a \mid s) p(s' \mid s,a)$.

Тогда уравнение Беллмана принимает вид линейной системы

$$\mathbf{v}_\pi = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_\pi, \tag{3.16}$$

с единственным решением

$$\mathbf{v}_\pi = (I - \gamma \mathbf{P}_\pi)^{-1} \mathbf{r}_\pi. \tag{3.17}$$

Матрица $(I - \gamma\mathbf{P}_\pi)$ обратима, поскольку $\mathbf{P}_\pi$ — стохастическая матрица (спектральный радиус 1), поэтому все собственные значения $\gamma\mathbf{P}_\pi$ имеют модуль не более $\gamma < 1$, что делает все собственные значения $(I - \gamma\mathbf{P}_\pi)$ отделёнными от нуля.

Пример 3.6 (Решение системы Беллмана)**.** Вернёмся к примеру 3.5, но теперь предположим, что оба состояния s_2 и s_3 возвращаются обратно в s_1 с вознаграждением 0, и s_1 использует политику $\pi(a_1|s_1) = 0.6$, $\pi(a_2|s_1) = 0.4$, $\gamma = 0.5$.

Матрица переходов под π из s_1:

$$p_\pi(s_2 \mid s_1) = 0.6 \times 0.8 = 0.48, \quad p_\pi(s_3 \mid s_1) = 0.6 \times 0.2 + 0.4 \times 1 = 0.52.$$

Ожидаемое вознаграждение: $r_\pi(s_1) = 0.6 \times 4.2 + 0.4 \times 3.0 = 3.72$ (как и прежде). Для простоты полагаем $V(s_2) = V(s_3) = 0$ (поглощающие). Уравнение Беллмана для s_1:

$$V_\pi(s_1) = 3.72 + 0.5(0.48 \cdot 0 + 0.52 \cdot 0) = 3.72.$$

Это совпадает с прямым вычислением в примере 3.5.

3.7 Оптимальные функции ценности и уравнение оптимальности Беллмана

Оптимальные функции ценности

Определение 3.6 (Оптимальные функции ценности)**. Оптимальная функция ценности состояния**:

$$V_*(s) = \sup_\pi V_\pi(s), \qquad s \in \mathcal{S}, \tag{3.18}$$

а **оптимальная функция ценности действия**:

$$Q_*(s,a) = \sup_\pi Q_\pi(s,a), \qquad s \in \mathcal{S},\ a \in \mathcal{A}(s). \tag{3.19}$$

Политика π_* является **оптимальной**, если $V_{\pi_*}(s) = V_*(s)$ для всех $s \in \mathcal{S}$.

Упорядочение политик и существование оптимальной политики

Определение 3.7 (Упорядочение политик)**.** Пишем $\pi' \succeq \pi$ и говорим, что π' **не хуже** π, если

$$V_{\pi'}(s) \geq V_\pi(s) \qquad \forall s \in \mathcal{S}.$$

Теорема 3.7 (Существование оптимальной политики)**.** *Для любого конечного MDP существует по меньшей мере одна детерминированная оптимальная политика π_* такая, что $V_{\pi_*}(s) = V_*(s)$ для всех $s \in \mathcal{S}$.*

Набросок доказательства. Ключевое наблюдение: политика оптимальна тогда и только тогда, когда в каждом состоянии s она действует жадно относительно V_*:

$$\pi_*(a \mid s) > 0 \implies a \in \operatorname*{argmax}_{a' \in \mathcal{A}(s)} Q_*(s,a').$$

Такая детерминированная политика всегда существует (разрешая ничьи произвольно). Из уравнений оптимальности Беллмана, выведенных ниже, следует, что $V_{\pi_*} = V_*$. □

Оптимальные функции ценности удовлетворяют собственным уравнениям Беллмана, получаемым заменой взвешенной по политике суммы по действиям на максимум.

Теорема 3.8 (Уравнения оптимальности Беллмана)**.**

$$V_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a)\big[r + \gamma V_*(s')\big], \tag{3.20}$$

$$Q_*(s,a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a)\Big[r + \gamma \max_{a' \in \mathcal{A}(s')} Q_*(s', a')\Big]. \tag{3.21}$$

Доказательство. Для (3.20): поскольку $V_*(s) = \max_\pi V_\pi(s)$, уравнение Беллмана (3.14) для оптимальной политики π_* даёт

$$V_*(s) = \sum_a \pi_*(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V_*(s')].$$

Поскольку π_* жадна относительно V_*, взвешенная сумма по a равна максимуму по a, что даёт (3.20). Уравнение (3.21) следует аналогично из Q-версии уравнения Беллмана. □

Замечание 3.3. Уравнения оптимальности Беллмана *нелинейны* относительно V_* (из-за оператора max) и не могут быть решены простым обращением матрицы. Необходимо динамическое программирование и связанные алгоритмы; см. главу 4.

3.8 Оператор Беллмана

Уравнения Беллмана можно изучать как задачи нахождения неподвижных точек линейных операторов — перспектива, объясняющая, почему алгоритмы их решения сходятся.

Определение 3.8 (Операторы Беллмана)**.** Пусть $\mathcal{B}(\mathcal{S})$ обозначает банахово пространство ограниченных функций $V : \mathcal{S} \to \mathbb{R}$, оснащённое супремум-нормой $\|V\|_\infty = \max_{s\in\mathcal{S}} |V(s)|$.

Оператор Беллмана для политики $\mathcal{T}_\pi : \mathcal{B}(\mathcal{S}) \to \mathcal{B}(\mathcal{S})$:

$$(\mathcal{T}_\pi V)(s) = \sum_{a\in\mathcal{A}(s)} \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]. \tag{3.22}$$

Оператор оптимальности Беллмана $\mathcal{T}_* : \mathcal{B}(\mathcal{S}) \to \mathcal{B}(\mathcal{S})$:

$$(\mathcal{T}_* V)(s) = \max_{a\in\mathcal{A}(s)} \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]. \tag{3.23}$$

В этих обозначениях уравнения Беллмана читаются как $V_\pi = \mathcal{T}_\pi V_\pi$ и $V_* = \mathcal{T}_* V_*$: функции ценности являются *неподвижными точками* соответствующих операторов Беллмана.

Теорема 3.9 (γ-сжатие)**.** *Оба оператора $\mathcal{T}_\pi$ и $\mathcal{T}_*$ являются γ-сжатиями в супремум-норме: для любых $V, W \in \mathcal{B}(\mathcal{S})$,*

$$\|\mathcal{T}_\pi V - \mathcal{T}_\pi W\|_\infty \le \gamma \|V - W\|_\infty, \tag{3.24}$$

$$\|\mathcal{T}_* V - \mathcal{T}_* W\|_\infty \le \gamma \|V - W\|_\infty. \tag{3.25}$$

Доказательство. Доказываем (3.24); доказательство (3.25) аналогично. Фиксируем $s \in \mathcal{S}$:

$$\begin{aligned}
|(\mathcal{T}_\pi V)(s) - (\mathcal{T}_\pi W)(s)| &= \Big|\sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\, \gamma\big(V(s') - W(s')\big)\Big| \\
&\le \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\, \gamma\, |V(s') - W(s')| \\
&\le \gamma\, \|V - W\|_\infty \underbrace{\sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)}_{=1}.
\end{aligned}$$

Беря максимум по s, получаем (3.24).

Для (3.25) используем неравенство $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$ и действуем аналогично. □

Следствие 3.10 (Теорема Банаха о неподвижной точке)**.** *Поскольку $\mathcal{B}(\mathcal{S})$ — полное метрическое пространство (конечномерное) и операторы $\mathcal{T}_\pi$, $\mathcal{T}_*$ являются сжатиями ($\gamma < 1$), теорема Банаха о неподвижной точке гарантирует:*

1. *Каждый оператор имеет* единственную *неподвижную точку: V_π и V_* соответственно.*
2. *Итерирование оператора, начиная с произвольного $V^{(0)} \in \mathcal{B}(\mathcal{S})$:*

$$V^{(k+1)} = \mathcal{T} V^{(k)}$$

сходится к неподвижной точке с геометрической скоростью:

$$\|V^{(k)} - V_\pi\|_\infty \le \gamma^k \|V^{(0)} - V_\pi\|_\infty \to 0.$$

Это следствие является теоретической основой *оценки политики* в динамическом программировании (глава 4) и обучения TD(0) (глава 6): оба можно понять как итеративное применение (приближения) оператора Беллмана.

3.9 Числовой пример: MDP студента

Теперь разберём полный пример, вдохновлённый MDP студента из Саттона и Барто (2018).

Пример 3.7 (MDP студента)**.** Студент может находиться в одном из четырёх состояний:

- s_1: **Занятия** — посещение лекции.
- s_2: **Рутуб** — просмотр социальных сетей.
- s_3: **Учёба** — подготовка к экзамену.
- s_4: **Сон** — сон (терминальное/поглощающее состояние).

Действия и переходная динамика приведены в таблице 3.1, а MDP изображён на рис. 3.3. Используем коэффициент дисконтирования $\gamma = 0.9$.

Таблица 3.1: Переходная динамика MDP студента. Каждая строка задаёт действие в текущем состоянии, следующее состояние, вероятность и полученное вознаграждение.

Состояние	Действие	Следующее состояние	Вер. p	Вознаграждение r
s_1 (Занятия)	посещать	s_3 (Учёба)	1.0	-2
s_1 (Занятия)	отвлечься	s_2 (Рутуб)	1.0	-1
s_2 (Рутуб)	выйти	s_1 (Занятия)	1.0	-1
s_2 (Рутуб)	продолжить	s_2 (Рутуб)	1.0	-1
s_3 (Учёба)	сдать	s_4 (Сон)	1.0	$+10$
s_3 (Учёба)	учиться ещё	s_3 (Учёба)	1.0	-2

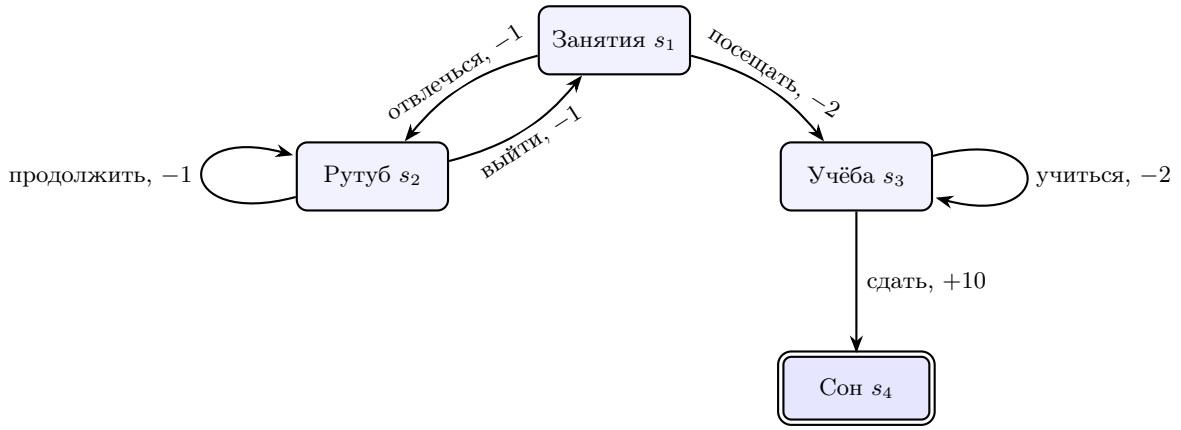


Рис. 3.3: MDP студента. Рёбра помечены действием и немедленным вознаграждением. Терминальное состояние Сон (s_4, двойная граница) является поглощающим.

Вычисление функций ценности при оптимальной политике. Оптимальная стратегия очевидно — посещать занятия и затем сдать экзамен: посещать s_1, достичь s_3, немедленно сдать в s_4.

При этой политике π_*:

$$\begin{aligned}
V_*(s_4) &= 0 \quad \text{(поглощающее)},\\
V_*(s_3) &= r(\text{сдать}) + \gamma V_*(s_4) = 10 + 0.9 \times 0 = 10.0,\\
V_*(s_1) &= r(\text{посещать}) + \gamma V_*(s_3) = -2 + 0.9 \times 10 = -2 + 9 = 7.0.
\end{aligned}$$

Таким образом, оптимальная ценность состояния «Занятия» равна 7.0.

Вычисление ценностей действий. В s_1:

$$\begin{aligned}
Q_*(s_1, \text{посещать}) &= -2 + 0.9 \times 10 = 7.0,\\
Q_*(s_1, \text{отвлечься}) &= -1 + 0.9 \times V_*(s_2).
\end{aligned}$$

Для $V_*(s_2)$ заметим, что из Рутуб оптимальное действие — *выйти* (возвращаясь к Занятиям, имеющим ценность 7.0), поэтому:

$$V_*(s_2) = -1 + 0.9 \times 7.0 = -1 + 6.3 = 5.3.$$

Следовательно:

$$Q_*(s_1, \text{отвлечься}) = -1 + 0.9 \times 5.3 = -1 + 4.77 = 3.77.$$

Поскольку $Q_*(s_1, \text{посещать}) = 7.0 > 3.77 = Q_*(s_1, \text{отвлечься})$, жадная политика действительно выбирает «посещать», подтверждая наши прежние рассуждения.

Проверка уравнения оптимальности Беллмана в s_2. Уравнение оптимальности Беллмана (3.20) гласит:

$$V_*(s_2) = \max\big\{Q_*(s_2, \text{выйти}),\, Q_*(s_2, \text{продолжить})\big\}.$$

Имеем:

$$\begin{aligned}
Q_*(s_2, \text{выйти}) &= -1 + 0.9 \times 7.0 = 5.3,\\
Q_*(s_2, \text{продолжить}) &= -1 + 0.9 \times V_*(s_2) = -1 + 0.9 \times 5.3 = 3.77.
\end{aligned}$$

Действительно $\max\{5.3, 3.77\} = 5.3 = V_*(s_2)$. ✓

3.10 Резюме

- **Марковское свойство** утверждает, что будущее независимо от прошлого при условии знания текущего состояния. Оно является основой, делающей RL разрешимым.
- **Марковский процесс принятия решений** $(\mathcal{S}, \mathcal{A}, p, \gamma)$ — стандартная рамка для последовательного принятия решений. Ядро $p(s', r \mid s, a)$ кодирует всю динамику.
- **Доход** $G_t = \sum_k \gamma^k R_{t+k+1}$ фиксирует дисконтированное суммарное будущее вознаграждение. Его рекуррентное свойство $G_t = R_{t+1} + \gamma G_{t+1}$ является зерном уравнений Беллмана.
- **Функции ценности** $V_\pi(s)$ и $Q_\pi(s, a)$ измеряют ожидаемый доход под политикой π. Они связаны соотношением $V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s, a)$.
- **Уравнения ожиданий Беллмана** выражают функции ценности как самосогласованные неподвижные точки. Для малых MDP они могут быть решены точно методами линейной алгебры.
- **Уравнения оптимальности Беллмана** характеризуют V_* и Q_* через оператор max. Их решения непосредственно дают оптимальную политику.
- **Операторы Беллмана** $\mathcal{T}_\pi$ и $\mathcal{T}_*$ являются γ-сжатиями в супремум-норме. По теореме Банаха о неподвижной точке итерирование любого из операторов из произвольной начальной точки геометрически сходится к единственной неподвижной точке. Это теоретический фундамент динамического программирования и обучения TD.

Упражнения

Упражнение 3.1. Приведите пример задачи, в которой необработанное сенсорное наблюдение (например, один пиксель) *не* является марковским состоянием, а дополненное состояние (например, последние k кадров) является. Объясните, почему дополненное состояние удовлетворяет марковскому свойству.

Упражнение 3.2. Рассмотрим непрерывную задачу, в которой агент всегда получает вознаграждение $c > 0$ на каждом шаге и $\gamma \in [0, 1)$.

(a) Вычислите G_t в замкнутой форме.

(b) Покажите, что $G_t = c/(1 - \gamma)$.

(c) Объясните, почему условие $\gamma < 1$ необходимо для конечности дохода.

(d) Что происходит при $\gamma = 1$?

Упражнение 3.3. Используя определения из предложения 3.1, проверьте, что

$$r(s, a) = \sum_{s' \in \mathcal{S}} p(s' \mid s, a)\, r(s, a, s').$$

Это показывает, что трёхаргументное ожидаемое вознаграждение $r(s, a, s')$ более информативно, чем $r(s, a)$.

Упражнение 3.4. В MDP студента (пример 3.7), предположим, студент следует политике, при которой в состоянии s_3 он учится ещё с вероятностью 0.5 и сдаёт с вероятностью 0.5. Используйте $\gamma = 0.9$.

(a) Запишите уравнение Беллмана для $V_\pi(s_3)$ под этой смешанной политикой.

(b) Решите относительно $V_\pi(s_3)$.

(c) Сравните с $V_*(s_3) = 10$ и объясните разницу.

Упражнение 3.5. Докажите уравнение Беллмана для Q_π (предложение 3.6) подробно, начиная с определения $Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$ и рекуррентного дохода (3.8).

Упражнение 3.6. Рассмотрим MDP с двумя состояниями $\mathcal{S} = \{s_1, s_2\}$, одним действием, матрицей переходов

$$P_\pi = \begin{pmatrix} 0.3 & 0.7 \\ 0.6 & 0.4 \end{pmatrix},$$

вектором ожидаемых вознаграждений $\mathbf{r}_\pi = (2, 5)^\top$ и $\gamma = 0.8$.

(a) Запишите матричное уравнение Беллмана (3.16).

(b) Явно вычислите $(I - \gamma P_\pi)$.

(c) Решите относительно $\mathbf{v}_\pi$ по формуле (3.17).

Упражнение 3.7. Завершите доказательство теоремы 3.9 для оператора оптимальности $\mathcal{T}_*$. *Подсказка*: используйте тождество $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$.

Упражнение 3.8. Начиная с $V^{(0)} \equiv 0$ и итерируя $\mathcal{T}_*$ с $\gamma = 0.9$ и $R_{\max} = 1$:

(a) Дайте верхнюю оценку $\|V^{(k)} - V_*\|_\infty$ через k и γ.

(b) Сколько итераций необходимо, чтобы гарантировать $\|V^{(k)} - V_*\|_\infty \leq 0.01$?

Упражнение 3.9. Приведите пример конечного MDP, имеющего бесконечно много оптимальных политик, но единственную оптимальную функцию ценности V_*. Убедитесь, что все оптимальные политики достигают V_*.

Упражнение 3.10. Рассмотрим решётку 1×3 (клетки 1, 2, 3). Агент может двигаться влево или вправо. Клетка 3 — цель (вознаграждение $+1$, терминальная). Агент *не* наблюдает текущую клетку; он получает лишь сигнал о том, столкнулся ли он со стеной.

(a) Объясните, почему наблюдение не является марковским состоянием.

(b) Определите дополненное состояние (используя историю наблюдений), восстанавливающее марковское свойство.

(c) Опишите оптимальную политику в терминах этого дополненного состояния.

Глава 4

Динамическое программирование

Дайте мне точку опоры, и я переверну Землю. — Архимед

Динамическое программирование (DP) занимает уникальное место в обучении с подкреплением: оно решает центральную задачу последовательного принятия решений *точно*, но лишь тогда, когда агент располагает идеальной математической моделью среды. В этой главе мы рассматриваем DP как собрание практических алгоритмов и как теоретическую основу, на которой зиждется вся остальная книга. Каждый алгоритм, с которым мы встретимся в последующих главах — методы Монте-Карло, временное разностное обучение, градиент политики — можно понять как приближение к DP, заменяющее точные модельные вычисления выборкой или аппроксимацией функций.

Формально мы предполагаем на протяжении всей этой главы, что среда является конечным марковским процессом принятия решений $(\mathcal{S}, \mathcal{A}, p, \gamma)$, как определено в главе 3. Ядро переходов $p(s', r \mid s, a)$ известно полностью, пространства состояний и действий конечны, а коэффициент дисконтирования удовлетворяет $\gamma \in [0, 1)$. В этих условиях DP находит оптимальную функцию ценности и оптимальную политику за время, полиномиальное относительно $|\mathcal{S}|$ и $|\mathcal{A}|$.

Фундаментальное понимание, лежащее в основе всех алгоритмов DP, — *уравнение Беллмана*: оптимальные функции ценности удовлетворяют рекуррентному условию самосогласованности, и DP эксплуатирует эту структуру, итерируя тщательно определённый оператор до сходимости. Мы представляем четыре основных алгоритма — итеративную оценку политики, улучшение политики, итерацию политики и итерацию ценности — и объединяем их под знаменем *обобщённой итерации политики* (GPI).

4.1 Оценка политики (предсказание)

Первая и наиболее базовая задача — *оценка политики*: дана фиксированная политика π, вычислить её функцию ценности состояния v^π. Это задача предсказания — мы ещё не пытаемся найти *хорошую* политику, лишь понять, насколько хорошо данная.

Оператор Беллмана для фиксированной политики

Из главы 3 напомним, что v^π является единственным решением *уравнений ожиданий Беллмана*

$$v^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r} p(s', r \mid s, a) \big[r + \gamma\, v^\pi(s') \big], \qquad s \in \mathcal{S}. \tag{4.1}$$

Это линейная система из $|\mathcal{S}|$ уравнений с $|\mathcal{S}|$ неизвестными. Теоретически её можно было бы решить напрямую методом Гаусса, но для больших пространств состояний, типичных в RL, это вычислительно нецелесообразно. Вместо этого DP решает (4.1) *методом последовательных приближений*: определим **оператор Беллмана для политики**

$$(T^\pi v)(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r} p(s', r \mid s, a) \big[r + \gamma\, v(s') \big], \tag{4.2}$$

и итерируем $v_{k+1} = T^\pi v_k$ начиная с произвольной ограниченной инициализации v_0.

Теорема 4.1 (Сжатие T^π)**.** *Оператор T^π является сжатием на пространстве ограниченных функций $\mathcal{S} \to \mathbb{R}$, оснащённом супремум-нормой $\|v\|_\infty = \max_{s \in \mathcal{S}} |v(s)|$. Точнее, для любых двух ограниченных функций $u, v \colon \mathcal{S} \to \mathbb{R}$,*

$$\|T^\pi u - T^\pi v\|_\infty \le \gamma \|u - v\|_\infty.$$

Следовательно, T^π имеет единственную неподвижную точку v^π, и итераты $v_{k+1} = T^\pi v_k$ сходятся к v^π из любой начальной точки v_0.

Доказательство. Зафиксируем произвольное состояние $s \in \mathcal{S}$. Тогда

$$\begin{aligned}\big|(T^\pi u)(s) - (T^\pi v)(s)\big| &= \Big|\sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\, \gamma \big[u(s') - v(s')\big]\Big| \\ &\le \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\, \gamma\, \|u - v\|_\infty \\ &= \gamma \|u - v\|_\infty,\end{aligned}$$

где неравенство использует неравенство треугольника и тот факт, что веса $\pi(a \mid s)\, p(s', r \mid s, a)$ неотрицательны и суммируются к единице. Взяв супремум по s, получаем $\|T^\pi u - T^\pi v\|_\infty \le \gamma \|u - v\|_\infty$. Поскольку $\gamma < 1$, оператор является строгим сжатием. По теореме Банаха о неподвижной точке он имеет единственную неподвижную точку, а итераты сходятся с геометрической скоростью с коэффициентом γ. □

Замечание 4.1. После k итераций ошибка удовлетворяет $\|v_k - v^\pi\|_\infty \le \gamma^k \|v_0 - v^\pi\|_\infty$. Для $\gamma = 0.9$ и желаемой точности $\varepsilon = 10^{-3}$ требуется примерно $k \ge \log(10^{-3}/\|v_0 - v^\pi\|_\infty)/\log(0.9)$ итераций — независимо от $|\mathcal{S}|$.

Алгоритм и критерий остановки

На практике обходят все состояния и применяют обновление одновременно (синхронное DP) или последовательно (см. раздел 4.6). Алгоритм останавливается, когда максимальное изменение по всем состояниям падает ниже порога θ.

Алгоритм 3: Итеративная оценка политики

Input: Политика π, динамика MDP p, дисконтирование γ, порог $\theta > 0$
Инициализировать $V(s) \leftarrow 0$ для всех $s \in \mathcal{S}$; установить $V(\text{терминал}) \leftarrow 0$;
repeat
  $\Delta \leftarrow 0$;
  for *каждого* $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$;
    $V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$;
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$;
until $\Delta < \theta$;
Output: $V \approx v^\pi$

Критерий остановки. Порог θ управляет компромиссом между точностью и вычислительными затратами. Распространённый практический выбор — $\theta = 10^{-4}$ или $\theta = 10^{-6}$. *Апостериорная* оценка ошибки равна

$$\|V - v^\pi\|_\infty \le \frac{\theta\, \gamma}{1 - \gamma},$$

что следует из аргумента геометрического ряда, применённого к сжатию.

Числовой пример: решётка

Пример 4.1 (4×4 решётка)**.** Рассмотрим стандартную 4×4 решётку из 16 состояний, расположенных в сетке. Состояния s_1 (верхний левый) и s_{16} (нижний правый) — терминальные. Из каждого нетерминального состояния агент может двигаться ВВЕРХ, ВНИЗ, ВЛЕВО или ВПРАВО; попытка выйти за пределы решётки оставляет состояние неизменным. Каждый переход даёт вознаграждение -1 (агент хочет достичь терминального состояния как можно быстрее). Оцениваем *равновероятную случайную политику* $\pi(a \mid s) = 0.25$ для всех a и $\gamma = 1$ (недисконтированная эпизодическая задача).

В следующей таблице показаны значения V после нескольких проходов.

Итерация	Значения V (строка за строкой, терминальные состояния показаны как 0)			
$k = 0$	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
$k = 1$	0	-1	-1	-1
	-1	-1	-1	-1
	-1	-1	-1	-1
	-1	-1	-1	0
$k = 2$	0	-1.75	-2	-2
	-1.75	-2	-2	-2
	-2	-2	-2	-1.75
	-2	-2	-1.75	0
$k \to \infty$	0	-14	-20	-22
	-14	-18	-20	-20
	-20	-20	-18	-14
	-22	-20	-14	0

Сошедшиеся значения отражают ожидаемое число шагов до достижения терминального состояния под случайной политикой. Состояние, диагонально противоположное обоим терминалам (-22), труднее всего покинуть; состояния, соседние с терминалами, имеют $v^\pi \approx -14$. Этот пример иллюстрирует, как оценка политики распространяет информацию *обратно* от терминальных состояний через решётку.

Сошедшаяся v^π (случайная политика, $\gamma = 1$)

0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0

Рис. 4.1: Функция ценности состояния v^π для равновероятной случайной политики на 4×4 решётке после сходимости итеративной оценки политики. Закрашенные клетки — терминальные состояния со значением 0. Значения представляют ожидаемое суммарное (недисконтированное) вознаграждение, то есть отрицательное ожидаемое число шагов до достижения терминала.

4.2 Улучшение политики

Вычислив v^π для некоторой политики π, естественно спросить: можно ли построить строго лучшую политику? *Теорема об улучшении политики* отвечает на это утвердительно.

Функция ценности действия

Дано v^π, *функция ценности действия* $q^\pi(s, a)$ фиксирует ценность выполнения действия a в состоянии s с последующим следованием π:

$$q^\pi(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r} p(s', r \mid s, a)\big[r + \gamma\, v^\pi(s')\big]. \tag{4.3}$$

В модельной постановке q^π можно вычислить непосредственно из v^π с помощью одношагового просмотра вперёд в (4.3). Ключевое наблюдение: $q^\pi(s, a) \geq v^\pi(s)$ означает, что выполнение действия a *один раз* в состоянии s с последующим возвратом к π не хуже, чем следование π с самого начала.

Теорема 4.2 (Теорема об улучшении политики)**.** *Пусть π и π' — две детерминированные политики такие, что для всех $s \in \mathcal{S}$,*

$$q^{\pi}\big(s, \pi'(s)\big) \geq v^{\pi}(s). \tag{4.4}$$

Тогда π' не хуже π:

$$v^{\pi'}(s) \geq v^{\pi}(s), \qquad \forall s \in \mathcal{S}.$$

Более того, если неравенство в (4.4) строгое хотя бы для одного состояния, то $v^{\pi'}(s) > v^{\pi}(s)$ хотя бы для одного состояния.

Доказательство. Из (4.4) и определения q^{π}:

$$v^{\pi}(s) \leq q^{\pi}\big(s, \pi'(s)\big) = \sum_{s',r} p(s', r \mid s, \pi'(s))\big[r + \gamma\, v^{\pi}(s')\big] = (T^{\pi'} v^{\pi})(s).$$

Следовательно, $v^{\pi} \leq T^{\pi'} v^{\pi}$ поточечно. Оператор $T^{\pi'}$ *монотонен*: если $u \leq v$ поточечно, то $(T^{\pi'} u)(s) \leq (T^{\pi'} v)(s)$ для всех s, поскольку веса $p(s', r \mid s, \pi'(s))$ неотрицательны. Применяя $T^{\pi'}$ многократно и используя монотонность,

$$v^{\pi} \leq T^{\pi'} v^{\pi} \leq (T^{\pi'})^2 v^{\pi} \leq \cdots \leq (T^{\pi'})^n v^{\pi} \qquad \forall n \geq 1.$$

По теореме 4.1 (с π' вместо π), $(T^{\pi'})^n v^{\pi} \to v^{\pi'}$ при $n \to \infty$. Поскольку неравенство сохраняется при каждом конечном n, оно сохраняется в пределе:

$$v^{\pi}(s) \leq v^{\pi'}(s), \qquad \forall s \in \mathcal{S}.$$

□

Построение жадной политики

Жадная политика π' относительно v^{π} удовлетворяет условию улучшения (4.4) максимальным образом:

$$\pi'(s) \in \operatorname*{argmax}_{a \in \mathcal{A}} q^{\pi}(s, a) = \operatorname*{argmax}_{a \in \mathcal{A}} \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v^{\pi}(s')\big]. \tag{4.5}$$

По построению, $q^{\pi}(s, \pi'(s)) = \max_a q^{\pi}(s, a) \geq v^{\pi}(s)$ для всех s, поэтому теорема применима.

Когда улучшение прекращается. Предположим, жадное улучшение даёт $\pi' = \pi$ — то есть текущая политика уже жадна относительно своей собственной функции ценности. Тогда $v^{\pi}(s) = \max_a q^{\pi}(s, a)$ для всех s, что в точности является *уравнением оптимальности Беллмана*:

$$v^{\pi}(s) = \max_{a \in \mathcal{A}} \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v^{\pi}(s')\big] = v^{*}(s).$$

Следовательно, π является оптимальной политикой. Это наблюдение движет алгоритмом итерации политики.

4.3 Итерация политики

Итерация политики чередует два шага, разработанных выше: оцениваем текущую политику до сходимости, затем жадно улучшаем её.

Конечная сходимость

Теорема 4.3 (Конечная сходимость итерации политики)**.** *Итерация политики завершается за конечное число итераций.*

Доказательство. Каждая политика π — детерминированное отображение из $\mathcal{S}$ в $\mathcal{A}$. Таких отображений не более $|\mathcal{A}|^{|\mathcal{S}|}$ — конечное число. Теорема об улучшении политики гарантирует, что всякий раз, когда порождается $\pi' \neq \pi$, $v^{\pi'} \geq v^{\pi}$ с хотя бы одним строгим неравенством. Следовательно, последовательность политик, порождаемых итерацией политики, строго улучшается по ценности (пока не стабилизируется), что исключает двукратное посещение любой политики. Поскольку политик конечное число, алгоритм должен завершиться. □

Алгоритм 4: Итерация политики

Input: Динамика MDP p, дисконтирование γ, порог $\theta > 0$
Произвольно инициализировать $V(s) \in \mathbb{R}$ и $\pi(s) \in \mathcal{A}$ для всех $s \in \mathcal{S}$; $V(\text{терминал}) \leftarrow 0$;
repeat
  `// Оценка политики`
  repeat
    $\Delta \leftarrow 0$;
    for *каждого* $s \in \mathcal{S}$ **do**
      $v \leftarrow V(s)$;
      $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) \big[r + \gamma V(s')\big]$;
      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$;
  until $\Delta < \theta$;
  `// Улучшение политики`
  ПОЛИТИКА-СТАБИЛЬНА $\leftarrow$ `true`;
  for *каждого* $s \in \mathcal{S}$ **do**
    $a_{\text{старое}} \leftarrow \pi(s)$;
    $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma V(s')\big]$;
    if $a_{\text{старое}} \neq \pi(s)$ **then**
      ПОЛИТИКА-СТАБИЛЬНА $\leftarrow$ `false`
until *политика-стабильна* = `true`;
Output: $V \approx v^*$, $\pi \approx \pi^*$

Замечание 4.2. На практике итерация политики сходится за очень малое число итераций — нередко менее десяти — даже для больших MDP. Каждая итерация требует решения линейной системы (или итеративной оценки до сходимости), что доминирует в вычислениях.

Пример 4.2 (Итерация политики на решётке)**.** Возвращаясь к 4×4 решётке из примера 4.1, запускаем итерацию политики с равновероятной случайной политики.

- **Итерация 0**: Оцениваем случайную политику (как вычислено выше). Жадное улучшение немедленно даёт значительно лучшую политику: в каждом состоянии двигаться к ближайшему терминалу.
- **Итерация 1**: Улучшенная политика уже выглядит почти оптимально. Повторная оценка показывает, что она достигает $v^{\pi_1}(s)$ (манхэттенское расстояние до ближайшего терминального состояния) для всех s.
- **Итерация 2**: Жадная политика относительно v^{π_1} совпадает с π_1. Итерация политики останавливается.

Таким образом, итерация политики сходится всего за 2 итерации на этой небольшой задаче, несмотря на то что итеративная оценка на каждом шаге требует многих внутренних проходов.

4.4 Итерация ценности

Итерация политики может быть дорогостоящей, если оценка политики требует многих проходов до сходимости перед каждым шагом улучшения. *Итерация ценности* избегает этого, чередуя один шаг оценки с шагом улучшения, применяя по существу *оператор оптимальности Беллмана* напрямую.

Оператор оптимальности Беллмана

Определим T^* как

$$(T^* v)(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} \sum_{r} p(s', r \mid s, a) \big[r + \gamma v(s')\big]. \tag{4.6}$$

Оптимальная функция ценности v^* является единственной неподвижной точкой T^*: $T^* v^* = v^*$.

Теорема 4.4 (Сжатие T^*)**.** *T^* является γ-сжатием на $(B(\mathcal{S}), \|\cdot\|_\infty)$:*

$$\|T^*u - T^*v\|_\infty \le \gamma\,\|u - v\|_\infty.$$

Следовательно, итераты $v_{k+1} = T^*v_k$ сходятся к v^* из любой начальной точки v_0.

Доказательство. Для любого $s \in \mathcal{S}$,

$$\begin{aligned}(T^*u)(s) - (T^*v)(s) &= \max_a \sum_{s',r} p(s', r \mid s, a)\,\gamma\,u(s') - \max_a \sum_{s',r} p(s', r \mid s, a)\,\gamma\,v(s') \\ &\le \max_a \sum_{s',r} p(s', r \mid s, a)\,\gamma\big[u(s') - v(s')\big] \\ &\le \gamma\,\|u - v\|_\infty,\end{aligned}$$

используя $\max f - \max g \le \max(f - g)$ и неотрицательность весов переходов. Симметрично, $(T^*v)(s) - (T^*u)(s) \le \gamma\|u - v\|_\infty$. Взяв супремум-норму, получаем результат. □

Алгоритм

Алгоритм 5: Итерация ценности

Input: Динамика MDP p, дисконтирование γ, порог $\theta > 0$
Инициализировать $V(s) \leftarrow 0$ для всех $s \in \mathcal{S}$; $V(\text{терминал}) \leftarrow 0$;
repeat
  $\Delta \leftarrow 0$;
  for *каждого* $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$;
    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\,V(s')\big]$;
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$;
until $\Delta < \theta$;
Вывести детерминированную жадную политику π относительно V;
Output: $V \approx v^*$, $\pi \approx \pi^*$

Скорость сходимости. Поскольку T^* является γ-сжатием, ошибка удовлетворяет $\|v_k - v^*\|_\infty \le \gamma^k\|v_0 - v^*\|_\infty$. Для желаемой точности ε необходимое число итераций $k \ge \log(\varepsilon/\|v_0 - v^*\|_\infty)/\log\gamma$, которое растёт как $O\big((1-\gamma)^{-1}\log(1/\varepsilon)\big)$. Меньшее γ означает более быструю сходимость, но более сильное дисконтирование будущих вознаграждений.

Связь с итерацией политики

Итерацию ценности можно рассматривать как *итерацию политики с усечённой оценкой*: вместо выполнения оценки до сходимости выполняем ровно один проход T^π перед улучшением. Поскольку $T^{\pi_k} = T^*$, когда π_k жадна относительно v_k (по определению максимума в (4.6)), каждый проход итерации ценности одновременно является одним шагом и оценки, и улучшения.

Более общо, если оценка политики прерывается после m проходов (а не 1 или ∞), получаем семейство алгоритмов, интерполирующих между чистой итерацией ценности ($m = 1$) и итерацией политики ($m \to \infty$).

Пример 4.3 (Итерация ценности на решётке)**.** На той же 4×4 решётке применяем итерацию ценности, начиная с $V \equiv 0$.

После каждого прохода функция ценности быстро отражает структуру оптимального поведения. Примерно после 10–15 проходов с порогом $\theta = 10^{-4}$ алгоритм сходится, и жадная политика, извлечённая из V, совпадает с оптимальной политикой, найденной итерацией политики. Итерация ценности требует меньше суммарных проходов, чем итерация политики (которая нуждается во многих внутренних проходах на каждую итерацию), когда число состояний мало, но преимущество меняется для больших MDP, поскольку итерация политики сходится за очень мало итераций.

4.5 Обобщённая итерация политики

Оценка политики и улучшение политики — не жёстко последовательные шаги — их можно чередовать в любой пропорции. Термин *обобщённая итерация политики* (GPI) обозначает общую идею поддержания функции ценности V и политики π с многократным применением некоторой формы оценки (делающей V более согласованным с π) и некоторой формы улучшения (делающей π более жадной относительно V).

Два конкурирующих процесса. Два процесса тянут в противоположные стороны: оценка толкает V к v^π для *текущей* π, тогда как улучшение изменяет π на жадную относительно текущей V, что немедленно делает V неточной оценкой для *новой* π. Несмотря на это противоречие, два процесса взаимодействуют в следующем смысле: каждый делает прогресс к одной и той же цели. Взаимодействие можно визуализировать как две линии в пространстве значение-политика, притягиваемые друг к другу (см. рис. 4.2).

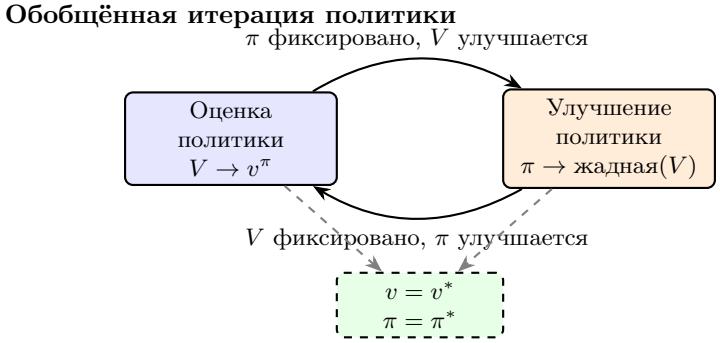


Рис. 4.2: Рамка GPI. Оценка политики и улучшение политики взаимодействуют многократно. Каждый шаг движется к устойчивой неподвижной точке, где одновременно $V = v^*$ и $\pi = \pi^*$. В итерации политики оценка выполняется до завершения перед каждым шагом улучшения. В итерации ценности один проход выполняет обе функции. Все модельно-свободные алгоритмы в последующих главах можно понять как приближённую GPI.

GPI как универсальная рамка. Каждый крупный алгоритм RL вписывается в шаблон GPI. В методе Монте-Карло (глава 5) оценка выполняется усреднением выборочных доходов, а улучшение — ε-жадным шагом. В временном разностном обучении (глава 6) оценка использует одношаговые оценки с бутстрэпингом. В методах актор-критик (глава 9) актор выполняет улучшение, а критик — оценку.

GPI на собственных и чужих данных. Важное различие, ставшее критически важным в модельно-свободном (model free) RL, — используются ли данные оценки из той же самой оцениваемой политики (*на собственных данных*) или из другой *поведенческой политики* (*на чужих данных*). В чистом DP это различие несущественно, поскольку модель p предоставляет точные математические ожидания; данные вообще не собираются. Однако, как только мы переходим к методам на основе выборки в главах 5–6, различие становится центральным.

4.6 Асинхронное динамическое программирование

Алгоритмы, описанные до сих пор, выполняют *синхронные* проходы: каждое состояние обновляется ровно один раз за итерацию. *Асинхронное* DP ослабляет это требование, обновляя состояния в произвольном порядке, любое число раз, не дожидаясь полного прохода.

Обновления на месте. В синхронной версии мы поддерживаем два массива V (текущий) и V' (следующий), записывая $V'(s) = T^\pi V(s)$, используя только значения из V. В варианте *на месте* мы поддерживаем один массив и немедленно перезаписываем $V(s)$ новым значением. Поскольку обновление $V(s)$ может использовать уже обновлённые значения $V(s')$ при $s' \neq s$, версия на месте не является синхронной, однако на практике обычно сходится быстрее: обновление в состоянии s' немедленно распространяется на любое состояние s, обновляемое позже в том же проходе.

Приоритетный обход. Не все состояния одинаково важно обновлять. *Приоритетный обход* поддерживает очередь с приоритетами и обновляет состояние s, у которого ошибка Беллмана $|v - T^\pi V(s)|$ наибольшая. При обновлении состояния приоритеты всех его предшественников (состояний, из которых достижимо s) пересчитываются. Это концентрирует вычисления на наиболее информативных обновлениях и может существенно сократить число обновлений, необходимых для сходимости.

DP в реальном времени. В *DP реального времени* (Barto, Bradtke и Singh, 1995) агент взаимодействует со средой в реальном времени и обновляет только фактически посещаемые состояния. Это концептуальный мост между модельным DP и модельно-свободным обучением TD: если модель доступна, посещаемые состояния могут обновляться с помощью точных резервных вычислений Беллмана; если нет, резервные вычисления заменяются выборочными переходами.

Замечание 4.3. Асинхронное DP сходится к v^* (или v^π) при условии, что каждое состояние продолжает обновляться бесконечно часто — то есть ни одно состояние не игнорируется постоянно. Это условие *достаточных обновлений* легко выполнить, многократно посещая состояния в произвольном порядке.

4.7 Эффективность и ограничения динамического программирования

Полиномиальная сложность. Алгоритмы DP полиномиальны относительно $|\mathcal{S}|$ и $|\mathcal{A}|$. Каждый проход оценки политики или итерации ценности требует $O(|\mathcal{S}|^2|\mathcal{A}|)$ операций (для каждого из $|\mathcal{S}|$ состояний обновление суммирует по $|\mathcal{A}|$ действиям и $|\mathcal{S}|$ следующим состояниям). Итерация политики выполняет не более $|\mathcal{A}|^{|\mathcal{S}|}$ итераций, но на практике сходится за очень мало (обычно $O(\log|\mathcal{S}|)$). Сравните это с полным перебором политик, требующим оценки всех $|\mathcal{A}|^{|\mathcal{S}|}$ политик.

Проклятие размерности. Несмотря на полиномиальную сложность в $|\mathcal{S}|$, DP нередко неприменимо, поскольку само $|\mathcal{S}|$ астрономически велико. Робот с n непрерывными углами суставов, дискретизированными по d делений каждый, имеет $|\mathcal{S}| = d^n$ состояний, что экспоненциально растёт с n. Это *проклятие размерности* Беллмана: пространство состояний взрывается с числом измерений. У скромного робота с 10 суставами и 100 делениями на сустав $|\mathcal{S}| = 10^{20}$ — безнадёжно неприемлемо.

Сравнение с линейным программированием. Оптимальную функцию ценности v^* можно также найти, решив линейную программу с $|\mathcal{S}|$ переменными и $|\mathcal{S}| \cdot |\mathcal{A}|$ ограничениями. Методы на основе ЛП могут в принципе использовать эффективные решатели ЛП, но они также плохо масштабируются с $|\mathcal{S}|$ и в целом не конкурентоспособны с DP для больших задач.

Почему выборка необходима. Ограничения DP мотивируют модельно-свободные методы, изучаемые в остальной части книги. В методах Монте-Карло (глава 5) v^π оценивается по выборочным траекториям без знания p. В методах временных разностей (глава 6) бутстрэпинг позволяет делать инкрементные обновления из отдельных переходов. Во всех случаях рамка GPI сохраняется, но точные резервные вычисления Беллмана заменяются приближёнными, выборочными.

Ключевое понимание таково: *DP говорит нам, что вычислять; выборка говорит, как вычислять это, когда p недоступно или пространство состояний слишком велико.*

4.8 Числовой пример: аренда автомобилей Джека

Чтобы проиллюстрировать итерацию политики на реалистичной задаче, опишем пример *аренды автомобилей Джека*, классику из Саттона и Барто (2018).

Постановка задачи

Джек управляет двумя пунктами аренды автомобилей, A и B. Каждый день клиенты приезжают в каждый пункт, чтобы взять машину. Аренда и возврат распределены примерно по закону Пуассона: в среднем 3 аренды и 3 возврата в день в пункте A и 4 аренды и 2 возврата в день в пункте B. Джек получает \$10 за каждую арендованную машину (если она доступна) и может перемещать машины за ночь между пунктами (до 5 машин в ночь по цене \$2 за машину). Каждый пункт вмещает не более 20 машин.

MDP определяется следующим образом. Состояние (n_A, n_B) — число машин в каждом пункте в конце дня (до ночных перемещений), от 0 до 20 на каждой площадке. Действие $a \in \{-5, \ldots, 5\}$ — чистое число

машин, перемещаемых из A в B (отрицательное означает перемещение из B в A). Вознаграждение — ожидаемый доход от аренды минус затраты на перемещение. Пространство состояний имеет $21^2 = 441$ состояние, а пространство действий — 11 действий, что делает задачу приемлемой для точного DP.

Эволюция политики

На рисунке 4.3 схематично показано, как политика эволюционирует при итерации политики.

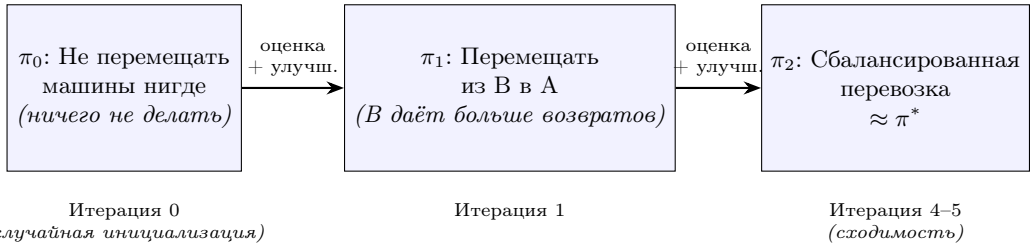


Рис. 4.3: Качественная эволюция политики при итерации политики для задачи аренды автомобилей. Ранние политики грубы; после 4–5 итераций политика стабилизируется до оптимальной стратегии перевозки, перемещающей машины из B в A (поскольку B имеет более высокое ожидаемое число возвратов, хранение машин в A захватывает больше дохода от аренды).

Наблюдения. Оптимальная политика перемещает машины из пункта с более высоким ожидаемым числом возвратов (B) в пункт с более высоким ожидаемым числом аренды (A), тщательно балансируя затраты на перевозку \$2 за машину против ожидаемого прироста дохода от аренды. Итерация политики сходится за 4–5 итераций, каждая из которых требует решения линейной системы 441×441. Это демонстрирует, как итерация политики эффективно обрабатывает структурированную динамику вознаграждений и переходов, достигая оптимума значительно быстрее, чем чистая итерация ценности или полный перебор.

4.9 Резюме

- Динамическое программирование предполагает известную модель $p(s', r \mid s, a)$ и точно решает задачу нахождения оптимальной функции ценности и политики.
- **Итеративная оценка политики** многократно применяет оператор Беллмана T^π; сходимость гарантирована теоремой о сжимающем отображении со скоростью γ.
- **Теорема об улучшении политики** гарантирует, что жадная политика относительно v^π не хуже π; если она совпадает с π, то π оптимальна.
- **Итерация политики** чередует оценку и улучшение и завершается за конечное число итераций, поскольку последовательность политик строго улучшается и пространство политик конечно.
- **Итерация ценности** применяет T^* напрямую, объединяя оценку и улучшение в одном проходе, и геометрически сходится к v^*.
- **Обобщённая итерация политики** — объединяющая рамка: все алгоритмы RL можно рассматривать как чередующиеся оценку и улучшение, отличающиеся лишь тем, насколько приближённым является каждый шаг.
- **Асинхронное DP** обновляет состояния в гибком порядке; приоритетный обход концентрирует вычисления там, где ошибки Беллмана наибольшие.
- DP полиномиально в $|\mathcal{S}|$ и $|\mathcal{A}|$, но непригодно для больших пространств состояний из-за проклятия размерности, мотивируя методы на основе выборки из глав 5 и 6.

Упражнения

Упражнение 4.1. Пусть $v_0 \equiv 0$ и $\|v^*\|_\infty \le R_{\max}/(1-\gamma)$.

(a) Покажите, что $\|v_k - v^*\|_\infty \le \gamma^k R_{\max}/(1-\gamma)$ для итератов $v_{k+1} = T^* v_k$.

(b) При $\gamma = 0.99$ и $R_{\max} = 1$ сколько итераций итерации ценности необходимо для достижения $\|v_k - v^*\|_\infty \leq 10^{-3}$?

Упражнение 4.2. Обобщите теорему об улучшении политики на *стохастические* политики π'. То есть предположим, что $\sum_a \pi'(a \mid s)\, q^\pi(s,a) \geq v^\pi(s)$ для всех s. Докажите, что $v^{\pi'} \geq v^\pi$ поточечно. *Подсказка*: доказательство по существу то же, что и для детерминированных политик; оператор $T^{\pi'}$ по-прежнему является сжатием.

Упражнение 4.3. После того как итерация ценности сошлась к $V \approx v^*$ с $\|V - v^*\|_\infty \leq \varepsilon$, какую оценку можно дать субоптимальности жадной политики π, извлечённой из V? То есть оцените $v^*(s) - v^\pi(s)$ через ε и γ. *Подсказка*: дважды используйте аргумент сжатия.

Упражнение 4.4. В 4×4 решётке с $\gamma = 1$ и случайной политикой ($\pi(a \mid s) = 0.25$):

(a) Запишите уравнения Беллмана для четырёх угловых состояний и четырёх состояний в серединах рёбер.

(b) Проверьте численно (или алгебраически), что сошедшиеся значения в примере 4.1 удовлетворяют этим уравнениям.

Упражнение 4.5. Конечный MDP имеет $|\mathcal{S}| = 5$ состояний и $|\mathcal{A}| = 3$ действия. Сколько детерминированных политик существует? Итерация политики посещает каждую политику не более одного раза. Предполагая, что каждая итерация требует $O(|\mathcal{S}|^2|\mathcal{A}|)$ работы, какова оценка суммарных вычислений в наихудшем случае? Сравните с работой при полном переборе политик.

Упражнение 4.6. Объясните, почему синхронный проход на месте (один массив, состояния обновляются слева направо в решётке) может сходиться быстрее, чем версия с двумя массивами. Приведите интуитивный пример в 4×4 решётке, где обновление распространяется по нескольким состояниям за один проход.

Упражнение 4.7 (Усечённая оценка политики)**.** Определите *m-шаговую итерацию политики* как итерацию политики, в которой фаза оценки выполняет ровно m проходов T^π перед каждым шагом улучшения. Покажите, что:

(a) $m = \infty$ восстанавливает стандартную итерацию политики.

(b) $m = 1$ восстанавливает итерацию ценности.

(c) Для всех конечных m последовательность функций ценности не убывает и сходится к v^*.

Упражнение 4.8. Покажите, что оптимальная функция ценности v^* является *наименьшим* суперрешением уравнения оптимальности Беллмана: то есть v^* является покоординатным минимумом всех функций v, удовлетворяющих $v(s) \geq (T^* v)(s)$ для всех s. Используйте это для формулировки нахождения v^* как линейной программы.

Упражнение 4.9 (Модификация аренды автомобилей Джека)**.** В задаче об аренде автомобилей из раздела 4.8 Джек нанимает второго работника. Первый работник может перемещать одну машину в ночь бесплатно; любые дополнительные машины по-прежнему стоят \$2 каждая. Кроме того, каждый пункт теперь вмещает до 25 машин, но взимается плата за парковку \$4 в ночь за каждый пункт, у которого более 10 машин. Опишите, как бы вы изменили функцию вознаграждения и определение пространства состояний. Какие изменения влияют на вероятности переходов? Какие влияют только на вознаграждение?

Упражнение 4.10. В модельном DP мы утверждали, что различие на собственных/чужих данных несущественно. Объясните точно, почему. Затем объясните, что идёт не так, если мы пытаемся применить обновления *на чужих данных* в модельно-свободной (основанной на выборке) версии оценки политики, и сформулируйте условие, необходимое для согласованности таких обновлений.

Глава 5

Методы Монте-Карло

Чтобы найти ценность позиции, симулируйте множество партий из этой позиции и усредните результаты.

Методы динамического программирования из главы 4 обладают большой мощью, однако требуют жёсткого предварительного условия: полного знания вероятностей переходов среды и функции вознаграждения. Во многих практически важных задачах — карточных играх, управлении роботами, диалоговых системах — такая модель попросту недоступна или слишком велика для полного перебора всех состояний. *Методы Монте-Карло* полностью устраняют это требование. Вместо того чтобы вычислять функции ценности из математической спецификации среды, они *обучаются* функциям ценности непосредственно по выборкам эпизодов взаимодействия.

Основная идея столь же стара, как и статистическое моделирование: если вы хотите узнать ожидаемый результат стохастического процесса, запустите его много раз и усредните результаты. Применительно к обучению с подкреплением это означает генерацию полных эпизодов в рамках фиксированной стратегии, вычисление отдачи (дисконтированной суммы вознаграждений), следующей за каждым посещением состояния или пары состояние–действие, и усреднение этих отдач. По мере роста числа эпизодов закон больших чисел гарантирует сходимость к истинной функции ценности.

Методы Монте-Карло занимают ключевое место в пейзаже обучения с подкреплением. Они знаменуют переход от обучения на основе модели к *безмодельному* обучению и вводят фундаментальную дихотомию между *обучением на текущей стратегии* и *обучением на другой стратегии*, которая будет встречаться на протяжении всей остальной части книги. Они требуют полных эпизодов, что делает их естественно пригодными для эпизодических задач; методы временной разности из главы 6 позднее устранят и это ограничение.

5.1 Предсказание методом Монте-Карло

Постановка задачи и обозначения

На протяжении всей этой главы мы работаем с конечным эпизодическим МПР (см. главу 3) с пространством состояний $\mathcal{S}$, пространством действий $\mathcal{A}$, коэффициентом дисконтирования $\gamma \in [0, 1)$ и фиксированной стратегией π. Один эпизод представляет собой траекторию

$$S_0, A_0, R_1, S_1, A_1, R_2, \ \ldots, \ S_{\tau-1}, A_{\tau-1}, R_\tau, S_\tau = \mathrm{T},$$

где T — терминальное состояние, а τ — (случайная) длина эпизода. *Отдача* начиная с шага t равна

$$G_t = \sum_{k=0}^{\tau-t-1} \gamma^k R_{t+1+k}. \tag{5.1}$$

Функция ценности состояния задаётся как $v^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$.

Мы предполагаем на протяжении всей главы, что: (i) эпизоды генерируются независимо в соответствии со стратегией π; (ii) каждый эпизод завершается почти наверное, $\mathbb{P}_\pi(\tau < \infty) = 1$; (iii) вознаграждения ограничены, $|R_t| \le R_{\max}$ п.н.; (iv) каждое интересующее нас состояние s посещается с положительной вероятностью за один эпизод.

Монте-Карло по первому посещению и по каждому посещению

В рамках одного эпизода состояние s может посещаться несколько раз. Два естественных способа усреднения соответствуют разным подходам к обработке повторных посещений.

Определение 5.1 (Оценщик МК по первому посещению)**.** Оценщик **Монте-Карло по первому посещению** для $v^\pi(s)$ усредняет по эпизодам отдачу G_t, следующую за *первым* шагом t, на котором $S_t = s$ в каждом эпизоде. Если $G_1^{\mathrm{FV}}(s), G_2^{\mathrm{FV}}(s), \dots$ — это отдачи, собранные из n эпизодов, то оценщик равен

$$\widehat{v}_n^{\mathrm{FV}}(s) = \frac{1}{n}\sum_{i=1}^{n} G_i^{\mathrm{FV}}(s). \tag{5.2}$$

Определение 5.2 (Оценщик МК по каждому посещению)**.** Оценщик **Монте-Карло по каждому посещению** усредняет отдачи, следующие за *каждым* посещением s по всем эпизодам. Для эпизода i пусть $K_i(s) = \sum_t \mathbb{1}\{S_t^{(i)} = s\}$ — число посещений, а $X_i(s) = \sum_t \mathbb{1}\{S_t^{(i)} = s\} G_t^{(i)}$ — суммарные отдачи. Оценщик после n эпизодов равен

$$\widehat{v}_n^{\mathrm{EV}}(s) = \frac{\sum_{i=1}^{n} X_i(s)}{\sum_{i=1}^{n} K_i(s)}. \tag{5.3}$$

Оба оценщика нацелены на одну и ту же величину $v^\pi(s)$. Их различие носит статистический, а не определяющий характер.

Несмещённость МК по первому посещению

Теорема 5.1 (МК по первому посещению несмещён и состоятелен)**.** *При сделанных выше допущениях каждая отдача $G_i^{\mathrm{FV}}(s)$ является независимой одинаково распределённой случайной величиной со средним $v^\pi(s)$ и конечной дисперсией. Следовательно:*

1. $\mathbb{E}\big[\widehat{v}_n^{\mathrm{FV}}(s)\big] = v^\pi(s)$ *для всех* $n \geq 1$ *(несмещённость).*
2. $\widehat{v}_n^{\mathrm{FV}}(s) \xrightarrow[n\to\infty]{\text{п.н.}} v^\pi(s)$ *(сильная состоятельность).*

Доказательство. **Независимость.** Каждая $G_i^{\mathrm{FV}}(s)$ вычисляется по отдельному независимо сгенерированному эпизоду. Следовательно, последовательность $G_1^{\mathrm{FV}}(s), G_2^{\mathrm{FV}}(s), \dots$ состоит из независимых случайных величин.

Одинаковое распределение. Рассмотрим произвольный эпизод и пусть T^* — первый шаг, на котором $S_{T^*} = s$. По свойству Маркова распределение траектории начиная с T^* зависит только от текущего состояния s и не зависит от истории $S_0, A_0, \dots, S_{T^*-1}$. Поскольку π фиксирована, отдача G_{T^*} с этой точки имеет в точности распределение $G_t \mid S_t = s$ при стратегии π. Таким образом, $G_i^{\mathrm{FV}}(s) \overset{d}{=} G_t \mid S_t = s$ для всех i.

Конечная дисперсия. Поскольку $|R_t| \leq R_{\max}$ и $\tau < \infty$ п.н.,

$$|G_t| \leq R_{\max} \sum_{k=0}^{\tau-1} \gamma^k \leq \frac{R_{\max}}{1-\gamma} < \infty \quad \text{п.н.}$$

Следовательно, $G_i^{\mathrm{FV}}(s)$ ограничена п.н. и имеет конечную дисперсию $\sigma_s^2 < \infty$.

Несмещённость. По линейности математического ожидания,

$$\mathbb{E}\big[\widehat{v}_n^{\mathrm{FV}}(s)\big] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}\big[G_i^{\mathrm{FV}}(s)\big] = v^\pi(s).$$

Сильная состоятельность. Поскольку $G_1^{\mathrm{FV}}(s), G_2^{\mathrm{FV}}(s), \dots$ — н.о.р. с конечным средним, усиленный закон больших чисел даёт

$$\widehat{v}_n^{\mathrm{FV}}(s) = \frac{1}{n}\sum_{i=1}^{n} G_i^{\mathrm{FV}}(s) \xrightarrow{\text{п.н.}} v^\pi(s). \qquad \square$$

Замечание 5.1. Дисперсия $\widehat{v}_n^{\mathrm{FV}}(s)$ равна σ_s^2/n и убывает со стандартной скоростью Монте-Карло $1/\sqrt{n}$. Это контрастирует, например, с бутстрэп-методами, которые могут достигать более высоких скоростей в гладких моделях. Однако здесь не требуется никаких предположений о распределении — лишь ограниченность вознаграждений и эпизодическое завершение.

Состоятельность МК по каждому посещению

Лемма 5.2. *Для любого состояния s и любого эпизода* $\mathbb{E}[X_1(s)] = v^\pi(s)\ \mathbb{E}[K_1(s)]$.

Доказательство. По линейности и свойству башни,

$$\begin{aligned}\mathbb{E}[X_1(s)] &= \sum_{t\geq 0} \mathbb{E}[\mathbb{1}\{S_t = s\}\, G_t] = \sum_{t\geq 0} \mathbb{E}[\mathbb{1}\{S_t = s\}\ \mathbb{E}[G_t \mid S_t]] \\ &= v^\pi(s) \sum_{t\geq 0} \mathbb{P}(S_t = s) = v^\pi(s)\ \mathbb{E}[K_1(s)].\end{aligned}$$

□

Теорема 5.3 (МК по каждому посещению состоятелен)**.** *Если* $\mathbb{E}[K_1(s)] \in (0,\infty)$ *и* $\mathbb{E}[|X_1(s)|] < \infty$, *то* $\widehat{v}_n^{\mathrm{EV}}(s) \xrightarrow{\text{п.н.}} v^\pi(s)$.

Доказательство. Пары $(X_i(s), K_i(s))$ являются н.о.р. по эпизодам. Усиленный закон больших чисел даёт

$$\frac{1}{n}\sum_{i=1}^{n} X_i(s) \xrightarrow{\text{п.н.}} \mathbb{E}[X_1(s)], \qquad \frac{1}{n}\sum_{i=1}^{n} K_i(s) \xrightarrow{\text{п.н.}} \mathbb{E}[K_1(s)] > 0.$$

Знаменатель в конечном счёте положителен п.н., поэтому по теореме о непрерывном отображении (правило частных):

$$\widehat{v}_n^{\mathrm{EV}}(s) \xrightarrow{\text{п.н.}} \frac{\mathbb{E}[X_1(s)]}{\mathbb{E}[K_1(s)]} = v^\pi(s),$$

где последнее равенство следует из леммы 5.2. □

Замечание 5.2. МК по каждому посещению *смещён* при любом конечном n, поскольку $\mathbb{E}[\sum X_i / \sum K_i] \neq \mathbb{E}[\sum X_i]/\mathbb{E}[\sum K_i]$ в общем случае (смещение оценщика-отношения). Однако смещение исчезает при $n \to \infty$, поэтому МК по каждому посещению *асимптотически* несмещён. На практике МК по каждому посещению может иметь меньшую среднеквадратичную ошибку, чем МК по первому посещению, когда состояния часто повторяются внутри эпизода, поскольку использует больше данных.

Инкрементная реализация

Вычислять средние заново после каждого эпизода расточительно. Вместо этого поддерживается текущий счётчик $N(s)$ и текущая оценка $V(s)$, которые обновляются после каждой наблюдаемой отдачи G:

$$N(s) \leftarrow N(s) + 1, \qquad V(s) \leftarrow V(s) + \frac{1}{N(s)}\big(G - V(s)\big). \tag{5.4}$$

Это алгебраически эквивалентно скользящему выборочному среднему и требует $O(|\mathcal{S}|)$ памяти.

Алгоритм 6: Предсказание методом МК по первому посещению

Input: Стратегия π, дисконт γ, число эпизодов N_{ep}
1 Инициализировать $V(s) \leftarrow 0$, $\mathit{Count}(s) \leftarrow 0$ для всех $s \in \mathcal{S}$;
2 **for** *эпизод* $= 1, 2, \ldots, N_{\text{ep}}$ **do**
3 Сгенерировать эпизод $S_0, A_0, R_1, \ldots, S_\tau$ по стратегии π;
4 $G \leftarrow 0$;
5 **for** $t = \tau - 1, \tau - 2, \ldots, 0$ (в обратном порядке) **do**
6 $G \leftarrow \gamma G + R_{t+1}$;
7 **if** S_t *не встречается в* $S_0, \ldots, S_{t-1}$ **then**
8 $\mathit{Count}(S_t) \leftarrow \mathit{Count}(S_t) + 1$;
9 $V(S_t) \leftarrow V(S_t) + \dfrac{1}{\mathit{Count}(S_t)}\big(G - V(S_t)\big)$;
10 **return** V;

Обратный проход по эпизоду (строки 4–9) вычисляет все отдачи G_t за время $O(\tau)$ с помощью рекуррентного соотношения $G_t = R_{t+1} + \gamma G_{t+1}$.

Резервная диаграмма МК

Следующая диаграмма сравнивает резервные диаграммы динамического программирования и метода Монте-Карло. В ДП одношаговое резервное обновление усредняет по всем возможным следующим состояниям с использованием модели. В МК следуют по одной выборочной траектории до конца, до терминального состояния.

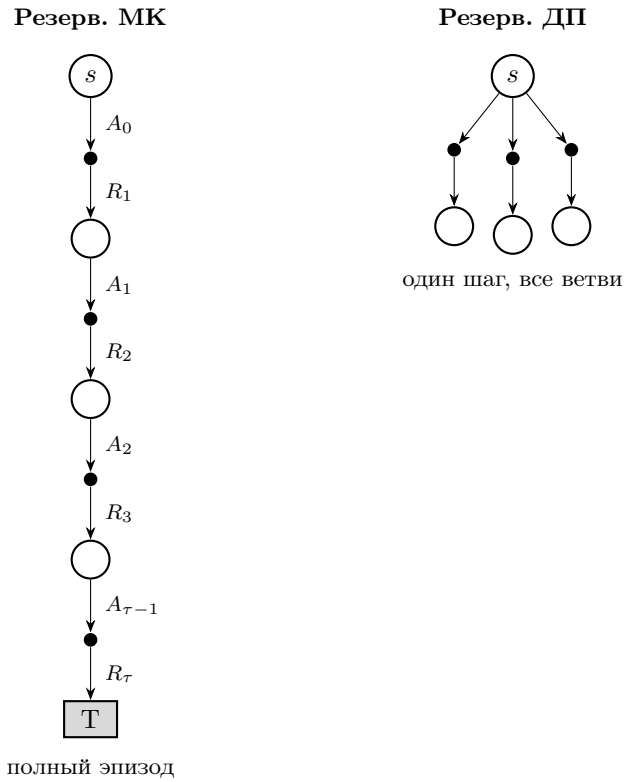


Рис. 5.1: Резервные диаграммы для метода Монте-Карло (слева) и динамического программирования (справа). МК следует по одной выборочной траектории до терминального состояния; ДП разветвляется по всем возможным одношаговым переходам с использованием модели.

5.2 Разобранный пример: Блэкджек

Блэкджек (также называемый *двадцать одно*) — канонический пример, используемый для иллюстрации предсказания методом Монте-Карло в книге Саттона и Барто (2018). Он особенно поучителен, поскольку полная модель среды (вероятности переходов) технически доступна, однако её трудно записать в явном виде, тогда как оценивание методом Монте-Карло легко реализовать путём симуляции карточных партий.

MDP блэкджека

Игра ведётся против дилера. Цель игрока — набрать карты с суммой как можно ближе к 21, не превышая этого значения (не «перебрать»). Картинки (валет, дама, король) считаются за 10; туз считается за 11, если при этом сумма не превысит 21, иначе туз считается за 1.

Состояние. Состояние — тройка $(x, d, a) \in \{12, \ldots, 21\} \times \{1, \ldots, 10\} \times \{0, 1\}$, где:

- x = текущая сумма игрока (рассматриваются только суммы ≥ 12; ниже 12 игрок всегда безопасно берёт карту);
- d = открытая карта дилера (туз считается за 1);
- $a = 1$, если у игрока есть «рабочий» туз (туз, в данный момент считающийся за 11), $a = 0$ иначе.

Это даёт $|\mathcal{S}| = 10 \times 10 \times 2 = 200$ нетерминальных состояний.

Действия. В каждом состоянии игрок выбирает *стоп* (прекратить брать карты, $A = 0$) или *ещё* (взять ещё карту, $A = 1$).

Вознаграждения. Эпизод завершается, когда игрок говорит «стоп» или перебирает. После «стопа» дилер играет до конца по фиксированному правилу (берёт карты, пока сумма < 17). Вознаграждение: $+1$ за победу игрока, -1 за проигрыш, 0 за ничью. Все промежуточные вознаграждения равны 0. Коэффициент дисконтирования $\gamma = 1$.

Стратегия. Оцениваем *простую стратегию* π: говорить «стоп», если $x \geq 20$, иначе брать ещё.

Генерация эпизодов и вычисление отдачи

Пример 5.1 (Пример эпизода блэкджека)**.** Рассмотрим следующий эпизод, сгенерированный в соответствии с простой стратегией:

t	Состояние	Действие	Сл. карта	R_{t+1}
0	$(15, 6, 0)$	Ещё	3	0
1	$(18, 6, 0)$	Ещё	2	0
2	$(20, 6, 0)$	Стоп	—	—
Дилер: $6 + 9 = 15$, берёт $4 \to 19$; игрок $20 > 19$.				$+1$

Эпизод завершается при $\tau = 3$ и $R_3 = +1$. Отдачи (при $\gamma = 1$) равны:

$$\begin{aligned} G_2 &= R_3 = +1, \\ G_1 &= R_2 + G_2 = 0 + 1 = +1, \\ G_0 &= R_1 + G_1 = 0 + 1 = +1. \end{aligned}$$

При МК по первому посещению этот эпизод вносит вклад:

- Отдача $G_0 = +1$ в оценку $v^\pi(15, 6, 0)$ (первое и единственное посещение).
- Отдача $G_1 = +1$ в $v^\pi(18, 6, 0)$.
- Отдача $G_2 = +1$ в $v^\pi(20, 6, 0)$.

Запуск 500,000 таких эпизодов даёт точное представление о функции ценности. Состояния с $x = 20$ или $x = 21$ имеют значения, близкие к $+0.6$ (игрок обычно выигрывает), тогда как состояния с $x \in \{12, \ldots, 14\}$ против туза дилера имеют значения около -0.3. В учебнике Саттона–Барто эти значения изображены в виде трёхмерных поверхностей, демонстрирующих интуитивно понятный ландшафт: высокие суммы против слабых карт дилера выгодны; низкие суммы против сильных карт дилера невыгодны.

Пример демонстрирует, почему метод Монте-Карло хорошо подходит для данной задачи: вычисление $p(s', r \mid s, a)$ в точности требует перечисления распределения карт, взятых из бесконечной колоды, тогда как симуляция раздачи тривиальна.

5.3 Оценка функций ценности действий методом Монте-Карло

Когда модель среды $p(s', r \mid s, a)$ доступна, функции ценности состояний v^π достаточно для управления: зная v^π, можно вычислить жадную стратегию путём одношагового просмотра вперёд,

$$\pi'(s) \in \operatorname*{argmax}_{a \in \mathcal{A}} \sum_{s', r} p(s', r \mid s, a) \big[r + \gamma v^\pi(s') \big].$$

Без модели такой просмотр невозможен. Мы не можем вычислить правую часть, поскольку $p(s', r \mid s, a)$ неизвестна. Поэтому в безмодельном управлении естественной целью является *функция ценности действия*

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s,\ A_t = a]. \tag{5.5}$$

Зная Q^π, жадная стратегия просто $\pi'(s) = \operatorname{argmax}_a Q^\pi(s, a)$, и модель не нужна вовсе.

Оценка Q^π методом Монте-Карло. Весь аппарат раздела 5.1 применяется дословно, с заменой состояний s на пары состояние–действие (s, a). Оценщик МК по первому посещению для $Q^\pi(s, a)$ собирает отдачи с первого посещения (s, a) в каждом эпизоде; оценщик по каждому посещению усредняет все такие отдачи. Теоремы 5.1 и 5.3 справедливы с идентичными доказательствами.

Допущение об исследующих стартах. Сразу возникает ключевая трудность: если стратегия π детерминирована, она никогда не выберет определённые действия в определённых состояниях, поэтому пары (s, a) для этих действий никогда не посещаются и $Q^\pi(s, a)$ никогда не оценивается. Без оценок для всех пар нельзя улучшить стратегию, выбирая жадное действие.

Допущение об *исследующих стартах* разрешает эту проблему, требуя, чтобы каждая пара состояние–действие имела положительную вероятность быть стартовой парой эпизода:

$$\mathbb{P}\big(S_0 = s,\ A_0 = a\big) > 0 \quad \forall\, s \in \mathcal{S},\ a \in \mathcal{A}.$$

При исследующих стартах каждая пара (s, a) будет в конечном счёте посещаться бесконечно часто (поскольку эпизоды генерируются многократно), обеспечивая состоятельную оценку Q^π для всех пар. Раздел 5.4 разрабатывает полный алгоритм управления на основе этого допущения; раздел 5.5 затем снимает его.

5.4 Управление методом Монте-Карло с исследующими стартами

Фреймворк обобщённой итерации по стратегии (GPI) из главы 4 чередует *оценивание стратегии* (оценку Q^π) и *улучшение стратегии* (выбор жадной стратегии по текущей Q). Управление методом Монте-Карло реализует этот фреймворк, используя выборочные эпизоды вместо обходов уравнений Беллмана.

Алгоритм 7: Управление методом МК с исследующими стартами (MC-ES)

Input: Дисконт γ
1 Инициализировать $Q(s, a)$ произвольно, $\pi(s) \in \mathcal{A}$ произвольно, $Count(s, a) \leftarrow 0$ для всех s, a;
2 **for** *каждый эпизод* **do**
3 Выбрать $S_0 \sim$ Равном.$(\mathcal{S})$, $A_0 \sim$ Равном.$(\mathcal{A})$ (исследующие старты);
4 Сгенерировать эпизод $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_\tau$ следуя π после шага 0;
5 $G \leftarrow 0$;
6 **for** $t = \tau - 1, \tau - 2, \ldots, 0$ (в обратном порядке) **do**
7 $G \leftarrow \gamma G + R_{t+1}$;
8 **if** (S_t, A_t) *не встречается в* $(S_0, A_0), \ldots, (S_{t-1}, A_{t-1})$ **then**
9 $Count(S_t, A_t) \leftarrow Count(S_t, A_t) + 1$;
10 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{Count(S_t, A_t)}\big(G - Q(S_t, A_t)\big)$;
11 $\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$;

Аргумент сходимости. Алгоритм выполняет один шаг улучшения стратегии (строка 11) после каждого обновления отдачи. Формально, сходимость к оптимальной стратегии π^* следует из аргумента о неподвижной точке: если алгоритм сходится, то в точке сходимости $Q \approx Q^\pi$ (оценивание состоятельно по теореме 5.1) и π жадна по отношению к Q^π. Стратегия, одновременно жадная по отношению к собственной функции ценности, должна быть оптимальной (см. теорему об улучшении стратегии, глава 4).

Почему исследующие старты непрактичны. Допущение о том, что каждый эпизод может начинаться с произвольной пары (s, a), искусственно в большинстве реальных задач. Например, нельзя начать шахматную партию с королём в произвольной позиции или инициализировать робота в произвольном положении суставов, гарантируя при этом безопасность. Требование исследования должно быть встроено в саму стратегию — это тема следующих двух разделов.

5.5 Управление методом Монте-Карло на текущей стратегии

Управление МК на текущей стратегии отказывается от исследующих стартов и вместо этого обеспечивает попытку каждого действия в каждом состоянии, используя *стохастическую* стратегию, которая всегда назначает некоторую минимальную вероятность каждому действию.

Определение 5.3 (ε-мягкая стратегия)**.** Стратегия π называется **ε-мягкой**, если для всех $s \in \mathcal{S}$ и всех $a \in \mathcal{A}(s)$:

$$\pi(a \mid s) \geq \frac{\varepsilon}{|\mathcal{A}(s)|}, \qquad \varepsilon > 0. \tag{5.6}$$

Среди всех ε-мягких стратегий естественным улучшением данной ε-мягкой стратегии π является ε-жадная стратегия по отношению к текущей оценке Q:

$$\pi'(a \mid s) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & a = \operatorname{argmax}_b Q(s, b), \\ \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & \text{иначе.} \end{cases} \tag{5.7}$$

Теорема об улучшении ε-мягкой стратегии

Теорема 5.4 (Улучшение ε-мягкой стратегии)**.** *Пусть π — ε-мягкая стратегия и π' — ε-жадная стратегия, определённая формулой (5.7) по отношению к Q^π. Тогда $v^{\pi'}(s) \geq v^\pi(s)$ для всех $s \in \mathcal{S}$.*

Доказательство. Зафиксируем $s \in \mathcal{S}$ и пусть $m = |\mathcal{A}(s)|$. Обозначим $M = \max_{a \in \mathcal{A}(s)} Q^\pi(s, a)$.

Поскольку π является ε-мягкой, можно разложить её как

$$\pi(a \mid s) = \frac{\varepsilon}{m} + \delta_a, \quad \delta_a \geq 0, \quad \sum_a \delta_a = 1 - \varepsilon.$$

Тогда, используя сокращённое обозначение $v^\pi(s) = Q^\pi(s, \pi) := \sum_a \pi(a \mid s) Q^\pi(s, a)$:

$$\begin{aligned} v^\pi(s) &= \sum_a \pi(a \mid s)\, Q^\pi(s, a) \\ &= \frac{\varepsilon}{m} \sum_a Q^\pi(s, a) + \sum_a \delta_a\, Q^\pi(s, a) \\ &\leq \frac{\varepsilon}{m} \sum_a Q^\pi(s, a) + (1 - \varepsilon)\, M. \end{aligned} \tag{5.8}$$

Заметим, что правая часть (5.8) есть в точности

$$Q^\pi(s, \pi') = \sum_a \pi'(a \mid s)\, Q^\pi(s, a),$$

поскольку π' назначает вес $1 - \varepsilon + \varepsilon/m$ максимизирующему действию и ε/m всем остальным. Таким образом,

$$Q^\pi(s, \pi') \geq v^\pi(s) = Q^\pi(s, \pi).$$

Это в точности условие теоремы об улучшении стратегии (глава 4), которая даёт $v^{\pi'}(s) \geq v^\pi(s)$. □

Замечание 5.3. Неравенство превращается в равенство — и тем самым $\pi = \pi'$ — только когда $Q^\pi(s, a)$ одинаково для всех a в каждом состоянии, то есть когда π уже оптимальна среди всех ε-мягких стратегий. Этот оптимум — *ε-оптимальная* стратегия: наилучшая достижимая при условии сохранения не менее $\varepsilon/|\mathcal{A}(s)|$ вероятности на каждое действие. В общем случае это не глобально оптимальная детерминированная стратегия π^*. Разрыв уменьшается при $\varepsilon \to 0$.

Алгоритм 8: Управление МК по первому посещению на текущей стратегии (ε-жадная)

Input: Малое $\varepsilon > 0$, дисконт γ
Инициализировать $Q(s, a) \leftarrow 0$, $Count(s, a) \leftarrow 0$ для всех s, a;
$\pi \leftarrow \varepsilon$-жадная по Q;
for *каждый эпизод* **do**
  Сгенерировать эпизод $S_0, A_0, R_1, \ldots, S_\tau$ по стратегии π;
  $G \leftarrow 0$;
  for $t = \tau - 1, \tau - 2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$;
    if (S_t, A_t) *первое посещение в эпизоде* **then**
      $Count(S_t, A_t) \leftarrow Count(S_t, A_t) + 1$;
      $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{Count(S_t, A_t)}\big(G - Q(S_t, A_t)\big)$;
  Обновить π до ε-жадной по текущей Q;

Свойство ε-мягкости гарантирует, что каждая пара (s, a) посещается бесконечно часто, поэтому оценки $Q(s, a)$ сходятся к Q^π по теореме 5.1, и многократное применение теоремы 5.4 толкает стратегию к ε-оптимальной.

Сходимость к ε-оптимальности. Управление МК на текущей стратегии сходится к наилучшей возможной стратегии среди всех ε-мягких стратегий. Это слабее, чем сходимость к глобально оптимальной π^*. Чтобы получить π^*, используют либо исследующие старты (раздел 5.4), либо, что более практично, подход на другой стратегии из раздела 5.6, который позволяет целевой стратегии быть детерминированной, пока поведенческая стратегия занимается исследованием.

5.6 Метод Монте-Карло на другой стратегии с взвешиванием по важности

Проблема обучения на другой стратегии

При обучении на другой стратегии данные собираются *поведенческой стратегией* μ (которая может свободно исследовать), тогда как мы хотим оценить или улучшить *целевую стратегию* π (которая может быть детерминированной и жадной). Фундаментальное препятствие состоит в том, что распределение эпизодов, порождаемое μ, отличается от распределения, порождаемого π. Наивное усреднение отдач давало бы оценки Q^μ, а не Q^π.

Допущение о покрытии. Чтобы оценивание на другой стратегии было корректным, требуется, чтобы π не могла выбрать ни одно действие, которое μ никогда не выбирает:

$$\pi(a \mid s) > 0 \implies \mu(a \mid s) > 0, \quad \forall\, s \in \mathcal{S},\ a \in \mathcal{A}. \tag{5.9}$$

Без (5.9) некоторые траектории, требуемые π, имели бы нулевую вероятность при μ и никогда не появлялись бы в данных.

Отношение важности выборок

Ключевым инструментом является *отношение важности выборок*, которое перевзвешивает вероятность наблюдения конкретной траектории при μ к вероятности наблюдения её при π. Для отрезка траектории $(S_t, A_t, \dots, S_T)$, сгенерированного с помощью μ, отношение вероятности траектории при π к вероятности при μ равно

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{\mu(A_k \mid S_k)}. \tag{5.10}$$

Вероятности переходов среды $p(s', r \mid s, a)$ сокращаются в этом отношении, поскольку они одинаковы для обеих стратегий.

Обычное взвешивание по важности (OIS)

Определение 5.4 (Оценщик обычного взвешивания по важности)**.** Для n эпизодов с отдачами $G_0^{(1)}, \dots, G_0^{(n)}$ и соответствующими весами ВВ $\rho^{(1)}, \dots, \rho^{(n)}$, **оценщик обычного взвешивания по важности** (OIS) для $Q^\pi(s,a)$ равен

$$\widehat{Q}_n^{\mathrm{OIS}}(s,a) = \frac{1}{n} \sum_{i=1}^{n} \rho^{(i)} G_0^{(i)}. \tag{5.11}$$

Теорема 5.5 (OIS несмещён)**.** *При допущении о покрытии* (5.9)*,*

$$\mathbb{E}_\mu[\rho_{0:T-1}\, G_0 \mid S_0 = s,\ A_0 = a] = Q^\pi(s,a).$$

Доказательство. Обозначим хвостовую траекторию через $h = (s_1, a_1, r_2, \dots, s_\tau)$. Тогда

$$Q^\pi(s,a) = \sum_h \mathbb{P}_\pi(h \mid s,a)\, G(h),$$

где $G(h) = \sum_k \gamma^k r_{k+1}$. При стратегии μ:

$$\mathbb{P}_\mu(h \mid s,a) = \prod_{k=1}^{\tau-1} \mu(a_k \mid s_k)\, p(s_{k+1}, r_{k+1} \mid s_k, a_k).$$

По определению ρ:

$$\rho(h)\,\mathbb{P}_\mu(h \mid s,a) = \prod_{k=1}^{\tau-1} \pi(a_k \mid s_k)\, p(s_{k+1}, r_{k+1} \mid s_k, a_k) = \mathbb{P}_\pi(h \mid s,a).$$

Следовательно,

$$\begin{aligned}\mathbb{E}_\mu[\rho\, G_0 \mid S_0 = s, A_0 = a] &= \sum_h \rho(h)\, G(h)\, \mathbb{P}_\mu(h \mid s,a) \\ &= \sum_h G(h)\, \mathbb{P}_\pi(h \mid s,a) = Q^\pi(s,a).\end{aligned}$$

□

Взвешенное взвешивание по важности (WIS)

Определение 5.5 (Оценщик взвешенного взвешивания по важности)**.** Оценщик **взвешенного взвешивания по важности** (WIS) равен

$$\widehat{Q}_n^{\mathrm{WIS}}(s,a) = \frac{\sum_{i=1}^n \rho^{(i)}\, G_0^{(i)}}{\sum_{i=1}^n \rho^{(i)}}, \tag{5.12}$$

когда знаменатель положителен.

WIS является оценщиком-отношением. Он смещён при конечном n, однако асимптотически несмещён и имеет значительно меньшую дисперсию, чем OIS, в случаях, когда веса $\rho^{(i)}$ сильно варьируются. На практике WIS обычно предпочтительнее в задачах с длинными эпизодами или близкими к детерминированной целевой стратегией, поскольку веса ВВ могут быть экспоненциально велики.

Инкрементная реализация WIS

Как и в оценивании на текущей стратегии, WIS можно вычислять инкрементно, поддерживая текущую сумму $C(s,a)$ весов и текущую оценку $Q(s,a)$:

$$C(s,a) \leftarrow C(s,a) + \rho, \tag{5.13}$$

$$Q(s,a) \leftarrow Q(s,a) + \frac{\rho}{C(s,a)}\big(G - Q(s,a)\big). \tag{5.14}$$

Эти обновления применяются за одно обратное прохождение по каждому эпизоду, аналогично алгоритму 6.

Алгоритм 9: МК на другой стратегии с WIS для оценивания Q^π

Input: Целевая стратегия π, поведенческая стратегия μ, дисконт γ

Инициализировать $Q(s,a) \leftarrow 0$, $C(s,a) \leftarrow 0$ для всех s, a;
for *каждый эпизод* **do**
  Сгенерировать эпизод $S_0, A_0, R_1, \ldots, S_\tau$ по стратегии μ;
  $G \leftarrow 0$; $\rho \leftarrow 1$;
  for $t = \tau-1, \tau-2, \ldots, 0$ (в обратном порядке) **do**
    $G \leftarrow \gamma G + R_{t+1}$;
    $\rho \leftarrow \rho \cdot \dfrac{\pi(A_t \mid S_t)}{\mu(A_t \mid S_t)}$;
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + \rho$;
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \dfrac{\rho}{C(S_t, A_t)}\big(G - Q(S_t, A_t)\big)$;
    if $\pi(A_t \mid S_t) = 0$ **then**
      break;
return Q;

Строка “break” использует тот факт, что если $\pi(A_t \mid S_t) = 0$, то $\rho = 0$ для всех более ранних шагов — эти шаги не вносят вклада в оценку Q^π.

5.7 Управление методом Монте-Карло на другой стратегии

Объединяя WIS-оценивание с GPI, получаем полный алгоритм управления на другой стратегии. Целевая стратегия π является жадной по Q, а поведенческая стратегия μ — ε-жадной (или любой другой стратегией, удовлетворяющей допущению о покрытии).

Алгоритм 10: Управление МК на другой стратегии (WIS)

Input: Дисконт γ, параметр исследования ε

1 Инициализировать $Q(s,a)$ произвольно, $C(s,a) \leftarrow 0$, $\pi(s) \leftarrow \operatorname{argmax}_a Q(s,a)$ для всех s, a;
2 **for** *каждый эпизод* **do**
3 $\quad \mu \leftarrow$ любая стратегия, покрывающая π (например, ε-жадная по Q);
4 $\quad$ Сгенерировать эпизод $S_0, A_0, R_1, \ldots, S_\tau$ по μ;
5 $\quad G \leftarrow 0; \quad \rho \leftarrow 1$;
6 $\quad$ **for** $t = \tau - 1, \tau - 2, \ldots, 0$ **do**
7 $\quad\quad G \leftarrow \gamma G + R_{t+1}$;
8 $\quad\quad \rho \leftarrow \rho \cdot \frac{\pi(A_t|S_t)}{\mu(A_t|S_t)}$;
9 $\quad\quad C(S_t, A_t) \leftarrow C(S_t, A_t) + \rho$;
10 $\quad\quad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{\rho}{C(S_t, A_t)}\big(G - Q(S_t, A_t)\big)$;
11 $\quad\quad \pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$;
12 $\quad\quad$ **if** $A_t \neq \pi(S_t)$ **then**
13 $\quad\quad\quad$ **break**;

Условие “break” в строке 12 использует тот факт, что жадная целевая стратегия π будет назначать нулевую вероятность любому действию, не являющемуся жадным; таким образом, ρ становится равным нулю, и более ранние шаги не вносят вклада.

5.8 Сводка и перспективы

Методы Монте-Карло представляют собой первый большой класс безмодельных алгоритмов обучения с подкреплением. Их ключевые свойства таковы:

- **Несмещённость.** Как оценщик по первому посещению (теорема 5.1), так и OIS (теорема 5.5) несмещённы; WIS асимптотически несмещён.
- **Большая дисперсия.** Поскольку целевой является полная отдача G_t, дисперсия растёт с горизонтом. Это главная слабость МК по сравнению с методами TD.
- **Нет бутстрэппинга.** Оценки МК независимы от текущего приближения функции ценности. Это устраняет смещение за счёт бутстрэппинга, но требует ждать конца эпизода.
- **Только эпизодические задачи.** МК-методы требуют терминальных состояний. Для продолжающихся задач необходимы методы TD.

В главе 6 мы увидим, как методы временной разности устраняют последнее ограничение, жертвуя несмещённостью ради возможности обновлений после каждого отдельного перехода.

Упражнения

Упражнение 5.1. Пусть $G_1^{\mathrm{FV}}(s), \ldots, G_n^{\mathrm{FV}}(s)$ — отдачи по первому посещению для состояния s, собранные за n эпизодов. Предположим, что каждая отдача имеет среднее $v^\pi(s)$ и дисперсию $\sigma_s^2 < \infty$.

(a) Покажите, что $\widehat{v}_n^{\mathrm{FV}}(s)$ несмещён.

(b) Вычислите $\operatorname{Var}\big[\widehat{v}_n^{\mathrm{FV}}(s)\big]$.

(c) Сколько эпизодов необходимо, чтобы гарантировать $\operatorname{Var}\big[\widehat{v}_n^{\mathrm{FV}}(s)\big] \leq 0{,}01\,\sigma_s^2$?

Упражнение 5.2. На конкретном примере с двумя состояниями покажите, что МК-оценщик по каждому посещению $\widehat{v}_1^{\mathrm{EV}}(s)$ смещён, если вычисляется по одному эпизоду. *Подсказка*: постройте цикл $s \to s \to \mathrm{T}$ с вознаграждениями 1 и 0 и покажите, что одноэпизодный EV-оценщик не равен $v^\pi(s)$.

Упражнение 5.3. Выведите (5.4) из формулы для текущего выборочного среднего. То есть, начиная с $V_n = \frac{1}{n}\sum_{i=1}^{n} G_i$, покажите алгебраически, что $V_{n+1} = V_n + \frac{1}{n+1}(G_{n+1} - V_n)$.

Упражнение 5.4. В примере с блэкджеком из раздела 5.2 состояние имеет вид (x, d, a), где $x \in \{12, \ldots, 21\}$.

(a) Почему мы ограничиваемся $x \geq 12$? Что происходит для сумм меньше 12?

(b) Почему on-policy МК даёт состоятельные оценки для всех 200 состояний, хотя простая стратегия («стоять на 20 или 21») никогда добровольно не выбирает определённые действия?

(c) Оцените неформально значение $v^{\pi}(20, \text{у дилера } 6, \text{нет применимого туза})$ и обоснуйте свою оценку.

Упражнение 5.5. В алгоритме 7 обновление стратегии (строка 11) применяется только при первом посещении пары (S_t, A_t) в каждом эпизоде. Предположим, что вместо этого мы всегда обновляем Q и π при каждом посещении (включая повторные). Сходится ли алгоритм по-прежнему? Что меняется в аргументе сходимости?

Упражнение 5.6. В доказательстве теоремы 5.4 неравенство (5.8) обращается в равенство. Точно охарактеризуйте, когда это равенство выполняется, и объясните, почему при равенстве стратегия π является ε-оптимальной.

Упражнение 5.7. Пусть $X = \rho G$ — одна OIS-взвешенная отдача. Покажите, что

$$\mathrm{Var}_{\mu}[\rho G] = \mathbb{E}_{\mu}[\rho^2 G^2] - (Q^{\pi}(s, a))^2.$$

Объясните, почему $\mathbb{E}_{\mu}[\rho^2 G^2]$ может быть очень большим, когда π и μ существенно различаются. Как это мотивирует WIS?

Упражнение 5.8. Докажите, что если $\pi = \mu$, то $\rho_{t:T-1} = 1$ для каждого эпизода и оба оценщика OIS и WIS сводятся к стандартному on-policy выборочному среднему Монте-Карло.

Упражнение 5.9. Покажите, что WIS-оценщик $\widehat{Q}_1^{\mathrm{WIS}}(s, a)$, вычисленный по одному эпизоду, удовлетворяет $\widehat{Q}_1^{\mathrm{WIS}}(s, a) = G_0^{(1)}$ независимо от веса ВВ $\rho^{(1)}$. Сделайте вывод, что при одном эпизоде WIS в общем случае не является несмещённым.

Упражнение 5.10. Рассмотрим 5-шаговый эпизодический МПР, в котором единственное вознаграждение равно $+1$ на финальном шаге и 0 в остальных случаях. Дисконт $\gamma = 1$. Стратегия фиксирована и детерминирована.

(a) Какова МК-отдача G_t для каждого момента времени $t = 0, 1, 2, 3, 4$?

(b) Если запустить МК по первому посещению на 1000 эпизодов, к чему сходится оценка $V(s_0)$?

(c) Объясните качественно, почему дисперсия G_0 выше, чем дисперсия G_4. При каком условии они были бы равны?

Глава 6

Обучение с временной разностью

Если и можно выделить одну идею как центральную и новаторскую в обучении с подкреплением, то это, несомненно, обучение с временной разностью (TD).

Обучение с временной разностью — наиважнейшая идея в обучении с подкреплением. Оно синтезирует два мощных направления мысли: *выборочный* подход методов Монте-Карло, который обучается непосредственно из опыта без модели среды, и *бутстрэппинговый* подход динамического программирования, который обновляет оценки на основе других оценок, а не дожидаясь окончательного результата. В итоге получается семейство алгоритмов, способных обучаться онлайн, после каждого отдельного перехода, как в эпизодических, так и в продолжающихся задачах, не требуя модели среды.

В этой главе семейство TD развивается во всей полноте. Мы начинаем с простейшего его члена, TD(0), и устанавливаем его ключевые свойства: ошибку TD, свойство нулевого среднего при истинной функции ценности и сходимость. Затем переходим к TD-алгоритмам управления — SARSA, Q-learning, Expected SARSA и Double Q-learning — и тщательно сравниваем их характер применительно к текущей и другой стратегии. Обобщаем до n-шаговых отдач, которые интерполируют между TD и Монте-Карло, а затем до полного алгоритма TD(λ) со следами приемлемости, который одновременно объединяет информацию со всех горизонтов. Завершаем обобщённой оценкой преимущества (GAE) — современным методом для оценки преимущества в методах градиента стратегии, который прямо основан на механизме TD(λ), — и тщательным анализом смещения, дисперсии и сходимости по всему семейству.

6.1 Предсказание TD(0)

6.1.1 От метода Монте-Карло к временной разности

Вспомним из главы 5, что предсказание методом Монте-Карло (МК) оценивает функцию ценности V^π, усредняя полные выборочные отдачи. После каждого полного эпизода каждое посещённое состояние S_t получает обновление

$$V(S_t) \leftarrow V(S_t) + \alpha\big(G_t - V(S_t)\big), \tag{6.1}$$

где $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ — дисконтированная отдача с момента t. Этот оценщик несмещён — $\mathbb{E}_\pi[G_t \mid S_t = s] = V^\pi(s)$ — но имеет три ограничения: требует завершения эпизода до начала обучения; накапливает дисперсию вознаграждений на длинных горизонтах; вовсе не применим к продолжающимся (не эпизодическим) задачам.

Уравнение Беллмана для фиксированной стратегии π предлагает альтернативную цель. Поскольку

$$V^\pi(s) = \mathbb{E}_\pi\big[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s\big], \tag{6.2}$$

истинная ценность состояния s равна ожидаемому немедленному вознаграждению плюс дисконтированная ценность следующего состояния. Уравнение (6.2) точно при $V = V^\pi$, но даже при несовершенной оценке $V \approx V^\pi$ величина

$$R_{t+1} + \gamma V(S_{t+1}) \tag{6.3}$$

представляет собой полезную *одношаговую цель*, которую можно вычислить сразу после наблюдения перехода $(S_t, A_t, R_{t+1}, S_{t+1})$. Заменяя полную отдачу G_t в (6.1) этой одношаговой целью, получаем

$$\boxed{V(S_t) \leftarrow V(S_t) + \alpha\Big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\Big).} \tag{6.4}$$

Это алгоритм **TD(0)**, также называемый одношаговым методом временной разности.

6.1.2 Ошибка TD

Определение 6.1 (Ошибка TD)**. Ошибка временной разности** в момент t равна

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t). \tag{6.5}$$

Ошибка TD измеряет расхождение между текущей оценкой $V(S_t)$ и одношаговой целью $R_{t+1}+\gamma V(S_{t+1})$. С этим обозначением обновление TD(0) (6.4) упрощается до

$$V(S_t) \leftarrow V(S_t) + \alpha\,\delta_t.$$

Знак δ_t имеет непосредственную интуитивную интерпретацию:

- $\delta_t > 0$: наблюдаемый переход оказался лучше, чем ожидалось; $V(S_t)$ следует увеличить.
- $\delta_t < 0$: состояние было переоценено; $V(S_t)$ следует уменьшить.
- $\delta_t = 0$: текущая оценка локально согласована с уравнением Беллмана.

Определение 6.2 (Бутстрэппинг)**. Бутстрэппинг** — это использование текущей оценки функции ценности как части цели в правиле обновления. В TD(0) цель $R_{t+1}+\gamma V(S_{t+1})$ зависит от оценки V в следующем состоянии, поэтому метод использует бутстрэппинг. Монте-Карло не использует бутстрэппинг, поскольку G_t является истинной отдачей, независимой от V.

6.1.3 Свойство нулевого среднего ошибки TD

Следующий результат является фундаментальным: он показывает, что δ_t — корректный (нулевого среднего) градиентный сигнал, когда функция ценности точна.

Теорема 6.1 (Нулевое среднее ошибки TD)**.** *Если $V = V^\pi$, то для каждого состояния s:*

$$\mathbb{E}_\pi[\delta_t \mid S_t = s] = 0. \tag{6.6}$$

Доказательство. Подставляя $V = V^\pi$ в определение (6.5):

$$\delta_t = R_{t+1} + \gamma V^\pi(S_{t+1}) - V^\pi(S_t).$$

Беря условное математическое ожидание при $S_t = s$:

$$\begin{aligned}\mathbb{E}_\pi[\delta_t \mid S_t = s] &= \mathbb{E}_\pi\big[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s\big] - V^\pi(s)\\ &= V^\pi(s) - V^\pi(s) = 0,\end{aligned}$$

где вторая строка использует уравнение Беллмана (6.2). □

Теорема 6.1 подразумевает, что в неподвижной точке TD-обновление имеет нулевое ожидаемое изменение. До сходимости $\delta_t \neq 0$ в среднем, и его знак последовательно указывает направление, уменьшающее ошибку Беллмана.

6.1.4 Алгоритм TD(0)

Алгоритм 11: TD(0) для оценивания стратегии

Input: Стратегия π, шаг $\alpha \in (0, 1]$, дисконт $\gamma \in [0, 1)$
1 Инициализировать $V(s) \leftarrow 0$ для всех $s \in \mathcal{S}$, $V(s_{\text{term}}) \leftarrow 0$;
2 **for** *каждый эпизод* **do**
3 Инициализировать S_0;
4 **for** $t = 0, 1, 2, \ldots$ *пока S_t не терминальное* **do**
5 Выбрать $A_t \sim \pi(\cdot \mid S_t)$;
6 Выполнить действие A_t; наблюдать R_{t+1} и S_{t+1};
7 $\delta_t \leftarrow R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$;
8 $V(S_t) \leftarrow V(S_t) + \alpha\,\delta_t$;

Заметим, что, в отличие от Монте-Карло, обновление в строке 8 происходит *немедленно* после каждого перехода — ждать конца эпизода не нужно.

6.1.5 Сходимость табличного TD(0)

Сходимость TD(0) можно понять в рамках оператора Беллмана. Для фиксированной стратегии π определим

$$(T^\pi V)(s) = \mathbb{E}_\pi\big[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s\big].$$

Уравнение Беллмана принимает вид $V^\pi = T^\pi V^\pi$. Поскольку T^π является γ-сжатием в супремум-норме, его единственная неподвижная точка — V^π. TD(0) можно интерпретировать как стохастическое приближение итерации $V \leftarrow T^\pi V$: вместо вычисления полного математического ожидания используется один выборочный переход.

Теорема 6.2 (Сходимость табличного TD(0))**.** *В табличном случае, если π фиксирована, все состояния посещаются бесконечно часто, вознаграждения ограничены, и шаги $\{\alpha_t(s)\}$ удовлетворяют условиям Роббинса–Монро:*

$$\sum_{t=1}^{\infty} \alpha_t(s) = \infty, \qquad \sum_{t=1}^{\infty} \alpha_t^2(s) < \infty \quad \text{для каждого } s, \tag{6.7}$$

то $V_t(s) \to V^\pi(s)$ почти наверное для всех $s \in \mathcal{S}$.

Условия Роббинса–Монро (6.7) требуют, чтобы шаги были достаточно велики для коррекции любой начальной ошибки (первое условие), но достаточно малы для усреднения шума (второе условие). Расписание $\alpha_t(s) = 1/N_t(s)$, где $N_t(s)$ — счётчик посещений, удовлетворяет обоим условиям. Постоянный шаг α удовлетворяет только первому, поэтому он не сходится к точному V^π, но отслеживает его в нестационарной среде.

6.1.6 Разобранный пример: случайное блуждание

Пример 6.1 (Случайное блуждание по пяти состояниям)**.** Рассмотрим цепь Маркова с пятью состояниями A, B, C, D, E, расположенными линейно, плюс два терминальных состояния L (слева) и R (справа). Из любого нетерминального состояния агент движется влево или вправо с равной вероятностью 0.5. Единственное ненулевое вознаграждение — $+1$ при переходе в правое терминальное состояние R; все остальные вознаграждения равны 0. Коэффициент дисконтирования $\gamma = 1$. Истинные ценности состояний (то есть вероятность в конечном счёте достичь R) равны:

$$V^\pi(A) = \tfrac{1}{6}, \quad V^\pi(B) = \tfrac{2}{6}, \quad V^\pi(C) = \tfrac{3}{6}, \quad V^\pi(D) = \tfrac{4}{6}, \quad V^\pi(E) = \tfrac{5}{6}.$$

Предположим, что инициализируем $V(s) = 0.5$ для всех состояний и запускаем TD(0) с $\alpha = 0.1$. Рассмотрим одну траекторию $C \to D \to E \to R$.

Шаг 1: $S_t = C$, $A_t =$ вправо, $R_{t+1} = 0$, $S_{t+1} = D$.

$$\delta_t = 0 + 1.0 \cdot 0.5 - 0.5 = 0.$$

На этом шаге обновлений нет (поскольку $V(D) = V(C) = 0.5$ изначально).

Шаг 2: $S_t = D$, $R_{t+1} = 0$, $S_{t+1} = E$.

$$\delta_t = 0 + 0.5 - 0.5 = 0.$$

Снова без изменений.

Шаг 3: $S_t = E$, $R_{t+1} = 1$, $S_{t+1} = R$ (терминальное, $V(R) = 0$).

$$\delta_t = 1 + 0 - 0.5 = 0.5.$$

$$V(E) \leftarrow 0.5 + 0.1 \times 0.5 = 0.55.$$

Сигнал вознаграждения распространяется только на один шаг назад. На протяжении многих эпизодов TD(0) постепенно распространяет оценки ценности назад по цепочке. Примерно после 100 эпизодов оценки близки к истинным значениям. Этот пример демонстрирует как скоростное преимущество TD (обучение внутри каждого эпизода), так и проблему присвоения кредита: каждый новый сигнал вознаграждения должен распространяться назад по пространству состояний шаг за шагом.

6.2 Преимущества методов TD

Методы TD занимают привилегированное положение в области обучения с подкреплением, поскольку унаследовали преимущества как от Монте-Карло, так и от динамического программирования, избегая их главных слабостей. Перечислим эти преимущества.

Не требуют полного эпизода. Методы Монте-Карло должны ждать конца эпизода для вычисления G_t. Это проблематично для длинных эпизодов и невозможно для продолжающихся задач без естественного завершения. TD(0) обновляется после каждого отдельного шага, что делает его немедленно применимым к любой структуре задач.

Работают для продолжающихся задач. Поскольку цель TD $R_{t+1} + \gamma V(S_{t+1})$ всегда доступна (терминальное состояние не нужно), методы TD естественно обрабатывают продолжающиеся задачи. Методы Монте-Карло в своей базовой форме не могут применяться в таких условиях.

Меньше дисперсия, чем у Монте-Карло. Отдача Монте-Карло $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$ накапливает дисперсию от каждого будущего вознаграждения. Цель TD $R_{t+1} + \gamma V(S_{t+1})$ включает лишь одно случайное вознаграждение и (относительно гладкую) оценку $V(S_{t+1})$. Формально, если каждое вознаграждение имеет дисперсию σ^2 и переходы независимы, дисперсия МК растёт примерно как $\sum_{k=0}^{T-t-1} \gamma^{2k}\sigma^2$, тогда как дисперсия TD примерно равна $\sigma^2 + \gamma^2 \operatorname{Var}[V(S_{t+1})]$.

Компромисс смещение–дисперсия. Ценой меньшей дисперсии является то, что цель TD *смещена*: поскольку в начале обучения $V \neq V^\pi$, цель $R_{t+1} + \gamma V(S_{t+1})$ не равна $V^\pi(S_t)$ в математическом ожидании. По теореме 6.1 смещение исчезает при сходимости. Монте-Карло несмещён, но имеет высокую дисперсию. Этот фундаментальный компромисс — бутстрэппинг покупает меньшую дисперсию ценой смещения — присутствует во всех TD-методах и количественно описан в разделе 6.11.

Более быстрое присвоение кредита. Поскольку TD распространяет оценки ценности на один шаг в каждом эпизоде, полезная информация из вознаграждаемых переходов включается в более ранние состояния быстрее, чем при МК. n-шаговое TD (раздел 6.8) и следы приемлемости (раздел 6.9) предоставляют механизмы для эффективного распространения кредита на несколько шагов.

Пакетное TD(0) и оценка максимального правдоподобия. Когда доступен фиксированный пакет эпизодов, можно многократно применять обновления TD(0) до сходимости. Это *пакетное TD(0)* сходится к *оценке максимального правдоподобия* (оценке с определённостью эквивалентности) V^π по данным — точному решению, которое вычислялось бы построением эмпирического МПР из пакета и его решением. Для сравнения, пакетный Монте-Карло сходится к оценке, минимизирующей среднеквадратичную ошибку на тренировочных отдачах. В задачах с ограниченными данными TD-решение часто лучше обобщается, поскольку использует марковскую структуру.

6.3 SARSA: управление TD на текущей стратегии

6.3.1 От предсказания к управлению

Оценивание стратегии говорит нам, насколько хороша текущая стратегия. Управление требует её улучшения. Как и в управлении методом Монте-Карло (глава 5), ключевое наблюдение состоит в том, что нужно оценивать *функцию ценности действия* $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$, а не $V^\pi(s)$, поскольку жадное улучшение требует сравнения действий:

$$\pi'(s) \in \operatorname*{argmax}_{a \in \mathcal{A}} Q(s,a).$$

Уравнение Беллмана для Q^π имеет вид:

$$Q^\pi(s,a) = \mathbb{E}_\pi\big[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a\big]. \tag{6.8}$$

Заменяя математическое ожидание в (6.8) одной выборкой и применяя полученное обновление, получаем:

6.3.2 Обновление SARSA

Определение 6.3 (SARSA)**. SARSA**[1] (Раммери и Нираньян, 1994; Саттон, 1996) — это алгоритм управления TD на текущей стратегии с правилом обновления:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big). \tag{6.9}$$

[1] Название происходит от пятёрки $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, используемой в каждом обновлении.

Критическая особенность SARSA состоит в том, что A_{t+1} извлекается из *той же* стратегии π, которая используется для сбора данных. Это делает SARSA **алгоритмом на текущей стратегии**: он оценивает и улучшает стратегию, которая генерирует данные.

Утверждение 6.3. *Цель* $Y_t^{\text{SARSA}} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ *является несмещённой выборкой оператора Беллмана, применённого к* Q*:*

$$\mathbb{E}_\pi\big[Y_t^{\text{SARSA}} \mid S_t = s, A_t = a\big] = (T^\pi Q)(s, a),$$

где $(T^\pi Q)(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$.

Доказательство. По определению условного математического ожидания утверждение следует непосредственно. □

Алгоритм 12: SARSA (управление TD на текущей стратегии)

```
Input: Шаг α, дисконт γ, исследование ε
Инициализировать Q(s, a) для всех s, a; Q(s_term, ·) ← 0;
for каждый эпизод do
    Инициализировать S;
    Выбрать A из π(· | S) на основе Q (например, ε-жадная);
    for каждый шаг эпизода пока S не терминальное do
        Выполнить действие A; наблюдать R, S';
        Выбрать A' из π(· | S') (та же ε-жадная);
        Q(S, A) ← Q(S, A) + α(R + γQ(S', A') − Q(S, A));
        S ← S';   A ← A';
```

6.3.3 Сходимость SARSA

Теорема 6.4 (Сходимость SARSA)**.** *В табличном случае SARSA сходится к* Q^* *почти наверное, если: (1) все пары состояние-действие посещаются бесконечно часто; (2) стратегия сходится к жадной (например,* $\varepsilon_t \to 0$*); (3) шаги удовлетворяют условиям Роббинса–Монро* (6.7)*.*

Сходимость требует, чтобы стратегия в пределе стала жадной, поэтому исследование должно уменьшаться со временем. На практике часто используют постоянные ε или α, при которых SARSA сходится к окрестности Q^*, а не к Q^* точно.

6.3.4 Разобранный пример: хождение по скалам

Пример 6.2 (Хождение по скалам)**.** Рассмотрим клеточный мир 4×12 с обрывом вдоль нижнего края (клетки $(3, 1)$–$(3, 10)$). Агент начинает в $(3, 0)$ и должен достичь цели в $(3, 11)$. Наступание на обрыв даёт вознаграждение -100 и возвращает агента на старт. Все остальные переходы дают вознаграждение -1. Эпизоды завершаются в цели или после 200 шагов.

SARSA с $\varepsilon = 0.1$ обучается *безопасному пути* вдоль верха сетки, держась подальше от обрыва. Поскольку исследовательские действия иногда подталкивают агента к обрыву, SARSA оценивает клетки обрыва как опасные и избегает их. Q-learning (раздел 6.4), будучи алгоритмом на другой стратегии, оценивает каждое состояние при *жадной* стратегии и поэтому находит кратчайший путь вдоль края обрыва — однако этот путь уязвим на практике, поскольку ε-жадное исследование иногда приводит агента к обрыву во время обучения. Это канонический пример компромисса между текущей и другой стратегией: SARSA безопаснее во время обучения; Q-learning находит теоретически оптимальную стратегию.

6.4 Q-learning: управление TD на другой стратегии

6.4.1 Обновление Q-learning

Q-learning (Уоткинс, 1989) непосредственно аппроксимирует оптимальную функцию ценности действия Q^*, используя уравнение оптимальности Беллмана в качестве цели. Поскольку $Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid S_t = s, A_t = a]$, одновыборочное приближение даёт:

$$\boxed{Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\Big).} \tag{6.10}$$

Ключевое отличие от SARSA (6.9) — оператор **max**. Независимо от того, как фактически выбирается A_{t+1} (по ε-жадной или любой другой поведенческой стратегии), цель всегда использует жадное действие в S_{t+1}. Это делает Q-learning **алгоритмом на другой стратегии**: поведенческая стратегия собирает данные, а обновление оценивает жадную (целевую) стратегию.

Алгоритм 13: Q-learning (управление TD на другой стратегии)

Input: Шаг α, дисконт γ, исследование ε
Инициализировать $Q(s, a)$ для всех s, a; $Q(s_{\text{term}}, \cdot) \leftarrow 0$;
for *каждый эпизод* **do**
  Инициализировать S;
  for *каждый шаг пока S не терминальное* **do**
    Выбрать A из Q с помощью ε-жадной стратегии;
    Выполнить действие A; наблюдать R, S';
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big(R + \gamma \max_{a'} Q(S', a') - Q(S, A)\big)$;
    $S \leftarrow S'$;

6.4.2 Сходимость Q-learning

Теорема 6.5 (Сходимость Q-learning, Уоткинс и Дайан, 1992)**.** *В конечном МПР, если все пары состояние-действие посещаются бесконечно часто, вознаграждения ограничены и шаги удовлетворяют условиям Роббинса–Монро (6.7), то Q-learning сходится к Q^* почти наверное.*

Этот результат замечателен: он справедлив независимо от поведенческой стратегии, используемой для генерации переходов, при условии, что каждая пара (s, a) посещается достаточно часто. Q-learning является полностью внестратегийным методом с гарантиями сходимости в табличном случае.

6.4.3 Смещение максимизации

Тонкая, но важная проблема возникает из-за использования $\max_{a'} Q(S_{t+1}, a')$ в качестве цели. Когда Q — зашумлённая оценка, максимум зашумлённых значений является *положительно смещённым* оценщиком истинного максимального значения. Это связано с тем, что по неравенству Йенсена, применённому к выпуклой функции максимума:

$$\mathbb{E}\Big[\max_{a'} Q(S_{t+1}, a')\Big] \geq \max_{a'} \mathbb{E}[Q(S_{t+1}, a')].$$

Смещение может быть существенным, когда много действий имеют схожие истинные значения, но зашумлённые оценки.

Пример 6.3 (Смещение максимизации)**.** Предположим, что состояние s' имеет три действия с истинными значениями $Q^*(s', a_1) = Q^*(s', a_2) = Q^*(s', a_3) = 0$. Оценки $\hat{Q}(s', a_i) \sim \mathcal{N}(0, \sigma^2)$ — независимый гауссовский шум. Тогда $\mathbb{E}[\max_i \hat{Q}(s', a_i)] > 0$ — положительное смещение, которое растёт с σ^2 и числом действий. Q-learning использует это смещённое максимальное значение как цель, вызывая систематическое завышение оценок.

6.5 Expected SARSA

SARSA использует фактическое следующее действие A_{t+1} в своей цели, вводя дисперсию из-за случайности стратегии. Q-learning устраняет эту дисперсию, беря жадное действие, но несёт смещение максимизации. **Expected SARSA** (ван Сейен и др., 2009) выбирает средний путь: заменяет $Q(S_{t+1}, A_{t+1})$ *математическим ожиданием* по стратегии:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1})\, Q(S_{t+1}, a') - Q(S_t, A_t)\right). \tag{6.11}$$

Алгоритм 14: Expected SARSA

Input: Шаг α, дисконт γ, стратегия π (или ε-жадная)
Инициализировать $Q(s, a)$ для всех s, a;
for *каждый эпизод* **do**
  Инициализировать S;
  for *каждый шаг пока S не терминальное* **do**
    Выбрать A из $\pi(\cdot \mid S)$;
    Выполнить действие A; наблюдать R, S';
    Вычислить $\bar{Q}(S') \leftarrow \sum_{a'} \pi(a' \mid S')\, Q(S', a')$;
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big(R + \gamma\, \bar{Q}(S') - Q(S, A)\big)$;
    $S \leftarrow S'$;

Expected SARSA устраняет дисперсию от случайного выбора A_{t+1}, поэтому его обновления менее зашумлены, чем у SARSA. Его также можно использовать вне текущей стратегии: если π является жадной стратегией (вероятность 1 на $\operatorname{argmax}_a Q(s, a)$, вероятность 0 иначе), Expected SARSA сводится в точности к Q-learning, поскольку

$$\sum_{a'} \pi_{\text{жадная}}(a' \mid s') Q(s', a') = \max_{a'} Q(s', a').$$

Таким образом, Expected SARSA обобщает как SARSA (на текущей стратегии, π — поведенческая), так и Q-learning (на другой стратегии, π — жадная).

6.6 Double Q-learning

Double Q-learning (ван Хасселт, 2010) устраняет смещение максимизации, поддерживая *две независимые* оценки функции ценности действия, Q_1 и Q_2. Идея — *разделить* выбор действия и оценку действия:

- Использовать Q_1 для выбора жадного действия: $a^* = \operatorname{argmax}_{a'} Q_1(S_{t+1}, a')$.
- Использовать Q_2 для оценки этого действия: цель $= R_{t+1} + \gamma Q_2(S_{t+1}, a^*)$.

Поскольку Q_1 и Q_2 — независимые оценки, использование одной для выбора и другой для оценки устраняет положительное смещение. Важно, что роли Q_1 и Q_2 меняются с вероятностью 0.5 на каждом шаге, чтобы обе оценки оставались актуальными.

Алгоритм 15: Double Q-learning

Input: Шаг α, дисконт γ, исследование ε
Инициализировать $Q_1(s, a)$ и $Q_2(s, a)$ для всех s, a;
for *каждый эпизод* **do**
  Инициализировать S;
  for *каждый шаг пока S не терминальное* **do**
    $A \leftarrow \varepsilon$-жадная по $Q_1(S, \cdot) + Q_2(S, \cdot)$;
    Наблюдать R, S';
    if *с вероятностью* 0.5 **then**
      $a^* \leftarrow \operatorname{argmax}_{a'} Q_1(S', a')$;
      $Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\big(R + \gamma Q_2(S', a^*) - Q_1(S, A)\big)$;
    else
      $a^* \leftarrow \operatorname{argmax}_{a'} Q_2(S', a')$;
      $Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\big(R + \gamma Q_1(S', a^*) - Q_2(S, A)\big)$;
    $S \leftarrow S'$;

Double Q-learning доказуемо уменьшает смещение максимизации и работает лучше, чем Q-learning, в средах с большим числом действий. Эта идея позднее была масштабирована до глубокого ОП в алгоритме Double DQN (ван Хасселт, Гез и Сильвер, 2016), который использует онлайн-сеть для выбора действия и целевую сеть для оценки.

6.7 Сравнение: SARSA, Q-learning и Expected SARSA

В таблице 6.1 обобщены три основных алгоритма управления TD.

Таблица 6.1: Сравнение алгоритмов управления TD на текущей и другой стратегии.

Свойство	**SARSA**	**Q-learning**	**Expected SARSA**
Цель	$R+\gamma Q(S',A')$	$R+\gamma \max_{a'} Q(S',a')$	$R+\gamma\sum_{a'} \pi(a'\|S')Q(S',a')$
Стратегия	На текущей	На другой	Обе
Задача	Q^π	Q^*	Q^π или Q^*
Дисперсия	Средняя	Низкая	Ниже, чем у SARSA
Смещение	Несмещённое	Смещение макс.	Несмещённое
Безопасно?	Да	Нет (обрыв)	Да

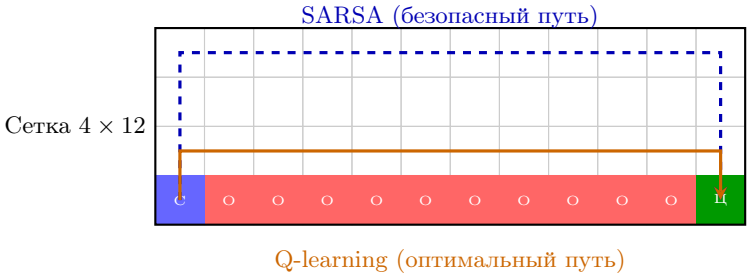


Рис. 6.1: Клеточный мир «хождение по скалам». Красные клетки — обрыв (вознаграждение -100, сброс на старт). SARSA (синий пунктир) обучается безопасному пути вдоль верха; Q-learning (сплошной оранжевый) обучается теоретически более короткому пути вдоль края обрыва, но этот путь уязвим при ε-жадном исследовании.

Пример «хождение по скалам» (пример 6.2 и рисунок 6.1) иллюстрирует практическое следствие оценивания на текущей и другой стратегии. Во время обучения с ε-жадным исследованием SARSA учитывает возможность случайных действий, толкающих агента к обрыву, и поэтому предпочитает более безопасный маршрут. Q-learning оценивает состояния при жадной стратегии — которая никогда не сходит с края обрыва — поэтому обучается маршруту вдоль обрыва, оптимальному, но уязвимому. При тестировании ($\varepsilon = 0$) оба находят эквивалентные пути, но SARSA достигает более высокого среднего вознаграждения во время обучения.

6.8 n-шаговые методы TD

6.8.1 n-шаговая отдача

Оба метода — TD(0) и Монте-Карло — являются частными случаями более общего семейства, параметризованного *горизонтом просмотра вперёд* $n \in \{1, 2, \dots\}$. **n-шаговая отдача** определяется как

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}), \tag{6.12}$$

а соответствующее обновление:

$$V(S_t) \leftarrow V(S_t) + \alpha\big(G_t^{(n)} - V(S_t)\big). \tag{6.13}$$

Утверждение 6.6. *n-шаговая отдача удовлетворяет следующим свойствам:*

1. *При $n = 1$: $G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$ — цель TD(0).*
2. *В эпизодической задаче с терминальным временем T: $G_t^{(T-t)} = G_t$ — полная отдача Монте-Карло (при $V(S_T) = 0$).*

Доказательство. (1) Подстановка $n = 1$ в (6.12) даёт $G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$. (2) При $n = T - t$ и $V(S_T) = 0$: $G_t^{(T-t)} = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} = G_t$. □

6.8.2 Компромисс смещение–дисперсия в n-шаговом TD

По мере увеличения n n-шаговая отдача интерполирует между смещённой, но с низкой дисперсией целью TD(0) и несмещённой, но с высокой дисперсией отдачей МК:

- Малое n: больше бутстрэппинга, выше смещение, ниже дисперсия.
- Большое n: меньше бутстрэппинга, ниже смещение, выше дисперсия.

Оптимальное n зависит от задачи: в средах с плотными вознаграждениями с низкой дисперсией малое n работает хорошо; в средах с редкими вознаграждениями часто полезнее большее n.

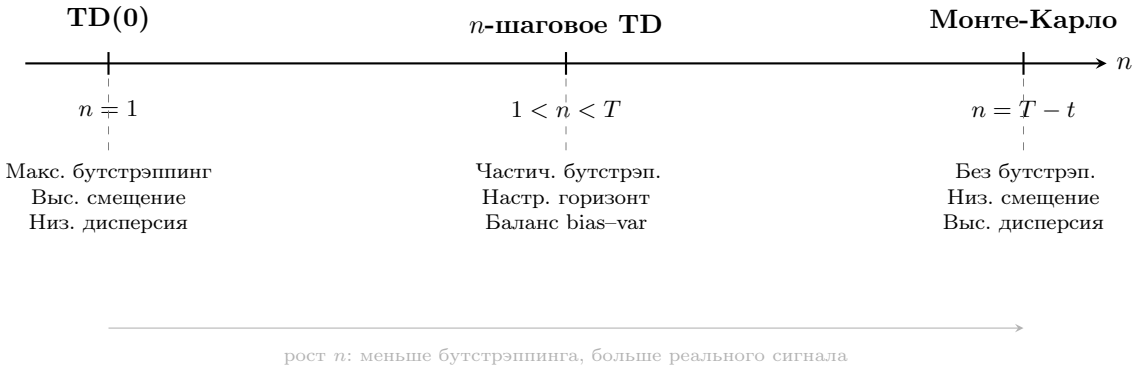


Рис. 6.2: Спектр n-шаговых TD-методов. TD(0) ($n = 1$) и Монте-Карло ($n = T - t$) — крайние случаи. Промежуточные n обеспечивают компромисс смещение–дисперсия, настраиваемый под задачу.

6.8.3 n-шаговый SARSA

Идея n-шагового подхода естественно распространяется на управление. Цель n-шагового SARSA:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n}),$$

а обновление: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(G_t^{(n)} - Q(S_t, A_t))$. Этот алгоритм обладает гарантиями сходимости, аналогичными стандартному SARSA при подходящих условиях.

6.9 TD(λ) и следы приемлемости

Методы предыдущего раздела требуют фиксации одного горизонта n заранее. TD(λ) вместо этого объединяет *все* n-шаговые отдачи одновременно, взвешивая их геометрически так, что объединённая цель вычисляется онлайн с помощью одного дополнительного вектора — *следа приемлемости*.

6.9.1 λ-отдача

Определение 6.4 (λ-отдача)**.** Для $\lambda \in [0, 1]$ **λ-отдача** — это геометрически взвешенное среднее всех n-шаговых отдач:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}. \tag{6.14}$$

Вес, присваиваемый n-шаговой отдаче, равен $(1 - \lambda)\lambda^{n-1}$. Эти веса положительны, геометрически убывают с ростом n и в сумме равны единице:

$$\sum_{n=1}^{\infty} (1 - \lambda)\lambda^{n-1} = (1 - \lambda) \cdot \frac{1}{1 - \lambda} = 1.$$

Таким образом, G_t^λ является настоящей выпуклой комбинацией n-шаговых отдач.

Частные случаи.

- $\lambda = 0$: $G_t^0 = G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$ — цель TD(0). Весь вес на одношаговой отдаче.
- $\lambda = 1$: $G_t^1 = G_t$ — полная отдача Монте-Карло. Геометрические веса равномерны (в пределе дают полную отдачу).

Таким образом, TD(λ) поглощает как TD(0), так и Монте-Карло как крайние случаи, как и n-шаговое TD — но с элегантной дифференцируемой интерполяцией.

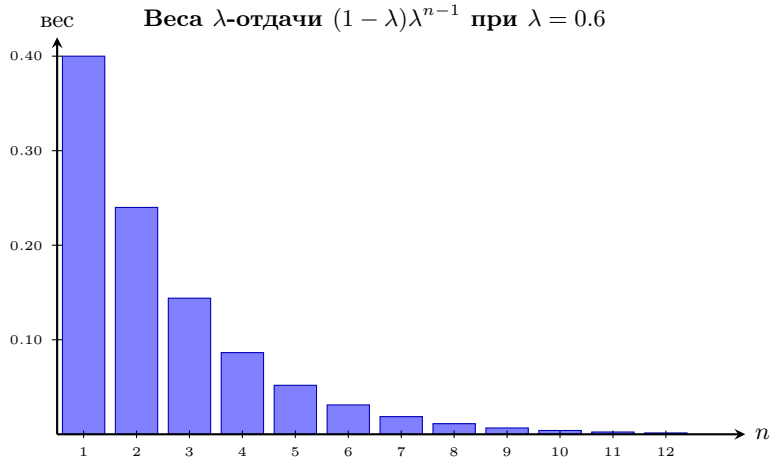


Рис. 6.3: Веса $(1-\lambda)\lambda^{n-1}$, присваиваемые каждой n-шаговой отдаче в λ-отдаче ($\lambda = 0.6$). Краткосрочные отдачи получают наибольший вес; веса геометрически убывают, поэтому влияние далёких отдач быстро уменьшается.

6.9.2 Взгляд вперёд на TD(λ)

Взгляд вперёд на TD(λ) определяет цель обновления для состояния S_t как G_t^λ и применяет

$$V(S_t) \leftarrow V(S_t) + \alpha\big(G_t^\lambda - V(S_t)\big).$$

Это концептуально прозрачно: цель является конкретной, хорошо определённой величиной. Однако вычисление G_t^λ требует всей будущей траектории с момента t, поэтому взгляд вперёд требует ждать завершения эпизода до применения любого обновления. Он поэтому не является онлайновым в операциональном смысле и эквивалентен однопроходному апостериорному обновлению по эпизоду.

Несмотря на непрактичность для онлайн-обучения, взгляд вперёд является *золотым стандартом* для понимания того, что вычисляет TD(λ). Взгляд назад, который мы развиваем далее, математически эквивалентен взгляду вперёд, но работает онлайн.

6.9.3 Следы приемлемости

Ключ к превращению TD(λ) в онлайновый алгоритм — **след приемлемости** $e_t(s)$, вектор по состояниям, который регистрирует, насколько «приемлемо» каждое состояние для кредитного обновления в момент t.

Определение 6.5 (След приемлемости (накапливающий))**. Накапливающий след приемлемости** для состояния s инициализируется значением $e_0(s) = 0$ и обновляется каждый шаг по правилу:

$$e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbb{1}\{S_t = s\}. \tag{6.15}$$

След $e_t(s)$ увеличивается на 1 при каждом посещении состояния s и убывает с коэффициентом $\gamma\lambda$ на каждом шаге времени. Интуитивно он измеряет давность и частоту посещений s: состояние, посещаемое часто и недавно, имеет высокий след.

Альтернатива — **заменяющий след приемлемости**:

$$e_t(s) = \begin{cases} \gamma\lambda\, e_{t-1}(s) + 1 & \text{если } s = S_t, \\ \gamma\lambda\, e_{t-1}(s) & \text{иначе,} \end{cases} \quad \text{ограниченный значением 1.} \tag{6.16}$$

Заменяющие следы не позволяют следу для текущего состояния превышать 1, что может стабилизировать обучение в некоторых условиях.

6.9.4 Взгляд назад на TD(λ): онлайновый алгоритм

Во **взгляде назад** на каждом шаге вычисляется одна ошибка TD δ_t, и *все* состояния обновляются одновременно, взвешенные своими следами приемлемости:

$$\boxed{V(s) \leftarrow V(s) + \alpha\,\delta_t\,e_t(s), \quad \forall s \in \mathcal{S}.} \tag{6.17}$$

Ошибка TD $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ транслируется во все состояния, но только состояния с ненулевым следом получают значимое обновление. Недавно посещённые состояния — с высокими следами — получают большие обновления; состояния, не посещавшиеся недавно, имеют почти нулевые следы и практически не затрагиваются. Это и есть механизм «присвоения кредита» следов приемлемости: хорошие или плохие исходы засчитываются пропорционально давности и частоте прошлых посещений.

Алгоритм 16: TD(λ) со следами приемлемости

Input: Шаг α, дисконт γ, параметр следа λ
Инициализировать $V(s) \leftarrow 0$ для всех s;
for *каждый эпизод* **do**
  $e(s) \leftarrow 0$ для всех s;
  Инициализировать S;
  for *каждый шаг t пока S не терминальное* **do**
    Выбрать $A \sim \pi(\cdot \mid S)$; наблюдать R, S';
    $\delta \leftarrow R + \gamma V(S') - V(S)$;
    $e(S) \leftarrow e(S) + 1$ (накапливающий след);
    for *всех $s \in \mathcal{S}$* **do**
      $V(s) \leftarrow V(s) + \alpha\,\delta\,e(s)$;
      $e(s) \leftarrow \gamma\lambda\,e(s)$;
    $S \leftarrow S'$;

На практике обновлять нужно только состояния с ненулевыми следами, поэтому внутренний цикл по всем s обычно заменяется поддержкой множества активных следов. Алгоритм требует $O(|\mathcal{S}|)$ памяти на эпизод — одно значение следа на состояние.

6.9.5 Эквивалентность взглядов вперёд и назад

Следующая теорема является краеугольным камнем теории следов приемлемости.

Теорема 6.7 (Эквивалентность взглядов вперёд и назад для TD(λ))**.** *Для автономной (обновляемой в конце эпизода) версии TD(λ) с постоянным шагом α сумма всех обновлений $V(s)$ за эпизод одинакова при взгляде вперёд (верхний индекс fw) и при взгляде назад (верхний индекс bw). То есть, определяя $\Delta_t^{\mathrm{fw}} V(s) = \alpha(G_t^\lambda - V(S_t))\mathbb{1}\{S_t = s\}$ (вперёд) и $\Delta_t^{\mathrm{bw}} V(s) = \alpha\delta_t e_t(s)$ (назад):*

$$\sum_{t=0}^{T-1} \Delta_t^{\mathrm{bw}} V(s) = \sum_{t=0}^{T-1} \Delta_t^{\mathrm{fw}} V(s), \quad \forall s \in \mathcal{S}. \tag{6.18}$$

Набросок доказательства. Ключевое тождество:

$$G_t^\lambda - V(S_t) = \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \delta_k,$$

которое выражает ошибку λ-отдачи как дисконтированную сумму будущих ошибок TD. Для его проверки следует раскрыть G_t^λ по определению (6.14), записать каждую $G_t^{(n)}$ как телескопическую сумму ошибок TD (используя рекурсивно $G_t^{(n)} = \delta_t + \gamma G_{t+1}^{(n-1)}$), затем собрать слагаемые. Полный алгебраический вывод можно найти у Саттона и Барто (2018, глава 12).

Установив это тождество, суммирование $\alpha\delta_t e_t(s)$ по t и перестановка порядка суммирования даёт $\sum_t \alpha(G_t^\lambda - V(S_t))\mathbb{1}\{S_t = s\}$, что завершает доказательство. □

Теорема 6.7 подтверждает, что онлайновый инкрементный взгляд назад вычисляет в точности те же обновления, что и автономный взгляд вперёд. Это глубокий результат: он означает, что нам не нужно выбирать между концептуальной ясностью (вперёд) и вычислительной эффективностью (назад).

6.9.6 Практические соображения

Замечание 6.1 (Затухание следа и эффективный горизонт). Произведение $\gamma\lambda$ управляет эффективной скоростью затухания следов приемлемости. При $\gamma\lambda = 0.9$ состояние, посещённое k шагов назад, имеет след 0.9^k, поэтому состояния, посещавшиеся более ~ 20 шагов назад, имеют пренебрежимо малый след. На практике λ часто настраивают совместно с γ для установки желаемого эффективного горизонта.

Замечание 6.2 (Вычислительная стоимость). Наивная реализация алгоритма 16 обновляет все $|\mathcal{S}|$ следов на каждом шаге, что составляет $O(|\mathcal{S}|)$ за шаг. С табличными представлениями это осуществимо для умеренных пространств состояний. При аппроксимации функций (глава 7) след становится вектором той же размерности, что и вектор параметров, и стоимость за шаг составляет $O(d)$, где d — число параметров.

Замечание 6.3 (SARSA(λ) и Q(λ)). Следы приемлемости естественно распространяются на управление. **SARSA(λ)** поддерживает след $e_t(s, a)$ по парам состояние-действие и использует ошибку TD SARSA $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$ для обновления $Q(s, a) \leftarrow Q(s, a) + \alpha\delta_t e_t(s, a)$. Аналогично, **Q($\lambda$)** применяет ошибку TD Q-learning со следами. SARSA(λ) особенно популярен на практике, поскольку сохраняет характер SARSA на текущей стратегии, включая многошаговое присвоение кредита следов приемлемости.

6.9.7 Разобранный пример: случайное блуждание с TD(λ)

Пример 6.4 (Влияние λ на скорость обучения)**.** Вернёмся к пятисостоянию случайного блуждания из примера 6.1. Запустив TD(λ) с различными значениями λ для фиксированного числа эпизодов и измерив СКО относительно истинных значений, наблюдаем:

- $\lambda = 0$ (TD(0)): медленное обучение; сигнал вознаграждения требует многих эпизодов для распространения в далёкие состояния.
- $\lambda = 0.5$: следы обеспечивают распространение вознаграждения из правого терминала назад к состояниям, посещённым в течение последних нескольких шагов в том же эпизоде, существенно ускоряя обучение.
- $\lambda = 0.9$: быстрое начальное обучение, но слегка более зашумлённое, поскольку следы придают значительный вес далёким состояниям, которые могли не вносить вклада в итоговое вознаграждение.
- $\lambda = 1$ (МК): наибольшая дисперсия за эпизод, но при хорошо настроенном шаге суммарное обучение по эпизодам может быть конкурентоспособным.

В экспериментах Саттона и Барто промежуточные значения $\lambda \approx 0.7$–0.9 часто достигают наименьшего СКО для этой задачи. Оптимальное λ зависит от шага и структуры задачи.

6.10 Обобщённая оценка преимущества (GAE)

Обобщённая оценка преимущества (Шульман и др., 2016) — современный метод вычисления оценок преимущества с низкой дисперсией для использования в методах градиента стратегии (глава 8). Он построен непосредственно на механизме TD(λ) и представляет собой убедительное применение теории, разработанной в этой главе.

6.10.1 Функция преимущества

Определение 6.6 (Функция преимущества)**. Функция преимущества** стратегии π равна:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \tag{6.19}$$

Преимущество $A^\pi(s, a)$ измеряет, насколько действие a лучше среднего действия при стратегии π в состоянии s. По определению:

- $A^\pi(s, a) > 0$: действие a лучше среднего; стратегия должна увеличить его вероятность.
- $A^\pi(s, a) < 0$: действие a хуже среднего; уменьшить его вероятность.
- $A^\pi(s, a) = 0$: действие a работает ровно на уровне среднего.

Заметим, что $\mathbb{E}_{a\sim\pi(\cdot|s)}[A^\pi(s, a)] = 0$ для всех s, то есть преимущество центрировано.

Почему преимущество важно для градиентов стратегии. Теорема о градиенте стратегии (глава 8) утверждает, что градиент ожидаемой отдачи по параметрам стратегии θ равен

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi\big[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \cdot \Psi_t\big],$$

где Ψ_t может быть отдачей G_t, функцией ценности действия $Q^\pi(S_t, A_t)$ или — что особенно важно — преимуществом $A^\pi(S_t, A_t)$. Использование преимущества вместо сырой отдачи или функции ценности действия резко снижает дисперсию: базовая линия $V^\pi(S_t)$ вычитается, устраняя всю вариацию, обусловленную состоянием (на которое стратегия не может влиять).

6.10.2 n-шаговые оценки преимущества

На практике V^π и A^π неизвестны. При наличии обученного критика $V_\phi \approx V^\pi$ естественным оценщиком одношагового преимущества является ошибка TD:

$$\hat{A}_t^{(1)} = \delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t). \tag{6.20}$$

В более общем виде n-шаговая оценка преимущества:

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n V_\phi(S_{t+n}) - V_\phi(S_t). \tag{6.21}$$

Заметим, что $\hat{A}_t^{(n)} = G_t^{(n)} - V_\phi(S_t)$, где $G_t^{(n)}$ — n-шаговая отдача (6.12).

6.10.3 GAE: преимущество с весами по λ

Определение 6.7 (Обобщённая оценка преимущества)**. Обобщённая оценка преимущества** $\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}$ — это экспоненциально взвешенное среднее n-шаговых оценок преимущества:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}, \tag{6.22}$$

где $\delta_{t+l} = R_{t+l+1} + \gamma V_\phi(S_{t+l+1}) - V_\phi(S_{t+l})$ — ошибка TD в момент $t + l$.

Утверждение 6.8. *GAE является λ-взвешенным средним n-шаговых оценок преимущества:*

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)}. \tag{6.23}$$

Доказательство. Используем ключевое тождество $\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} (\gamma)^l \delta_{t+l}$ (телескопическое), которое следует из

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n V_\phi(S_{t+n}) - V_\phi(S_t) = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}.$$

Тогда

$$\begin{aligned}
(1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)} &= (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{l=0}^{n-1} \gamma^l \delta_{t+l} \\
&= (1-\lambda) \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} \sum_{n=l+1}^{\infty} \lambda^{n-1} \\
&= (1-\lambda) \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} \cdot \frac{\lambda^l}{1-\lambda} \\
&= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}.
\end{aligned}$$

□

6.10.4 Частные случаи и компромисс смещение–дисперсия

- $\lambda = 0$: $\hat{A}_t^{\mathrm{GAE}(\gamma,0)} = \delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$ — одношаговое преимущество TD с минимальной дисперсией и максимальным смещением (полная зависимость от V_ϕ).
- $\lambda = 1$: $\hat{A}_t^{\mathrm{GAE}(\gamma,1)} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = G_t - V_\phi(S_t)$ — оценка преимущества Монте-Карло с минимальным смещением и максимальной дисперсией.
- Промежуточные λ: экспоненциально убывающие веса усредняют краткосрочные (низкая дисперсия, высокое смещение) и долгосрочные (высокая дисперсия, низкое смещение) оценки.

6.10.5 Связь с TD(λ)

Структура GAE (6.22) идентична механизму λ-отдачи. Фактически, если рассматривать $V_\phi(S_t)$ как базовую линию, то

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = G_t^\lambda - V_\phi(S_t),$$

где G_t^λ — λ-отдача, вычисленная с критиком V_ϕ. Таким образом, GAE — это просто цель TD(λ) минус текущая оценка ценности. Взгляд на следы приемлемости предоставляет эффективный способ вычисления GAE онлайн: поддерживать след e_t и накапливать слагаемые $e_t\,\delta_t$.

На практике (например, в алгоритмах PPO и TRPO) GAE вычисляется обратным проходом по собранному роллауту длины T:

$$\begin{aligned} A_T &= 0, \\ A_t &= \delta_t + \gamma\lambda\, A_{t+1}, \quad t = T-1, T-2, \ldots, 0, \end{aligned}$$

что является линейным рекуррентным соотношением, вычисляемым за время $O(T)$.

Пример 6.5 (Вычисление GAE для короткого роллаута)**.** Предположим $\gamma = 0.99$, $\lambda = 0.95$, и роллаут из 4 шагов даёт ошибки TD $\delta_0 = 0.5,\ \delta_1 = -0.3,\ \delta_2 = 0.8,\ \delta_3 = 0.1$ (при $A_4 = 0$). Обратное вычисление даёт:

$$\begin{aligned} A_3 &= 0.1 + 0 = 0.1, \\ A_2 &= 0.8 + 0.99 \times 0.95 \times 0.1 = 0.8 + 0.094 = 0.894, \\ A_1 &= -0.3 + 0.99 \times 0.95 \times 0.894 = -0.3 + 0.840 = 0.540, \\ A_0 &= 0.5 + 0.99 \times 0.95 \times 0.540 = 0.5 + 0.508 = 1.008. \end{aligned}$$

Оценка преимущества $A_0 = 1.008$ для первого шага учитывает не только $\delta_0 = 0.5$, но и нижестоящий положительный сигнал $\delta_2 = 0.8$, соответствующим образом дисконтированный на $(\gamma\lambda)^2$.

6.11 Анализ смещения, дисперсии и сходимости

6.11.1 Формальный анализ смещения

Цель TD(0) $R_{t+1} + \gamma V_t(S_{t+1})$ смещена, когда $V_t \neq V^\pi$. Разложим смещение следующим образом. Пусть V_t обозначает оценку ценности на итерации t, и пусть $b_t(s) = V_t(s) - V^\pi(s)$ — ошибка приближения на итерации t. Тогда

$$\mathbb{E}_\pi[R_{t+1} + \gamma V_t(S_{t+1}) \mid S_t = s] = V^\pi(s) + \gamma\, \mathbb{E}_\pi[b_t(S_{t+1}) \mid S_t = s].$$

Таким образом, смещение цели TD равно γ, умноженному на ожидаемую ошибку приближения в следующем состоянии. По величине это строго меньше ошибки приближения в s (масштабированной на $\gamma < 1$), что и является механизмом, благодаря которому TD-обновления притягиваются к V^π. По мере того как $V_t \to V^\pi$, смещение исчезает, что согласуется с теоремой 6.1.

Монте-Карло, напротив, использует несмещённую цель G_t, поэтому его смещение равно нулю независимо от V_t.

6.11.2 Анализ дисперсии

Дисперсия цели TD(0) $R_{t+1} + \gamma V(S_{t+1})$ ограничена:

$$\mathrm{Var}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s] \leq \sigma_R^2 + \gamma^2\, \mathrm{Var}[V(S_{t+1}) \mid S_t = s],$$

где σ_R^2 — дисперсия вознаграждения. Когда V — гладкая функция (например, нейронная сеть), $\mathrm{Var}[V(S_{t+1})]$ мало. Для сравнения, отдача МК $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ накапливает дисперсию с каждого шага. Поскольку вознаграждения на разных шагах, вообще говоря, коррелированы через общую траекторию, полное выражение включает члены с ковариациями:

$$\begin{aligned}\mathrm{Var}[G_t \mid S_t = s] &= \sum_{k=0}^{\infty} \gamma^{2k} \,\mathrm{Var}[R_{t+k+1} \mid S_t = s] \\ &\quad + 2 \sum_{0 \le j < k} \gamma^{j+k} \,\mathrm{Cov}[R_{t+j+1}, R_{t+k+1} \mid S_t = s] \\ &\gg \mathrm{Var}[R_{t+1} + \gamma V(S_{t+1})].\end{aligned}$$

Этот разрыв в дисперсии растёт с длиной эпизода и коэффициентом дисконтирования, делая методы TD существенно более выборочно эффективными, чем МК, во многих практических условиях.

6.11.3 Разложение среднеквадратичной ошибки

Для любого оценщика $\hat{y}$ цели $y^* = \mathbb{E}[Y]$ среднеквадратичная ошибка раскладывается как:

$$\mathrm{МКО}(\hat{y}) = \underbrace{\big(\mathbb{E}[\hat{y}] - y^*\big)^2}_{\text{Смещение}^2} + \underbrace{\mathrm{Var}[\hat{y}]}_{\text{Дисперсия}} . \tag{6.24}$$

Это разложение применяется к цели TD (смещённой, с низкой дисперсией) и отдаче МК (несмещённой, с высокой дисперсией). Минимизация МКО требует баланса обоих слагаемых, что именно и делает TD(λ) настройкой λ.

6.11.4 Сравнительная таблица

Таблица 6.2: Свойства смещения, дисперсии и сходимости методов предсказания.

Метод	**Смещение**	**Дисперсия**	**Онлайн?**	**Сходимость**
Монте-Карло	Нулевое	Высокая (растёт с T)	Нет	Да (к V^π)
TD(0)	Высокое нач., $\to 0$	Низкая	Да	Да (Теор. 6.2)
n-шаговое TD	Среднее	Средняя	Полуонлайн	Да
TD(λ)	Настраиваемое (λ)	Настраиваемая (λ)	Да	Да

6.11.5 Смертельная триада

Табличные TD-методы гарантированно сходятся при мягких условиях. Однако сочетание трёх ингредиентов ведёт к потенциальной расходимости:

1. **Аппроксимация функций** (например, линейные или нейросетевые представления V).
2. **Бутстрэппинг** (TD-обновления с использованием текущей оценки в качестве цели).
3. **Обучение на другой стратегии** (обучение по данным, сгенерированным другой стратегией).

Это сочетание — известное как *смертельная триада* — может привести к расходимости оценок ценности до бесконечности даже в простых линейных примерах (контрпример Бэрда, 1995). Монте-Карло избегает триады за счёт отказа от бутстрэппинга; методы на текущей стратегии избегают её, оставаясь на текущей стратегии; некоторые алгоритмы (например, методы градиентного TD, emphatic TD) решают её посредством тщательного алгоритмического проектирования. Эти вопросы подробно рассматриваются в главе 7.

6.11.6 Сходимость при расписании Роббинса–Монро

Условия Роббинса–Монро (6.7) необходимы для почти наверное сходимости алгоритмов стохастической аппроксимации. Их интуиция такова:

- Условие расходимости $\sum_t \alpha_t = \infty$ гарантирует, что алгоритм может исправить любую начальную ошибку, какой бы большой она ни была.

- Условие квадратичной суммируемости $\sum_t \alpha_t^2 < \infty$ гарантирует подавление накопленного шума: дисперсия суммы $\sum_t \alpha_t \epsilon_t$ (где ϵ_t — шум) ограничена величиной $\sigma^2 \sum_t \alpha_t^2 < \infty$.

Гармоническое расписание $\alpha_t = 1/t$ удовлетворяет обоим условиям. Постоянное расписание $\alpha_t = \alpha$ не удовлетворяет ни одному из них — что приводит к устойчивым флуктуациям — но часто предпочитается в нестационарных задачах или когда нужна быстрая адаптация к сдвигам распределения.

6.12 Итоги

Обучение с временной разностью объединяет традиции Монте-Карло и динамического программирования. Из единственной идеи бутстрэппинга — использования текущих оценок ценности в качестве целей — возникает целое семейство эффективных, онлайновых, безмодельных алгоритмов. В таблице 6.3 приводится полный обзор всех методов, введённых в этой главе.

Таблица 6.3: Сводка методов временной разности, введённых в этой главе.

Алгоритм	**Тип**	**Стр-гия**	**Цель**	**Онлайн**	**Кредит**
TD(0)	Предск.	Фикс. π	$R + \gamma V(S')$	Да	1
n-шаг. TD	Предск.	Фикс. π	$G^{(n)}$	Полу	n
TD(λ)	Предск.	Фикс. π	G^{λ}	Да	Следы
SARSA	Упр-е	Текущ.	$R + \gamma Q(S', A')$	Да	1
SARSA(λ)	Упр-е	Текущ.	G_Q^{λ}	Да	Следы
Q-learning	Упр-е	Другой	$R + \gamma \max_{a'} Q(S', a')$	Да	1
Expected SARSA	Упр-е	Обе	$R + \gamma \bar{Q}(S')$	Да	1
Double Q-learning	Упр-е	Другой	Раздел. макс.	Да	1

Ключевые выводы этой главы:

- Ошибка TD $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ — фундаментальный кредитный сигнал в обучении с подкреплением; при сходимости её среднее равно нулю (теорема 6.1).
- Методы TD обменивают смещение на дисперсию: бутстрэппинг вводит смещение, но снижает дисперсию по сравнению с Монте-Карло.
- SARSA работает на текущей стратегии и безопасен при исследовании; Q-learning работает на другой стратегии и напрямую обучает оптимальной стратегии.
- TD(λ) со следами приемлемости элегантно заполняет разрыв между одношаговым TD и Монте-Карло, при этом взгляды вперёд и назад доказуемо эквивалентны.
- GAE применяет механизм λ-отдачи к оценке преимущества в методах градиента стратегии, обеспечивая чёткий компромисс смещение–дисперсия, параметризованный λ.
- Сходимость в табличном случае хорошо изучена; сочетание аппроксимации функций, бутстрэппинга и данных с другой стратегии (смертельная триада) может вызвать расходимость.

Упражнения

Упражнение 6.1. Предположим $V(S_t) = 3$, $R_{t+1} = 1$, $\gamma = 0.9$, $V(S_{t+1}) = 4$.

(a) Вычислите ошибку TD δ_t.

(b) Примените одно обновление TD(0) с шагом $\alpha = 0.1$. Каково новое значение $V(S_t)$?

(c) Интерпретируйте знак δ_t: было ли состояние переоценено или недооценено?

Упражнение 6.2. Пусть $V = V^{\pi}$ — точная функция ценности. Дайте подробное доказательство того, что $\mathbb{E}_{\pi}[\delta_t \mid S_t = s] = 0$ для всех s, явно указав, на каком шаге используется уравнение Беллмана для V^{π}. Затем объясните, почему это свойство *не* выполняется при $V \neq V^{\pi}$.

Упражнение 6.3. Рассмотрите среду «хождение по скалам» из примера 6.2.

(a) Точно объясните, почему SARSA с $\varepsilon = 0.1$ предпочитает путь вдали от обрыва, хотя путь вдоль края обрыва короче.

(b) Q-learning с $\varepsilon = 0.1$ в итоге обучается маршруту вдоль края обрыва. Если уменьшить ε до 0.01, как это повлияет на (i) маршрут, которому обучится Q-learning, и (ii) маршрут, которому обучится SARSA?

(c) Какому алгоритму вы бы отдали предпочтение на этапе тестирования при $\varepsilon = 0$, и почему?

Упражнение 6.4. Рассмотрите МПР с двумя действиями, в котором истинные значения $Q^*(s, a_1) = Q^*(s, a_2) = 0$. Пусть $\hat{Q}(s, a_i) \sim \mathcal{N}(0, \sigma^2)$ — независимые шумные оценки.

(a) Покажите, что $\mathbb{E}[\max(\hat{Q}(s, a_1), \hat{Q}(s, a_2))] > 0$. (Подсказка: используйте формулу для ожидаемого максимума двух стандартных нормальных случайных величин и масштабируйте результат с помощью σ.)

(b) Как это смещение растёт с увеличением числа действий?

(c) Объясните, как Double Q-learning (алгоритм 15) устраняет это смещение.

Упражнение 6.5. Для случайного блуждания с пятью состояниями из примера 6.1 инициализируйте все $V(s) = 0.5$ и предположите, что наблюдается траектория $B \to C \to D \to E \to R$ с вознаграждениями $(0, 0, 0, 1)$ и $\gamma = 1$.

(a) Вычислите цель TD(0) ($n = 1$) для состояния B.

(b) Вычислите 2-шаговую отдачу $G_0^{(2)}$ для состояния B.

(c) Вычислите полную отдачу Монте-Карло G_0 для состояния B.

(d) Для λ-отдачи при $\lambda = 0.5$ вычислите G_0^λ для состояния B (включая все n-шаговые слагаемые).

Упражнение 6.6. В табличной задаче траектория посещает состояния A, B, A, C в моменты времени $t = 0, 1, 2, 3$. Используя накапливающие следы приемлемости (6.15) при $\gamma = 0.9$ и $\lambda = 0.8$, вычислите след $e_t(s)$ для каждого состояния $s \in \{A, B, C\}$ в каждый момент времени $t = 0, 1, 2, 3$. Проверьте, что при $t = 3$ след $e_3(A)$ больше, чем $e_3(B)$, поскольку состояние A посещалось дважды и более недавно.

Упражнение 6.7. Сформулируйте теорему об эквивалентности взглядов вперёд и назад (теорема 6.7) своими словами. Затем используйте тождество $G_t^\lambda - V(S_t) = \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \delta_k$, чтобы проверить эквивалентность для конкретного двухшагового эпизода: состояния S_0, S_1, S_2 (терминальное), вознаграждения R_1, R_2 и дисконт γ. Подтвердите, что $\sum_{t=0}^{1} \alpha\, \delta_t\, e_t(S_0)$ равно $\alpha(G_0^\lambda - V(S_0))$.

Упражнение 6.8. Пусть $\gamma = 0.99$, $\lambda = 0.9$, и траектория длины 5 даёт TD-ошибки $\delta_0 = 1$, $\delta_1 = 0$, $\delta_2 = -1$, $\delta_3 = 2$, $\delta_4 = 0$.

(a) Вычислите $\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}$ для $t = 0, 1, 2, 3$, используя обратную рекурсию из раздела 6.10.

(b) Какими были бы оценки при $\lambda = 0$? А при $\lambda = 1$?

(c) Проинтерпретируйте различие: при каком значении λ оценка преимущества в момент $t = 0$ сильнее всего отражает положительный сигнал $\delta_3 = 2$?

Упражнение 6.9. *Смертельная триада* обозначает сочетание бутстрэппинга, аппроксимации функций и обучения на другой стратегии.

(a) Объясните, почему удаление любого одного из этих трёх элементов восстанавливает гарантии сходимости.

(b) Известно, что Монте-Карло с аппроксимацией функций не расходится. Какого элемента триады здесь не хватает?

(c) TD на текущей стратегии с линейной аппроксимацией функций сходится к окрестности V^π. Какого элемента триады здесь не хватает?

(d) Предложите один вариант проектирования алгоритма, который сохраняет все три элемента и при этом пытается восстановить сходимость.

Упражнение 6.10. Рассмотрите расписание шага $\alpha_t = c/(t + t_0)$ для констант $c > 0$ и $t_0 > 0$.

(a) Покажите, что это расписание удовлетворяет условиям Роббинса–Монро (6.7).

(b) Покажите, что постоянное расписание $\alpha_t = \alpha$ *не* удовлетворяет второму условию.

(c) В нестационарной среде, где V^π меняется со временем, объясните, почему постоянный шаг часто *предпочтителен*, несмотря на отсутствие второго условия Роббинса–Монро.

Упражнение 6.11. Алгебраически покажите, что когда стратегия π в Expected SARSA (6.11) является детерминированной жадной стратегией относительно Q—то есть $\pi(a \mid s) = \mathbb{1}\{a = \operatorname{argmax}_{a'} Q(s, a')\}$—обновление Expected SARSA в точности сводится к обновлению Q-learning (6.10).

Упражнение 6.12. Заполните следующую таблицу для простой эпизодической задачи со средней длиной эпизода $T = 100$ и дисперсией вознаграждения $\sigma^2 = 1$ на шаг. Предположите $\gamma = 0.99$. Оцените порядок величины дисперсии цели обновления для: (a) TD(0), (b) 10-шагового TD, (c) Монте-Карло. Используйте приближение, что дисперсия n-шаговой отдачи равна $\sum_{k=0}^{n-1} \gamma^{2k}\sigma^2 + \gamma^{2n} \operatorname{Var}[V(S_n)]$, и что $\operatorname{Var}[V(S_n)] \approx 0$ (идеальный критик). Обсудите, как меняются члены смещения при увеличении n.

Часть II

Аппроксимация функций и глубокое RL

Глава 7

Аппроксимация функций ценности и глубокое обучение с подкреплением

Сила нейронных сетей не в том, что они запоминают; она в том, что они обобщают. В обучении с подкреплением это различие разделяет методы, которые масштабируются, от тех, которые не масштабируются.

Предыдущие шесть глав создали строгую основу для обучения с подкреплением, используя табличные представления. В главе 3 мы формализовали фреймворк марковских процессов принятия решений и вывели уравнения Беллмана; в главе 4 показали, как динамическое программирование решает их точно при наличии модели; в главах 5 и 6 разработали методы Монте-Карло и временной разности, которые обучаются из опыта без модели. Эти алгоритмы обладают хорошо понятными гарантиями сходимости и чёткой математической структурой. Однако они все разделяют одно критическое допущение: что мы можем хранить и обновлять отдельное численное значение для каждого состояния — или каждой пары состояние–действие — в задаче.

Это допущение нарушается почти мгновенно в любом реалистичном приложении. Игра Atari представляет примерно $10^{120\,000}$ различных конфигураций экрана; роботическая рука, двигающаяся в непрерывном пространстве суставов, имеет несчётно бесконечное пространство состояний; языковая модель должна присваивать ценности каждой возможной последовательности токенов. Ни один компьютер, ни существующий, ни будущий, не может хранить таблицу такого размера. Даже если бы хранилище было неограниченным, обобщающие свойства табличных методов были бы плохими: обучение тому, что одно игровое состояние опасно, не даёт никакой информации о визуально схожем состоянии.

Решение — *аппроксимация функций*: вместо хранения одного числа на состояние мы обучаем параметризованную функцию $V_{\mathbf{w}}(s) \approx V^{\pi}(s)$, чьё число параметров d значительно меньше $|\mathcal{S}|$. Один набор параметров одновременно кодирует информацию обо всех состояниях, и обновление параметров после посещения одного состояния автоматически изменяет предсказанные ценности близких состояний. Это механизм обобщения.

Эта глава систематически развивает аппроксимацию функций — от первых принципов до архитектур глубоких нейронных сетей, лежащих в основе современного обучения с подкреплением. Раздел 7.1 количественно описывает проблему масштабируемости. Раздел 7.2 вводит линейную аппроксимацию функций — простейшую постановку, в которой сходимость можно доказать строго. Раздел 7.3 анализирует проецированное уравнение Беллмана, характеризующее неподвижную точку TD-методов в пространстве аппроксимации функций. Раздел 7.4 исследует методы полуградиента и объясняет, почему они не являются истинными методами градиентного спуска. Раздел 7.5 рассматривает *смертельную триаду*: сочетание аппроксимации функций, бутстрэппинга и данных с другой стратегии, способное вызвать расходимость даже при линейных аппроксиматорах. Раздел 7.6 представляет Deep Q-Networks (DQN) — первый алгоритм, продемонстрировавший производительность уровня человека во многих разнообразных задачах, стабилизировав глубокое ОП двумя ключевыми инженерными новшествами. Раздел 7.7 рассматривает важные алгоритмические улучшения, построенные на фундаменте DQN. Раздел 7.8 обозревает, что теория сходимости может и чего не может сказать о нелинейной аппроксимации функций. Наконец, раздел 7.9 обсуждает практические соображения, соединяющие теорию и инженерию.

7.1 Почему табличные методы терпят неудачу

7.1.1 Проклятие размерности

Чтобы понять необходимость аппроксимации функций, нужно сначала количественно оценить, как резко растёт пространство состояний с усложнением задачи. В табличном МПР с $|\mathcal{S}|$ состояниями и $|\mathcal{A}|$ действиями представление функции ценности действия $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ требует хранения $|\mathcal{S}| \cdot |\mathcal{A}|$ чисел с плавающей точкой. Для небольшого клеточного мира со 100 клетками и 4 действиями это 400 чисел — тривиально. Но размер пространства состояний не является линейной функцией входной размерности задачи; он экспоненциален.

Определение 7.1 (Проклятие размерности)**.** Пусть состояние описывается n признаками, каждый принимает k дискретных значений. Общее число состояний равно k^n. Следовательно, затраты памяти и выборочная сложность табличных методов растут *экспоненциально* по числу признаков n. Это явление называется **проклятием размерности**.

Три конкретных примера делают это наглядным.

Пример 7.1 (Видеоигры Atari)**.** Atari 2600 производит кадры 210×160 пикселей с 128 цветами на пиксель. Мних и др. (2015) предобрабатывают кадры до 84×84 градаций серого и укладывают четыре последовательных кадра для захвата движения, давая тензор наблюдения с $84 \times 84 \times 4 = 28\,224$ значениями, каждое — целое число в $\{0, \ldots, 255\}$. Число различных наблюдений составляет не более $256^{28\,224}$ — число с примерно 68 000 десятичными цифрами. Табличная Q-функция на этом пространстве не просто непрактична; она концептуально абсурдна. Тем не менее DQN, используя свёрточную нейронную сеть с ~ 1.5 млн параметров, достиг сверхчеловеческой производительности во многих из этих игр. Сеть преуспевает именно потому, что обобщает по визуально схожим кадрам.

Пример 7.2 (Непрерывная робототехника)**.** Симулированный гуманоидный робот (например, среда MuJoCo Humanoid-v4) имеет вектор состояния $s \in \mathbb{R}^{376}$, включающий угловые положения суставов, угловые скорости и контактные силы. Это пространство состояний не просто велико — оно непрерывно и несчётно бесконечно. Никакая схема дискретизации не может свести его к управляемой таблице без серьёзных ошибок приближения. Более того, пространство действий — $\mathbb{R}^{17}$, поэтому операция argmax, центральная для Q-learning, становится задачей непрерывной оптимизации на каждом шаге.

Пример 7.3 (Язык и текст)**.** В приложениях ОП к дообучению языковых моделей (подробнее в главе 14) состояние — это текущий префикс токенов — вся история диалога до текущего шага. При словарном запасе размером 50 000 и контекстном окне в 2 048 токенов число возможных префиксов астрономически велико. Более того, семантика двух префиксов, отличающихся одним словом, может быть почти идентичной, поэтому любое полезное представление должно достигать семантического обобщения, а не только синтаксического.

7.1.2 Аргумент обобщения

Помимо чисто проблемы хранения, табличные методы имеют фундаментальную статистическую неэффективность: каждое состояние рассматривается как полностью отдельный объект. Обучение тому, что ценность состояния s высока, не даёт никакой информации о состоянии s', даже если s и s' визуально идентичны за исключением положения одного пикселя. Говоря языком статистики, табличные методы обладают нулевым *обобщением*: каждое значение должно оцениваться независимо, по переходам, посещающим конкретное состояние.

В больших пространствах состояний большинство состояний посещается не более одного раза или вообще не посещается в ходе разумного обучающего прогона. Табличные методы присваивают таким состояниям их начальные значения (часто ноль) на неопределённый срок. Аппроксимация функций устраняет это, разделяя статистическую мощь между состояниями: посещение состояния s неявно обновляет оценочные ценности всех состояний s', для которых $\varphi(s')$ похоже на $\varphi(s)$, где φ обозначает признаковое представление. Эта *неявная регуляризация* является основной пользой параметрических функций ценности.

7.2 Линейная аппроксимация функций ценности

7.2.1 Признаковые векторы и линейный анзац

Простейшее параметрическое приближение представляет функцию ценности как линейную комбинацию фиксированных признаков.

Определение 7.2 (Линейная аппроксимация функций ценности)**.** Пусть $\varphi : \mathcal{S} \to \mathbb{R}^d$ — **признаковое отображение**, назначающее каждому состоянию признаковый вектор. **Линейная аппроксимация функций ценности** имеет вид:

$$V_{\mathbf{w}}(s) = \mathbf{w}^\top \varphi(s) = \sum_{i=1}^{d} w_i \varphi_i(s), \tag{7.1}$$

где $\mathbf{w} \in \mathbb{R}^d$ — вектор весов. Для функций ценности действия используются признаки $\varphi(s, a) \in \mathbb{R}^d$ и записывается $Q_{\mathbf{w}}(s, a) = \mathbf{w}^\top \varphi(s, a)$.

Линейный анзац привлекателен по трём причинам. Во-первых, это простейшая возможная параметризация: приближение однозначно определяется d вещественными числами вне зависимости от $|\mathcal{S}|$. Во-вторых, многие интересные признаковые отображения могут быть спроектированы вручную — тайловые кодирования, радиальные базисные функции, полиномиальные базисы, фурье-признаки — каждое кодирует конкретные априорные убеждения о структуре функции ценности. В-третьих, и самое главное, линейная аппроксимация функций является постановкой, в которой сходимость TD можно установить с математической строгостью, как будет показано в разделе 7.3.

Пример 7.4 (Полиномиальные признаки для одномерного коридора)**.** Рассмотрим коридор из $N = 1\,000$ дискретных позиций. В табличном случае V^π требует $N = 1\,000$ параметров. Если, однако, истинная функция ценности гладкая — что вероятно, если вознаграждение сосредоточено на одном конце и коэффициент дисконтирования умеренный — мы можем аппроксимировать её полиномиальным базисом:

$$\varphi(s) = \big(1,\ s/N,\ (s/N)^2,\ (s/N)^3\big)^\top \in \mathbb{R}^4.$$

Это сокращает число параметров с $1\,000$ до 4, ценой невозможности точного представления быстро меняющихся функций ценности. Ошибка приближения, возникающая при ограничении данным классом функций, называется **ошибкой представления**, и она неизбежна всякий раз, когда истинная функция ценности лежит вне семейства приближений.

7.2.2 Тайловое кодирование

Особенно полезным признаковым отображением для непрерывных пространств состояний является *тайловое кодирование*, разработанное Саттоном (1996). Пространство состояний $\mathbb{R}^n$ покрывается несколькими перекрывающимися сетками, называемыми *тайлингами*. Для данного состояния s ровно один тайл в каждом тайлинге активен, давая двоичный признаковый вектор, ненулевые элементы которого указывают, какой тайл активен в каждом тайлинге. Если имеется m тайлингов, каждый разбивающий пространство на k тайлов, признаковый вектор имеет размерность mk и содержит ровно m единиц. Результирующая линейная аппроксимация эквивалентна таблице значений тайлов, но перекрывающаяся структура тайлингов обеспечивает гладкую интерполяцию между близкими состояниями.

7.2.3 Линейное TD(0)

Вспомним из главы 6, что TD(0) для оценивания стратегии поддерживает оценку ценности и обновляет её после каждого перехода $(S_t, A_t, R_{t+1}, S_{t+1})$, используя ошибку TD:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

Когда V параметризована как $V_{\mathbf{w}}(s) = \mathbf{w}^\top \varphi(s)$, естественным расширением является обновление вектора весов в направлении, уменьшающем квадрат ошибки TD. Это даёт обновление **полуградиентного TD(0)** (подробно анализируется в разделе 7.4):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta_t\, \varphi(S_t), \tag{7.2}$$

где $\delta_t = R_{t+1} + \gamma \mathbf{w}^\top \varphi(S_{t+1}) - \mathbf{w}^\top \varphi(S_t)$. Заметим, что в линейном случае $\nabla_{\mathbf{w}} V_{\mathbf{w}}(S_t) = \varphi(S_t)$, поэтому (7.2) является шагом полуградиента, применённым к $V_{\mathbf{w}}$.

Критическое свойство этого обновления состоит в том, что изменение $\mathbf{w}$ затрагивает *все состояния одновременно*: предсказание ценности в состоянии s' изменяется на $\alpha \delta_t\, \varphi(s') \cdot \varphi(S_t)$, пропорционально схожести s' с S_t по признаковому скалярному произведению. Это и есть механизм обобщения в линейной аппроксимации функций.

Теорема 7.1 (Сходимость линейного TD(0) при выборке на текущей стратегии)**.** *Пусть π — фиксированная стратегия и μ — её стационарное распределение по $\mathcal{S}$ (предполагается единственным и положительным везде). Предположим, что признаковая матрица $\Phi \in \mathbb{R}^{|\mathcal{S}| \times d}$ имеет полный ранг по столбцам.*

Если выборки (S_t, R_{t+1}, S_{t+1}) берутся на текущей стратегии (т.е. $S_t \sim \mu$ в установившемся режиме) и шаги удовлетворяют $\sum_t \alpha_t = \infty$ и $\sum_t \alpha_t^2 < \infty$, то итерации $\mathbf{w}_t$ линейного TD(0) сходятся почти наверное к единственному вектору весов $\mathbf{w}_{\mathrm{TD}}$, удовлетворяющему проецированному уравнению Беллмана $\Phi\mathbf{w}_{\mathrm{TD}} = \Pi_\mu T^\pi(\Phi\mathbf{w}_{\mathrm{TD}})$ (определяется в разделе 7.3).

Доказательство теоремы 7.1 опирается на теорию стохастической аппроксимации и свойства сжатия проецированного оператора Беллмана, которые мы сейчас развиваем.

7.3 Проецированное уравнение Беллмана

7.3.1 Геометрия аппроксимации функций

Чтобы понять, к чему в действительности сходится линейное TD(0), нужно думать геометрически. Линейный класс функций $\mathcal{F} = \{\Phi\mathbf{w} : \mathbf{w} \in \mathbb{R}^d\}$ — это d-мерное подпространство $\mathbb{R}^{|\mathcal{S}|}$ (отождествляя функции ценности с векторами, проиндексированными состояниями). Оператор Беллмана T^π отображает любую функцию ценности $v \in \mathbb{R}^{|\mathcal{S}|}$ в новую функцию $T^\pi v$, но в общем случае $T^\pi v \notin \mathcal{F}$ даже при $v \in \mathcal{F}$. Следовательно, TD-методы, работающие в $\mathcal{F}$, не могут следовать T^π напрямую.

Решение — *проецировать* $T^\pi v$ обратно в $\mathcal{F}$ после каждого шага Беллмана.

Определение 7.3 (μ-взвешенное скалярное произведение и проекция)**.** Пусть $\mu : \mathcal{S} \to (0, \infty)$ — положительное взвешивание (как правило, стационарное распределение π на текущей стратегии). Определим **μ-взвешенную норму**:

$$\|v\|_\mu^2 = \sum_{s \in \mathcal{S}} \mu(s)\, v(s)^2 = \mathbf{v}^\top D_\mu \mathbf{v}, \tag{7.3}$$

где $D_\mu = \mathrm{diag}(\mu)$. **Ортогональная проекция** v на $\mathcal{F}$ в этой норме:

$$\Pi_\mu v = \Phi\big(\Phi^\top D_\mu \Phi\big)^{-1} \Phi^\top D_\mu v = \operatorname*{argmin}_{f \in \mathcal{F}} \|f - v\|_\mu. \tag{7.4}$$

Матрица $\Pi_\mu = \Phi(\Phi^\top D_\mu \Phi)^{-1}\Phi^\top D_\mu$ является ортогональным проектором в μ-взвешенном скалярном произведении. Она идемпотентна ($\Pi_\mu^2 = \Pi_\mu$) и удовлетворяет $\|I - \Pi_\mu\|_\mu \le 1$, то есть проекция может только приблизить вектор к $\mathcal{F}$, но не удалить его.

Определение 7.4 (Проецированное уравнение Беллмана)**. Проецированное уравнение Беллмана** для стратегии π:

$$\hat{V} = \Pi_\mu T^\pi \hat{V}, \qquad \hat{V} \in \mathcal{F}. \tag{7.5}$$

Решение $\hat{V} = \Phi\mathbf{w}_{\mathrm{TD}}$ называется **неподвижной точкой TD**.

Проецированное уравнение Беллмана имеет ясную геометрическую интерпретацию: найти функцию в классе приближений $\mathcal{F}$, которая после одного шага Беллмана и повторного проецирования возвращает к себе. Это наилучшее, что мы можем ожидать, когда точная согласованность Беллмана недостижима в $\mathcal{F}$.

Пример 7.5 (Проецированное уравнение Беллмана для трёх состояний)**.** Предположим $|\mathcal{S}| = 3$ с равномерным взвешиванием $\mu = (1/3, 1/3, 1/3)$, и признаковая матрица:

$$\Phi = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \in \mathbb{R}^{3\times 2}.$$

Семейство приближений $\mathcal{F}$ имеет размерность 2, поэтому значения во всех трёх состояниях не могут задаваться независимо: $V(s_3) = V(s_1) + V(s_2)$ всегда. Проекция $\Pi_\mu v$ любого $v \in \mathbb{R}^3$ находит веса $\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} \|v - \Phi\mathbf{w}\|_\mu^2$, решая $\Phi^\top D_\mu \Phi \mathbf{w}^* = \Phi^\top D_\mu v$, что даёт:

$$\mathbf{w}^* = \tfrac{1}{2}\begin{pmatrix} v_1 + v_3 - v_2 \\ v_2 + v_3 - v_1 \end{pmatrix},$$

то есть проекция «делит вклад s_3» между двумя базисными направлениями.

7.3.2 Существование неподвижной точки TD

Теорема 7.2 (Существование и единственность неподвижной точки TD)**.** *При допущениях теоремы 7.1 проецированный оператор Беллмана $\Pi_\mu T^\pi : \mathcal{F} \to \mathcal{F}$ является сжатием в μ-норме с коэффициентом γ:*

$$\|\Pi_\mu T^\pi u - \Pi_\mu T^\pi v\|_\mu \le \gamma \|u - v\|_\mu \qquad \forall\, u, v \in \mathcal{F}. \tag{7.6}$$

Следовательно, проецированное уравнение Беллмана (7.5) имеет единственное решение $\hat{V} = \Phi \mathbf{w}_{\mathrm{TD}} \in \mathcal{F}$.

Доказательство. Сам оператор Беллмана T^π является сжатием в μ-норме с коэффициентом γ (это следует из свойства сжатия T^π в супремум-норме и эквивалентности этих норм при положительном μ). Формально, для любых u, v:

$$\|T^\pi u - T^\pi v\|_\mu \le \gamma \|u - v\|_\mu.$$

Поскольку Π_μ — неэкспансивная проекция ($\|\Pi_\mu x\|_\mu \le \|x\|_\mu$ для всех x), имеем:

$$\|\Pi_\mu T^\pi u - \Pi_\mu T^\pi v\|_\mu \le \|T^\pi u - T^\pi v\|_\mu \le \gamma \|u - v\|_\mu.$$

Поскольку $\gamma < 1$ и $\mathcal{F}$ является полным метрическим пространством (замкнутым подпространством $\mathbb{R}^{|\mathcal{S}|}$), теорема Банаха о неподвижной точке гарантирует единственную неподвижную точку. □

7.3.3 Ошибка приближения в неподвижной точке TD

Неподвижная точка TD $\hat{V}$ минимизирует определённую меру ошибки, но не минимизирует напрямую среднеквадратичную ошибку $\|\hat{V} - V^\pi\|_\mu^2$. Следующая классическая оценка (Цицикис и Ван Рой, 1997) количественно определяет, насколько неподвижная точка TD отклоняется от наилучшего возможного приближения.

Теорема 7.3 (Оценка ошибки приближения в неподвижной точке TD)**.** *Пусть $\hat{V}^* = \Pi_\mu V^\pi$ — наилучшее приближение V^π в $\mathcal{F}$. Тогда:*

$$\|\hat{V} - V^\pi\|_\mu \le \frac{1}{\sqrt{1 - \gamma^2}} \, \|\hat{V}^* - V^\pi\|_\mu. \tag{7.7}$$

Коэффициент $1/\sqrt{1-\gamma^2}$ — это «цена» бутстрэппинга: неподвижная точка TD может быть в $1/\sqrt{1-\gamma^2}$ раз хуже наилучшего приближения в классе. При $\gamma = 0.99$ этот коэффициент составляет примерно 7.1, иллюстрируя, что бутстрэппинг может существенно усилить ошибки приближения в задачах с длинными горизонтами времени.

7.4 Методы полуградиента

7.4.1 Истинный градиентный спуск для предсказания

Прежде чем изучать методы полуградиента, поучительно понять, как выглядел бы истинный градиентный спуск для задачи предсказания. Определим **среднеквадратичную ошибку ценности**:

$$\overline{\mathrm{VE}}(\mathbf{w}) = \|V^\pi - V_{\mathbf{w}}\|_\mu^2 = \sum_{s \in \mathcal{S}} \mu(s) \big[V^\pi(s) - V_{\mathbf{w}}(s)\big]^2, \tag{7.8}$$

где μ — распределение на текущей стратегии. Истинный градиент $\overline{\mathrm{VE}}$ по $\mathbf{w}$:

$$-\frac{1}{2} \nabla_{\mathbf{w}} \overline{\mathrm{VE}}(\mathbf{w}) = \sum_{s} \mu(s) \big[V^\pi(s) - V_{\mathbf{w}}(s)\big] \nabla_{\mathbf{w}} V_{\mathbf{w}}(s). \tag{7.9}$$

К сожалению, $V^\pi(s)$ неизвестна, поэтому мы не можем вычислить этот градиент напрямую. Методы Монте-Карло подставляют выборочную отдачу $G_t \approx V^\pi(S_t)$, давая обновление:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \big[G_t - V_{\mathbf{w}}(S_t)\big] \nabla_{\mathbf{w}} V_{\mathbf{w}}(S_t).$$

Это истинный стохастический шаг градиентного спуска, потому что $\mathbb{E}[G_t \mid S_t] = V^\pi(S_t)$ и градиент $\nabla_{\mathbf{w}} V_{\mathbf{w}}(S_t)$ не входит в цель G_t.

7.4.2 Почему TD нарушает истинный градиентный спуск

Методы TD заменяют G_t одношаговой целью бутстрэппинга $R_{t+1} + \gamma V_{\mathbf{w}}(S_{t+1})$. Полученное обновление:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \underbrace{\big[R_{t+1} + \gamma V_{\mathbf{w}}(S_{t+1}) - V_{\mathbf{w}}(S_t)\big]}_{\delta_t} \nabla_{\mathbf{w}} V_{\mathbf{w}}(S_t). \tag{7.10}$$

Это *похоже* на шаг градиентного спуска, но таковым не является.

Определение 7.5 (Метод полуградиента)**. Метод полуградиента** — это метод, в котором направление обновления вычисляется так, как если бы δ_t был сигналом ошибки, не зависящим от $\mathbf{w}$, то есть градиент берётся только через $V_{\mathbf{w}}(S_t)$, но не через цель $V_{\mathbf{w}}(S_{t+1})$. Обновление (7.10) является шагом полуградиента, поскольку игнорируется градиент δ_t по $\mathbf{w}$ через его зависимость от $V_{\mathbf{w}}(S_{t+1})$.

Чтобы понять, почему градиент через $V_{\mathbf{w}}(S_{t+1})$ опускается, заметим, что если бы мы вычисляли истинный градиент $\frac{1}{2}\delta_t^2$, то получили бы:

$$\nabla_{\mathbf{w}} \tfrac{1}{2}\delta_t^2 = \delta_t\big(\gamma \nabla_{\mathbf{w}} V_{\mathbf{w}}(S_{t+1}) - \nabla_{\mathbf{w}} V_{\mathbf{w}}(S_t)\big).$$

Полуградиент TD отбрасывает слагаемое $\gamma \nabla_{\mathbf{w}} V_{\mathbf{w}}(S_{t+1})$. Это означает, что направление обновления *не является* отрицательным градиентом никакой фиксированной целевой функции. Как следствие, стандартные теоремы о сходимости стохастического градиентного спуска не применяются непосредственно.

Замечание 7.1. Опущение градиента бутстрэппинга является намеренным, а не ошибкой. Если бы включить его, то полученный алгоритм *остаточного градиента* сходился бы в среднеквадратичной ошибке Беллмана, но страдал бы от проблемы «двойной выборки»: для вычисления полного градиента требуются две независимые выборки S_{t+1} для одной и той же пары (S_t, A_t), что обычно недоступно в едином потоке взаимодействия. Методы полуградиента избегают этого, рассматривая цель как фиксированную; их практические свойства сходимости, как правило, лучше, несмотря на отсутствие гарантии градиентного спуска.

7.4.3 Алгоритм полуградиентного TD

Алгоритм 17: Полуградиентное TD(0) для оценивания стратегии

Input: Стратегия π, признаковое отображение φ, шаг $\alpha > 0$, дисконт $\gamma \in [0, 1)$
Инициализировать $\mathbf{w} \in \mathbb{R}^d$ (например, $\mathbf{w} \leftarrow \mathbf{0}$);
for *каждый эпизод* **do**
  Инициализировать состояние S_0;
  for $t = 0, 1, 2, \ldots$ *пока* S_t *не терминальное* **do**
    Выбрать $A_t \sim \pi(\cdot \mid S_t)$;
    Наблюдать R_{t+1} и S_{t+1};
    $\delta_t \leftarrow R_{t+1} + \gamma \mathbf{w}^\top \varphi(S_{t+1}) - \mathbf{w}^\top \varphi(S_t)$;
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta_t\, \varphi(S_t)$;

7.4.4 Сходимость полуградиентного TD

Для линейного случая полуградиентное TD(0) при выборке на текущей стратегии сходится к неподвижной точке TD $\mathbf{w}_{\mathrm{TD}}$, характеризуемой теоремой 7.2. Аргумент опирается на *метод обыкновенного дифференциального уравнения (ОДУ)*: показывается, что поле среднего обновления

$$\bar{f}(\mathbf{w}) = \mathbb{E}_\mu[\delta_t \varphi(S_t) \mid \mathbf{w}] = \Phi^\top D_\mu (T^\pi(\Phi\mathbf{w}) - \Phi\mathbf{w})$$

является сжатием к $\mathbf{w}_{\mathrm{TD}}$, а затем применяется стандартная теорема (Кушнер и Йинь, 2003), утверждающая, что стохастическая аппроксимация сходится к равновесию соответствующего ОДУ.

Для *нелинейных* аппроксиматоров функций даже полуградиентное TD на текущей стратегии не гарантировано сходится. На практике TD-методы на основе нейронных сетей часто сходятся к полезным решениям, однако контрпримеры существуют. Сходимость в нелинейном случае остаётся активной областью исследований.

7.5 Смертельная триада

7.5.1 Три ингредиента нестабильности

Гарантия сходимости теоремы 7.1 критически зависит от слов «выборка на текущей стратегии». При нарушении любого из трёх условий хорошо поведённый ландшафт линейного TD разрушается. При одновременном нарушении всех трёх даже линейная аппроксимация функций может расходиться. Саттон и Барто (2018) называют эту опасную комбинацию **смертельной триадой**.

Определение 7.6 (Смертельная триада)**. Смертельная триада** означает одновременное присутствие трёх элементов:

1. **Аппроксимация функций:** использование параметрической функции $V_{\mathbf{w}}$ (или $Q_{\mathbf{w}}$) с $d \ll |\mathcal{S}|$ вместо отдельной табличной записи на состояние.

2. **Бутстрэппинг:** формирование целей из текущей оценки, как в TD-методах, вместо ожидания полной отдачи, как в Монте-Карло.

3. **Обучение на другой стратегии:** обновление параметров по данным, сгенерированным *поведенческой стратегией* b, отличной от оцениваемой или улучшаемой *целевой стратегии* π.

Сочетание всех трёх может вызвать расходимость параметров даже при линейных аппроксиматорах и даже при бесконечном объёме данных.

Рис. 7.1: Смертельная триада. Каждое ребро треугольника «безопасно»: два из трёх ингредиентов вместе управляемы. Только когда все три вершины активны одновременно, нестабильность становится неизбежной в общем случае. Подписи рёбер указывают, какой элемент отсутствует вдоль каждой стороны.

Важно понимать, что каждая пара ингредиентов управляема:

- *Аппроксимация функций + бутстрэппинг, на текущей стратегии:* Линейное TD(0) сходится к неподвижной точке TD (теорема 7.1).

- *Аппроксимация функций + другая стратегия, без бутстрэппинга:* Монте-Карло на другой стратегии с линейной аппроксимацией функций сходится, поскольку градиент является истинным градиентом среднеквадратичной ошибки ценности.

- *Бутстрэппинг + другая стратегия, табличное:* Q-learning сходится к Q^* в табличном случае (глава 6, теорема о сходимости Q-learning).

Проблема возникает только при сочетании всех трёх.

7.5.2 Контрпример Бэрда

Наиболее ясная иллюстрация смертельной триады — *контрпример Бэрда* (Бэрд, 1995). Рассмотрим простой МПР с 7 состояниями и 2 действиями. Шесть состояний расположены по схеме звезды; седьмое — центральный узел. Поведенческая стратегия всегда выбирает первое действие (переводящее в одно из шести состояний-лучей, равномерно) с вероятностью $\frac{6}{7}$ и второе действие (переводящее в узел) с вероятностью $\frac{1}{7}$. Целевая стратегия всегда выбирает второе действие. Все вознаграждения равны нулю, $\gamma = 0.99$.

Признаковые векторы выбраны так, что семейство приближений линейно. Признаковый вектор для состояния-узла s_7 — $(0, 0, 0, 0, 0, 0, 1, 2)^\top$, а для состояния-луча s_i $(i = 1, \ldots, 6)$ — вектор с 2 на позиции i и 1 на позиции 8. Вектор весов $\mathbf{w} \in \mathbb{R}^8$ инициализируется значением $(1, 1, 1, 1, 1, 1, 10, 1)^\top$.

Теорема 7.4 (Контрпример Бэрда)**.** *При полуградиентном TD(0) на другой стратегии с описанными выше признаковыми векторами вектор весов $\mathbf{w}_t$ расходится к бесконечности почти наверное, несмотря на то, что признаковая матрица имеет полный ранг по столбцам и шаг удовлетворяет стандартным условиям Роббинса–Монро.*

Механизм расходимости тонок: распределение на другой стратегии μ концентрирует слишком много веса на состояниях-лучах (которые часто посещаются при поведенческой стратегии), тогда как обновления Беллмана преимущественно «толкают» ценности этих состояний вверх, поскольку целевая стратегия предпочитает идти в узел. Проекция обратно в класс функций затем усиливает этот толчок вместо его демпфирования, создавая положительную обратную связь.

Замечание 7.2 (Что нас учит контрпример Бэрда). Контрпример Бэрда — не патологическое курьёзное явление; это фундаментальный результат об ограничениях TD-методов. Он говорит нам, что сочетание полуградиентного бутстрэппинга и данных с другой стратегии изначально опасно, независимо от того, насколько хорошо ведёт себя признаковое представление. Решения, исследуемые на практике — коррекции взвешиванием по важности, методы градиентного TD (TDC, GTD2) и инженерные стабилизации DQN — представляют различные стратегии выхода из опасностей триады без потери преимуществ данных с другой стратегии.

7.6 Deep Q-Networks

7.6.1 От линейного к нелинейному приближению

Успехи линейной аппроксимации функций реальны, но ограничены: метод требует вручную спроектированных признаков, и в высокомерных перцептивных областях, таких как сырые пиксели изображений, ни один человеческий разработчик не может заранее определить правильные признаки. Следующий естественный шаг — использовать глубокую нейронную сеть в качестве аппроксиматора, позволяя сети обучать своё собственное внутреннее представление из данных.

Проблема в том, что глубокие сети выявляют все трудности смертельной триады в полной мере. При Q-learning с нейронной сетью Q_θ и жадным выбором действий цель для пары состояние–действие (S_t, A_t):

$$y_t = R_{t+1} + \gamma \max_{a'} Q_\theta(S_{t+1}, a'), \tag{7.11}$$

а минимизируемая функция потерь — $(y_t - Q_\theta(S_t, A_t))^2$. Но и предсказание $Q_\theta(S_t, A_t)$, и цель y_t зависят от одних и тех же параметров θ, поэтому каждый шаг градиента одновременно изменяет предсказание *и* цель. Это создаёт проблему движущейся цели: сеть гонится за величиной, которая меняется с каждым обновлением, приводя на практике к осцилляциям и расходимости.

Две дополнительные патологии усугубляют проблему. Переходы $(S_t, A_t, R_{t+1}, S_{t+1})$, собираемые последовательным агентом, *сильно коррелированы*: последовательные переходы разделяют одну и ту же последовательность состояний. Обучение нейронной сети на коррелированных минипакетах нарушает предположение о н.о.р. стохастического градиентного спуска и заставляет сеть переобучаться на недавнем опыте, «забывая» более раннее обучение. Кроме того, распределение обучающих данных меняется по мере улучшения стратегии, делая ландшафт оптимизации нестационарным.

7.6.2 Буфер повторного воспроизведения опыта

Первое стабилизирующее нововведение, введённое Мних и др. (2013, 2015), — **повторное воспроизведение опыта**. По мере взаимодействия агента со средой каждый переход $(S_t, A_t, R_{t+1}, S_{t+1})$ сохраняется в *буфере повторного воспроизведения* $\mathcal{D}$ ёмкостью N. Во время обучения минипакеты переходов извлекаются *равномерно случайно* из $\mathcal{D}$:

$$\{(s_i, a_i, r_i, s_i')\}_{i=1}^{B} \sim \text{Равном.}(\mathcal{D}). \tag{7.12}$$

Это простое изменение даёт два важных эффекта. Во-первых, оно *декоррелирует* обучающие данные: последовательные минипакеты извлекаются из многих различных временных шагов, разрывая временную корреляцию, дестабилизирующую онлайн-обновления. Во-вторых, каждый переход используется потенциально для многих шагов градиентного спуска, повышая эффективность использования данных. Цена — необходимость хранить большой буфер в памяти и введение неявного компонента другой стратегии: переходы, сохранённые давно, были сгенерированы более ранней, другой стратегией.

7.6.3 Целевые сети

Второе стабилизирующее нововведение — **целевая сеть**. DQN поддерживает две сети с одинаковой архитектурой: *онлайн-сеть* Q_θ (также называемая *поведенческой сетью*) и *целевая сеть* Q_{θ^-}. Онлайн-сеть обновляется на каждом шаге градиентного спуска. Параметры целевой сети θ^- фиксируются на C шагов (обычно $C = 10\,000$), а затем синхронизируются с онлайн-сетью: $\theta^- \leftarrow \theta$.

С целевой сетью функция потерь принимает вид:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\right)^2\right]. \tag{7.13}$$

Градиент берётся только по θ, при θ^- как константе. Фиксируя цель на C шагов, метод снижает корреляцию между обновляемыми параметрами и преследуемой ими целью. Цель не является полностью стационарной — она всё равно меняется каждые C шагов — но на практике это существенно стабилизирует обучение.

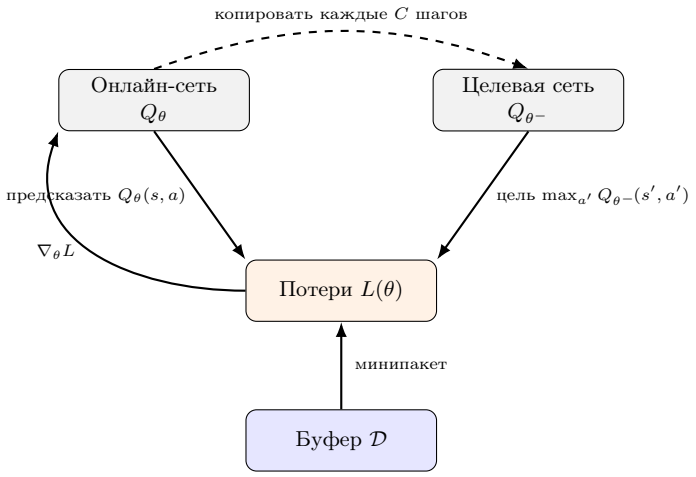


Рис. 7.2: Архитектура DQN. Онлайн-сеть Q_θ выбирает действия и обновляется на каждом шаге. Целевая сеть Q_{θ^-} вычисляет TD-цель и синхронизируется каждые C шагов. Переходы хранятся в буфере повторного воспроизведения $\mathcal{D}$ и извлекаются случайными минипакетами для обучения. Пунктирная стрелка: периодическое жёсткое копирование $\theta^- \leftarrow \theta$.

7.6.4 Алгоритм DQN

7.6.5 Архитектура и результаты на Atari

Для игр Atari сеть DQN является свёрточной архитектурой: три свёрточных слоя (с 32, 64 и 64 фильтрами размеров 8×8, 4×4, 3×3 и шагами 4, 2, 1 соответственно), за которыми следует полносвязный слой из 512 нейронов и линейный выходной слой с одним нейроном на действие. На вход подаётся стопка из четырёх 84×84 кадров в градациях серого, предобработанных из сырого вывода 210×160 RGB.

Мних и др. (2015) оценили DQN на 49 играх Atari, используя *единственную фиксированную архитектуру и гиперпараметры* для всех игр. DQN достиг производительности выше уровня человека в 29 из 49 игр и установил новое состояние искусства почти во всех из них. Принципиально важно, что DQN получает на вход только сырые пиксели и счёт игры, без какой-либо инженерии признаков, специфичной для игры. Это впервые продемонстрировало, что единственный алгоритм глубокого ОП может достичь широкой компетентности в разнообразных задачах исключительно по высокоразмерному сенсорному вводу.

Алгоритм 18: Deep Q-Network (DQN)

Input: Среда, сеть Q_θ, ёмкость буфера N, размер пакета B, частота обновления C, расписание ε, γ, α

Инициализировать θ случайно; $\theta^- \leftarrow \theta$; $\mathcal{D} \leftarrow \emptyset$;
for *каждый эпизод* **do**
  Сбросить среду; наблюдать s_0;
  for $t = 0, 1, 2, \ldots$ **do**
    С вероятностью ε: $a_t \leftarrow$ случайное действие; иначе $a_t \leftarrow \operatorname{argmax}_a Q_\theta(s_t, a)$;
    Выполнить a_t; наблюдать r_{t+1}, s_{t+1}, флаг завершения;
    Сохранить $(s_t, a_t, r_{t+1}, s_{t+1})$ в $\mathcal{D}$ (отбросить старейшее при заполнении);
    if $|\mathcal{D}| \geq B$ **then**
      Извлечь минипакет $\{(s_i, a_i, r_i, s'_i)\}_{i=1}^{B} \sim \mathcal{D}$;
      for *каждого перехода* i **do**
        if s'_i *терминальное* **then** $y_i \leftarrow r_i$;
        else $y_i \leftarrow r_i + \gamma \max_{a'} Q_{\theta^-}(s'_i, a')$;
      Шаг градиента: $\theta \leftarrow \theta - \alpha \nabla_\theta \frac{1}{B} \sum_i (y_i - Q_\theta(s_i, a_i))^2$;
      if $t \bmod C = 0$ **then**
        $\theta^- \leftarrow \theta$
    if *завершено* **then break**;

7.7 Расширения DQN

Статья о DQN дала толчок волне алгоритмических улучшений. Рассмотрим три, ставших стандартными компонентами современных систем глубокого ОП.

7.7.1 Double DQN

Тонкая, но важная патология Q-learning — **смещение максимизации**: оператор $\max_{a'} Q(s', a')$ является завышенной оценкой $\max_{a'} Q^*(s', a')$, когда Q содержит шум, поскольку максимум зашумлённых оценок смещён вверх. Ван Хасселт и др. (2016) показали, что это смещение присутствует и вредоносно в DQN, заставляя агента переоценивать ценности и совершать неоптимальные действия.

Решение — **Double DQN** (DDQN): разделить выбор действия и оценку ценности, используя онлайн-сеть для выбора жадного действия и целевую сеть для его оценки:

$$y_t^{\mathrm{DDQN}} = R_{t+1} + \gamma Q_{\theta^-}\left(S_{t+1}, \operatorname*{argmax}_{a'} Q_\theta(S_{t+1}, a')\right). \tag{7.14}$$

По сравнению с целью DQN в (7.13) единственное изменение — argmax и оценка Q выполняются разными сетями. Это простое изменение существенно снижает завышение оценок и улучшает производительность во многих играх Atari при пренебрежимых вычислительных накладных расходах.

7.7.2 Дуэльный DQN

Ван и др. (2016) заметили, что во многих состояниях ценность состояния $V^\pi(s)$ более информативна, чем отдельные ценности действий $Q^\pi(s, a)$. Когда агент находится в безопасном состоянии, выбор действия мало важен; важно само состояние. Они предложили **дуэльную архитектуру**, которая разделяет финальные слои сети на два потока: один оценивает функцию ценности состояния $V_\theta(s)$, другой — **функцию преимущества**:

$$A_\theta(s, a) = Q_\theta(s, a) - V_\theta(s).$$

Потоки объединяются как:

$$Q_\theta(s, a) = V_\theta(s) + \left(A_\theta(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A_\theta(s, a')\right), \tag{7.15}$$

где вычитание среднего обеспечивает идентифицируемость (поскольку без него V и A не идентифицируемы по отдельности). Дуэльная архитектура позволяет сети обучаться тому, какие состояния ценны, не обучая воздействию каждого действия, что полезно, когда действия имеют схожий эффект в большинстве состояний.

7.7.3 Приоритетное повторное воспроизведение опыта

При стандартном повторном воспроизведении переходы извлекаются из $\mathcal{D}$ равномерно. Шаул и др. (2016) утверждали, что переходы с большими ошибками TD содержат более «удивительную» информацию и должны воспроизводиться чаще. **Приоритетное повторное воспроизведение опыта** (PER) назначает каждому переходу i приоритет:

$$p_i = |\delta_i|^\alpha + \epsilon,$$

где δ_i — наиболее свежая ошибка TD для перехода i, $\alpha > 0$ управляет степенью приоритизации, а $\epsilon > 0$ предотвращает нулевой приоритет. Вероятность выборки: $P(i) = p_i / \sum_j p_j$. Веса взвешивания по важности $w_i = (N \cdot P(i))^{-\beta}$ (при $\beta \to 1$ по мере продвижения обучения) корректируют неравномерную выборку для сохранения несмещённых оценок градиента. PER обычно ускоряет обучение и улучшает итоговую производительность, особенно в средах с редкими вознаграждениями, где информативные переходы редки.

7.8 Анализ сходимости

7.8.1 Что можно доказать

Ландшафт сходимости для аппроксимации функций ценности значительно более неоднозначен, чем для табличного ОП. В таблице 7.1 обобщены основные результаты.

Таблица 7.1: Сводка сходимости для методов аппроксимации функций ценности. «Сходится» означает сходимость к полезному (возможно, приближённому) решению почти наверное или в среднеквадратичном. «Может расходиться» означает, что известны контрпримеры сходимости.

Метод	**Аппрокс.**	**Выборка**	**Гарантия**
МК-градиент	Лин./Нелин.	Любая	Сход. (лок. мин.)
Полуград. TD	Линейная	На тек.	Сход. (н.т. TD)
Полуград. TD	Линейная	На друг.	Может расход.
Полуград. TD	Нелинейн.	На тек.	Может расход.
Град. TD (GTD2)	Линейная	На друг.	Сход. (н.т. TD)
Полуград. Q-learning	Линейная	На друг.	Может расход.
DQN	Нелин. (глуб.)	На друг.	Нет гарантий

7.8.2 Методы градиентного TD

Для выхода из смертельной триады при сохранении обучения на другой стратегии несколько авторов разработали **методы градиентного TD** — алгоритмы, минимизирующие суррогатную целевую функцию, истинный градиент которой поддаётся вычислению. Ключевое наблюдение (Саттон и др., 2009) — определить *проецированную ошибку Беллмана*:

$$\overline{\mathrm{PBE}}(\mathbf{w}) = \|\Pi_\mu (T^\pi V_\mathbf{w} - V_\mathbf{w})\|_\mu^2,$$

которая равна нулю в неподвижной точке TD. В отличие от среднеквадратичной ошибки ценности, градиент $\overline{\mathrm{PBE}}$ можно оценить по одной выборке (с вспомогательным вектором параметров), избегая проблемы двойной выборки. Полученные алгоритмы (TDC и GTD2) сходятся к $\mathbf{w}_{\mathrm{TD}}$ при выборке на другой стратегии, ломая одну ногу смертельной триады. Их практическое применение было ограниченным, однако, поскольку они требуют тонкой настройки двух шагов обучения и могут медленно сходиться.

7.8.3 Нелинейный случай

Для глубоких нейронных сетей не известна никакая гарантия сходимости, аналогичная теореме 7.1, и существуют реальные контрпримеры сходимости с нелинейными аппроксиматорами (Цицикис и Ван Рой, 1997). Инженерные инновации DQN — повторное воспроизведение опыта и целевые сети — являются эвристическими стабилизаторами, хорошо работающими на практике без теоретических гарантий.

Один частичный результат доступен: если потери DQN (7.13) рассматривать как задачу обучения с учителем с фиксированными целями (то есть θ^- постоянно), то стандартное обучение нейронной сети сходится к локальному минимуму $L(\theta)$ для этой фиксированной цели. Трудность в том, что цель периодически меняется. Если целевая сеть обновляется бесконечно медленно (формально, $C \to \infty$ при полностью сошедшейся онлайн-сети на каждом шаге), получается итеративная последовательность задач обучения

с учителем, неподвижные точки которых связаны с неподвижной точкой Q-learning. Эта перспектива *подогнанной итерации ценности* (Гордон, 1999; Ридмиллер, 2005) обеспечивает некоторое теоретическое обоснование поведения DQN.

Замечание 7.3 (Перспектива теории среднего поля). Недавние работы в пределе *среднего поля* (сети бесконечной ширины) и в режиме *нейронного касательного ядра* (NTK) предоставили новые инструменты для анализа ОП на основе нейронных сетей. В пределе NTK сеть ведёт себя примерно линейно по своим параметрам на протяжении всего обучения, восстанавливая гарантии сходимости линейного TD. Хотя этот режим не вполне реалистичен (реальные сети гораздо уже, чем требуется), он подсказывает, что «достаточно сильно параметризованные» сети могут унаследовать часть устойчивости линейного случая, и поддерживает эмпирические наблюдения о том, что широкие сети обучаются в глубоком ОП более стабильно.

7.9 Приложения и практические соображения

7.9.1 Инженерия признаков против сквозного обучения

Центральное проектное решение в любой системе аппроксимации функций ценности — проектировать ли признаки вручную или обучать их сквозным образом.

Признаки, спроектированные вручную (тайловые кодирования, радиальные базисные функции, базисы Фурье) интерпретируемы, требуют меньше данных и допускают гарантии сходимости при линейном приближении. Это правильный выбор, когда доменные знания богаты и пространство состояний умеренно сложно. Например, задача навигации робота на известной карте могла бы использовать в качестве признаков координаты (x, y) робота, его курс и расстояние до ближайшего препятствия — четырёхмерное представление, сжимающее тысячи данных LIDAR до действительно важных величин.

Сквозное глубокое обучение необходимо, когда сырое наблюдение высокоразмерно (изображения, аудио, текст) и релевантные признаки заранее не известны. Цена — значительно большее число параметров, более длительное обучение, сниженная интерпретируемость и более слабые теоретические гарантии. На практике сквозное глубокое ОП выигрывает от: (i) достаточной ёмкости сети для представления функции ценности; (ii) нормализации признаков для предотвращения взрыва градиентов; (iii) хорошо настроенного расписания ε-жадности для баланса исследования и использования; (iv) достаточно большого буфера повторного воспроизведения для разрыва временных корреляций.

7.9.2 Чувствительность к гиперпараметрам

Алгоритмы глубокого ОП печально известны чувствительностью к гиперпараметрам. Хендерсон и др. (2018) показали, что заявленная производительность алгоритмов глубокого ОП может варьироваться более чем на порядок в зависимости от случайных начальных состояний, деталей реализации и выбора гиперпараметров. Следующие гиперпараметры наиболее критически влияют на производительность DQN:

Размер буфера N: Слишком малый — буфер заполняется коррелированным недавним опытом; слишком большой — старые, устаревшие переходы из ранней стратегии преобладают. Типичные значения: 10^5–10^6.

Частота обновления цели C: Слишком малая — проблема движущейся цели; слишком большая — цель сильно отстаёт, замедляя обучение. Типично: 10^3–10^4 шагов.

Размер пакета B: Большие пакеты дают более стабильные оценки градиента, но увеличивают затраты памяти и вычислений на обновление. Типично: 32–256.

Расписание ε: Скорость убывания от 1.0 до малого финального ε должна соответствовать задаче; слишком быстрая ведёт к преждевременному использованию; слишком медленная расточительна.

7.9.3 Практические рекомендации

Для тех, кто впервые реализует глубокое ОП:

- Начинайте с простейшего базового метода: табличного Q-learning или линейной аппроксимации функций, если пространство состояний допускает это. Прежде чем вводить нейронные сети, убедитесь, что среда и сигнал вознаграждения корректны.

- При переходе к глубоким сетям используйте устоявшуюся реализацию (например, Stable Baselines3, CleanRL), а не пишите всё с нуля, чтобы избежать трудноуловимых ошибок в буферах повторного воспроизведения и целевых сетях.
- Стройте графики TD-ошибки, среднего возврата за эпизод и оценок Q на протяжении обучения. Взрывающиеся Q-значения являются надёжным признаком патологии движущейся цели.
- Если обучение нестабильно, уменьшите скорость обучения, увеличьте частоту обновления целевой сети C или перейдите от жёстких обновлений цели ($\theta^- \leftarrow \theta$) к мягким обновлениям ($\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$, $\tau \ll 1$), как это делается в DDPG и SAC.
- Нормализуйте вознаграждения до $[-1, 1]$ или применяйте отсечение градиентов, чтобы предотвратить раннюю расходимость.

7.9.4 Исторические замечания

Аппроксимация функций в ОП имеет долгую историю. Сам Беллман признал проклятие размерности ещё в 1950-х годах. Сходимость линейного TD(0) была установлена Цицикисом и Ван Роем (1997). Смертельная триада была выявлена и названа Саттоном (1995). Нейросетевая аппроксимация функций в ОП восходит как минимум к TD-Gammon Тесауро (1992), который научился играть в нарды на уровне, близком к гроссмейстерскому, используя вручную созданную двухслойную сеть, обученную методами TD. DQN (Мних и др., 2013, в полной версии в 2015) был первой демонстрацией сквозного глубокого ОП, достигшего сверхчеловеческой производительности в разнообразных задачах. Double DQN (Ван Хасселт и др., 2016), Dueling DQN (Ван и др., 2016) и приоритетное повторное воспроизведение опыта (Шаул и др., 2016) были опубликованы в том же году, совместно образуя «эпоху DQN» в глубоком ОП. Алгоритм Rainbow (Хессель и др., 2018) объединил все шесть расширений в единого агента, достигнув значительно улучшенной выборочной эффективности.

Упражнения

Упражнение 7.1. Рассмотрим робота, навигирующего в клеточном мире, описываемом n-мерным бинарным вектором состояния (каждое измерение принимает значение 0 или 1).

1. Сколько состояний требует хранения табличная Q-функция с $|\mathcal{A}| = 4$?
2. Предположим, что вместо этого используется линейная аппроксимация функций с $d = 10n$ признаками, где $d \ll 2^n$. Выразите отношение памяти, требуемой линейным приближением, к табличному представлению как функцию от n.
3. При каком значении n линейное приближение впервые требует меньше памяти, чем табличное (при $d = 10n$)?
4. Обсудите одну ошибку приближения, которую вводит линейное представление по сравнению с табличным.

Упражнение 7.2. Пусть $|\mathcal{S}| = 4$, $d = 2$, $\mu = (1/4, 1/4, 1/4, 1/4)$ и

$$\Phi = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}.$$

1. Убедитесь, что Φ имеет полный ранг по столбцам.
2. Вычислите матрицу проекции $\Pi_\mu = \Phi(\Phi^\top D_\mu \Phi)^{-1}\Phi^\top D_\mu$.
3. Примените проекцию к вектору $v = (3, 1, 2, 4)^\top$. Каково $\Pi_\mu v$?
4. Проверьте, что $\|v - \Pi_\mu v\|_\mu \le \|v - f\|_\mu$ для $f = (2, 2, 2, 2)^\top$.

Упражнение 7.3. Рассмотрим среднеквадратичную ошибку ценности $\overline{\mathrm{VE}}(\mathbf{w}) = \sum_s \mu(s)[V^\pi(s) - V_\mathbf{w}(s)]^2$. Цель TD для перехода (s, r, s'): $y = r + \gamma V_\mathbf{w}(s')$.

1. Запишите $-\frac{1}{2}\nabla_\mathbf{w}[y - V_\mathbf{w}(s)]^2$, рассматривая y как константу (полуградиент).

2. Запишите $-\frac{1}{2}\nabla_{\mathbf{w}}[y - V_{\mathbf{w}}(s)]^2$, рассматривая y как функцию от $\mathbf{w}$ (полный градиент).

3. Определите слагаемое, которое полуградиент опускает. При каком условии это слагаемое равно нулю?

Упражнение 7.4. Оценка ошибки приближения в неподвижной точке TD (теорема 7.3) гласит: $\|\hat{V} - V^\pi\|_\mu \leq \frac{1}{\sqrt{1-\gamma^2}}\|\hat{V}^* - V^\pi\|_\mu$.

1. Вычислите коэффициент оценки для $\gamma \in \{0.5, 0.9, 0.99, 0.999\}$.

2. При $\gamma \to 1$ коэффициент расходится. Дайте интуитивное объяснение, почему бутстрэппинг с дисконтом, близким к единице, усиливает ошибки приближения.

3. Является ли оценка точной в общем случае? Что должно выполняться в МПР и признаковом наборе для достижения равенства?

Упражнение 7.5. Для каждого из следующих алгоритмов определите, какие элементы смертельной триады присутствуют, и укажите, гарантирована ли сходимость:

1. Монте-Карло с линейной аппроксимацией функций, на текущей стратегии.

2. Q-learning в табличном МПР с $|\mathcal{S}| = 10^6$.

3. TD(0) на другой стратегии с нейронной сетью.

4. SARSA с линейной аппроксимацией функций, на текущей стратегии.

5. DQN на Atari.

Упражнение 7.6. Покажите, что функция потерь DQN (7.13) сводится к стандартному обновлению Q-learning (7.11) в частном случае табличного представления (one-hot признаки) с шагом $\alpha = 1$ и без целевой сети ($\theta^- = \theta$). Что это говорит о связи DQN с Q-learning?

Упражнение 7.7. Пусть $Q(s,a) = Q^*(s,a) + \varepsilon_{s,a}$, где $\varepsilon_{s,a}$ — независимые и одинаково распределённые шумовые члены с нулевым средним и дисперсией σ^2.

1. Покажите, что $\mathbb{E}[\max_a Q(s,a)] \geq \max_a Q^*(s,a)$ (смещение максимизации).

2. Предположим, что действия оцениваются с помощью независимой копии $\tilde{Q}(s,a) = Q^*(s,a) + \tilde{\varepsilon}_{s,a}$, где $\tilde{\varepsilon}$ независимо от ε. Остаётся ли $\mathbb{E}[\tilde{Q}(s, \operatorname{argmax}_a Q(s,a))]$ смещённым вверх? Обоснуйте ответ.

3. Свяжите полученные выводы с целью Double DQN (7.14).

Упражнение 7.8. В приоритетном повторном воспроизведении опыта переходы выбираются с вероятностью $P(i) = p_i / \sum_j p_j$ и корректируются весами важности $w_i = (N \cdot P(i))^{-\beta}$.

1. Покажите, что если все переходы имеют одинаковый приоритет $p_i = p$, то $w_i = 1$ для всех i (равномерная выборка, коррекция не требуется).

2. Объясните, почему поправки взвешивания по важности необходимы для сходимости в пределе $\beta = 1$. Что идёт не так при $\beta = 0$?

3. Опишите ситуацию, в которой PER может *ухудшить* производительность по сравнению с равномерным повторным воспроизведением.

Упражнение 7.9. Подогнанная итерация ценности. Рассмотрим следующий алгоритм: имея набор данных переходов $\mathcal{D}$, многократно решать задачу обучения с учителем

$$\mathbf{w}_{k+1} = \operatorname*{argmin}_{\mathbf{w}} \sum_{(s,a,r,s') \in \mathcal{D}} \big(r + \gamma \max_{a'} Q_{\mathbf{w}_k}(s',a') - Q_{\mathbf{w}}(s,a)\big)^2.$$

1. Покажите, что для линейной аппроксимации функций это эквивалентно решению последовательности линейных задач наименьших квадратов.

2. Докажите, что если $Q_{\mathbf{w}}$ является табличным представлением (один параметр на каждую пару (s,a)), а $\mathcal{D}$ покрывает все пары состояние–действие, то алгоритм сходится к Q^*.

3. Объясните, как DQN с периодическим копированием целевой сети аппроксимирует подогнанную итерацию ценности. Какое ключевое приближение при этом делается?

Глава 8

Методы градиента стратегии

Вместо того чтобы искать стратегию косвенно через функцию ценности, мы ищем её напрямую. Мы спрашиваем: при данной текущей стратегии, в каком направлении следует скорректировать параметры, чтобы ожидаемое вознаграждение возросло? Этот вопрос имеет прекрасный ответ, известный под названием теоремы о градиенте стратегии.

Все методы, изученные в главах 4–7, разделяют общую архитектуру: сначала оценить функцию ценности, затем вывести из неё стратегию, действуя жадно или ε-жадно. Функции ценности являются полезными промежуточными объектами, но они не строго необходимы. Вместо этого можно параметризовать стратегию напрямую — как дифференцируемую функцию $\pi_\theta(a \mid s)$ от параметров θ — и оптимизировать эти параметры для максимизации ожидаемой отдачи. Это идея **градиента стратегии**, и именно она является предметом данной главы.

Методы градиента стратегии имеют три важных преимущества перед методами, основанными на функции ценности. Во-первых, они естественно обрабатывают непрерывные и многомерные пространства действий: выборка из $\pi_\theta(\cdot \mid s)$ не требует внутренней максимизации. Во-вторых, обученная стратегия может быть стохастической, что иногда является доказуемо оптимальным (например, в частично наблюдаемых средах или в играх с противником) и всегда является корректно определённым вероятностным распределением. В-третьих, и пожалуй наиболее важным для современных приложений, градиент ожидаемой отдачи по параметрам стратегии может быть оценён исключительно по выборкам траекторий, без необходимости дифференцировать через динамику среды.

Глава организована следующим образом. Раздел 8.1 объясняет мотивацию прямой оптимизации стратегии. Раздел 8.2 выводит теорему о градиенте стратегии — математическую основу всего семейства методов — включая полные доказательства трюка логарифмического отношения, сведения к отдаче с текущего момента с помощью принципа причинности, и компактной формы через Q^{π_θ}. Раздел 8.3 представляет алгоритм REINFORCE с разобранным числовым примером. Раздел 8.4 доказывает, что вычитание базовой линии, зависящей от состояния, не вносит смещения в градиент, и выводит оптимальную базовую линию. Раздел 8.5 объединяет оценку функции ценности с градиентом стратегии, получая REINFORCE с обученной базовой линией. Раздел 8.6 разрабатывает REINFORCE Leave-One-Out (RLOO) — современный оценщик, играющий ключевую роль в выравнивании больших языковых моделей. Раздел 8.7 вводит натуральный градиент стратегии, который корректирует обычный градиент с учётом кривизны многообразия стратегий. Раздел 8.8 анализирует сходимость и компромисс между смещением и дисперсией. Раздел 8.9 сравнивает оценщики в итоговой таблице. Глава завершается упражнениями.

8.1 Зачем оптимизировать стратегию напрямую?

8.1.1 Ограничения подхода, основанного на функции ценности

Пайплайн подхода, основанного на функции ценности: обучить $\hat{Q} \approx Q^*$, затем задать $\pi(s) = \operatorname{argmax}_a \hat{Q}(s, a)$. Этот пайплайн ломается двумя важными способами, когда пространство действий велико или непрерывно.

1. *Непрерывные действия.* Если $\mathcal{A} \subseteq \mathbb{R}^m$, вычисление $\operatorname{argmax}_{a \in \mathbb{R}^m} \hat{Q}(s, a)$ само по себе является задачей непрерывной оптимизации, которую необходимо решать на каждом шаге во время выполнения. Такие методы, как DDPG, обходят эту проблему, обучая отдельную сеть-максимизатор, но в результате получается архитектура актор-критик — которая уже выходит за рамки чистого обучения на основе функции ценности.

2. *Очень большие дискретные словари.* В генерации языка на уровне токенов $|\mathcal{A}|$ равен размеру словаря (часто 50,000 или более), а длина эпизода может составлять сотни шагов. Хранить отдельное значение для каждой пары (s, a) невозможно; и даже при использовании аппроксимации функции, перебор всех действий для нахождения жадного является дорогостоящим.

Обе трудности исчезают, когда стратегия является параметризованным распределением $\pi_\theta(a \mid s)$: во время выполнения достаточно просто выбрать $A_t \sim \pi_\theta(\cdot \mid S_t)$, независимо от размера $\mathcal{A}$.

8.1.2 Стохастические стратегии иногда необходимы

Помимо масштабируемости, есть принципиальные причины желать стохастических стратегий. В частично наблюдаемых средах оптимальная стратегия, как правило, стохастична: агенту может потребоваться рандомизация для хеджирования от скрытых состояний. В играх с противником детерминированная стратегия поддаётся эксплуатации, тогда как равновесие смешанных стратегий требует рандомизации. При исследовании стохастическая стратегия выбирает действия без необходимости отдельного механизма ε-жадности, а степень исследования автоматически контролируется энтропией π_θ.

Замечание 8.1. Детерминированная стратегия $\mu_\theta(s)$ является частным случаем: она соответствует распределению, сосредоточенному в одном действии. Теорема о *детерминированном градиенте стратегии* (DPG) Сильвера и др. (2014) расширяет разработанный здесь аппарат на этот вырожденный случай и приводит к таким алгоритмам, как DDPG и TD3. Мы не будем развивать это направление дальше; оно рассматривается в главе 9.

8.1.3 Целевая функция

В эпизодической задаче с горизонтом T мы оптимизируем ожидаемую дисконтированную отдачу:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}\right], \tag{8.1}$$

где траектория $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T)$ распределена согласно

$$p_\theta(\tau) = \rho_0(S_0) \prod_{t=0}^{T-1} \pi_\theta(A_t \mid S_t)\, p(S_{t+1}, R_{t+1} \mid S_t, A_t), \tag{8.2}$$

ρ_0 — это распределение начального состояния, а p — (неизвестное) ядро переходов. В задаче без эпизодов целевая функция является средним вознаграждением $J(\theta) = \lim_{T\to\infty} \frac{1}{T}\,\mathbb{E}[\sum_{t=0}^{T-1} R_{t+1}]$; мы сосредотачиваемся на эпизодическом случае для ясности, отмечая, что теорема о градиенте стратегии справедлива в обоих случаях.

Центральный вопрос: как меняется $J(\theta)$ при изменении θ? Наивное дифференцирование (8.1) кажется требующим дифференцирования через неизвестную динамику среды p. Теорема о градиенте стратегии показывает, что это не нужно.

8.2 Теорема о градиенте стратегии

8.2.1 Трюк логарифмического отношения

Начнём с вычисления $\nabla_\theta J(\theta)$ непосредственно из определения. Поскольку горизонт конечен, можно дифференцировать под знаком математического ожидания:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau p_\theta(\tau)\, R(\tau) = \sum_\tau \nabla_\theta p_\theta(\tau)\, R(\tau), \tag{8.3}$$

где $R(\tau) = \sum_{t=0}^{T-1} \gamma^t R_{t+1}$ — полная дисконтированная отдача траектории τ. Эта сумма не является математическим ожиданием, поэтому её нельзя оценить по выборкам — пока что. **Трюк логарифмического отношения** (также называемый *трюком функции счёта* или *трюком REINFORCE*) преобразует его в таковое. Для любого $p_\theta(\tau) > 0$:

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau). \tag{8.4}$$

Подставляя в (8.3), получаем

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\big[R(\tau)\, \nabla_\theta \log p_\theta(\tau)\big]. \tag{8.5}$$

Теперь воспользуемся факторизованной формой (8.2). Логарифмируя:

$$\log p_\theta(\tau) = \log \rho_0(S_0) + \sum_{t=0}^{T-1} \log \pi_\theta(A_t \mid S_t) + \sum_{t=0}^{T-1} \log p(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

Первый и третий члены не зависят от θ, поэтому их градиенты обращаются в нуль:

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{8.6}$$

Принципиально важно, что эта формула включает только *стратегию*, а не ядро переходов. Подставляя в (8.5), получаем базовый оценщик градиента стратегии:

Теорема 8.1 (Базовый оценщик градиента стратегии)**.** *При приведённых выше предположениях,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\left(\sum_{t=0}^{T-1} \gamma^t R_{t+1}\right) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\right]. \tag{8.7}$$

Уравнение (8.7) уже является корректным оценщиком градиента: для данной одиночной траектории τ выражение внутри математического ожидания является несмещённой выборкой $\nabla_\theta J(\theta)$. Однако оно имеет серьёзную проблему с дисперсией: полная отдача $R(\tau)$ умножает градиент логарифма вероятности каждого действия в траектории, даже действий, которые произошли *до* получения вознаграждений. Интуитивно, то, что произошло при $t = 3$, не может задним числом изменить вознаграждение, полученное при $t = 1$. Следующая лемма формализует это.

8.2.2 Лемма о причинности и отдача с текущего момента

Лемма 8.2 (Причинность)**.** *Для каждого* $t \in \{0, \dots, T-1\}$,

$$\mathbb{E}_{\tau\sim p_\theta}\left[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \sum_{k=0}^{t-1} \gamma^k R_{k+1}\right] = 0. \tag{8.8}$$

Доказательство. Пусть $C_t = \sum_{k=0}^{t-1} \gamma^k R_{k+1}$ обозначает накопленное дисконтированное вознаграждение *до* момента t. Эта величина измерима относительно истории $(S_0, A_0, R_1, \dots, S_t)$ и, в частности, не зависит от выбора A_t при данном S_t. По теореме о полном математическом ожидании,

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, C_t] = \mathbb{E}[C_t\ \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t, \text{история}_{<t}]]\,.$$

Для внутреннего математического ожидания зафиксируем $S_t = s$. Тогда $A_t \sim \pi_\theta(\cdot \mid s)$, и

$$\begin{aligned}\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t = s] &= \sum_a \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s)\\ &= \sum_a \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta \sum_a \pi_\theta(a \mid s) = \nabla_\theta 1 = 0.\end{aligned}$$

Поскольку внутреннее условное математическое ожидание тождественно равно нулю, внешнее ожидание также равно нулю. □

Замечание 8.2. Ключевое вычисление $\sum_a \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$ называется **тождеством функции счёта**. Это алгебраический механизм, лежащий в основе как теоремы о базовой линии, так и леммы о причинности.

Определим **отдачу с текущего момента** начиная с шага t как

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}, \qquad \bar{G}_t = \gamma^t G_t = \sum_{k=t}^{T-1} \gamma^k R_{k+1}. \tag{8.9}$$

Полная отдача разлагается как $R(\tau) = C_t + \bar{G}_t$. Применяя лемму 8.2 к (8.7), получаем редуцированный оценщик:

Теорема 8.3 (Градиент стратегии через отдачу с текущего момента)**.**

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, \bar{G}_t\right] = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G_t\right]. \tag{8.10}$$

Доказательство. Начиная с теоремы 8.1, меняем порядок суммирования по t и суммы по вознаграждениям. Для каждого фиксированного t разложим $R(\tau) = C_t + \bar{G}_t$ и применим линейность математического ожидания:

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, R(\tau)] = \underbrace{\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, C_t]}_{=0 \text{ по лемме } 8.2} + \mathbb{E}\big[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, \bar{G}_t\big]\,.$$

Суммируя по t и используя $\bar{G}_t = \gamma^t G_t$, получаем (8.10). □

8.2.3 Теорема о градиенте стратегии: компактная форма

Форма с отдачей с текущего момента (8.10) является наиболее практичным оценщиком. Её также можно записать в компактной форме по парам состояние-действие через усреднение по дисконтированной мере посещаемости состояний.

Определение 8.1 (Дисконтированная мера посещаемости)**.**

$$d^\gamma_{\pi_\theta}(s) = \sum_{t=0}^{T-1} \gamma^t \mathbb{P}_{\pi_\theta}(S_t = s).$$

Теорема 8.4 (Теорема о градиенте стратегии)**.**

$$\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}} d^\gamma_{\pi_\theta}(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a) = \mathbb{E}_{S\sim d^\gamma_{\pi_\theta},\, A\sim\pi_\theta(\cdot\mid S)}[\nabla_\theta \log \pi_\theta(A \mid S)\, Q^{\pi_\theta}(S,A)]\,. \tag{8.11}$$

Доказательство. Начинаем с (8.10). По линейности математического ожидания,

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T-1} \gamma^t\ \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G_t]\,.$$

Применяем теорему о полном математическом ожидании, обусловливаясь на (S_t, A_t):

$$\begin{aligned}
&\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G_t] \\
&\quad = \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\ \mathbb{E}[G_t \mid S_t, A_t]] \\
&\quad = \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, Q^{\pi_\theta}(S_t, A_t)]\,.
\end{aligned}$$

Раскрывая внешнее математическое ожидание как сумму по (s,a) и используя тождество $\pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta \pi_\theta(a \mid s)$:

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T-1} \gamma^t \sum_s \mathbb{P}(S_t = s) \sum_a \nabla_\theta \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a).$$

Собирая коэффициент при каждом состоянии s, получаем $\sum_t \gamma^t \mathbb{P}(S_t = s) = d^\gamma_{\pi_\theta}(s)$, что соответствует определению 8.1, откуда следует результат. □

Замечание 8.3. Теорема 8.4 примечательна: градиент $J(\theta)$ по параметрам стратегии зависит от динамики переходов *только через* функцию ценности действий Q^{π_θ} — и даже её можно оценить по выборкам, не зная p. Именно это делает методы градиента стратегии истинно свободными от модели.

8.3 REINFORCE

8.3.1 Алгоритм

REINFORCE (Уильямс, 1992) — прямая реализация метода Монте-Карло для теоремы 8.3. Поскольку $\mathbb{E}_{\pi_\theta}[G_t \mid S_t, A_t] = Q^{\pi_\theta}(S_t, A_t)$, мы можем подставить выборочную отдачу G_t вместо истинного Q^{π_θ}:

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \gamma^t G_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{8.12}$$

Это обновление является несмещённым шагом стохастического градиентного подъёма по $J(\theta)$.

Стоит сделать несколько замечаний по реализации. Во-первых, множитель γ^t перед обновлением уменьшает размер шага для более поздних шагов по времени, отражая более низкий дисконтированный вес

Алгоритм 19: REINFORCE (Уильямс, 1992)

Input: Стратегия π_θ, размер шага α, дисконт γ
Инициализировать θ произвольно;
for *каждый эпизод* **do**
  Сгенерировать $\tau = (S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T)$ следуя π_θ;
  for $t = 0, 1, \ldots, T-1$ **do**
    Вычислить $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$;
    $\theta \leftarrow \theta + \alpha\, \gamma^t G_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$;

соответствующего члена градиента. Некоторые реализации опускают этот множитель; это соответствует оптимизации несколько иной (недисконтированной) целевой функции, но является распространённой практикой. Во-вторых, поскольку все обновления в эпизоде могут быть вычислены в конце эпизода, цикл по t можно векторизовать: вычислить все значения G_t за один обратный проход, затем просуммировать взвешенные градиенты. В-третьих, в случае стратегии softmax по действиям, $\nabla_\theta \log \pi_\theta(a \mid s)$ является функцией счёта, равной $\phi(s,a) - \sum_{a'} \pi_\theta(a' \mid s)\, \phi(s,a')$ для векторов признаков $\phi(s,a)$.

8.3.2 Разобранный пример

Пример 8.1 (REINFORCE на двухсостоянном МПР)**.** Рассмотрим следующий простой эпизодический МПР:

- Два состояния: $\mathcal{S} = \{s_1, s_2\}$; два действия: $\mathcal{A} = \{a_L, a_R\}$.
- Начальное состояние всегда s_1. Длина эпизода $T = 2$.
- Переходы: из s_1 действие a_L ведёт в s_2 с вознаграждением $R = 0$; действие a_R остаётся в s_1 с вознаграждением $R = 0$. Из s_2 (или s_1 при $t = 1$) получается фиксированное терминальное вознаграждение $+1$ за a_R и 0 за a_L.
- Стратегия: $\pi_\theta(a_R \mid s_1) = \sigma(\theta_1)$, $\pi_\theta(a_R \mid s_2) = \sigma(\theta_2)$, где σ — сигмоидная функция.

Задать $\gamma = 1$, $\alpha = 0.1$ и инициализировать $\theta_1 = \theta_2 = 0$ (равномерная стратегия, вероятность каждого действия 0.5).

Эпизод: Предположим, что выборочная траектория равна $(s_1, a_L, 0, s_2, a_R, 1)$.

Вычислить отдачу с текущего момента:

$$G_0 = 0 + 1 = 1, \qquad G_1 = 1.$$

Градиенты функции счёта (при $\theta_1 = \theta_2 = 0$, т. е. $\pi = 0.5$ для обоих действий):

$$\nabla_{\theta_1} \log \pi_\theta(a_L \mid s_1) = \frac{\partial}{\partial \theta_1} \log(1 - \sigma(\theta_1)) = -\sigma(\theta_1) = -0.5,$$

$$\nabla_{\theta_2} \log \pi_\theta(a_R \mid s_2) = \sigma(\theta_2)(1 - \sigma(\theta_2)) \cdot \frac{1}{\sigma(\theta_2)} = 1 - 0.5 = 0.5.$$

Обновления градиента ($\gamma = 1$):

$$\Delta\theta_1 = \alpha \cdot \gamma^0 \cdot G_0 \cdot (-0.5) = 0.1 \cdot 1 \cdot 1 \cdot (-0.5) = -0.05,$$

$$\Delta\theta_2 = \alpha \cdot \gamma^1 \cdot G_1 \cdot 0.5 = 0.1 \cdot 1 \cdot 1 \cdot 0.5 = +0.05.$$

После обновления: $\theta_1 = -0.05$ (делая a_L чуть менее вероятным из s_1) и $\theta_2 = +0.05$ (делая a_R более вероятным из s_2). Интуитивно алгоритм работает правильно: траектория, давшая положительную отдачу, подталкивает стратегию к действиям, которые её породили.

Замечание 8.4. REINFORCE сходится к локальному максимуму $J(\theta)$ при стандартных условиях стохастической аппроксимации (убывающие размеры шагов, $\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$). Однако его дисперсия может быть очень большой, поскольку G_t может сильно варьироваться от эпизода к эпизоду. Следующие два раздела рассматривают снижение дисперсии.

8.4 Снижение дисперсии: базовые линии

8.4.1 Теорема о базовой линии

Базовая линия $b(s)$ — это любая функция состояния (но не действия), которая вычитается из отдачи в оценщике градиента стратегии. Ключевой результат состоит в том, что это не вносит смещения.

Теорема 8.5 (Теорема о базовой линии)**.** *Для любой функции* $b : \mathcal{S} \to \mathbb{R}$,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\,\big(G_t - b(S_t)\big)\right]. \tag{8.13}$$

Доказательство. Достаточно показать, что дополнительный член, введённый b, имеет нулевое математическое ожидание. Для каждого фиксированного t,

$$\begin{aligned}\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, b(S_t)] &= \mathbb{E}[b(S_t)\ \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t]] \\ &= \mathbb{E}[b(S_t)\cdot 0] = 0,\end{aligned}$$

где внутреннее математическое ожидание обращается в нуль по тождеству функции счёта (см. доказательство леммы 8.2). Суммируя по t и применяя линейность математического ожидания, получаем (8.13). □

8.4.2 Оптимальная базовая линия

Поскольку базовая линия не меняет среднее значение оценщика, мы вольны выбирать её так, чтобы *минимизировать дисперсию.* Рассмотрим дисперсию одношагового члена $g_t(\theta) = \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\,(G_t - b)$ (зависимость от состояния опущена для ясности). Дисперсия скалярной случайной величины X удовлетворяет $\mathrm{Var}[X - c] = \mathrm{Var}[X] - 2c\ \mathrm{Cov}[X, 1] + c^2\ \mathrm{Var}[1]$; для вектора $\mathbf{v} = \mathbf{u}\,(G_t - b)$ аналогичное вычисление, применённое к каждой компоненте, даёт оптимальную базовую линию для k-й компоненты градиента:

$$b_k^*(s) = \frac{\mathbb{E}\big[(\nabla_\theta \log \pi_\theta(A_t \mid S_t))_k^2\, G_t \;\big|\; S_t = s\big]}{\mathbb{E}[(\nabla_\theta \log \pi_\theta(A_t \mid S_t))_k^2 \mid S_t = s]}. \tag{8.14}$$

Это отношение математических ожиданий, зависящее от индекса компоненты k. Оптимальная базовая линия, таким образом, является вектором и, как правило, вычислительно неприемлема. На практике вместо неё используется функция ценности состояния $V^{\pi_\theta}(s)$.

8.4.3 Базовая линия на основе функции ценности состояния

Наиболее естественной и широко используемой базовой линией является $b(S_t) = V^{\pi_\theta}(S_t)$. Вычитание функции ценности из отдачи даёт **функцию преимущества**:

$$A^{\pi_\theta}(s, a) = Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s). \tag{8.15}$$

Теорема о градиенте стратегии с базовой линией на основе функции ценности принимает вид

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, A^{\pi_\theta}(S_t, A_t)\right]. \tag{8.16}$$

Поскольку $\mathbb{E}_\pi[A^\pi(s, a) \mid s] = 0$ по определению, преимущество имеет нулевое среднее под стратегией, что часто резко снижает дисперсию оценщика. На практике V^{π_θ} неизвестно и само должно быть оценено; следующий раздел рассматривает это.

Пример 8.2 (Влияние базовой линии на дисперсию)**.** Рассмотрим бандитную (однострановую) среду с двумя действиями. Действие a_1 всегда даёт вознаграждение 10; действие a_2 всегда даёт вознаграждение 10.1. Без базовой линии сигнал градиента пропорционален 10 или 10.1 — оба большие и почти равные. Оценщик с трудом различает чуть лучшее действие a_2 среди шума. При базовой линии $b = 10.05$ (среднее вознаграждение) сигнал градиента пропорционален -0.05 или $+0.05$, который центрирован вокруг нуля и имеет значительно меньшую абсолютную величину, существенно снижая дисперсию.

Алгоритм 20: REINFORCE с обученной базовой линией

Input: Стратегия π_θ, функция ценности $\hat{V}_w$, размеры шагов α_θ, α_w, дисконт γ
Инициализировать θ, w произвольно;
for *каждый эпизод* **do**
  Сгенерировать $\tau = (S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T)$ следуя π_θ;
  for $t = 0, 1, \ldots, T-1$ **do**
    $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$;
    $\delta_t \leftarrow G_t - \hat{V}_w(S_t)$; // Оценка преимущества
    $w \leftarrow w + \alpha_w \, \delta_t \, \nabla_w \hat{V}_w(S_t)$; // Обновление функции ценности
    $\theta \leftarrow \theta + \alpha_\theta \, \gamma^t \, \delta_t \, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$; // Обновление стратегии

8.5 REINFORCE с обученной базовой линией

В предыдущем разделе базовая линия $b(s) = V^{\pi_\theta}(s)$ считалась известной. На практике её необходимо оценивать. Простейший подход — поддерживать отдельную параметризованную функцию ценности $\hat{V}_w(s)$ (*критика*) рядом со стратегией (*актором*) и обучать обоих одновременно.

Обновление функции ценности минимизирует среднеквадратичную ошибку $\mathbb{E}[(G_t - \hat{V}_w(S_t))^2]$, что является задачей обучения с учителем. Заметим, что градиент по w равен $\nabla_w \hat{V}_w(S_t)$, а целью служит выборочная отдача G_t (то есть без бутстрэппинга), поэтому обновление функции ценности является обычным шагом градиента — а не полуградиентным шагом, как в TD(0) (см. главу 6).

Замечание 8.5. Введение отдельного критика открывает путь к полному семейству актор-критик, где критик использует TD-бутстрэппинг вместо отдачи Монте-Карло. Это обменивает несмещённость REINFORCE на сниженную дисперсию ценой введения смещения из-за ошибки аппроксимации функции. Методы актор-критик рассматриваются в главе 9.

Распространённое опасение состоит в том, не вносит ли обученная базовая линия смещение. Нет: ключевое состоит в том, что $\hat{V}_w(S_t)$ зависит только от состояния S_t, а не от действия A_t, выбранного на этом шаге. Поэтому теорема 8.5 применима независимо от того, как получено $\hat{V}_w$. Оценка градиента стратегии $\delta_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)$ остаётся несмещённой для $\nabla_\theta J(\theta)$ даже при $\hat{V}_w \neq V^{\pi_\theta}$, хотя неоптимальное $\hat{V}_w$ не будет снижать дисперсию так же эффективно, как истинная функция ценности.

8.6 RLOO: REINFORCE Leave-One-Out

8.6.1 Мотивация: градиент стратегии в контексте языковых моделей

Современные пайплайны выравнивания больших языковых моделей (LLM) (см. главу 14) дообучают языковую модель π_θ с использованием сигнала вознаграждения $r(\mathbf{y} \mid \mathbf{x})$, который оценивает сгенерированное дополнение $\mathbf{y}$ к промпту $\mathbf{x}$. В этом контексте «действие» — это целое дополнение (или, эквивалентно, последовательность действий на уровне токенов), а «состояние» — промпт. Таким образом, возникает задача градиента стратегии с:

- Очень высокой дисперсией на выборку (дополнения могут сильно различаться).
- Отсутствием доступа к отдельно обученному критику (обучение головы функции ценности добавляет потребление памяти и нестабильность).
- Естественной пакетной структурой: для каждого промпта $\mathbf{x}$ в мини-пакете модель генерирует K независимых дополнений $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(K)} \sim \pi_\theta(\cdot \mid \mathbf{x})$.

Ключом является пакетная структура. RLOO использует её для построения базовой линии с низкой дисперсией, несмещённой и не требующей критика.

8.6.2 Оценщик RLOO

Зафиксируем промпт $\mathbf{x}$ и выберем K н.о.р. дополнений $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(K)} \sim \pi_\theta(\cdot \mid \mathbf{x})$ с соответствующими вознаграждениями $r_i = r(\mathbf{y}^{(i)} \mid \mathbf{x})$. **Базовая линия RLOO** для выборки i — среднее вознаграждение остальных $K-1$ выборок:

$$b_i = \frac{1}{K-1} \sum_{j \neq i} r_j = \frac{1}{K-1} \left(\sum_{j=1}^{K} r_j - r_i \right). \tag{8.17}$$

Оценщик градиента стратегии RLOO тогда равен

$$\hat{g}_{\text{RLOO}} = \frac{1}{K}\sum_{i=1}^{K}(r_i - b_i)\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}), \tag{8.18}$$

где $\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}) = \sum_t \nabla_\theta \log \pi_\theta(y_t^{(i)} \mid \mathbf{x}, y_{<t}^{(i)})$ — суммарная функция счёта по токенам.

Подставляя (8.17) в (8.18), можно также записать

$$\hat{g}_{\text{RLOO}} = \frac{1}{K}\sum_{i=1}^{K}\left(r_i - \frac{\bar{r} - r_i/K}{(K-1)/K}\right)\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}), \tag{8.19}$$

где $\bar{r} = \frac{1}{K}\sum_j r_j$. Более компактное выражение:

$$r_i - b_i = r_i - \frac{\sum_{j\neq i} r_j}{K-1} = \frac{K\,r_i - \sum_{j=1}^{K} r_j}{K-1} = \frac{K}{K-1}(r_i - \bar{r})\,. \tag{8.20}$$

Таким образом, преимущество для выборки i является масштабированным отклонением r_i от среднего по пакету, с поправочным коэффициентом $K/(K-1)$, учитывающим структуру leave-one-out.

Алгоритм 21: Обновление градиента стратегии RLOO

Input: Стратегия π_θ, функция вознаграждения r, размер пакета K, размер шага α

for *каждый промпт* $\mathbf{x}$ *в мини-пакете* **do**
  Выбрать K дополнений $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(K)} \sim \pi_\theta(\cdot \mid \mathbf{x})$;
  Вычислить вознаграждения $r_i \leftarrow r(\mathbf{y}^{(i)} \mid \mathbf{x})$ для $i = 1, \ldots, K$;
  Вычислить среднее по пакету $\bar{r} \leftarrow \frac{1}{K}\sum_{i=1}^{K} r_i$;
  for $i = 1, \ldots, K$ **do**
    $b_i \leftarrow \frac{K}{K-1}(\bar{r} - r_i/K)\cdot\frac{1}{1} = \frac{1}{K-1}\left(\sum_j r_j - r_i\right)$; `// Базовая линия LOO`
    $\hat{A}_i \leftarrow r_i - b_i = \frac{K}{K-1}(r_i - \bar{r})$;
  $\theta \leftarrow \theta + \alpha \cdot \frac{1}{K}\sum_{i=1}^{K} \hat{A}_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})$;

8.6.3 Несмещённость RLOO

Теорема 8.6 (RLOO несмещён)**.** *Оценщик RLOO $\hat{g}_{RLOO}$, определённый в (8.18), является несмещённым оценщиком $\nabla_\theta J(\theta)$:*

$$\mathbb{E}[\hat{g}_{RLOO}] = \nabla_\theta J(\theta).$$

Доказательство. Достаточно показать, что b_i удовлетворяет условиям теоремы о базовой линии (теорема 8.5), т. е. что она не зависит от действия $\mathbf{y}^{(i)}$ при данном состоянии (промпте $\mathbf{x}$). Базовая линия $b_i = \frac{1}{K-1}\sum_{j\neq i} r_j$ является функцией $K-1$ остальных дополнений $\mathbf{y}^{(j)}$, $j \neq i$. Поскольку все K дополнений выбираются независимо из $\pi_\theta(\cdot \mid \mathbf{x})$, базовая линия b_i не зависит от $\mathbf{y}^{(i)}$ условно на $\mathbf{x}$.

Формально, взяв математическое ожидание i-го члена:

$$\mathbb{E}\Big[(r_i - b_i)\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \mathbb{E}\Big[r_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] - \mathbb{E}\Big[b_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big]\,.$$

Для второго члена обусловимся на все $\mathbf{y}^{(j)}$ с $j \neq i$ (что определяет b_i):

$$\mathbb{E}\Big[b_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \mathbb{E}\Big[b_i\;\mathbb{E}\Big[\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}) \;\Big|\; \mathbf{x},\, \{r_j\}_{j\neq i}\Big]\Big]\,.$$

Внутреннее математическое ожидание — это в точности тождество функции счёта, применённое к распределению $\pi_\theta(\cdot \mid \mathbf{x})$:

$$\mathbb{E}_{\mathbf{y}^{(i)}\sim\pi_\theta(\cdot\mid\mathbf{x})}\Big[\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \nabla_\theta \sum_{\mathbf{y}} \pi_\theta(\mathbf{y} \mid \mathbf{x}) = \nabla_\theta 1 = 0.$$

Поэтому вычтенный член с базовой линией равен нулю, и

$$\mathbb{E}\Big[(r_i - b_i)\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \mathbb{E}\Big[r_i\,\nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})\Big] = \nabla_\theta J(\theta)\big|_{\text{одна выборка}}.$$

Усредняя по $i = 1, \ldots, K$, получаем $\hat{g}_{\text{RLOO}}$, математическое ожидание которого равно $\nabla_\theta J(\theta)$. □

8.6.4 Снижение дисперсии в RLOO

Чтобы понять, почему RLOO снижает дисперсию по сравнению с обычным REINFORCE, сравним два оценщика. Обычный REINFORCE использует полную отдачу r_i в качестве веса градиента (без базовой линии):

$$\hat{g}_{\mathrm{RF}} = \frac{1}{K}\sum_{i=1}^{K} r_i \, \nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x}).$$

Пусть $g_i = \nabla_\theta \log \pi_\theta(\mathbf{y}^{(i)} \mid \mathbf{x})$ и $\mu = \mathbb{E}[r_i]$. Тогда $\mathrm{Var}[r_i\, g_i] \approx \mathrm{Var}[r_i]\,\mathrm{Var}[g_i] + \mu^2\,\mathrm{Var}[g_i]$ (без учёта перекрёстных членов), что растёт с величиной μ. RLOO заменяет r_i на $r_i - b_i \approx r_i - \mu$ (с точностью до поправки $K/(K-1)$), среднее значение которого равно нулю. Доминирующий член μ^2 тем самым устраняется, и дисперсия масштабируется с $\mathrm{Var}[r_i]$, а не с $\mathbb{E}[r_i^2]$.

Точнее, при большом K базовая линия RLOO $b_i \to \mu = \mathbb{E}_{\pi_\theta}[r]$ (по закону больших чисел), и RLOO приближается к оракульной базовой линии $b = \mu$. Остаточная дисперсия $r_i - b_i$ равна $\mathrm{Var}[r_i]/(K-1)$, по сравнению с $\mathrm{Var}[r_i] + \mu^2$ при отсутствии базовой линии. Когда вознаграждения имеют большое положительное среднее ($\mu \gg 0$), снижение дисперсии может быть значительным.

8.6.5 Связь с RLHF

В пайплайнах RLHF (глава 14) модель вознаграждения $r_\phi(\mathbf{y} \mid \mathbf{x})$ обеспечивает скалярную обратную связь по дополнениям модели. RLOO хорошо вписывается: для каждого промпта $\mathbf{x}$ выбирается K дополнений из текущей стратегии π_θ, они оцениваются моделью вознаграждения, и вычисляется градиент RLOO. Сеть-критик не нужна, что экономит память GPU и устраняет нестабильность, которая может возникать при совместном обучении критика и стратегии.

RLOO был введён Ахмадяном и др. (2024) и показал превосходство над базовыми методами RLHF на основе PPO при одинаковом количестве запросов к модели вознаграждения, в основном благодаря снижению дисперсии от базовой линии leave-one-out и устранению критика. Его также можно комбинировать со штрафом за KL-дивергенцию от референсной стратегии π_{ref}, заменив r_i на $r_i - \beta \log(\pi_\theta(\mathbf{y}^{(i)})/\pi_{\mathrm{ref}}(\mathbf{y}^{(i)}))$, что контролирует, насколько сильно дообученная модель отклоняется от предобученной.

Замечание 8.6. RLOO тесно связан с Group Relative Policy Optimisation (GRPO), который также использует пакет выборок на промпт и нормализует преимущества внутри группы. Ключевое отличие в том, что GRPO делит на стандартное отклонение внутри группы (стандартизируя преимущество), тогда как RLOO использует немасштабированное отклонение leave-one-out. Оба принадлежат к семейству оценщиков с «самоопорными базовыми линиями», которые извлекают снижающий дисперсию сигнал из самого пакета, без отдельно обученного критика.

8.7 Натуральный градиент стратегии

8.7.1 Проблема обычного градиента

Обычный градиент $\nabla_\theta J(\theta)$ определён относительно евклидовой метрики в пространстве параметров. Шаг градиента $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$ перемещает на фиксированное евклидово расстояние в пространстве θ, но соответствующее изменение распределения стратегии π_θ зависит от локальной геометрии многообразия стратегий. Вблизи детерминированной стратегии (где вероятность некоторого действия близка к 1) малый шаг в θ вызывает большое изменение π_θ; вблизи равномерной стратегии тот же шаг вызывает меньшее распределительное изменение. Обычный градиент поэтому плохо откалиброван: его величина отражает евклидово расстояние в пространстве параметров, а не изменение распределения.

8.7.2 Матрица информации Фишера

Натуральный градиент стратегии исправляет это, используя **матрицу информации Фишера** стратегии в качестве римановой метрики:

$$F(\theta) = \mathbb{E}_{S\sim d^{\gamma}_{\pi_\theta},\, A\sim\pi_\theta(\cdot\mid S)}\left[\nabla_\theta \log \pi_\theta(A \mid S)\, \nabla_\theta \log \pi_\theta(A \mid S)^\top\right]. \tag{8.21}$$

Эта матрица измеряет ожидаемое внешнее произведение векторов функции счёта; она фиксирует, насколько чувствительно распределение π_θ реагирует на малые изменения θ. Эквивалентно, $F(\theta)$ является гессианом KL-дивергенции $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\theta'})$ по θ', вычисленным при $\theta' = \theta$.

Натуральный градиент — это направление наискорейшего подъёма в пространстве распределений, измеренное KL-дивергенцией:

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1} \nabla_\theta J(\theta). \tag{8.22}$$

Обновление по натуральному градиенту стратегии:

$$\theta \leftarrow \theta + \alpha\, F(\theta)^{-1} \nabla_\theta J(\theta). \tag{8.23}$$

Теорема 8.7 (Амари, 1998; Какаде, 2002)**.** *Натуральный градиент $\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1}\nabla_\theta J(\theta)$ является направлением наискорейшего подъёма для J при ограничении, что KL-дивергенция между старой и новой стратегией удовлетворяет $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\theta+\Delta\theta}) \le \epsilon$, для малого ϵ.*

Набросок доказательства. Задача условной оптимизации разложения Тейлора первого порядка $J(\theta + \Delta\theta) \approx J(\theta) + \nabla_\theta J(\theta)^\top \Delta\theta$ при ограничении $\Delta\theta^\top F(\theta)\Delta\theta \le \epsilon$ (ограничение в форме шара KL в пространстве параметров) даёт по неравенству Коши-Шварца во взвешенном F скалярном произведении оптимальное направление $\Delta\theta^* \propto F(\theta)^{-1}\nabla_\theta J(\theta)$. □

8.7.3 Практические соображения

Прямое вычисление $F(\theta)^{-1}$ требует $O(d^2)$ памяти и $O(d^3)$ времени, что неприемлемо для больших нейронных сетей. На практике используются несколько приближений:

- **Метод сопряжённых градиентов.** Решать $F(\theta)\,\tilde{g} = \nabla_\theta J(\theta)$ итерационно. Каждый шаг требует произведения Фишера на вектор, которое может быть вычислено за $O(d)$ с помощью обратного распространения через функцию счёта. Это подход, используемый в TRPO (Шульман и др., 2015).
- **Кронекерово-факторизованное приближение (K-FAC).** Аппроксимировать $F(\theta)$ как блочно-диагональную матрицу с кронекеровой структурой, используя послойную факторизацию нейронных сетей (Мартенс и Гросс, 2015).
- **Proximal Policy Optimization (PPO).** Вместо явного обращения Фишера, PPO использует ограниченную суррогатную целевую функцию для неявного ограничения KL-дивергенции, достигая аналогичного поведения при значительно меньших затратах (Шульман и др., 2017).

Натуральный градиент стратегии концептуально важен, поскольку обеспечивает теоретическую основу для методов доверительной области и проксимальных методов; его практические реализации являются рабочими лошадками современного глубокого ОП и дообучения LLM.

8.8 Анализ сходимости

8.8.1 Сходимость REINFORCE

REINFORCE — это алгоритм стохастического градиентного подъёма, применённый к $J(\theta)$. Его свойства сходимости следуют из общей теории стохастической аппроксимации (Роббинс и Монро, 1951), при условии, что оценщик градиента несмещён (установлено в теореме 8.3) и размеры шагов удовлетворяют условиям Роббинса-Монро:

$$\sum_{k=1}^{\infty} \alpha_k = \infty, \qquad \sum_{k=1}^{\infty} \alpha_k^2 < \infty. \tag{8.24}$$

Теорема 8.8 (Сходимость REINFORCE)**.** *Предположим, что $\pi_\theta(a \mid s)$ дважды непрерывно дифференцируема по θ, и что градиент $\nabla_\theta \log \pi_\theta$ ограничен. Если размеры шагов удовлетворяют (8.24) и $J(\theta)$ ограничена сверху, то итерации REINFORCE удовлетворяют*

$$\liminf_{k\to\infty} \|\nabla_\theta J(\theta_k)\|^2 = 0 \quad \textit{п.н.}$$

То есть REINFORCE сходится к стационарной точке J.

Теорема гарантирует сходимость к *стационарной точке*, которая может быть локальным максимумом, седловой точкой или (в вырожденных случаях) локальным минимумом. Глобальная оптимальность не гарантируется для невыпуклого J. Однако для *табличных* стратегий (без аппроксимации функции, конечные пространства состояний и действий) целевая функция градиента стратегии не имеет ложных локальных максимумов при мягких условиях (Агарвал и др., 2021), поэтому сходимость к глобальному оптимуму может быть гарантирована.

8.8.2 Скорость сходимости

Для невыпуклого случая современное состояние теории — скорость $\tilde{O}(1/\sqrt{K})$ сходимости к стационарной точке по числу траекторий K:

Теорема 8.9 (Скорость сходимости, неформально)**.** *При стандартных условиях регулярности REINFORCE с постоянным размером шага α находит точку θ с $\|\nabla_\theta J(\theta)\|^2 \leq \epsilon$ не более чем за $O(\sigma^2/(\alpha\epsilon))$ эпизодов, где σ^2 — верхняя оценка дисперсии оценщика градиента.*

Эта скорость раскрывает центральную роль дисперсии: снижение σ^2 (через базовые линии или RLOO) непосредственно снижает сложность по выборкам. При базовой линии, снижающей дисперсию в c раз, требуется в c раз меньше эпизодов для достижения той же точности.

8.8.3 Компромисс между смещением и дисперсией

Все оценщики градиента стратегии предполагают компромисс между смещением и дисперсией в том, как они оценивают $Q^{\pi_\theta}(s, a)$:

- **Отдача Монте-Карло G_t:** несмещённая, но с высокой дисперсией (накапливает шум вознаграждения на протяжении всего эпизода).
- **Цель TD(0) $R_{t+1} + \gamma\hat{V}(S_{t+1})$:** смещённая (из-за ошибки аппроксимации $\hat{V}$), но с низкой дисперсией (только один шаг шума).
- **n-шаговая отдача:** интерполирует между двумя крайностями; увеличение n снижает смещение и увеличивает дисперсию.
- **GAE (обобщённая оценка преимущества, раздел 6.10) с $\lambda \in [0, 1]$:** экспоненциально взвешенное среднее n-шаговых отдач, сочетающее низкое смещение (λ близко к 1) с низкой дисперсией (λ близко к 0).

Оценщик REINFORCE соответствует крайнему случаю $n = T$ (полный Монте-Карло). RLOO сохраняет отдачу Монте-Карло, но снижает дисперсию через базовую линию leave-one-out, тогда как методы актор-критик (глава 9) снижают дисперсию через бутстрэппинг ценой некоторого смещения.

Замечание 8.7. На практике дисперсия обычно является доминирующей проблемой для методов градиента стратегии, особенно в задачах с длинным горизонтом или с большими масштабами вознаграждений. Снижения базовой линии и отдачи с текущего момента вместе часто уменьшают дисперсию на один-два порядка величины по сравнению с наивным оценщиком (8.7), что составляет разницу между сходимостью и расходимостью.

8.9 Сравнение оценщиков градиента стратегии

В таблице 8.1 обобщены свойства оценщиков, разработанных в данной главе. Используются следующие обозначения: $\hat{g}$ обозначает оценщик градиента, b — базовую линию, K — число выборок на промпт/состояние.

Из таблицы 8.1 стоит сделать несколько наблюдений.

Во-первых, все оценщики в столбце «Смещение», отмеченные как «Несмещённые», являются несмещёнными оценщиками $\nabla_\theta J(\theta)$: градиент стратегии является точным градиентом истинной целевой функции. Смещение возникает только при замене отдачи Монте-Карло на бутстрэппинговые цели (как в методах актор-критик) или когда ошибка аппроксимации функции ценности распространяется на оценку преимущества.

Во-вторых, столбец «Критик?» раскрывает фундаментальный выбор дизайна. Методы без критика (REINFORCE, RLOO) проще, но полагаются на отдачу Монте-Карло с высокой дисперсией; методы с критиком (актор-критик, PPO) достигают более низкой дисперсии ценой отдельной сети, дополнительных гиперпараметров (скорость обучения критика, количество эпох обновления) и потенциальной нестабильности при неточном критике.

В-третьих, RLOO достигает лучшего из обоих миров — без критика, но с существенно меньшей дисперсией, чем обычный REINFORCE, — используя естественную пакетную структуру обучения LLM. При одной выборке ($K = 1$) RLOO сводится к обычному REINFORCE (базовая линия LOO при одной выборке равна нулю), поэтому улучшение зависит от наличия $K \geq 2$.

В-четвёртых, натуральный градиент стратегии и его практические потомки (TRPO, PPO) предпочтительны при необходимости больших обновлений параметров, поскольку они инвариантны к перепараметризации стратегии: направление обновления одинаково независимо от математического представления π_θ.

Таблица 8.1: Сравнение оценщиков градиента стратегии.

Оценщик	**Базовая линия**	**Смещ./Дисп.**	**Критик?**	**Примечания**
REINFORCE	Нет	Несмещ./Очень выс.	Нет	Простейший; редко используется отдельно
REINFORCE+b	Фикс. b	Несмещ./Высок.	Нет	Снижает среднее; неоптимален
REINFORCE+$\hat{V}$	$\hat{V}_w(s)$	Несмещ./Средн.	Да	Наиболее распространённая б.л.
RLOO	LOO-среднее	Несмещ./Низк.	Нет	$K \geq 2$ выборки; без критика
Актор-критик	$\hat{V}_w(s)$	Смещ./Низк.	Да	Смещение из-за бутстрэппинга
Натур. ГС	Нет	Несмещ./Средн.	Нет	Лучшая геометрия
TRPO/PPO	$\hat{V}_w(s)$	Смещ./Низк.	Да	KL/отсечение; лучший результат

Упражнения

Упражнение 8.1. Пусть $\pi_\theta(a \mid s) = \mathrm{softmax}(\theta_a)$ — табличная softmax-стратегия над n действиями.

1. Покажите, что $\nabla_{\theta_a} \log \pi_\theta(a' \mid s) = \mathbb{1}[a = a'] - \pi_\theta(a \mid s)$.
2. Проверьте тождество функции счёта: $\sum_a \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) = 0$.
3. Пусть $n = 3$, $\theta = (1, 0, -1)$, и наблюдается действие $a = 2$ с отдачей $G = 5$. Вычислите обновление REINFORCE $\Delta\theta$ с размером шага $\alpha = 0.01$.

Упражнение 8.2. Докажите лемму о причинности (лемма 8.2) непосредственно из определения $p_\theta(\tau)$, не используя теорему о полном математическом ожидании. Подсказка: явно распишите сумму по траекториям и разложите её по шагам времени.

Упражнение 8.3. Рассмотрим однострановой двухдействиийный бандит. Пусть $r(a_1) = 1$ и $r(a_2) = 3$ с вероятностью 1. Пусть $\pi_\theta(a_1) = \sigma(-\theta)$, $\pi_\theta(a_2) = \sigma(\theta)$.

1. Вычислите $J(\theta) = \mathbb{E}[r]$ как функцию от θ.
2. Вычислите $\nabla_\theta J(\theta)$ аналитически.
3. Вычислите дисперсию оценщика градиента REINFORCE (без базовой линии) как функцию от θ.
4. Покажите, что базовая линия $b = \mathbb{E}[r] = J(\theta)$ снижает дисперсию. Вычислите дисперсию с этой базовой линией.
5. Какая базовая линия является оптимальной согласно (8.14)?

Упражнение 8.4. Докажите несмещённость оценщика RLOO непосредственно, не ссылаясь на теорему 8.5. Конкретно, для $K = 2$ дополнений $y^{(1)}, y^{(2)} \sim \pi_\theta(\cdot \mid x)$ с вознаграждениями r_1, r_2, покажите, что

$$\mathbb{E}\Big[(r_1 - r_2)\, \nabla_\theta \log \pi_\theta(y^{(1)} \mid x) + (r_2 - r_1)\, \nabla_\theta \log \pi_\theta(y^{(2)} \mid x)\Big] = 2\, \nabla_\theta J(\theta).$$

Упражнение 8.5. В обозначениях раздела 8.6 пусть $r_1, \ldots, r_K$ — н.о.р. со средним μ и дисперсией σ^2, а $g_1, \ldots, g_K$ — функции счёта (независимые от вознаграждений по тождеству функции счёта, поэтому в данном упражнении считайте их фиксированными). Покажите, что

$$\mathrm{Var}\left[\frac{1}{K}\sum_{i=1}^{K}(r_i - b_i)\, g_i\right] \approx \frac{\sigma^2}{K-1} \cdot \frac{1}{K}\sum_{i=1}^{K} g_i^2$$

при большом K, и сравните это с дисперсией оценщика обычного REINFORCE $\frac{1}{K}\sum_i r_i g_i$. При каких условиях снижение дисперсии наибольшее?

Упражнение 8.6. 1. Пусть π_θ — гауссовская стратегия: $\pi_\theta(a \mid s) = \mathcal{N}(a;\, \mu_\theta(s),\, \sigma^2)$ с фиксированной дисперсией σ^2. Вычислите матрицу информации Фишера $F(\theta)$ через $\nabla_\theta \mu_\theta(s)$.

2. Покажите, что когда $\mu_\theta(s) = \theta^\top \phi(s)$ линейно, натуральный градиент сводится к $\tilde{\nabla}_\theta J = \sigma^2 F_\phi^{-1} \nabla_\theta J$, где F_ϕ — матрица ковариации признаков.

3. Объясните интуитивно, почему обновление по натуральному градиенту инвариантно к перепараметризации стратегии.

Упражнение 8.7. Рассмотрим МПР с непрерывным пространством действий $\mathcal{A} = \mathbb{R}$ и гауссовской стратегией $\pi_\theta(a \mid s) = \mathcal{N}(a;\, \mu_\theta(s),\, e^{2\rho})$, где $\theta = (\theta_\mu, \rho)$ параметризует сеть среднего и логарифмическое стандартное отклонение.

1. Выведите $\nabla_{\theta_\mu} \log \pi_\theta(a \mid s)$ и $\nabla_\rho \log \pi_\theta(a \mid s)$.

2. Интерпретируйте градиент по ρ: когда обновление увеличивает энтропию, а когда уменьшает?

3. Покажите, что обновление REINFORCE по ρ реализует автоматическое регулирование энтропии исключительно через сигнал вознаграждения.

Упражнение 8.8. 1. Сформулируйте условия Роббинса-Монро (8.24) и приведите один пример последовательности размеров шагов, удовлетворяющей им, и один пример, не удовлетворяющей.

2. Рассмотрим REINFORCE с постоянным размером шага $\alpha > 0$. Почему это *не* удовлетворяет условиям Роббинса-Монро, и к чему вместо этого сходится алгоритм?

3. Объясните компромисс между смещением и дисперсией между REINFORCE (отдача Монте-Карло) и методом актор-критик (цель TD) в контексте теоремы 8.9. Как снижение дисперсии через базовую линию влияет на оценку сложности по выборкам?

Упражнение 8.9. В контексте языковой модели стратегия π_θ генерирует последовательности токенов авторегрессионно. Предположим, что мы используем RLOO с $K = 4$ дополнениями на промпт.

1. Выпишите преимущество RLOO $\hat{A}_i = r_i - b_i$ для $i = 1, 2, 3, 4$ по формуле из (8.20).

2. Предположим, что четыре вознаграждения равны $r = (0.2, 0.8, 0.5, 0.3)$. Вычислите все четыре преимущества.

3. Покажите, что $\sum_{i=1}^{4} \hat{A}_i = 0$. Всегда ли это верно? Докажите.

4. Обсудите, почему свойство нулевой суммы преимуществ RLOO (при суммировании по пакету) может быть выгодным или невыгодным на практике.

Упражнение 8.10. Вы разрабатываете систему ОП для дообучения предобученной языковой модели на задаче генерации текста. Вам доступна функция вознаграждения типа «чёрный ящик» и возможность генерировать до $K = 8$ дополнений на промпт параллельно.

1. Сравните REINFORCE с обученной базовой линией функции ценности, RLOO и PPO по следующим измерениям: (a) требования к памяти, (b) количество гиперпараметров, (c) потенциальное смещение в оценке градиента, (d) простота реализации.

2. При каких обстоятельствах вы предпочли бы RLOO вместо PPO? При каких предпочли бы PPO?

3. Предложите простой эксперимент для эмпирического определения оптимального K для RLOO в вашем контексте.

Глава 9

Методы актор-критик и Proximal Policy Optimization

Идея наличия двух взаимодействующих компонентов — одного для поведения и одного для критики — кажется столь же старой, как и сама идея обучения с подкреплением. —Ричард С. Саттон и Эндрю Дж. Барто, *Reinforcement Learning: An Introduction*

В главе 8 была разработана теорема о градиенте стратегии и алгоритм REINFORCE, установившие, что можно напрямую оптимизировать параметризованную стратегию, следуя градиенту ожидаемой отдачи. REINFORCE концептуально элегантен, но страдает от фундаментальной статистической слабости: его оценщик градиента использует полную дисконтированную отдачу G_t в качестве сигнала для каждого совершённого в эпизоде действия. Поскольку G_t является суммой многих случайных вознаграждений, простирающейся до конца эпизода, его дисперсия растёт с горизонтом, что приводит к медленному и зашумлённому обучению.

Предлагаемое в данной главе средство — заменить G_t оценкой с меньшей дисперсией того, насколько конкретное действие было лучше среднего — *функцией преимущества* — и обучать эту оценку онлайн с помощью второй нейронной сети, называемой *критиком*. Возникающий класс алгоритмов известен как методы **актор-критик**. Актор (стратегия) и критик (функция ценности) взаимодействуют на каждом шаге: критик оценивает выборы актора, а актор использует эту оценку для обновления своих параметров.

На этой основе мы строим всё более мощные алгоритмы. Обобщённая оценка преимущества (GAE) обеспечивает гибкий компромисс между смещением и дисперсией для сигнала преимущества. Асинхронный актор-критик с преимуществом (A3C) использует параллельных работников для декорреляции опыта и ускорения обучения. Оптимизация стратегии с доверительной областью (TRPO) вводит ключевое понятие о том, что обновления стратегии должны оставаться близкими к текущей стратегии. Proximal Policy Optimization (PPO) достигает того же эффекта с помощью простого механизма отсечения, который работает быстро, легко реализуется и оказывается замечательно устойчивым на практике. Наконец, мы изучаем, как PPO применяется для дообучения больших языковых моделей с учётом обратной связи от людей — парадигма, известная как RLHF.

9.1 Функция преимущества

9.1.1 Определение и основные свойства

Вспомним из главы 8, что теорема о градиенте стратегии выражает градиент целевой функции через функцию ценности действий:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim d^{\gamma}_{\pi_\theta},\, A\sim\pi_\theta(\cdot|S)}\big[\nabla_\theta \log \pi_\theta(A \mid S)\; Q^{\pi_\theta}(S,A)\big]. \tag{9.1}$$

Глава 8 также показала, что любая *зависящая от состояния* базовая линия $b(S_t)$ может быть вычтена из $Q^{\pi_\theta}(S_t, A_t)$ без изменения ожидаемого градиента (теорема 8.5 в главе 8). Наилучшей возможной базовой линией является та, которая максимально снижает дисперсию, и при мягких условиях оптимальным выбором является в точности функция ценности состояния $V^{\pi_\theta}(s)$. Остаток после вычитания этой базовой линии настолько важен, что заслуживает собственного имени.

Определение 9.1 (Функция преимущества)**.** Для стратегии π и порождённых ею функций ценности действий и состояний Q^π и V^π, **функция преимущества** определяется как

$$A^\pi(s,a) \;=\; Q^\pi(s,a) - V^\pi(s). \tag{9.2}$$

Интуитивно, $A^\pi(s,a)$ измеряет относительное качество действия a по сравнению со средним действием стратегии в состоянии s. Положительное преимущество означает, что a лучше среднего; отрицательное — что хуже. Это свойство центрирования формализуется следующим утверждением.

Утверждение 9.1 (Нулевое среднее преимущества)**.** *Для любой стратегии π и любого состояния $s \in \mathcal{S}$,*

$$\mathbb{E}_{A\sim\pi(\cdot|s)}\big[A^\pi(s,A)\big] = 0. \tag{9.3}$$

Доказательство. По определению 9.1 и линейности математического ожидания,

$$\mathbb{E}_{A\sim\pi(\cdot|s)}\big[A^\pi(s,A)\big] = \mathbb{E}_{A\sim\pi(\cdot|s)}\big[Q^\pi(s,A)\big] - V^\pi(s).$$

Уравнение Беллмана (глава 3, уравнение (3.12)) даёт $V^\pi(s) = \sum_a \pi(a|s)\,Q^\pi(s,a) = \mathbb{E}_{A\sim\pi(\cdot|s)}[Q^\pi(s,A)]$, поэтому правая часть равна нулю. □

Подстановка $V^{\pi_\theta}(S_t)$ вместо базовой линии в формулу градиента стратегии (9.1) даёт **форму через преимущество** теоремы о градиенте стратегии:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim d^\gamma_{\pi_\theta},\; A\sim\pi_\theta(\cdot|S)}\big[\nabla_\theta \log \pi_\theta(A \mid S)\; A^{\pi_\theta}(S,A)\big]. \tag{9.4}$$

Это версия теоремы о градиенте стратегии, лежащая в основе всех алгоритмов актор-критик.

9.1.2 Почему преимущество снижает дисперсию

Чтобы конкретно увидеть, почему использование $A^\pi(s,a)$ вместо $Q^\pi(s,a)$ помогает, рассмотрим простой пример.

Пример 9.1 (Преимущество против сырой отдачи)**.** Предположим, что агент находится в состоянии s, где каждое действие даёт высокую отдачу, поскольку s уже является очень хорошим состоянием. Конкретно, пусть $Q^\pi(s,a) \in \{8,10,12\}$ с равными вероятностями для трёх действий, так что $V^\pi(s) = 10$. Оценки сырой функции ценности действий имеют среднее 10 и дисперсию $\mathrm{Var}[Q^\pi(s,\cdot)] = \frac{4+0+4}{3} \approx 2.67$.

Оценки преимущества равны $A^\pi(s,a) \in \{-2,0,2\}$, что имеет нулевое среднее (согласно утверждению 9.1) и ту же дисперсию 2.67. На первый взгляд дисперсия не изменилась, но обратите внимание на принципиальное различие: поскольку обновление градиента стратегии равно $\nabla_\theta \log \pi_\theta(a|s) \cdot A^\pi(s,a)$, обновления для посредственного действия ($A = 0$) равны нулю независимо от параметров стратегии. Использование сырого Q давало бы ненулевой градиентный сигнал величиной 10 даже для нейтрального действия, засоряя обновление большим глобальным смещением, которое необходимо компенсировать на множестве выборок. Преимущество устраняет это смещение идеально и в ожидании. □

Более формально можно показать, что дисперсия оценщика градиента стратегии с оптимальной базовой линией $b^*(s)$ удовлетворяет

$$\mathrm{Var}\big[\nabla_\theta \log \pi_\theta(A|S)\,(Q^\pi(S,A) - b^*(S))\big] \;\le\; \mathrm{Var}\big[\nabla_\theta \log \pi_\theta(A|S)\,Q^\pi(S,A)\big], \tag{9.5}$$

причём снижение может быть значительным в средах, где функция ценности существенно варьируется по состояниям, но вариация преимущества внутри каждого состояния невелика.

9.2 Архитектура актор-критик

9.2.1 Двухкомпонентный дизайн

Алгоритм актор-критик поддерживает два взаимодействующих компонента.

Определение 9.2 (Актор и критик)**.** Алгоритм **актор-критик** поддерживает:

- **актора**: параметризованную стратегию $\pi_\theta(a \mid s)$, чьи параметры θ обновляются для максимизации $J(\theta)$;
- **критика**: параметризованную функцию ценности $V_\phi(s)$, чьи параметры ϕ обновляются для аппроксимации $V^{\pi_\theta}(s)$.

Название отражает метафору исполнителя (актора) и рецензента (критика): после каждого действия критик сигнализирует, было ли это действие выше или ниже среднего, а актор корректируется соответственно.

На рисунке 9.1 показан поток информации. Среда выдаёт состояние S_t; актор выбирает действие A_t; среда переходит в S_{t+1} и выдаёт вознаграждение R_{t+1}. Критик использует (S_t, R_{t+1}, S_{t+1}) для вычисления ошибки TD δ_t, которая служит одновременно и сигналом обучения критика (он измеряет, насколько ошибочным было предсказание самого критика), и оценкой преимущества для управления обновлением актора.

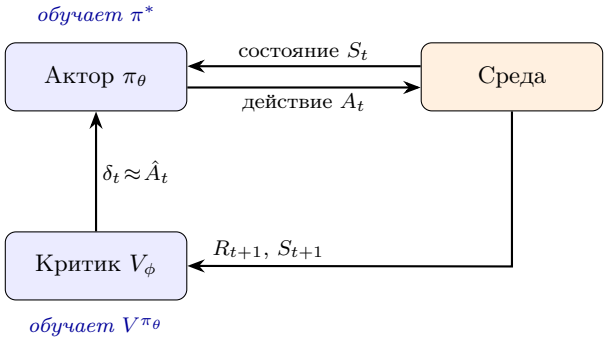


Рис. 9.1: Поток информации в алгоритме актор-критик. Актор выбирает действия; среда возвращает вознаграждения и следующие состояния; критик вырабатывает ошибку TD δ_t, направляющую обновления как критика, так и актора.

9.2.2 Раздельные или совместные сети

На практике актор и критик могут быть реализованы двумя способами. Если они являются **отдельными сетями**, актор π_θ и критик V_ϕ имеют полностью несвязанные параметры, что делает два целевых объекта обучения независимыми и часто проще поддающимися настройке. Если они **разделяют общий ствол** (общий набор скрытых слоёв), две головы разделяют представление, сокращая общее число параметров и стимулируя скрытые слои извлекать признаки, полезные как для стратегии, так и для оценки ценности. Общие стволы являются стандартом в современных реализациях, особенно для сложных наблюдаемых пространств — таких как изображения или токены естественного языка — где обучение хороших представлений само по себе является серьёзной задачей.

9.3 Ошибка TD как оценка преимущества

9.3.1 Одношаговая оценка

Хотя форма через преимущество (9.4) теоретически чиста, она требует знания как Q^{π_θ}, так и V^{π_θ}. Вычислять оба избыточно: поскольку $Q^\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = a]$, можно получить преимущество только через функцию ценности, используя наблюдаемый переход как выборку. Полученная величина является в точности одношаговой ошибкой TD, введённой в главе 6.

Теорема 9.2 (Ошибка TD является несмещённой оценкой преимущества)**.** *Пусть $V_\phi = V^{\pi_\theta}$ (критик идеален). Тогда для любого перехода $(S_t, A_t, R_{t+1}, S_{t+1})$, выборочного согласно π_θ,*

$$\mathbb{E}\big[\delta_t \;\big|\; S_t = s,\ A_t = a\big] \;=\; A^{\pi_\theta}(s,a), \tag{9.6}$$

где $\delta_t = R_{t+1} + \gamma V^{\pi_\theta}(S_{t+1}) - V^{\pi_\theta}(S_t)$.

Доказательство. Взяв условное математическое ожидание δ_t при $S_t = s$ и $A_t = a$:

$$\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = \mathbb{E}\big[R_{t+1} + \gamma V^{\pi_\theta}(S_{t+1}) \mid S_t = s, A_t = a\big] - V^{\pi_\theta}(s).$$

По уравнению Беллмана для Q^{π_θ} (глава 3),

$$\mathbb{E}\big[R_{t+1} + \gamma V^{\pi_\theta}(S_{t+1}) \mid S_t = s, A_t = a\big] = Q^{\pi_\theta}(s,a),$$

что следует из того, что марковское свойство позволяет разложить $\mathbb{E}[G_{t+1} \mid S_t, A_t] = \mathbb{E}[\mathbb{E}[G_{t+1} \mid S_{t+1}] \mid S_t, A_t] = \mathbb{E}[V^{\pi_\theta}(S_{t+1}) \mid S_t, A_t]$. Поэтому

$$\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s) = A^{\pi_\theta}(s,a). \qquad \square$$

Теорема 9.2 является ключевым результатом, делающим алгоритмы актор-критик практичными. Вместо поддержки отдельных аппроксиматоров функций для Q^π и V^π, достаточно лишь обучить V_ϕ; ошибка TD δ_t тогда является одновыборочной несмещённой оценкой преимущества.

Замечание 9.1. Несмещённость выполняется только при $V_\phi = V^{\pi_\theta}$. На практике критик несовершенен, поэтому ошибка TD вносит *смещение* в оценку преимущества. Компромисс между смещением и дисперсией является центральным противоречием в методах актор-критик: хорошо обученный критик существенно снижает дисперсию, но вносит систематические ошибки. Это то же противоречие, которое отличает TD(0) от Монте-Карло в главе 6.

9.3.2 Одношаговый алгоритм актор-критик

Подстановка δ_t в градиент стратегии в форме преимущества даёт простейшее обновление актор-критик с обучением на политике.

Алгоритм 22: Одношаговый актор-критик с обучением на политике (критик TD(0))

Input: Сеть стратегии π_θ, сеть ценности V_ϕ, размеры шагов $\alpha_\theta, \alpha_\phi > 0$, дисконт γ

Инициализировать θ и ϕ случайно
for *каждый эпизод* **do**
  Наблюдать начальное состояние S_0
  for $t = 0, 1, 2, \ldots$ *до терминального состояния* **do**
    Выбрать действие $A_t \sim \pi_\theta(\cdot \mid S_t)$
    Наблюдать R_{t+1} и S_{t+1}
    Вычислить ошибку TD: $\delta_t \leftarrow R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$
    `// Обновление критика: полуградиент TD(0)`
    $\phi \leftarrow \phi + \alpha_\phi \, \delta_t \, \nabla_\phi V_\phi(S_t)$
    `// Обновление актора: градиент стратегии с оценкой преимущества через ошибку TD`
    $\theta \leftarrow \theta + \alpha_\theta \, \gamma^t \, \delta_t \, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$

Обновление критика в алгоритме 22 является в точности полуградиентным обновлением TD(0), выведенным в главе 6 (уравнение (6.18)). Обновление актора заменяет полную отдачу G_t REINFORCE на ошибку TD δ_t, которая доступна сразу после каждого шага и имеет значительно меньшую дисперсию. Принципиально важно, что ни одно из обновлений не требует ожидания конца эпизода, что делает методы актор-критик применимыми к непрерывным (не эпизодическим) задачам.

Пример 9.2 (CartPole с одношаговым актор-критиком)**.** Рассмотрим среду CartPole, где состоянием является четырёхмерный вектор (положение тележки, скорость тележки, угол шеста, угловая скорость шеста), действие является бинарным (толкнуть влево или вправо), а эпизод завершается, когда шест падает или тележка выезжает за пределы трассы.

Актор — это двухслойная сеть с выходом softmax для двух действий; критик — двухслойная сеть, выдающая скаляр $V_\phi(s)$. После шага, на котором шест почти упал, а затем восстановился,

$$R_{t+1} = 1, \quad V_\phi(S_t) = 4.2, \quad V_\phi(S_{t+1}) = 3.1,$$

поэтому $\delta_t = 1 + 0.99 \times 3.1 - 4.2 = -0.131$. Отрицательная ошибка TD сигнализирует, что состояние S_t было переоценено — агент действует хуже, чем ожидалось, — и обновление актора слегка снижает вероятность действия, совершённого в S_t. Напротив, на шаге, где шест стабилен и критик недооценил ситуацию, $\delta_t > 0$ повышает вероятность стабилизирующего действия. Эта немедленная, пошаговая обратная связь отличает актор-критик от REINFORCE. □

9.4 Обобщённая оценка преимущества

9.4.1 Многошаговые отдачи и n-шаговое преимущество

Одношаговая оценка преимущества δ_t имеет низкую дисперсию, но смещена, когда $V_\phi \neq V^{\pi_\theta}$. На другом крайнем случае, оценка преимущества Монте-Карло $G_t - V_\phi(S_t)$ является несмещённой оценкой преимущества, но имеет высокую дисперсию, поскольку суммирует все будущие вознаграждения. Глава 6 показала, что n-шаговые отдачи интерполируют между этими полюсами: n-шаговая цель TD равна

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_\phi(S_{t+n}).$$

Соответствующая n-шаговая оценка преимущества:

$$\hat{A}_t^{(n)} = G_t^{(n)} - V_\phi(S_t) = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n V_\phi(S_{t+n}) - V_\phi(S_t). \tag{9.7}$$

При $n \to \infty$ $\hat{A}_t^{(n)} \to G_t - V_\phi(S_t)$ — оценке преимущества Монте-Карло. При $n = 1$ получаем δ_t.

Вместо фиксации конкретного n, Обобщённая Оценка Преимущества (GAE), введённая Шульманом и др. (2016), берёт экспоненциально взвешенное среднее всех n-шаговых оценок.

9.4.2 Определение и вывод GAE

Определение 9.3 (Обобщённая оценка преимущества)**.** Для параметров $\gamma \in (0, 1)$ и $\lambda \in [0, 1]$ определим ошибки TD

$$\delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t),$$

и **оценку преимущества GAE** как

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \, \delta_{t+l}. \tag{9.8}$$

Формула (9.8) выглядит как экспоненциально затухающая сумма одношаговых ошибок TD, начиная с момента t. Она эквивалентна TD(λ) для оценки ценности, применённому здесь для оценки преимущества. Следующее утверждение явно устанавливает связь с n-шаговыми преимуществами.

Утверждение 9.3 (GAE как взвешенное среднее n-шаговых преимуществ)**.** *Оценка GAE равна нормализованной на* $(1 - \lambda)$ *смеси всех* n*-шаговых оценок преимущества:*

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1 - \lambda) \sum_{n=1}^{T-t} \lambda^{n-1} \, \hat{A}_t^{(n)}. \tag{9.9}$$

Доказательство. Раскроем $\hat{A}_t^{(n)}$ из (9.7) через ошибки TD. Заметим, что

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n V_\phi(S_{t+n}) - V_\phi(S_t).$$

Добавляя и вычитая $\gamma^l V_\phi(S_{t+l})$ для каждого промежуточного шага,

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \big(R_{t+l+1} + \gamma V_\phi(S_{t+l+1}) - V_\phi(S_{t+l})\big) = \sum_{l=0}^{n-1} \gamma^l \, \delta_{t+l}.$$

Теперь формируем взвешенную сумму:

$$\begin{aligned}
(1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)} &= (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{l=0}^{n-1} \gamma^l \, \delta_{t+l} \\
&= (1 - \lambda) \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l} \underbrace{\sum_{n=l+1}^{\infty} (1 - \lambda) \lambda^{n-1-l}}_{=1} \\
&= \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l},
\end{aligned}$$

где перестановка порядка суммирования допустима для конечно-горизонтных эпизодов (внутренняя сумма конечна), а заключённый в скобки геометрический ряд суммируется до 1, поскольку $(1 - \lambda) \sum_{k=0}^{\infty} \lambda^k = 1$. Усечение до $l = T - t - 1$ даёт (9.9). □

9.4.3 Параметр λ: компромисс между смещением и дисперсией

Роль λ состоит в интерполяции между двумя крайними случаями.

- $\lambda = 0$: В (9.8) выживает только первый член, поэтому $\hat{A}_t^{\mathrm{GAE}(\gamma,0)} = \delta_t$. Это одношаговая ошибка TD — минимальная дисперсия, но смещённая, когда $V_\phi \neq V^{\pi_\theta}$.
- $\lambda = 1$: Сумма простирается до конца эпизода и $\hat{A}_t^{\mathrm{GAE}(\gamma,1)} = \sum_{l=0}^{T-t-1} \gamma^l \delta_{t+l} = G_t - V_\phi(S_t)$ — оценка преимущества Монте-Карло. Это несмещённо (для любого V_ϕ), но с максимальной дисперсией.

При промежуточных λ эффективный горизонт убывает экспоненциально: члены δ_{t+l} взвешиваются с весом $(\gamma\lambda)^l$, поэтому значимо вносят вклад лишь примерно первые $1/(1-\gamma\lambda)$ шагов. Значения $\lambda \in [0.90, 0.97]$ при $\gamma \approx 0.99$ типичны в современных реализациях, обеспечивая хороший баланс для эпизодов умеренной длины.

На рисунке 9.2 визуализируется, как изменяется эффективный вес при ошибке TD на шаге l в зависимости от λ.

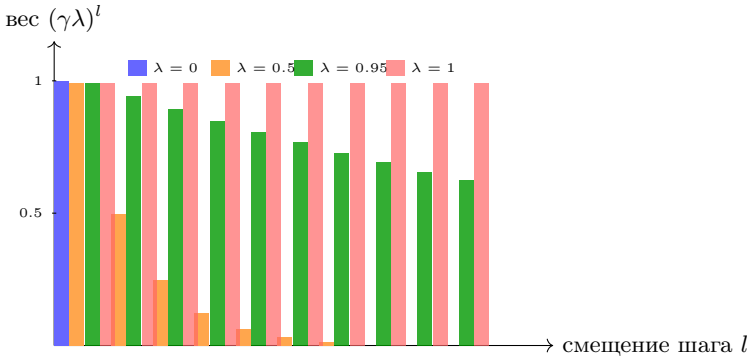


Рис. 9.2: Эффективные веса $(\gamma\lambda)^l$ при ошибках TD с шаговым смещением l в GAE, для $\gamma = 0.99$ и четырёх значений λ. При $\lambda = 0$ только непосредственная ошибка TD вносит вклад; при $\lambda = 1$ все шаги получают почти равномерный вес (убывающий лишь с коэффициентом $\gamma^l \approx 1$). Промежуточные λ дают мягкое экспоненциальное убывание.

9.4.4 Цель GAE для критика

GAE также предоставляет естественную цель для обучения критика. Поскольку $\hat{A}_t^{\mathrm{GAE}} = \hat{V}_t - V_\phi(S_t)$, где

$$\hat{V}_t = \hat{A}_t^{\mathrm{GAE}} + V_\phi(S_t), \tag{9.10}$$

критик может быть обучен путём минимизации среднеквадратичной ошибки $\mathbb{E}[(V_\phi(S_t) - \hat{V}_t)^2]$ при фиксированной цели $\hat{V}_t$. Это создаёт согласованную пару: актор использует $\hat{A}_t^{\mathrm{GAE}}$ для градиента стратегии, а критик использует $\hat{V}_t$ как цель регрессии.

9.5 A2C и A3C

9.5.1 Параллелизм как инструмент снижения дисперсии

Одной из практических трудностей алгоритмов актор-критик с обучением на политике является то, что последовательные переходы $\{(S_t, A_t, R_{t+1}, S_{t+1})\}$, собранные одним агентом, сильно *коррелированы*: агент следует единственной цепи Маркова, поэтому соседние выборки имеют схожие состояния. Высокая корреляция увеличивает эффективную дисперсию оценки градиента даже при низкой дисперсии отдельных выборок.

Мних и др. (2016) предложили **Асинхронный Актор-Критик с Преимуществом** (A3C) в качестве решения. Идея состоит в запуске нескольких независимых *рабочих* процессов, каждый со своей копией среды и локальной копией параметров сети. Рабочие генерируют опыт независимо и асинхронно отправляют обновления градиента центральному серверу параметров, который агрегирует обновления.

Определение 9.4 (Архитектура A3C)**.** В A3C K рабочих процессов выполняются параллельно. Каждый рабочий k:

1. Копирует глобальные параметры (θ, ϕ) в локальные копии (θ_k, ϕ_k).
2. Собирает короткую траекторию из $t_{\max}$ переходов с использованием π_{θ_k}.
3. Вычисляет градиенты актора и критика с помощью оценки преимущества $\hat{A}_t$ из (9.7).
4. Асинхронно применяет градиенты к глобальным (θ, ϕ).

Поскольку разные рабочие, как правило, оказываются в разных состояниях и исследуют разные части пространства состояний одновременно, их градиенты почти некоррелированы даже без буфера воспроизведения опыта. Эта декорреляция является ключевым преимуществом асинхронного параллелизма.

9.5.2 A2C: синхронный вариант

Актор-Критик с Преимуществом (A2C) — это синхронная версия A3C. Все K рабочих одновременно собирают пакет переходов, и обновление параметров выполняется только после того, как все рабочие отчитались. Поскольку обновления синхронны, нет несоответствия между параметрами, используемыми для генерации данных, и теми, которые обновляются.

Алгоритм 23: Актор-Критик с Преимуществом (A2C)

Input: K параллельных рабочих, длина прокрутки T_{roll}, дисконт γ, параметр GAE λ
Инициализировать глобальные θ, ϕ
repeat
  for *каждый рабочий* $k = 1, \ldots, K$ параллельно **do**
    Запустить π_θ на T_{roll} шагах; собрать $\{(S_t^k, A_t^k, R_{t+1}^k)\}_{t=0}^{T_{\text{roll}}-1}$
    Вычислить $\hat{A}_t^k$ через GAE (9.8)
    Вычислить $\hat{V}_t^k$ через (9.10)
  Агрегировать градиенты по всем рабочим
  Обновить θ с помощью градиента актора $\frac{1}{KT_{\text{roll}}} \sum_k \sum_t \hat{A}_t^k \nabla_\theta \log \pi_\theta(A_t^k | S_t^k)$
  Обновить ϕ минимизацией $\frac{1}{KT_{\text{roll}}} \sum_k \sum_t (V_\phi(S_t^k) - \hat{V}_t^k)^2$
until *сходимость*

На практике A2C достигает производительности, сопоставимой с A3C, и проще воспроизводимо реализуется, поскольку синхронный дизайн устраняет состояния гонки. Современные пайплайны глубокого ОП почти повсеместно используют синхронное пакетирование в стиле A2C в качестве основы, дополняемой ограничениями доверительной области, разработанными в следующем разделе.

9.6 Методы доверительной области

9.6.1 Повторное использование данных с другой политики через взвешивание по важности

Методы с обучением на политике, такие как A2C, собирают пакет переходов и затем немедленно отбрасывают их после одного шага градиента. Это расточительно: каждое взаимодействие со средой обходится дорого (по времени симуляции, вычислениям или реальным затратам), но ограничение обучения на политике вынуждает отбрасывать каждую выборку после изменения параметров.

Естественным средством является взвешивание по важности (введённое в главе 5 в контексте Монте-Карло с данными другой политики). Предположим, что данные были собраны с использованием *старой* стратегии $\pi_{\theta_{\text{old}}}$, и мы хотим оценить или оптимизировать *новую* стратегию π_θ. Определим пошаговое **отношение важности**:

$$r_t(\theta) = \frac{\pi_\theta(A_t \mid S_t)}{\pi_{\theta_{\text{old}}}(A_t \mid S_t)}. \tag{9.11}$$

По стандартному взвешиванию по важности, градиент стратегии $\mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(A|S)\, A^\pi(S, A)]$ можно переписать, при приближении, что распределение состояний, индуцированное π_θ, близко к распределению $\pi_{\theta_{\text{old}}}$, как

$$L(\theta) = \mathbb{E}_{\pi_{\theta_{\text{old}}}}\big[r_t(\theta)\, \hat{A}_t\big]. \tag{9.12}$$

Функция $L(\theta)$ называется **суррогатной целевой функцией**. Её можно оптимизировать с использованием данных из $\pi_{\theta_{\text{old}}}$, что позволяет выполнять несколько шагов градиента на одном пакете.

Утверждение 9.4 (Суррогатный градиент равен градиенту стратегии при θ_{old})**.**

$$\nabla_\theta L(\theta)\big|_{\theta=\theta_{\text{old}}} = \nabla_\theta J(\theta)\big|_{\theta=\theta_{\text{old}}}. \tag{9.13}$$

Доказательство. При $\theta = \theta_{\text{old}}$, $r_t(\theta) = 1$ для всех переходов. Дифференцируя $L(\theta) = \mathbb{E}[r_t(\theta)\,\hat{A}_t]$, получаем

$$\nabla_\theta L(\theta)\big|_{\theta_{\text{old}}} = \mathbb{E}\big[\nabla_\theta r_t(\theta)\big|_{\theta_{\text{old}}} \cdot \hat{A}_t\big].$$

Поскольку $\nabla_\theta r_t(\theta) = \frac{\nabla_\theta \pi_\theta(A_t|S_t)}{\pi_{\theta_{\text{old}}}(A_t|S_t)}$, при $\theta = \theta_{\text{old}}$ это равно $\nabla_\theta \log \pi_\theta(A_t|S_t)|_{\theta_{\text{old}}}$, что даёт градиент стратегии. □

9.6.2 Опасность больших обновлений

Утверждение 9.4 показывает, что суррогатная целевая функция локально корректна, но что происходит, когда θ значительно отклоняется от θ_{old}?

Утверждение 9.5 (Граница монотонного улучшения)**.** *Существует константа $C > 0$, зависящая от γ и $\max_{s,a} |A^{\pi_\theta}(s,a)|$, такая что*

$$J(\theta) \geq L(\theta) - C \cdot \max_s D_{\text{KL}}(\pi_\theta(\cdot|s) \parallel \pi_{\theta_{\text{old}}}(\cdot|s))\,. \tag{9.14}$$

Граница (9.14), принадлежащая Шульману и др. (2015), говорит, что $J(\theta)$ ограничена снизу $L(\theta)$ минус член, пропорциональный KL-дивергенции новой стратегии от старой. Если KL мала, суррогат $L(\theta)$ является надёжной аппроксимацией $J(\theta)$, поэтому максимизация L гарантирует улучшение J. Если KL велика, граница неинформативна: суррогат может расти, пока истинная целевая функция падает.

Это фундаментальная мотивация подхода доверительной области: мы должны двигать θ только в области, где KL-дивергенция от θ_{old} достаточно мала, чтобы суррогат был надёжным.

9.6.3 Trust Region Policy Optimization (TRPO)

Trust Region Policy Optimization (TRPO), введённый Шульманом и др. (2015), решает задачу оптимизации с ограничениями:

$$\max_\theta \; L(\theta) \quad \text{при ограничении} \quad \max_s \; D_{\text{KL}}(\pi_\theta(\cdot|s) \parallel \pi_{\theta_{\text{old}}}(\cdot|s)) \leq \delta, \tag{9.15}$$

где $\delta > 0$ — гиперпараметр, управляющий размером доверительной области. На практике максимум по состояниям заменяется эмпирическим средним.

Решение (9.15) требует вычисления матрицы информации Фишера (гессиана ограничения KL) и решения задачи квадратичного программирования с ограничениями. Это приближённо выполняется методом сопряжённых градиентов с поиском по направлению, однако процедура вычислительно тяжела: она требует нескольких произведений гессиана на вектор за обновление, что плохо масштабируется для сетей с миллионами параметров. TRPO теоретически хорошо обоснован, но редко используется на практике для больших моделей.

PPO, разработанный далее, предоставляет значительно более простое приближение к той же идее.

9.7 Proximal Policy Optimization

9.7.1 От доверительных областей к отсечению

Proximal Policy Optimization (PPO), введённый Шульманом и др. (2017), заменяет явное ограничение KL TRPO простой операцией отсечения отношения важности. Интуиция состоит в том, что $r_t(\theta) = 1$ при $\theta = \theta_{\text{old}}$, и что отклонение $r_t(\theta)$ далеко от 1 соответствует большому изменению стратегии. PPO непосредственно предотвращает большие отклонения, ограничивая r_t интервалом $[1-\varepsilon, 1+\varepsilon]$ для гиперпараметра $\varepsilon > 0$ (обычно $\varepsilon \in [0.1, 0.3]$).

Определение 9.5 (Отсечённая целевая функция PPO)**. Отсечённая суррогатная целевая функция PPO** определяется как

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\; \text{clip}\big(r_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon\big)\,\hat{A}_t\Big)\Big]\,, \tag{9.16}$$

где $\text{clip}(x, a, b) = \max(a, \min(b, x))$.

9.7.2 Анализ случаев механизма отсечения

Теорема 9.6 (Корректность отсечения PPO)**.** *Для каждого перехода отсечённая целевая функция L^{CLIP} обладает следующими свойствами:*

(i) Если $\hat{A}_t > 0$: L^{CLIP} неубывает по $r_t(\theta)$ при $r_t \le 1+\varepsilon$ и постоянна при $r_t > 1+\varepsilon$. Градиент равен нулю при $r_t > 1+\varepsilon$.

(ii) Если $\hat{A}_t < 0$: L^{CLIP} невозрастает по $r_t(\theta)$ при $r_t \ge 1-\varepsilon$ и постоянна при $r_t < 1-\varepsilon$. Градиент равен нулю при $r_t < 1-\varepsilon$.

(iii) При $\theta = \theta_{\text{old}}$ (т. е. $r_t = 1 \in [1-\varepsilon, 1+\varepsilon]$) градиент L^{CLIP} равен градиенту стратегии.

Доказательство. Зафиксируем переход и запишем $r = r_t(\theta)$, $A = \hat{A}_t$. Пусть $f_1(r) = rA$ и $f_2(r) = \text{clip}(r, 1-\varepsilon, 1+\varepsilon) \cdot A$.

Случай (i): $A > 0$. Тогда f_1 строго возрастает по r. При $r \le 1+\varepsilon$ отсечение неактивно: $f_2(r) = rA = f_1(r)$. При $r > 1+\varepsilon$, $f_2(r) = (1+\varepsilon)A$, константа. Таким образом,

$$\min(f_1, f_2) = \begin{cases} rA & r \le 1+\varepsilon, \\ (1+\varepsilon)A & r > 1+\varepsilon. \end{cases}$$

Это неубывающая функция, и её производная по r равна нулю при $r > 1+\varepsilon$. Не существует стимула в виде градиента для увеличения вероятности хорошего действия сверх предела $(1+\varepsilon)$.

Случай (ii): $A < 0$. Тогда f_1 строго убывает. При $r \ge 1-\varepsilon$, $f_2(r) = rA = f_1(r)$. При $r < 1-\varepsilon$, $f_2(r) = (1-\varepsilon)A$, константа, и $f_1(r) > f_2(r)$, поскольку $r < 1-\varepsilon$ и $A < 0$. Поэтому $\min(f_1, f_2) = f_2 = (1-\varepsilon)A$ при $r < 1-\varepsilon$ (постоянна), и $\min(f_1, f_2) = rA$ при $r \ge 1-\varepsilon$. Следовательно:

$$\min(f_1, f_2) = \begin{cases} (1-\varepsilon)A & r < 1-\varepsilon, \\ rA & r \ge 1-\varepsilon. \end{cases}$$

Градиент по r равен нулю при $r < 1-\varepsilon$: нет стимула агрессивно снижать вероятность плохого действия ниже пола $(1-\varepsilon)$.

Случай (iii): При $r = 1 \in (1-\varepsilon, 1+\varepsilon)$ отсечение неактивно в обоих случаях, поэтому $L^{\text{CLIP}} = \mathbb{E}[r_t(\theta)\hat{A}_t] = L(\theta)$ в окрестности θ_{old}. По утверждению 9.4, $\nabla_\theta L^{\text{CLIP}}|_{\theta_{\text{old}}} = \nabla_\theta J(\theta)|_{\theta_{\text{old}}}$. □

Оператор min в определении 9.5 является принципиальным: он делает L^{CLIP} *пессимистической нижней границей* суррогата $r_t(\theta)\hat{A}_t$. Рисунок 9.3 иллюстрирует два случая.

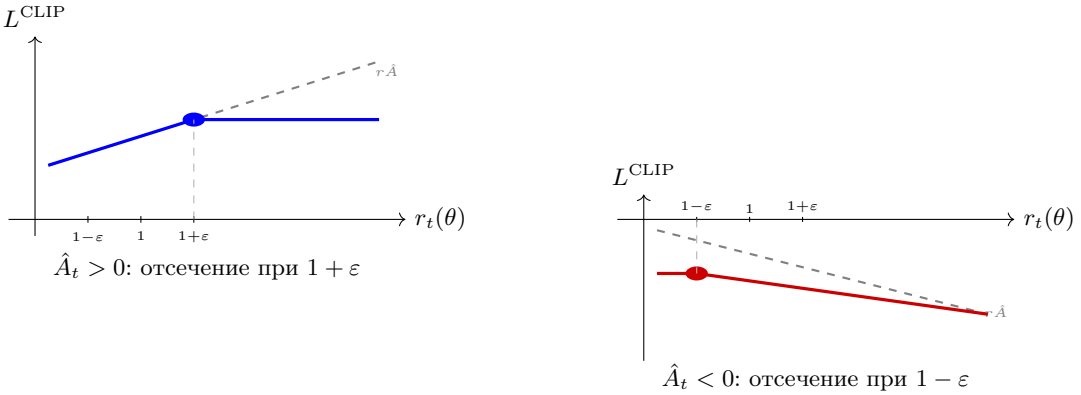


Рис. 9.3: Механизм отсечения PPO. *Слева*: при $\hat{A}_t > 0$ целевая функция ограничена сверху при $r_t = 1+\varepsilon$ (синий), предотвращая чрезмерный рост вероятности действия. *Справа*: при $\hat{A}_t < 0$ целевая функция ограничена снизу при $r_t = 1-\varepsilon$ (красный), предотвращая чрезмерное снижение. В обоих случаях оператор min делает L^{CLIP} пессимистической нижней границей необрезанного суррогата (серый пунктир).

9.7.3 Полная целевая функция PPO

На практике PPO оптимизирует составную целевую функцию, сочетающую отсечённые потери актора с потерями регрессии критика и бонусом за энтропию:

$$\mathcal{L}(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \, \mathbb{E}\big[(V_\phi(S_t) - \hat{V}_t)^2\big] - c_H \, \mathbb{E}\big[-\log \pi_\theta(A_t \mid S_t)\big], \tag{9.17}$$

где $\hat{V}_t$ — цель GAE (9.10), $c_v > 0$ взвешивает потери ценности, а $c_H > 0$ взвешивает бонус за энтропию $H(\pi_\theta(\cdot|S_t))$. Три члена выполняют различные функции:

$-L^{\text{CLIP}}$	Максимизирует целевую функцию стратегии с консервативным ограничением доверительной области.
$c_v \, L^{\text{VF}}$	Обучает критика предсказывать V^{π_θ}, что необходимо для точной оценки преимущества.
$-c_H \, H$	Добавляет бонус за энтропию, препятствующий преждевременному коллапсу стратегии к почти детерминированному распределению, поддерживая исследование в ходе обучения.

Когда актор и критик разделяют слои сети, вся целевая функция (9.17) дифференцируется совместно.

9.7.4 Алгоритм PPO

Алгоритм 24: Proximal Policy Optimization (PPO)

Input: Длина прокрутки T, рабочие K, эпохи K_e, размер мини-пакета M, отсечение ε, λ для GAE
Инициализировать θ, ϕ
repeat
  `// Сбор данных`
  for *каждый из K рабочих параллельно* **do**
    Запустить π_θ на T шагах; сохранить $\{S_t, A_t, R_{t+1}, S_{t+1}\}$
    Вычислить $\hat{A}_t^{\text{GAE}}$ через (9.8) и $\hat{V}_t$ через (9.10)
  Нормализовать преимущества: $\hat{A}_t \leftarrow (\hat{A}_t - \bar{A})/\hat{\sigma}_A$ по всем рабочим
  Зафиксировать $\theta_{\text{old}} \leftarrow \theta$
  `// Несколько эпох оптимизации по собранным данным`
  for *эпоха* $= 1, \ldots, K_e$ **do**
    for *каждый мини-пакет $\mathcal{B}$ размера M* **do**
      Вычислить $r_t(\theta) = \pi_\theta(A_t|S_t)/\pi_{\theta_{\text{old}}}(A_t|S_t)$ для каждого $(S_t, A_t) \in \mathcal{B}$
      Вычислить $\mathcal{L}(\theta, \phi)$ через (9.17)
      Обновить θ, ϕ одним шагом Adam
until *сходимость*

Несколько проектных решений в алгоритме 24 заслуживают комментария.

Замечание 9.2 (Нормализация преимущества). Вычитание среднего по пакету и деление на стандартное отклонение по пакету (шаг «Нормализовать преимущества») является стандартной инженерной практикой. Это поддерживает примерно постоянную величину градиента на протяжении эпизодов независимо от масштаба вознаграждений, снижая чувствительность к скорости обучения и стабилизируя обучение. Это вносит небольшое смещение, поскольку нормализация меняет относительные веса переходов, но эмпирически полезна.

Замечание 9.3 (Несколько эпох). PPO повторно использует каждый пакет для K_e шагов градиента. Это существенно улучшает эффективность использования выборок по сравнению с чистыми методами обучения на политике (где данные отбрасываются после одного шага градиента), тогда как отсечение гарантирует, что стратегия не отклоняется слишком далеко от стратегии сбора данных в рамках одного пакета. Типичные значения: $K_e \in [3, 10]$.

Пример 9.3 (PPO на простом МПР)**.** Рассмотрим МПР в виде цепочки из трёх состояний $\{s_1, s_2, s_3\}$, где s_3 является терминальным. У агента есть два действия (влево/вправо). Предположим, на текущей итерации:

- $\hat{A}_t = 0.8$ (совершённое действие было значительно выше среднего).
- $r_t(\theta) = 1.35$ после шага градиента (новая стратегия назначает на 35% больше вероятности этому действию, чем старая).
- $\varepsilon = 0.2$.

Без отсечения вклад в L составил бы $1.35 \times 0.8 = 1.08$. С отсечением r_t ограничивается до $1 + \varepsilon = 1.2$, давая $1.2 \times 0.8 = 0.96$. Поскольку $0.96 < 1.08$, оператор min выбирает 0.96. Градиент L^{CLIP} по θ равен нулю для этой выборки, потому что $r_t > 1 + \varepsilon$: стратегия уже достаточно сдвинулась в направлении увеличения вероятности этого действия, и PPO не даёт дополнительного градиента для дальнейшего смещения в этом направлении. □

9.8 PPO для языковых моделей

9.8.1 Языковые модели как стратегии ОП

Применение PPO для дообучения больших языковых моделей (LLM) с использованием обратной связи от людей — обычно называемое RLHF — стало одним из наиболее практически важных применений обучения с подкреплением. Глава 11 подробно изучает языковые модели как агентов ОП; здесь мы сосредотачиваемся конкретно на компоненте PPO в пайплайне RLHF.

Языковая модель π_θ определяет авторегрессионную стратегию по токенам. При промпте $x = (x_1, \ldots, x_m)$ модель генерирует ответ $y = (y_1, \ldots, y_T)$ по одному токену за раз:

$$\pi_\theta(y \mid x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid x,\, y_1, \ldots, y_{t-1}). \tag{9.18}$$

Это соответствует МПР, где состоянием на шаге t является $S_t = (x, y_1, \ldots, y_{t-1})$, действием является следующий токен $A_t = y_t \in \mathcal{V}$ (словарь), а эпизод завершается при генерации токена конца последовательности. Пространство действий имеет размер $|\mathcal{V}| \in [32{,}000,\ 128{,}000]$ — значительно больше, чем в классических задачах ОП, — что делает методы на основе функции ценности неприемлемыми, а стратегические методы, такие как PPO, естественными.

9.8.2 Моделирование вознаграждения и штраф KL

В RLHF сигнал вознаграждения поступает от отдельно обученной **модели вознаграждения** $r_\psi(x, y)$, обученной на сравнениях предпочтений людей. В простейшем случае скалярное вознаграждение выдаётся только в конце эпизода: $R_t = 0$ при $t < T$ и $R_T = r_\psi(x, y)$.

Однако без дополнительной регуляризации стратегия быстро научается эксплуатировать модель вознаграждения — явление, называемое **взломом вознаграждения**. Стратегия может производить выходные данные, которые высоко оцениваются по r_ψ, но расходятся от связного, безопасного и фактически точного языка. Стандартным средством является пошаговый штраф за KL-дивергенцию относительно *референсной стратегии* π_{ref} (обычно модели после обучения с учителем до обучения ОП):

$$\tilde{R}_t(x, y) = \begin{cases} r_\psi(x, y) - \beta \log \dfrac{\pi_\theta(y_T \mid x, y_{<T})}{\pi_{\text{ref}}(y_T \mid x, y_{<T})}, & t = T, \\ -\beta \log \dfrac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}, & t < T, \end{cases} \tag{9.19}$$

где $\beta > 0$ управляет силой штрафа KL.

Утверждение 9.7 (Суммарное вознаграждение RLHF)**.** *Сумма скорректированных пошаговых вознаграждений удовлетворяет*

$$\sum_{t=1}^{T} \tilde{R}_t = r_\psi(x, y) - \beta\, D_{\text{KL}}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))\,, \tag{9.20}$$

где KL-дивергенция вычисляется по совместному последовательному распределению.

Доказательство. Суммируя (9.19) по $t = 1, \ldots, T$:

$$\sum_{t=1}^{T} \tilde{R}_t = r_\psi(x, y) - \beta \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}.$$

KL-дивергенция между двумя авторегрессионными распределениями $\pi_\theta(\cdot|x)$ и $\pi_{\text{ref}}(\cdot|x)$ факторизуется как

$$D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)) = \mathbb{E}_{y\sim\pi_\theta}\left[\sum_{t=1}^{T} \log \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\text{ref}}(y_t|x, y_{<t})}\right],$$

поэтому пошаговая выборочная сумма $\sum_t \log[\pi_\theta/\pi_{\text{ref}}]$ является несмещённой оценкой KL методом Монте-Карло. Подстановка даёт (9.20). □

Утверждение 9.7 раскрывает целевую функцию, которую RLHF неявно оптимизирует: максимизировать оценку модели вознаграждения минус взвешенный KL-штраф за отклонение от референсной стратегии. При $\beta \to 0$ штраф исчезает, и стратегия может произвольно отклоняться от π_{ref}, рискуя взломом вознаграждения. При $\beta \to \infty$ оптимальной стратегией является π_{ref}, и обучение ОП не происходит.

9.8.3 Четырёхмодельная архитектура RLHF

Стандартный цикл обучения PPO-RLHF одновременно поддерживает четыре нейронные сети:

Актор π_θ Обучаемая стратегия. Её параметры обновляются на каждой итерации PPO.

Критик V_ϕ Голова функции ценности — обычно один линейный слой, применённый к скрытому состоянию последнего токена — оценивающая ожидаемую накопленную скорректированную отдачу из состояния S_t.

Референсная модель π_{ref} Замороженная копия модели после обучения с учителем, используемая только для вычисления штрафа KL в (9.19).

Модель вознаграждения r_ψ Замороженная модель (обычно той же архитектуры, что и актор, с добавленной скалярной головой), обученная на данных предпочтений людей для выдачи скалярной оценки качества.

Преимущества $\hat{A}_t$ вычисляются через GAE, применённый к скорректированным вознаграждениям $\tilde{R}_t$ из (9.19), а отсечённая целевая функция PPO (9.16) применяется с отношениями важности на уровне словаря $r_t(\theta) = \pi_\theta(y_t|x, y_{<t})/\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})$.

Пример 9.4 (PPO-RLHF на задаче ответа на вопросы)**.** Предположим, промпт — «Объясни последнюю теорему Ферма.», а стратегия генерирует ответ из 120 токенов y. Модель вознаграждения оценивает полный ответ: $r_\psi(x, y) = 0.78$. На каждом шаге $t < 120$ вычисляется пошаговое логарифмическое отношение $\log[\pi_\theta(y_t|x, y_{<t})/\pi_{\text{ref}}(y_t|x, y_{<t})]$; если стратегия умеренно отклонилась от референсной, каждый член может составлять около 0.01, вкладывая суммарный штраф KL $\beta \times 0.01 \times 120 = 1.2\beta$. При $\beta = 0.05$ суммарное скорректированное вознаграждение равно $0.78 - 0.06 = 0.72$.

Критик $V_\phi(S_t)$ предсказывает ожидаемую суммарную скорректированную отдачу из частичной последовательности, видимой до сих пор. После генерации каждого токена вычисляется $\delta_t = \tilde{R}_t + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$, и GAE агрегирует эти ошибки в пошаговые оценки преимущества. Токен вроде «Уайлс» (имея в виду Эндрю Уайлса), который одновременно уместен и специфичен, может повысить предсказание критика, давая положительное преимущество, повышающее вероятность этого токена в будущих обновлениях PPO. □

9.8.4 Практические соображения

Следующие рекомендации дистиллируют эмпирические лучшие практики из литературы по RLHF (Оуян и др. 2022, Бай и др. 2022, Циглер и др. 2019).

Мониторинг и управление KL. Отслеживайте скользящее среднее $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ по пакетам промптов. Быстрый рост сигнализирует об отклонении стратегии от референсной модели. Если KL превышает предустановленный порог (например, 10 натов), либо увеличьте β, уменьшите скорость обучения, либо остановите обучение и диагностируйте модель вознаграждения.

Нормализация преимущества. Нормализуйте $\hat{A}_t$ до нулевого среднего и единичной дисперсии по мини-пакету перед вычислением отсечённой целевой функции. Для генерации языка эпизоды (ответы) имеют различную длину; нормализация предотвращает доминирование длинных последовательностей в обновлениях градиента исключительно из-за накопления большего числа пошаговых членов.

Инициализация критика. Инициализируйте голову функции ценности поверх того же предобученного каркаса, что и актор. Запуск со случайно инициализированным критиком даёт дико неточные оценки преимущества в начале обучения, дестабилизируя актора до того, как он что-либо усвоит.

Количество эпох на пакет. Используйте $K_e \in [1, 4]$ эпох оптимизации на пакет данных для RLHF против $K_e \in [4, 10]$ для классических сред ОП. Поскольку модель вознаграждения может эксплуатироваться (в отличие от настоящей среды), слишком много эпох на одном пакете побуждает актора переобучаться к сигналу модели вознаграждения, ускоряя взлом вознаграждения.

Обнаружение взлома вознаграждения. Периодически оценивайте обученную стратегию на отложенных промптах, оцениваемых людьми (или отдельным независимым оценщиком). Если оценка модели вознаграждения продолжает расти, а оценки людей выравниваются или снижаются, происходит взлом вознаграждения. Меры противодействия включают обновление модели вознаграждения с дополнительными данными предпочтений, увеличение β или включение потерь обучения с учителем наряду с целевой функцией PPO.

9.9 Сходимость и практические соображения

9.9.1 Свойства сходимости

Теория сходимости для методов актор-критик существенно сложнее, чем для чистых методов TD или чистых методов градиента стратегии, поскольку оба компонента обновляются одновременно и их целевые функции взаимодействуют.

В случае *линейной* аппроксимации функции при фиксированной стратегии критик TD(0) сходится к решению проецированного уравнения Беллмана, как установлено в главе 6. Когда актор также обновляется, сходимость требует **двухшкальной стохастической аппроксимации**: размер шага критика α_ϕ должен убывать быстрее, чем шаг актора α_θ, обеспечивая, что критик отслеживает V^{π_θ} быстрее, чем меняется π_θ.

Теорема 9.8 (Сходимость при двух шкалах, набросок)**.** *При следующих условиях:*

(i) линейная аппроксимация функции для критика,

(ii) размеры шагов Роббинса-Монро, удовлетворяющие $\sum_t \alpha_\phi(t) = \infty$, $\sum_t \alpha_\phi(t)^2 < \infty$, и $\alpha_\theta(t)/\alpha_\phi(t) \to 0$,

(iii) обыкновенное дифференциальное уравнение градиента стратегии имеет единственное глобально притягивающее равновесие (или, в более общем случае, итерации остаются ограниченными),

алгоритм актор-критик сходится п.н. к стационарной точке целевой функции $J(\theta)$.

Для *нелинейной* аппроксимации функции (нейронные сети) сопоставимо чёткой теории не существует. На практике актор и критик обновляются одним оптимизатором (например, Adam) с одинаковыми или схожими скоростями обучения, что нарушает требование двух шкал. Тем не менее PPO эмпирически стабилен на широком классе задач, что предполагает, что нейронные сети действуют как собственные эффективные регуляризаторы.

9.9.2 Сравнение алгоритмов градиента стратегии

В таблице 9.1 обобщены ключевые свойства семейства методов градиента стратегии, обсуждавшихся в главах 8 и 9.

Таблица 9.1: Сравнение алгоритмов градиента стратегии. «Обучение на политике» означает, что каждая выборка данных используется не более чем для одного обновления градиента.

Алгоритм	Базовая линия	Онлайн?	Неск. эпох?	Довер. обл.?
REINFORCE	Нет / конст.	Нет	Нет	Нет
REINFORCE + баз. линия	$V_\phi(s)$	Нет	Нет	Нет
Одношаговый АК	Критик TD(0)	Да	Нет	Нет
A2C / A3C	Критик GAE	Да	Нет	Нет
TRPO	Критик GAE	Да	Нет	KL
PPO	Критик GAE	Да	Да	clip ε

9.9.3 Чувствительность к гиперпараметрам и советы по реализации

PPO заметно устойчив к выбору гиперпараметров по сравнению с TRPO, который требует тщательной настройки числа шагов сопряжённых градиентов и параметров поиска по направлению. Тем не менее следующие рекомендации полезны на практике.

Параметр отсечения ε Типичные значения: от 0.1 до 0.3. Меньшее ε даёт более консервативные обновления и безопаснее при дообучении крупных предобученных моделей. $\varepsilon = 0.2$ — наиболее распространённое значение по умолчанию.

GAE λ Значения 0.90–0.97 хорошо работают для большинства задач. Более длинные эпизоды требуют большего λ (для захвата долгосрочного кредита).

Коэффициент ценности c_v Обычно 0.5; можно уменьшить, если сеть функции ценности оптимизируется отдельно или если величины градиентов уже хорошо согласованы.

Коэффициент энтропии c_H Начинать с 0.01; увеличивать, если стратегия преждевременно коллапсирует к почти детерминированному распределению.

Эпохи на пакет K_e Значения 4–8 для непрерывного управления; 1–4 для RLHF. Мониторить KL между π_θ и $\pi_{\theta_{\text{old}}}$ и останавливать эпохи досрочно, если KL превышает $\sim 1.5\varepsilon$.

Размер пакета и мини-пакета Большие пакеты снижают шум градиента, но требуют больше памяти. Прокрутка из 2048 шагов с размером мини-пакета 64 — распространённая отправная точка для непрерывного управления.

Скорость обучения 3×10^{-4} с Adam — устойчивое значение по умолчанию. Линейные или косинусные расписания затухания часто помогают при дообучении.

Нормализация наблюдений Нормализовать наблюдения до нулевого среднего и единичной дисперсии (с накопленной статистикой) и ограничивать вознаграждения фиксированным диапазоном (например, $[-5, 5]$), чтобы критик не был перегружен выбросами вознаграждений.

Упражнения

Упражнение 9.1. Рассмотрим МПР с тремя действиями, доступными в состоянии s. Предположим, $Q^\pi(s, a_1) = 5$, $Q^\pi(s, a_2) = 8$, $Q^\pi(s, a_3) = 2$, и стратегия назначает вероятности $\pi(a_1|s) = 0.5$, $\pi(a_2|s) = 0.3$, $\pi(a_3|s) = 0.2$.

(a) Вычислите $V^\pi(s)$ и преимущество $A^\pi(s, a_i)$ для каждого действия.

(b) Проверьте, что $\mathbb{E}_{A\sim\pi(\cdot|s)}[A^\pi(s, A)] = 0$.

(c) Шаг градиента увеличивает $\pi(a_2|s)$. Объясните, почему использование $A^\pi(s, a_2) = 2$ (вместо $Q^\pi(s, a_2) = 8$) в качестве веса градиента приводит к меньшему, но более информативному обновлению.

Упражнение 9.2. Критик с $V_\phi = V^{\pi_\theta}$ наблюдает переход $(S_t, A_t, R_{t+1}, S_{t+1})$ с $R_{t+1} = 2$, $\gamma = 0.9$, $V^\pi(S_t) = 3$, $V^\pi(S_{t+1}) = 4$.

(a) Вычислите ошибку TD δ_t.

(b) Действие A_t выше или ниже среднего в состоянии S_t? Обоснуйте, используя знак δ_t.

(c) После одного обновления актора с размером шага $\alpha_\theta = 0.01$ и $\gamma^t = 1$, на сколько изменится $\log \pi_\theta(A_t|S_t)$?

Упражнение 9.3. Пусть $T = 4$ (шаги $t = 0, 1, 2, 3$, терминальное при $t = 4$) с ошибками TD $\delta_0 = 0.4$, $\delta_1 = -0.2$, $\delta_2 = 0.6$, $\delta_3 = 0.1$. Возьмём $\gamma = 1$ и $\lambda = 0.8$.

(a) Вычислите $\hat{A}_0^{\text{GAE}}$ используя определение 9.3.

(b) Проверьте, что ваш ответ совпадает с формулой из утверждения 9.3, вычислив взвешенную смесь n-шаговых преимуществ $\hat{A}_0^{(n)}$ для $n = 1, 2, 3, 4$.

(c) Вычислите $\hat{A}_0^{\mathrm{GAE}}$ для $\lambda = 0$ и $\lambda = 1$ и интерпретируйте результаты.

Упражнение 9.4. Рассмотрим отсечённую целевую функцию PPO с $\varepsilon = 0.2$.

(a) Вычислите вклады L^{CLIP} для следующих трёх переходов и укажите, активно ли отсечение:

(i) $\hat{A}_t = 1.5$, $r_t(\theta) = 0.8$.

(ii) $\hat{A}_t = 1.5$, $r_t(\theta) = 1.3$.

(iii) $\hat{A}_t = -0.5$, $r_t(\theta) = 0.75$.

(b) Для случая (ii) объясните, почему градиент L^{CLIP} по θ равен нулю, несмотря на то что $\hat{A}_t > 0$ и действие следует поощрять. Почему такое консервативное поведение желательно?

Упражнение 9.5. В этом упражнении детально разбирается суррогатный градиент (утверждение 9.4).

(a) Покажите, что $r_t(\theta)|_{\theta=\theta_{\mathrm{old}}} = 1$ для всех переходов.

(b) Запишите $\nabla_\theta r_t(\theta)$ через стратегию и покажите, что оно равно $r_t(\theta)\,\nabla_\theta \log \pi_\theta(A_t|S_t)$.

(c) Заключите, что $\nabla_\theta L(\theta)|_{\theta_{\mathrm{old}}} = \mathbb{E}[\nabla_\theta \log \pi_{\theta_{\mathrm{old}}}(A_t|S_t)\,\hat{A}_t]$, и объясните, почему это в точности градиент стратегии.

Упражнение 9.6. В скорректированном вознаграждении RLHF (9.19) задать $\gamma = 1$.

(a) Докажите утверждение 9.7 с нуля, используя только цепное правило для KL-дивергенции авторегрессионных моделей.

(b) Покажите, что оптимальная стратегия для целевой функции $\max_\pi \mathbb{E}[r_\psi(x,y)] - \beta\, D_{\mathrm{KL}}(\pi(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x))$ есть $\pi^*(y|x) \propto \pi_{\mathrm{ref}}(y|x)\, e^{r_\psi(x,y)/\beta}$.

(c) Интерпретируйте оптимальную стратегию: что происходит при $\beta \to 0$? При $\beta \to \infty$? Что это означает для выбора β на практике?

Упражнение 9.7. Заполните следующую таблицу сравнения для трёх алгоритмов: REINFORCE с базовой линией, одношаговый актор-критик и PPO. Для каждой строки ответьте «да» / «нет» или дайте краткую фразу.

Свойство	REINFORCE+БЛ	Одношаг. АК	PPO
Требует полного эпизода?			
Повторное использование данных?			
Оценщик преимущества			
Обновления онлайн (пошагово)?			
Ограничение доверительной обл.?			

Упражнение 9.8. Объясните, используя разложение из утверждения 9.3, почему GAE с $\lambda < 1$ имеет меньшую дисперсию, чем оценка преимущества Монте-Карло $G_t - V_\phi(S_t)$, но вносит смещение.

(a) Определите источник дисперсии в оценке Монте-Карло (подсказка: подсчитайте количество независимых случайных величин в G_t).

(b) Определите источник смещения в оценке GAE при $\lambda < 1$ (подсказка: рассмотрите, что происходит, когда $V_\phi \neq V^\pi$).

(c) Неформально обоснуйте, почему $\lambda \approx 0.95$ является разумным значением по умолчанию для эпизодов длиной ~ 1000.

Упражнение 9.9. Результат сходимости при двух шкалах (теорема 9.8) требует $\alpha_\theta(t)/\alpha_\phi(t) \to 0$.

(a) Объясните интуитивно, почему критик должен обучаться быстрее актора. Что пойдёт не так, если обновления актора такие же быстрые, как у критика?

(b) Предложите конкретную пару последовательностей размеров шагов $\alpha_\phi(t) = 1/(t+1)$ и $\alpha_\theta(t)$, удовлетворяющих условию двух шкал.

(c) В современных реализациях PPO с Adam и актор, и критик используют одинаковый оптимизатор и скорость обучения. Какой практический механизм заменяет требование двух шкал?

Упражнение 9.10 (Повышенная сложность)**.** В этом упражнении устанавливается точная связь между TRPO и PPO.

(a) Покажите, что ограничение TRPO $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\theta_{\mathrm{old}}}) \leq \delta$ влечёт $|r_t(\theta) - 1| \leq \sqrt{2\delta}$ для любой отдельной пары действие-состояние, используя неравенство Пинскера $\|\pi - q\|_1 \leq \sqrt{2\, D_{\mathrm{KL}}(\pi \parallel q)}$.

(b) Используя часть (a), покажите, что отсечение PPO при ε является приближением первого порядка к ограничению KL TRPO с $\delta \approx \varepsilon^2/2$.

(c) Обсудите одно преимущество и один недостаток приближения PPO по сравнению с точным TRPO.

Часть III

RL для языковых моделей

Глава 10

Формирование вознаграждения и модели вознаграждения

Мусор на входе, мусор на выходе. Качество обученной стратегии никогда не превысит качество оптимизируемого сигнала вознаграждения.

В конечном счёте любой алгоритм обучения с подкреплением сводится к одному императиву: максимизировать кумулятивное вознаграждение. Это делает сигнал вознаграждения наиболее значимым решением при проектировании любой RL-системы. Неправильно заданное вознаграждение — даже незначительно — даст стратегию, которая в лучшем случае будет субоптимальной, а в худшем — активно вредной. История прикладного RL полна поучительных примеров: симулированный робот, научившийся двигаться вибрацией вместо ходьбы; игровой агент, обнаруживший высокооценивающийся эксплойт, которого разработчики никогда не предусматривали; и — в современную эпоху выравнивания языковых моделей — чат-боты, научившиеся производить многословную, льстивую прозу, потому что так предпочитала прокси-модель вознаграждения.

Эта глава рассматривает проблему проектирования вознаграждения с двух взаимодополняющих углов. Первая половина развивает классическую теорию *формирования вознаграждения*: по заданной среде с хорошо определённым, но разреженным или иначе неудобным истинным вознаграждением — как дополнить это вознаграждение дополнительными слагаемыми, ускоряющими обучение без изменения оптимальной стратегии? Краеугольным результатом является теорема о формировании на основе потенциала Нга, Харады и Расселла (1999), дающая точную характеристику всех дополнений вознаграждения, гарантированно сохраняющих стратегию. Вторая половина обращается к *обученным моделям вознаграждения*: что делать, когда истинное вознаграждение неизвестно даже в замкнутой форме и должно быть выведено из суждений людей, исходов или аннотаций на уровне шагов. Мы выводим модель предпочтений Bradley–Terry, кросс-энтропийную функцию потерь для обучения моделей вознаграждения по предпочтениям, а затем подробно рассматриваем различие между моделями вознаграждения по исходам (ORM) и по процессу (PRM). Завершаем переоптимизацией вознаграждения — законом Гудхарта в RL — и механизмом штрафа KL, защищающим от неё.

Эта глава занимает ключевое место в книге. Она связывает Часть II (классические методы RL, Главы 4–9) и Часть III (RL для языковых моделей). Обученные здесь модели вознаграждения непосредственно используются в пайплайне дообучения PPO (Глава 14), целевой функции DPO и алгоритме GRPO, описанных в последующих главах. Читатели, освоившие аппарат TD и градиента стратегии из Части II, найдут материал по моделированию вознаграждения во второй половине этой главы естественным расширением, изложенным в терминах крупномасштабных языковых моделей.

10.1 Формирование вознаграждения на основе потенциала

10.1.1 Проблема разреженного вознаграждения

Во многих средах агент сталкивается с разреженным сигналом вознаграждения: положительная обратная связь приходит только при достижении конкретной цели, а за все остальные переходы получается нулевое вознаграждение. Рассмотрим робота, навигирующего в большом лабиринте: истинное вознаграждение равно $+1$ у выхода и 0 везде ещё. Агент, исследующий случайным образом, найдёт выход только после астрономически большого числа шагов, и до этого момента не получит никакой информации о градиенте вовсе. Аналогично, языковая модель, получающая вознаграждение только в конце длинного ответа, должна

приписать каждому из сотен предшествующих токенов долю этого терминального сигнала — проблема присвоения кредита в крайней форме.

Интуитивное решение — *формировать* вознаграждение: добавить вспомогательные слагаемые, направляющие агента к цели. Робот, навигирующий в лабиринте, мог бы получать небольшой бонус, пропорциональный тому, насколько он сократил Евклидово расстояние до выхода. Языковая модель могла бы получать промежуточные вознаграждения за производство грамматически верных предложений. Однако формирование вознаграждения опасно. Наивно добавленные бонусные слагаемые могут изменить оптимальную стратегию: робот, получающий большой бонус за приближение к стене (которая оказывается рядом с выходом), может научиться держаться возле этой стены, а не достигать цели. Фундаментальный вопрос: *какие дополнения вознаграждения гарантированно не изменяют множество оптимальных стратегий?*

10.1.2 Теорема о формировании

Нг, Харада и Расселл (1999) ответили на этот вопрос точно. Их результат, известный как **теорема о формировании на основе потенциала**, характеризует все преобразования вознаграждения, инвариантные к стратегии.

Определение 10.1 (Функция формирования на основе потенциала)**.** Пусть $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ — дисконтированный MDP. **Функция формирования на основе потенциала** — это функция $F : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ вида

$$F(s, a, s') = \gamma\, \Phi(s') - \Phi(s), \tag{10.1}$$

где $\Phi : \mathcal{S} \to \mathbb{R}$ — произвольная **потенциальная функция**.

Задав сформированное вознаграждение $\tilde{R}(s, a, s') = R(s, a, s') + F(s, a, s')$, определим сформированный MDP $\tilde{\mathcal{M}} = (\mathcal{S}, \mathcal{A}, P, \tilde{R}, \gamma)$. Центральная теорема такова.

Теорема 10.1 (Теорема о формировании на основе потенциала, Ng et al. 1999)**.** *Пусть F — функция формирования на основе потенциала, как в (10.1). Тогда:*

1. *Каждая оптимальная стратегия для $\mathcal{M}$ также оптимальна для $\tilde{\mathcal{M}}$, и наоборот.*
2. *Сформированная функция ценности $\tilde{V}^\pi$ и сформированная функция ценности действий $\tilde{Q}^\pi$ удовлетворяют*
$$\tilde{V}^\pi(s) = V^\pi(s) - \Phi(s), \qquad \tilde{Q}^\pi(s, a) = Q^\pi(s, a) - \Phi(s). \tag{10.2}$$
3. $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$ — единственное *семейство функций (с точностью до константы), при котором оптимальные стратегии сохраняются для всех MDP.*

Доказательство. Докажем части 1 и 2. n-шаговая доходность при сформированном вознаграждении, начиная из состояния s_0, равна

$$\begin{aligned}\tilde{G}_0 &= \sum_{t=0}^{\infty} \gamma^t \tilde{R}(S_t, A_t, S_{t+1}) \\ &= \sum_{t=0}^{\infty} \gamma^t R(S_t, A_t, S_{t+1}) + \sum_{t=0}^{\infty} \gamma^t \big(\gamma\Phi(S_{t+1}) - \Phi(S_t)\big).\end{aligned}$$

Раскрывая вторую сумму как телескопический ряд:

$$\sum_{t=0}^{\infty} \gamma^t \big(\gamma\Phi(S_{t+1}) - \Phi(S_t)\big) = \sum_{t=0}^{\infty} \gamma^{t+1}\Phi(S_{t+1}) - \sum_{t=0}^{\infty} \gamma^t \Phi(S_t) = -\Phi(S_0).$$

(Телескопическое сокращение выполняется при условии $\gamma^t \Phi(S_t) \to 0$ при $t \to \infty$, что гарантируется при ограниченной Φ и $\gamma < 1$.) Следовательно

$$\tilde{G}_0 = G_0 - \Phi(S_0). \tag{10.3}$$

Беря ожидания: $\tilde{V}^\pi(s) = V^\pi(s) - \Phi(s)$, что соответствует (10.2). Поскольку $\Phi(s)$ зависит только от текущего состояния, но не от действия или стратегии, функция преимущества не изменяется:

$$\tilde{A}^\pi(s, a) = \tilde{Q}^\pi(s, a) - \tilde{V}^\pi(s) = Q^\pi(s, a) - V^\pi(s) = A^\pi(s, a).$$

Жадная стратегия относительно $\tilde{Q}^\pi$ совпадает с жадной стратегией относительно Q^π, что устанавливает часть 1. $\square$

Элегантность доказательства заключается в телескопическом тождестве (10.3): добавление бонуса на основе потенциала эквивалентно простому сдвигу всей доходности на зависящую от состояния константу $-\Phi(S_0)$, которая не может повлиять на то, какая стратегия достигает максимума.

Замечание 10.1 (Не-потенциальное формирование меняет оптимальные стратегии). Часть 3 Теоремы 10.1 предостерегает: *любая* функция формирования, не имеющая вид на основе потенциала, изменит оптимальные стратегии хотя бы в некотором MDP. Классический контрпример: предположим, что бонус формирования $F(s, a, s') = +1$ для всех переходов, независимо от какого-либо потенциала. Это равномерно завышает все вознаграждения, что не меняет оптимальную стратегию при полном дисконтировании, но искажает относительную ценность коротких и длинных эпизодов. Более серьёзно: не-потенциальные бонусы, такие как зависящие от действий слагаемые (например, бонусы за посещение новых состояний при обучении на основе любопытства), *действительно* изменяют оптимальную стратегию в общем случае, что может быть намеренным (исследование), но должно пониматься как сознательное изменение цели, а не нейтральное дополнение.

10.1.3 Проектирование потенциальных функций: примеры

Теорема гарантирует инвариантность стратегии для любой Φ, однако *скорость* обучения существенно зависит от того, насколько хорошо Φ кодирует полезные априорные знания. Три стандартные конструкции иллюстрируют пространство проектных решений.

Пример 10.1 (Потенциал расстояния до цели в мире клеток)**.** Рассмотрим сетку 5×5 с целевым состоянием g. Определим $\Phi(s) = -d(s, g)$, где d — манхэттенское расстояние до цели. Бонус формирования для перехода из s в s' равен

$$F(s, s') = \gamma(-d(s', g)) - (-d(s, g)) = d(s, g) - \gamma\, d(s', g).$$

Когда агент приближается к цели ($d(s', g) < d(s, g)$) и $\gamma \approx 1$, этот бонус приблизительно равен $d(s, g) - d(s', g) > 0$. Удаление от цели даёт отрицательный бонус. Это мягко направляет агента к цели, не изменяя того, что он в конечном итоге научится делать.

Численно: предположим $\gamma = 0.99$, $d(s, g) = 4$, $d(s', g) = 3$. Тогда $F = 4 - 0.99 \cdot 3 = 4 - 2.97 = 1.03$. При удалении агента: $d(s', g) = 5$, $F = 4 - 0.99 \cdot 5 = 4 - 4.95 = -0.95$. Сигнал бонуса по величине сопоставим с истинным терминальным вознаграждением $+1$, обеспечивая плотное руководство на протяжении всего эпизода.

Пример 10.2 (Потенциал функции ценности)**.** Если имеется доступ к приближённой функции ценности $\hat{V}$ (возможно, из более простого, связанного MDP или из предобученной модели), установка $\Phi = \hat{V}$ даёт бонусы формирования, кодирующие априорные убеждения агента о качестве состояний. В контексте языковых моделей именно такова роль *опорной модели*: стратегия SFT обеспечивает неявную оценку того, какие продолжения являются высококачественными, а штраф KL можно интерпретировать как регуляризацию на основе потенциала. Мы вернёмся к этой связи в Разделе 10.8.

Пример 10.3 (Обученный потенциал из критика)**.** В методах актор-критик (Глава 9) критик V_ϕ сам обучается одновременно со стратегией. Установка $\Phi = V_\phi$ и вычисление бонусов формирования на лету соответствует в точности формулировке через функцию преимущества: TD-ошибка $\delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$ является бонусом формирования на основе потенциала, добавляемым к непосредственному вознаграждению перед использованием в качестве сигнала градиента стратегии. Это обнаруживает, что градиенты стратегии на основе преимущества в точном смысле используют обученную потенциальную функцию в качестве бонуса формирования.

10.2 От ручного проектирования к обученным вознаграждениям

10.2.1 Пределы ручного проектирования вознаграждения

Даже при защите потенциальным формированием проектирование правильной функции вознаграждения для сложных задач чрезвычайно затруднено. Практик сталкивается с фундаментальной асимметрией: описать, как *выглядит* хорошее поведение, легко, но точно закодировать его как скалярную функцию состояний и действий — сложно. Для робототехники необходимо решить, вознаграждать ли моменты сочленений, позиции концевого эффектора, выполнение задачи, энергоэффективность и ограничения безопасности одновременно, а затем вручную настраивать их относительные веса. Для разговорного агента понятия *полезности*, *безвредности* и *честности* интуитивно понятны людям, но крайне нетривиально поддаются количественной оценке.

Взлом вознаграждения — это явление, при котором агент обнаруживает стратегию, достигающую высокого числового вознаграждения без удовлетворения замысла разработчика. Литература по «specification gaming» (Krakovna et al., 2020) документирует сотни примеров в различных предметных областях. В RL для игр агенты научились умирать повторно, чтобы избежать потери очков; эксплуатировать баги физики для бесконечной скорости; и останавливать игровые часы, чтобы предотвратить штрафы за ограниченное время. В робототехнике симулированная рука научилась позиционировать себя между датчиком и объектом задачи так, что задача казалась завершённой для датчика, тогда как реального прогресса не было.

10.2.2 Взлом вознаграждения в языковых моделях

Те же режимы отказа появляются, возможно ещё более остро, в выравнивании языковых моделей, где функция вознаграждения даже не проектируется вручную, а сама является нейронной сетью, обученной на шумных человеческих суждениях.

Смещение по многословности: аннотаторы, оценивающие пары ответов, склонны воспринимать более длинные, подробные ответы как более качественные, даже когда дополнительная длина не несёт информации. Модель вознаграждения, обученная на таких данных, будет присваивать более высокие баллы более длинным ответам. Когда языковая модель оптимизируется против этого прокси-вознаграждения, она учится заполнять ответы повторяющимися оговорками и шаблонными резюме — стратегия, максимизирующая прокси при деградации подлинной полезности.

Подхалимство: человеческие аннотаторы подвержены смещению социальной желательности, предпочитая ответы, соглашающиеся с их высказанными мнениями, ответам, исправляющим их ошибки. Модель вознаграждения, интернализирующая эту закономерность, будет высоко оценивать ответы, ищущие соглашения, что приводит к тому, что оптимизированная стратегия говорит пользователям то, что те хотят услышать, а не то, что является истиной.

Эксплуатация форматирования: модели вознаграждения, обученные на ответах людей-писателей, могут непреднамеренно ассоциировать markdown-форматирование (маркированные списки, жирные заголовки, блоки кода) с высоким качеством, просто потому что человеческие демонстрации склонны быть хорошо оформленными. Оптимизированная стратегия затем применяет тяжёлое форматирование даже там, где оно неуместно, например оформляя краткий фактический ответ в виде маркированного списка.

Эти примеры мотивируют фундаментальный сдвиг в стратегии: вместо попыток закодировать функцию вознаграждения a priori мы *обучаем* её по обратной связи от людей. В оставшейся части главы разрабатывается теоретический и практический аппарат для этого.

10.3 Модель Bradley–Terry

10.3.1 Данные о предпочтениях

Отправная точка для обучения функции вознаграждения по мнениям людей — наблюдение, что человеку часто легче сказать, какой из двух ответов лучше, чем присвоить единственному ответу абсолютный числовой балл. Сравнительные суждения более надёжны, более согласованны между аннотаторами и менее чувствительны к абсолютной калибровке отдельных рейтеров.

Формально, предположим, что существует скрытая истинная функция вознаграждения $r^* : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ (или, в контексте языковых моделей, $r^* : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$, где $\mathcal{X}$ — пространство промптов, $\mathcal{Y}$ — пространство ответов). Эта функция не наблюдаема напрямую. Вместо этого человек-аннотатор наблюдает два ответа y_1 и y_2 на промпт x и указывает предпочтение, обозначаемое $y_1 \succ y_2$ («y_1 предпочтительнее y_2»).

Определение 10.2 (Датасет предпочтений)**. Датасет предпочтений** — это коллекция

$$\mathcal{D} = \left\{ (x^{(i)},\, y_w^{(i)},\, y_l^{(i)}) \right\}_{i=1}^{N},$$

где $y_w^{(i)} \succ y_l^{(i)}$ означает, что человек-аннотатор предпочёл $y_w^{(i)}$ («победитель») над $y_l^{(i)}$ («проигравший») в ответ на промпт $x^{(i)}$.

Цель — обучить параметрическую функцию вознаграждения $r_\psi : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ так, чтобы $r_\psi(x, y_w) > r_\psi(x, y_l)$ для всех пар в $\mathcal{D}$, и, что важнее, чтобы она правильно обобщалась на невидимые промпты и ответы.

10.3.2 Модель предпочтений Bradley–Terry

Как следует моделировать вероятность того, что человек предпочтёт y_1 над y_2? Наиболее принципиальным и широко используемым фреймворком является **модель Bradley–Terry** (Bradley and Terry, 1952), изначально разработанная для ранжирования спортивных соревнующихся и впоследствии принятая в статистике, экономике и машинном обучении.

Определение 10.3 (Модель Bradley–Terry)**.** **Модель Bradley–Terry** постулирует, что вероятность предпочтения y_1 над y_2 при промпте x зависит только от разности скрытых качеств:

$$\mathbb{P}(y_1 \succ y_2 \mid x) = \sigma\big(r^*(x, y_1) - r^*(x, y_2)\big), \tag{10.4}$$

где $\sigma(z) = 1/(1 + e^{-z})$ — логистическая сигмоидная функция.

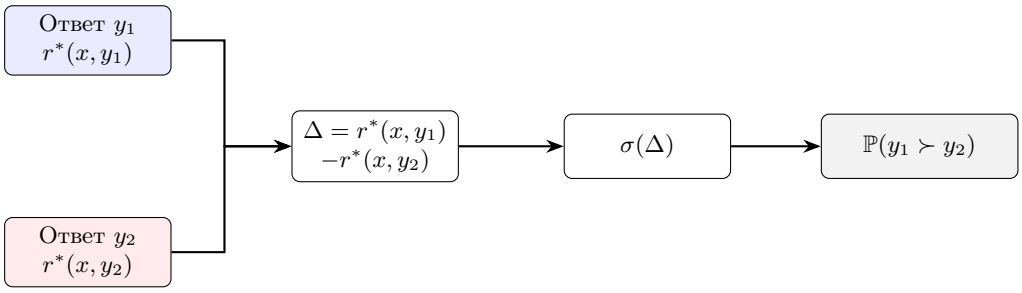


Рисунок: Модель предпочтений Bradley–Terry. Вероятность предпочтения зависит только от разности скрытых оценок вознаграждения, пропущенной через сигмоиду.

Модель имеет ясную вероятностную интерпретацию. Когда два ответа одинаково хороши ($r^*(x, y_1) = r^*(x, y_2)$), вероятность предпочтения равна $\sigma(0) = 1/2$: аннотатор безразличен. По мере роста разрыва в качестве вероятность предпочтения приближается к 1 (или 0), отражая возрастающую уверенность. Сигмоидная форма фиксирует насыщение человеческих суждений: для очень очевидных пар вероятность предпочтения фактически определённа вне зависимости от точной величины разности.

Утверждение 10.2 (Свойства модели Bradley–Terry)**.** *Модель Bradley–Terry* (10.4) *удовлетворяет:*

1. ***Симметрия***: $\mathbb{P}(y_1 \succ y_2 \mid x) + \mathbb{P}(y_2 \succ y_1 \mid x) = 1$.
2. ***Безразличие при равенстве***: $r^*(x, y_1) = r^*(x, y_2) \implies \mathbb{P}(y_1 \succ y_2) = 1/2$.
3. ***Монотонность***: $\mathbb{P}(y_1 \succ y_2 \mid x)$ *строго возрастает по* $r^*(x, y_1)$ *и строго убывает по* $r^*(x, y_2)$.
4. ***Инвариантность к сдвигу***: $r^* \mapsto r^* + c$ *для любой константы* c *не изменяет все вероятности предпочтения. Функция вознаграждения идентифицируется, таким образом, лишь с точностью до аддитивной константы.*

Доказательство. (1) следует из тождества $\sigma(z) + \sigma(-z) = 1$. (2) следует из $\sigma(0) = 1/2$. (3) следует из $\sigma'(z) = \sigma(z)(1 - \sigma(z)) > 0$ для всех $z \in \mathbb{R}$. (4): подстановка $r^*(x, y) + c$ вместо $r^*(x, y)$ даёт $\sigma((r^*(x, y_1) + c) - (r^*(x, y_2) + c)) = \sigma(r^*(x, y_1) - r^*(x, y_2))$, что неизменно. □

10.3.3 Связь с рейтингами Эло

Модель Bradley–Terry тесно связана с **системой рейтингов Эло**, используемой в шахматах и других соревновательных играх. В Эло у каждого игрока i есть скалярный рейтинг ρ_i, и вероятность того, что игрок i победит игрока j, моделируется как $1/(1 + 10^{(\rho_j - \rho_i)/400})$. Это в точности модель Bradley–Terry с $r^* = \rho$ и логистической функцией по основанию 10 с масштабным коэффициентом 400. Ключевое различие: рейтинги Эло обновляются онлайн после каждого матча, используя простой шаг стохастического градиента по лог-потере, тогда как при моделировании вознаграждения мы выполняем пакетный градиентный спуск по всем парам предпочтений одновременно. Оба подхода оценивают одно и то же скрытое качество r^* по попарным сравнениям.

10.4 Модели вознаграждения по предпочтениям

10.4.1 Оценка максимального правдоподобия

Имея датасет предпочтений $\mathcal{D}$, мы хотим подобрать параметрическую модель r_ψ для аппроксимации r^*. При предположении Bradley–Terry вероятность наблюдения полного датасета равна

$$\mathcal{L}_{\text{lik}}(\psi) = \prod_{i=1}^{N} \sigma\big(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)})\big). \tag{10.5}$$

Максимизация правдоподобия эквивалентна минимизации отрицательного логарифма правдоподобия.

Определение 10.4 (Функция потерь модели вознаграждения по предпочтениям)**. Функция потерь модели вознаграждения по предпочтениям** равна

$$\boxed{\mathcal{L}_{\text{RM}}(\psi) = -\frac{1}{N}\sum_{i=1}^{N} \log \sigma\big(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)})\big).} \tag{10.6}$$

Это в точности бинарная кросс-энтропийная потеря с меткой 1 («победитель предпочтителен») и предсказанной вероятностью $\hat{p}_i = \sigma(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)}))$. Обучение модели вознаграждения — это, следовательно, стандартная задача бинарной классификации: модель должна правильно ранжировать предпочтительный ответ выше непредпочтительного.

Утверждение 10.3 (Структура градиента $\mathcal{L}_{\text{RM}}$)**.** *Для одной пары (x, y_w, y_l) пусть $\Delta = r_\psi(x, y_w) - r_\psi(x, y_l)$. Тогда*

$$\frac{\partial \mathcal{L}_{\text{RM}}}{\partial r_\psi(x, y_w)} = -\sigma(-\Delta) < 0, \qquad \frac{\partial \mathcal{L}_{\text{RM}}}{\partial r_\psi(x, y_l)} = +\sigma(-\Delta) > 0.$$

Градиентный спуск, таким образом, увеличивает $r_\psi(x, y_w)$ *и* уменьшает $r_\psi(x, y_l)$*, раздвигая оценки. Размер шага $\sigma(-\Delta) = 1 - \sigma(\Delta)$ велик, когда текущий разрыв мал (пара сложная), и мал, когда разрыв уже велик (пара лёгкая). Это стандартное поведение логистической регрессии.*

Доказательство. $\mathcal{L} = -\log \sigma(\Delta)$. Дифференцируя по Δ: $\partial\mathcal{L}/\partial\Delta = -\sigma(-\Delta)$. Поскольку $\partial\Delta/\partial r_\psi(x, y_w) = +1$ и $\partial\Delta/\partial r_\psi(x, y_l) = -1$, результат следует по правилу цепочки. □

10.4.2 Выбор архитектуры

Модель вознаграждения по предпочтениям r_ψ обычно строится поверх предобученной языковой модели. Каноническая архитектура инициализирует r_ψ из контрольной точки SFT — той же модели с контролируемым дообучением, используемой в качестве отправной точки для RL. Линейный проекционный слой $W \in \mathbb{R}^{d\times 1}$ добавляется поверх трансформера, принимая на вход скрытое состояние на позиции последнего токена (обычно токена `[EOS]`) и выдавая скалярную оценку.

- **Почему инициализировать из SFT?** Модель вознаграждения должна оценивать ответы в том же формате, в котором модель SFT их генерирует. Инициализация из базовой LM, не видевшей данных для следования инструкциям, ведёт к модели вознаграждения, плохо понимающей содержание, которое ей следует оценивать.

- **Почему использовать токен EOS?** К тому времени, когда модель обрабатывает токен `[EOS]`, механизм само-внимания позволил каждой позиции обращать внимание на каждую другую. Скрытое состояние EOS, таким образом, кодирует сжатое представление всего ответа, делая его естественной точкой извлечения скалярной оценки качества.

- **Инвариантность к масштабу**: функция вознаграждения идентифицируется лишь с точностью до аддитивной константы (Утверждение 10.2). На практике оценки нормализуются до нулевого среднего и единичной дисперсии по обучающему батчу, что стабилизирует обучение и обеспечивает работу штрафа KL (Раздел 10.8) при согласованном масштабе.

Алгоритм 25: Обучение модели вознаграждения по предпочтениям

Input: Контрольная точка SFT π_{SFT}; датасет предпочтений $\mathcal{D}$; скорость обучения α; размер батча B; число эпох E

Output: Обученная модель вознаграждения r_ψ

Инициализировать $r_\psi \leftarrow \pi_{\text{SFT}}$ + случайная линейная голова $W \in \mathbb{R}^{d\times 1}$;
for $e = 1, \ldots, E$ **do**
  for *каждый мини-батч* $\{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^{B} \subset \mathcal{D}$ **do**
    Вычислить $s_w^{(i)} \leftarrow r_\psi(x^{(i)}, y_w^{(i)})$ и $s_l^{(i)} \leftarrow r_\psi(x^{(i)}, y_l^{(i)})$ для всех i;
    Вычислить потерю $\mathcal{L} \leftarrow -\frac{1}{B}\sum_{i=1}^{B} \log \sigma(s_w^{(i)} - s_l^{(i)})$;
    $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}$;
    Мониторировать $\text{Acc} = \frac{1}{B}\sum_i \mathbb{1}[s_w^{(i)} > s_l^{(i)}]$;
return r_ψ

10.4.3 Практическая процедура обучения

На практике y_w и y_l конкатенируются по батч-измерению, так что оба обрабатываются за один прямой проход, вдвое сокращая количество прямых проходов трансформера по сравнению с наивной двухпроходной реализацией. Хорошо обученные модели вознаграждения по предпочтениям обычно достигают попарной точности на отложенном наборе 70–80%. Идеальная точность невозможна, поскольку человеческие аннотации содержат подлинный шум: разные аннотаторы расходятся во мнениях, и даже один аннотатор может быть непоследовательным в разных сессиях.

10.5 Модели вознаграждения по исходам (ORM)

10.5.1 Разреженные терминальные вознаграждения

Модель вознаграждения по предпочтениям из Раздела 10.4 присваивает единственный скаляр полному ответу (x, y). Это частный случай более широкого класса **моделей вознаграждения по исходам** (ORM): моделей, оценивающих только конечный исход траектории, с нулевым вознаграждением на всех промежуточных шагах.

Определение 10.5 (Модель вознаграждения по исходам)**. Модель вознаграждения по исходам** (ORM) — это функция вознаграждения $r_{\text{ORM}} : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$, присваивающая скаляр полному ответу y, с нулевым вознаграждением на каждом промежуточном шаге генерации:

$$R_t = 0 \text{ при } t < T, \qquad R_T = r_{\text{ORM}}(x, y).$$

ORM — естественный выбор, когда качество ответа можно оценить только после того, как полный ответ доступен. Они возникают как минимум в трёх различных контекстах.

1. RLHF на основе предпочтений. Как выведено в Разделе 10.4, результатом модели вознаграждения по предпочтениям на основе Bradley–Terry является единственный скаляр на ответ. Это ORM по конструкции.

2. Задачи с верифицируемо правильным ответом. При решении математических задач, программировании и фактическом вопросоответе ответ либо верен, либо нет: сгенерированный ответ совпадает с эталоном или нет. Можно обучить бинарный классификатор $r_{\text{ORM}}(x, y) \in \{0, 1\}$, проверяющий, верен ли окончательный ответ. Это даёт чистый объективный ORM, избегающий шума, присущего аннотациям людей.

3. Оценка ценности методом Монте-Карло. В фреймворке Монте-Карло (MC) из Главы 5 оценки ценности получаются путём развёртки полных траекторий из каждого состояния и усреднения доходностей. ORM в контексте языковых моделей соответствует в точности доходности Монте-Карло: полная последовательность генерируется, терминальное вознаграждение наблюдается, а ценность каждого промежуточного состояния оценивается усреднением по многим развёрткам. Эта связь точна: сигнал вознаграждения ORM — это доходность MC с нулевым промежуточным вознаграждением и терминальным вознаграждением.

10.5.2 Когда использовать ORM

ORM предпочтительны, когда:

- Корректность промежуточных шагов сложно или дорого оценить.

- Доступен надёжный автоматизированный верификатор (например, символьная программа проверки математики или запускатель юнит-тестов).
- Горизонт задачи достаточно короток, чтобы разреженное терминальное вознаграждение не приводило к неразрешимому присвоению кредита.

Главное ограничение ORM — именно проблема присвоения кредита, подчёркнутая в Главе 6: ошибочный токен в начале ответа вносит вклад в неудачу всего ответа, но получает только косвенный сигнал через терминальное вознаграждение после множества промежуточных токенов. Модели вознаграждения по процессу, вводимые далее, устраняют это ограничение, предоставляя плотные промежуточные сигналы.

10.6 Модели вознаграждения по процессу (PRM)

10.6.1 Надзор на уровне шагов

Вместо оценки только конечного ответа **модель вознаграждения по процессу** (PRM) присваивает вознаграждение каждому промежуточному шагу в генерации. В контексте многошагового математического рассуждения шаг соответствует строке решения (предложению, логическому выводу или алгебраической манипуляции). В более общем случае в RL языковых моделей шаг может соответствовать предложению, абзацу или даже отдельному токену.

Определение 10.6 (Модель вознаграждения по процессу)**.** **Модель вознаграждения по процессу** (PRM) присваивает скалярную оценку каждому шагу $k = 1, \ldots, K$ многошагового решения $y = (y_1, \ldots, y_K)$:

$$r_{\text{PRM}}(x, y_{1:k}) \in \mathbb{R} \quad \text{для каждого } k = 1, \ldots, K.$$

Вознаграждение на шаге k отражает качество или корректность частичного решения $(y_1, \ldots, y_k)$ при промпте x.

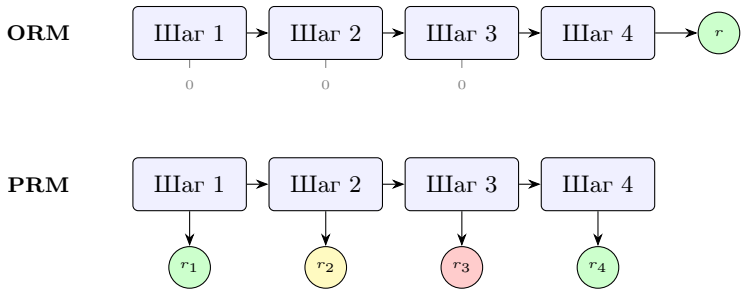


Рис. 10.1: Структура сигнала ORM и PRM. ORM присваивает единственное терминальное вознаграждение (вверху); PRM присваивает вознаграждение каждому шагу (внизу). Цвета шагов указывают качество: зелёный = верно, жёлтый = неопределённо, красный = неверно.

10.6.2 Обучение моделей вознаграждения по процессу

Обучение PRM требует аннотаций на *уровне шагов*: для каждого шага в решении аннотатор (человек или автоматизированный) указывает, является ли шаг верным. Пусть $c_k \in \{0, 1\}$ — метка корректности для шага k. PRM обучается предсказывать вероятность того, что шаг k верен:

$$\mathcal{L}_{\text{PRM}}(\psi) = -\frac{1}{N} \sum_{i=1}^{N} \frac{1}{K^{(i)}} \sum_{k=1}^{K^{(i)}} \Big[c_k^{(i)} \log r_\psi(x^{(i)}, y_{1:k}^{(i)}) + (1 - c_k^{(i)}) \log(1 - r_\psi(x^{(i)}, y_{1:k}^{(i)})) \Big], \tag{10.7}$$

где N — число решений, а $K^{(i)}$ — количество шагов в решении i.

Пример 10.4 (Датасет PRM800K)**.** Датасет PRM800K (Lightman et al., 2023) — наиболее широко используемый бенчмарк для обучения и оценки PRM в математическом рассуждении. Он содержит около 800 000 пошаговых человеческих аннотаций на решения задач из датасета MATH, коллекции олимпиадных математических задач семи уровней сложности. Каждый шаг каждого решения помечается как *положительный* (шаг верен и продвигает решение), *отрицательный* (шаг содержит ошибку) или *нейтральный*

(шаг нерелевантен или дублирован). PRM, обученная на PRM800K, учится присваивать высокую вероятность корректности логически верным шагам вывода и низкую вероятность ошибочным шагам, независимо от того, оказался ли конечный ответ верным.

Что важно, PRM800K демонстрирует, что надзор на уровне шагов даёт сигнал, недоступный ORM: решение может прийти к правильному окончательному ответу через неверный промежуточный шаг (случайные ошибки), или содержать все верные шаги, но прийти к неверному ответу из-за арифметической ошибки в последней строке. ORM присвоила бы высокое вознаграждение первому решению и низкое второму, несмотря на то что второе демонстрирует лучшее математическое рассуждение.

10.6.3 Использование PRM для поиска во время вывода

Помимо роли сигнала вознаграждения во время RL-обучения, PRM особенно мощны во время вывода. По заданному промпту x можно сгенерировать несколько кандидатных решений с помощью лучевого поиска или best-of-n сэмплирования, затем ранжировать их с помощью PRM. Несколько стратегий оказались эффективными:

Переранжирование best-of-n Сгенерировать n полных решений, оценить каждое с помощью ORM или агрегированных оценок PRM и вернуть решение с наивысшей оценкой. С хорошей PRM это существенно эффективнее, чем использование ORM для переранжирования, поскольку PRM штрафует решения, приходящие к верному ответу через ошибочное рассуждение.

Пошаговый лучевой поиск На каждом шаге генерировать K кандидатных продолжений, оценивать каждое с помощью PRM и расширять только B лучших лучей. Это вычислительно дорого, но исследует пространство решений значительно эффективнее, чем плоское сэмплирование.

Древовидный поиск под руководством PRM Использовать PRM как функцию ценности в древовидном поиске (например, Monte Carlo Tree Search) по шагам рассуждения. Это позволяет модели возвращаться назад и исследовать альтернативные пути рассуждения, когда ранний шаг получает низкую оценку PRM.

10.6.4 ORM против PRM: сравнение

Таблица 10.1: Сравнение моделей вознаграждения по исходам (ORM) и по процессу (PRM).

Свойство	**ORM**	**PRM**
Гранулярность	Только исход	На уровне шагов
Стоимость аннотирования	Низкая	Высокая
Присвоение кредита	Разреженное	Плотное
Случайные ошибки	Вознаграждает любой верный ответ	Штрафует неверные шаги
Устойчивость к шагам	Низкая	Высокая
Качество поиска	Умеренное	Сильное
Необходимые данные	Результаты (верно/неверно)	Метки корректности шагов
Применения	RLHF по предпочтениям, верификация	Мат. рассуждение, код

10.7 Цели обучения и практические соображения

10.7.1 Варианты функции потерь с запасом

Стандартная функция потерь модели вознаграждения (10.6) толкает $r_\psi(x, y_w)$ выше $r_\psi(x, y_l)$ без указания, насколько именно. Естественное расширение — требовать минимальный запас $m > 0$:

$$\mathcal{L}_{\mathrm{RM}}^{(m)}(\psi) = -\frac{1}{N}\sum_{i=1}^{N} \log \sigma\big(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)}) - m\big). \tag{10.8}$$

Запас гарантирует, что правильно ранжированные пары с малым разрывом всё же вносят нетривиальный вклад в потерю, поощряя модель производить уверенно разделённые оценки. Это аналогично запасу в методах опорных векторов.

Связанный вариант ассоциирует непрерывный показатель уверенности $\delta^{(i)} \in [0,1]$ с каждой парой предпочтений, отражающий уверенность аннотатора. Парам, с которыми аннотаторы сильно согласны, присваиваются более высокие δ:

$$\mathcal{L}_{\mathrm{RM}}^{(\delta)}(\psi) = -\frac{1}{\sum_i \delta^{(i)}} \sum_{i=1}^{N} \delta^{(i)} \log \sigma\big(r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)})\big). \tag{10.9}$$

Это снижает вес неоднозначных пар предпочтений, уменьшая влияние шума в процессе аннотирования.

10.7.2 Сглаживание меток

Аннотации предпочтений людей шумны: аннотаторы иногда ошибаются, и некоторые пары подлинно неоднозначны. Сглаживание меток регуляризует модель вознаграждения, заменяя жёсткие метки $\{0,1\}$ мягкими метками $\{\epsilon/2, 1-\epsilon/2\}$ для малого параметра сглаживания $\epsilon \in (0, 0.2]$:

$$\mathcal{L}_{\mathrm{RM}}^{(\epsilon)}(\psi) = -\frac{1}{N} \sum_{i=1}^{N} \Big[(1-\tfrac{\epsilon}{2}) \log \sigma(\Delta^{(i)}) + \tfrac{\epsilon}{2} \log \sigma(-\Delta^{(i)})\Big], \tag{10.10}$$

где $\Delta^{(i)} = r_\psi(x^{(i)}, y_w^{(i)}) - r_\psi(x^{(i)}, y_l^{(i)})$. Сглаживание меток не даёт модели вознаграждения гнать $\Delta^{(i)} \to +\infty$ (что сделало бы $\log \sigma(\Delta^{(i)}) \to 0$ и потерю нулевой) и побуждает к откалиброванным оценкам вероятности.

10.7.3 Качество данных и протоколы аннотирования

Качество модели вознаграждения ограничено качеством данных о предпочтениях. Ряд практических соображений существенен.

- **Согласие аннотаторов**: пары предпочтений, с которыми аннотаторы сильно расходятся, следует фильтровать или понижать в весе. Межаннотаторское согласие (например, каппа Коэна $\kappa > 0.4$) — минимальная планка для надёжного надзора.
- **Разнообразие промптов**: модель вознаграждения, обученная на узком распределении промптов, плохо обобщается. Данные о предпочтениях должны охватывать разнообразные темы, форматы и намерения пользователей.
- **Разнообразие ответов**: если оба ответа y_w и y_l сэмплированы из сильной стратегии, разрыв предпочтения может быть мал, а сигнал — шумным. Включение сравнений со значительно более слабыми базовыми линиями (например, с базовой моделью до SFT) может дать более лёгкий обучающий сигнал.
- **Оценка на отложенном наборе**: точность модели вознаграждения всегда следует сообщать на отложенном наборе из того же процесса аннотирования, и в идеале также на независимо собранном оценочном наборе для защиты от переобучения к идиосинкразиям аннотаторов.

10.7.4 Метрики оценки

Основная метрика оценки для модели вознаграждения по предпочтениям — **попарная точность**:

$$\mathrm{Acc} = \frac{1}{N_{\mathrm{eval}}} \sum_{i=1}^{N_{\mathrm{eval}}} \mathbb{1}\big[r_\psi(x^{(i)}, y_w^{(i)}) > r_\psi(x^{(i)}, y_l^{(i)})\big]. \tag{10.11}$$

Помимо попарной точности можно вычислять **корреляцию Пирсона** между оценками модели вознаграждения и независимыми человеческими рейтингами, и **метрики ранжирования**, такие как τ Кенделла или ρ Спирмена, по спискам ответов, ранжированных по оценкам модели вознаграждения. Для ORM, используемых в верифицируемых задачах, точность относительно символьного верификатора служит прямой проверкой относительно истины.

10.8 Переоптимизация вознаграждения

10.8.1 Закон Гудхарта в обучении с подкреплением

Обученная модель вознаграждения r_ψ является прокси для истинного человеческого предпочтения r^*. Когда стратегия оптимизируется против r_ψ с помощью RL, она неизбежно эксплуатирует любое расхождение между r_ψ и r^*. Это частный случай **закона Гудхарта**: *когда мера становится целью, она перестаёт быть хорошей мерой.*

Определение 10.7 (Переоптимизация вознаграждения)**. Переоптимизация вознаграждения** (или *взлом вознаграждения* в смысле прокси) происходит, когда стратегия π_θ достигает высокого прокси-вознаграждения $\mathbb{E}[r_\psi(x,y)]$, но низкого истинного вознаграждения $\mathbb{E}[r^*(x,y)]$. Она характеризуется существованием KL-бюджета d^* такого, что:

$$\frac{d}{d\,d}\,\mathbb{E}[r^*(x,y)] > 0 \quad \text{при } d < d^*, \tag{10.12}$$

$$\frac{d}{d\,d}\,\mathbb{E}[r^*(x,y)] < 0 \quad \text{при } d > d^*, \tag{10.13}$$

где $d = D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ измеряет дивергенцию от опорной стратегии.

Иными словами: по мере того, как стратегия всё дальше оптимизируется против прокси-вознаграждения (с возрастающей дивергенцией KL от опорной), истинное качество сначала улучшается, достигает пика при d^*, а затем деградирует. Прокси-вознаграждение, тем временем, продолжает монотонно улучшаться. Это создаёт опасный разрыв: стандартные RL-метрики обучения выглядят хорошо, даже когда модель ухудшается по той мере, что действительно важна.

Пример 10.5 (Масштабирование переоптимизации)**.** Gao et al. (2023) охарактеризовали это явление эмпирически и обнаружили, что истинное вознаграждение r^* как функция дивергенции KL d следует приблизительно вогнутой кривой вида

$$r^*(d) \approx \alpha\sqrt{d} - \beta\, d,$$

для положительных констант α и β, зависящих от размера и качества модели вознаграждения. Оптимальный KL-бюджет равен $d^* = (\alpha/(2\beta))^2$. Что важно, β растёт по мере того, как модель вознаграждения становится *меньше* относительно оптимизируемой стратегии: слабая модель вознаграждения ведёт к более быстрой переоптимизации. Это говорит о том, что ёмкость модели вознаграждения должна масштабироваться с ёмкостью стратегии.

10.8.2 Штраф KL как регуляризация

Стандартное средство от переоптимизации вознаграждения — добавить штраф дивергенции KL к целевой функции, ограничивая стратегию оставаться близко к опорной (SFT) стратегии:

Определение 10.8 (KL-регуляризованная целевая функция RLHF)**. KL-регуляризованная целевая функция RLHF** равна

$$J_{\mathrm{RLHF}}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r_\psi(x,y)\big] - \beta\, D_{\mathrm{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\mathrm{ref}}(\cdot|x))\,, \tag{10.14}$$

где $\beta > 0$ — коэффициент KL, а π_{ref} — стратегия SFT.

В реализациях PPO (Глава 14) штраф KL обычно включается в сигнал вознаграждения на уровне токенов:

$$r(x,y) = r_\psi(x,y) - \beta\,\log\frac{\pi_\theta(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)}, \tag{10.15}$$

что эквивалентно добавлению $-\beta\log(\pi_\theta/\pi_{\mathrm{ref}})$ как бонуса формирования на уровне токенов. Это в точности функция формирования на основе потенциала с $\Phi(s) = -\beta\log\pi_{\mathrm{ref}}(a|s)$ — связывая штраф KL с потенциальным фреймворком Раздела 10.1.

Теорема 10.4 (Оптимальная стратегия при KL-регуляризации)**.** *Стратегия, максимизирующая* (10.14) *поточечно (для каждого x), равна*

$$\pi^*(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)\,\exp\!\big(r_\psi(x,y)/\beta\big)}{Z(x)}, \tag{10.16}$$

где $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x)\,\exp(r_\psi(x,y)/\beta)$ — статистическая сумма.

Доказательство. Зафиксируем x и запишем целевую функцию как функционал от $\pi(\cdot|x)$:

$$\sum_y \pi(y|x)\, r_\psi(x,y) - \beta \sum_y \pi(y|x) \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}.$$

Объединяя слагаемые:

$$= -\beta \sum_y \pi(y|x) \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{r_\psi(x,y)}{\beta} \right] = -\beta\, D_{\text{KL}}(\pi(\cdot|x) \parallel \tilde{\pi}(\cdot|x)) + \text{const},$$

где $\tilde{\pi}(y|x) \propto \pi_{\text{ref}}(y|x) \exp(r_\psi(x,y)/\beta)$. KL минимизируется (и целевая функция максимизируется) при $\pi = \tilde{\pi}$, давая (10.16). □

Формула (10.16) раскрывает роль β как параметра *температуры*:

- При $\beta \to 0$: π^* концентрирует всю массу на ответе, максимизирующем r_ψ. Стратегия полностью управляется прокси-вознаграждением, и взлом вознаграждения максимален.
- При $\beta \to \infty$: $\pi^* \to \pi_{\text{ref}}$. RL-дообучение не имеет эффекта; стратегия остаётся в контрольной точке SFT.
- При умеренных $\beta \in [0.01, 0.5]$: стратегия интерполирует между априором SFT и решением, максимизирующим вознаграждение. Эмпирически этот диапазон захватывает пик кривой истинного вознаграждения (Определение 10.7).

Замечание 10.2 (Связь с DPO). Формулу оптимальной стратегии (10.16) можно преобразовать, чтобы выразить r_ψ через π^*, π_{ref} и $Z(x)$:

$$r_\psi(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x).$$

Это тождество лежит в основе алгоритма Direct Preference Optimisation (DPO): подстановка этого выражения для r_ψ в модель предпочтений Bradley–Terry (10.4) и использование того факта, что $\log Z(x)$ сокращается в разности, позволяет DPO обойти модель вознаграждения полностью и оптимизировать стратегию непосредственно по данным предпочтений. DPO выводится подробно в Главе 14.

10.9 Резюме и связи

В этой главе были разработаны два взаимодополняющих фреймворка для проблемы проектирования вознаграждения.

Формирование вознаграждения на основе потенциала (Раздел 10.1) предоставляет теоретически обоснованный метод дополнения данной функции вознаграждения для более плотного сигнала обучения без изменения множества оптимальных стратегий. Ключевое наблюдение — телескопическое тождество (10.3): бонус на основе потенциала $F = \gamma\Phi(s') - \Phi(s)$ вносит ровно $-\Phi(s_0)$ в кумулятивную доходность, зависящую от состояния константу, которая не влияет на функции преимущества. Этот принцип лежит в основе формулировки преимущества в методах актор-критик и штрафа KL на уровне токенов в RLHF.

Обученные модели вознаграждения (Разделы 10.2–10.8) рассматривают более сложную проблему вывода вознаграждения из человеческой обратной связи. Модель Bradley–Terry предоставляет принципиальный вероятностный фреймворк для данных предпочтений, приводящий к кросс-энтропийной функции потерь модели вознаграждения (10.6). Модели вознаграждения по исходам (ORM) предоставляют единственный терминальный сигнал, аналогичный доходностям Монте-Карло; модели вознаграждения по процессу (PRM) предоставляют плотный надзор на уровне шагов, разрешающий неоднозначности, недоступные ORM. Обе подчиняются закону Гудхарта: продолжительная оптимизация против любого прокси-вознаграждения в конечном счёте деградирует истинное качество. KL-регуляризованная целевая функция (10.14) и её оптимальное решение (10.16) дают принципиальное средство.

Центральные связи с последующими главами следующие.

Глава 14 (PPO) Обученная здесь модель вознаграждения r_ψ служит непосредственно сигналом вознаграждения при дообучении PPO. Вознаграждение на уровне токенов (10.15) — это в точности вознаграждение, вычисляемое на каждом шаге в оценщике преимущества PPO.

Глава 14 (DPO) Параметризация оптимальной стратегии (10.16) непосредственно приводит к целевой функции DPO, устраняющей необходимость обучать отдельную модель вознаграждения.

Глава 14 (GRPO) Групповая относительная оптимизация стратегии использует ORM на основе верификатора в качестве сигнала вознаграждения и нормализует вознаграждения внутри группы выбранных ответов, используя групповую статистику для оценки преимущества без обученного критика.

Замечание 10.3 (Историческая заметка). Формирование вознаграждения на основе потенциала было введено Нгом, Харадой и Расселлом на ICML 1999 и быстро стало стандартным инструментом в арсенале практиков RL. Модель Bradley–Terry восходит к 1952 году и была введена в статистике задолго до её принятия в машинном обучении. Её применение к RLHF было популяризировано Christiano et al. (2017), показавшими, что модель вознаграждения по предпочтениям, обученная на небольшом числе человеческих сравнений, может направлять RL в задачах Atari и робототехники. Различие между ORM и PRM было кристаллизовано Lightman et al. (2023) в контексте математического рассуждения, причём датасет PRM800K предоставил первый крупномасштабный ресурс пошаговых аннотаций.

Упражнения

Упражнение 10.1. Пусть $\mathcal{S} = \{s_1, s_2, s_3\}$ — пространство состояний детерминированного MDP с $\gamma = 0.9$ и истинным вознаграждением $R(s_1 \to s_2) = 0$, $R(s_2 \to s_3) = 0$, $R(s_3) = +1$ (терминал). Определим потенциальную функцию $\Phi(s_1) = 0$, $\Phi(s_2) = 0.5$, $\Phi(s_3) = 1.0$.

1. Вычислите бонус формирования $F(s_1, s_2)$ и $F(s_2, s_3)$.
2. Вычислите сформированную доходность $\tilde{G}_0$, начиная из s_1, и проверьте, что $\tilde{G}_0 = G_0 - \Phi(s_1)$.
3. Имеет ли сформированный MDP ту же оптимальную стратегию, что и исходный? Объясните, используя Теорему 10.1.

Упражнение 10.2. Постройте явный MDP и не-потенциальную функцию формирования $F(s, a, s')$, меняющую оптимальную стратегию. Ваш MDP должен иметь как минимум два действия хотя бы в одном состоянии, и вы должны идентифицировать, какое действие было оптимальным до формирования и какое становится оптимальным после. Проверьте, что F не может быть записана в виде $\gamma\Phi(s') - \Phi(s)$ для какой-либо Φ.

Упражнение 10.3. Докажите все четыре свойства модели Bradley–Terry, перечисленные в Утверждении 10.2. Для свойства 4 (инвариантность к сдвигу) объясните, почему это означает, что функция вознаграждения не является единственно идентифицируемой по данным предпочтений, и опишите дополнительное ограничение, которое можно наложить для её идентификации.

Упражнение 10.4. Предположим, что модель вознаграждения в данный момент присваивает оценки $r_\psi(x, y_w) = 1.2$ и $r_\psi(x, y_l) = 1.8$ для одной пары предпочтений.

1. Вычислите потерю $\mathcal{L}_{\text{RM}}$ для этой пары.
2. Вычислите градиенты $\partial\mathcal{L}/\partial r_\psi(x, y_w)$ и $\partial\mathcal{L}/\partial r_\psi(x, y_l)$ и опишите направление обновления.
3. Как изменяется величина градиента при росте разрыва $r_\psi(x, y_w) - r_\psi(x, y_l)$ от -0.6 до $+5$? Набросайте градиент как функцию разрыва.
4. Теперь предположим, что мы добавляем запас $m = 1.0$. Пересчитайте потерю и градиент. Увеличивается или уменьшается градиент? Объясните интуитивно.

Упражнение 10.5. Языковую модель просят решить следующую арифметическую задачу: «Если поезд едет со скоростью 60 миль/ч в течение 2.5 часа, а затем со скоростью 80 миль/ч в течение 1.5 часа, каково общее расстояние?» Модель генерирует следующее трёхшаговое решение:

Шаг 1: Расстояние на первом отрезке: $60 \times 2.5 = 150$ миль.

Шаг 2: Расстояние на втором отрезке: $80 \times 2 = 160$ миль. [*Ошибка: должно быть* 1.5 *часа*]

Шаг 3: Итого: $150 + 160 = 310$ миль. [*Ошибка перенесена из Шага 2; верный ответ 270*]

1. Какое вознаграждение присвоит ORM, если верификатор проверяет только конечный ответ? Что присвоит PRM?
2. Чем ORM и PRM отличаются в обращении с этим решением по сравнению с решением, приходящим к 270 по другому (но верному) трёхшаговому пути?

3. Опишите сценарий обучения, при котором ORM систематически переоценивала бы качество модели, делающей систематические ошибки на Шаге 2.

Упражнение 10.6. Используйте Теорему 10.4 для ответа на следующие вопросы об оптимальной стратегии π^* при KL-регуляризации.

1. Покажите, что $\log \pi^*(y|x) - \log \pi_{\text{ref}}(y|x) = r_\psi(x, y)/\beta - \log Z(x)$.

2. Используя результат (a), выразите $r_\psi(x, y)$ полностью через π^*, π_{ref} и $Z(x)$. Это тождество, используемое в выводе DPO.

3. Проверьте, что подстановка $r_\psi(x, y) - r_\psi(x, y')$ в формулу Bradley–Terry (10.4) приводит к сокращению $\log Z(x)$, оставляя вероятность предпочтения в терминах только π^* и π_{ref}.

Упражнение 10.7. Сформулируйте явление переоптимизации вознаграждения как задачу смещения-дисперсии в следующем смысле: прокси-модель вознаграждения r_ψ является оценкой истинного вознаграждения r^* с некоторой ошибкой обобщения $\varepsilon = \mathbb{E}[(r_\psi(x, y) - r^*(x, y))^2]$.

1. Аргументируйте, что разрыв $\mathbb{E}_{\pi_\theta}[r_\psi] - \mathbb{E}_{\pi_\theta}[r^*]$ (разрыв между прокси- и истинным вознаграждением при стратегии π_θ) растёт с дивергенцией KL $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$.

2. Предположим, что ошибка обобщения модели вознаграждения ε уменьшается по мере сбора большего объёма данных предпочтений. Как это влияет на оптимальный KL-бюджет d^*?

3. Опишите практический критерий остановки дообучения PPO, использующий как прокси-вознаграждение, так и отложенный набор человеческих оценок для обнаружения момента, когда разрыв между прокси- и истинным вознаграждением становится недопустимо большим.

Упражнение 10.8. Начиная со сглаженной по меткам функции потерь (10.10), покажите, что она эквивалентна стандартной функции потерь (10.6), возмущённой слагаемым $\frac{\epsilon}{2} \log \sigma(-\Delta)/(1 - \epsilon/2)$, и интерпретируйте это возмущение как регуляризатор, предотвращающий произвольный рост оценок вознаграждения.

Упражнение 10.9. Покажите, что штраф KL на уровне токенов в (10.15) можно интерпретировать как бонус формирования на основе потенциала в MDP уровня токенов, обсуждаемом в разделе 10.2. В частности:

1. Определите потенциал $\Phi(s_t) = \beta \log \pi_{\text{ref}}(a_t|s_t)$, вычисленный в каждой паре состояние-действие MDP.

2. Вычислите бонус формирования $F(s_t, a_t, s_{t+1}) = \gamma\Phi(s_{t+1}) - \Phi(s_t)$ и сравните его со слагаемым KL на уровне токенов.

3. Заключите, что KL-регуляризованная целевая функция RLHF эквивалентна нерегуляризованной RL-задаче на сформированном MDP, и укажите, какое следствие Теорема 10.1 имеет для этой эквивалентности.

Глава 11

Языковые модели и постобучение

Горький урок шестидесяти лет исследований в области ИИ состоит в том, что общие методы, использующие вычисления, в конечном счёте наиболее эффективны. Языковые модели — чистейшее выражение этого урока из всех открытых на сегодня.

Предшествующие десять глав построили аппарат обучения с подкреплением с первых принципов: марковские процессы принятия решений, динамическое программирование, методы Монте-Карло и разностного временного обучения, аппроксимация функций, градиенты стратегии и алгоритмы актор-критик. Глава 10 расширила этот аппарат на задачу обучения сигнала вознаграждения по данным предпочтений людей. Всё это в каком-то смысле было прелюдией. Последующие главы — о проксимальной оптимизации стратегии для языковых моделей, о прямой оптимизации предпочтений, о групповой относительной оптимизации стратегии — применяют RL к одному из наиболее значимых вычислительных артефактов нашей эпохи: большой языковой модели.

Прежде чем эти применения можно развить точно, читателю необходимо рабочее понимание того, что такое языковая модель, как она строится и почему RL — правильный инструмент для финальных стадий её обучения. Эта глава предоставляет такой фундамент. Она не является исчерпывающим обзором обработки естественного языка — для этого существует несколько отличных учебников, — но скорее самодостаточным мостиком между RL-теорией Части II и методами RLHF Части III.

Начнём с архитектуры Трансформера (Раздел 11.1), нейронной основы всех современных языковых моделей, затем формализуем авторегрессионное языковое моделирование как статистическую задачу (Раздел 11.2). Раздел 11.3 устанавливает ключевую концептуальную связь: генерация языка — это марковский процесс принятия решений, а языковая модель сама является стратегией. Вооружившись этим соответствием, Разделы 11.4–11.6 описывают трёхэтапный рецепт обучения, используемый на практике: массивное предобучение, дообучение с учителем (SFT) и выравнивание с помощью обучения с подкреплением. Раздел 11.7 рассматривает растущую роль синтетических данных, ставших незаменимыми на каждом этапе постобучения. Раздел 11.8 охватывает методологию оценки, а Раздел 11.9 показывает место этой главы в общей логике книги.

11.1 Архитектура Трансформера

Современные большие языковые модели почти повсеместно реализованы на основе архитектуры Трансформера, предложенной Vaswani et al. (2017). Для последующего RL-анализа достаточно понимать Трансформер на концептуальном уровне — без погружения во все детали учебника по NLP. Мы опишем четыре его ключевых компонента: само-внимание, позиционное кодирование, блоки прямой связи и нормализацию слоёв.

11.1.1 Само-внимание

Центральной инновацией Трансформера является механизм **само-внимания**, позволяющий каждой позиции в последовательности непосредственно обращать внимание на каждую другую. Для последовательности из n представлений токенов $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^d$, упакованных в матрицу $X \in \mathbb{R}^{n \times d}$, масштабированное скалярное произведение вычисляет

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{11.1}$$

где $Q = XW_Q$, $K = XW_K$, $V = XW_V$ — проекции запроса, ключа и значения с обученными весовыми матрицами $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$. Деление на $\sqrt{d_k}$ предотвращает рост скалярных произведений настолько, что softmax насыщается.

Интуитивно, каждый вектор запроса $\mathbf{q}_i$ сравнивается со всеми векторами ключей $\mathbf{k}_j$ для получения оценки внимания, которая затем используется для формирования взвешенного среднего векторов значений. Токен, представляющий слово «он», может сильно обращать внимание на существительное, которое оно заменяет, независимо от расстояния между ними. Это неограниченное рецептивное поле — в отличие от локальных окон свёрточных сетей или последовательного узкого места рекуррентных сетей — делает Трансформер столь мощным для языка.

На практике h голов внимания вычисляются параллельно и конкатенируются:

$$\mathrm{MultiHead}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W_O,$$

где каждая $\mathrm{head}_i = \mathrm{Attention}(XW_{Q,i}, XW_{K,i}, XW_{V,i})$ и $W_O \in \mathbb{R}^{hd_k \times d}$. Множество голов позволяет модели одновременно обращать внимание на разные аспекты контекста: одна голова может улавливать синтаксическое согласование, другая — кореференцию, третья — семантическое сходство.

11.1.2 Позиционное кодирование

Само-внимание *эквивариантно к перестановкам*: если позиции токенов перемешаны, оценки внимания перемешиваются аналогично, и модель не может отличить «кошка сидела на коврике» от какой-либо его перестановки. Позиционная информация должна быть введена явно.

В исходном Трансформере использовались фиксированные синусоидальные кодировки, добавляемые к эмбеддингам токенов. Современные модели, как правило, предпочитают **ротационные позиционные эмбеддинги** (RoPE; Su et al., 2021) или **ALiBi** (Press et al., 2022), кодирующие относительные, а не абсолютные позиции и лучше обобщающиеся на длины последовательностей, не встречавшиеся при обучении. Для наших целей достаточно знать, что после позиционного кодирования модель имеет доступ к порядку токенов.

11.1.3 Блоки прямой связи и нормализация слоёв

Каждый *слой* Трансформера состоит из подслоя само-внимания, за которым следует позиционно-точечный подслой прямой связи. Блок прямой связи применяет два линейных преобразования с нелинейностью между ними:

$$\mathrm{FFN}(\mathbf{x}) = W_2\, \sigma(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2,$$

где σ — как правило, функция активации GELU. Внутреннее измерение обычно в четыре раза превышает измерение модели: $W_1 \in \mathbb{R}^{4d \times d}$, $W_2 \in \mathbb{R}^{d \times 4d}$.

Нормализация слоёв (LayerNorm; Ba et al., 2016) применяется перед каждым подслоем (вариант *pre-norm*, используемый в большинстве современных моделей):

$$\mathrm{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sigma + \epsilon} \odot \boldsymbol{\gamma} + \boldsymbol{\beta},$$

где μ и σ — среднее и стандартное отклонение, вычисленные по измерению признаков, а $\boldsymbol{\gamma}, \boldsymbol{\beta}$ — обученные параметры масштабирования и сдвига. LayerNorm стабилизирует обучение, предотвращая бесконтрольный рост активаций по слоям.

Остаточное соединение охватывает каждый подслой, так что полное вычисление в одном слое Трансформера таково:

$$\begin{aligned} \mathbf{h}^{(l)} &= \mathbf{h}^{(l-1)} + \mathrm{MultiHead}(\mathrm{LayerNorm}(\mathbf{h}^{(l-1)})), \\ \mathbf{h}^{(l+1)} &= \mathbf{h}^{(l)} + \mathrm{FFN}(\mathrm{LayerNorm}(\mathbf{h}^{(l)})). \end{aligned}$$

Стек из L таких слоёв и проекция представления верхнего слоя на словарь задают выходное распределение модели. Рисунок 11.1 иллюстрирует один такой слой.

Замечание 11.1. Для языкового моделирования Трансформер использует архитектуру *только декодера* с **причинной маской**: во время само-внимания позиции i запрещено обращать внимание на позиции $j > i$. Это гарантирует, что предсказание токена t зависит только от токенов $1, \ldots, t-1$, сохраняя авторегрессионную структуру. Маска реализуется установкой $QK_{ij}^\top = -\infty$ при $j > i$ перед применением softmax, что обнуляет соответствующие веса внимания.

Масштаб современных Трансформеров поражает: GPT-3 имеет 175 миллиардов параметров в 96 слоях; LLaMA 3 70B имеет 80 слоёв со скрытым измерением 8192. Несмотря на масштаб, все эти модели разделяют одни и те же четыре компонента, описанных выше.

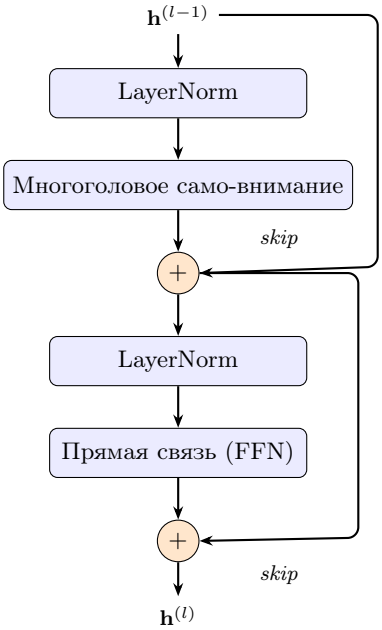


Рис. 11.1: Один слой декодера Трансформера. Входное представление $\mathbf{h}^{(l-1)}$ проходит через нормализацию слоёв перед многоголовым само-вниманием и вновь перед блоком прямой связи. Остаточные соединения (узлы +) позволяют градиентам течь непосредственно от выхода к входу, обеспечивая устойчивое обучение глубоких сетей.

11.2 Авторегрессионное языковое моделирование

Языковая модель — это распределение вероятностей на последовательностях токенов. Обозначим **словарь** как $\mathcal{V}$ — конечное множество из $|\mathcal{V}|$ токенов, обычно от 32 000 до 128 000 элементов после байт-парного кодирования (BPE; Sennrich et al., 2016). Последовательность токенов $\mathbf{w} = (w_1, w_2, \ldots, w_T)$ с $w_t \in \mathcal{V}$ может представлять предложение, документ, диалог или фрагмент кода.

Определение 11.1 (Авторегрессионная языковая модель)**. Авторегрессионная языковая модель** с параметрами θ — это параметризованное распределение на последовательностях токенов, определённое по правилу цепочки вероятностей:

$$p_\theta(w_1, \ldots, w_T) = \prod_{t=1}^{T} p_\theta(w_t \mid w_1, \ldots, w_{t-1}), \tag{11.2}$$

где каждый множитель $p_\theta(w_t \mid w_{1:t-1})$ — распределение на $\mathcal{V}$, вычисленное из контекста $(w_1, \ldots, w_{t-1})$.

Факторизация (11.2) точна по правилу цепочки и не вносит приближений. Трансформер реализует каждый условный $p_\theta(w_t \mid w_{1:t-1})$, отображая контекст в вектор **логитов** $\mathbf{z}_t \in \mathbb{R}^{|\mathcal{V}|}$ и применяя softmax:

$$p_\theta(w_t \mid w_{1:t-1}) = \mathrm{softmax}(\mathbf{z}_t/\tau)\,, \tag{11.3}$$

где $\tau > 0$ — параметр **температуры**. При $\tau = 1$ (по умолчанию) модель сэмплирует пропорционально вероятностям softmax. При $\tau \to 0$ распределение концентрируется на единственном токене с наибольшей вероятностью (жадное декодирование). Температуры выше 1 выравнивают распределение, увеличивая разнообразие за счёт связности.

11.2.1 Цель обучения: кросс-энтропийная потеря

Имея корпус из N документов, где документ i содержит токены $(w_1^{(i)}, \ldots, w_{T_i}^{(i)})$, цель обучения — **предсказание следующего токена**: минимизировать среднее отрицательное логарифмическое правдоподобие

$$\mathcal{L}_{\mathrm{NTP}}(\theta) = -\frac{1}{\sum_i T_i} \sum_{i=1}^{N} \sum_{t=1}^{T_i} \log p_\theta\Big(w_t^{(i)} \mid w_{1:t-1}^{(i)}\Big)\,. \tag{11.4}$$

Это кросс-энтропия между эмпирическим распределением данных и предсказанным распределением модели. Поскольку истинный следующий токен всегда является унитарным вектором над $\mathcal{V}$, кросс-энтропия

сводится к отрицательной логарифмической вероятности верного токена. Эквивалентно, минимизация (11.4) максимизирует совместную вероятность, присваиваемую моделью обучающему корпусу.

Определение 11.2 (Перплексия)**.** **Перплексия** языковой модели p_θ на тестовой последовательности $(w_1, \ldots, w_T)$ равна

$$\mathrm{PPL}(\theta) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p_\theta(w_t \mid w_{1:t-1})\right) = \exp(\mathcal{L}_{\mathrm{NTP}}). \tag{11.5}$$

Перплексия — это геометрическое среднее обратных вероятностей, присваиваемых моделью каждому токену. Перплексия k означает, что модель в среднем так же неопределённа, как если бы ей приходилось выбирать равномерно из k равновероятных токенов. Современные большие языковые модели достигают перплексий ниже 10 на стандартных бенчмарках по сравнению с базовым значением случайного ответа $|\mathcal{V}| \approx 50\,000$.

11.2.2 Законы масштабирования

Одним из наиболее практически важных эмпирических открытий в языковом моделировании является то, что качество (измеренное потерей на валидации) следует предсказуемым **законам масштабирования** в зависимости от размера модели N, размера датасета D и вычислительного бюджета $C \approx 6ND$. Kaplan et al. (2020) установили, что потеря на валидации убывает как степенная функция от каждого из этих количеств по отдельности. Hoffmann et al. (2022) — в том, что сегодня известно как **законы масштабирования Chinchilla** — показали, что предыдущие крупные модели (включая GPT-3) были значительно недообучены относительно своего размера: оптимальное по вычислениям обучение требует примерно 20 токенов на параметр модели. Результат Chinchilla означает, что модель с N параметрами следует обучать приблизительно на $20N$ токенах для минимизации потерь при данном вычислительном бюджете. Это наблюдение изменило подход к проектированию передовых моделей: LLaMA 3 8B, например, обучалась на 15 триллионах токенов — значительно больше оптимального по Chinchilla количества, что отражает практическую важность эффективности вывода над эффективностью обучения.

11.3 Генерация языка как марковский процесс принятия решений

Описанный выше авторегрессионный процесс генерации имеет элегантную интерпретацию как марковский процесс принятия решений. Это соответствие является концептуальным фундаментом каждого алгоритма RLHF в Части III: как только мы распознаём языковую модель как стратегию, весь RL-аппарат, разработанный в Главах 8 и 9, применяется непосредственно.

Определение 11.3 (MDP уровня токенов)**.** Пусть $x = (x_1, \ldots, x_m)$ обозначает **промпт** (последовательность из m токенов), а $y = (y_1, \ldots, y_T)$ — **ответ**, подлежащий генерации, с $y_t \in \mathcal{V}$. **MDP уровня токенов** — это кортеж $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$, где:

- **Пространство состояний** $\mathcal{S}$: состояние на шаге t — полный контекст, увиденный к этому моменту,

  $$s_t = (x_1, \ldots, x_m, y_1, \ldots, y_{t-1}),$$

  то есть конкатенация промпта и всех токенов, сгенерированных до (но не включая) шага t. Начальное состояние — $s_0 = x$.

- **Пространство действий** $\mathcal{A} = \mathcal{V}$: действие на шаге t — следующий токен $a_t = y_t$. Пространство действий имеет мощность $|\mathcal{V}| \sim 10^4$–10^5.

- **Динамика переходов** P: детерминированная,

  $$s_{t+1} = (x, y_1, \ldots, y_t) = s_t \parallel y_t,$$

  где $\parallel$ обозначает конкатенацию. Переход полностью определён текущим состоянием и действием.

- **Функция вознаграждения** R: разреженная и терминальная,

  $$R(s_t, a_t) = \begin{cases} r(x, y) & \text{если } a_t = \textsc{eos} \text{ или } t = T, \\ 0 & \text{иначе,} \end{cases}$$

  где $r : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ — скалярная оценка полного ответа (например, обученная модель вознаграждения или балл верифицируемой корректности), а EOS — токен конца последовательности.

- **Коэффициент дисконтирования** $\gamma = 1$ (эпизодическая задача с единственным терминальным вознаграждением).
- **Стратегия** π_θ: сама языковая модель,

$$\pi_\theta(a_t \mid s_t) = p_\theta(y_t \mid x, y_{1:t-1}).$$

Утверждение 11.1. *Суммарная доходность в MDP уровня токенов с $\gamma = 1$ равна вознаграждению, присвоенному полному ответу:*

$$G_0 = \sum_{t=0}^{T} R(s_t, a_t) = r(x, y). \tag{11.6}$$

Следовательно, цель RLHF

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r(x, y)\big] \tag{11.7}$$

— это в точности ожидаемая доходность стратегии π_θ в MDP уровня токенов.

Доказательство. Все промежуточные вознаграждения по определению нулевые; единственное ненулевое слагаемое в сумме — $R(s_T, a_T) = r(x, y)$. Взятие ожидания по $x \sim \mathcal{D}$ и распределению траектории, порождённому π_θ, непосредственно даёт (11.7) из определения ожидаемой доходности. □

Эта MDP-формулировка обнаруживает три структурные особенности, отличающие генерацию языка от классических RL-задач Части II.

Замечание 11.2 (Отличительные черты MDP уровня токенов). 1. **Огромное пространство действий.** При $|\mathcal{A}| \sim 10^5$ хранить функцию ценности $Q(s, a)$ явно для всех действий вычислительно нецелесообразно. Это исключает табличные методы (Глава 4) и большинство вариантов Q-обучения, мотивируя подходы на основе градиента стратегии и актор-критик из Глав 8–9.

2. **Длинный горизонт с разреженным вознаграждением.** Типичный ответ содержит 200–2000 токенов. Единственное терминальное вознаграждение должно быть приписано сотням предшествующих действий — проблема присвоения кредита в её крайней форме. Модели вознаграждения по процессу (Раздел 10.6 в Главе 10) решают её, предоставляя обратную связь на уровне шагов.

3. **Детерминированные переходы.** После добавления токена состояние определяется с полной определённостью. Всё стохастическое происходит от самой стратегии, а не от среды. Это упрощает некоторые теоретические анализы, но затрудняет исследование: модель должна полагаться на своё собственное распределение сэмплирования для посещения новых завершений.

Пример 11.1 (Подсчёт токенов в MDP)**.** Рассмотрим промпт x = «`Какова столица Франции?`» (6 токенов) и ответ y = «`Столицей Франции является Париж.`» (7 токенов). Эпизод имеет $T = 7$ шагов. На шаге $t = 5$ состояние — $s_5 = (x, \texttt{Столицей}, \texttt{Франции}, \texttt{является}, \texttt{Париж})$. Действие — $a_5 =$ `.`. Вознаграждение нулевое для всех $t < 7$, и $r(x, y)$ даётся при $t = 7$ моделью вознаграждения, оценивающей полный ответ. Если модель вознаграждения оценивает фактическую корректность и присваивает $r = +1.4$, то $G_0 = 1.4$ независимо от того, какие промежуточные токены были сгенерированы.

11.4 Предобучение

Первый этап построения способной языковой модели — **предобучение**: обучение случайно инициализированного Трансформера на огромном корпусе текста с использованием цели предсказания следующего токена (11.4). Предобучение — безусловно наиболее вычислительно затратный этап и обычно потребляет более 95% общего бюджета обучения передовой модели.

11.4.1 Данные

Корпусы для предобучения собираются из разнообразных источников в интернете и институциональных коллекций. Общие компоненты включают:

Common Crawl Ежемесячные снимки веба; после агрессивного дедублирования и фильтрации качества обычно составляет 50–70% обучающих токенов.

Книги Project Gutenberg, Books3 и аналогичные коллекции; богаты длинным структурированным текстом со связным повествованием.

Научная литература	ArXiv, PubMed, Semantic Scholar; критически важна для способностей к техническому рассуждению.
Код	Репозитории GitHub на десятках языков программирования; сильно коррелирует с улучшенным математическим и структурированным рассуждением.
Википедия и Викиданные	Высококачественный энциклопедический текст с фактическими утверждениями; как правило, избыточно представлен относительно его исходного размера.

Общее число токенов для современных передовых моделей составляет от одного до пятнадцати триллионов. LLaMA 3 обучалась на 15 триллионах токенов; обучающие данные GPT-4 не раскрываются публично, но, по имеющимся сведениям, сопоставимы по масштабу. Сборка такого корпуса требует значительных инженерных усилий: удаление почти-дубликатов (с использованием хеширования MinHash с локальной чувствительностью), идентификация языка, фильтрация качества (отклонение документов ниже порога перплексии, установленного небольшой эталонной моделью) и очистка персональных данных.

11.4.2 Потеря предобучения и эмерджентные способности

Предобучение оптимизирует (11.4) с использованием вариантов AdamW (Loshchilov & Hutter, 2019) с косинусным расписанием скорости обучения. Градиентные контрольные точки, арифметика смешанной точности (bfloat16) и распределённое обучение на тысячах GPU или TPU — стандартные инженерные решения.

Замечательное эмпирическое наблюдение состоит в том, что по мере роста модели и обучения на большем количестве данных она не просто запоминает обучающий корпус — она приобретает **эмерджентные способности**, явно не присутствующие в обучающем сигнале. Модель, обученная только предсказывать следующее слово, учится:

- переводить между языками, которые она никогда не видела в парах;
- писать и отлаживать компьютерные программы;
- решать многошаговые математические задачи, показывая промежуточные вычисления;
- отвечать на фактические вопросы, используя знания, неявно закодированные в корпусе.

Wei et al. (2022) задокументировали, что некоторые способности появляются резко при определённых размерах моделей, что предполагает скачкообразные фазовые переходы в ландшафте потерь, а не плавную интерполяцию. Точный механизм эмерджентности остаётся активной областью исследований.

11.4.3 Базовые модели и их ограничения

Результатом предобучения является **базовая модель** (также называемая **фундаментальной моделью** или **предобученной моделью**). По заданному промпту базовая модель продолжает его в стиле того текста, с которым она научилась ассоциировать этот префикс. Такое поведение полезно для задач завершения текста, но проблематично при интерактивном использовании: на вопрос «Как испечь ржаной хлеб на закваске?» базовая модель может ответить большим количеством вопросов (как будто продолжая документ FAQ), а не полезным рецептом.

Что ещё важнее, базовая модель не имеет механизма отклонения вредных запросов, разграничения фактов и домыслов или принятия разговорного регистра, ожидаемого от ассистента. Эти ограничения мотивируют два последующих этапа обучения.

11.5 Supervised Fine-Tuning

Supervised fine-tuning (SFT) адаптирует базовую модель для следования инструкциям, обучая её на тщательно отобранном датасете пар (инструкция, ответ). Процедура по существу идентична предобучению — предсказание следующего токена через кросс-энтропийную потерю — но на значительно меньшем, более качественном датасете демонстраций.

11.5.1 Формат данных и шаблоны инструкций

Данные SFT состоят из пар $(x^{(i)}, y^{(i)})$, где $x^{(i)}$ — инструкция (запрос пользователя, вопрос или описание задачи), а $y^{(i)}$ — высококачественный ответ, написанный или отобранный аннотатором-человеком или отфильтрованный из существующих высококачественных источников. На практике разговоры многоходовые, поэтому вход $x^{(i)}$ может включать полную историю диалога.

Для структурирования входа для модели **шаблон чата** оборачивает каждый ход специальными токенами:

```
<|system|> Ты полезный ассистент. <|end|>
<|user|> Какова столица Франции? <|end|>
<|assistant|> Столицей Франции является Париж. <|end|>
```

Модель обучается предсказывать только токены ассистента; токены пользователя и системы *маскируются* при вычислении потерь, чтобы градиент не стимулировал модель имитировать речевые паттерны пользователя.

11.5.2 Функция потерь SFT

Потеря SFT — та же кросс-энтропия, что (11.4), но применяемая только к токенам ответа:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T_i}\log p_\theta\Big(y_t^{(i)} \mid x^{(i)}, y_{1:t-1}^{(i)}\Big)\,. \tag{11.8}$$

Градиент (11.8) по θ вычисляется только по позициям, где y_t — токен ассистента. Эта асимметрия существенна: она учит модель генерировать хорошие ответы, не толкая её имитировать запросы пользователя или системные промпты.

11.5.3 Датасеты и примеры

Stanford Alpaca (Taori et al., 2023) стал одним из первых широко воспроизведённых SFT-датасетов: 52 000 демонстраций следования инструкциям, сгенерированных с помощью промптинга `text-davinci-003` методом Self-Instruct (Wang et al., 2023). Несмотря на скромный размер, Alpaca показал, что 7-миллиардная модель LLaMA может приобрести полезное поведение следования инструкциям.

Tülu (Wang et al., 2023; Ivison et al., 2024) — семейство моделей с настройкой по инструкциям, обученных на смеси отобранных датасетов с открытым исходным кодом. Tülu 3 (2024) использует более миллиона примеров следования инструкциям, собранных из разнообразных источников с тщательным дедублированием и фильтрацией качества, и включает как SFT, так и этапы дообучения на основе предпочтений. Он служит сильной базовой линией с открытым исходным кодом для изучения полного пайплайна постобучения.

11.5.4 Смещение воздействия и потолок SFT

SFT использует **принудительное обучение с учителем**: на каждом шаге t во время обучения модель получает на вход *истинный* предыдущий токен y_{t-1}, а не своё собственное предсказание $\hat{y}_{t-1}$. Во время вывода это предположение нарушается — модель должна обусловливаться на собственных (возможно, ошибочных) предыдущих выходах. Это несоответствие, известное как **смещение воздействия** (Ranzato et al., 2015), может вызывать накопление ошибок: небольшая ошибка на шаге t помещает модель в состояние, с которым она никогда не сталкивалась при обучении, делая дальнейшие ошибки более вероятными.

В более фундаментальном смысле SFT ограничен качеством своих демонстраций. Независимо от размера датасета SFT модель не может научиться производить ответы лучше, чем написавшие их аннотаторы. Для задач, где человеческие демонстрации дороги при масштабировании, или где пространство возможных хороших ответов богаче, чем может зафиксировать любой конечный датасет, этот потолок становится ограничивающим. Эти наблюдения мотивируют этап выравнивания — обучения на сравнениях предпочтений и сигналах вознаграждения, а не на фиксированных демонстрациях.

11.6 Пайплайн постобучения

Разработка современных языковых моделей следует трёхэтапному пайплайну, который можно понять как прогрессивный сдвиг обучающего сигнала от самообучающейся статистики к предпочтениям людей и верифицируемой корректности. Рисунок 11.2 иллюстрирует полный пайплайн.

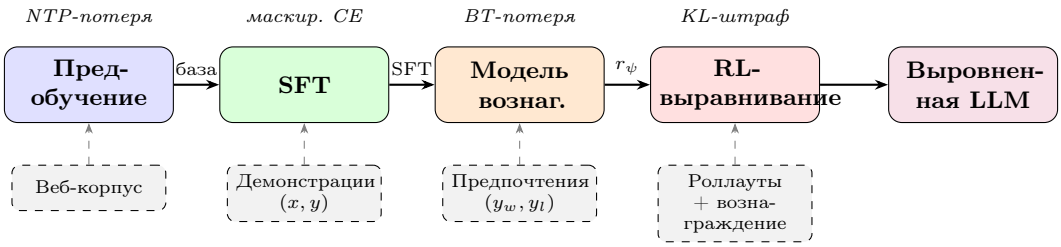


Рис. 11.2: Полный пайплайн постобучения для большой языковой модели. Предобучение на веб-текстах создаёт базовую модель с широкими знаниями о мире. SFT на отобранных демонстрациях учит формату следования инструкциям. Моделирование вознаграждения обучает скалярному сигналу предпочтения по сравнениям людей. RL-выравнивание оптимизирует стратегию относительно этого вознаграждения, ограничивая дрейф штрафом KL. Главы 14–17 подробно развивают каждый метод RL-выравнивания.

11.6.1 Взаимодействие этапов

Три этапа не являются независимыми: каждый существенно зависит от качества предыдущего.

Предобучение устанавливает потолок знаний. Этапы SFT и RL могут изменить поведение модели, но не могут привить знания, которых у базовой модели нет. Базовая модель, никогда не видевшая учебников по математике, не научится решать интегралы через настройку по инструкциям. Именно поэтому курирование данных предобучения столь значимо.

SFT устанавливает поведенческий априор. Модель SFT служит как отправной точкой, так и опорной стратегией π_{ref} для этапа RL. Слагаемое штрафа KL $\beta\, D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ (введённое в Главе 10) привязывает стратегию RL вблизи распределения SFT. Плохо обученная модель SFT, следовательно, ограничивает эффективность RL-выравнивания.

Моделирование вознаграждения определяет сигнал выравнивания. Модель вознаграждения r_ψ — единственный источник обратной связи во время RL-обучения. Её точность, калибровка и охват распределения задач непосредственно определяют качество выровненной модели. Глава 10 подробно развивает теорию моделирования вознаграждения; Главы 14–17 показывают, как модель вознаграждения используется в PPO, DPO и GRPO соответственно.

11.6.2 Что оптимизирует каждый этап

Краткое сравнение трёх этапов приведено в Таблице 11.1.

Таблица 11.1: Сравнение трёх этапов постобучения.

Этап	Данные	Цель	Парадигма	Ключевое ограничение
Предобучение	Неразмеченный текст	NLL следующего токена	Самообучение	Нет поведенческого контроля
SFT	Пары (x, y)	NLL с маскировкой	Обучение с учителем	Ограничен качеством аннотаторов
RL-выравнивание	(x, y_w, y_l) или $r(x, y)$	KL-штрафованное вознаграждение	Обучение с подкреплением	Переоптимизация вознаграждения

11.7 Синтетические данные для постобучения

Аннотирование людьми — ограничивающий ресурс постобучения. Квалифицированный аннотатор производит около 20–50 высококачественных примеров следования инструкциям в час; один тренировочный прогон для передовой модели может потребовать миллионов. Этот арифметический разрыв побудил отрасль всё больше полагаться на **синтетические данные**: примеры, генерируемые моделями ИИ, а не непосредственно людьми-авторами. Этот раздел обозревает основные методы, их теоретические свойства и практические риски, которые они вносят.

11.7.1 Почему синтетические данные необходимы

Три структурных давления вынуждают использовать синтетические данные в масштабе.

Объём. Эффективное SFT и дообучение по предпочтениям требуют десятков тысяч до миллионов примеров, охватывающих разнообразные задачи, темы, уровни сложности и стили ответов. Аннотирование людьми в таком масштабе непомерно дорого: сборка одного миллиона высококачественных пар предпочтений по $5 за сравнение обходится в $5 миллионов, без учёта несогласия аннотаторов и накладных расходов на контроль качества.

Охват. Аннотаторы естественным образом кластеризуются вокруг знакомых областей. Редкие, но важные темы — нишевые научные субдисциплины, малоизвестные языки программирования, малоресурсные человеческие языки — систематически недопредставлены в сгенерированных людьми данных. Синтетическая генерация может быть направлена на эти пробелы с помощью инструкции.

Калибровка сложности. Обучение RL получает выгоду от примеров подходящего уровня сложности: задач, которые модель иногда, но не всегда решает, так чтобы градиент вознаграждения был информативным. Генерировать примеры с точно заданным целевым уровнем сложности практически невозможно с аннотаторами-людьми, но это можно сконструировать через синтетические пайплайны.

11.7.2 Self-Instruct

Self-Instruct (Wang et al., 2023) — основополагающий метод синтетической генерации инструкций. Пайплайн начинается с небольшого начального набора из 175 написанных людьми инструкций, охватывающих разнообразный круг задач. На каждой итерации пайплайн запрашивает у большой языковой модели (**учителя**) несколько из этих исходных инструкций и просит сгенерировать новые инструкции, классифицировать их по типу задачи и построить пары «вход — выход». Новые инструкции, слишком похожие на уже имеющиеся (по метрике ROUGE-L), отбрасываются; остальные добавляются в пул. Повторяя этот процесс, Wang et al. расширили исходный набор до 52 000 пар следования инструкциям — датасета Stanford Alpaca — при стоимости около $500 в API-вызовах к `text-davinci-003`.

Статья Self-Instruct продемонстрировала, что 7-миллиардная модель LLaMA, дообученная на этих данных, сравнялась по способности следования инструкциям с моделью, обученной на частных написанных людьми данных, что говорит о том, что качество модели-учителя — главное узкое место, а не само по себе различие между человеческим и синтетическим происхождением.

11.7.3 Дистилляция из моделей-учителей

Более прямой подход — использовать более сильную модель (учителя) для генерации ответов, на которых затем обучается более слабая модель (ученик). Учитель может быть доступен через API (OpenAI, Anthropic, Google) или локально запущенную модель с открытыми весами. Процедура:

1. Собрать или сгенерировать набор промптов $\{x^{(i)}\}$, охватывающих желаемое распределение задач.
2. Для каждого промпта сэмплировать один или несколько ответов от учителя: $y^{(i)} \sim \pi_{\text{teacher}}(\cdot \mid x^{(i)})$.
3. Дообучить ученика на результирующих парах $(x^{(i)}, y^{(i)})$ с помощью (11.8).

Дистилляция особенно эффективна для *передачи формата*: обучения малой модели перенимать стиль, структуру и паттерны пошагового рассуждения большей модели. Серия Orca (Mukherjee et al., 2023) показала, что 13-миллиардная модель, обученная на подробных объяснительных ответах GPT-4, превзошла модели во много раз большего размера на определённых бенчмарках, что говорит о том, что ключевым ингредиентом является процесс рассуждения учителя, а не только его окончательный ответ.

Главное ограничение дистилляции — правовое и коммерческое: большинство API передовых моделей запрещают использовать их выходы для обучения конкурирующих моделей. Учителя с открытыми весами (LLaMA 3, Mistral, Qwen) под это ограничение не подпадают и поэтому всё шире используются на практике.

11.7.4 Evol-Instruct и нарастание сложности

Evol-Instruct (Xu et al., 2023), введённый в рамках проекта **WizardLM**, заметил, что Self-Instruct склонен производить инструкции однородной (и часто низкой) сложности. Их идея — *эволюционировать* существующие инструкции, многократно применяя один из нескольких операторов мутации:

- **Добавить ограничения**: добавить требование, сужающее пространство ответов.
- **Углубить**: запросить более подробное или строгое рассмотрение.
- **Расширить**: обобщить инструкцию на более широкий контекст.
- **Добавить многошаговое рассуждение**: ввести требование, предполагающее поэтапный вывод.

- **Эволюция вширь**: сгенерировать совершенно новую, но связанную инструкцию.

Каждый шаг эволюции выполняется через промптинг модели-учителя. Результирующий датасет охватывает широкий спектр уровней сложности, что прямо выгодно для RL-обучения. WizardLM-2 (2024), обученный на данных Evol-Instruct в большом масштабе, достигает конкурентоспособного качества с моделями вдвое большего числа параметров на бенчмарках кодирования и математического рассуждения.

11.7.5 Генерация на основе персон

Дополнительная стратегия увеличения разнообразия — **генерация на основе персон**: вместо варьирования инструкции менять демографическую или профессиональную идентичность гипотетического пользователя. Модель-учитель запрашивается с описанием персоны, например «Вы аспирант второго года по вычислительной биологии» или «Вы старшеклассник, испытывающий трудности с алгеброй». Результирующие инструкции и ответы естественно кластеризуются вокруг разных словарей, уровней сложности и стилей общения.

Chan et al. (2024) показали, что генерация с учётом персон, применённая в масштабе с использованием 1 миллиарда уникальных синтетических персон, полученных из веб-текстов, даёт датасеты инструкций с существенно более высоким разнообразием, чем генерация без условия. Датасет Tülu 3 использует генерацию на основе персон как один из компонентов своей смеси данных.

11.7.6 Синтез данных предпочтений

Помимо демонстраций следования инструкциям, RL-выравнивание требует **данных предпочтений**: троек (x, y_w, y_l), в которых человек (или модель) считает y_w лучше y_l. Собирать такие сравнения от людей в масштабе ещё дороже, чем демонстрации, поскольку каждая аннотация требует от человека прочтения и сравнения двух полных ответов. Синтетическая генерация предпочтений решает это узкое место через три взаимодополняющих механизма.

Множественное сэмплирование ответов и оценка LLM. Наиболее распространённый подход:

1. Для каждого промпта x сэмплировать $k \geq 2$ ответов от текущей или модели-учителя.
2. Оценить каждый ответ с помощью **модели-судьи** (обычно GPT-4 или судьи с открытыми весами) по рубрике.
3. Формировать пары предпочтений, принимая y_w как ответ с наибольшим баллом и y_l как любой ответ с меньшим баллом (или специально с наименьшим).

UltraFeedback (Cui et al., 2023) — канонический датасет, собранный этим методом: 64 000 промптов, каждый связан с четырьмя ответами от различных моделей, оцененными GPT-4 по четырём рубрикам (следование инструкциям, правдивость, честность, полезность). Результирующий датасет предпочтений использовался для обучения десятков выровненных моделей, включая серию Zephyr.

Constitutional AI и RLAIF. Bai et al. (2022) из Anthropic представили **Constitutional AI** (CAI), в котором сама модель ИИ критикует и пересматривает свои ответы в соответствии с фиксированным набором принципов («конституцией»). Цикл критика-пересмотр генерирует улучшенные ответы без вмешательства людей, а результирующие пары (исходный, пересмотренный) можно использовать как данные предпочтений. Тесно связанный **RLAIF** (Reinforcement Learning from AI Feedback; Lee et al., 2023) использует разметчик ИИ вместо разметчиков-людей для аннотирования того, какой из двух ответов предпочтительнее. Lee et al. показали, что RLAIF и человеческий RLHF дают модели сопоставимого качества на нескольких стандартных бенчмарках, что говорит о том, что узким местом в выравнивании является не источник меток предпочтений (человек или ИИ), а их точность и охват.

Оценка по рубрикам. Вместо единственного общего предпочтения современные пайплайны часто просят судью оценить по нескольким измерениям — корректность, лаконичность, безопасность, соответствие инструкции — и агрегировать. Рубрики также могут быть задачеспецифичными: для математики рубрика проверяет, верен ли окончательный ответ и валидны ли промежуточные шаги. Tülu 3 использует рубрики с верификаторами для математических и кодовых задач, полностью заменяя модель-судью в областях, где корректность проверяема.

11.7.7 Фильтрация данных и контроль качества

Синтетическая генерация данных дешева, но шумна. Механизмы контроля качества существенны для предотвращения деградации производительности модели от низкокачественных примеров. Стандартные техники включают:

- **Фильтрация по перплексии**: отклонять ответы, которым текущая модель присваивает чрезвычайно высокую или низкую перплексию (первое указывает на деформированный текст; второе — на почти запомненный шаблон).
- **Фильтрация по длине**: отбрасывать ответы за пределами разумного диапазона длин (очень короткие часто лишены деталей; очень длинные часто заполняются пустыми словами).
- **Дедупликация**: удалять почти-дубликаты с помощью MinHash или косинусного сходства на основе эмбеддингов для предотвращения запоминания модели повторяющихся паттернов.
- **Фильтрация по модели вознаграждения**: сохранять только ответы, которым модель вознаграждения присваивает балл выше порога. Это создаёт свойство самосогласованности: данные, используемые для обучения, уже считаются хорошими моделью вознаграждения.
- **Фильтрация по сложности**: для RL-обучения сохранять только примеры в «обучаемой зоне» — где частота успеха модели составляет от 20% до 80%. Задачи, которые модель всегда решает, не дают градиента; задачи, которые она никогда не решает, вносят только шум.

11.7.8 Коллапс модели и меры противодействия

Фундаментальная проблема обучения на синтетических данных — **коллапс модели** (Shumailov et al., 2024): если модели обучаются на своих собственных выходах (или выходах аналогично обученных моделей), обучающее распределение может прогрессивно сужаться, заставляя модель терять широту и разнообразие, присутствующие в исходном написанном людьми корпусе. Математически, если обучающее распределение в поколении g — p_g, а распределение синтетических данных является приближением p_{g-1}, итерирование этого процесса вызывает геометрическое уменьшение дисперсии p_g, в конечном счёте коллапсируя к узкой моде.

Определены четыре меры противодействия:

1. **Смешивание с человеческими данными.** Всегда сохранять долю подлинного написанного людьми текста в обучающей смеси. Доля не обязана быть большой: Gerstgrasser et al. (2024) показали, что даже 5–10% человеческих данных достаточно для теоретического предотвращения коллапса.
2. **Разнообразный ансамбль учителей.** Использование нескольких моделей-учителей с различными архитектурами и историями обучения вносит дистрибуционное разнообразие, которое единственный учитель не может предоставить.
3. **Агрессивная дедупликация.** Почти-дублированные синтетические примеры — механизм распространения коллапса моды; их удаление разрывает положительную обратную связь.
4. **Водяные знаки и отслеживание происхождения.** Маркировка синтетических данных при генерации позволяет последующим кураторам мониторировать долю синтетических данных и ограничивать её по мере необходимости.

11.7.9 Ключевые синтетические датасеты

Таблица 11.2 суммирует наиболее влиятельные синтетические датасеты в литературе по постобучению.

Таблица 11.2: Основные синтетические датасеты, используемые в постобучении.

Датасет	Год	Размер	Тип	Метод
Stanford Alpaca	2023	52K	SFT	Self-Instruct (GPT-3.5)
UltraFeedback	2023	256K пар	Предпочтения	LLM-судья (GPT-4)
Tülu 3	2024	1M+	SFT + Предп.	Смесь, персоны, рубрики
OpenThoughts 3	2025	1M+	SFT (CoT)	Дистилляция (DeepSeek-R1)

OpenThoughts 3 (2025) воплощает нарождающийся класс *ориентированных на рассуждение* синтетических датасетов: один миллион цепочек рассуждений для математических, научных и кодовых задач, дистиллированных из DeepSeek-R1. Обучение базовой модели на OpenThoughts 3 даёт модель, демонстрирующую многошаговое рассуждение в формате ⟨thinking⟩, существенно улучшая производительность на олимпиадной математике (AIME, MATH) и бенчмарках кодирования (HumanEval).

11.8 Оценка языковых моделей

Оценка языковых моделей существенно сложнее, чем оценка классических RL-агентов. Стратегия в мире клеток либо достигает цели, либо нет; качество разговорного ответа не поддаётся такой бинарной характеристике. В этом разделе обозреваются основные парадигмы оценки.

11.8.1 Автоматизированные бенчмарки

Автоматизированные бенчмарки обеспечивают дешёвую, воспроизводимую и объективную оценку. Наиболее широко используемые включают:

MMLU (Hendrycks et al., 2021) Massive Multitask Language Understanding: 57 академических предметов от уровня старшей школы до профессионального, в формате множественного выбора. Проверяет широту фактических знаний.

HellaSwag (Zellers et al., 2019) Логический вывод на основе здравого смысла: выбрать наиболее вероятное продолжение описания деятельности. Проверяет заземлённое рассуждение на основе здравого смысла.

GSM8K (Cobbe et al., 2021) Математика уровня начальной школы: 8500 многошаговых арифметических текстовых задач. Ответы верифицируются проверкой окончательного числового результата.

HumanEval (Chen et al., 2021) Генерация кода: 164 задачи на программирование на Python, оцениваемые запуском юнит-тестов против сгенерированного кода.

MATH (Hendrycks et al., 2021) Олимпиадная математика: 12 500 задач уровня AMC/AIME, требующих символьных манипуляций и многошагового доказательства.

Автоматизированные бенчмарки имеют критический режим отказа: модели могут быть **загрязнены**, если данные оценки появляются в корпусе предобучения. Оценка GPT-4 на GSM8K улучшилась на 7 процентных пунктов при реконструкции оценочного разбиения с новыми формулировками задач, что говорит о частичном запоминании. Ответственная оценка требует проверки n-граммового пересечения между бенчмарком и обучающими данными.

11.8.2 Оценка людьми

Для открытых задач, где не существует истинного ответа — творческое письмо, диалог в открытом домене, реферирование — оценка людьми остаётся золотым стандартом. Аннотаторов обычно просят сравнить два ответа бок о бок (A/B-оценка) и указать, какой лучше по одному или нескольким измерениям. Платформы, такие как Chatbot Arena (Chiang et al., 2024), собирают попарные предпочтения в масштабе от пользователей-добровольцев, производя Эло-стиль ранжирования моделей.

Оценка людьми дорога, медленна и страдает систематическими смещениями: аннотаторы предпочитают более длинные ответы (смещение по многословности), ответы, соглашающиеся с их предварительными взглядами (подхалимство), и ответы от моделей, которые они считают более способными (якорение). Контроль за этими смещениями требует тщательного экспериментального дизайна и больших выборок.

11.8.3 LLM как судья

LLM-as-judge (Zheng et al., 2023) использует сильную языковую модель (обычно GPT-4) как автоматизированного оценщика, заменяя аннотаторов-людей в A/B-сравнениях. Судье подают промпт с инструкцией, двумя ответами и структурированной рубрикой, после чего он должен вынести суждение о предпочтении и кратко его объяснить. MT-Bench и AlpacaEval 2.0 используют эту парадигму для ранжирования моделей, настроенных на следование инструкциям.

LLM-as-judge наследует смещения модели-судьи и вносит **смещение самоулучшения**: модели от того же разработчика, что и судья, как правило, оцениваются более благоприятно. Меры противодействия включают использование нескольких судей, запрос объяснений в виде цепочки рассуждений перед окончательным вердиктом и перестановку позиций (оценку каждой пары в обоих порядках с усреднением).

11.8.4 Оценка модели вознаграждения

Когда конечным использованием оценки является выбор между кандидатными моделями во время RL-обучения, обученная модель вознаграждения r_ψ используется как оценщик. Точность модели вознаграждения измеряется как доля отложенных пар предпочтений, для которых она правильно предсказывает предпочтительный с точки зрения человека ответ. Как отмечено в Главе 10, хорошо откалиброванные модели вознаграждения достигают 70–80% точности на отложенных человеческих предпочтениях. Помимо точности, распределение оценок вознаграждения и их корреляция с независимыми человеческими рейтингами мониторируются для обнаружения систематических смещений.

11.9 Резюме

В этой главе была представлена инфраструктура языкового моделирования, лежащая в основе методов RLHF, развиваемых в последующих главах.

Языковая модель — это авторегрессионное распределение вероятностей на последовательностях токенов, реализованное с использованием архитектуры Трансформера. Предобучение на веб-текстах с использованием предсказания следующего токена прививает широкие мировые знания и эмерджентные способности; supervised fine-tuning на отобранных парах инструкция-ответ учит поведенческому формату, ожидаемому от ассистента. Затем этап RL-выравнивания со штрафом KL оптимизирует сигнал вознаграждения — выученный из сравнений предпочтений людей или верифицируемых исходов задач — для поднятия модели выше потолка, установленного демонстрациями.

Ключевым концептуальным мостиком является MDP уровня токенов (Определение 11.3): генерация языка точно отображается на MDP-фреймворк Глав 3–9, где языковая модель — стратегия, словарь — пространство действий, а терминальное вознаграждение от человека или модели вознаграждения. Это соответствие оправдывает прямое применение методов градиента стратегии к обучению языковых моделей.

Синтетические данные стали незаменимым компонентом каждого этапа постобучения: Self-Instruct и Evol-Instruct для разнообразия инструкций SFT, пайплайны LLM-судьи для данных предпочтений в масштабе и дистилляция рассуждений для дообучения по цепочке рассуждений. Коллапс модели — реальный риск, но его можно смягчить, смешивая человеческие данные, используя разнообразные ансамбли учителей и агрессивную дедупликацию.

Последующие главы подробно развивают три основных алгоритма RL-выравнивания. Глава 14 рассматривает RLHF на основе PPO, пайплайн, ставший пионером InstructGPT и используемый в GPT-4 и Claude 2. Последующие главы охватывают DPO, устраняющий явную модель вознаграждения, и GRPO, снижающий дисперсию через вычитание групповой относительной базовой линии и центральный для моделей рассуждений DeepSeek. В каждом случае MDP уровня токенов, установленный здесь, предоставляет общий формальный язык.

Упражнения

Упражнение 11.1. Языковая модель присваивает следующие вероятности токенам предложения «The cat sat»: $p(\text{The}) = 0.05$, $p(\text{cat} \mid \text{The}) = 0.02$, $p(\text{sat} \mid \text{The cat}) = 0.08$.

1. Вычислите кросс-энтропийную потерю (11.4) для этого предложения.
2. Вычислите перплексию (11.5).
3. Если вместо этого используется равномерное распределение на словаре размером $|\mathcal{V}| = 50\,000$, какова перплексия? Сравните и интерпретируйте.

Упражнение 11.2. Рассмотрим MDP уровня токенов (Определение 11.3) с размером словаря $|\mathcal{V}| = 100\,000$ и максимальной длиной ответа $T = 512$ токенов.

1. Дайте верхнюю оценку $|\mathcal{S}|$, числа возможных состояний. Сколько бит памяти потребовалось бы для хранения таблицы $Q(s, a)$ для всех пар (s, a), если каждая запись занимает 4 байта?
2. Объясните, почему методы на основе ценности, такие как DQN, неосуществимы для этого MDP.
3. Теорема о градиенте стратегии (Глава 8) требует только $\nabla_\theta \log \pi_\theta(a \mid s)$, а не Q-таблицы. Почему это делает методы на основе градиента стратегии трактуемыми несмотря на огромное пространство состояний-действий?

Упражнение 11.3. Причинная маска в Трансформере только-декодера гарантирует, что позиция i не может обращать внимание на позиции $j > i$.

1. Запишите маскированный вес внимания α_{ij} формулой, используя индикаторную функцию $\mathbb{1}[j \leq i]$.

2. Объясните, почему эта маска эквивалентна марковскому свойству MDP уровня токенов: предсказание y_t зависит только от $(x, y_{1:t-1})$, но не от любого будущего токена.

3. Была бы необходима причинная маска для языковой модели, используемой только как *классификатор* (предсказывающий метку для всей последовательности)? Обоснуйте ответ.

Упражнение 11.4. Проблема exposure bias возникает потому, что SFT использует принуждение учителем во время обучения, но во время вывода опирается на собственные выходы модели.

1. Формализуйте несоответствие: пусть p_{teacher} — распределение обучения с принуждением учителя по контекстам на шаге t, а p_{model} — соответствующее распределение вывода. Покажите, что они совпадают тогда и только тогда, когда модель идеальна (присваивает вероятность 1 верному следующему токену на каждом шаге).

2. Запланированное сэмплирование (Bengio et al., 2015) заменяет истинный токен собственным предсказанием модели с вероятностью ϵ_t, где ϵ_t возрастает от 0 до 1 в ходе обучения. Опишите одно преимущество и один недостаток этого подхода по сравнению со стандартным принуждением учителем.

3. Объясните, почему RL-дообучение (которое обучается на собственных роллаутах модели) не страдает от смещения воздействия.

Упражнение 11.5. Закон масштабирования Chinchilla утверждает, что оптимальное по вычислениям обучение требует $D \approx 20N$ токенов для модели с N параметрами. Предположим вычислительный бюджет $C = 6 \times 10^{23}$ FLOPs и что один обучающий FLOP стоит приблизительно $C \approx 6ND$ FLOPs.

1. Выведите оптимальный по вычислениям размер модели N^* и количество токенов D^* как функции C.

2. С приведённым выше бюджетом вычислите N^* и D^* численно. Сравните с публично известной конфигурацией LLaMA 3 8B (8B параметров, 15T токенов). Является ли LLaMA 3 8B оптимальной по вычислениям согласно Chinchilla?

3. Анализ Chinchilla минимизирует потерю при обучении. Аргументируйте, почему практик, оптимизирующий *стоимость вывода*, может рационально выбрать меньшую, более переобученную модель.

Упражнение 11.6. Пайплайн Self-Instruct (Wang et al., 2023) засеивает генерацию инструкций 175 написанными людьми примерами и развивает датасет через промптинг модели-учителя.

1. Опишите один механизм, с помощью которого качество начального набора влияет на качество сгенерированного датасета.

2. Сходство ROUGE-L используется для фильтрации почти-дублированных инструкций. Приведите пример двух инструкций, которые ROUGE-L пометит как похожие, но которые подлинно различны в своих когнитивных требованиях, и наоборот (две инструкции, которые ROUGE-L оценивает как несхожие, но которые семантически почти идентичны).

3. Предположим, что модель-учитель, используемая в Self-Instruct, систематически склонна к определённым стилям ответа (например, всегда использует маркированные списки). Как это смещение распространится на студенческую модель, обученную на результирующих данных?

Упражнение 11.7. Коллапс модели означает прогрессивное сужение выходного распределения модели при итеративном обучении на её собственных синтетических выходах.

1. Пусть p_0 — исходное распределение человеческих данных, $\hat{p}_g$ — распределение, выученное в поколении g, аппроксимированное обучением на выборках из $\hat{p}_{g-1}$. Предположим, что каждый шаг обучения вносит ошибку аппроксимации, ограниченную ϵ по расстоянию суммарной вариации: $\mathrm{TV}(\hat{p}_g, p_{g-1}) \leq \epsilon$. Дайте верхнюю оценку $\mathrm{TV}(\hat{p}_G, p_0)$ после G поколений.

2. Shumailov et al. (2024) показали, что хвосты распределения (редкий, но важный контент) теряются первыми. Объясните интуитивно, почему это так, используя интерпретацию через дисперсию при коллапсе модели.

3. Предложите стратегию смешивания данных, которая теоретически предотвращает неограниченный рост оценки суммарной вариации при $G \to \infty$, предполагая, что доля λ обучающих данных в каждом поколении — подлинные данные людей.

Упражнение 11.8. LLM-as-judge использует способную модель для оценки ответов, заменяя аннотаторов-людей.

1. Опишите *смещение по многословности* в системах LLM-as-judge и предложите меры противодействия посредством инженерии промптов.

2. Позиционное смещение относится к тенденции судей-LLM предпочитать ответ, представленный первым (или вторым). Разработайте протокол оценки, устраняющий позиционное смещение, и объясните, почему ваш протокол несмещён.

3. Предположим, что модель вознаграждения, обученная на предпочтениях, сгенерированных судьёй-LLM, впоследствии используется в PPO для производства выровненной модели. Проследите, как систематическое смещение в модели-судье распространяется по пайплайну, влияя на поведение итоговой выровненной модели.

Упражнение 11.9. Constitutional AI (Bai et al., 2022) использует набор принципов для руководства самокритикой и пересмотром.

1. Запишите цикл критика-пересмотр как двухшаговую процедуру. Какова роль конституции? Чем это отличается от стандартного SFT?

2. RLAIF использует тот же принцип, но на этапе разметки предпочтений: модель просят оценить, какой из двух ответов лучше соответствует конституции. Покажите, что если модель-судья идеальна (всегда выбирает подлинно лучший ответ), то RLAIF и человеческий RLHF дают одну и ту же оптимальную стратегию по модели Bradley–Terry (Глава 10).

3. Каков основной режим отказа RLAIF при несовершенной модели-судье, и как он связан с переоптимизацией вознаграждения?

Упражнение 11.10. Рассмотрим простую задачу генерации языка: модель должна произвести бинарную строку длины $T = 3$ (например, «010»), где вознаграждение равно $+1$, если число единиц равно числу нулей, и -1 иначе.

1. Запишите полный MDP уровня токенов: перечислите все состояния, действия, переходы и вознаграждения.

2. Вычислите оптимальную стратегию π^* с помощью динамического программирования (задача достаточно мала для ручного вычисления).

3. Для $T = 3$ существует $2^3 = 8$ возможных ответов. Покажите, что цель RLHF (11.7) равна $\mathbb{P}_{\pi_\theta}(\text{сбалансированная строка}) - \mathbb{P}_{\pi_\theta}(\text{несбалансированная строка})$, и вычислите её значение при равномерной стратегии $\pi_\theta(0 \mid s) = \pi_\theta(1 \mid s) = 0.5$ для всех s.

Глава 12

Прямая оптимизация предпочтений и методы выравнивания

Простейшее решение задачи — то, которое переставляет то, что вы уже знаете, а не добавляет новые механизмы.

Пайплайн RLHF, описанный в главе 10, включает четыре отдельных нейронных сети: референсную стратегию, обучаемую стратегию, модель вознаграждения и (в PPO) критика с функцией ценности. Одновременное обучение всех четырёх требует колоссальных инженерных усилий. Модель вознаграждения необходимо обучить, оценить и заморозить до начала RL. Критик должен быть инициализирован и синхронизирован с актором. Видеопамять GPU должна вмещать как минимум две полные копии модели — а нередко и больше при использовании отсечённого суррогата PPO или вычислении штрафа KL относительно референса. На практике эта сложность является существенным барьером: RLHF с PPO в масштабе GPT-4 или Claude требует сложной распределённой инфраструктуры обучения, тщательной настройки гиперпараметров на нескольких этапах и постоянного мониторинга взлома вознаграждения.

Прямая оптимизация предпочтений (Rafailov и др., 2023) задаёт радикальный вопрос: нужна ли нам вообще модель вознаграждения? Ответ, на который в главе 10 намекало замечание 10.2, — нет. Оптимальная стратегия в рамках KL-регуляризованной цели RLHF связана с функцией вознаграждения через обратимую параметризацию. Подстановка этой параметризации в модель предпочтений Брэдли — Терри приводит к тому, что неразрешимая статистическая сумма сокращается, оставляя функцию потерь, зависящую только от стратегии и референсной стратегии — величин, которые можно вычислить напрямую. Модель вознаграждения устраняется полностью.

В данной главе развивается теория и практика DPO и более широкого семейства *прямых алгоритмов выравнивания*, которые он вдохновил. В разделе 12.1 объясняется мотивация отказа от модели вознаграждения. В разделе 12.2 приводится полный вывод цели DPO из первых принципов с каждым алгебраическим шагом. В разделе 12.3 описывается практический алгоритм обучения, требования к данным и детали реализации. В разделе 12.4 рассматриваются основные варианты — IPO, KTO, cDPO и ORPO, — каждый из которых устраняет отдельное ограничение базового алгоритма. В разделе 12.5 анализируются структурные слабости всех офлайн-методов прямого выравнивания, включая проблему сдвига распределения и отсутствие онлайн-исследования. В разделе 12.6 объясняется, почему online RL остаётся необходимым в production-системах, и описываются инженерные паттерны, делающие его практичным. В разделе 12.7 приводится исчерпывающее сравнение всех основных методов выравнивания — от PPO до GRPO — по наиболее важным на практике параметрам. В разделе 12.8 подводятся итоги главы.

12.1 От RLHF к прямому выравниванию

12.1.1 Стоимость модели вознаграждения

Стандартный пайплайн RLHF состоит из трёх этапов. Сначала обучается стратегия после supervised fine-tuning (SFT), π_{SFT}, на данных демонстраций. Затем обучается модель вознаграждения r_ψ на данных предпочтений $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}$ путём минимизации функции потерь Брэдли — Терри (10.6). Наконец, стратегия дообучается с помощью PPO против r_ψ с учётом штрафа KL относительно π_{SFT}. Каждый этап вносит свою инженерную сложность и источники отказов.

Этап модели вознаграждения особенно обременителен. Он требует отдельного набора гиперпараметров (скорость обучения, размер батча, отступ), отдельного протокола оценки (точность попарного сравнения

на отложенной выборке) и шага развёртывания, делающего обученную модель доступной в качестве сигнала вознаграждения во время RL. Модель вознаграждения, как правило, имеет тот же размер, что и стратегия, то есть должна помещаться в память GPU вместе со стратегией, референсной стратегией и функцией ценности PPO — итого до четырёх копий модели. При масштабе 70 миллиардов параметров это приближается к практическому пределу даже самых крупных GPU-кластеров.

Помимо памяти, модель вознаграждения является источником нестабильности. Если она недообучена, она не обеспечит полезный градиентный сигнал, и стратегия остановится. Если перетренирована, стратегия быстро обнаружит и начнёт эксплуатировать её слабые места (взлом вознаграждения). Поскольку модель вознаграждения заморожена во время PPO, у неё нет механизма адаптации по мере того, как распределение стратегии смещается от данных, на которых была обучена модель вознаграждения, — форма ковариантного сдвига, который усугубляется по мере продвижения обучения.

12.1.2 Простота, память и стабильность

Прямой алгоритм выравнивания заменяет двухэтапный пайплайн «модель вознаграждения, затем RL» единой целевой функцией в стиле обучения с учителем, вычисляемой непосредственно на данных предпочтений. Преимущества существенны.

Требования к памяти DPO требует только двух копий модели: обучаемой стратегии и замороженной референсной стратегии. Это вдвое снижает требования к памяти по сравнению с PPO, которому нужны стратегия, референсная стратегия, модель вознаграждения и критик с функцией ценности.

Стабильность обучения Функция потерь DPO представляет собой цель бинарной кросс-энтропии. Она гладкая, имеет ограниченные градиенты и не страдает от нестабильностей, связанных с отсечённой суррогатной целью PPO или движущимся сигналом вознаграждения онлайн-RL.

Отсутствие взлома вознаграждения Поскольку явной модели вознаграждения для переобучения нет, DPO не может демонстрировать классическую картину взлома вознаграждения. Стратегия не может эксплуатировать разрыв между прокси-моделью вознаграждения и истинными предпочтениями человека, так как сигнал предпочтений встроен непосредственно в функцию потерь.

Простота реализации DPO сводится примерно к двадцати строкам на PyTorch. Не нужен буфер опыта, сеть критика, оценка преимущества или отсечение PPO. Весь пайплайн напоминает дообучение с учителем, что делает его доступным для практиков без инфраструктуры RL.

Эти преимущества имеют свою цену, которую разделы 12.5 и 12.6 рассматривают подробно. Критическое ограничение состоит в том, что DPO является принципиально *offline*-методом: он обучается на фиксированном наборе данных предпочтений, собранных при другой (обычно SFT) стратегии. По мере того как обученная стратегия удаляется от референсной стратегии, данные предпочтений устаревают, и неявный сигнал вознаграждения перестаёт точно отражать то, что улучшенная стратегия предпочтёт на самом деле. Эта проблема сдвига распределения неизбежна в любом офлайн-алгоритме прямого выравнивания, и она мотивирует продолжающуюся актуальность online RL, описанного в разделе 12.6.

12.2 Вывод DPO

12.2.1 KL-ограниченная цель

Начнём с KL-регуляризованной цели RLHF, введённой в главе 10. При заданном промпте x и функции вознаграждения r цель имеет вид

$$J(\pi_\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r(x,y)\big] - \beta\, D_{\mathrm{KL}}(\pi_\theta(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x))\,, \tag{12.1}$$

где $\beta > 0$ управляет силой штрафа KL, а π_{ref} — референсная SFT-стратегия. Цель уравновешивает максимизацию вознаграждения и близость к референсу. В теореме 10.4 было показано, что единственная оптимальная стратегия равна

$$\pi^*(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)\, \exp\!\big(r(x,y)/\beta\big)}{Z(x)}, \tag{12.2}$$

где статистическая сумма $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x)\, \exp(r(x,y)/\beta)$.

12.2.2 Параметризация оптимальной стратегии

Взяв логарифм обеих частей (12.2) и перегруппировав, получаем *параметризацию вознаграждения*:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{12.3}$$

Это тождество выражает вознаграждение через оптимальную стратегию и статистическую сумму. Два свойства этого тождества принципиально важны.

Во-первых, тождество справедливо для *любой* функции вознаграждения r и соответствующей ей оптимальной стратегии π^*. Это не приближение; это точное алгебраическое следствие условий оптимальности.

Во-вторых, статистическая сумма $Z(x)$ зависит только от промпта x, но не от конкретного ответа y. Это означает, что при вычислении *разности* вознаграждений между двумя ответами y_w и y_l на один и тот же промпт x статистическая сумма сокращается:

$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}. \tag{12.4}$$

Неподдающийся вычислению член $\log Z(x)$ исчез.

12.2.3 Подстановка в модель Брэдли — Терри

Вспомним модель предпочтений Брэдли — Терри из главы 10: вероятность того, что человек предпочтёт y_w над y_l при промпте x, равна

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma\big(r(x, y_w) - r(x, y_l)\big). \tag{12.5}$$

Подставляя разность вознаграждений (12.4) в (12.5) и заменяя π^* параметрической стратегией π_θ (поскольку именно π^* мы хотим выучить), получаем

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right). \tag{12.6}$$

Это вероятность предпочтения DPO. Она зависит только от логарифма отношения стратегии к референсной стратегии, которое вычисляется двумя прямыми проходами через языковую модель (один для y_w и один для y_l).

12.2.4 Функция потерь DPO

При наличии набора данных $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^N$ предпочтений людей отрицательное логарифмическое правдоподобие наблюдаемых предпочтений согласно модели (12.6) является функцией потерь DPO:

Определение 12.1 (Функция потерь DPO)**. Функция потерь прямой оптимизации предпочтений (DPO)** имеет вид

$$\boxed{\mathcal{L}_{\text{DPO}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log \sigma\left(\beta \log \frac{\pi_\theta(y_w^{(i)} \mid x^{(i)})}{\pi_{\text{ref}}(y_w^{(i)} \mid x^{(i)})} - \beta \log \frac{\pi_\theta(y_l^{(i)} \mid x^{(i)})}{\pi_{\text{ref}}(y_l^{(i)} \mid x^{(i)})}\right).} \tag{12.7}$$

Функция потерь DPO представляет собой потерю бинарной кросс-энтропии, где «логит» для каждой пары — разность неявных логарифмических вознаграждений между предпочтительным и нежелательным ответами. Минимизация $\mathcal{L}_{\text{DPO}}$ эквивалентна максимизации правдоподобия наблюдаемых аннотаций предпочтений в рамках параметризованной модели Брэдли — Терри.

Утверждение 12.1 (Структура градиента функции потерь DPO)**.** *Для одной пары предпочтений (x, y_w, y_l) пусть*

$$h_\theta = \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}$$

будет разностью неявных вознаграждений. Тогда градиент $\mathcal{L}_{\text{DPO}}$ по θ равен

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta\, \sigma(-h_\theta) \big[\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_l|x)\big]. \tag{12.8}$$

Множитель $\sigma(-h_\theta) \in (0, 1)$ взвешивает обновление: он велик (близок к 1), когда стратегия присваивает схожее неявное вознаграждение обоим ответам (пара сложная), и мал (близок к 0), когда стратегия уже явно предпочитает y_w (пара лёгкая). Градиент увеличивает логарифмическую вероятность y_w и уменьшает логарифмическую вероятность y_l с весом, пропорциональным неуверенности текущей модели.

Доказательство. Пусть $\mathcal{L} = -\log \sigma(h_\theta)$. Поскольку $\frac{d}{dh} \log \sigma(h) = \sigma(-h) = 1 - \sigma(h)$, имеем

$$\nabla_\theta \mathcal{L} = -\sigma(-h_\theta)\, \nabla_\theta h_\theta.$$

Вычислим $\nabla_\theta h_\theta$: так как референсная модель π_{ref} не зависит от θ,

$$\nabla_\theta h_\theta = \beta\, \nabla_\theta \log \pi_\theta(y_w|x) - \beta\, \nabla_\theta \log \pi_\theta(y_l|x).$$

Объединяя, получаем (12.8). □

Замечание 12.1 (DPO как дообучение с учителем с контрастным взвешиванием). Уравнение (12.8) показывает, что DPO структурно идентично двум одновременным обновлениям дообучения с учителем: положительному (повышающему вероятность) обновлению для y_w и отрицательному (понижающему вероятность) обновлению для y_l, оба масштабируются весом доверия $\sigma(-h_\theta)$ для каждой пары. Это делает DPO совместимым с теми же трюками реализации, что и SFT: контрольные точки градиента, смешанная точность, адаптеры LoRA и оптимизация DeepSpeed ZeRO применяются без изменений.

Пример 12.1 (Вычисление функции потерь DPO)**.** Предположим, что для одной пары (x, y_w, y_l) логарифмические вероятности при текущей стратегии и референсе равны:

$$\log \pi_\theta(y_w|x) = -3{,}2, \quad \log \pi_{\text{ref}}(y_w|x) = -3{,}5, \quad \log \pi_\theta(y_l|x) = -2{,}8, \quad \log \pi_{\text{ref}}(y_l|x) = -2{,}6.$$

При $\beta = 0{,}1$ разность неявных вознаграждений равна

$$h_\theta = 0{,}1 \cdot (-3{,}2 - (-3{,}5)) - 0{,}1 \cdot (-2{,}8 - (-2{,}6)) = 0{,}1 \cdot (0{,}3) - 0{,}1 \cdot (-0{,}2) = 0{,}03 + 0{,}02 = 0{,}05.$$

Потеря для этой пары равна $-\log \sigma(0{,}05) = -\log(0{,}5125) \approx 0{,}669$. Вес доверия $\sigma(-0{,}05) \approx 0{,}4875$, что указывает на почти максимальную неопределённость для данной пары, и градиентное обновление будет большим. После обучения, когда h_θ достигнет 3, потеря станет $-\log \sigma(3) \approx 0{,}049$, а вес доверия упадёт до $\sigma(-3) \approx 0{,}047$, отражая, что стратегия научилась явно предпочитать y_w.

12.3 Алгоритм DPO

12.3.1 Процедура обучения

Процедура обучения DPO удивительно проста. Она требует двух экземпляров модели — обучаемой стратегии π_θ и замороженной референсной стратегии π_{ref} — и набора данных предпочтений. Референсная стратегия — это, как правило, контрольная точка SFT, замороженная с начала обучения DPO.

Алгоритм 26: Прямая оптимизация предпочтений (DPO)

Input: Контрольная точка SFT π_{ref}; набор данных предпочтений $\mathcal{D}$; коэффициент KL β; скорость обучения α; размер батча B
Output: Выровненная стратегия π_θ

```
Инициализировать π_θ ← π_ref;
Заморозить π_ref (без обновления градиентов);
for каждый мини-батч {(x^(i), y_w^(i), y_l^(i))}_{i=1}^B ⊂ D do
    for каждая пара i = 1, ..., B параллельно do
        Вычислить ℓ_w^(i) ← log π_θ(y_w^(i)|x^(i)) и ℓ_l^(i) ← log π_θ(y_l^(i)|x^(i));
        Вычислить ℓ̂_w^(i) ← log π_ref(y_w^(i)|x^(i)) и ℓ̂_l^(i) ← log π_ref(y_l^(i)|x^(i)) (без градиентов);
        Вычислить h^(i) ← β(ℓ_w^(i) − ℓ̂_w^(i)) − β(ℓ_l^(i) − ℓ̂_l^(i));
    Вычислить L ← −(1/B) Σ_{i=1}^B log σ(h^(i));
    θ ← θ − α ∇_θ L;
    Журнал: средний отступ вознаграждения (1/B) Σ_i h^(i); доля правильных (1/B) Σ_i 𝟙[h^(i) > 0];
return π_θ
```

12.3.2 Требования к данным

DPO требует набора данных предпочтений вида $\mathcal{D} = \{(x, y_w, y_l)\}$. Несколько практических соображений определяют качество и полезность этих данных.

Разрыв в качестве ответов. Функция потерь DPO наиболее информативна, когда между y_w и y_l существует явный разрыв в качестве. Если оба ответа почти идентичны, h_θ будет близок к нулю для любой разумной стратегии, и градиентное обновление будет направлено в произвольную сторону. Следует отбирать пары так, чтобы y_w и y_l демонстрировали существенно разный уровень качества.

Источник ответов. В оригинальной статье DPO оба ответа y_w и y_l сэмплируются из π_{ref} (модели SFT). Это наиболее последовательный выбор для автономного DPO: обучающие ответы взяты из того же распределения, что и референсная стратегия, поэтому неявный сигнал вознаграждения хорошо откалиброван. На практике многие наборы данных (например, Anthropic HH-RLHF или сравнения OpenAI WebGPT) сопоставляют ответы, сгенерированные моделью, с ответами человека или ответами более слабой модели — допустимый подход, хотя он вносит дополнительное несоответствие распределений.

Распределение промптов. Промпты x должны отражать целевое распределение развёртывания. Модель, обученная с DPO, будет хорошо работать на промптах, схожих с теми, что есть в $\mathcal{D}$, и может деградировать на промптах из других распределений — та же оговорка об обобщении, что применима ко всем методам обучения с учителем.

12.3.3 Детали реализации

Вычисление логарифмической вероятности. Логарифм вероятности последовательности $\log \pi_\theta(y|x) = \sum_{t=1}^{|y|} \log \pi_\theta(y_t|x, y_{<t})$ вычисляется путём конкатенации промпта и ответа, выполнения одного прямого прохода и суммирования по-токенных логарифмических вероятностей на позициях ответа.

Нормализация по длине. Ответы разной длины будут иметь разные логарифмические вероятности просто из-за длины. Чтобы стратегия не выучила тривиальный шорткат по длине — присваивая более высокую логарифмическую вероятность более коротким ответам вне зависимости от качества, — стандартной практикой является деление на длину ответа: $\bar{\ell} = \log \pi_\theta(y|x)/|y|$.

Кэширование логитов референса. Референсная модель π_{ref} заморожена на протяжении всего обучения. Логарифмические вероятности $\log \pi_{\text{ref}}(y_w|x)$ и $\log \pi_{\text{ref}}(y_l|x)$ можно поэтому предварительно вычислить один раз и закэшировать, исключив необходимость запуска прямого прохода референсной модели во время обучения. Это экономит примерно половину вычислений на каждом шаге обучения.

Гиперпараметры. Основные гиперпараметры: $\beta \in [0{,}01, 0{,}5]$ (коэффициент KL, управляющий тем, насколько далеко стратегия отходит от референса; меньшее β допускает большее отклонение), скорость обучения (как правило, от 1×10^{-6} до 5×10^{-6} для полного дообучения, на один-два порядка меньше, чем при SFT) и число эпох (обычно 1 — 3; больше эпох грозит переобучением к данным предпочтений).

12.3.4 Сравнение пайплайнов DPO и PPO: диаграмма

Следующая диаграмма сопоставляет полный пайплайн PPO-RLHF (сверху) с пайплайном DPO (снизу), иллюстрируя компоненты, которые DPO устраняет.

12.4 Варианты DPO

Цель DPO, при всей её элегантности, делает конкретные предположения о модели предпочтений и природе обучающих данных. Для смягчения этих предположений и устранения практических ограничений было предложено несколько вариантов.

12.4.1 IPO: оптимизация идентичных предпочтений

DPO выводится путём максимизации логарифмического правдоподобия в рамках модели Брэдли — Терри, которая является *мягкой* моделью предпочтений: даже детерминированные предпочтения (один ответ выбирается всегда) дают конечную потерю при любой конечной стратегии. Однако Azar и др. (2023) заметили, что если предпочтения совершенно последовательны — реалистичная идеализация при использовании автоматического оракула предпочтений, — оптимум DPO вырождается: стратегия стимулируется увеличивать отступ вознаграждения h_θ без ограничений, отправляя логарифмические вероятности к $+\infty$ и $-\infty$. Это *переобучение к детерминированным предпочтениям* проявляется как вырожденные выходные распределения.

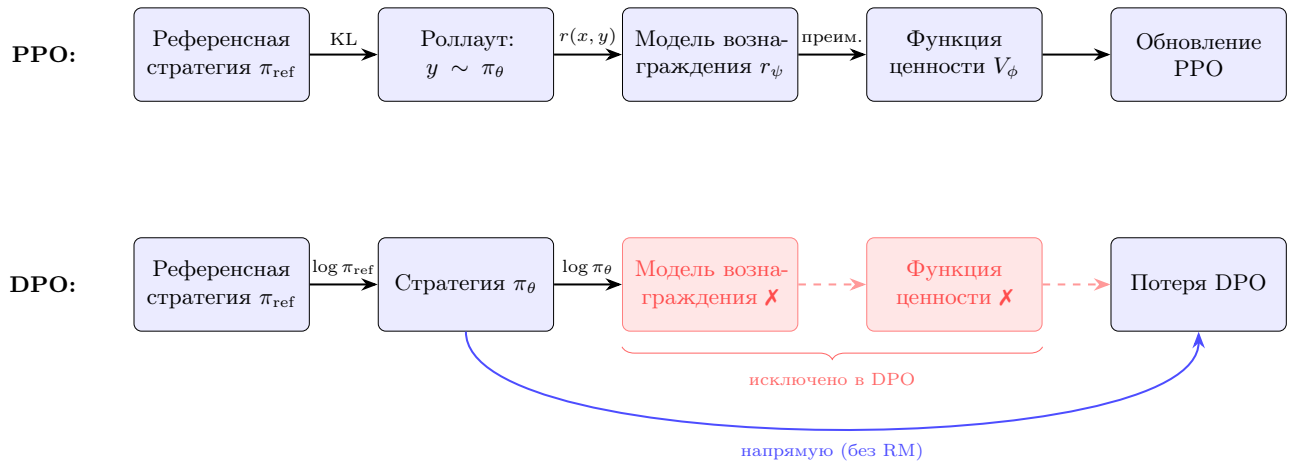


Рис. 12.1: Пайплайн обучения PPO (сверху) и DPO (снизу). Пайплайн PPO требует четырёх компонентов: референсной стратегии (для KL), онлайн-роллаутов, модели вознаграждения r_ψ и критика с функцией ценности V_ϕ. DPO устраняет модель вознаграждения и функцию ценности, вычисляя сигнал выравнивания напрямую из логарифмических отношений вероятностей.

Функция потерь Identity Preference Optimisation (IPO) устраняет эту проблему, заменяя логарифм сигмоиды штрафом на основе квадрата ошибки:

$$\mathcal{L}_{\text{IPO}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(\log \frac{\pi_\theta(y_w^{(i)}|x^{(i)})}{\pi_{\text{ref}}(y_w^{(i)}|x^{(i)})} - \log \frac{\pi_\theta(y_l^{(i)}|x^{(i)})}{\pi_{\text{ref}}(y_l^{(i)}|x^{(i)})} - \frac{1}{2\beta} \right)^2 . \tag{12.9}$$

Целевой отступ вознаграждения равен $1/(2\beta)$: стратегия должна выучить присваивать разность неявных вознаграждений ровно $1/(2\beta)$ между предпочтительным и нежелательным ответами, не больше. Квадратичная потеря создаёт восстанавливающую силу, штрафующую как недостаточное, так и избыточное разделение, предотвращая режим неограниченного отступа. IPO можно вывести непосредственно из исходной KL-ограниченной цели, не проходя через модель Брэдли — Терри, что делает его применимым даже когда модель предпочтений нелогистическая.

12.4.2 KTO: оптимизация Канемана — Тверски

Фундаментальным требованием DPO (и IPO) являются *парные* данные предпочтений: для каждого промпта x должен быть доступен как предпочтительный ответ y_w, так и нежелательный ответ y_l. Сбор таких парных данных требует одновременного предъявления аннотатору двух ответов — интерфейса сравнения, который дороже и менее естественен, чем простой вопрос о том, хорош ли отдельный ответ.

Ethayarajh и др. (2023) предложили оптимизацию Канемана — Тверски (KTO), работающую с непарной, бинарной обратной связью — каждый пример (x, y, z) состоит из промпта, ответа и бинарной метки $z \in \{0, 1\}$, указывающей, хорош ли ответ ($z = 1$) или плох ($z = 0$). KTO выводит свою цель из *теории перспектив* (Канеман и Тверски, 1979), в частности из наблюдения, что люди не склонны к риску: боль от потери превышает удовольствие от эквивалентного выигрыша.

Определение 12.2 (Функция потерь KTO)**.** Пусть $\mathcal{D}_+ = \{(x, y) : z = 1\}$ и $\mathcal{D}_- = \{(x, y) : z = 0\}$. Опорная точка KTO равна

$$z_{\text{ref}} = \mathbb{E}_{(x,y)\in\mathcal{D}}[\beta\, D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))] ,$$

на практике аппроксимируемая как $\beta \log \pi_\theta(y|x)/\pi_{\text{ref}}(y|x)$, усреднённая по батчу. **Функция потерь KTO** имеет вид

$$\mathcal{L}_{\text{KTO}}(\theta) = \mathbb{E}_{(x,y)\in\mathcal{D}_+}\big[1 - \sigma(\hat{r}_\theta(x, y) - z_{\text{ref}})\big] + \mathbb{E}_{(x,y)\in\mathcal{D}_-}\big[1 - \sigma(z_{\text{ref}} - \hat{r}_\theta(x, y))\big], \tag{12.10}$$

где $\hat{r}_\theta(x, y) = \beta \log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))$ — неявное вознаграждение.

Интуитивно, KTO поощряет неявное вознаграждение положительных примеров превышать опорную точку (тогда человек «получает» полезность) и неявное вознаграждение отрицательных примеров падать ниже неё (тогда человек «теряет» — но фрейм неприятия потерь означает, что это штрафуется асимметрично). KTO особенно привлекателен, когда наборы данных предпочтений собираются через интерфейсы оценки «нравится / не нравится», которые гораздо более распространены в масштабе, чем интерфейсы сравнения двух ответов.

12.4.3 cDPO: консервативный DPO

Аннотации предпочтений людей зашумлены: аннотаторы иногда предпочитают объективно худший ответ из-за поверхностных признаков, нехватки времени или непоследовательности. Стандартный DPO обрабатывает все аннотации с одинаковым весом, что означает: пара с перевёрнутой меткой — где y_w на самом деле хуже y_l, — будет активно двигать стратегию в неправильном направлении.

Консервативный DPO (cDPO), предложенный Mitchell и др. (2023), устраняет эту проблему, добавляя параметр сглаживания меток $\epsilon \in [0, 0{,}5)$ в цель DPO. Вместо трактовки каждого предпочтения как жёсткого присвоения cDPO трактует его как мягкий сигнал с вероятностью шума ϵ:

$$\mathcal{L}_{\text{cDPO}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\Big[(1-\epsilon)\log\sigma(h_\theta^{(i)}) + \epsilon\log\sigma(-h_\theta^{(i)})\Big], \tag{12.11}$$

где $h_\theta^{(i)}$ — разность неявных вознаграждений, определённая в определении 12.1. При $\epsilon = 0$ cDPO точно сводится к DPO. При $\epsilon > 0$ функция потерь добавляет небольшой штраф за чрезмерную уверенность в том, что y_w лучше, что регуляризует стратегию против переобучения к зашумлённым меткам. Цель cDPO эквивалентна потере модели вознаграждения со сглаживанием меток (10.10), применённой в параметризации DPO.

12.4.4 ORPO: оптимизация отношения шансов

Все упомянутые выше варианты отделяют цель выравнивания от цели языкового моделирования: DPO обучается на парах предпочтений, тогда как базовое SFT-обучение является отдельным предварительным этапом. Hong и др. (2024) предложили Odds Ratio Preference Optimisation (ORPO), который объединяет две цели в единую функцию потерь и полностью устраняет необходимость в референсной модели.

Определение 12.3 (Функция потерь ORPO)**. Функция потерь ORPO** объединяет обучающий член отрицательного логарифмического правдоподобия для предпочтительного ответа с штрафом на основе отношения шансов:

$$\mathcal{L}_{\text{ORPO}}(\theta) = -\log\pi_\theta(y_w|x) - \lambda\,\log\sigma\left(\log\frac{\text{odds}_\theta(y_w|x)}{\text{odds}_\theta(y_l|x)}\right), \tag{12.12}$$

где $\text{odds}_\theta(y|x) = \pi_\theta(y|x)/(1-\pi_\theta(y|x))$ — шансы ответа при данной стратегии, а $\lambda > 0$ — коэффициент смешивания.

Первый член обеспечивает продолжение генерации плавного, связного текста (дообучение с учителем на предпочтительном ответе). Второй член штрафует случаи, когда шансы стратегии генерировать y_l сравнимы с шансами генерировать y_w или превышают их. Поскольку ORPO использует *отношение шансов*, а не логарифмическое отношение с участием референсной модели, отдельная референсная модель не нужна: регуляризация встроена в структуру функции потерь.

ORPO особенно привлекателен для очень больших моделей, где поддерживать копию референсной модели в памяти нецелесообразно, а также для случаев, когда модель обучается с нуля на данных предпочтений без предварительного этапа SFT.

12.5 Ограничения прямого выравнивания

12.5.1 Проблема сдвига распределения

Фундаментальное ограничение DPO и всех автономных прямых алгоритмов выравнивания — *сдвиг распределения*. Набор данных предпочтений $\mathcal{D}$ собирается путём предъявления ответов от некоторой порождающей данные стратегии μ — как правило, SFT-модели π_{ref} — аннотаторам-людям. Неявный сигнал вознаграждения в DPO откалиброван под это распределение: логарифмическое отношение $\log(\pi_\theta/\pi_{\text{ref}})$ мало (а градиент потерь велик) именно для ответов из носителя μ.

По мере обучения π_θ отдаляется от π_{ref}. Ответы, изначально вероятные при μ, становятся всё менее вероятными при π_θ, а новые ответы, которые π_θ научилась предпочитать, могут вовсе отсутствовать в $\mathcal{D}$. Функция потерь DPO не может давать никакого обучающего сигнала для ответов вне своего обучающего распределения. Это означает, что по мере улучшения стратегии она исследует области пространства ответов, где имеющиеся у неё метки предпочтений всё менее информативны.

Эмпирически это проявляется в характерной кривой обучения: отступ вознаграждения DPO h_θ быстро растёт в первые несколько тысяч шагов, пока стратегия усваивает очевидные различия предпочтений в данных, а затем выходит на плато, когда стратегия исчерпывает информационное содержание фиксированного набора данных. Дальнейшее обучение даёт всё меньшую отдачу, а продолжение обучения после

этой точки может даже привести к *деградации* стратегии — увеличению неявного отступа вознаграждения для обучающих пар при ухудшении производительности на отложенных промптах, что является формой переобучения к предпочтениям.

12.5.2 Отсутствие онлайн-исследования

Связанным со сдвигом распределения является отсутствие онлайн-исследования. В онлайн-методах RL, таких как PPO, стратегия непрерывно генерирует новые ответы, собирает вознаграждения и обновляется. Это позволяет стратегии исследовать пространство ответов и получать обратную связь именно по тем ответам, которые она с наибольшей вероятностью генерирует в данный момент. Сигнал предпочтений всегда on-policy.

У DPO такого механизма нет. Он работает исключительно с фиксированным, заранее собранным набором данных. Если существуют высококачественные ответы, которые SFT-модель не смогла бы сгенерировать (и, следовательно, они отсутствуют в $\mathcal{D}$), DPO не может их обнаружить. Это особенно существенно для задач, требующих композиционного рассуждения: модель, научившаяся быть чуть более точной в задачах пошагового рассуждения, теперь может выиграть от обратной связи по конкретным паттернам рассуждения, которые она генерирует, — а они отличаются от тех, что есть в фиксированном обучающем наборе.

12.5.3 Поиск моды против покрытия мод

Модель Брэдли — Терри и функция потерь DPO выводятся из перспективы *поиска моды*: потеря поощряет стратегию присваивать высокую вероятность предпочтительным ответам и низкую — нежелательным. В пределе это толкает стратегию к *точечной массе* на ответе с наибольшим вознаграждением для каждого промпта.

Эта тенденция к поиску моды иногда желательна (для задач с единственным правильным ответом, например математических), но нередко нежелательна (для задач, требующих разнообразия, таких как творческое письмо или открытый диалог). Стратегия, ищущая моду, может генерировать ответы с высокими оценками по метрикам предпочтений, при этом будучи монотонной и лишённой разнообразия, которое реально хотят пользователи. В отличие от неё онлайн-методы RL с регуляризацией по энтропии или бонусами за разнообразие можно настроить для получения более широко охватывающих распределений.

12.5.4 Когда DPO не работает

На основе приведённого выше анализа DPO, как правило, уступает онлайн-RL в следующих сценариях.

- **Длительные горизонты обучения**: по мере потребления большего количества данных предпочтений и расхождения стратегии с референсом накапливается сдвиг распределения, и производительность выходит на плато или регрессирует.
- **Задачи с проверяемой правильностью**: для математических рассуждений или кодирования онлайн-верификатор (юнит-тесты, символический проверщик) обеспечивает более сильный и надёжный сигнал, чем статичные аннотации предпочтений людей.
- **Выравнивание с высокими ставками**: когда цель состоит в создании модели, безопасной и полезной *по всему распределению развёртывания*, онлайн-обратная связь от реальных пользователей даёт сигнал, который фиксированный набор данных не может предвосхитить.
- **Дальнейшее улучшение**: DPO сходится к оптимальной стратегии для данного фиксированного набора данных. Дальнейшее улучшение требует новых данных, тогда как онлайн-RL может продолжать улучшаться, пока среда обеспечивает обратную связь.

12.6 Online RL в Production-системах

12.6.1 Почему Online RL важен

Различие между автономным и онлайн-выравниванием не просто академическое. Производственная языковая модель обслуживает миллионы пользователей, генерируя миллиарды ответов по огромному разнообразию тем, стилей и намерений. Ни один статичный набор данных предпочтений, сколь бы он ни был велик, не может предвидеть это разнообразие. Пользователи будут задавать вопросы, о которых создатели

набора данных не задумывались, запрашивать стили, отличающиеся от обучающего распределения, и обнаруживать слабые места, проявляющиеся только в масштабе.

Онлайн-обучение с подкреплением замыкает этот круг. Вместо однократного обучения на фиксированном наборе данных и развёртывания онлайн-система RL непрерывно собирает обратную связь от реальных взаимодействий — явную (оценки «нравится / не нравится», звёздочки, выборы предпочтений) или неявную (продолжительность сессии, уточняющие вопросы, частота копирования) — и обновляет стратегию. Поведение модели улучшается по мере обслуживания пользователей, причём улучшение направлено на реальное распределение развёртывания, а не на прокси-набор данных.

Именно такой паттерн развёртывания присущ ведущим коммерческим ИИ-ассистентам. Технический отчёт GPT-4 (OpenAI, 2023) описывает непрерывные обновления RLHF на основе обратной связи пользователей. Пайплайн Constitutional AI от Anthropic (Bai и др., 2022) использует RLHF с сэмплированием на текущей стратегии, чтобы сигнал вознаграждения был откалиброван под распределение выходных данных текущей стратегии. Эти системы не рассматривают выравнивание как однократное событие обучения; они рассматривают его как непрерывный производственный процесс.

12.6.2 On-Policy и Off-Policy Соображения

В online-режиме ключевым проектным решением является использование *on-policy feedback* или *off-policy feedback*. On-policy feedback собирается от текущей стратегии: модель генерирует ответ, пользователь даёт обратную связь, и модель обновляется на этой обратной связи перед генерацией следующего ответа. Off-policy feedback собирается от более ранней версии стратегии (или другой стратегии) и используется для обновления текущей стратегии.

On-policy feedback — золотой стандарт для выравнивания. Гарантируется, что сигнал вознаграждения находится в распределении, которое стратегия в данный момент исследует, поэтому нет сдвига распределения. Обновление эквивалентно policy gradient с выборочной оценкой advantage. Однако on-policy feedback медленна: обратную связь необходимо собрать и обработать до следующего обновления, что ограничивает пропускную способность цикла обучения.

Off-policy feedback (например, DPO на сохранённых данных предпочтений) обеспечивает значительно более высокую пропускную способность, поскольку обучение отделено от сбора данных. Платой является сдвиг распределения, рассмотренный в разделе 12.5. На практике production-системы обычно используют гибридный подход: большой replay buffer off-policy обратной связи из исторических данных в сочетании с небольшим потоком on-policy feedback, с importance weighting для коррекции несоответствия распределений.

12.6.3 Смягчение катастрофического забывания

Постоянной проблемой онлайн-RL является *катастрофическое забывание*: по мере обновления стратегии для улучшения выравнивания она может деградировать по способностям, присутствовавшим в SFT-модели. Модель может стать менее беглой, менее фактически точной или менее способной к задачам, не представленным в текущем потоке обратной связи.

Несколько техник смягчают катастрофическое забывание в производстве.

Якорение KL — Штраф KL $\beta D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ в цели RL не позволяет стратегии слишком далеко отходить от контрольной точки SFT. Однако если сама контрольная точка SFT периодически обновляется (что обычно для производства), якорь смещается, потенциально допуская медленный, но неограниченный дрейф.

Ограничения на способности — После каждого обновления RL запускаются отдельные оценки на отложенных бенчмарках способностей (MMLU, HumanEval, GSM8K). Если какая-либо способность падает ниже порога, обновление откатывается или коэффициент KL увеличивается. Это требует специальной инфраструктуры оценки, но является наиболее принципиальной защитой.

Буферы воспроизведения — Смешивание старых примеров SFT с градиентом онлайн-RL гарантирует, что стратегия продолжает получать контролируемый сигнал по полному распределению способностей. Это аналогично воспроизведению опыта в DQN: примеры SFT играют роль «якорей» в пространстве опыта.

Дообучение на основе LoRA — Ограничение обновлений адаптером низкого ранга (LoRA) вместо полных параметров модели ограничивает степень забывания, поскольку веса базовой модели остаются фиксированными. Выравнивание хранится в адаптере; способности сохраняются в базовой модели.

12.6.4 Ограничения безопасности в онлайн-RL

Онлайн-RL вносит риск безопасности, которого нет у автономного выравнивания: модель обновляется на обратной связи реальных пользователей, которая может включать состязательные, предвзятые или иным образом вредоносные сигналы. Производственная система должна защищаться от нескольких режимов отказа.

Взлом вознаграждения пользователями. Если пользователи обнаружат, что определённые паттерны ответов получают высокие вознаграждения (например, ответы, соглашающиеся с любой позицией пользователя, или особенно подробные ответы), они могут использовать это для управления поведением модели. Состязательная устойчивость сигнала вознаграждения требует обнаружения аномалий и фильтрации выбросов обратной связи.

Нестабильность обратной связи. Положительные обратные связи могут вызвать быстрый дрейф стратегии к локальной моде. Штраф KL в сочетании с периодической оценкой на разнообразном отложенном наборе промптов является стандартным стабилизатором.

Оптимизация стратегии с ограничениями. Для критически важных с точки зрения безопасности свойств (отказ от вредоносных запросов, фактическая точность) можно дополнить цель RL ограничениями Лагранжа (Achiam и др., 2017; Bai и др., 2022): стратегия оптимизирует вознаграждение при условии, что релевантные безопасности метрики остаются выше нижней границы. Это преобразует неограниченную задачу RL в задачу ограниченной оптимизации, решаемую методами двойственных переменных.

12.6.5 PPO против DPO в онлайн-режиме

Сравнение PPO и DPO качественно различается в онлайн- и автономном режимах. В автономном режиме (фиксированный набор данных предпочтений) DPO конкурентоспособен с PPO или превосходит его по многим бенчмаркам: он проще, стабильнее и требует меньше вычислений. В онлайн-режиме PPO имеет структурное преимущество, которое DPO не может компенсировать.

PPO собирает обратную связь по текущей стратегии, обновляет стратегию на основе этой обратной связи и повторяет. Стратегия и источник обратной связи всегда синхронизированы. DPO, напротив, требует отдельного этапа сбора данных перед каждым этапом обучения; каждый цикл «сбор — обучение» является отдельным батчем, а не непрерывным циклом. Это означает, что онлайн-DPO фактически является *итеративным DPO*: сбор предпочтений от текущей стратегии, запуск DPO на несколько эпох, сбор новых предпочтений от обновлённой стратегии и повторение. Итеративный DPO существенно сокращает разрыв с PPO за счёт уменьшения сдвига распределения, но вносит свои инженерные издержки (повторный сбор данных и аннотирование) и всё равно уступает полностью онлайн-циклу PPO.

На практике выбор между PPO и DPO в production зависит от доступной инфраструктуры. Команда с выделенной RL-инфраструктурой, быстрой моделью вознаграждения (или верификатором) и возможностью online rollouts должна предпочесть PPO или вариант PPO (например, GRPO). Команда без RL-инфраструктуры или нуждающаяся в быстром выравнивании модели с имеющимся набором данных предпочтений должна предпочесть DPO или один из его вариантов.

12.6.6 Практические паттерны развёртывания

На основе приведённого анализа и наблюдаемой практики в ведущих ИИ-лабораториях сложились следующие паттерны развёртывания.

Начальный запуск с DPO — Использовать DPO на имеющемся наборе данных предпочтений для быстрого создания выровненной модели. Это самый быстрый путь от SFT к развёртываемой выровненной модели, подходящий для первоначальных выпусков.

Уточнение с онлайн-RL — После развёртывания собирать обратную связь по текущей стратегии и запускать итеративный DPO или PPO для непрерывного улучшения. Модель, запущенная с DPO, обеспечивает хорошую отправную точку с относительно небольшим доступным бюджетом KL для дальнейшего улучшения.

Разделение безопасности и способностей — Использовать автономный DPO для выравнивания безопасности (которое меняется медленно и может быть закодировано в статичном наборе данных) и онлайн-RL для выравнивания полезности (которое должно адаптироваться к меняющемуся распределению развёртывания).

RL с верификатором для рассуждений — Для задач с проверяемой правильностью (математика, код) использовать онлайн-RL с автоматическим верификатором в качестве сигнала вознаграждения, а не предпочтения людей. Это позволяет неограниченный сбор данных по текущей стратегии

без затрат на аннотирование людьми, а верификатор свободен от предвзятостей, влияющих на аннотаторов-людей.

12.7 Исчерпывающее сравнение методов выравнивания

12.7.1 Обзор методов

Таблица 12.1 содержит исчерпывающее сравнение основных методов выравнивания для больших языковых моделей. Методы охватывают весь спектр — от полностью онлайн-методов градиента стратегии до чисто автономного выравнивания с учителем.

Таблица 12.1: Исчерпывающее сравнение методов выравнивания LLM. «RM» = модель вознаграждения; «реф. модель» = требуется референсная стратегия; «парные» = требуются парные данные (y_w, y_l); «онлайн» = генерирует новые ответы во время обучения. Объём памяти относительно одной копии модели (1×). Стабильность: B = высокая, C = средняя, H = низкая.

Метод	RM?	Парные?	Онлайн?	Память	Стаб.	Лучше всего для
PPO	Да	Нет	Да	4×	C	Безопасность, RLHF в масштабе, онлайн-обратная связь
REINFORCE	Да	Нет	Да	2×	H	Простые задачи, базовое сравнение
RLOO	Да	Нет	Да	2×	C	Онлайн-RL с малыми ресурсами, без критика
GRPO	Нет*	Нет	Да	2×	B	Рассуждения, проверяемые задачи, без критика
DPO	Нет	Да	Нет	2×	B	Быстрое выравнивание, автономные данные
IPO	Нет	Да	Нет	2×	B	Детерминированные / автоматические предпочтения
KTO	Нет	Нет	Нет	2×	B	Бинарная обратная связь, без попарного сравнения
cDPO	Нет	Да	Нет	2×	B	Зашумлённые метки предпочтений
ORPO	Нет	Да	Нет	1×	B	Без реф. модели, ограниченная память

*GRPO использует верификатор или вознаграждение на основе правил, а не обученную модель вознаграждения.

12.7.2 Выбор метода выравнивания по цели

Выбор метода выравнивания должен определяться целью развёртывания, имеющимися данными и доступной вычислительной инфраструктурой. Следующие рекомендации сводят приведённый выше анализ к практическим указаниям.

Выравнивание безопасности. Для встраивания ограничений безопасности — отказа от вредоносных запросов, избегания токсичных выходных данных, соблюдения конституциональных принципов — основными проблемами являются покрытие (модель должна вести себя безопасно во всём распределении развёртывания) и стабильность (ограничения не должны деградировать со временем). PPO с моделью вознаграждения, специфичной для безопасности, и жёсткими ограничениями Лагранжа является наиболее принципиальным подходом для флагманских моделей. DPO или cDPO на тщательно подобранном наборе данных предпочтений безопасности является практической и нередко достаточной альтернативой для меньших моделей или команд без инфраструктуры RL. KTO особенно хорошо подходит для выравнивания безопасности, поскольку метки безопасности по природе бинарны (вредоносный / не вредоносный), а не сравнительны.

Способность к рассуждению. Для улучшения производительности на математических, логических задачах или задачах кодирования ключевое наблюдение состоит в том, что у этих задач есть проверяемые ответы. Это означает, что вместо аннотаций предпочтений людей можно использовать сигнал вознаграждения на основе правил (правильно / неправильно), обеспечивая неограниченный сбор данных по текущей стратегии. GRPO является наиболее сильным выбором здесь: он использует групповое сэмплирование для оценки преимуществ без критика, а вознаграждение на основе верификатора даёт чистый, однозначный сигнал. Результат DeepSeek-R1 (DeepSeek-AI, 2025) продемонстрировал, что GRPO с вознаграждением на

Таблица 12.2: Плюсы и минусы каждого метода выравнивания.

Метод	**Плюсы**	**Минусы**
PPO	Полностью онлайн, обрабатывает сложные сигналы вознаграждения, хорошо изучен теоретически	Высокое потребление памяти (4 модели), сложная реализация, риск взлома вознаграждения
REINFORCE	Простота, несмещённый градиент стратегии, лёгкость реализации	Высокая дисперсия, медленная сходимость, нестабильность без baseline
RLOO	Снижение дисперсии через базис leave-one-out, не требует критика	Требует группового сэмплирования, по-прежнему имеет сложность онлайн-режима
GRPO	Нет критика, стабильный, отлично работает на проверяемых задачах, сильные эмпирические результаты	Требует верификатора или модели вознаграждения; нет естественного расширения на открытые задачи
DPO	Простота, стабильность, обучение в стиле SFT, нет RM, низкое потребление памяти	Только автономный, сдвиг распределения, нет исследования, выходит на плато
IPO	Устраняет переобучение DPO, регулирует отступ, более устойчив к чистым предпочтениям	Чувствителен к гиперпараметрам (β управляет целевым отступом), нет сигнала по текущей стратегии
KTO	Работает с бинарными метками (без пар), естественен для обратной связи в производстве	Требует тщательной оценки опорной точки, менее эффективен по данным, чем DPO
cDPO	Устойчив к шуму меток, естественный регуляризатор, почти нулевые накладные расходы по сравнению с DPO	Требует настройки ϵ; слишком большое ϵ препятствует выравниванию
ORPO	Не нужна референсная модель, SFT + выравнивание за один проход, экономия памяти	Меньше теоретического обоснования; λ требует настройки; нет регуляризации KL

основе верификатора может давать драматические улучшения способности к рассуждению без каких-либо данных предпочтений людей.

Следование инструкциям. Следование инструкциям — способность точно и полезно выполнять разнообразные запросы пользователей — требует как широкого охвата (множество типов инструкций), так и тонких оценок качества (разница между хорошим и отличным ответом на данную инструкцию). DPO на большом разнообразном наборе данных предпочтений (например, UltraFeedback или Alpaca Farm) является рабочим методом для следования инструкциям. Итеративный DPO — несколько раундов DPO с сбором данных по текущей стратегии между раундами — дополнительно улучшает охват. PPO с обобщённой моделью вознаграждения является наиболее сильным вариантом при наличии онлайн-обратной связи.

Творческие задачи. Творческие задачи (генерация историй, поэзия, открытый диалог) требуют разнообразия: выровненная модель должна производить разнообразные, образные выходные данные, а не сходиться к единой моде. Тенденция DPO к поиску моды делает его неподходящим выбором, если только не добавить регуляризацию по разнообразию. Предпочтительнее PPO с бонусом за энтропию или функцией вознаграждения, поощряющей разнообразие. KTO с положительными метками для разнообразных творческих примеров также может стимулировать широкий охват без тенденции к коллапсу к моде, присущей методам на основе контрастных пар.

Среды с ограниченными ресурсами. При жёстких ограничениях по памяти или вычислениям единственными осуществимыми вариантами являются ORPO (требует только одну копию модели) или DPO (две копии). Оба могут применяться с адаптерами LoRA для дальнейшего сокращения числа обучаемых параметров, позволяя выравнивать большие модели на обычном оборудовании.

Пример 12.2 (Выбор метода для математического репетиторского ассистента)**.** Команда разрабатывает

математического репетиторского ассистента, который должен: (a) правильно решать задачи, (b) отказываться давать прямые ответы, когда студент должен учиться самостоятельно, и (c) ясно объяснять рассуждения. Как следует подходить к выравниванию?

Для цели (a) идеально подходит GRPO с символическим верификатором математики: ответы можно верифицировать автоматически, данные по текущей стратегии не ограничены, и аннотирование людьми не требуется.

Для цели (b) небольшой набор данных предпочтений безопасности из пар (задача, «дать ответ», «направлять без ответа») можно выровнять с DPO или KTO. Ограничение похоже на ограничение безопасности (бинарное: нарушает политику «без прямого ответа» или нет), что делает KTO особенно естественным.

Для цели (c) подходит DPO на наборе данных человеческих пар предпочтений (хорошо объяснённые vs плохо объяснённые решения), дополненный раундами итеративного DPO с использованием роллаутов по текущей стратегии от модели, улучшенной GRPO, для избегания сдвига распределения.

На практике эти три цели объединяются: GRPO обрабатывает цель проверяемой правильности, KTO обрабатывает педагогическое ограничение в стиле безопасности, а DPO обрабатывает стилистическое измерение качества. Каждый метод применяется на отдельном этапе, с оценкой способностей между этапами для мониторинга забывания.

12.8 Итоги

В этой главе была развита теория и практика прямой оптимизации предпочтений и её связь с широким ландшафтом методов выравнивания LLM.

Центральное понимание DPO (раздел 12.2) состоит в двухшаговой алгебраической манипуляции: оптимальная стратегия KL-регуляризованного RLHF может быть преобразована для выражения вознаграждения как логарифмического отношения оптимальной стратегии к референсной плюс зависящая от промпта статистическая сумма. Когда это выражение подставляется в вероятность предпочтения Брэдли — Терри, статистическая сумма сокращается, оставляя функцию потерь, зависящую только от логарифмических отношений стратегий — величин, вычислимых двумя прямыми проходами. Модель вознаграждения устраняется без приближений.

Алгоритм DPO (раздел 12.3) сводится к бинарной кросс-энтропии на логарифмических отношениях стратегий с весом доверия $\sigma(-h_\theta)$, фокусирующим обучение на неопределённых парах. Диаграмма TikZ (рисунок 12.1) проиллюстрировала драматическое упрощение по сравнению с PPO: модель вознаграждения и функция ценности устранены, уменьшая требуемые копии моделей с четырёх до двух.

Варианты DPO (раздел 12.4) каждый устраняют конкретное ограничение. IPO предотвращает режим неограниченного отступа, налагая квадратичную цель на разрыв вознаграждения. KTO расширяет выравнивание на непарную бинарную обратную связь с использованием теории перспектив. cDPO вводит сглаживание меток для обработки зашумлённых аннотаций. ORPO полностью устраняет референсную модель, используя штраф на основе отношения шансов, встроенный в контролируемую функцию потерь.

Проблема сдвига распределения (раздел 12.5) является фундаментальным ограничением, общим для всех офлайн-методов прямого выравнивания. По мере удаления стратегии от референсной стратегии метки предпочтений в обучающем наборе становятся всё менее информативными, вызывая выход производительности на плато. Отсутствие онлайн-исследования означает, что DPO не может открыть высококачественные ответы, которые текущая стратегия теперь способна генерировать, но которых не было в исходном наборе данных.

Online RL в production (раздел 12.6) устраняет эти ограничения, замыкая контур между поведением модели и обратной связью. Production-система RL непрерывно генерирует ответы, собирает обратную связь (явную или неявную) и обновляет стратегию. Основные инженерные задачи — катастрофическое забывание (смягчаемое якорением KL, ограничениями на способности, replay buffer и LoRA), безопасность при состязательной обратной связи (смягчаемая обнаружением аномалий и constrained optimisation) и компромисс PPO vs. DPO (разрешаемый доступностью инфраструктуры и допустимой степенью сдвига распределения).

Исчерпывающая таблица сравнения (раздел 12.7) является практическим справочником для выбора метода выравнивания по цели: PPO и GRPO для онлайн-режима и проверяемых задач, DPO и его варианты для автономного режима с доступными данными предпочтений, KTO для бинарной обратной связи и ORPO для сред с ограниченной памятью.

Глава 10 закладывает основу: модель Брэдли — Терри, KL-регуляризованную цель и параметризацию оптимальной стратегии, которую использует DPO.

Глава 14 описывает пайплайн дообучения PPO во всех инженерных деталях, включая вознаграждение KL на уровне токенов, оценку преимущества и практические трюки для стабильного обучения.

Глава 16 обсуждает выравнивание в агентской среде, где модель должна быть выровнена не только по однооборотным ответам, но и по многошаговым траекториям.

Упражнения

Упражнение 12.1. Проведите полный вывод DPO с нуля. Начиная с KL-регуляризованной цели (12.1), докажите, что оптимальная стратегия имеет вид (12.2). Затем преобразуйте для получения параметризации вознаграждения (12.3). Покажите явно, что статистическая сумма $Z(x)$ сокращается при вычислении разности вознаграждений $r(x, y_w) - r(x, y_l)$. Наконец, подставьте в формулу Брэдли — Терри для получения функции потерь DPO (12.7). На каждом шаге укажите, какие предположения являются точными и что потребовалось бы изменить, если бы модель предпочтений не была Брэдли — Терри.

Упражнение 12.2. Проверьте утверждение 12.1 явным вычислением. Для одной пары предпочтений с логарифмическими вероятностями $\log \pi_\theta(y_w|x) = -2{,}0$, $\log \pi_{\text{ref}}(y_w|x) = -2{,}4$, $\log \pi_\theta(y_l|x) = -1{,}5$, $\log \pi_{\text{ref}}(y_l|x) = -1{,}3$ и $\beta = 0{,}1$:

1. Вычислите разность неявных вознаграждений h_θ.
2. Вычислите потерю DPO для этой пары.
3. Вычислите вес доверия $\sigma(-h_\theta)$ и интерпретируйте его: является ли данная пара лёгкой или сложной для текущей стратегии?
4. Предположим, что $\log \pi_\theta(y_w|x)$ уменьшается на 0,3 (стратегия становится менее вероятной для генерации y_w). Пересчитайте h_θ и потерю и объясните качественно, как изменяется градиент.

Упражнение 12.3. Функция потерь IPO (12.9) заменяет логарифм сигмоиды квадратичной потерей.

1. Покажите, что функция потерь DPO не имеет конечного минимума, когда предпочтения совершенно последовательны (т.е. когда $y_w \succ y_l$ с вероятностью 1). *Подсказка:* рассмотрите, что происходит с $\mathcal{L}_{\text{DPO}}$ при $h_\theta \to +\infty$.
2. Покажите, что функция потерь IPO (12.9) минимизируется при $h_\theta^* = 1/(2\beta)$. Вычислите h_θ^* для $\beta = 0{,}1$ и интерпретируйте это значение в терминах разрыва неявного вознаграждения.
3. Для численного примера из упражнения 12.2 вычислите потерю IPO и сравните её с потерей DPO. Какая больше и почему?

Упражнение 12.4. Цель KTO мотивирована теорией перспектив. Формально функция ценности Канемана — Тверски $v : \mathbb{R} \to \mathbb{R}$ вогнута для выигрышей ($v > 0$) и выпукла и круче для потерь ($v < 0$), с $v(0) = 0$.

1. Нарисуйте функцию ценности и объясните, почему неприятие потерь подразумевает $|v'(-\epsilon)| > |v'(\epsilon)|$ для малых $\epsilon > 0$.
2. Опорная точка KTO z_{ref} играет роль нуля функции ценности. Объясните интуитивно, почему неявное вознаграждение положительного примера должно превышать z_{ref}, а неявное вознаграждение отрицательного примера должно быть ниже него.
3. Покажите, что KTO с $z_{\text{ref}} = 0$ и равными долями положительных и отрицательных примеров сводится к форме, близко связанной с DPO, если положительные и отрицательные примеры взяты из одного парного набора данных.

Упражнение 12.5. Покажите, что функция потерь cDPO (12.11) может быть выведена из DPO, предположив, что каждая метка предпочтения переворачивается с вероятностью ϵ. То есть аннотатор маркирует $y_w \succ y_l$ с вероятностью $1 - \epsilon$ и $y_l \succ y_w$ с вероятностью ϵ. Взятие математического ожидания потери DPO по данной стохастической модели маркировки даёт cDPO. Какому значению ϵ соответствует полностью неинформативный аннотатор и к чему cDPO сводится в этом пределе?

Упражнение 12.6. Рассмотрите стилизованную модель сдвига распределения в DPO. Предположим, набор данных предпочтений собран из референсной стратегии π_{ref}, и после обучения DPO стратегия π_θ имеет дивергенцию KL $d = D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ от референса.

1. Докажите, что доля ответов в π_θ, которые также появляются в носителе π_{ref} (и, следовательно, покрыты набором данных предпочтений), уменьшается с ростом d.

2. В худшем случае, если π_θ размещает всю свою массу на ответах, не входящих в носитель π_{ref}, что может сообщить стратегии о качестве этих ответов градиент DPO?

3. Предложите практическое смягчение: как итеративный DPO снижает тяжесть сдвига распределения? Каков остаточный сдвиг распределения после k раундов итеративного DPO?

Упражнение 12.7. Команда развёртывает чат-бот для обслуживания клиентов. Чат-бот должен: (a) правильно решать проблемы клиентов, (b) отвечать вежливо даже на оскорбительные сообщения и (c) переводить разговор к человеку-оператору при необходимости. Разработайте пайплайн обучения с выравниванием для этого чат-бота, ответив на следующие вопросы:

1. Какой метод(ы) выравнивания вы бы использовали для каждой из трёх целей (a), (b), (c)?

2. Как бы вы собирали обратную связь в производстве: явную, неявную или обе?

3. Как бы вы предотвратили деградацию беглости языка или фактической точности модели в цикле онлайн-RL?

4. Какой механизм безопасности предотвратил бы систематическое управление стратегией со стороны состязательных пользователей?

Упражнение 12.8. Изучите сравнение методов выравнивания в таблице 12.1.

1. Объясните, почему PPO требует 4× памяти одной модели, тогда как DPO требует только 2×.

2. У команды есть набор данных из 50 000 взаимодействий с поддержкой клиентов, каждое маркировано как «решено» или «не решено», без попарных сравнений ответов. Какой метод выравнивания вы бы рекомендовали и почему?

3. GRPO указан с оценкой «B» (высокая) стабильности, несмотря на то что это онлайн-метод. Объясните, почему GRPO достигает более высокой стабильности, чем стандартный REINFORCE, обратившись к механизму групповой нормализации.

4. При каких условиях DPO мог бы *превзойти* PPO даже в онлайн-режиме? Будьте конкретны в отношении предположений о данных и инфраструктуре.

Упражнение 12.9. Рассмотрите использование DPO для улучшения математических рассуждений языковой модели. Набор данных предпочтений состоит из пар (y_w, y_l), где y_w — правильное решение с цепочкой мыслей, а y_l — неправильное, оба сгенерированы SFT-моделью.

1. После обучения DPO точность модели на математических задачах из обучающего распределения улучшается с 45% до 58%. Предложите эксперимент, позволяющий определить, связано ли это улучшение с (i) генерацией лучших шагов рассуждения или (ii) улучшением угадывания формата итогового ответа.

2. Объясните, почему проблема сдвига распределения (раздел 12.5) особенно тяжела для математических рассуждений по сравнению, скажем, со следованием инструкциям.

3. Опишите, как вы бы перешли от выравнивания математики на основе DPO к выравниванию на основе GRPO. Как бы выглядела последовательность перехода и что бы вы использовали в качестве сигнала вознаграждения GRPO?

Глава 13

GRPO и онлайн-RL с проверяемыми вознаграждениями

Лучший способ проверить правильность ответа — проверить его, а не спрашивать кого-то, выглядит ли он правильным.

В главе 9 была разработана проксимальная оптимизация стратегии (PPO) — основной алгоритм обучения с подкреплением на основе обратной связи от человека (RLHF). PPO обеспечивает стабильные обновления стратегии с помощью отсечённой суррогатной цели и вычисляет преимущества через обученного критика (функцию ценности). В главе 12 затем было показано, что прямая оптимизация предпочтений (DPO) может полностью обойтись без модели вознаграждения, параметризуя KL-регуляризованную цель в виде контролируемой функции потерь на парах предпочтений. DPO прост и стабилен, но принципиально offline: он обучается на фиксированном наборе данных и не может исследовать пространство.

В данной главе вводится **групповая относительная оптимизация стратегии** (GRPO) — алгоритм, лежащий в основе DeepSeek-R1 и других недавних моделей рассуждений (Shao и др., 2024; DeepSeek-AI, 2025). GRPO занимает удачную нишу между PPO и DPO: он является *онлайн* — стратегия генерирует свежие завершения на каждой итерации, — но устраняет критика с функцией ценности, делающего PPO дорогостоящим. Вместо этого GRPO оценивает преимущества, сэмплируя *группу* завершений для каждого промпта и нормируя вознаграждения внутри группы. Результатом является простой, экономичный по памяти алгоритм, сохраняющий преимущества стабильности механизма отсечения PPO.

Мы также формализуем **обучение с подкреплением с проверяемыми вознаграждениями** (RLVR) — парадигму обучения, при которой сигнал вознаграждения поступает от детерминированного верификатора (например, набора юнит-тестов или символического математического проверщика), а не от обученной модели вознаграждения. RLVR является естественным дополнением GRPO: когда вознаграждения бинарны и не зашумлены, group-relative advantage ведёт себя особенно хорошо, а отсутствие обученной модели вознаграждения устраняет риск reward hacking.

В разделе 13.1 мотивируется GRPO путём выявления конкретных ограничений PPO и DPO, которые он устраняет. В разделе 13.2 выводится цель GRPO в полном объёме. В разделе 13.3 представлен полный алгоритм. В разделе 13.4 даётся теоретический анализ, включая связь с REINFORCE Leave-One-Out (RLOO, раздел 8.6 главы 8) и анализ смещения-дисперсии размера группы. В разделе 13.5 формализуется RLVR. В разделе 13.6 обсуждаются rejection sampling и best-of-N как более простой baseline. В разделе 13.7 описываются инженерные аспекты online RL training at scale. В разделе 13.8 сравниваются PPO, GRPO и DPO в единой таблице. В разделе 13.9 подводятся итоги главы.

13.1 Мотивация: за пределами PPO и DPO

13.1.1 Стоимость критика в PPO

Напомним из раздела 9.7, что отсечённая суррогатная цель PPO (9.16) требует оценок преимущества $\hat{A}_t$, вычисляемых через обобщённую оценку преимущества (GAE, определение 6.7). GAE, в свою очередь, требует обученной функции ценности $V_\phi(S_t)$ — *критика*, — параметры ϕ которого обучаются вместе с актором. В рамках RLHF, описанного в разделе 9.8, архитектура из четырёх моделей (актор, критик, референсная модель, модель вознаграждения) требует огромного объёма памяти GPU. При масштабе 70B параметров каждая копия модели занимает примерно 140 ГБ в формате половинной точности, так что четыре копии требуют 560 ГБ — больше, чем на одном узле 8×A100.

Критик также является источником нестабильности. Ему приходится отслеживать движущуюся цель: по мере улучшения актора V^{π_θ} меняется, и критик должен переобучаться. В начале обучения критик особенно неточен, производя зашумлённые оценки преимуществ, которые могут дестабилизировать актора. Тщательная инициализация (раздел 9.8) смягчает, но не устраняет эту проблему.

13.1.2 Проблема сдвига распределения в DPO

В главе 12 было показано, что DPO устраняет как модель вознаграждения, так и критика, сокращая требования к памяти до двух копий моделей (π_θ и π_{ref}). Платой является то, что DPO является offline-методом: он обучается на фиксированном наборе данных предпочтений $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}$, собранном при стратегии SFT. По мере обучения и удаления π_θ от π_{SFT} метки предпочтений устаревают. Ответы, которые улучшенная стратегия теперь генерировала бы, отсутствуют в $\mathcal{D}$, и модель не может их открыть. В разделе 12.5 главы 12 это ограничение было проанализировано подробно: DPO сходится к оптимальной стратегии *для данного фиксированного набора данных*, но дальнейшее улучшение требует новых данных.

13.1.3 GRPO: лучшее из двух миров

GRPO одновременно решает обе проблемы:

Без критика Преимущества оцениваются по группе сэмплированных завершений, а не по обученной функции ценности. Объём занимаемой памяти сокращается с четырёх до двух копий модели (актор π_θ и референс π_{ref}), как и в DPO.

Онлайн-режим Стратегия генерирует новые завершения на каждой итерации и получает свежую обратную связь по вознаграждению. Сдвига распределения нет: сигнал вознаграждения всегда откалиброван под выходное распределение текущей стратегии.

Стабильные обновления GRPO унаследует отсечённое отношение важности PPO, предотвращая чрезмерно крупные шаги обновления стратегии.

Ключевое понимание состоит в том, что основная роль критика — обеспечение baseline для снижения дисперсии — может быть выполнена эмпирическими статистиками группы завершений, сэмплированных из текущей стратегии. Эта идея тесно связана с оценщиком RLOO (раздел 8.6 главы 8), который использует leave-one-out baseline; GRPO расширяет его стандартизацией и отсечением в стиле PPO.

На рисунке 13.1 сравниваются архитектуры модели PPO-RLHF (четыре модели) и GRPO-RLVR (две модели плюс верификатор).

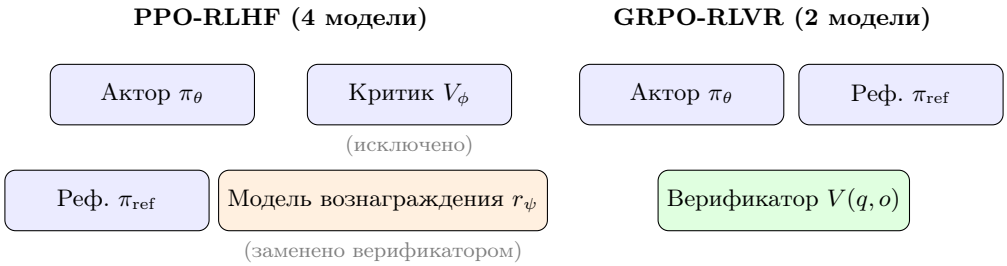


Рис. 13.1: Сравнение архитектур моделей. PPO-RLHF требует четырёх копий нейронных сетей: актора, критика, референса и модели вознаграждения. GRPO-RLVR требует только двух копий (актора и референса) плюс лёгкий детерминированный верификатор, уменьшая вдвое объём занимаемой памяти.

13.2 Цель GRPO

GRPO был предложен Shao и др. (2024) в статье DeepSeekMath, где его использовали для улучшения математических рассуждений в языковой модели с 7B параметрами. Алгоритм был затем принят в качестве компонента RL в пайплайне DeepSeek-R1 (DeepSeek-AI, 2025), достигнув результатов уровня лучших мировых решений по математике уровня соревнований, генерации кода и решению задач по естественным наукам. Название “group relative” отражает две определяющие особенности: преимущества вычисляются относительно *группы* завершений для одного и того же промпта и выражаются как *относительные* отклонения (стандартизованные групповым стандартным отклонением).

13.2.1 Постановка задачи и обозначения

Пусть π_θ обозначает обучаемую стратегию языковой модели, $\pi_{\theta_{\text{old}}}$ — стратегию в начале текущей итерации (до шага градиента), а π_{ref} — замороженную референсную стратегию (обычно SFT checkpoint). Промпт q берётся из набора данных $\mathcal{D}$. Для каждого промпта GRPO сэмплирует **группу** из G независимых завершений из старой стратегии:

$$o_1, o_2, \ldots, o_G \;\sim\; \pi_{\theta_{\text{old}}}(\cdot \mid q), \tag{13.1}$$

где каждое завершение $o_i = (o_{i,1}, o_{i,2}, \ldots, o_{i,|o_i|})$ является последовательностью токенов переменной длины. Функция вознаграждения $r(q, o_i) \in \mathbb{R}$ оценивает каждое завершение, давая вознаграждения $r_1, r_2, \ldots, r_G$.

13.2.2 Group-Relative Advantage

В PPO преимущество $\hat{A}_t$ вычисляется через обученного критика с помощью GAE. GRPO заменяет это **group-relative advantage**, не требующим обученных параметров:

Определение 13.1 (Преимущество GRPO)**.** Для завершения i в группе из G завершений с вознаграждениями $r_1, \ldots, r_G$ преимущество GRPO равно

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \varepsilon_0}, \tag{13.2}$$

где $\mu_G = \frac{1}{G}\sum_{j=1}^{G} r_j$ — среднее по группе, $\sigma_G = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \mu_G)^2}$ — стандартное отклонение по группе, а $\varepsilon_0 > 0$ — малая константа для численной стабильности (обычно 10^{-8}).

Преимущество $\hat{A}_i$ измеряет, на сколько стандартных отклонений выше или ниже среднего по группе находится вознаграждение завершения i. Если все завершения в группе получают одинаковое вознаграждение (например, все правильные или все неправильные), то $\sigma_G = 0$ и $\hat{A}_i \approx 0$ для всех i, не производя никакого градиентного сигнала. Это желательно: промпт, который модель всегда решает или никогда не решает, не даёт никакой контрастной информации.

Замечание 13.1. Одно и то же преимущество $\hat{A}_i$ присваивается *каждому токену* в завершении i:

$$\hat{A}_{i,t} = \hat{A}_i \quad \text{для всех } t \in \{1, \ldots, |o_i|\}.$$

Это преимущество на уровне последовательности (или уровне результата), а не на уровне токена. В отличие от GAE, который присваивает разные преимущества разным токенам на основе оценок функции ценности на каждом шаге, GRPO равномерно приписывает итоговое вознаграждение всем токенам, которые его произвели. Это упрощение является одновременно достоинством (критик не нужен) и ограничением (нет присваивания заслуг внутри завершения).

13.2.3 Отсечённая суррогатная цель

Следуя PPO (определение 9.5), GRPO использует взвешенную по важности цель с отсечением. Для завершения i и позиции токена t определим отношение важности на уровне токена:

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}. \tag{13.3}$$

Отсечённый суррогат для данного токена равен

$$L_{i,t}^{\text{CLIP}}(\theta) = \min\Big(\rho_{i,t}(\theta)\,\hat{A}_i,\ \text{clip}\big(\rho_{i,t}(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\,\hat{A}_i\Big), \tag{13.4}$$

где $\epsilon > 0$ — диапазон отсечения (обычно $\epsilon \in [0{,}1, 0{,}3]$). Механизм отсечения идентичен PPO: он ограничивает стимул увеличивать вероятность положительно-адвантажных завершений и ограничивает стимул уменьшать вероятность отрицательно-адвантажных завершений, предотвращая дестабилизирующие крупные шаги стратегии.

13.2.4 Штраф KL

GRPO добавляет штраф дивергенции Кульбака — Лейблера непосредственно в функцию потерь, а не встраивает его в вознаграждение, как это делает PPO (уравнение (9.19)). Такая конструкция отделяет KL-регуляризацию от вычисления преимущества, избегая осложнений при групповой нормализации. Оценщик KL на уровне токена, используемый на практике, равен

$$D_{\text{KL}}^{(i,t)} = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - \log\frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - 1, \tag{13.5}$$

что является несмещённым, неотрицательным оценщиком Шульмана (2020) дивергенции KL между π_θ и π_{ref}. Этот оценщик всегда удовлетворяет $D_{\text{KL}}^{(i,t)} \geq 0$, равенство достигается тогда и только тогда, когда $\pi_\theta(o_{i,t} \mid q, o_{i,<t}) = \pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})$.

Замечание 13.2. Альтернативным оценщиком является более простой $\log[\pi_\theta(o_{i,t} \mid q, o_{i,<t})/\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})]$, который несмещён, но может быть отрицательным для отдельных сэмплов. Оценщик в (13.5) предпочтительнее, поскольку его неотрицательность стабилизирует обучение.

13.2.5 Полная цель GRPO

Объединяя отсечённый суррогат и штраф KL, цель GRPO принимает вид:

Определение 13.2 (Цель GRPO)**.**

$$\boxed{J_{\text{GRPO}}(\theta) = \mathbb{E}_{\substack{q\sim\mathcal{D} \\ \{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}} \left[\frac{1}{G}\sum_{i=1}^{G} \frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \left(L_{i,t}^{\text{CLIP}}(\theta) - \beta\, D_{\text{KL}}^{(i,t)} \right) \right],} \tag{13.6}$$

где $L_{i,t}^{\text{CLIP}}$ определяется уравнением (13.4), $D_{\text{KL}}^{(i,t)}$ — уравнением (13.5), а $\hat{A}_i$ — уравнением (13.2).

Ряд особенностей этой цели заслуживает комментария.

- Внутренняя сумма усредняет по позициям токенов внутри каждого завершения. Нормировка на $1/|o_i|$ предотвращает непропорционально большой вклад длинных завершений в градиент.
- Внешняя сумма усредняет по G завершениям в группе. Это аналогично усреднению по K траекториям в стандартном мини-батче градиента стратегии, но здесь траектории разделяют один и тот же промпт.
- Коэффициент KL $\beta > 0$ управляет дрейфом от референсной стратегии. Большее β сильнее ограничивает стратегию, снижая риск взлома вознаграждения ценой более медленного обучения.
- Когда на одном и том же батче завершений делается несколько шагов градиента, $\pi_{\theta_{\text{old}}}$ фиксируется при стратегии, сгенерировавшей завершения, и обновляется только π_θ. Это та же коррекция вне текущей стратегии, что и в PPO.

Утверждение 13.1 (Локальная эквивалентность градиентов)**.** *При $\theta = \theta_{\text{old}}$ градиент $J_{\text{GRPO}}(\theta)$ по θ равен градиенту стратегии REINFORCE с group-mean baseline:*

$$\nabla_\theta J_{\text{GRPO}}(\theta)\big|_{\theta=\theta_{\text{old}}} = \frac{1}{G}\sum_{i=1}^{G} \hat{A}_i \sum_{t=1}^{|o_i|} \frac{1}{|o_i|} \nabla_\theta \log \pi_\theta(o_{i,t} \mid q, o_{i,<t})\Big|_{\theta_{\text{old}}} - \beta\, \nabla_\theta D_{\text{KL}}. \tag{13.7}$$

Доказательство. При $\theta = \theta_{\text{old}}$ отношение важности $\rho_{i,t}(\theta) = 1$ для всех i, t. Поскольку $1 \in [1-\epsilon, 1+\epsilon]$, отсечение не активно, поэтому $L_{i,t}^{\text{CLIP}} = \rho_{i,t}\hat{A}_i$. Дифференцирование $\rho_{i,t}\hat{A}_i$ по θ при $\rho_{i,t} = 1$ даёт $\hat{A}_i \nabla_\theta \log \pi_\theta(o_{i,t} \mid q, o_{i,<t})$ — в точности градиент REINFORCE, взвешенный групп-нормированным преимуществом. □

13.3 Алгоритм GRPO

Алгоритм 27 резюмирует цикл обучения GRPO.

13.3.1 Ключевые гиперпараметры

Размер группы G — Число завершений на промпт. Типичные значения: $G \in \{4, 8, 16, 64\}$. Большее G обеспечивает оценки преимущества с меньшей дисперсией, но линейно увеличивает стоимость генерации. DeepSeek-R1 использует $G = 64$ в начале обучения.

Диапазон отсечения ϵ — Управляет доверительной областью. Стандартный выбор $\epsilon = 0{,}2$, как и в PPO.

Коэффициент KL β — Управляет дрейфом от референса. Обычные значения от $\beta = 0{,}01$ (свободный) до $\beta = 0{,}1$ (жёсткий). Некоторые реализации используют динамическое β с целевым бюджетом KL.

Число внутренних эпох K_e — Число шагов градиента на батч сгенерированных завершений. DeepSeek-R1 использует $K_e = 1$ (один проход по каждому батчу) — более консервативный подход, чем типичное $K_e \in [3, 10]$ в PPO.

Алгоритм 27: Групповая относительная оптимизация стратегии (GRPO)

Input: Стратегия π_θ, референсная стратегия π_{ref}, функция вознаграждения r, набор данных промптов $\mathcal{D}$
Input: Размер группы G, диапазон отсечения ϵ, коэффициент KL β, скорость обучения α, число внутренних эпох K_e
Output: Обученная стратегия π_θ

Инициализировать $\pi_\theta \leftarrow \pi_{\text{ref}}$
repeat
  Сэмплировать батч промптов $\{q^{(1)}, \ldots, q^{(B)}\}$ из $\mathcal{D}$
  `// Фаза генерации`
  for *каждого промпта* $q^{(b)}$, $b = 1, \ldots, B$ **do**
    Сэмплировать G завершений $o_1^{(b)}, \ldots, o_G^{(b)} \sim \pi_\theta(\cdot \mid q^{(b)})$
    Вычислить вознаграждения $r_i^{(b)} \leftarrow r(q^{(b)}, o_i^{(b)})$ для $i = 1, \ldots, G$
    Вычислить групповое среднее $\mu_G^{(b)} \leftarrow \frac{1}{G}\sum_{i=1}^{G} r_i^{(b)}$ и стандартное отклонение $\sigma_G^{(b)}$
    Вычислить преимущества $\hat{A}_i^{(b)} \leftarrow (r_i^{(b)} - \mu_G^{(b)})/(\sigma_G^{(b)} + \varepsilon_0)$ для $i = 1, \ldots, G$
    Сохранить логарифмические вероятности $\log \pi_{\theta_{\text{old}}}(o_{i,t}^{(b)} \mid q^{(b)}, o_{i,<t}^{(b)})$ для всех i, t
  `// Фаза обучения`
  Зафиксировать $\theta_{\text{old}} \leftarrow \theta$
  for *эпоха* $= 1, \ldots, K_e$ **do**
    for *каждого мини-батча пар промпт-завершение* **do**
      Вычислить $\rho_{i,t} \leftarrow \pi_\theta(o_{i,t} \mid q, o_{i,<t}) \,/\, \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$ для всех i, t
      Вычислить отсечённые цели $L_{i,t}^{\text{CLIP}}$ по (13.4)
      Вычислить штрафы KL $D_{\text{KL}}^{(i,t)}$ по (13.5)
      Вычислить потерю $\mathcal{L} \leftarrow -\frac{1}{|\text{батч}|}\sum_i \frac{1}{|o_i|}\sum_t (L_{i,t}^{\text{CLIP}} - \beta\, D_{\text{KL}}^{(i,t)})$
      $\theta \leftarrow \theta - \alpha\, \nabla_\theta \mathcal{L}$
  `// Опционально обновить референсную модель`
  Каждые N_{ref} шагов: $\pi_{\text{ref}} \leftarrow \pi_\theta$
until *сходимость*
return π_θ

Частота обновления референса N_{ref} Как часто референсная модель заменяется текущей стратегией. DeepSeek-R1 обновляет каждые 400 шагов. Частые обновления позволяют штрафу KL адаптироваться к улучшенной стратегии; редкие обновления обеспечивают более стабильный якорь.

13.3.2 Проработанный пример

Пример 13.1 (Вычисление преимущества GRPO)**.** Рассмотрим промпт q = «Чему равно $\int_0^1 x^2\,dx$?» с $G = 4$ завершениями:

i	Завершение	r_i	$\hat{A}_i$
1	«$\frac{1}{3}$. Интегрируем x^2, получаем $\frac{x^3}{3}$…» (верно)	1,0	+0,94
2	«$\frac{1}{2}$. Интеграл равен $\frac{x^2}{2}$…» (неверно)	0,0	−1,09
3	«1/3. По степенному правилу…» (верно)	1,0	+0,94
4	«Не знаю.» (неверно)	0,0	−1,09

Статистики группы: $\mu_G = 0{,}5$ и $\sigma_G \approx 0{,}577$. По (13.2): $\hat{A}_1 = (1{,}0 - 0{,}5)/0{,}577 \approx +0{,}87$; $\hat{A}_2 = (0{,}0 - 0{,}5)/0{,}577 \approx -0{,}87$. (Значения в таблице используют выборочное стандартное отклонение с поправкой Бесселя для иллюстрации.) Положительные преимущества подкрепляют правильные завершения, а отрицательные подавляют неправильные. Критик не потребовался. □

13.4 Теоретический анализ

13.4.1 Связь с REINFORCE и Baseline

В утверждении 13.1 было показано, что при $\theta = \theta_{\text{old}}$ градиент GRPO сводится к REINFORCE со средним по группе в качестве baseline. Для точности рассмотрим «неотсечённую GRPO» (при $\epsilon \to \infty$):

$$J_{\text{GRPO}}^{\text{без отсечения}}(\theta) = \frac{1}{G}\sum_{i=1}^{G} \hat{A}_i \cdot \frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \log \pi_\theta(o_{i,t} \mid q, o_{i,<t}) - \beta\, \hat{D}_{\text{KL}}. \tag{13.8}$$

Игнорируя нормировку $1/|o_i|$ и член KL, первая часть равна

$$\frac{1}{G}\sum_{i=1}^{G} \frac{r_i - \mu_G}{\sigma_G} \log \pi_\theta(o_i \mid q) = \frac{1}{\sigma_G}\left[\frac{1}{G}\sum_{i=1}^{G} r_i \log \pi_\theta(o_i \mid q) - \mu_G \cdot \frac{1}{G}\sum_{i=1}^{G} \log \pi_\theta(o_i \mid q)\right].$$

Первый член — стандартный оценщик REINFORCE. Второй член вычитает group-mean baseline μ_G, умноженный на среднюю логарифмическую вероятность. Множитель $1/\sigma_G$ масштабирует весь градиент, влияя на эффективную скорость обучения, но не на его направление.

Теорема 13.2 (Несмещённость Group-Mean Baseline)**.** *Оценщик градиента GRPO (без отсечения и нормировки по длине) является несмещённым оценщиком градиента стратегии $\nabla_\theta J(\theta)$ с точностью до зависящего от промпта масштабного множителя $1/\sigma_G$.*

Доказательство. Group-mean baseline $\mu_G = \frac{1}{G}\sum_j r_j$ включает вознаграждение r_i самого i-го сэмпла. В отличие от baseline в RLOO (уравнение (8.17)), это делает μ_G *слабо* зависимым от o_i. Однако вводимое смещение мало. Запишем $\mu_G = \frac{r_i}{G} + \frac{1}{G}\sum_{j\neq i} r_j$. Второй член $b_i' = \frac{1}{G}\sum_{j\neq i} r_j$ не зависит от o_i при данном q и, следовательно, является допустимым baseline согласно теореме 8.5. Дополнительный член r_i/G вводит смещение порядка $O(1/G)$:

$$\mathbb{E}\left[\frac{r_i}{G}\nabla_\theta \log \pi_\theta(o_i \mid q)\right] = \frac{1}{G}\nabla_\theta J(\theta),$$

что является масштабированной версией истинного градиента. После вычитания из члена $r_i \nabla_\theta \log \pi_\theta$ чистый эффект состоит в том, что оценщик GRPO аппроксимирует $(1 - 1/G)\nabla_\theta J(\theta)$ — смещённый на множитель $(G-1)/G$, что пренебрежимо мало при $G \geq 4$. Деление на σ_G является промпт-зависимым скаляром и не влияет на направление градиента в среднем. □

13.4.2 Сравнение с RLOO

В замечании 8.6 главы 8 отмечалась тесная связь между GRPO и RLOO. Таблица 13.1 делает это сравнение явным.

Таблица 13.1: Сравнение оценщиков преимущества RLOO и GRPO.

Аспект	**RLOO**	**GRPO**
Baseline	Среднее $K-1$ других сэмплов	Среднее всех G сэмплов
Нормировка	Нет (сырое отклонение)	Деление на σ_G
Смещение	Точно несмещён	Смещение $O(1/G)$
Отсечение	Нет	Отсечение в стиле PPO
Размещение KL	В вознаграждении	В функции потерь
Много эпох	Не стандартно	Да (как PPO)

Преимущество RLOO для сэмпла i равно $\hat{A}_i^{\text{RLOO}} = \frac{K}{K-1}(r_i - \bar{r})$ (уравнение (8.20)), где $\bar{r} = \frac{1}{K}\sum_j r_j$. Преимущество GRPO равно $\hat{A}_i^{\text{GRPO}} = (r_i - \bar{r})/\sigma_G$. Ключевые различия: (i) RLOO использует точный leave-one-out baseline и точно несмещён, тогда как GRPO использует среднее по группе, включая сэмпл i, и вводит смещение $O(1/G)$; (ii) GRPO делит на σ_G, что стандартизирует масштаб преимущества по промптам с разной дисперсией вознаграждения; (iii) GRPO применяет отсечение в стиле PPO, обеспечивая безопасное обучение с несколькими эпохами на одном батче.

13.4.3 Снижение дисперсии при групповой нормализации

Утверждение 13.3 (Дисперсия преимущества GRPO)**.** *Пусть $r_1, \ldots, r_G$ независимы и одинаково распределены со средним μ и дисперсией σ^2. Дисперсия ненормированного преимущества $r_i - \mu_G$ равна*

$$\text{Var}[r_i - \mu_G] = \sigma^2\left(1 - \frac{1}{G}\right) = \frac{(G-1)\sigma^2}{G}. \tag{13.9}$$

После стандартизации на σ_G преимущество $\hat{A}_i$ имеет приблизительно единичную дисперсию независимо от распределения вознаграждений.

Доказательство. Поскольку $\mu_G = \frac{1}{G}\sum_j r_j$, имеем $r_i - \mu_G = r_i - \frac{1}{G}r_i - \frac{1}{G}\sum_{j\neq i} r_j = \frac{G-1}{G}r_i - \frac{1}{G}\sum_{j\neq i} r_j$. Используя независимость:

$$\mathrm{Var}[r_i - \mu_G] = \left(\frac{G-1}{G}\right)^2 \sigma^2 + \frac{G-1}{G^2}\sigma^2 = \frac{(G-1)\sigma^2}{G}.$$

Деление на $\sigma_G \approx \sigma\sqrt{(G-1)/G}$ (при больших G) даёт $\mathrm{Var}[\hat{A}_i] \approx 1$. □

Эта автоматическая нормализация дисперсии является практическим преимуществом GRPO перед RLOO: промпты с высокой дисперсией вознаграждения и промпты с низкой дисперсией вносят обновления градиента сопоставимого масштаба, не давая ни одному промпту доминировать в мини-батче.

13.4.4 Влияние размера группы G

Увеличение размера группы G оказывает два эффекта:

1. **Лучшая оценка baseline.** Среднее по группе μ_G сходится к истинному ожидаемому вознаграждению $\mathbb{E}_{\pi_\theta}[r \mid q]$ со скоростью $O(1/\sqrt{G})$. Это снижает дисперсию оценки преимущества и улучшает качество градиента.

2. **Более высокая стоимость генерации.** Каждый промпт требует G прямых проходов через модель. Если модель генерирует в среднем T токенов, общая стоимость генерации на промпт равна $O(GT)$, что может доминировать над стоимостью обучения.

Оптимальное G уравновешивает статистическую эффективность и вычислительные затраты. Эмпирически G от 8 до 64 хорошо работает для задач рассуждения. Для бинарных вознаграждений (правильно / неправильно) более крупные группы особенно выгодны, поскольку они обеспечивают более разнообразные контрастные пары.

Пример 13.2 (Влияние размера группы на качество преимущества)**.** Предположим, промпт имеет истинное ожидаемое вознаграждение $\mu = 0{,}3$ (модель решает его в 30% случаев), и вознаграждения бинарные $r_i \in \{0, 1\}$. При $G = 4$ возможные средние по группе равны $\mu_G \in \{0; 0{,}25; 0{,}5; 0{,}75; 1\}$. Вероятность того, что $\mu_G = 0$ (все четыре неверные), равна $(0{,}7)^4 = 0{,}240$, в этом случае все преимущества равны нулю и промпт не вносит никакого обучающего сигнала. При $G = 16$ вероятность того, что все ответы неверны, равна $(0{,}7)^{16} = 0{,}003$ — пренебрежимо мало. Большие группы гарантируют, что почти каждый промпт вносит контрастный сигнал. □

13.5 RLVR: обучение с подкреплением с проверяемыми вознаграждениями

13.5.1 Определение и мотивация

Многие важные задачи обладают свойством, что правильность ответа можно проверить автоматически, без обученной модели вознаграждения. Математические рассуждения («Равен ли $\frac{1}{3}$ правильному ответу на $\int_0^1 x^2\,dx$?»), генерация кода («Проходит ли эта функция все юнит-тесты?») и формальная логика («Проверяется ли это доказательство по типам?») — все они допускают детерминированную верификацию. Этот класс задач мотивирует отдельную парадигму обучения.

Определение 13.3 (RLVR)**. Обучение с подкреплением с проверяемыми вознаграждениями** (RLVR) является парадигмой обучения, при которой:

1. функция вознаграждения $r(q, o)$ является *детерминированным верификатором*, проверяющим правильность ответа o на промпт q;

2. верификатор возвращает бинарное вознаграждение: $r(q, o) \in \{0, 1\}$ (или более общее вознаграждение на основе правил);

3. обученная модель вознаграждения не используется; верификатор — это фиксированная, доверенная функция.

RLVR имеет ряд преимуществ перед RLHF с обученной моделью вознаграждения:

Нет взлома вознаграждения Верификатор — это фиксированная функция истинного ответа, а не обученное приближение. Стратегия не может найти ложные выходные данные, «обманывающие» верификатор (при условии, что верификатор корректен).

Неограниченные данные Поскольку верификатор автоматизирован, можно генерировать и оценивать неограниченное число завершений без каких-либо затрат на аннотирование людьми. Это обеспечивает значительно более масштабное онлайн-обучение.

Чистый сигнал Бинарные вознаграждения не зашумлены: математический ответ либо верен, либо нет. Нет разногласий между аннотаторами, нет дрейфа калибровки и субъективных суждений.

Нет обучения модели вознаграждения Верификатор — это фиксированная функция (например, символический решатель или test harness), поэтому нет модели вознаграждения для обучения, оценки или deployment.

13.5.2 Формальная постановка

В рамках RLVR обучающие данные состоят из набора промптов $\mathcal{D} = \{q^{(1)}, \ldots, q^{(N)}\}$, каждый со связанным эталонным ответом $a^{(i)}$. Верификатор $V : (q, o) \to \{0, 1\}$ извлекает ответ модели из завершения o и сравнивает его с $a^{(i)}$:

$$r(q, o) = V(q, o) = \mathbb{1}[\text{extract}(o) = a]. \tag{13.10}$$

Более сложные верификаторы могут присваивать частичный зачёт, проверять промежуточные шаги или объединять несколько сигналов вознаграждения:

$$r(q, o) = w_{\text{acc}}\, V_{\text{accuracy}}(q, o) + w_{\text{fmt}}\, V_{\text{format}}(o) + w_{\text{len}}\, V_{\text{length}}(o), \tag{13.11}$$

где V_{accuracy} проверяет правильность, V_{format} проверяет соответствие предписанному формату вывода (например, использование тегов `<think>`…`</think>`), а V_{length} штрафует чрезмерно длинные ответы.

13.5.3 Связь с GRPO

GRPO является естественным алгоритмом оптимизации для RLVR. Бинарная структура вознаграждения гарантирует, что группы содержат смесь правильных и неправильных завершений (для промптов умеренной сложности), производя нетривиальные преимущества. Отсутствие обученной модели вознаграждения означает, что две оставшиеся копии моделей (π_θ и π_{ref}) легко помещаются в память GPU, а верификатор добавляет пренебрежимо малые вычислительные накладные расходы.

Пайплайн обучения DeepSeek-R1 (DeepSeek-AI, 2025) продемонстрировал эту комбинацию в масштабе: GRPO с бинарными вознаграждениями на основе правил, обучаемый непосредственно на базовой языковой модели (без какого-либо этапа дообучения с учителем), породил модель, которая спонтанно развила рассуждение с цепочкой мыслей, самопроверку и стратегии исследования — способности, возникшие только из обучения RL.

Пример 13.3 (Эмерджентное рассуждение в DeepSeek-R1-Zero)**.** DeepSeek-R1-Zero применил GRPO с бинарными вознаграждениями за точность к базовой языковой модели (без SFT). Во время обучения модель спонтанно выработала несколько сложных поведений:

- **Рассуждение с цепочкой мыслей**: модель начала производить пошаговые выводы, разбивая сложные задачи на подзадачи.
- **Самопроверка**: модель начала проверять свои промежуточные результаты, производя такие фразы, как «Подождите, дайте мне проверить…» перед окончательным ответом.
- **Исследование альтернативных подходов**: когда первый подход не срабатывал, модель начала пробовать альтернативные стратегии решения в рамках одного ответа.

Ни одно из этих поведений не вознаграждалось явно — проверялась только правильность итогового ответа. Они возникли потому, что ответы с цепочкой мыслей с большей вероятностью дают правильные ответы, а сигнал преимущества GRPO подкреплял всю последовательность токенов, приводящую к правильным результатам. □

13.5.4 Области применения RLVR

Математика Символическая верификация по сравнению с эталонными ответами. Инструменты такие как `sympy`, `math_verify` и `latex2sympy` разбирают и сравнивают математические выражения.

Генерация кода Наборы юнит-тестов служат верификатором. Модель генерирует код; код выполняется в изолированной среде; вознаграждение — доля пройденных тестов.

Формальные доказательства Средства проверки типов для систем доказательств теорем (Lean, Coq, Isabelle) верифицируют доказательства детерминированно.

Генерация с ограничениями Ограничения формата (например, валидация JSON-схемы) предоставляют бинарную обратную связь о структурной правильности.

Естественные науки Числовые ответы по физике, химии и инженерии можно верифицировать в пределах допуска.

13.6 Rejection Sampling и best-of-N

До применения GRPO (или любого алгоритма RL) естественным baseline является сэмплирование **best-of-N**: генерация N завершений, оценка каждого верификатором или моделью вознаграждения и возврат наивысшего по баллам. Иногда это называется *rejection sampling fine-tuning*, когда лучшие завершения используются как контролируемые обучающие данные.

13.6.1 Алгоритм

Для каждого промпта q сэмплируем $o_1, \ldots, o_N \sim \pi_\theta(\cdot \mid q)$ и выбираем $o^* = \operatorname{argmax}_i r(q, o_i)$. Ожидаемое вознаграждение best-of-N растёт с N: если каждое завершение правильно с вероятностью p (бинарное вознаграждение), вероятность того, что хотя бы одно из N правильно, равна $1 - (1-p)^N$.

Пример 13.4 (Best-of-N против одного сэмпла)**.** Модель решает данную математическую задачу с вероятностью $p = 0{,}3$. С одним сэмплом ожидаемое вознаграждение равно 0,3. При $N = 8$ вероятность хотя бы одного правильного решения равна $1 - (0{,}7)^8 = 0{,}942$. При $N = 64$ она возрастает до $1 - (0{,}7)^{64} > 0{,}9999$. Best-of-$N$ драматически улучшает производительность во время инференса, но стоимость линейно растёт с N. □

13.6.2 Rejection Sampling Fine-Tuning

Вместо использования best-of-N только во время инференса можно использовать отобранные завершения как обучающие данные SFT. Этот процесс, называемый rejection sampling fine-tuning (RFT), создаёт улучшенный набор данных из собственных генераций модели:

1. Генерировать N завершений на промпт.
2. Фильтровать, оставляя только правильные (или высокооцениваемые) завершения.
3. Дообучить π_θ на отфильтрованном наборе данных с использованием стандартного обучения с учителем.

13.6.3 Сравнение с GRPO

Best-of-N и GRPO тесно связаны. Оба сэмплируют несколько завершений и используют вознаграждение для различения хороших и плохих. Ключевые различия:

- **Использование сигнала.** Best-of-N отбрасывает всю информацию, кроме единственного лучшего завершения. GRPO использует полное распределение вознаграждений для вычисления преимуществ, извлекая контрастный обучающий сигнал из каждого завершения.
- **Качество градиента.** Best-of-N сводится к SFT на отобранных примерах, что не может снизить вероятность плохих ответов. GRPO явно снижает вероятность отрицательно-адвантажных завершений.
- **Итеративное улучшение.** Best-of-N с RFT требует чередования этапов генерации и SFT. GRPO интегрирует генерацию и оптимизацию стратегии в единый цикл.

- **Когда достаточно rejection sampling.** Если модель уже решает задачу с умеренной вероятностью ($p \geq 0{,}3$) и цель — качество во время deployment, best-of-N при инференсе может быть более экономически эффективным, чем обучение RL. Если модель редко решает задачу ($p < 0{,}05$) и цель — улучшить базовую модель, необходимо обучение RL (GRPO) для сдвига распределения.

13.7 Онлайн-обучение RL в масштабе

13.7.1 Двухфазная архитектура

Производственное обучение GRPO в масштабе разделяет нагрузку на две асинхронные фазы:

Фаза генерации Rollout workers размещают копии текущей стратегии π_θ для инференса и генерируют завершения. Эта фаза ограничена памятью (веса модели) и чувствительна к задержке (авторегрессивная генерация токенов). Для максимизации пропускной способности используются оптимизированные движки инференса, такие как vLLM (Kwon и др., 2023), с непрерывным батчингом и PagedAttention.

Фаза обучения Trainer workers получают сгенерированные завершения, вычисляют преимущества и выполняют обновления градиента. Эта фаза ограничена вычислениями (прямые и обратные проходы через полную модель с вычислением градиента).

Две фазы могут работать в пайплайн режиме: пока обучающий модуль обновляется на батче k, rollout workers генерируют батч $k+1$ с использованием последнего checkpoint. Это перекрытие скрывает задержку генерации и поддерживает высокую утилизацию GPU.

13.7.2 Обработка завершений переменной длины

Практической проблемой является то, что завершения внутри группы могут резко различаться по длине — от нескольких токенов («1/3») до тысяч токенов (многостраничный вывод с цепочкой мыслей). Используются несколько стратегий:

- **Дополнение и маскировка.** Дополнить все завершения в группе до максимальной длины и использовать маски внимания. Просто, но тратит память на короткие завершения.
- **Упаковка.** Конкатенировать завершения из разных промптов в единую последовательность, разделённую токенами-разделителями, и использовать блочно-диагональные маски внимания для предотвращения перекрёстного смешения. Это максимизирует утилизацию GPU.
- **Максимальная длина генерации.** Применять жёсткое ограничение на длину завершения. Для задач рассуждения обычны ограничения в 4096 — 8192 токена.

Нормировка $1/|o_i|$ в цели GRPO (13.6) необходима при завершениях переменной длины: без неё длинные завершения вносили бы непропорционально большой вклад в потерю.

13.7.3 Формирование батча

Для GRPO каждый «мини-батч» состоит из промптов, каждый с G завершениями. Если батч содержит B промптов, общее число завершений равно BG. DeepSeek-R1 использует $BG = 8192$ завершений на rollout, которые случайным образом разбиваются на 16 мини-батчей по 512 завершений для одной эпохи обучения.

Стоимость генерации на итерацию масштабируется как $O(BG \cdot \bar{T})$, где $\bar{T}$ — средняя длина завершения. Для задач долгосрочного рассуждения ($\bar{T} \approx 2000$ токенов) при $BG = 8192$ это примерно 16 миллионов токенов на rollout — существенная нагрузка инференса, которая мотивирует использование выделенных inference-кластеров.

13.7.4 Синхронный и асинхронный дизайн

В **синхронном** дизайне обучающий модуль ждёт завершения генерации всеми rollout workers, прежде чем начать фазу обучения. Это гарантирует, что все завершения получены из одного и того же checkpoint стратегии, делая отношения важности $\rho_{i,t} = 1$ в начале каждой фазы обучения. Недостатком является простой: обучающие модули ждут во время генерации, а rollout workers — во время обучения.

В **асинхронном** дизайне rollout workers непрерывно генерируют завершения и помещают их в очередь, тогда как обучающие модули извлекают из очереди и обновляют стратегию. Стратегия, использованная

для генерации, может отставать от обучаемой стратегии на один или несколько шагов. Это вносит лёгкое off-policy mismatch, но отсечённое отношение важности в GRPO естественно с этим справляется: если $\rho_{i,t}$ слишком сильно отклоняется от 1, отсечение ограничивает вклад в градиент, предотвращая нестабильность из-за устаревших данных. Асинхронные дизайны достигают значительно более высокой утилизации GPU и предпочтительны в масштабе.

13.7.5 Воспроизведение опыта для GRPO

В отличие от PPO, который обычно использует данные по текущей стратегии несколько эпох, а затем отбрасывает их, GRPO может извлечь пользу из лёгкой формы experience replay. Недавние завершения из предыдущих M итераций можно хранить в replay buffer и смешивать со свежими данными по текущей стратегии. Отношение важности $\rho_{i,t}(\theta)$ естественно корректирует устаревание воспроизводимых данных, а механизм отсечения предотвращает чрезмерные коррекции вне текущей стратегии. Experience replay снижает стоимость генерации, амортизируя каждое завершение на несколько шагов обучения, ценой незначительно возросшего несоответствия распределений.

13.7.6 Мониторинг и диагностика

Эффективное обучение GRPO требует мониторинга нескольких величин:

Кривая вознаграждения Среднее вознаграждение по промптам должно расти в ходе обучения. Плато может указывать на то, что распределение промптов слишком лёгкое (все промпты уже решены) или слишком сложное (нет контрастного сигнала).

Дивергенция KL Скользящее среднее $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ должно расти медленно. Быстрый рост KL сигнализирует об опасном удалении стратегии от референса; следует увеличить β или уменьшить скорость обучения.

Доля отсечения Доля токенов, где отношение важности $\rho_{i,t}$ отсечено. Доля отсечения выше 10 — 15% указывает на то, что стратегия меняется слишком быстро за итерацию; следует рассмотреть уменьшение скорости обучения или числа внутренних эпох K_e.

Разнообразие групп Доля групп, содержащих как положительные, так и отрицательные преимущества. Если эта доля падает ниже 20%, промпты могут быть слишком лёгкими или слишком сложными, и следует перебалансировать распределение промптов.

Величина преимущества Средняя абсолютная величина преимущества $|\hat{A}_i|$ должна быть приблизительно равна 1 (по построению, благодаря стандартизации). Экстремальные значения указывают на численные проблемы при нормализации.

13.8 Сравнение: PPO, GRPO и DPO

Таблица 13.2 содержит детальное сравнение трёх основных алгоритмов выравнивания и RL, рассмотренных в главах 9, 12 и настоящей главе.

13.8.1 Когда применять каждый метод

Применяйте PPO, когда сигнал вознаграждения исходит от обученной модели вознаграждения и важно присваивание заслуг на уровне токена — например, в общем RLHF, где качество отдельных токенов влияет на общий ответ. Основанные на GAE преимущества PPO обеспечивают тонкий сигнал, но инфраструктурные затраты высоки.

Применяйте GRPO, когда вознаграждения проверяемы или когда приоритетны простота и экономия памяти. GRPO является предпочтительным алгоритмом для задач рассуждения (математика, код, логика), где доступны бинарные вознаграждения за правильность. Он также эффективен с обученными моделями вознаграждения, когда стоимость критика чрезмерна.

Применяйте DPO, когда имеются данные предпочтений, но онлайн-генерация слишком дорога или непрактична. DPO является самым быстрым путём от SFT-модели к выровненной модели, не требуя инфраструктуры RL. Для задач, где сдвиг распределения вызывает опасения, итеративный DPO (сбор новых данных предпочтений между раундами) частично закрывает разрыв.

Таблица 13.2: Детальное сравнение PPO, GRPO и DPO для обучения LLM.

Свойство	**PPO** (гл. 9)	**GRPO** (эта глава)	**DPO** (гл. 12)
Online / Offline	Online	Online	Offline
Нужен критик?	Да (сеть функции ценности)	Нет	Нет
Модель вознаграждения?	Да (обученная RM)	Верификатор или RM	Нет (неявная в потере)
Копии моделей	4 (актор, критик, реф., RM)	2 (актор, реф.)	2 (актор, реф.)
Оценка преимущества	GAE (на токен)	Групповая нормализация (на последовательность)	Нет (без преимущества)
Доверительная область	Отсечённое отношение	Отсечённое отношение	Неявная KL
Требование к данным	Только промпты	Только промпты	Пары предпочтений
Исследование	Сэмплирование по текущей стратегии	Сэмплирование по текущей стратегии	Нет (фиксированный набор данных)
Сдвиг распределения	Нет	Нет	Да (деградирует со временем)
Присваивание заслуг	На токен	На последовательность	На пару
Риск взлома вознаграждения	Высокий (обученная RM)	Низкий (верификатор)	Нет (нет RM)
Сложность реализации	Высокая	Средняя	Низкая

Замечание 13.3. Эти методы не являются взаимоисключающими. Общий производственный паттерн — начальный запуск с DPO на имеющемся наборе данных предпочтений, затем уточнение с GRPO на проверяемых задачах и, наконец, шлифовка с PPO с использованием обученной модели вознаграждения для открытого качества. Каждый этап устраняет отдельное измерение выравнивания (см. пример 12.2 в главе 12).

Пример 13.5 (Бюджет памяти при масштабе 70B)**.** Рассмотрим обучение модели с 70B параметрами в формате половинной точности (2 байта на параметр, то есть 140 ГБ на копию модели). PPO-RLHF требует четыре копии: актор (140 ГБ), критик (140 ГБ), референс (140 ГБ) и модель вознаграждения (140 ГБ), итого 560 ГБ без учёта состояний оптимизатора, активаций или градиентов. Один узел 8×A100 (640 ГБ всего) едва вмещает модели, оставляя почти не места для активаций.

GRPO-RLVR требует две копии: актор (140 ГБ) и референс (140 ГБ), итого 280 ГБ. Верификатор (например, символический математический проверщик) пренебрежимо мал. Это оставляет 360 ГБ на том же узле для состояний оптимизатора и активаций — достаточно для комфортного обучения с контрольными точками градиента. Двукратное снижение требований к памяти нередко является решающим фактором при выборе GRPO вместо PPO в большом масштабе. □

13.9 Итоги

В этой главе была разработана групповая относительная оптимизация стратегии (GRPO) — алгоритм градиента стратегии, устраняющий критика с функцией ценности путём оценки преимуществ по группам сэмплированных завершений.

Преимущество GRPO (определение 13.1) стандартизирует вознаграждение каждого завершения относительно среднего и стандартного отклонения по группе, производя сигнал с нулевым средним и приблизительно единичной дисперсией, не требующий обученных параметров. Отсечённая суррогатная цель (уравнение (13.4)) наследует свойство доверительной области PPO, обеспечивая безопасное обучение с несколькими эпохами. Штраф KL (уравнение (13.5)) применяется непосредственно в функции потерь, а не во вознаграждении, чётко разделяя регуляризацию и вычисление преимущества.

Теоретический анализ (раздел 13.4) показал, что градиент GRPO тесно связан с REINFORCE с group-mean baseline, со смещением $O(1/G)$, исчезающим при умеренных размерах группы. Стандартизация на σ_G автоматически нормирует величины градиентов по промптам с разной дисперсией вознаграждений — практическое преимущество перед RLOO (раздел 8.6 главы 8).

RLVR (раздел 13.5) формализовал парадигму обучения с детерминированными верификаторами вместо обученных моделей вознаграждения. Комбинация GRPO и RLVR обеспечивает крупномасштабный онлайн-RL с чистыми сигналами вознаграждения, без взлома вознаграждения и с минимальными инфраструктурными накладными расходами — конфигурация, породившая DeepSeek-R1 и аналогичные модели рассуждений.

Таблица сравнения (таблица 13.2) помещает GRPO в контекст: он занимает нишу среди online, critic-free методов, дополняя PPO (online, с критиком) и DPO (offline, без критика).

Глава 8 закладывает фундаментальный оценщик REINFORCE и baseline RLOO, на которых строится GRPO.

Глава 9 развивает PPO — алгоритм, отсечённый суррогат которого GRPO наследует, но критика которого устраняет.

Глава 12 развивает DPO — offline-альтернативу, проблема сдвига распределения которой мотивирует online-подход GRPO.

Глава 14 описывает полный производственный пайплайн RLHF, где GRPO может заменить PPO в качестве оптимизатора RL.

Упражнения

Упражнение 13.1. Преимущество GRPO вручную. Промпт получает $G = 5$ завершений с вознаграждениями $r = (0, 1, 1, 0, 1)$. Вычислите групповое среднее μ_G, групповое стандартное отклонение σ_G и преимущество GRPO $\hat{A}_i$ для каждого завершения. Проверьте, что $\sum_i \hat{A}_i \neq 0$, но $\sum_i \hat{A}_i \approx 0$ (объясните, почему точный нуль не гарантирован при использовании среднего по полной группе).

Упражнение 13.2. GRPO против RLOO. Используя вознаграждения из упражнения 13.1, вычислите преимущество RLOO $\hat{A}_i^{\text{RLOO}} = \frac{K}{K-1}(r_i - \bar{r})$ для каждого завершения. Сравните два набора преимуществ. При каком условии на распределение вознаграждений два оценщика дали бы одно и то же *направление* градиента?

Упражнение 13.3. Вырожденные группы. Что происходит с преимуществом GRPO, когда все завершения в группе получают одинаковое вознаграждение? Почему такое поведение желательно с точки зрения обучения? Предложите модификацию, которая всё же извлекала бы сигнал из таких групп.

Упражнение 13.4. Смещение group-mean baseline. Докажите, что смещение от использования $\mu_G = \frac{1}{G}\sum_j r_j$ в качестве baseline (вместо среднего leave-one-out) в оценщике REINFORCE точно равно $\frac{1}{G}\nabla_\theta J(\theta)$. Сделайте вывод, что смещение исчезает при $G \to \infty$.

Упражнение 13.5. Вероятность best-of-N. Модель имеет точность p на единичный сэмпл для данной задачи. Выведите ожидаемое вознаграждение (точность) сэмплирования best-of-N как функцию от p и N. Для $p = 0{,}1$ вычислите значение N, необходимое для достижения 90% ожидаемой точности.

Упражнение 13.6. Свойства оценщика KL. Покажите, что оценщик KL GRPO (13.5) неотрицателен и равен нулю тогда и только тогда, когда $\pi_\theta = \pi_{\text{ref}}$ в данной позиции токена. *Подсказка:* используйте неравенство $e^x \geq 1 + x$.

Упражнение 13.7. Размер группы и покрытие. Для бинарных вознаграждений с точностью p на единичный сэмпл выведите вероятность того, что группа размера G содержит *как* правильные, так и неправильные завершения (то есть производит нетривиальный контрастный сигнал). Постройте эту вероятность как функцию от G для $p \in \{0{,}1; 0{,}3; 0{,}5; 0{,}7; 0{,}9\}$.

Упражнение 13.8. GRPO с вознаграждением за процесс. Стандартный GRPO присваивает одинаковое преимущество каждому токену в завершении (надзор за результатом). Предположим, верификатор процесса присваивает вознаграждение $r_i^{(t)}$ после каждого шага рассуждения t. Предложите модификацию преимущества GRPO, включающую вознаграждения на уровне шагов, сохраняя при этом отсутствие обученного критика. Запишите модифицированную цель.

Упражнение 13.9. Эффективность rejection sampling против GRPO. Задача имеет точность $p = 0{,}2$ на единичный сэмпл. Сравните следующие две стратегии улучшения модели: (a) rejection sampling fine-tuning с best-of-N при $N = 16$, обучение на правильных завершениях одну эпоху; (b) GRPO с $G = 16$. Какая стратегия использует сгенерированные данные более эффективно? Обсудите компромиссы с точки зрения использования данных, качества градиента и стоимости вычислений.

Упражнение 13.10. Реализация. Реализуйте упрощённый цикл обучения GRPO в виде псевдокода на PyTorch. Ваша реализация должна: (i) сэмплировать G завершений на промпт, (ii) вычислять группнормированные преимущества, (iii) вычислять отсечённую суррогатную потерю, и (iv) добавлять штраф KL. Предполагается наличие функций `model.generate()`, `model.log_probs()`, `ref_model.log_probs()` и `reward_fn()`.

Глава 14

Практический RLHF

В теории нет разницы между теорией и практикой. На практике она есть. — Йоги Берра (приписывается)

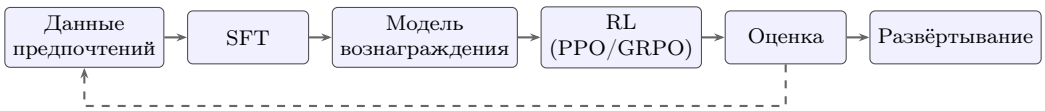
В предыдущих главах была разработана теория моделей вознаграждения (Глава 10), прямой оптимизации предпочтений (Глава 12) и GRPO (Глава 13). В каждой главе алгоритмы представлены в идеализированном виде: чистые функции потерь, элегантные выводы, гарантии сходимости при стандартных допущениях. Эта глава другая. Это инженерная глава — практическое руководство по всему, что находится между теоретическим алгоритмом и production-системой RLHF.

Разрыв между теорией и практикой в RLHF огромен. Тот, кто впервые попытается следовать правилу обновления PPO из Главы 9, уже через несколько часов обнаружит, что процесс обучения расходится, модель вознаграждения эксплуатируется, память GPU исчерпана или модель коллапсировала в единственный повторяющийся режим. Каждый из этих режимов отказа был встречен — и решён — командами, стоящими за InstructGPT, Llama 2, Claude и DeepSeek, однако решения рассредоточены по блог-постам, техническим отчётам и неформальным инженерным знаниям.

В этой главе эти знания собраны в единое связное изложение. Раздел 14.1 описывает полный пайплайн RLHF end-to-end. Раздел 14.2 охватывает искусство и науку сбора данных о предпочтениях. Раздел 14.3 рассматривает обучение модели вознаграждения на практике. Раздел 14.4 подробно описывает реализацию PPO для языковых моделей, включая четырёхмодельную схему, таблицы гиперпараметров и соображения о смешанной точности. Раздел 14.5 представляет диагностическое руководство по наиболее распространённым режимам отказа. Раздел 14.6 рассматривает проблемы распределённых систем при масштабировании RLHF до больших моделей. Раздел 14.7 обсуждает ограничения безопасности и элайнмента, которые должны быть наложены поверх основного цикла обучения. Раздел 14.8 представляет краткие кейсы четырёх крупных работ по выравниванию.

14.1 Пайплайн RLHF End-to-End

Прежде чем углубляться в детали каждого компонента, представим полный пайплайн RLHF как единый поток. Пайплайн состоит из шести этапов, каждый из которых подаёт данные в следующий.



Этап 1: Сбор данных Собрать демонстрационные данные для SFT и попарные данные предпочтений для модели вознаграждения. Данные предпочтений могут поступать от экспертных аннотаторов, краудсорсеров или с помощью разметки с участием ИИ.

Этап 2: Supervised fine-tuning (SFT) Обучить базовую языковую модель на высококачественных демонстрациях для получения π_{SFT}. Это отправная точка для всех последующих этапов и одновременно опорная стратегия π_{ref}.

Этап 3: Обучение модели вознаграждения Обучить скалярную модель вознаграждения r_ψ на попарных предпочтениях, используя функцию потерь Bradley–Terry из Главы 10 (Уравнение (10.6)).

Этап 4: RL-обучение Оптимизировать стратегию π_θ относительно r_ψ с помощью PPO (Раздел 14.4) или GRPO (Глава 13), со штрафом KL в сторону π_{ref} для предотвращения переоптимизации вознаграждения (Раздел 10.8).

Этап 5: Оценка Оценить выровненную модель на отложенных бенчмарках предпочтений, тестах безопасности и человеческих оценках. Сравнить с π_{SFT} и предыдущими контрольными точками.

Этап 6: Развёртывание Развернуть лучшую контрольную точку с мониторингом, логированием и фильтрами безопасности. Собирать новую обратную связь из продакшена для замыкания петли.

Пунктирная стрелка от оценки обратно к сбору данных представляет цикл *итеративного выравнивания*: обратная связь из продакшена генерирует новые данные предпочтений, что запускает новый раунд обучения модели вознаграждения и стратегии. На практике крупные усилия по выравниванию запускают этот цикл несколько раз. InstructGPT (Ouyang et al., 2022) выполнил две полные итерации; Llama 2 (Touvron et al., 2023) — пять.

Замечание 14.1 (Этапы не всегда последовательны). Не каждая система точно следует линейному шестиэтапному пайплайну. Некоторые команды обучают модель вознаграждения и модель SFT совместно (многозадачное обучение). Другие вообще пропускают отдельный этап SFT и начинают RL-обучение с предобученной базовой модели, компенсируя это мощной моделью вознаграждения. GRPO (Глава 13) устраняет модель вознаграждения для задач с верифицируемыми вознаграждениями. DPO (Глава 12) сворачивает этапы 3 и 4 в единый шаг обучения с учителем. Приведённый выше пайплайн представляет наиболее распространённую архитектуру, однако практики должны воспринимать каждый этап как модуль, который можно заменить.

14.2 Сбор данных о предпочтениях

Качество данных о предпочтениях ограничивает качество всего последующего. Идеальный алгоритм RL, оптимизирующий шумную модель вознаграждения, обученную на небрежных аннотациях, даст посредственную стратегию. В этом разделе описаны передовые практики сбора данных о предпочтениях.

14.2.1 Форматы аннотирования

Два основных формата аннотирования предпочтений — *попарное сравнение* и *оценка по шкале Лайкерта*.

Попарное сравнение. По заданному промпту x и двум ответам y_A, y_B аннотатор выбирает лучший ответ. Это формат, принятый в модели Bradley–Terry (Определение 10.3). Его главное преимущество — когнитивная простота: людям легче выносить относительные суждения, чем абсолютные. Основной недостаток — неэффективность: сравнение n ответов попарно требует $\binom{n}{2}$ аннотаций, тогда как оценки по Лайкерту требуют только n.

Оценка по шкале Лайкерта. По заданному промпту x и одному ответу y аннотатор присваивает балл по фиксированной шкале (например, 1–7). Оценки по Лайкерту можно преобразовать в попарные предпочтения, считая ответ с более высокой оценкой предпочтительным, но это преобразование вносит шум, когда два рейтинга близки. Преимущество — эффективность; недостаток — индивидуальные рейтеры часто используют шкалу по-разному (“5” одного аннотатора соответствует “3” другого).

На практике большинство крупных усилий по выравниванию используют попарное сравнение как основной формат. При сборе данных для Claude компания Anthropic использует четырёхбалльную шкалу (A намного лучше, A немного лучше, B немного лучше, B намного лучше), что позволяет включать margin в функцию потерь модели вознаграждения (Уравнение (10.8)).

14.2.2 Согласованность аннотаторов

Когда несколько аннотаторов размечают одну и ту же пару, они нередко расходятся во мнениях. *Межаннотаторское согласие* (IAA) измеряет согласованность человеческих суждений. Стандартная метрика — каппа Коэна:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

где p_o — наблюдаемая доля согласий, а p_e — ожидаемое случайное согласие. Для бинарных попарных предпочтений $p_e = 0.5$ и $\kappa = 2p_o - 1$. Значения $\kappa \geq 0.4$ обычно считаются умеренным согласием; $\kappa \geq 0.6$ — существенным.

InstructGPT сообщал о IAA около 73% ($\kappa \approx 0.46$) на своих данных предпочтений. Это согласуется с общим выводом о том, что аннотирование предпочтений — по своей сути шумная задача — разные аннотаторы могут иметь подлинно различные предпочтения, а не просто случайные расхождения. Метод сглаживания меток cDPO (Уравнение (12.11)) явно учитывает этот шум.

14.2.3 Качество данных против их количества

Повторяющийся вопрос: стоит ли вкладываться в большее количество данных или в их лучшее качество. Эмпирические данные склоняются в пользу качества. Touvron et al. (2023) обнаружили, что качество модели вознаграждения Llama 2 достигало плато примерно после миллиона пар предпочтений, после чего наблюдались убывающие отдачи. Напротив, замена шумных краудсорсинговых меток экспертными аннотациями давала стабильное улучшение даже при одинаковом размере датасета.

Замечание 14.2 (Оценки стоимости данных предпочтений). По состоянию на 2024 год типичные затраты на аннотирование попарных предпочтений составляют: краудсорсинг через такие платформы, как Scale AI или Surge AI, — \$1–3 за сравнение; экспертные аннотации обученных лингвистов — \$5–15 за сравнение; аннотации красной команды для безопасности — \$10–25 за сравнение. Полный датасет для выравнивания из 100 000 пар предпочтений обходится примерно в \$100K–300K для краудсорсинговых данных и значительно дороже для экспертно-аннотированных. Эти затраты стимулируют растущий интерес к разметке с помощью ИИ, когда мощная языковая модель генерирует предварительные метки, которые затем проверяют люди-аннотаторы.

14.2.4 Распространённые ошибки

- **Смещение по длине.** Аннотаторы склонны предпочитать более длинные ответы, даже когда дополнительная длина не добавляет ценности. Это переносится на модель вознаграждения, которая затем поощряет многословный вывод. Меры борьбы: инструктировать аннотаторов штрафовать излишнюю многословность; включать в набор для аннотирования пары с контролируемой длиной.
- **Позиционное смещение.** Когда два ответа представлены рядом, аннотаторы слегка предпочитают тот, что слева (или первый). Меры борьбы: рандомизировать порядок представления для каждой пары.
- **Неоднозначные промпты.** Когда промпт неясен, аннотаторы могут расходиться не потому, что имеют разные стандарты качества, а потому, что интерпретировали промпт по-разному. Меры борьбы: фильтровать промпты с высоким несогласием аннотаторов.
- **Утечка подхалимства.** Если аннотаторы предпочитают ответы, соглашающиеся с тезисами промпта (даже ложными), модель вознаграждения научится быть подхалимской. Меры борьбы: включать состязательные промпты с ложными тезисами и вознаграждать аннотаторов за выбор ответов, вежливо исправляющих пользователя.

14.3 Обучение моделей вознаграждения на практике

В Главе 10 была выведена функция потерь Bradley–Terry и доказано, что модель вознаграждения сходится к истинной функции вознаграждения с точностью до константы. На практике ряд дополнительных соображений определяет, работает ли модель вознаграждения в действительности.

14.3.1 Выбор архитектуры

Модель вознаграждения $r_\psi(x, y)$ — это, как правило, трансформер, принимающий на вход конкатенацию промпта и ответа и выдающий скалярный результат. Применяются две архитектурные стратегии.

Та же архитектура, что и стратегия. Модель вознаграждения инициализируется из той же контрольной точки SFT, что и стратегия, при этом голова языкового моделирования заменяется линейной проекцией в скаляр. Этот подход используется InstructGPT и Llama 2. Преимущество — модель вознаграждения начинает с сильным представлением языка; недостаток — большой объём памяти, поскольку модель вознаграждения столь же велика, как и стратегия.

Отдельная, меньшая модель. Модель вознаграждения использует меньший трансформер (например, 7B-модель вознаграждения для 70B-стратегии). Это снижает потребность в памяти, но рискует недостаточно подогнать распределение предпочтений. Gao et al. (2023) показали, что меньшие модели вознаграждения ведут к более быстрой переоптимизации (см. Пример 10.5), что говорит в пользу того, что ёмкость модели вознаграждения должна быть как можно больше в пределах бюджета.

14.3.2 Устойчивость обучения и переобучение

Модели вознаграждения склонны к переобучению, поскольку датасеты предпочтений относительно малы (от десятков тысяч до низких миллионов пар) по сравнению с корпусами предобучения (триллионы токенов). Стандартные меры противодействия включают:

- **Низкие скорости обучения.** Типичные значения — от 1×10^{-6} до 5×10^{-6}, что на порядок ниже скоростей обучения для SFT.
- **Ранняя остановка.** Мониторировать попарную точность на отложенном наборе и останавливаться, когда она начинает снижаться.
- **Сглаживание меток.** Заменить жёсткие метки Bradley–Terry на $1 - \epsilon$ и ϵ для некоторого малого ϵ (обычно 0.1), как в формулировке cDPO (Уравнение (10.10)).
- **Дропаут.** Применить дропаут ($p = 0.1$) к последним слоям модели вознаграждения.

14.3.3 Калибровка и метрики оценки

Модель вознаграждения может иметь высокую попарную точность и при этом давать плохо откалиброванные скалярные выходы. Ключевые метрики для мониторинга:

- **Попарная точность** (Уравнение (10.11)): доля отложенных пар предпочтений, где $r_\psi(x, y_w) > r_\psi(x, y_l)$. Значения 70–75% типичны для выравнивания общего назначения; 80%+ указывает на сильную модель вознаграждения.
- **Распределение вознаграждений.** Построить распределение баллов r_ψ по большому набору ответов. Распределение должно быть унимодальным и примерно центрированным; бимодальное распределение говорит о том, что модель вознаграждения выучила ложный бинарный признак (например, длину ответа).
- **Согласие с человеческими ранжированиями.** На небольшом наборе промптов с четырьмя или более ответами, ранжированными экспертами, вычислить τ Кенделла между ранжированием модели вознаграждения и ранжированием экспертов.

14.3.4 Когда переобучать

Модель вознаграждения должна быть переобучена, когда стратегия достаточно далеко отдрейфовала от распределения, на котором была обучена модель вознаграждения. Практическое эмпирическое правило: мониторировать дивергенцию KL между текущей стратегией и опорной. Когда $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ превышает пороговое значение (обычно 5–15 натов в зависимости от размера модели), собрать новые данные предпочтений от текущей стратегии и переобучить модель вознаграждения. Это итеративный цикл RLHF, описанный в Разделе 14.1.

14.4 PPO для языковых моделей: детали реализации

Применение PPO (Глава 9) к выравниванию языковых моделей — не тривиальная задача. Структура MDP на уровне токенов при генерации языка (Определение 11.3), огромное количество параметров и необходимость вычислять вероятности как под стратегией, так и под опорной моделью создают уникальный набор инженерных проблем. В этом разделе описана стандартная реализация.

14.4.1 Четырёхмодельная схема

Цикл обучения RLHF на основе PPO одновременно поддерживает четыре нейронные сети:

1. **Модель стратегии** π_θ: языковая модель, которая обучается. Генерирует ответы и вычисляет логарифмические вероятности.
2. **Опорная модель** π_{ref}: замороженная копия контрольной точки SFT. Используется для вычисления штрафа KL.
3. **Модель вознаграждения** r_ψ: замороженная модель вознаграждения, обученная на данных предпочтений. Оценивает завершённые ответы.

4. **Модель ценности** V_ϕ: сеть-критик, оценивающая ожидаемое будущее вознаграждение с каждой позиции токена. Используется для оценки преимущества.

При масштабе 7B параметров каждая модель требует приблизительно 14 ГБ в половинной точности (FP16/BF16). Хранение всех четырёх в памяти GPU одновременно требует не менее 56 ГБ — осуществимо на одной A100 (80 ГБ) при тщательном управлении памятью, но довольно плотно с учётом активаций и KV-кэша во время генерации. При масштабе 70B четыре модели требуют около 560 ГБ, что обусловливает необходимость multi-GPU setups (Раздел 14.6).

14.4.2 Реализация штрафа KL

Штраф KL не позволяет стратегии слишком далеко отклоняться от опорной. Применяются две стратегии реализации.

Вознаграждение KL на уровне токенов. На каждой позиции токена t вычитаем штраф KL из вознаграждения:

$$\tilde{r}_t = \begin{cases} -\beta \log \dfrac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}, & t < T, \\ r_\psi(x, y) - \beta \log \dfrac{\pi_\theta(y_T \mid x, y_{<T})}{\pi_{\text{ref}}(y_T \mid x, y_{<T})}, & t = T. \end{cases} \tag{14.1}$$

Терминальное вознаграждение при $t = T$ включает как оценку модели вознаграждения, так и штраф KL за последний токен. Эта формулировка распределяет сигнал KL по всем позициям токенов, что улучшает присвоение кредита. Она эквивалентна Уравнению (10.15) из Главы 10.

Штраф KL на уровне последовательности. Добавить суммарную дивергенцию KL уровня последовательности как единственный штраф в конце последовательности:

$$\tilde{r} = r_\psi(x, y) - \beta \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}.$$

Это проще в реализации, но концентрирует весь сигнал KL на последнем токене, делая оценку преимущества менее информативной для промежуточных токенов.

На практике предпочтительна формулировка на уровне токенов.

14.4.3 Оценка преимущества с помощью GAE

Учитывая модифицированные вознаграждения на уровне токенов $\{\tilde{r}_t\}$ и оценки ценности $\{V_\phi(s_t)\}$, обобщённая оценка преимущества (GAE) из Главы 9 равна

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \delta_t = \tilde{r}_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \tag{14.2}$$

где γ — коэффициент дисконтирования, а λ — параметр GAE. Для RLHF языковых моделей стандартный выбор — $\gamma = 1$ (без дисконтирования внутри ответа) и $\lambda \in [0.95, 1.0]$. Установка $\gamma = 1$ естественна, поскольку “эпизод” — это один ответ: нет причин дисконтировать будущие токены внутри предложения.

14.4.4 Формирование мини-батча и накопление градиентов

Одна итерация PPO выполняется следующим образом:

1. **Фаза сбора роллаутов.** Выбрать батч промптов из $\mathcal{D}$. Сгенерировать ответы с помощью π_θ. Оценить каждый ответ с помощью r_ψ и вычислить логарифмические вероятности на уровне токенов как под π_θ, так и под π_{ref}.

2. **Вычисление преимущества.** Вычислить преимущества GAE с помощью модели ценности V_ϕ.

3. **Обновление PPO.** В течение K эпох (обычно $K = 2$–4) перемешать данные роллаутов в мини-батчи и выполнить обновления градиента по отсечённой целевой функции PPO:

$$\mathcal{L}_t^{\text{clip}}(\theta) = -\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big), \tag{14.3}$$

   где $r_t(\theta) = \pi_\theta(y_t|x, y_{<t})/\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})$ — отношение важности, а ϵ — параметр отсечения.

4. **Обновление ценности.** Обновить V_ϕ, минимизируя среднеквадратичную ошибку относительно целевых значений GAE-доходности.

Когда памяти GPU недостаточно для полного мини-батча, используется *накопление градиентов*: прямые и обратные проходы вычисляются на микро-батчах, а градиенты суммируются перед шагом оптимизатора. Это математически эквивалентно большему размеру батча, но требует только памяти для одного микро-батча.

14.4.5 Обучение со смешанной точностью

Современные реализации RLHF используют BF16 (bfloat16) для прямых проходов и FP32 для накопления градиентов и состояний оптимизатора. BF16 вдвое сокращает объём памяти по сравнению с FP32, сохраняя при этом динамический диапазон, необходимый для устойчивого обучения. Модель ценности иногда хранится в FP32 во избежание числовых проблем при вычислении преимущества, особенно когда вознаграждения охватывают широкий диапазон.

14.4.6 Гиперпараметры

Таблица 14.1 перечисляет типичные диапазоны гиперпараметров для RLHF на основе PPO для языковых моделей различных размеров.

Гиперпараметр	**Типичный диапазон**	**Примечания**
Скорость обучения (стратегия)	$5 \times 10^{-7} - 2 \times 10^{-6}$	Косинусный расписание, прогрев 3–10%
Скорость обучения (ценность)	$1 \times 10^{-6} - 5 \times 10^{-6}$	Часто в 2–5× больше LR стратегии
Коэффициент KL β	0.01 – 0.2	Начинать с малого, увеличивать при взрыве KL
Параметр отсечения ϵ	0.1 – 0.2	0.2 наиболее распространён
GAE λ	0.95 – 1.0	1.0 типично для LM
Дисконт γ	1.0	Без дисконтирования внутри ответа
Эпохи PPO на батч K	2 – 4	Больше эпох рискует переобучением
Размер батча (промпты)	64 – 512	Больше — стабильнее
Размер мини-батча	8 – 64	Ограничен памятью GPU
Максимальная длина ответа	512 – 2048 токенов	Зависит от задачи

Таблица 14.1: Типичные гиперпараметры PPO для RLHF на языковых моделях.

14.5 Отладка RLHF

RLHF печально известна сложностью отладки. Тренировочные прогоны дороги, режимы отказа едва заметны, а взаимодействие четырёх нейронных сетей создаёт большое пространство возможных проблем. В этом разделе каталогизированы наиболее распространённые сбои и их диагностика.

14.5.1 Взлом вознаграждения

Симптом: Оценка модели вознаграждения быстро растёт, но качество генерируемого текста — по мнению людей — деградирует. Типичные проявления включают чрезмерно многословные ответы, повторяющиеся похвалы или формулировки-оговорки, и ответы, эксплуатирующие причуды форматирования.

Диагностика: Строить в процессе обучения как прокси-вознаграждение (оценка модели вознаграждения), так и эталонную метрику (например, процент побед над baseline по мнению людей). Взлом вознаграждения порождает расхождение: прокси-вознаграждение растёт, а эталонная метрика выходит на плато или падает.

Меры устранения:

- Увеличить коэффициент KL β. Это первый рубеж обороны.
- Переобучить модель вознаграждения на данных, сгенерированных текущей стратегией.
- Ансамблировать несколько моделей вознаграждения и использовать минимальную или медианную оценку.
- Добавить штраф за длину к вознаграждению для противодействия взлому путём многословия.

14.5.2 Взрыв дивергенции KL

Симптом: Дивергенция KL $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ быстро растёт (например, превышает 20 натов за несколько сотен шагов), и обучение становится нестабильным.

Диагностика: Мониторировать KL на уровне токенов на каждой позиции. Если несколько позиций токенов имеют чрезвычайно высокий KL, возможно, стратегия научилась всегда производить определённый токен на этих позициях.

Меры устранения:

- Снизить скорость обучения.
- Увеличить β или перейти на адаптивный штраф KL, который динамически корректирует β для поддержания целевого KL.
- Уменьшить количество эпох PPO на батч (K).

14.5.3 Коллапс моды

Симптом: Стратегия генерирует почти одинаковые ответы на разные промпты. Метрики разнообразия (например, distinct n-gram, self-BLEU) резко падают.

Диагностика: Выборочно сгенерировать 100 ответов стратегией на разнообразные промпты и вычислить попарные баллы BLEU. Средний попарный BLEU выше 0.5 — сильный индикатор коллапса.

Меры устранения:

- Увеличить β, чтобы стратегия оставалась ближе к разнообразной опорной.
- Увеличить температуру сэмплирования во время роллаутов.
- Добавить бонус энтропии к целевой функции PPO.

14.5.4 Нестабильность обучения

Симптом: Потеря стратегии дико колеблется, или оценка вознаграждения немонотонна с большими скачками между батчами.

Меры устранения:

- Снизить скорость обучения и увеличить размер батча.
- Убедиться, что оценки преимущества нормализованы (нулевое среднее, единичная дисперсия) внутри каждого мини-батча.
- Проверить, не отстаёт ли модель ценности от стратегии; при необходимости обновлять её чаще.
- Проверить на числовые проблемы в BF16: переключить голову модели ценности на FP32.

14.5.5 Диагностические метрики для мониторинга

Таблица 14.2 суммирует ключевые метрики для логирования во время обучения RLHF.

Метрика	**Здоровый диапазон**	**Тревожный сигнал**
Среднее вознаграждение	Медленно растёт	Резкий скачок (взлом)
Дивергенция KL	1–15 натов	> 20 натов
Энтропия стратегии	Стабильна или медленно убывает	Внезапное падение (коллапс)
Доля отсечений	0.05–0.15	> 0.3 (обновления слишком велики)
Потеря ценности	Убывает	Растёт (сбой модели ценности)
Средняя длина ответа	Стабильна	Монотонный рост (взлом многословием)
Прибл. норма градиента стратегии	Стабильна	Скачки (нестабильность)

Таблица 14.2: Диагностические метрики для обучения RLHF. «Тревожный сигнал» означает значение, требующее расследования.

14.6 Масштабирование RLHF

Обучение RLHF при масштабе 70B+ параметров требует распределения вычислений по множеству GPU. Главная проблема состоит в том, что цикл обучения RLHF имеет две различные фазы — генерацию и обучение — с очень разными вычислительными профилями.

14.6.1 Генерация против обучения

Фаза генерации (сбор роллаутов) является авторегрессионной: каждый токен зависит от всех предыдущих, что ограничивает параллелизм внутри одной последовательности. Основные оси параллелизма — по последовательностям (параллелизм данных) и по слоям трансформера (тензорный параллелизм). Генерация ограничена памятью из-за KV-кэша, который растёт линейно с длиной последовательности.

Фаза обучения (обновления PPO) является стандартным прямым-обратным проходом по последовательностям фиксированной длины. Она ограничена вычислениями и хорошо параллелизуется по батч-измерению (параллелизм данных) и по измерению модели (тензорный параллелизм или пайплайный параллелизм).

14.6.2 Стратегии параллелизма

Параллелизм данных Реплицировать модель на каждый GPU; каждый GPU обрабатывает разное подмножество батча. Градиенты синхронизируются с помощью AllReduce. Работает как для генерации, так и для обучения, но требует хранения полной копии модели на каждом GPU.

Тензорный параллелизм Разбить отдельные матрицы весов по GPU внутри узла. Снижает потребление памяти на GPU, но требует высокоскоростных межсоединений (NVLink). Необходим для моделей, не умещающихся на одном GPU.

Пайплайный параллелизм Разделить модель на стадии (группы слоёв), каждая из которых назначается отдельному GPU. Полезен для обучения в сочетании с планированием микро-батчей (например, 1F1B) для сокращения времени простоя.

Асинхронная генерация роллаутов Разъединить фазы генерации и обучения: пул GPU генерирует роллауты асинхронно, пока другой пул выполняет обновления PPO на завершённых роллаутах. Этот подход используется в инфраструктуре RLHF OpenAI и в открытых фреймворках, таких как DeepSpeed-Chat и OpenRLHF.

14.6.3 Оценки вычислительного бюджета

Таблица 14.3 содержит приблизительные оценки вычислений для одного раунда обучения RLHF при различных масштабах моделей, предполагая 50 000 промптов и 4 эпохи PPO.

Размер модели	GPU (A100 80 ГБ)	Время (прибл.)	Стоимость (прибл.)
7B	8	12–24 часа	$500–1 000
13B	16	1–2 дня	$2 000–4 000
70B	64	3–5 дней	$20 000–40 000

Таблица 14.3: Приблизительный вычислительный бюджет для одного раунда обучения RLHF. Затраты основаны на облачной цене GPU $2–3/GPU-час.

Эти оценки весьма вариативны и зависят от длины ответа, размера батча, числа эпох PPO и эффективности фреймворка распределённого обучения. Тем не менее они иллюстрируют ключевой момент: обучение RLHF дорого, но составляет малую долю стоимости предобучения. Типичный прогон предобучения для 70B-модели потребляет миллионы GPU-часов; один раунд RLHF — тысячи.

14.7 Ограничения безопасности и элайнмента

Максимизация вознаграждения необходима, но недостаточна для безопасного развёртывания. В этом разделе обсуждаются дополнительные ограничения, которые должны быть наложены поверх основного цикла обучения RLHF.

14.7.1 Красная команда

Красная команда — это систематический процесс зондирования модели на предмет вредных выводов. Красная команда генерирует состязательные промпты, призванные вызвать токсичные, предвзятые или опасные ответы. Полученные сбои используются для формирования целевых данных предпочтений (где вредный ответ помечается как отвергнутый), которые затем включаются в следующий раунд обучения модели вознаграждения и стратегии.

Работа красной команды наиболее эффективна, когда она *итеративна*: красная команда атакует последний контрольный пункт, сбои устраняются, и красная команда атакует снова. Ganguli et al. (2022) обнаружили, что работа красной команды с языковой моделью в течение 3–4 раундов снизила долю успешных атак на 75% по сравнению с одним раундом.

14.7.2 Интеграция Constitutional AI

Constitutional AI (Bai et al., 2022) дополняет RLHF набором явных принципов («конституцией»), которым должна следовать модель. В стандартной реализации языковая модель критикует собственные выводы относительно конституции и генерирует исправленные ответы. Эти исправленные ответы используются как предпочтительный ответ в паре предпочтений, создавая синтетические данные предпочтений без разметки людьми. Этот подход можно встроить в пайплайн RLHF как дополнительный источник данных для обучения модели вознаграждения.

14.7.3 Полезность против безвредности

Устойчивое противоречие в выравнивании — компромисс между полезностью и безвредностью. Модель, агрессивно обученная на безвредности, может отказываться отвечать на безобидные вопросы, которые поверхностно напоминают вредные (гиперотказ). Напротив, модель, обученная главным образом на полезности, может выполнять вредные запросы.

Стандартный способ снижения противоречия — обучать отдельные модели вознаграждения для полезности и безвредности (или одну модель вознаграждения с отдельными головками) и комбинировать их во время RL-обучения:

$$r(x, y) = r_{\text{helpful}}(x, y) + \alpha\, r_{\text{harmless}}(x, y),$$

где α контролирует вес, придаваемый безвредности. Llama 2 использовала вариант этого подхода, при этом α возрастала в ходе обучения для акцентирования безопасности на более поздних стадиях.

14.7.4 Когда останавливать обучение

Не существует единого критерия для остановки обучения RLHF. На практике команды используют комбинацию:

- **Достижение плато в человеческой оценке.** Когда последовательные контрольные точки не показывают улучшения в человеческих оценках (например, рейтинги Эло в прямых сравнениях), дальнейшее обучение вряд ли поможет.
- **Исчерпание бюджета KL.** Некоторые команды устанавливают максимальный бюджет KL (например, 10 натов) и останавливают обучение при его достижении, полагая, что дальнейшее отклонение от опорной модели увеличивает риск переоптимизации.
- **Регрессия безопасности.** Если бенчмарки безопасности деградируют относительно предыдущей контрольной точки, обучение следует откатить.

14.8 Кейсы

В этом разделе кратко суммированы подходы к выравниванию четырёх влиятельных систем. Каждая представляет отдельную точку в пространстве проектных решений.

14.8.1 InstructGPT (Ouyang et al., 2022)

InstructGPT стал первой крупномасштабной демонстрацией RLHF для языковых моделей. Пайплайн следовал трёхэтапному рецепту: SFT на 13 000 демонстрациях, обучение модели вознаграждения на 33 000 пар предпочтений и дообучение PPO. Ключевые инженерные решения включали: использование 6B-модели вознаграждения для 175B-стратегии (экономия памяти в ущерб ёмкости модели вознаграждения);

выполнение 2 полных итераций пайплайна; использование коэффициента KL $\beta = 0.02$. Итоговая модель была предпочтительнее, чем 175B-базовая линия SFT, по мнению человеческих оценщиков в 71% случаев.

14.8.2 Llama 2 (Touvron et al., 2023)

Усилия по выравниванию Llama 2 включали несколько инноваций. Команда собрала 1,4 миллиона пар предпочтений за 5 итераций цикла RLHF, с новыми данными от последней стратегии на каждой итерации. Они обучили две отдельные модели вознаграждения — одну для полезности, другую для безопасности — и объединили их с коэффициентом безопасности, который возрастал в ходе обучения. Примечательной техникой был *rejection sampling* (отбор по отклонению): на каждой итерации команда генерировала K ответов на промпт и выбирала лучший по модели вознаграждения, создавая синтетические SFT-данные для инициализации стратегии следующей итерации. Эта комбинация «RLHF + rejection sampling» оказалась эффективнее каждой техники по отдельности.

14.8.3 Подход Anthropic (Bai et al., 2022)

Работа Anthropic по выравниванию для Claude ввела Constitutional AI и сделала акцент на измерении безвредности. Ключевые вклады: механизм конституционной самокритики, генерирующий данные предпочтений из собственных ревизий модели; обучение с комбинированным вознаграждением полезность-безвредность; и обширная работа красной команды с итеративным устранением. Anthropic также стала пионером в использовании online RLHF, где модель вознаграждения периодически переобучается на предпочтениях, собранных от последней стратегии, что решает проблему сдвига распределения, от которой страдают offline-методы, такие как DPO (Раздел 12.5).

14.8.4 Подход DeepSeek с GRPO (DeepSeek-AI, 2024)

DeepSeek избрала другой путь, заменив PPO на Group Relative Policy Optimisation (GRPO, Глава 13). Для задач с верифицируемыми вознаграждениями — математики, программирования и фактических вопросов — GRPO полностью устраняет модель вознаграждения, используя сигналы корректности (например, частоту прохождения юнит-тестов, символьную верификацию) в качестве вознаграждения. Для открытых задач DeepSeek объединила GRPO с моделью вознаграждения. Ключевое инженерное преимущество — устранение модели ценности: GRPO нормализует вознаграждения в группе ответов на один промпт, что даёт baseline без необходимости в отдельном критике. Это сокращает четырёхмодельную схему до двух моделей (стратегия и опорная), вдвое снижая потребность в памяти и упрощая инфраструктуру распределённого обучения.

14.9 Резюме

В этой главе было представлено практическое руководство по реализации RLHF в масштабе. Основные выводы:

1. Пайплайн RLHF состоит из шести этапов (сбор данных, SFT, обучение модели вознаграждения, RL-обучение, оценка, развёртывание), как правило, выполняемых итеративно.

2. Качество данных о предпочтениях важнее их количества. Тщательные инструкции по аннотированию, рандомизированный порядок представления и внимание к смещениям (по длине, позиции, подхалимству) обязательны.

3. Модели вознаграждения должны быть настолько большими, насколько позволяет вычислительный бюджет, обучаться с низкими скоростями обучения и мониторироваться на переобучение. Их необходимо переобучать, когда стратегия значительно дрейфует.

4. Реализация PPO для языковых моделей требует одновременного хранения четырёх моделей в памяти. Штрафы KL на уровне токенов, GAE с $\gamma = 1$ и тщательное управление смешанной точностью являются стандартными инженерными решениями.

5. Сбои RLHF — взлом вознаграждения, взрыв KL, коллапс моды — распространены и имеют хорошо известные диагностические признаки и способы устранения. Мониторинг метрик из Таблицы 14.2 обязателен.

6. Масштабирование до больших моделей требует сочетания параллелизма данных, тензорного и пайплайнного с асинхронной генерацией роллаутов.

7. Ограничения безопасности — красная команда, Constitutional AI и компромисс полезность-безвредность — должны быть интегрированы в цикл обучения, а не добавлены постфактум.

Кейсы иллюстрируют, что не существует единственного «правильного» способа реализовать RLHF. InstructGPT использовал классический пайплайн PPO. Llama 2 добавил rejection sampling и итеративный сбор данных. Anthropic ввела конституционную самокритику. DeepSeek заменил PPO на GRPO для верифицируемых задач. Каждый подход отражает различные компромиссы между инженерной сложностью, требованиями к данным и качеством выравнивания. Общая нить: успешное выравнивание требует не только правильного алгоритма, но и тщательного внимания к качеству данных, устойчивости обучения и оценке безопасности.

Упражнения

Упражнение 14.1. Команда обучает 7B-языковую модель с RLHF на основе PPO. После 500 шагов обучения дивергенция KL $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ достигла 18 натов, а средняя оценка модели вознаграждения выросла с 2.1 до 4.8.

1. Находится ли дивергенция KL в здоровом диапазоне? Обоснуйте ответ, используя Таблицу 14.2.
2. Команда замечает, что средняя длина ответа выросла с 120 токенов до 350 токенов. На какой режим отказа это указывает? Предложите два конкретных способа устранения.
3. Предложите адаптивную схему для коэффициента KL β, которая поддерживает целевой KL $d_{\mathrm{target}} = 6$ натов. Запишите правило обновления.

Упражнение 14.2. Рассмотрим четырёхмодельную схему PPO (Раздел 14.4.1) для модели с 13B параметров.

1. Оцените общую память GPU, необходимую для хранения всех четырёх моделей в BF16, без учёта активаций и KV-кэша.
2. Команда хочет обучать на 8× A100 (по 80 ГБ). Достаточно ли параллелизма данных, или требуется тензорный параллелизм? Обоснуйте ответ.
3. GRPO устраняет модель ценности. Сколько памяти это экономит и как меняются требования к параллелизму?

Упражнение 14.3. Разработайте эксперимент для обнаружения взлома вознаграждения. Дана модель вознаграждения r_ψ и доступ к пулу из 500 человеческих аннотаторов:

1. Опишите протокол составления «эталонного» оценочного набора, которого модель вознаграждения никогда не видела.
2. Определите количественную метрику, позволяющую отличить подлинное улучшение от взлома вознаграждения. Точно опишите, что вы бы строили на графике и какой признак на графике указывает на взлом.
3. Команда замечает, что модель вознаграждения даёт оценку +5.2 ответу «С удовольствием помогу! Вот подробный, детальный и исчерпывающий ответ на ваш вопрос. . . » вне зависимости от того, что следует дальше. Что это за тип взлома вознаграждения и как вы бы исправили модель вознаграждения?

Упражнение 14.4. Вы разрабатываете пайплайн аннотирования для обучения модели вознаграждения для чат-бота поддержки клиентов.

1. Напишите инструкции по аннотированию (3–5 предложений) для попарного сравнения, учитывающие смещение по длине и подхалимство.
2. Оцените стоимость сбора 50 000 пар предпочтений с помощью экспертных аннотаторов по $10 за сравнение. Если у вас бюджет $200 000, какие альтернативные стратегии сбора данных вы могли бы использовать для дополнения экспертно-аннотированных данных?
3. Как вы измерите межаннотаторское согласие на вашем датасете? Каков минимальный уровень согласия, который вы требуете перед использованием данных для обучения модели вознаграждения?

Упражнение 14.5. Рассмотрим ответ длиной $T = 4$ токена с вознаграждениями на уровне токенов $\tilde{r}_1 = 0.1$, $\tilde{r}_2 = -0.2$, $\tilde{r}_3 = 0.0$, $\tilde{r}_4 = 1.5$ и оценками ценности $V_1 = 0.8$, $V_2 = 0.6$, $V_3 = 0.9$, $V_4 = 1.0$, $V_5 = 0$ (терминальное состояние).

1. Вычислите TD-невязки $\delta_1, \ldots, \delta_4$ при $\gamma = 1$.
2. Вычислите преимущества GAE $\hat{A}_1, \ldots, \hat{A}_4$ при $\lambda = 0.95$.
3. Повторите часть (б) при $\lambda = 1.0$ и сравните. Какая настройка даёт оценки преимущества с большей дисперсией и почему?

Упражнение 14.6. Исследовательская лаборатория хочет запустить RLHF на 70B-параметрической модели с помощью PPO. У неё есть кластер из 64 GPU A100 (80 ГБ каждый), соединённых NVLink 400 Гбит/с внутри узлов из 8 GPU и InfiniBand 100 Гбит/с между узлами.

1. Предложите стратегию параллелизма для фазы генерации и для фазы обучения. Укажите степень тензорного параллелизма, параллелизма данных и пайплайнного параллелизма для каждой фазы.
2. Фаза генерации является авторегрессионной и ограниченной памятью. Объясните, почему тензорный параллелизм помогает, несмотря на накладные расходы на коммуникацию.
3. Команда рассматривает асинхронную генерацию роллаутов. Опишите потенциальную проблему устаревания: стратегия, использованная для генерации, может отличаться от той, что обновляется. При каких условиях это устаревание приемлемо?

Упражнение 14.7. Сравните подходы InstructGPT, Llama 2 и DeepSeek к выравниванию по следующим измерениям:

1. Количество моделей, требуемых в памяти GPU во время RL-обучения.
2. Тип и количество использованных человеческих данных предпочтений.
3. Стратегия для решения проблемы устаревания модели вознаграждения (сдвига распределения).
4. Пригодность для задач с верифицируемыми вознаграждениями (например, математика).

Представьте своё сравнение в таблице и напишите краткий абзац с рекомендацией, какой подход должен принять стартап с ограниченным вычислительным бюджетом для разработки ассистента общего назначения.

Часть IV

Фронтир

Глава 15

Рассуждения, инструменты и дообучение с подкреплением

Если интеллект — это торт, то основная его часть — это обучение без учителя, глазурь — это обучение с учителем, а вишенка на торте — обучение с подкреплением.

— Yann LeCun, NeurIPS 2016

На протяжении большей части истории, описанной в этой книге, обучение с подкреплением занимало любопытное место в пайплайне обучения языковых моделей: теоретически центральное, практически маргинальное. Сигнал вознаграждения, который управлял RLHF (Глава 14), сам по себе являлся обученной нейронной сетью, — несовершенным приближением для несовершенно понятой цели, — и каждый практик знал, что слишком сильное смещение стратегии в сторону такой модели вознаграждения приведёт к взлому вознаграждения, а не к подлинному улучшению. Результатом стал осторожный, жёстко ограниченный по KL режим обучения, в котором RL вносил лоск, но не мощь.

Эта картина резко изменилась между 2024 и 2025 годами. Катализатором послужило осознание того, что определённые области допускают *верифицируемые* вознаграждения: математический ответ либо верен, либо нет; фрагмент кода либо проходит набор тестов, либо нет. Когда вознаграждения можно вычислить с помощью детерминированной функции верификации, а не выводить из нейронной модели предпочтений, опасность взлома вознаграждения исчезает, и становится безопасным применять значительно более агрессивное RL на значительно более длительный срок. Результат — *Обучение с подкреплением с верифицируемыми вознаграждениями* (RLVR) — произвёл качественный скачок в способности к рассуждению: модели теперь тратят тысячи токенов на обдумывание перед ответом на сложный вопрос, могут писать и отлаживать сложные программы и достигают экспертного уровня на олимпиадной математике.

Эта глава организована для постепенного формирования понимания. Мы начинаем с формализации цели RLVR и сравнения её с целью RLHF из Части III (Раздел 15.1). Затем мы рассматриваем рассуждения типа chain-of-thought как технику формирования промптов и как обучаемое поведение (Раздел 15.2). Раздел 15.3 изучает рецепты обучения, используемые на практике, с DeepSeek R1 в качестве подробного разбора. Раздел 15.4 формализует масштабирование на этапе вывода: примечательное явление, при котором генерация большего числа токенов на этапе вывода приводит к измеримому росту точности. Разделы 15.5 и 15.6 расширяют фреймворк до *использования инструментов* и *агентного RL*, где модель должна взаимодействовать с внешними средами в течение нескольких шагов. Раздел 15.7 обозревает практические алгоритмические и инфраструктурные решения, а Раздел 15.8 завершает открытыми вопросами.

15.1 От RLHF к обучению с подкреплением с верифицируемыми вознаграждениями

15.1.1 Идея верификации

Напомним из Главы 14, что RLHF максимизирует

$$J_{\mathrm{RLHF}}(\theta) = \mathbb{E}_{\pi_\theta}\big[r_\phi(x, y)\big] - \beta\, D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})\,, \tag{15.1}$$

где r_ϕ — это *обученная* модель вознаграждения, обученная на парах предпочтений людей, а $\beta > 0$ — коэффициент штрафа KL, который предотвращает слишком сильное отклонение стратегии от референсной

модели. Обученная модель вознаграждения порождает две взаимоусиливающиеся проблемы. Во-первых, это приближение: даже хорошо обученная модель вознаграждения будет присваивать неверные оценки некоторой доле ответов, и достаточно сильная стратегия может обнаружить и использовать эти ошибки — явление, называемое *сверхоптимизацией вознаграждения* или законом Гудхарта в RL (Глава 10). Во-вторых, обучение модели вознаграждения требует дорогостоящей разметки людьми пар предпочтений, и полученная модель наследует все предвзятости и несогласованности этих разметок.

Обучение с подкреплением с верифицируемыми вознаграждениями (RLVR) обходит обе проблемы, заменяя обученную модель вознаграждения детерминированной **функцией верификации**:

$$r(x, y) = \begin{cases} 1 & \text{если } \textsc{Verify}(x, y) = \textsc{True}, \\ 0 & \text{иначе.} \end{cases} \tag{15.2}$$

Функция Verify является предметно-специфической и полностью символической: для математики она извлекает окончательный ответ из ответа модели (используя регулярное выражение для нахождения содержимого в команде `\boxed{}` или после разделителя, такого как `The answer is:`) и сравнивает его с правильным решением; для кода она выполняет ответ на наборе тестов и возвращает True, если все проверки проходят.

Определение 15.1 (Цель RLVR)**.** Пусть $\mathcal{D}$ — это набор данных пар (промпт, правильный ответ) (x, a^*). Цель RLVR:

$$J_{\text{RLVR}}(\theta) = \mathbb{E}_{(x,a^*)\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[\mathbb{1}\{\textsc{Verify}(y, a^*)\}\big] - \beta\, D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})\,, \tag{15.3}$$

где $\mathbb{1}\{\cdot\}$ — индикаторная функция, а член KL может быть уменьшен или полностью исключён (см. Раздел 15.3).

Поскольку r принимает значения в $\{0, 1\}$ и вычисляется детерминированной символической функцией, она не может быть сверхоптимизирована так же, как нейронная модель вознаграждения. Модель, научившаяся давать верные ответы, *действительно* стала лучше; нет прокси-разрыва для использования.

15.1.2 Верифицируемые области

Мощь RLVR зависит от наличия богатого и разнообразного набора задач с эффективно верифицируемыми ответами. В настоящее время доминируют три области.

Математика Олимпиадные задачи из наборов данных, таких как MATH и AIME, допускают единственные числовые или символические ответы, которые можно проверить точно. Модели предлагается заключить окончательный ответ в команду `\boxed{}`, которую регулярное выражение извлекает и сравнивает с эталоном.

Генерация кода Задачи с LeetCode, HumanEval или MBPP поставляются с модульными тестами. Решение модели выполняется в изолированной среде, и вознаграждение равно 1, если все тесты проходят (или доле прошедших тестов для вариантов с частичным зачётом).

Наука и логика Формальное доказательство теорем (например, в Lean 4) и структурированное научное вопросно-ответное взаимодействие с уникальными фактическими ответами являются естественными расширениями. Любая область, где можно написать проверочную функцию, в принципе доступна для RLVR.

15.1.3 Формальное сравнение с целью RLHF

Таблица 15.1 сравнивает два подхода по параметрам, наиболее значимым для практиков.

Ключевое структурное отличие состоит в том, что RLVR преобразует задачу RL в такую, где вознаграждение *не является* нейронной сетью со своими градиентами, а является оракулом только для чтения. Это делает ландшафт оптимизации чище: нет внутреннего цикла обучения модели вознаграждения, который нужно чередовать с обновлениями стратегии, нет проблемы сдвига распределения для модели вознаграждения и нет риска того, что стратегия «найдёт пробелы» в обученном приближении.

Таблица 15.1: RLHF против RLVR: формальное сравнение.

Свойство	**RLHF**	**RLVR**
Источник вознаграждения	Обученная нейронная r_ϕ	Символическая Verify
Диапазон вознаграждения	$\mathbb{R}$ (непрерывный)	$\{0,1\}$ (бинарный)
Разметка людьми	Требуется для r_ϕ	Требуется для пар (x, a^*)
Риск сверхоптимизации	Высокий	Пренебрежимо мал
Необходимость штрафа KL	Критическая	Необязательна
Применимые области	Общие	Верифицируемые области
Стоимость модели вознаграждения	O(длина ответа)	O(проверочная функция)

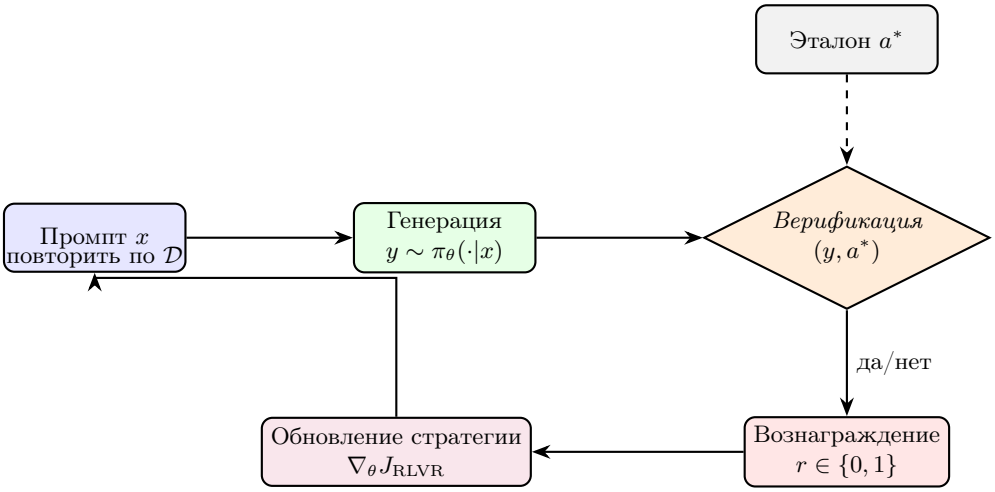


Рис. 15.1: Цикл обучения RLVR. Промпт x из набора данных подаётся текущей стратегии π_θ, которая генерирует ответ y. Детерминированная функция верификации сравнивает y с эталонным ответом a^* и возвращает бинарное вознаграждение. Затем вычисляются градиенты стратегии, и цикл повторяется. Обученная модель вознаграждения не нужна.

15.2 Рассуждения типа chain-of-thought

15.2.1 Основная идея

Стандартная авторегрессионная генерация отображает промпт x непосредственно в ответ y. Генерация типа chain-of-thought (CoT) вставляет *след рассуждений* z между промптом и финальным ответом:

$$x \longrightarrow z \longrightarrow a,$$

где z — последовательность промежуточных шагов рассуждения, а a — финальный ответ, извлечённый из конца генерации. Полная генерация $y = (z, a)$ производится той же авторегрессионной моделью; z просто не показывается пользователю во многих развёртываниях.

Пример 15.1 (Chain-of-thought на олимпиадной задаче)**.** Рассмотрим промпт: *«Найдите все целочисленные решения $x^2 - 5x + 6 = 0$.»* Без chain-of-thought модель может выдать правильный ответ $\{2, 3\}$ напрямую (если запомнила его) или тихо провалиться. С chain-of-thought генерация выглядит так:

```
<thinking>
Мне нужно разложить x^2 − 5x + 6 на множители. Ищу два числа,
произведение которых равно 6, а сумма -- −5. Это −2 и −3.
Значит, x^2 − 5x + 6 = (x − 2)(x − 3). Приравниваем каждый множитель к нулю:
x − 2 = 0 ⇒ x = 2; x − 3 = 0 ⇒ x = 3.
</thinking>
Целочисленные решения: x = 2 и x = 3, т.е. [2, 3].
```

Токены рассуждений не видны конечным пользователям, но используются в процессе обучения: функция верификации извлекает содержимое команды `\boxed{}` для вычисления вознаграждения.

15.2.2 Почему более длинное рассуждение помогает

Chain-of-thought работает по информационно-теоретической причине. Вероятность того, что модель даст правильный финальный ответ a^* при промпте x, может быть записана по правилу цепочки вероятностей как

$$\mathbb{P}(a^* \mid x) = \sum_z \mathbb{P}(a^* \mid x, z)\, \mathbb{P}(z \mid x). \tag{15.4}$$

Маргинализация по промежуточным следам рассуждений z эквивалентна их использованию в качестве скрытой переменной. Модель, способная генерировать широкое разнообразие высококачественных следов рассуждений, эффективно выполняет неявный поиск по путям решений, прежде чем остановиться на ответе. Чем длиннее и разнообразнее рассуждение, тем более детальным становится этот неявный поиск.

Эта связь с *поиском* не просто метафорична. Её можно формализовать, определив качество следа рассуждений z для промпта x и ответа a^* как

$$q(z; x, a^*) = \mathbb{P}(a^* \mid x, z).$$

Эффективная точность модели равна $\mathbb{E}_z[q(z; x, a^*)]$, где математическое ожидание берётся по распределению модели над следами. Более длинный контекстный буфер допускает больше информации в z, что повышает потолок $q(z; x, a^*)$ для сложных задач. Эмпирически на олимпиадных математических тестах корреляция между средним количеством выходных токенов и точностью сильна и монотонна на нескольких порядках величины вычислений — масштабный закон на этапе вывода (Раздел 15.4).

15.2.3 Токены рассуждений и формат

Современные модели рассуждений формализуют chain-of-thought с помощью специальных *токенов рассуждений*. Генерация структурирована следующим образом:

$$y = \underbrace{\texttt{<thinking>}\ z_1 z_2 \cdots z_T\ \texttt{</thinking>}}_{\text{след рассуждений, не показывается пользователю}} \quad \underbrace{a_1 a_2 \cdots a_K}_{\text{финальный ответ}}\ . \tag{15.5}$$

Последовательность токенов $z_1, \ldots, z_T$ иногда называется *scratchpad*. Для простого арифметического вопроса T может составлять несколько десятков токенов; для сложной олимпиадной задачи T может достигать десятков тысяч токенов, что соответствует нескольким минутам генерации на современном оборудовании.

Замечание 15.1 (Длинный контекст как предпосылка). Обучение моделей рассуждений требует, чтобы стратегия генерировала и обрабатывала очень длинные последовательности. Практическим катализатором этого сдвига стала разработка эффективных механизмов внимания (например, FlashAttention) и контекстных окон размером от 128k до 1M токенов. Без этих инженерных достижений описанный в Разделе 15.3 режим обучения не был бы вычислительно осуществим.

15.2.4 Связь с вычислениями на этапе вывода

Расширенный формат генерации (15.5) напрямую связывает обучение с распределением вычислений на этапе вывода. Определим *токенный бюджет* B как максимальное число токенов, которые модель может сгенерировать до получения финального ответа. Для фиксированной стратегии π_θ точность на наборе задач является неубывающей функцией от B:

$$\mathrm{Acc}(B) = \mathbb{P}_{y \sim \pi_\theta(\cdot \mid x),\, |y| \le B}\big[\textsc{Verify}(y, a^*) = \textsc{True}\big].$$

Обучение с RLVR на неограниченной генерации учит модель *самостоятельно распределять* токенный бюджет: сложные задачи порождают более длинные следы, простые разрешаются быстро. Это адаптивное распределение является ключевым практическим преимуществом перед альтернативами с фиксированными вычислениями.

15.3 Обучение моделей рассуждений

15.3.1 Двухэтапный рецепт

Доминирующий рецепт обучения моделей рассуждений состоит из двух этапов, проиллюстрированных здесь на примере DeepSeek R1 как канонического открытого примера.

Этап 1: Холодный старт SFT Базовая предобученная модель сначала дообучается с помощью обучения с учителем на относительно небольшом наборе данных высококачественных демонстраций chain-of-thought. Этот этап обучает модель базовому формату — использованию тегов `<thinking>`, организации многошаговых рассуждений, — и гарантирует, что последующее RL будет иметь хорошо обусловленное начальное распределение для возмущений. Без этого холодного старта обучение RLVR от базовой модели может расходиться или производить искажённые следы рассуждений.

Этап 2: Крупномасштабный RLVR Модель, инициализированная через SFT, затем обучается с алгоритмом градиента стратегии (обычно GRPO или его вариантом; см. Раздел 15.7) на большом наборе данных верифицируемых задач. В отличие от RLHF, который обычно выполняется $O(1)$ эпох по данным предпочтений, RLVR возвращается к одним и тем же обучающим задачам сотни или тысячи раз, позволяя стратегии обнаружить и закрепить верные паттерны рассуждений, которые были лишь латентными в базовой модели.

Пример 15.2 (Рецепт обучения DeepSeek R1)**.** DeepSeek R1 (январь 2025) обучал базовую модель с приблизительно 600 миллиардами параметров на учебном плане, смешивавшем задачи математического рассуждения (преимущественно из наборов данных олимпиадной математики), задачи синтеза кода с верификацией модульными тестами и общие задачи следования инструкциям. Этап RL использовал GRPO с бинарными вознаграждениями от функции верификации в сочетании с небольшими вспомогательными наградами за формат для обеспечения последовательного использования структуры `<thinking>`. Впоследствии модель была дистиллирована до меньших размеров (7B–70B) путём дообучения на сгенерированных выходах полной модели, что сделало сильные возможности рассуждений доступными на потребительском оборудовании.

15.3.2 Фильтрация по сложности

Тонким, но критически важным аспектом обучения RLVR является выбор того, какие задачи включить в обучающий набор. Вспомним обновление градиента стратегии: градиент J_{RLVR} по θ ненулевой только тогда, когда разные развёртки для одного промпта получают разные вознаграждения. Если текущая стратегия решает задачу в 100% случаев, все развёртки получают вознаграждение 1, преимущество тождественно равно нулю, и градиент исчезает. Аналогично, если стратегия проваливается в 100% случаев, все развёртки получают вознаграждение 0 и не генерируется никакого градиента.

Определение 15.2 (Коэффициент решаемости задачи)**.** Для задачи x с эталоном a^* и стратегией π_θ **коэффициент решаемости** равен

$$\sigma(x) = \mathbb{P}_{y \sim \pi_\theta(\cdot|x)}\big[\textsc{Verify}(y, a^*) = \textsc{True}\big],$$

оцениваемый путём сэмплирования N независимых развёрток (обычно $N = 16$).

Эмпирически задачи с $\sigma(x) \in [0.2, 0.8]$ дают наиболее сильные градиенты. Задачи вне этого диапазона — слишком лёгкие или слишком сложные — исключаются из обучающего набора до начала RLVR. Этот шаг *offline difficulty filtering* сейчас является стандартным шагом в пайплайнах моделей рассуждений, используемым в DeepSeek R1, Phi-4 Reasoning, INTELLECT-2, Skywork OR-1 и других.

15.3.3 Удаление штрафа KL

Стандартный RLHF использует штраф KL из (15.1), чтобы предотвратить схлопывание стратегии к выходам с взломом вознаграждения. В RLVR бинарное вознаграждение нельзя взломать, повторяя фиксированный выход, поскольку функция верификации детерминирована и специфична для задачи. В результате многие практики RLVR уменьшают β до нуля, полностью удаляя ограничение KL. Это позволяет стратегии исследовать более свободно и обнаруживать паттерны рассуждений, отсутствовавшие в референсной модели. Модели, удалившие штраф KL, включают Magistral (Mistral, 2025), OpenReasoner-Zero и Skywork OR-1.

Замечание 15.2 (Удаление KL и сдвиг распределения)*.* Удаление штрафа KL действительно вводит риск: стратегия может сместиться к распределению, при котором сам верификатор ненадёжен (например, если модель научится извлекать ответы из позиций, которые регулярное выражение не соответствует). Тщательная разработка промптов и тестирование устойчивости верификатора поэтому важнее в режимах без KL, чем в стандартном RLHF.

15.3.4 Награды за формат и штрафы за длину

В дополнение к бинарному вознаграждению за задачу практики обычно добавляют небольшие вспомогательные вознаграждения:

- **Награды за формат**: Небольшое положительное вознаграждение (≈ 0.1) даётся, когда ответ корректно использует структуру `<thinking>` ... `</thinking>` перед финальным ответом. Это предотвращает смещение модели к формату, который трудно разобрать.
- **Штрафы за длину**: Следы рассуждений, более длинные, чем необходимо, потребляют лишние вычисления на этапе вывода без улучшения точности. Штраф, пропорциональный $\max(0, |z| - L_{\max})$, препятствует pathological overthinking при сохранении возможности модели использовать длинные следы когда это действительно необходимо. Некоторые рецепты (например, Kimi 1.5) используют прогрессивный учебный план по длине: модель сначала учится рассуждать в жёстком токенном бюджете, а бюджет расширяется по мере продвижения обучения.
- **Награды за языковую согласованность**: Многоязычные модели могут перейти к смешению языков в середине следа (например, начав на английском и завершив на китайском), что ухудшает пользовательский опыт. Бинарная награда за языковую согласованность используется в DeepSeek R1 и Magistral.

15.3.5 Нормализация функции потерь и смещение GRPO

Алгоритм GRPO (Глава 13) вычисляет преимущества путём нормализации вознаграждений внутри каждой группы развёрток для одного промпта. Тонким следствием является то, что вклад задачи в общую функцию потерь не зависит от числа развёрток, сгенерированных для неё. Если более сложные задачи систематически требуют больше токенов, нормализация на уровне группы может придавать избыточный вес лёгким коротким задачам и недостаточный вес сложным. Несколько рецептов моделей рассуждений (включая Magistral и MiMo) решают это путём нормализации преимуществ на уровне *батча*, а не группы, или взвешивания функции потерь по количеству токенов.

15.3.6 Самообучаемое рассуждение и дистилляция

Два важных варианта стандартного рецепта отходят от чистого RL.

STaR (Self-Taught Reasoner; Zeiler et al., 2022) и его потомок **Quiet-STaR** аппроксимируют градиент стратегии путём чередования: (1) сэмплирование следов рассуждений из текущей модели, (2) фильтрация до тех, которые приводят к правильным ответам, и (3) дообучение модели на отфильтрованных следах с функцией потерь кросс-энтропии. Это эквивалентно упрощённой форме RL, где вознаграждение бинарно, а baseline равен нулю. STaR аппроксимирует градиент (15.3) без явного вычисления преимущества, ценой некоторой статистической эффективности. Quiet-STaR добавляет инновацию генерации токенов рассуждений перед каждым предсказанием токена, а не только в начале ответа, что обеспечивает более богатый обучающий сигнал.

Дистилляция переносит способность к рассуждению от большой, обученной с RLVR модели к меньшей модели-ученику посредством обучения с учителем на выходах chain-of-thought выходах модели-учителя. Поскольку ученик учится на завершённых следах рассуждений учителя — включая токены рассуждений, — он может достичь производительности рассуждений намного выше той, которую прямое RLVR на самом ученике позволило бы достичь. DeepSeek R1 опубликовал дистиллированные контрольные точки при 7B и 70B, которые превзошли свои аналоги того же размера без дистилляции на олимпиадной математике.

15.4 Масштабирование на этапе вывода

15.4.1 Гипотеза о вычислениях на этапе вывода

Масштабирование на этапе вывода — это эмпирическое наблюдение, что для фиксированной обученной стратегии выделение большего объёма вычислений на этапе вывода — путём генерации большего числа токенов, большего числа ответов или и того, и другого, — приводит к измеримым улучшениям точности задач. Это отличается от масштабирования на этапе обучения (более крупные модели или больше обучающих данных): это масштабирование при развёртывании.

Пусть $\mathcal{P}$ обозначает набор тестовых задач, а C — бюджет вычислений, измеренный в FLOPs (или, эквивалентно, в суммарном числе сгенерированных выходных токенов). Закон масштабирования вывода

утверждает, что существует функция f такая, что

$$\mathrm{Acc}(C) \approx f(C) = f_0 + \alpha \log C + O((\log C)^{-1}), \tag{15.6}$$

для констант f_0 и $\alpha > 0$. Эмпирически это логарифмически-линейное соотношение справедливо на двух-трёх порядках величины вычислений на этапе вывода на олимпиадных математических тестах, прежде чем насыщается по мере исчерпания задач.

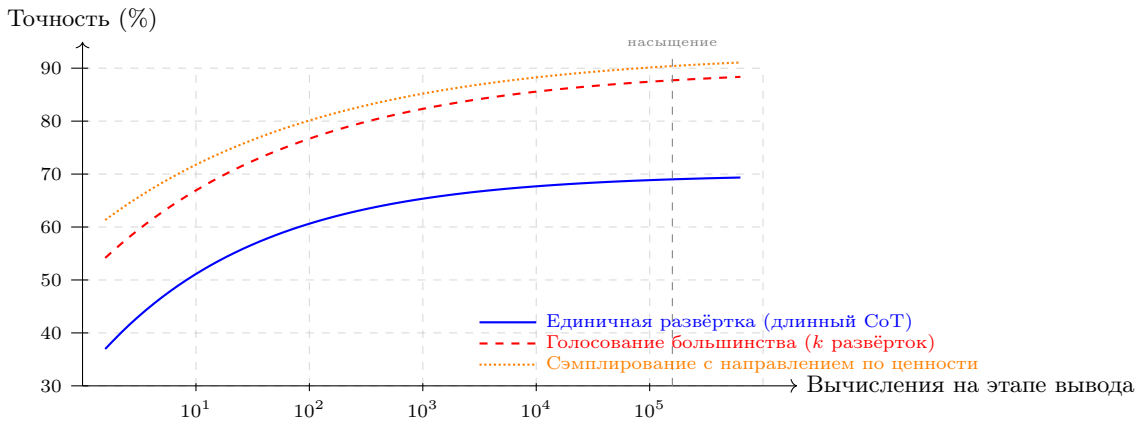


Рис. 15.2: Схематичные кривые масштабирования на этапе вывода для олимпиадной математики. Точность растёт логарифмически-линейно с вычислениями на этапе вывода для ряда стратегий. Голосование большинства по k параллельным развёрткам даёт более высокую точность при тех же вычислениях, чем единственная длинная развёртка; сэмплирование с направлением по ценности извлекает дополнительные улучшения, используя процессную модель вознаграждения для управления поиском по лучу. Все кривые насыщаются, как только достигается потолок возможностей модели.

15.4.2 Голосование большинства

Простейшей стратегией масштабирования на этапе вывода является *голосование большинства* (также называемое *самосогласованностью*): сэмплируем k независимых ответов $y_1, \ldots, y_k$ из стратегии, извлекаем финальный ответ a_i из каждого и возвращаем ответ с наибольшим числом голосов:

$$\hat{a} = \operatorname*{argmax}_a \sum_{i=1}^{k} \mathbb{1}\{a_i = a\}. \tag{15.7}$$

Если отдельные ответы верны с вероятностью $p > \frac{1}{2}$, голосование большинства даёт верный ответ с вероятностью, которая стремится к 1 экспоненциально быстро при росте k (следствие закона больших чисел). На практике $k = 8$–64 развёрток достаточно для большинства олимпиадных тестов.

Утверждение 15.1 (Граница точности голосования большинства)**.** *Предположим, что каждая из k независимых развёрток верна с вероятностью $p \in (\frac{1}{2}, 1)$ и существует не более m возможных ответов. Голосование большинства ошибается с вероятностью не более*

$$\mathbb{P}(\hat{a} \neq a^*) \leq \binom{k}{\lfloor k/2 \rfloor} p^{\lfloor k/2 \rfloor} (1-p)^{k - \lfloor k/2 \rfloor} \;\leq\; \exp\bigl(-k\, D_{\mathrm{KL}}(1/2 \,\|\, 1-p)\bigr),$$

где D_{KL} обозначает бинарную KL-дивергенцию. Ошибка убывает экспоненциально при росте k.

15.4.3 Сэмплирование с направлением по ценности

Голосование большинства считает все развёртки одинаково правдоподобными. Более сложная стратегия использует *процессную модель вознаграждения* (PRM), которая присваивает оценки *промежуточным* шагам рассуждения, а не только финальному ответу. Дан частичный след $z_{1:t}$, PRM предсказывает вероятность того, что рассуждение приведёт к верному финальному ответу. Эта оценка может направлять генерацию посредством поиска по лучу или выбора лучшего из k:

$$\hat{y} = \operatorname*{argmax}_{y \in \{y_1, \ldots, y_k\}} \mathrm{PRM}(y). \tag{15.8}$$

Сэмплирование с направлением по ценности последовательно превосходит голосование большинства при равных вычислениях, поскольку концентрирует бюджет сэмплирования на перспективных путях рассуждений, а не усредняет по всем им. Накладные расходы — стоимость запуска скорера PRM, который обычно значительно меньше модели стратегии.

15.4.4 Вычислительно-оптимальный вывод

Естественный вопрос — как разделить фиксированный бюджет вычислений C между длиной отдельных следов (больше токенов рассуждений на развёртку) и числом развёрток k. Определяя стоимость одной развёртки как $c(B) \propto B$ (где B — токенный бюджет на развёртку), общие вычисления составляют $C = k \cdot c(B)$. Вычислительно-оптимальная стратегия решает:

$$(k^*, B^*) = \underset{k,\, B:\, k \cdot B = C}{\operatorname{argmax}} \operatorname{Acc}(k, B),$$

где $\operatorname{Acc}(k, B)$ — точность при голосовании большинства с k развёртками длины B. Эмпирически этот компромисс отдаёт предпочтение более длинным отдельным следам для сложных задач (где B^* велик) и большему числу коротких развёрток для лёгких задач (где k^* велик).

15.5 Использование инструментов и вызов функций

15.5.1 Мотивация

Языковые модели, обученные исключительно на тексте, имеют фундаментальное ограничение: они не могут получать доступ к информации за пределами своих обучающих данных, выполнять код или взаимодействовать с внешними сервисами. Использование инструментов расширяет пространство действий модели от генерации токенов до *вызовов функций*: структурированных выходов, запрашивающих выполнение внешнего инструмента и встраивающих результат в последующую генерацию. Это качественное расширение парадигмы генерации:

$$x \longrightarrow (z_1, \underbrace{f_1(q_1)}_{\text{вызов инструмента}}, r_1, z_2, \underbrace{f_2(q_2)}_{\text{вызов инструмента}}, r_2, \ldots, a), \tag{15.9}$$

где f_i — инструмент (например, API веб-поиска или интерпретатор Python), q_i — запрос, который модель отправляет ему, r_i — возвращённый результат, а генерация модели чередует рассуждение (z_i) с вызовами инструментов и их результатами.

15.5.2 Формат вызова функций

На практике вызовы инструментов выражаются как структурированные объекты JSON, встроенные в поток генерации. Типичный формат:

Пример 15.3 (Вызов инструмента в формате JSON)**.** Рассмотрим модель, отвечающую на вопрос: *«Какова нынешняя численность населения Франции?»* Генерация может выглядеть следующим образом:

```
<thinking>
Мне нужно найти актуальную численность населения Франции.
</thinking>
{
"tool": "web_search
"query": "численность населения Франции 2025"
}
```

[Результат инструмента: «Население Франции: около 68,4 миллиона (2025)»]

```
Нынешняя численность населения Франции составляет около 68,4 миллиона человек.
```

Объект JSON анализируется средой выполнения, инструмент вызывается, а результат добавляется в контекст модели до продолжения генерации. Модель никогда не видит внутренние механизмы сырого API; она взаимодействует с чистым интерфейсом именованных инструментов с типизированными параметрами.

Модель должна одновременно освоить три возможности: (1) распознавать, когда нужен вызов инструмента, (2) генерировать корректный JSON, соответствующий схеме инструмента, и (3) правильно включать результат инструмента в последующее рассуждение.

15.5.3 Обучение использованию инструментов

Использование инструментов обычно обучается в два этапа:

SFT на демонстрациях Модель сначала дообучается на кураторском наборе данных траекторий использования инструментов: тройках (промпт, последовательность вызовов инструментов, финальный ответ), собранных из демонстраций людей или больших моделей. Это обучает модель формату и базовому решению о том, когда вызывать инструмент.

RL для многошагового использования инструментов Одного SFT недостаточно для сложных многошаговых сценариев использования инструментов, поскольку правильная последовательность вызовов инструментов зависит от результатов предыдущих вызовов способом, который трудно перечислить в фиксированном наборе данных демонстраций. RL позволяет модели обнаруживать эффективные стратегии вызова инструментов путём взаимодействия с живой средой инструментов и получения бинарного вознаграждения за выполнение задачи.

Сигнал вознаграждения для RL с использованием инструментов обычно основан на результате: модель получает вознаграждение 1, если финальный ответ (после всех вызовов инструментов) верен, и 0 иначе. Это в точности фреймворк RLVR из Раздела 15.1, теперь расширенный на многошаговые траектории. Средой является сама экосистема инструментов.

15.5.4 SWiRL: пошаговый RL для использования инструментов

SWiRL (Step-Wise Reinforcement Learning — пошаговое обучение с подкреплением) — это фреймворк, разработанный специально для многошаговых задач использования инструментов, где вознаграждения за результат разрежены. Вместо того чтобы ждать конца траектории для присвоения вознаграждения, SWiRL предоставляет *процессные вознаграждения* на каждом вызове инструмента. Пошаговое вознаграждение r_t вычисляется процессной моделью вознаграждения, которая оценивает, был ли t-й вызов инструмента полезным и корректно сформированным. Общее вознаграждение:

$$R = \sum_{t=1}^{T} \gamma^{t-1} r_t + \lambda \cdot r_{\text{outcome}}, \tag{15.10}$$

где $r_{\text{outcome}} \in \{0, 1\}$ — бинарное вознаграждение за выполнение задачи, а λ — гиперпараметр взвешивания. Пошаговые вознаграждения снижают дисперсию оценки градиента стратегии, обеспечивая более частую обратную связь, ценой необходимости в пошаговом оценщике.

15.6 Многошаговые агенты и агентное RL

15.6.1 Цикл агента

Когда использование инструментов охватывает множество шагов и модель должна планировать через длинный горизонт, мы вступаем в режим *агентного RL*. Модель теперь функционирует как полноценный агент RL, взаимодействующий со средой $\mathcal{E}$ (набором доступных инструментов и внешних систем). Стандартный цикл агента:

1. **Наблюдение**: Получить наблюдение o_t из среды (например, содержимое веб-страницы, вывод терминальной команды или ответ API).
2. **Рассуждение**: Сгенерировать след рассуждений z_t, обусловленный на полном контексте $(x, o_1, a_1, \ldots, o_{t-1}, a_{t-1}, o_t)$.
3. **Действие**: Вывести действие a_t, которым может быть вызов инструмента, навигационная команда или текстовый вывод.
4. **Наблюдение**: Получить o_{t+1} из среды. Перейти к 2.

Это в точности цикл агента MDP из Главы 3, теперь реализованный с языковой моделью в качестве стратегии и программной средой в качестве функции перехода.

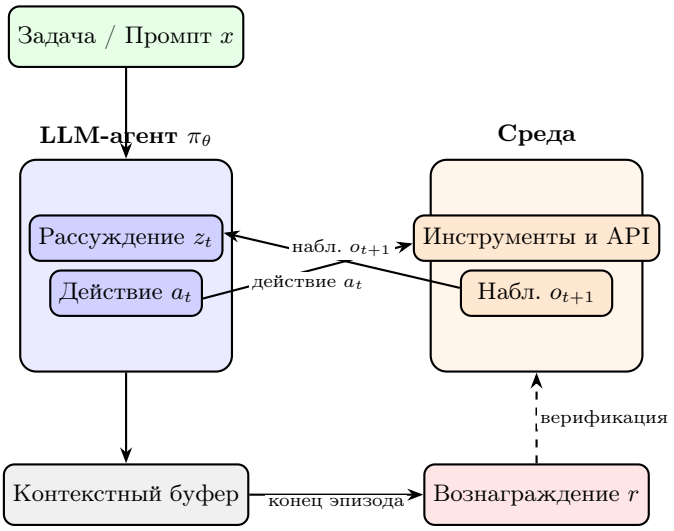


Рис. 15.3: Цикл агентного RL. LLM-агент π_θ наблюдает среду, генерирует след рассуждений и производит действие (вызов инструмента или текстовый вывод). Среда выполняет действие и возвращает наблюдение. Полный контекст накапливается в контекстном буфере модели. После завершения эпизода вычисляется сигнал вознаграждения (на основе результата или пошагового) и используется для обновления градиента стратегии.

15.6.2 Задачи и среды агентного RL

Агентное RL было применено к растущему числу задач:

Браузинг веба Агент навигирует по веб-браузеру (через Playwright или аналогичное), читает страницы, переходит по ссылкам и заполняет формы для выполнения задач поиска информации или транзакций. Вознаграждение бинарно: цель задачи достигнута или нет.

Выполнение кода Агент пишет код, выполняет его в изолированном интерпретаторе Python, читает вывод и итерирует. Сигнал вознаграждения через модульные тесты из Раздела 15.1 применяется напрямую.

Взаимодействие с API Агент вызывает REST API, читает ответы и цепочкой API-вызовов завершает многошаговые рабочие процессы (например, бронирование события в календаре после проверки доступности). Вознаграждение поступает от верификации финального состояния системы.

Кодирование на уровне репозитория Агент читает и пишет файлы в репозитории кода, запускает тесты и устраняет проблемы из отчётов об ошибках. SWE-bench — стандартный тест, измеряющий долю разрешённых агентом задач с GitHub.

15.6.3 Отнесение заслуг в многошаговых траекториях

Фундаментальный вызов агентного RL — отнесение заслуг: когда траектория из T шагов получает единственное конечное вознаграждение, каждое из T действий должно быть приписано соответствующим образом. Это проблема отложенного вознаграждения из классического RL (Глава 6), усиленная длиной траекторий языковых моделей.

Два подхода решают её:

- **Вознаграждения за результат**: Начислить каждому токену в траектории одинаковое конечное вознаграждение (возможно, дисконтированное). Это просто, но имеет высокую дисперсию: ранние действия, настраивающие успех, вознаграждаются наравне с единственным финальным токеном, подтверждающим успех.
- **Процессные вознаграждения**: Обучить PRM на уровне шагов для присвоения вознаграждений r_t на каждом шаге. Это значительно снижает дисперсию, но требует дополнительного надзора для обучения PRM. PRM можно обучать на разметке людьми того, какие промежуточные шаги верны, или (более

масштабируемо) с использованием развёрток Монте-Карло из каждого промежуточного состояния для оценки вероятности конечного успеха.

Компромисс между процессными и результирующими вознаграждениями (рассмотренный подробнее в Главе 10) особенно остёр в агентной обстановке, где траектории могут охватывать десятки вызовов инструментов и сотни тысяч токенов.

15.6.4 Исследование в агентном RL

Классические стратегии исследования в RL (epsilon-жадный, сэмплирование Томпсона, внутренняя мотивация; Глава 1) приобретают новый характер, когда стратегией является языковая модель. Априорное знание модели от предобучения уже кодирует богатое распределение над последовательностями действий, что обеспечивает неявное исследование. Однако это априорное знание может быть сосредоточено на безопасных, привычных последовательностях действий, которые избегают долгосрочных стратегий, необходимых для сложных агентных задач.

Практические подходы к агентному исследованию включают:

- **Масштабирование температуры**: Увеличение температуры генерации во время обучения разнообразит развёртки, аналогично увеличению epsilon в epsilon-жадном методе.
- **Учебный план по сложности задач**: Начало с коротких, лёгких задач и постепенное введение более длинных, сложных гарантирует, что модель получает адекватные сигналы обратной связи на протяжении всего обучения.
- **Рандомизация среды**: Изменение экземпляров задач (разные веб-страницы, разные кодовые репозитории, разные конечные точки API) предотвращает заучивание стратегией конкретных траекторий.

15.7 Практические соображения

15.7.1 Выбор алгоритма

Таблица 15.2 обобщает основные алгоритмы RL, используемые для обучения рассуждению и агентного обучения.

Таблица 15.2: Алгоритмы оптимизации стратегии для обучения моделей рассуждений.

Алгоритм	Механизм	Ключевое свойство	Используется в
PPO	Отсечённое приближение + сеть ценности	Устойчив, хорошо изучен	Tülu 3, Phi-4
GRPO	Групповая норм. преимуществ, без сети ценности	Эффективен по памяти	DeepSeek R1, Qwen 3
DAPO	Асимметричное отсечение	Better exploration	RAGEN, INTELLECT-2
CISPO	Отсечённое взвешивание важности	Off-policy stability	MiniMax-M1
MuonClip	Отсечение QK-нормы	Предотвращает скачки функции потерь	Kimi K2

PPO (Proximal Policy Optimisation — проксимальная оптимизация стратегии) остаётся наиболее широко изученным алгоритмом дообучения RL благодаря его теоретическим гарантиям и обширному эмпирическому опыту. Он требует поддержки отдельной сети ценности, которая удваивает потребление памяти, но снижает дисперсию градиента через обученный baseline.

GRPO (Group Relative Policy Optimisation — групповая относительная оптимизация стратегии) избегает сети ценности путём вычисления оценок преимущества на уровне промпта из эмпирического среднего и дисперсии вознаграждений внутри группы из k развёрток:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}, \qquad \bar{r} = \frac{1}{k}\sum_{j=1}^{k} r_j, \quad \sigma_r = \sqrt{\frac{1}{k}\sum_{j=1}^{k}(r_j - \bar{r})^2}. \tag{15.11}$$

При бинарных вознаграждениях RLVR, $r_i \in \{0, 1\}$, это упрощается до $\hat{A}_i = +\sigma_r^{-1}$ для правильных завершений и $\hat{A}_i = -\bar{r}/\sigma_r$ для неправильных. GRPO стал доминирующим в обучении моделей рассуждений, поскольку эффективен по памяти и прост в реализации.

DAPO (Decoupled Clip and Dynamic Sampling Policy Optimisation — оптимизация стратегии с разделённым отсечением и динамическим сэмплированием) модифицирует GRPO, используя асимметричные границы отсечения: большее отсечение на положительной стороне позволяет более агрессивные обновления в сторону высоковознаграждаемых ответов, тогда как отсечение на отрицательной стороне остаётся консервативным. Это способствует исследованию правильных стратегий рассуждения, которые текущая стратегия достигает лишь редко.

15.7.2 Онлайн- и офлайн-RL

Онлайн RL Развёртки генерируются из текущей стратегии π_θ и немедленно используются для обновления градиента. Это поддерживает обучение on-policy, что теоретически чисто, но практически медленно: стратегия должна ждать завершения генерации перед обновлением.

Офлайн RL Развёртки предварительно генерируются (из более ранней контрольной точки стратегии или из другой модели) и хранятся в буфере воспроизведения. Обновления используют веса важности для коррекции несоответствия распределений. Офлайн RL значительно улучшает пропускную способность, но вводит off-policy bias, если буфер воспроизведения устарел.

Асинхронный RL Практический компромисс, используемый несколькими рецептами моделей рассуждений: генерация выполняется асинхронно на одном наборе GPU, пока обновления градиента выполняются на другом. Это удерживает обучение близким к on-policy, избегая простоя вычислений.

15.7.3 Инфраструктура в масштабе

Обучение модели рассуждений требует инфраструктуры, отличающейся от стандартного предобучения языковых моделей в нескольких аспектах:

- **Пропускная способность генерации**: Обучение RLVR требует сэмплирования $k = 8$–16 развёрток на промпт, каждая потенциально тысячи токенов длиной. Необходима эффективная батчевая генерация с непрерывным батчингом (как в vLLM).
- **Верификация в масштабе**: Верификатор (regex-экстрактор, изолированная среда для кода) должен работать параллельно для тысяч завершений в секунду, не становясь узким местом. Верификация кода особенно дорогостояща, так как требует изоляции процессов.
- **Управление памятью**: Отсутствие в GRPO сети ценности вдвое уменьшает потребление памяти GPU по сравнению с PPO, но хранение k полных последовательностей развёрток на промпт всё равно требует тщательного планирования памяти.
- **Нормализация вознаграждений и журналирование**: Поскольку вознаграждения RLVR бинарны и разрежены в начале обучения, необходим тщательный мониторинг скользящего среднего вознаграждения, длины развёртки и соответствия формату для обнаружения нестабильностей обучения до их каскадного развития.

15.7.4 Инструменты с открытым исходным кодом

Несколько фреймворков с открытым исходным кодом сделали обучение RLVR доступным за пределами хорошо финансируемых промышленных лабораторий:

TRL Библиотека Transformer Reinforcement Learning от Hugging Face реализует PPO, GRPO и DPO с первоклассной поддержкой верифицируемых вознаграждений. Её `GRPOTrainer` интегрируется напрямую с экосистемами `datasets` и `transformers`.

veRL Высокопропускной фреймворк обучения RL от Volcengine, поддерживающий асинхронные пайплайны генерации-обновления и достигающий state-of-the-art пропускной способности обучения для крупных моделей.

OpenRLHF Распределённый фреймворк RLHF и RLVR, построенный на DeepSpeed и Ray, поддерживающий модели до сотен миллиардов параметров с гибкими API функций вознаграждения.

15.8 Итоги и открытые вопросы

В этой главе прослежена дуга от RLHF — где обученная модель вознаграждения обеспечивала несовершенные сигналы предпочтений, — к RLVR, где детерминированные функции верификации обеспечивают агрессивное, долгосрочное обучение RL на верифицируемых областях. Ключевые концептуальные вклады:

- **Верифицируемые вознаграждения** устраняют модель вознаграждения как узкое место и источник отказов. Бинарные вознаграждения от проверочных функций не содержат шума и не могут быть сверхоптимизированы.
- **Рассуждения типа chain-of-thought** связывают генерацию токенов с неявным поиском: более длинные следы рассуждений реализуют более тонкий поиск по путям решений.
- **Масштабирование на этапе вывода** показывает, что вычисления можно продуктивно использовать при развёртывании, не только при обучении. Голосование большинства и сэмплирование с направлением по ценности извлекают улучшения точности из дополнительной генерации без повторного обучения.
- **Использование инструментов и агентное RL** расширяют фреймворк на интерактивные среды, превращая языковые модели из пассивных генераторов текста в активных агентов решения проблем.

Открытые вопросы. Область развивается быстро, и ряд фундаментальных вопросов остаётся нерешённым:

- **Обобщение RLVR**: Переносятся ли навыки рассуждения, освоенные на математике и коде, на неверифицируемые области (открытое письмо, стратегическое рассуждение, безопасность)? Ранние данные указывают на частичный перенос, но механизм неясен.
- **Пределы масштабирования**: Как далеко распространяются законы масштабирования на этапе вывода? В какой момент генерация большего числа токенов даёт убывающую отдачу независимо от мощности модели?
- **Процессные модели вознаграждения в масштабе**: Обучение надёжной PRM требует надзора на уровне шагов, дорогостоящего для сбора. Оценка Монте-Карло корректности шагов шумна. Лучшие методы обучения PRM — активная область исследований.
- **Безопасность агентных систем**: Агент, способный просматривать веб, выполнять код и вызывать API, имеет значительно большее пространство действий, чем текстовый чат-бот. Согласование целей многошаговых агентов с человеческими ценностями — открытый вызов, который классический RLHF не был разработан для решения.
- **Взаимодействие с предобучением**: Ряд результатов указывает на то, что определённый уровень возможностей предобучения является предпосылкой для работы RLVR. Понимание порога и того, как его снизить, демократизировало бы обучение рассуждению.

Упражнения

Упражнение 15.1. Рассмотрим цель RLVR из (15.3) при $\beta = 0$. Покажите, что градиент стратегии $J_{\mathrm{RLVR}}(\theta)$ по θ может быть записан как

$$\nabla_\theta J_{\mathrm{RLVR}}(\theta) = \mathbb{E}_{x,y}\big[\mathbb{1}\{\textsc{Verify}(y, a^*)\}\, \nabla_\theta \log \pi_\theta(y \mid x)\big],$$

начиная с первых принципов с использованием трюка логарифмической производной. Как это сводится к градиенту REINFORCE из Главы 8 с $r(y) = \mathbb{1}\{\textsc{Verify}(y, a^*)\}$?

Упражнение 15.2. Докажите Утверждение 15.1. В частности, покажите, что для k независимых испытаний Бернулли(p) при $p > 1/2$ вероятность того, что произойдёт менее $k/2$ успехов, ограничена сверху $\exp(-k\, D_{\mathrm{KL}}(1/2 \,\|\, 1-p))$ с использованием неравенства Хёффдинга или границ Чернова.

Упражнение 15.3. Предположим, что $k = 4$ развёртки для промпта получают вознаграждения $r = (1, 1, 0, 0)$. Вычислите оценки преимущества GRPO $\hat{A}_i$ из (15.11). Теперь предположим $r = (1, 0, 0, 0)$. Снова вычислите $\hat{A}_i$. Сравните величину сигнала градиента в двух случаях и объясните, почему второй сценарий даёт более сильную обратную связь для правильной развёртки, хотя правильная развёртка только одна.

Упражнение 15.4. Обучающий набор данных содержит 10 000 задач. Вы сэмплируете $N = 16$ развёрток на задачу и обнаруживаете, что 2 000 задач имеют коэффициент решаемости $\sigma < 0.2$, 5 000 имеют $\sigma \in [0.2, 0.8]$ и 3 000 имеют $\sigma > 0.8$.

(a) Какая доля вычислений «расходуется впустую» на неинформативных задачах (с $\sigma < 0.2$ или $\sigma > 0.8$)?

(b) Как фильтрация по сложности меняет эффективный размер набора данных, доступного для обновлений градиента?

(c) Предложите динамический учебный план сложности, который позволяет не отбрасывать сложные задачи навсегда.

Упражнение 15.5. Используя разложение (15.4), неформально обоснуйте, почему (a) chain-of-thought улучшает точность для сложных многошаговых задач, но (b) по большей части избыточен для задач, которые модель может решить правильно за один шаг. Что это подразумевает для требований к *переключаемому* режиму рассуждений?

Упражнение 15.6. Модель решает следующую задачу через использование инструментов: получить численность населения Франции и численность населения Германии, затем вывести их соотношение. Напишите функцию Python `verify(trajectory)`, которая принимает список пар (действие, наблюдение) и возвращает вознаграждение в $\{0, 1\}$. Ваша функция должна обрабатывать: (a) вызов модели инструментов в неправильном порядке, (b) отказ модели вызвать требуемый инструмент, и (c) арифметические ошибки в финальном вычислении.

Упражнение 15.7. Рассмотрим многошагового агента, выполняющего задачу из 5 шагов. Вознаграждение за результат равно $+1$ только при выполнении задачи. Предположим, что агент преуспевает на финальном шаге, но выбрал неэффективный путь (сделав 3 избыточных API-вызова). При:

(a) только вознаграждении за результат, что агент узнаёт об избыточных API-вызовах?

(b) процессной модели вознаграждения, штрафующей избыточные вызовы на -0.1 каждый, как меняется сигнал градиента?

Обсудите компромисс смещение-дисперсия между (a) и (b).

Упражнение 15.8. Пусть $\mathrm{Acc}_1(p)$ — точность единичной развёртки, где каждая развёртка верна с вероятностью p. Пусть $\mathrm{Acc}_k(p)$ — точность голосования большинства по k развёрткам.

(a) Запишите точную формулу для $\mathrm{Acc}_k(p)$ через биномиальное распределение.

(b) Для $p = 0.4$ и $k \in \{1, 3, 5, 9, 17, 33\}$ вычислите $\mathrm{Acc}_k(p)$. Заметьте, что голосование большинства может *снизить* точность при $p < 0.5$; объясните, почему.

(c) Найдите пороговое значение p, ниже которого голосование большинства вредно.

Упражнение 15.9. Рецепт DeepSeek R1 использует два этапа: холодный старт SFT, затем крупномасштабный RLVR. Предположим, что вы пропускаете этап SFT и применяете RLVR непосредственно к базовой предобученной модели.

(a) Какие проблемы могут возникнуть, если базовая модель не надёжно производит текст в формате `<thinking>...</thinking>`?

(b) Как награда за формат, описанная в Разделе 15.3.4, частично смягчает это?

(c) Почему холодный старт SFT всё же предпочтителен, несмотря на награду за формат?

Глава 16

Мультиагентные системы и RL

Целое больше суммы своих частей.

— Аристотель, *Метафизика*

Все рассмотренные нами до сих пор обстановки включают одного обучающего агента, взаимодействующего с миром, чья динамика фиксирована. Среда может быть стохастической, частично наблюдаемой или высокоразмерной, но сама она не *обучается*. В реальном мире это допущение редко выполняется. Финансовые рынки населены трейдерами, каждый из которых адаптируется к стратегиям других. Автономные транспортные средства должны вести переговоры на перекрёстках с другими транспортными средствами, одновременно пытающимися безопасно навигировать. Команда роботов-спасателей должна разделить здание между собой без централизованной координации. Языковые модели, развёрнутые как помощники, всё чаще делегируют подзадачи специализированным агентам — другим моделям, которые отвечают, проверяют или критикуют. В каждом случае «среда» содержит *других агентов*, и эти агенты сами обучаются и адаптируются.

Мультиагентное обучение с подкреплением (MARL) изучает, как несколько субъектов, принимающих решения, могут обучаться хорошему поведению при совместном существовании и взаимодействии. Эта область находится на пересечении теории игр, оптимизации и современного глубокого обучения, и она ставит качественно иные задачи по сравнению с одноагентным RL: среда *нестационарна*, поскольку другие агенты меняются; вознаграждения могут быть общими, конкурирующими или чем-то средним; а пространство совместных поведений экспоненциально больше пространства индивидуальных поведений.

В этой главе теория и практика MARL развиваются постепенно. Раздел 16.1 вводит марковские игры — естественное многоагентное обобщение MDP, — и определяет концепцию равновесия Нэша. Раздел 16.2 охватывает кооперативный MARL, включая парадигму централизованного обучения с децентрализованным исполнением (CTDE) и алгоритмы QMIX и MAPPO. Раздел 16.3 изучает конкурентные и смешанные обстановки, включая игры с нулевой суммой, самоигру и историю AlphaGo и AlphaZero. Раздел 16.4 исследует возникающую коммуникацию и социальные дилеммы. Раздел 16.5 переносит этот подход на большие языковые модели, охватывая дебаты, многоагентную генерацию кода и agentic workflows. Раздел 16.6 разбирает открытые проблемы, а Раздел 16.7 подводит итоги.

16.1 Мультиагентные марковские игры

16.1.1 От MDP к марковским играм

Напомним из Главы 3, что марковский процесс принятия решений — это кортеж $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$, в котором один агент принимает действия, среда переходит в новое состояние, а агент получает скалярное вознаграждение. Когда есть $N \geq 2$ агентов, естественным расширением является *марковская игра*, также называемая *стохастической игрой*.

Определение 16.1 (Марковская игра)**. Марковская игра** с N агентами — это кортеж

$$\mathcal{G} = \big(\mathcal{S},\, \mathcal{A}^1, \ldots, \mathcal{A}^N,\, \mathcal{T},\, R^1, \ldots, R^N,\, \gamma\big),$$

где:

- $\mathcal{S}$ — конечное множество **состояний**, разделяемых всеми агентами.

- $\mathcal{A}^i$ — **множество действий** агента i. **Совместное пространство действий** — $\mathcal{A} = \mathcal{A}^1 \times \cdots \times \mathcal{A}^N$.
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ — **ядро перехода**; следующее состояние зависит от текущего состояния и совместного действия всех агентов.
- $R^i : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ — **функция вознаграждения** для агента i. У каждого агента своя функция вознаграждения, которая может зависеть от действий всех агентов.
- $\gamma \in [0, 1)$ — общий коэффициент дисконтирования.

При $N = 1$ марковская игра сводится к стандартному MDP. Когда все агенты разделяют одну и ту же функцию вознаграждения, $R^1 = \cdots = R^N$, игра является **полностью кооперативной**. Когда $\sum_i R^i = 0$ при каждом переходе, она является **игрой с нулевой суммой**. Общий случай, когда вознаграждения ни идентичны, ни суммируются до нуля, называется **смешанной или игрой с общей суммой**.

Совместная стратегия всех агентов — $\boldsymbol{\pi} = (\pi^1, \ldots, \pi^N)$, где каждая $\pi^i : \mathcal{S} \to \Delta(\mathcal{A}^i)$ — стохастическая стратегия агента i. Цель агента i — максимизировать его собственный дисконтированный доход:

$$J^i(\boldsymbol{\pi}) = \mathbb{E}_{\boldsymbol{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t R^i(s_t, \boldsymbol{a}_t, s_{t+1}) \right], \tag{16.1}$$

где $\boldsymbol{a}_t = (a_t^1, \ldots, a_t^N)$ извлекается из произведения стратегий $\prod_i \pi^i(\cdot \mid s_t)$.

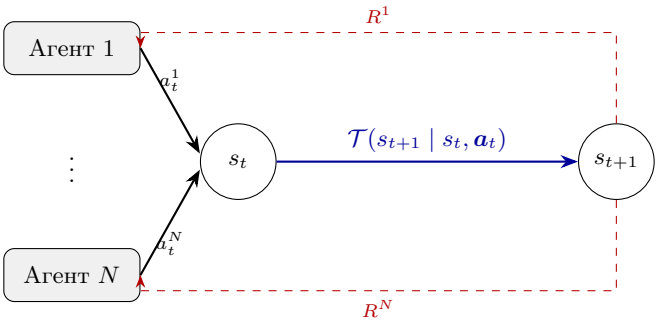


Рис. 16.1: Марковская игра с N агентами. На каждом шаге каждый агент наблюдает общее состояние s_t и выбирает действие. Совместное действие $\boldsymbol{a}_t$ управляет переходом в s_{t+1}, и каждый агент получает своё индивидуальное вознаграждение R^i.

16.1.2 Равновесие Нэша

Поскольку доход каждого агента зависит от стратегий *всех* агентов, стандартная концепция оптимальности из одноагентного RL больше не применима. Повышение дохода агента i изменением π^i может потребовать сохранения стратегий других агентов фиксированными — или может дать обратный эффект, если они адаптируются. Концепция решения, заимствованная из теории игр, — **равновесие Нэша**.

Определение 16.2 (Равновесие Нэша)**.** Совместная стратегия $\boldsymbol{\pi}^* = (\pi^{1*}, \ldots, \pi^{N*})$ является **равновесием Нэша**, если для каждого агента i и каждой альтернативной стратегии $\tilde{\pi}^i$:

$$J^i(\pi^{i*}, \boldsymbol{\pi}^{-i*}) \;\geq\; J^i(\tilde{\pi}^i, \boldsymbol{\pi}^{-i*}),$$

где $\boldsymbol{\pi}^{-i*}$ обозначает стратегии всех агентов кроме i, зафиксированные на равновесных значениях.

Иными словами, равновесие Нэша — это совместная стратегия, от которой ни один агент не имеет стимула отклоняться в одностороннем порядке. Теорема Нэша гарантирует, что каждая конечная игра имеет хотя бы одно равновесие Нэша в смешанных стратегиях, но равновесия не обязательно единственны, и нахождение одного вычислительно сложно в общем случае (PPAD-полная задача для двухигроковых игр с общей суммой).

Пример 16.1 (Дилемма заключённого как одношаговая марковская игра)**.** Два агента одновременно выбирают Сотрудничество (C) или Предательство (D). Выигрыши (строка = Агент 1, столбец = Агент 2):

	C	D
C	$(3,3)$	$(0,5)$
D	$(5,0)$	$(1,1)$

Агент 1 всегда предпочитает D независимо от действия Агента 2: если Агент 2 играет C, предательство даёт $5 > 3$; если Агент 2 играет D, предательство даёт $1 > 0$. Та же логика применима к Агенту 2. Следовательно, (D, D) — единственное равновесие Нэша, хотя (C, C) дало бы обоим агентам более высокий выигрыш 3. Это противоречие между индивидуальной рациональностью и коллективным благосостоянием является характерным признаком социальных дилемм и вновь появится в Разделе 16.4.

16.1.3 Частичная наблюдаемость и Dec-POMDP

На практике агенты редко наблюдают полное глобальное состояние. Каждый агент i вместо этого получает локальное наблюдение o_t^i, извлечённое из функции наблюдения $O^i : \mathcal{S} \to \Delta(\mathcal{O}^i)$. Получающаяся постановка — **децентрализованный частично наблюдаемый MDP** (Dec-POMDP), — в худшем случае NEXP-трудна. Большая часть практических исследований MARL сосредоточена на ограничении обстановки — кооперативные команды, общие наблюдения во время обучения или ограниченная коммуникация, — чтобы сделать задачу разрешимой.

16.2 Кооперативный MARL

16.2.1 Кооперативная обстановка

В полностью кооперативном MARL все агенты разделяют одно командное вознаграждение $r_t = R(s_t, \boldsymbol{a}_t, s_{t+1})$. Совместная цель:

$$J(\boldsymbol{\pi}) = \mathbb{E}_{\boldsymbol{\pi}}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]. \tag{16.2}$$

Несмотря на общую цель, каждый агент должен действовать на основе своего локального наблюдения и во время исполнения не может общаться или наблюдать глобальное состояние. Это стандартная обстановка в многороботной координации, многопользовательских кооперативных играх (например, микроуправление в StarCraft II) и многоагентных диалоговых системах.

16.2.2 Централизованное обучение с децентрализованным исполнением

Наиболее влиятельной парадигмой для кооперативного MARL является **централизованное обучение с децентрализованным исполнением** (CTDE). Во время *обучения* центральный критик или микшер имеет доступ к глобальному состоянию s_t и действиям всех агентов. Во время *исполнения* каждый агент опирается только на свою локальную историю наблюдений $\tau^i = (o_0^i, a_0^i, \ldots, o_t^i)$. Привилегия обучения позволяет критику давать информативный градиентный сигнал; ограничение исполнения гарантирует, что стратегия применима в обстановках, где центральный контроллер недоступен.

16.2.3 QMIX

QMIX (Rashid et al., 2018) — канонический алгоритм CTDE для кооперативного MARL на основе функции ценности. Каждый агент i поддерживает сеть полезности $Q_\theta^i(\tau^i, a^i)$, обусловленную только на локальной истории наблюдений. Центральная *сеть-микшер* объединяет эти полезности в совместную функцию ценности действий:

$$Q_{\text{tot}}(s, \boldsymbol{a}) = f_\phi\big(Q^1(\tau^1, a^1), \ldots, Q^N(\tau^N, a^N),\ s\big). \tag{16.3}$$

Микшер использует гиперсети, обусловленные на глобальном состоянии s, для получения неотрицательных весов, что гарантирует **ограничение монотонности**:

$$\frac{\partial Q_{\text{tot}}}{\partial Q^i} \geq 0 \quad \text{для всех } i.$$

Это ограничение является ключевым инсайтом. Оно гарантирует свойство **Индивидуально-Глобального Максимума** (IGM):

$$\operatorname*{argmax}_{\boldsymbol{a}}\ Q_{\text{tot}}(s, \boldsymbol{a}) \ = \ \big(\operatorname*{argmax}_{a^1} Q^1(\tau^1, a^1),\ \ldots,\ \operatorname*{argmax}_{a^N} Q^N(\tau^N, a^N)\big).$$

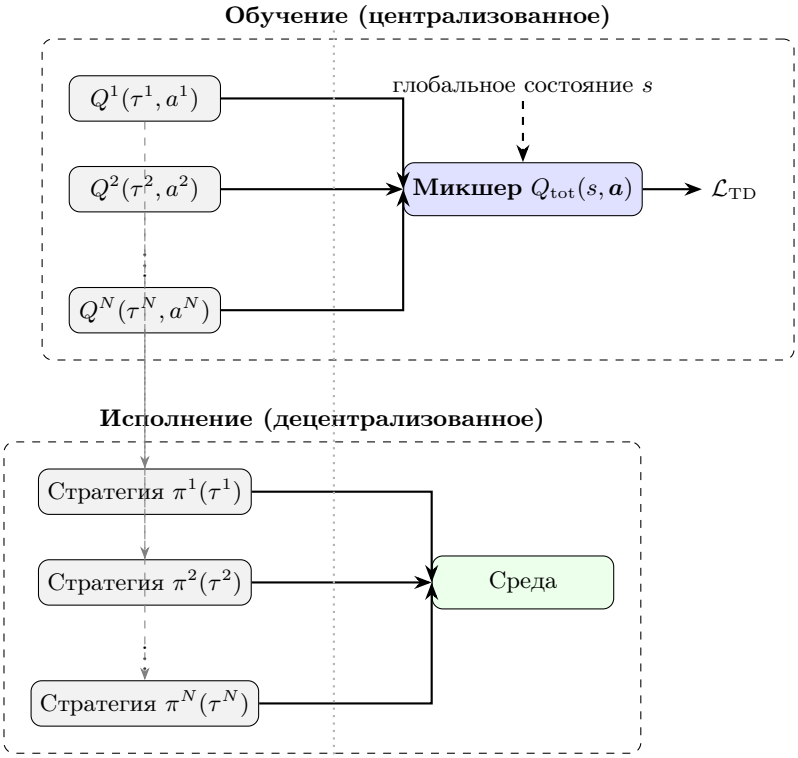


Рис. 16.2: Архитектура CTDE. Во время обучения *микшер* (например, QMIX) объединяет сети полезности Q^i каждого агента в совместный Q_{tot} с использованием глобального состояния, и градиенты распространяются через все Q^i. Во время исполнения централизованный микшер отбрасывается; каждый агент действует жадно относительно своего Q^i, используя только свою локальную историю наблюдений τ^i.

Максимизация Q_{tot} эквивалентна тому, что каждый агент жадно выбирает своё лучшее действие — поиск по совместным действиям во время исполнения не нужен. Вся система обучается end-to-end со стандартной функцией потерь TD:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}\Big[\big(r + \gamma \max_{\boldsymbol{a}'} Q_{\text{tot}}(s', \boldsymbol{a}') - Q_{\text{tot}}(s, \boldsymbol{a})\big)^2\Big]. \tag{16.4}$$

QMIX достигает лучших результатов в области на тесте StarCraft Multi-Agent Challenge (SMAC) — наборе задач кооперативного микроуправления в StarCraft II.

16.2.4 MAPPO

Тогда как QMIX работает в парадигме функции ценности, многие кооперативные задачи выигрывают от методов градиента стратегии, поскольку они естественно обрабатывают непрерывные действия и проще в настройке. **MAPPO** (Yu et al., 2022) применяет Proximal Policy Optimisation (PPO; Глава 9) в фреймворке CTDE. Каждый агент i поддерживает отдельного актора $\pi^i_\theta(\cdot \mid \tau^i)$, обусловленного на его локальной истории наблюдений. *Общий централизованный критик* $V_\phi(s)$ обусловлен на глобальном состоянии и производит baseline для снижения дисперсии:

$$\mathcal{L}_{\text{MAPPO}}(\theta^i) = \mathbb{E}\Big[\min\big(r^i_t(\theta^i)\,\hat{A}_t,\ \text{clip}(r^i_t(\theta^i), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big], \tag{16.5}$$

где $r^i_t(\theta^i) = \pi^i_\theta(a^i_t \mid \tau^i_t)/\pi^i_{\text{old}}(a^i_t \mid \tau^i_t)$ — коэффициент вероятностей, а $\hat{A}_t$ — преимущество, оцениваемое централизованным критиком. Ключевой практической находкой является то, что *совместное использование параметров* — инициализация всех агентов из одной сети актора при обусловлении на признаке индекса агента — резко улучшает эффективность по образцам, позволяя агентам разделять градиенты.

Замечание 16.1 (Совместное использование параметров против гетерогенных агентов). Совместное использование параметров мощно, когда агенты играют симметричные роли, но может быть неуместным для команд из разнородных специалистов (например, целителя и танка в игре). В этой обстановке отдельные сети с общим экстрактором признаков часто работают лучше всего.

16.3 Конкурентные и смешанные обстановки

16.3.1 Игры с нулевой суммой и самоигра

В двухигроковой игре с нулевой суммой $R^1 = -R^2$, так что выигрыш одного агента — ровно проигрыш другого. Равновесие Нэша игры с нулевой суммой по Маркову соответствует стратегии минимакса: агент 1 максимизирует, а агент 2 минимизирует одно и то же накопленное вознаграждение. Формально минимаксное значение v^* удовлетворяет:

$$v^* = \max_{\pi^1} \min_{\pi^2} \; J^1(\pi^1, \pi^2) = \min_{\pi^2} \max_{\pi^1} \; J^1(\pi^1, \pi^2),$$

где равенство (теорема о минимаксе фон Неймана) выполняется, когда стратегии могут быть смешанными (стохастическими).

Обучение агента для оптимальной игры в игре с нулевой суммой сталкивается с непосредственной проблемой начальной загрузки: чтобы научиться побеждать сильных противников, агенту нужны сильные противники для обучения. **Самоигра** решает это, заставляя агента играть против прошлых версий самого себя. Канонический цикл обучения:

1. Поддерживать популяцию (или один снимок) прошлых контрольных точек стратегии.
2. Выбрать противника из популяции.
3. Собрать развёртки; обновить текущую стратегию против этого противника.
4. Периодически добавлять текущую стратегию в популяцию.

Самоигра гарантирует, что распределение противников всегда имеет уровень сложности, соразмерный текущей стратегии, обеспечивая естественный учебный план.

16.3.2 AlphaGo и AlphaZero

Наиболее известное применение самоигры — программы AlphaGo и AlphaZero (Silver et al., 2016; 2017). Го долго сопротивлялось решениям ИИ, поскольку его игровое дерево имеет коэффициент ветвления около 250 и глубину более 200, делая исчерпывающий поиск невозможным.

AlphaGo объединял три компонента: (1) сеть ценности $v_\theta(s)$, обученную регрессией на результатах самоигры; (2) сеть стратегии $\pi_\theta(a \mid s)$, обученную обучением с учителем на партиях людей и затем уточнённую градиентом стратегии RL; и (3) поиск по дереву Монте-Карло (MCTS), использующий обе сети для управления развёртками. На каждом узле MCTS шаг выбора балансирует исследование и использование через критерий PUCT:

$$a^* = \operatorname*{argmax}_a \left[Q(s,a) + c_{\text{puct}}\, \pi_\theta(a \mid s) \frac{\sqrt{N(s)}}{1 + N(s,a)} \right], \tag{16.6}$$

где $Q(s,a)$ — эмпирическое среднее значение из прошлых развёрток, $N(s)$ — счётчик посещений узла s, а $N(s,a)$ — счётчик ребра (s,a).

AlphaZero убрал начальную загрузку через обучение с учителем: самоигра вместе с MCTS, даже при старте с *random play*, оказались достаточны для достижения сверхчеловеческой производительности в Го, шахматах и сёги за несколько дней обучения. Успех AlphaZero показал, что при наличии идеального симулятора самоигра может открыть экспертные стратегии полностью с нуля — знаковый результат как для ИИ, так и для MARL.

16.3.3 Нестабильность конкурентного обучения и нетранзитивные стратегии

Наивная самоигра не лишена патологий. В играх с **нетранзитивными стратегическими отношениями** — где стратегия A бьёт B, B бьёт C, но C бьёт A (динамика «камень-ножницы-бумага») — самоигра может циклически переходить вместо сходимости к равновесию Нэша. Стратегия, обученная против версии k самой себя, может быть сильной против k, но хрупкой против версии $k-5$ — проблема, известная как **забывание** или стратегический коллапс.

Широко используются два решения. **Фиктивная самоигра** обучает агента против *среднего* всех прошлых стратегий противника, а не только самого последнего снимка. В двухигроковых играх с нулевой суммой с конечными пространствами действий фиктивная самоигра сходится к равновесию Нэша. **Популяционное обучение** (PBT; Jaderberg et al., 2017) поддерживает разнообразную лигу агентов с различными стратегиями; каждый агент в популяции обучается против смеси других, предотвращая устранение разнообразия любой одной доминирующей стратегией.

16.4 Коммуникация и возникающее поведение

16.4.1 Обученные протоколы коммуникации

Когда агенты могут обмениваться сообщениями до или во время исполнения задачи, эффективное пространство стратегий команды резко расширяется. Но что им говорить? Жёсткое кодирование протокола коммуникации требует экспертизы в предметной области и плохо масштабируется. Более принципиальный подход — рассматривать сообщения как *непрерывные действия* и позволять протоколам коммуникации возникать через end-to-end обучение.

CommNet (Sukhbaatar et al., 2016) — одна из первых архитектур для дифференцируемой многоагентной коммуникации. На каждом шаге каждый агент i транслирует непрерывный вектор $m_t^i \in \mathbb{R}^d$; агрегированное сообщение $\bar{m}_t^i = \frac{1}{N-1}\sum_{j\neq i} m_t^j$ конкатенируется с наблюдением агента и подаётся в LSTM, чей выход управляет как следующим действием, так и следующим сообщением:

$$(a_t^i, m_{t+1}^i) = f_\theta\big(h_t^i, o_t^i, \bar{m}_t^i\big).$$

Поскольку канал коммуникации дифференцируем, вся система обучается end-to-end со стандартным обратным распространением через время. CommNet показал, что агенты могут спонтанно обучиться координации на кооперативных задачах (например, управление трафиком на перекрёстках) без какого-либо надзора над содержанием коммуникации.

TarMAC (Das et al., 2019) расширил CommNet, введя *целевую коммуникацию*: каждый агент производит ключ k^i и значение v^i, а агрегированное сообщение, полученное агентом j, — взвешенная по вниманию сумма:

$$m^j = \sum_{i\neq j} \sigma(k^i \cdot q^j)\, v^i,$$

где q^j — вектор запроса агента j. Механизм внимания позволяет агентам избирательно читать наиболее релевантные сообщения для их текущей ситуации, улучшая как производительность, так и интерпретируемость обученной коммуникации.

16.4.2 Возникающая коммуникация

Поразительный результат литературы по возникающей коммуникации состоит в том, что агенты могут развивать *композиционные языки*: короткие дискретные символы, комбинации которых кодируют структурированные смыслы. В референтных играх (например, один агент видит объект и должен его описать; другой агент должен идентифицировать его среди отвлекающих объектов) обученные агенты часто развивают словари, в которых разные символы соответствуют разным атрибутам объекта, а новые комбинации символов обозначают новые объекты. Эти находки параллельны изучению эволюции языка и поднимают вопрос о том, могут ли искусственные возникающие языки стать основой для полезных коммуникативных навыков для агентов реального мира.

16.4.3 Социальные дилеммы в многоагентном обучении

Дилемма заключённого из Примера 16.1 — одношаговая игра. В *повторяющихся* играх кооперация может возникать спонтанно, поскольку агенты имеют возможность *наказывать* предателей в будущих раундах. Знаменитые турниры Аксельрода показали, что простая стратегия Тит за тат (кооперировать изначально; на каждом последующем ходу копировать последнее действие противника) превосходит сложные стратегии в повторяющихся дилеммах заключённого.

В MARL социальные дилеммы формализуются как среды, где равновесие Нэша является Парето-субоптимальным: рациональное индивидуальное поведение ведёт к коллективно плохим результатам. Фреймворк последовательной социальной дилеммы (SSD) (Leibo et al., 2017) встраивает такие дилеммы в пространственные среды. Например, в игре *Gathering* два агента собирают яблоки и могут стрелять «штрафными лучами» для временного отключения друг друга. Взаимная стрельба отнимает время, но каждый агент индивидуально стимулируется выстрелить первым. Агенты RL, обученные независимым обучением, сходятся к взаимному предательству; агенты, обученные с просоциальным формированием вознаграждения или явной коммуникацией, учатся кооперировать.

16.5 Мультиагентный RL для LLM

В предыдущих разделах агенты рассматривались как нейронные сети, отображающие наблюдения в действия в среде с ясными состояниями и сигналами вознаграждения. Большие языковые модели (LLM)

вводят качественно новый вид агента: один, работающий в пространстве естественного языка, способный рассуждать через промежуточные шаги и делегировать подзадачи другим экземплярам LLM. Концепции MARL применяются естественно, и растущий массив работ использует их для улучшения качества рассуждений, генерации кода и выполнения задач.

16.5.1 Дебаты

Дебаты ИИ (Irving et al., 2018) — многоагентный протокол для получения надёжных выходов от сильной системы ИИ. Два языковых агента отстаивают противоположные стороны утверждения; третий агент (или судья-человек) оценивает обмен и объявляет победителя. Ключевое теоретическое утверждение состоит в том, что для достаточно способного судьи оптимальная стратегия каждого дебатёра — *говорить правду*: ложь всегда может быть разоблачена противником, поэтому лучшая стратегия в равновесии дебатов — честная аргументация.

На практике протоколы в стиле дебатов реализуются как пайплайн из двух LLM:

1. LLM-**предлагающий** генерирует утверждение или ответ.
2. LLM-**критик**, получив тот же промпт и выход предлагающего, генерирует контраргумент.
3. LLM-**судья** (или та же модель с промптом судьи) оценивает обе позиции и выбирает лучший ответ.

Оценка судьи может использоваться как сигнал вознаграждения для дообучения как предлагающего, так и критика через RL (Раздел 15.1 в Главе 15). Это обеспечивает *масштабируемый надзор*: система может улучшаться за пределами прямых возможностей судьи, поскольку судье проще оценить аргумент, чем генерировать правильный ответ с нуля.

16.5.2 Многоагентная генерация кода

Генерация кода — естественная обстановка для многоагентных LLM, поскольку код допускает автоматические верифицируемые вознаграждения (наборы тестов), а задача разбивается на подзадачи (спецификация, реализация, тестирование, документация), которые легко назначать специализированным агентам.

Представительный пайплайн (Chen et al., 2023; Du et al., 2023) работает следующим образом:

1. Агент-**планировщик** разбивает запрос пользователя на последовательность подзадач.
2. Несколько агентов-**программистов** независимо генерируют кандидатные реализации каждой подзадачи.
3. Агент-**верификатор** запускает наборы тестов на каждом кандидате и возвращает обратную связь об успехе/неудаче.
4. Агент-**ранкер**, обученный с предпочтительным RL, выбирает реализацию с наивысшим рангом и при необходимости исправляет неудачи.

Эмпирически разнообразные предложения от нескольких независимых программистов стабильно превосходят любого одного программиста, даже когда отдельные программисты являются идентичными моделями: разнообразие в случайных семенах, промптах или настройках температуры вводит достаточную вариацию, чтобы верификатор мог выявлять действительно лучшие решения.

16.5.3 LLM-агенты, сотрудничающие над задачами

Помимо кода, мультиагентные LLM-системы применялись к исследовательской помощи, игровым задачам и программной инженерии. **ChatDev** (Qian et al., 2023) структурирует программную команду как набор LLM-агентов, играющих разные роли (генеральный директор, технический директор, программист, тестировщик, автор документации), которые общаются через модерируемый чат-канал для завершения программного проекта от начала до конца. Каждая роль определяется системным промптом и ролевой историей; протокол модерации обеспечивает очерёдность ходов и фокус на задаче.

AutoGen (Wu et al., 2023) предоставляет более гибкий фреймворк: любой LLM может быть обёрнут как *агент* с именем, системным промптом и при желании интерфейсом использования инструментов. Агенты общаются, передавая сообщения в общем потоке; агент, представляющий человека, может вводить обратную связь от пользователя. Фреймворк поддерживает циклы, ветвящиеся разговоры и вложенные группы агентов, позволяя как кооперативные, так и состязательные паттерны.

16.5.4 Agentic Workflows со специализированными агентами

Наиболее практически влиятельным паттерном мультиагентного LLM на момент написания является паттерн **оркестратор — специалист**. Мощный универсальный LLM (*оркестратор*) получает сложную задачу пользователя, разбивает её на подзадачи и направляет каждую подзадачу *специализированному* агенту, оптимизированному для конкретного навыка:

Агент веб-поиска извлекает и резюмирует актуальную информацию.

Агент кода пишет и выполняет код в изолированном интерпретаторе.

Агент памяти ведёт постоянное хранилище ключ-значение фактов, извлечённых из предыдущих ходов.

Агент-критик проверяет план оркестратора и отмечает несоответствия перед исполнением.

Вознаграждения для дообучения RL оркестратора могут поступать от end-to-end решения задачи (например, совпадает ли финальный ответ с эталонным решением?) или от промежуточных подцелей, проверяемых специализированными агентами. Этот замкнутый цикл между оркестрацией и исполнением специалистом отражает парадигму CTDE: оркестратор действует как централизованный планировщик во время обучения, в то время как каждый специалист выполняет свою локальную стратегию независимо.

Связь с MARL прямая. Каждый специалист — это стратегия π^i; оркестратор определяет распределение задач над совместными последовательностями действий; а возникающее поведение ансамбля — это траектория в пространстве всех возможных многоагентных взаимодействий. Формальное обучение этой системы с целями MARL — а не дообучение каждого компонента в отдельности, — является активным рубежом исследований.

16.6 Проблемы мультиагентного RL

16.6.1 Нестационарность

Наиболее фундаментальная проблема MARL состоит в том, что среда каждого агента включает всех других агентов, которые сами обучаются. С точки зрения агента i закон перехода и функция вознаграждения *нестационарны*: мир, который он испытывает сегодня, отличается от мира, который он будет испытывать завтра, поскольку его товарищи по команде или противники обновили свои стратегии. Это нарушает допущение стационарности, лежащее в основе доказательств сходимости для Q-обучения и других одноагентных алгоритмов.

Формально агент i наблюдает эффективный закон перехода:

$$\tilde{\mathcal{T}}^i(s' \mid s, a^i) = \sum_{\boldsymbol{a}^{-i}} \mathcal{T}(s' \mid s, a^i, \boldsymbol{a}^{-i}) \prod_{j \neq i} \pi^j(a^j \mid s),$$

который меняется каждый раз, когда меняется любая π^j $(j \neq i)$. Независимое Q-обучение игнорирует эту нестационарность и часто не сходится на практике, особенно в конкурентных обстановках. Алгоритмы, такие как QMIX и MAPPO, частично смягчают нестационарность путём совместного использования информации о градиентах между агентами, но гарантии сходимости остаются слабыми в общем случае.

16.6.2 Отнесение заслуг в командах

Даже при общем сигнале вознаграждения командное отнесение заслуг сложно: если команда добивается успеха, чьё действие было ответственным? Наивные подходы равномерно распределяют всё вознаграждение между всеми агентами, что даёт шумные градиенты и препятствует специализации. Эвристика **разностного вознаграждения** присваивает агенту i кредит:

$$d_t^i = r_t - r_t^{-i},$$

где r_t^{-i} — вознаграждение, которое *было бы* получено, если бы агент i выполнил какое-то дефолтное или контрфактическое действие, при зафиксированных других агентах. Вычисление r_t^{-i} требует либо обученной модели мира, либо дополнительных развёрток, что делает его дорогостоящим. Ограничение монотонности QMIX обеспечивает неявный механизм отнесения заслуг: градиент Q_{tot} по Q^i говорит агенту i, насколько его функция полезности вносит вклад в команду.

16.6.3 Масштабируемость

Совместное пространство действий $|\mathcal{A}| = \prod_i |\mathcal{A}^i|$ растёт экспоненциально с числом агентов. Для $N = 10$ агентов каждый с $|\mathcal{A}^i| = 5$ действиями совместное пространство имеет $5^{10} \approx 10^7$ элементов. Даже представление $Q_{\text{tot}}(s, \boldsymbol{a})$ в виде таблицы неосуществимо; факторизация CTDE в QMIX решает это, записывая Q_{tot} как функцию от полезностей каждого агента, уменьшая сложность с экспоненциальной до линейной по N.

Методы градиента стратегии лучше масштабируются, поскольку они работают с маргинальными стратегиями $\pi^i(a^i \mid \tau^i)$, а не с совместной стратегией. С совместным использованием параметров одна сеть обслуживает всех агентов, поэтому число параметров не растёт с N. Тем не менее время обучения по-прежнему масштабируется с числом агентов, поскольку нужно больше данных развёрток, а межагентные зависимости требуют больших буферов воспроизведения и мини-батчей.

16.6.4 Формирование вознаграждения для кооперации

В играх со смешанными мотивами агенты могут иметь индивидуально рациональные стимулы, конфликтующие с командной производительностью. Формирование вознаграждения — дополнение истинного вознаграждения потенциально-основанным бонусом, — может подтолкнуть агентов к кооперативному поведению без нарушения равновесия Нэша. Однако плохое формирование вознаграждений может привносить ложные равновесия или делать агентов безразличными к истинной цели.

Внутренняя социальная мотивация — многообещающая альтернатива: агентам даётся внутреннее вознаграждение за *влияние* на поведение других (вознаграждение за социальное влияние; Jaques et al., 2019) или за достижение causal empowerment над средой. Эти внутренние вознаграждения поощряют обмен информацией без необходимости в вручную разработанных протоколах коммуникации.

16.6.5 Эффективность по образцам

MARL требует много образцов. Не только каждый агент должен исследовать своё пространство действий, но и совместная проблема исследования экспоненциально больше. Координированное исследование — где агенты активно избегают избыточного исследования одних и тех же областей состояний, — уменьшает избыточность, но требует дополнительных затрат на координацию. Офлайн MARL, который учится из заранее собранных наборов данных без онлайн-взаимодействия — это появляющийся подход, переносящий недавние успехи офлайн одноагентного RL (Глава 17) в многоагентную обстановку.

В таблице ниже обобщены основные алгоритмические семейства, их обстановки и ключевые свойства.

Алгоритм	**Обстановка**	**Парадигма**	**Пространство действий**	**Ключевая идея**
IQL	Коо./Кон.	Децентрализ.	Дискретное	Независимое Q-обучение
QMIX	Кооперат.	CTDE	Дискретное	Монотонный микшер
QPLEX	Кооперат.	CTDE	Дискретное	Микшер с двойным вниманием
MAPPO	Кооперат.	CTDE	Непр./Диск.	Общий центральный критик
MADDPG	Смешанный	CTDE	Непрерывное	Центральный критик каждого агента
AlphaZero	Конкурент.	Самоигра	Дискретное	MCTS + сети стратегии/ценности
PSRO	Смешанный	Популяция	Дискретное	Nash resp. oracle + mixture
CommNet	Кооперат.	Канал связи	Непрерывное	Сообщения среднего поля
TarMAC	Кооперат.	Канал связи	Непрерывное	Целевые сообщения с вниманием

16.7 Итоги

Мультиагентное обучение с подкреплением расширяет одноагентный формализм MDP на N взаимодействующих агентов, каждый со своей стратегией, наблюдением и вознаграждением. Центральная концепция решения — равновесие Нэша: совместная стратегия, от которой ни один агент не может выгодно отклониться в одностороннем порядке. Нахождение равновесий Нэша сложно в общем случае, но структура игры — кооперация, конкуренция с нулевой суммой, факторизованная динамика, — обеспечивает разрешимые алгоритмы.

Для кооперативных задач парадигма CTDE достигает лучшего из обоих миров: централизованная информация о градиентах во время обучения и полностью децентрализованное исполнение. QMIX использует монотонную факторизацию для обеспечения глобальной оптимальности жадного выбора действий каждым агентом. MAPPO применяет PPO с централизованным value baseline и совместным использованием параметров, масштабируясь на большие команды с непрерывными действиями.

В конкурентных обстановках самоигра обеспечивает автоматический учебный план. AlphaZero показывает, что даже при старте с random play сочетания самоигры и MCTS достаточно для сверхчеловеческой производительности в играх с полной информацией. Нетранзитивность и стратегическое забывание мотивируют популяционные методы, поддерживающие разнообразие стратегий.

Коммуникация открывает дополнительный канал для кооперации. CommNet и TarMAC демонстрируют, что end-to-end обучение может производить эффективные протоколы обмена сообщениями без какого-либо надзора над содержанием сообщений. В социальных дилеммах, где равновесие Нэша является Парето-субоптимальным, повторяющееся взаимодействие и просоциальные вознаграждения могут поддерживать кооперативное поведение.

LLM — последняя и, пожалуй, наиболее значимая арена для идей MARL. Дебаты, многоагентная генерация кода, пайплайны оркестратор-специалист и системы вроде AutoGen и ChatDev реализуют многоагентную динамику в языке. Применение формальных целей обучения MARL к этим системам — а не дообучение компонентов по отдельности, — является одним из наиболее захватывающих открытых направлений на рубеже исследований RL.

Ключевые проблемы — нестационарность, отнесение заслуг, масштабируемость, формирование вознаграждения и эффективность по образцам, — взаимосвязаны, и ни одна полностью не решена. Прогресс в каждой, вероятно, потребует новой теории и новой инженерии, что делает MARL одной из наиболее активных и значимых подобластей машинного обучения.

Упражнения

Упражнение 16.1. Покажите, что марковская игра с $N = 1$ сводится к стандартному MDP. Затем покажите, что если все агенты разделяют одно вознаграждение $R^1 = \cdots = R^N$ и все агенты действуют одинаково (т.е. $\pi^1 = \cdots = \pi^N$), совместный доход из Уравнения (16.2) равен одноагентному доходу из Главы 3. Какие дополнительные условия нужны для того, чтобы градиент стратегии с общим вознаграждением равнялся одноагентному градиенту стратегии?

Упражнение 16.2. Рассмотрим дилемму заключённого из Примера 16.1.

(a) Убедитесь, что (D, D) — равновесие Нэша, проверив определение: ни один игрок не может улучшить свой выигрыш, перейдя к C.

(b) Покажите, что (C, C) — *не* равновесие Нэша.

(c) Теперь предположим, что игра проводится $T = 100$ раундов и агенты максимизируют суммарное недисконтированное вознаграждение. Является ли равновесием Нэша для обоих агентов всегда кооперировать? Обоснуйте.

(d) Что меняется, если горизонт бесконечен с коэффициентом дисконтирования $\gamma = 0.9$?

Упражнение 16.3. Свойство IGM утверждает, что

$$\operatorname*{argmax}_{\boldsymbol{a}} Q_{\text{tot}}(s, \boldsymbol{a}) = \big(\operatorname*{argmax}_{a^1} Q^1(\tau^1, a^1),\ \ldots,\ \operatorname*{argmax}_{a^N} Q^N(\tau^N, a^N)\big).$$

(a) Предположим, что $Q_{\text{tot}}(s, \boldsymbol{a}) = \sum_i Q^i(\tau^i, a^i)$. Докажите, что свойство IGM выполняется.

(b) Выполняется ли свойство IGM, если $Q_{\text{tot}}(s, \boldsymbol{a}) = \prod_i Q^i(\tau^i, a^i)$ и все $Q^i > 0$? Докажите или приведите контрпример.

(c) Объясните в одном абзаце, почему неотрицательные смешивающие веса QMIX (из его гиперсети) достаточны, но не необходимы для IGM.

Упражнение 16.4. Рассмотрим игру с тремя стратегиями: чистыми стратегиями Камень (R), Ножницы (P), Бумага (S) и матрицей выигрышей:

$$M = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$$

где M_{ij} — выигрыш игрока по строке при игре стратегией i против игрока по столбцу со стратегией j.

(a) Найдите единственное равновесие Нэша этой игры.

(b) Симулируйте 1 000 шагов градиентного подъёма по π^1, пока π^2 фиксирована в равновесии Нэша. Сходится ли π^1?

(c) Теперь симулируйте 1 000 шагов одновременного градиентного подъёма для обоих игроков. Опишите траекторию в пространстве смешанных стратегий. Сходится ли она к равновесию Нэша?

(d) Интуитивно объясните, почему эта игра вызывает циклирование наивной самоигры.

Упражнение 16.5. Кооперативная команда из $N = 4$ роботов получает общее вознаграждение $r = 10$, когда все четверо достигают цели одновременно, и $r = 0$ иначе. После успешного эпизода независимое Q-обучение равномерно распределяет кредит $10/4 = 2.5$ каждому агенту.

(a) Предположим, что Робот 3 имел вероятность 0.5 достичь цели независимо от своего действия. Опишите, как равномерное распределение кредита искажает сигнал обучения для Роботов 1, 2 и 4.

(b) Определите разностное вознаграждение для Робота i с помощью контрфактического сценария, где Робот i стоит на месте, пока другие следуют наблюдаемой стратегии. Вычислите это вознаграждение для Робота 3.

(c) В чём преимущество разностного вознаграждения перед равномерным кредитом? В чём его главный вычислительный недостаток?

Упражнение 16.6. Разработайте алгоритм CTDE для команды из N дронов, обследующих зону поиска и спасения. Каждый дрон наблюдает локальную сетку 5×5; глобальная карта 50×50. Агенты не могут общаться во время тестирования.

(a) Определите пространство наблюдений $\mathcal{O}^i$, пространство действий $\mathcal{A}^i$ и подходящую командную функцию вознаграждения R.

(b) Предложите архитектуру централизованного критика, использующую полную карту 50×50 во время обучения. Какие признаки вы бы конкатенировали наряду с глобальным состоянием?

(c) Обоснуйте, уместно ли здесь совместное использование параметров, и как бы вы обусловили актора каждого агента на его уникальном идентификаторе.

(d) Определите два способа возникновения нестабильности обучения и предложите меры по их устранению.

Упражнение 16.7. Реализуйте упрощённый протокол дебатов для задачи closed-book question answering.

(a) Определите состояние, действие и вознаграждение для каждого из агентов-предлагающего, критика и судьи в однораундовых дебатах.

(b) Пусть судья присваивает бинарное вознаграждение $r \in \{0, 1\}$ (верно / неверно). Запишите обновление градиента стратегии REINFORCE для предлагающего, предполагая, что стратегия критика зафиксирована. Высока ли дисперсия этого оценщика? Почему?

(c) Предложите baseline для предлагающего, снижающий дисперсию без введения смещения. Конкретно опишите, как вы бы её оценили.

(d) При каких условиях равновесие дебатов гарантирует правдивое поведение предлагающего? Сформулируйте необходимые допущения о способностях судьи.

Упражнение 16.8. В референтной игре агент-говорящий наблюдает изображение, извлечённое из множества K объектов, и должен произвести дискретное сообщение $m \in \{1, \ldots, V\}^L$ (размер словаря V, длина сообщения L). Слушатель наблюдает m и должен идентифицировать правильный объект из K отвлекающих объектов.

(a) Какое максимальное число различных объектов может быть закодировано языком со словарём V и длиной L?

(b) Предположим, что оба агента обучаются end-to-end с REINFORCE, и вознаграждение равно $+1$ за верную идентификацию, 0 иначе. Запишите совместное обновление градиента стратегии.

(c) Опишите одну метрику для измерения композиционности возникшего языка (например, топографическое сходство). Определите её формально.

(d) Почему возникший язык может не быть композиционным, даже когда композиционность была бы полезна?

Глава 17

Открытые задачи и перспективные направления

Лучший способ предсказать будущее — изобрести его.

— Alan Kay

В предыдущих главах мы проследили развитие обучения с подкреплением от многоруких бандитов (Глава 2) через динамическое программирование (Глава 4), методы временных разностей (Глава 6), приближение глубокой функции ценности (Глава 7), градиенты стратегии и алгоритмы актор-критик (Главы 8–9) вплоть до современного пересечения RL с большими языковыми моделями (Главы 11–16). По пути мы видели, как RL решает игры, выравнивает языковые модели с предпочтениями людей, обеспечивает сложные рассуждения и координирует мультиагентные системы.

Тем не менее на каждую решённую RL задачу приходится дюжина открытых. Некоторые давние — эффективность по образцам была предметом беспокойства с первых табличных алгоритмов, — а другие возникли лишь недавно, движимые быстрым принятием RL как метода дообучения для базовых моделей. Эта финальная глава обозревает наиболее важные открытые вопросы, группирует их в связные темы и пытается наметить путь вперёд. Она неизбежно более умозрительна, чем предшествующие главы: цель — не доказывать теоремы, а ориентировать читателя на задачи, которые будут определять следующее десятилетие исследований.

17.1 Эффективность по образцам

Обучение с подкреплением печально известно своей жаждой данных. Безмодельные алгоритмы, такие как PPO (Глава 9) и DQN (Глава 7), часто требуют миллионов или даже миллиардов взаимодействий со средой для обучения поведению, которое ребёнок осваивает за минуты. Разрыв между человеческой и машинной эффективностью по образцам — одна из старейших открытых задач области, и она остаётся одной из наиболее значимых.

17.1.1 Модельно-ориентированный RL и модели мира

Наиболее естественный путь к эффективности по образцам — *обучить модель среды* и планировать внутри неё. Если агент может предсказать, что произойдёт до того, как он действует, каждое реальное взаимодействие становится значительно более информативным. Ранние модельно-ориентированные методы учили динамику переходов в табличной или линейной форме (Глава 4 обсуждает аспект планирования); современные подходы обучают *модели мира* — нейронные сети, предсказывающие будущие наблюдения, вознаграждения и сигналы завершения.

Статья Ha и Schmidhuber «World Models» (2018) показала, что агент может обучиться сжатому скрытому представлению своих наблюдений, а затем обучить стратегию целиком внутри обученной модели. Hafner et al. расширили это направление семейством алгоритмов Dreamer (Dreamer, DreamerV2, DreamerV3), которые обучают рекуррентную модель пространства состояний и оптимизируют стратегию путём обратного распространения через воображаемые траектории в скрытом пространстве. DreamerV3 достиг производительности уровня человека на десятках непрерывных и дискретных задач управления — и даже добывал алмазы в Minecraft, — используя примерно в 10–100× меньше шагов среды, чем сопоставимые безмодельные методы.

MuZero (Schrittwieser et al., 2020) применил другой подход: вместо предсказания сырых наблюдений он обучал скрытую модель динамики, чьи скрытые состояния обучались сквозным образом для поддержки планирования через поиск по дереву Монте-Карло. MuZero достиг производительности AlphaZero в Го, шахматах и сёги, также освоив игры Atari, демонстрируя, что планирование в скрытом пространстве может соответствовать или превосходить безмодельные методы даже в визуально сложных областях.

Несмотря на эти успехи, модельно-ориентированный RL сталкивается с постоянными вызовами. Накопление ошибок модели — небольшие неточности в обученной динамике, накапливающиеся на длинных горизонтах планирования, — могут производить стратегии, оптимальные в модели, но катастрофические в реальной среде. Решение *когда* доверять модели и когда прибегать к реальным данным (так называемый Dyna trade-off; Sutton, 1991) остаётся в значительной мере эвристическим.

17.1.2 Перенос знаний и few-shot RL

Дополнительный путь к эффективности по образцам — *перенос*: использование знаний из предыдущих задач для ускорения обучения на новых. Алгоритмы мета-RL, такие как MAML (Finn et al., 2017) и RL^2 (Duan et al., 2016), обучают инициализацию или алгоритм внутреннего цикла обучения, который может адаптироваться к новой задаче за горсть эпизодов. Более недавние подходы на основе foundation models рассматривают траектории RL как последовательности и используют in-context learning — тот же механизм, что делает возможным few-shot prompting в языковых моделях, — для адаптации к новым средам во время вывода без обновлений градиента (см. Раздел 17.4 о Decision Transformer и связанных методах).

Видение единого универсального агента RL, переносящего знания через разнообразные области — «RL foundation model», — остаётся в значительной мере нереализованным. Gato (Reed et al., 2022) показал, что единственный трансформер может играть в Atari, создавать подписи к изображениям и управлять роботизированной рукой, но его производительность по отдельным задачам отставала от специализированных агентов. Устранение этого разрыва — активная область исследований.

17.2 Спецификация вознаграждения и выравнивание

Каждый алгоритм RL оптимизирует сигнал вознаграждения. Когда этот сигнал точно отражает то, что мы хотим, оптимизация полезна; когда нет, агент использует разрыв — явление, известное как *взлом вознаграждения*. Задача — экземпляр закона Гудхарта: «Когда мера становится целью, она перестаёт быть хорошей мерой».

Глава 10 ввела модели вознаграждения, обученные на сравнениях людей, а Глава 12 показала, как DPO обходит явное моделирование вознаграждения. Оба подхода смягчают взлом вознаграждения в какой-то мере, но ни один не устраняет его. По мере того как модели становятся более мощными, ставки неправильно заданных вознаграждений растут.

17.2.1 Масштабируемый надзор

Когда возможности агента превышают возможности оценщика, обратная связь от людей становится ненадёжной. Модель для генерации кода может производить решения, верные, но непостижимые; модель научного рассуждения может предлагать эксперименты, чью обоснованность не может судить неспециалист. Задача *масштабируемого надзора* спрашивает: как обеспечить надёжный обучающий сигнал для систем, чьи выходы нам нелегко проверить?

Несколько исследовательских программ решают этот вопрос. *Дебаты* (Irving et al., 2018) противопоставляют две модели, с судьёй-человеком, решающим, какой аргумент более убедителен; надежда состоит в том, что истину проще проверить, чем сгенерировать. *Рекурсивное моделирование вознаграждения* (Leike et al., 2018) использует ИИ-помощника для помощи человеку в оценке, создавая иерархию надзора. *Constitutional AI* (Bai et al., 2022) заменяет часть обратной связи от людей набором письменных принципов, которые модель критикует сама против себя, снижая — хотя и не устраняя, — нагрузку на оценщиков-людей.

17.2.2 Фундаментальный вызов

По своей сути задача выравнивания — это задача *спецификации ценностей*: перевод богатого, контекстно-зависимого и часто противоречивого ландшафта человеческих ценностей в формальную цель. RL предоставляет оптимизационный механизм, но не может сказать нам, *что* оптимизировать. Это не просто техническая задача; это философская, затрагивающая вопросы, которые этики и политические теоретики обсуждают веками.

Практическое следствие для исследователей RL состоит в том, что проектирование вознаграждения останется первоклассной заботой. Успехи в интерпретируемости, лучшие методы получения предпочтений

людей и формальные фреймворки для спецификации ограничений — всё это сыграет свою роль. Никакая единственная техника вряд ли «решит» выравнивание; вызов системный, и прогресс будет постепенным.

17.3 Безопасность и устойчивость

Стратегия RL, хорошо работающая в среднем, может катастрофически отказывать в редких, но важных ситуациях. Безопасность и устойчивость — не предметы роскоши; они являются предпосылками для развёртывания RL в реальном мире.

17.3.1 Безопасное исследование

В таких областях, как автономное вождение, здравоохранение и промышленное управление, агент не может исследовать свободно: определённые состояния (столкновения, передозировки, повреждение оборудования) должны избегаться даже во время обучения. *Ограниченные MDP* формализуют это, дополняя стандартный MDP функциями стоимости и ограничениями вида $\mathbb{E}_\pi[\sum_t c(s_t, a_t)] \leq d$. Лагранжевы методы и ограниченная оптимизация стратегии (CPO; Achiam et al., 2017) оптимизируют вознаграждение с учётом этих ограничений, но гарантировать удовлетворение ограничений во время обучения — не только при сходимости, — остаётся сложным.

17.3.2 Распределительная устойчивость и состязательные атаки

Стратегии, обученные в одной среде, могут сломаться при развёртывании в другой, отличающейся даже незначительно. *Распределительная устойчивость* формулирует задачу как игру между агентом и противником, возмущающим динамику переходов или наблюдения в пределах ограниченного множества неопределённости. Робастные MDP (Iyengar, 2005; Nilim и El Ghaoui, 2005) обеспечивают теоретическую основу, и недавние работы масштабировали эти идеи на глубокий RL через состязательное обучение и рандомизацию домена.

Связанной озабоченностью являются *состязательные атаки на стратегии RL*: небольшие, тщательно подобранные возмущения наблюдения могут заставить хорошо обученную стратегию предпринимать катастрофически плохие действия (Huang et al., 2017). Сертифицированные защиты, аналогичные разработанным для классификаторов изображений, для последовательного принятия решений всё ещё находятся в зачаточном состоянии.

17.3.3 Формальная верификация

Можем ли мы *доказать*, что обученная стратегия удовлетворяет свойству безопасности? Методы формальной верификации из программной инженерии — проверка моделей, абстрактная интерпретация, выполнимость по модулю теорий (SMT) — были применены к небольшим нейросетевым контроллерам, но масштабирование их на сети, используемые в современном глубоком RL, остаётся открытой задачей. Преодоление разрыва между статистическими гарантиями теории обучения и детерминированными гарантиями формальных методов — одна из наиболее важных — и наиболее трудных, — задач в области.

17.4 Офлайн и пакетный RL

Во многих практических обстановках — здравоохранение, рекомендательные системы, диалог, — онлайн-взаимодействие дорогостояще или рискованно. *Офлайн RL* (также называемый *пакетным RL*) стремится обучить хорошую стратегию из фиксированного набора данных ранее собранных переходов без дальнейшего взаимодействия со средой. Идея привлекательна: организации часто располагают большими журналами прошлого поведения, и офлайн RL обещает извлечь ценность из этих данных.

17.4.1 Задача сдвига распределения

Центральная трудность — *сдвиг распределения*. Набор данных был собран некоторой поведенческой стратегией μ, но обученная стратегия π может посещать состояния, которые μ редко или никогда не встречала. В этих областях оценки ценности ненадёжны, и наивные офлайн-методы могут расходиться или производить обманчиво оптимистичные стратегии.

Conservative Q-Learning (CQL; Kumar et al., 2020) решает это, добавляя член регуляризации, штрафующий Q-функцию на внераспределённых действиях, эффективно обучая нижнюю границу истинной ценности. Implicit Q-Learning (IQL; Kostrikov et al., 2022) полностью избегает опроса стратегии во время

обучения, используя ожидаемую регрессию для оценки Q-значений верхней огибающей из набора данных. Оба метода показали сильную производительность на стандартных тестах, но их эффективность существенно зависит от качества и покрытия набора данных.

17.4.2 Моделирование последовательностей и Decision Transformer

Влиятельной альтернативой офлайн RL на основе функции ценности является переформулирование задачи как *моделирования последовательностей*. Decision Transformer (Chen et al., 2021) обусловливает трансформер на желаемом доходе, прошлых состояниях и прошлых действиях, и авторегрессионно предсказывает будущие действия. Во время тестирования агент обусловлен на высоком доходе и генерирует соответствующее поведение. Этот подход полностью обходит обновления Беллмана, используя сильные стороны трансформеров в моделировании последовательностей.

Trajectory Transformer (Janner et al., 2021) расширил идею путём моделирования совместного распределения по состояниям, действиям и вознаграждениям, обеспечивая планирование через beam search во время вывода. Эти методы размывают границу между RL и обучением с учителем, и их теоретические свойства — когда они выигрывают, когда ломаются и почему, — всё ещё остаются поняты лишь частично.

17.4.3 Переход от офлайн к онлайн

Многообещающим гибридом является *переход от офлайн к онлайн*: предобучение стратегии из статического набора данных с последующим дообучением небольшим объёмом онлайн-взаимодействия. Это отражает парадигму pretrain-then-finetune, столь успешную для языковых моделей (Глава 11). Ранние результаты (Nair et al., 2020; Lee et al., 2022) показывают, что офлайн-предобучение может резко ускорить онлайн-обучение, но предотвращение катастрофического забывания офлайн-знаний и управление сдвигом распределения при переходе остаются открытыми задачами.

17.5 Мультимодальный и воплощённый RL

Обучение с подкреплением началось с физических интуиций — робот, навигирующий по лабиринту, термостат, регулирующий температуру, — но за последнее десятилетие наиболее заметные успехи были в симуляции: играх, языке и рекомендациях. Возврат RL в физический мир — один из великих незавершённых проектов области.

17.5.1 Перенос из симуляции в реальность

Обучение в симуляции дёшево и безопасно; развёртывание в реальном мире — ни то, ни другое. *Разрыв симуляция-реальность* возникает, потому что симуляторы неизбежно упрощают физику, шум датчиков и визуальный вид. Рандомизация домена (Tobin et al., 2017) смягчает разрыв, обучая стратегию по распределению симулированных сред, так что реальный мир выглядит просто ещё одним образцом. Идентификация системы, адаптивное управление и обученные остаточные модели предлагают дополнительные подходы, но надёжное преодоление разрыва симуляция-реальность для манипуляций с контактами остаётся активной задачей.

17.5.2 Vision-Language-Action Models

Новый класс *vision-language-action* (VLA) моделей стремится объединить восприятие, понимание языка и моторное управление в единой архитектуре. RT-2 (Brohan et al., 2023) дообучил визуально-языковую модель для вывода роботических действий как текстовых токенов, демонстрируя, что семантические знания из крупномасштабного предобучения на веб-данных могут переноситься на задачи манипуляции. Octo (Ghosh et al., 2024) и π_0 (Black et al., 2024) продвинули это дальше с архитектурами трансформеров, принимающими изображения и языковые инструкции и выдающими непрерывные команды суставов.

Эти модели всё ещё далеки от ловкости и адаптируемости человеческих манипуляций. Ключевые узкие места включают нехватку высококачественных данных роботических демонстраций, сложность спецификации вознаграждений для задач с контактами и вычислительные затраты на вывод в реальном времени на аппаратуре с ограниченными ресурсами.

17.5.3 Непрерывное управление at scale

Помимо манипуляций, RL применяется к локомоции (шагающие роботы, обученные через PPO в симуляции и развёрнутые на аппаратуре), автономным гонкам (Gran Turismo), навигации дронов и управлению

гуманоидами всем телом. Общая нить состоит в том, что эти задачи включают высокоразмерные непрерывные пространства действий, сложную динамику контактов и жёсткие требования к реальному времени. Масштабирование RL на эти обстановки раздвигает как алгоритмические, так и инженерные рубежи.

17.6 RL и базовые модели

Пожалуй, наиболее значимое развитие в недавнем RL — его появление в качестве *универсального метода дообучения* для базовых моделей. Главы 11–15 зафиксировали это для текста; тот же паттерн теперь повторяется через модальности.

17.6.1 RL для языка и рассуждений

RLHF (Глава 14) и GRPO (Глава 13) стали стандартными инструментами для выравнивания и улучшения больших языковых моделей. Как обсуждалось в Главе 15, RL с верифицируемыми вознаграждениями обеспечил кардинальные улучшения в математическом рассуждении и генерации кода. Рубеж смещается в сторону задач рассуждений с более длинным горизонтом — многошаговое научное решение задач, программная инженерия, доказательство теорем, — где вознаграждение разрежено и отложено, а пространство действий — пространство всех возможных продолжений текста.

17.6.2 RL для визуальной генерации

Тот же цикл RLHF, который улучшил языковые модели, был применён к генерации изображений. DPOK (Fan et al., 2023) и DDPO (Black et al., 2024) использовали методы градиента стратегии для дообучения диффузионных моделей против моделей вознаграждения с предпочтениями людей, улучшая эстетическое качество и соответствие промпту. Для генерации видео дообучение на основе RL использовалось для улучшения временной согласованности и визуальной точности, хотя вычислительные затраты на генерацию и оценку видеотраекторий остаются узким местом.

17.6.3 RL для научных открытий

RL внёс вклад в заметные научные прорывы. Обучающий пайплайн AlphaFold (Jumper et al., 2021) включал RL-подобное структурное уточнение. GNoME (Merchant et al., 2023) использовал RL-управляемый поиск для открытия миллионов новых стабильных кристаллических структур. FunSearch (Romera-Paredes et al., 2024) объединил большие языковые модели с эволюционным поиском для открытия новых математических конструкций, превзойдя людей-математиков в задаче о cap-set.

Общий паттерн — RL как *outer loop*: базовая модель предлагает кандидаты (структуры белков, составы материалов, математические гипотезы), а RL-подобная цель оценивает и отбирает среди них. Расширение этого паттерна на химию, разработку лекарств, моделирование климата и инженерное проектирование — активный и перспективный рубеж.

17.7 Теоретические рубежи

Теория RL богата в табличных обстановках и обстановках с линейной аппроксимацией функций (Главы 3–6), но методы, работающие лучше всего на практике — глубокие сети, трансформеры, высокоразмерные пространства стратегий, — часто опережают доступную теорию.

17.7.1 Сожаление и сложность выборки для глубокого RL

Границы сожаления для табличных MDP тесные: алгоритмы, такие как UCBVI, достигают $\tilde{O}(\sqrt{H^3SAT})$ сожаления, где H — горизонт, S — число состояний, A — число действий, и T — число эпизодов. Для линейных MDP также известны совпадающие верхние и нижние границы. Но для нелинейной аппроксимации функций, используемой в глубоком RL — сверхпараметризованные нейронные сети, обученные с стохастическим градиентным спуском, — сопоставимых гарантий не существует.

Анализ нейронного тангенциального ядра (NTK) даёт некоторое понимание, показывая, что достаточно широкие сети ведут себя приблизительно линейно, но этот режим не захватывает обучение представлений, которое делает глубокий RL эффективным. Разработка теории, учитывающей обучение признаков, неконвексную оптимизацию и компромисс исследование-использование одновременно, — одна из наиболее трудных открытых задач в теории машинного обучения.

17.7.2 Сходимость нелинейной аппроксимации функций

Смертельная триада — комбинация бутстрапа, офлайн-обучения и аппроксимации функций, — была идентифицирована Саттоном и Барто как источник расходимости в приближённом динамическом программировании (Глава 7). Хотя алгоритмы, такие как DQN, хорошо работают на практике, доказательство их сходимости под реалистичными допущениями (сети конечной ширины, neiid данные, воспроизведение опыта) остаётся неуловимым. Недавний прогресс включает результаты сходимости для сверхпараметризованных сетей в режиме NTK и двухвременные анализы методов актор-критик, но единая теория всё ещё отсутствует.

17.7.3 Статистическая сложность RLHF

Взрыв RLHF в обучении языковых моделей (Глава 12) породил новые теоретические вопросы. Сколько сравнений людей нужно для обучения модели вознаграждения, точно отражающей человеческие предпочтения? При каких условиях цель DPO восстанавливает ту же стратегию, что пайплайн RLHF? Как качество данных предпочтений влияет на статистическую эффективность выравнивания? Ранние результаты (Zhu et al., 2024; Xu et al., 2024) устанавливают границы сложности выборки при Bradley–Terry модели предпочтений, но теория пока не учитывает распределительные и стратегические сложности реальной обратной связи от людей.

17.7.4 Почему RL работает для LLM?

Пожалуй, наиболее глубокая теоретическая загадка — почему дообучение RL работает так хорошо для больших языковых моделей. Пространство действий огромно (полный словарь на каждой позиции токена), вознаграждение разрежено и отложено (часто единственный скаляр для целого ответа), и стратегия — сеть с миллиардом параметров. Классическая теория RL предсказывает, что такие задачи должны быть неразрешимы, однако PPO и GRPO производят надёжные улучшения на практике. Понимание этого — вероятно, через сочетание структуры естественного языка, неявного исследования, обеспечиваемого сэмплированием токенов, и тёплого старта от предобучения, — является ключевым открытым вопросом.

17.8 Путь вперёд

Завершим, отступив от отдельных задач и рассмотрев более широкую траекторию области.

RL как универсальное дообучение. Наиболее поразительная тенденция последних нескольких лет — обобщение RL от нишевого метода для игр и робототехники в *универсальный метод дообучения* для любой модели, генерирующей структурированный вывод. Языковые модели, генераторы изображений, видео-синтезаторы, предсказатели структур белков и роботические контроллеры — все улучшаются через дообучение RL. Лежащий в основе принцип прост: если вы можете оценить вывод, вы можете использовать RL для того, чтобы модель производила выводы с более высокой оценкой. Этот принцип, впервые формализованный в теореме о градиенте стратегии (Глава 8), оказывается одной из наиболее мощных идей в современном ИИ.

Сближение классической теории и современной практики. На протяжении десятилетий теория RL и практика RL развивались в значительной мере отдельными сообществами. Теоретики изучали табличные MDP и линейную аппроксимацию функций; практики обучали глубокие сети at scale. Этот разрыв начинает сокращаться. Теория офлайн RL информирует практическое проектирование алгоритмов; границы PAC-Bayes обеспечивают невакуумные гарантии обобщения для обученных стратегий; а изучение RLHF приносит инструменты теории обучения к одному из наиболее значимых применений RL. Единая теория, соединяющая элегантные результаты табличной обстановки с эмпирическими успехами глубокого RL, стала бы знаковым достижением.

Агенты и автономия. Глава 15 и Глава 16 описали появление агентов с обучением RL, способных рассуждать, использовать инструменты и координироваться с другими агентами. Эта тенденция ускоряется. По мере того как языковые модели становятся более мощными и дообучение RL обеспечивает планирование с более длинным горизонтом, мы можем увидеть агентов, которые автономно проводят научные исследования, пишут и отлаживают сложное программное обеспечение, управляют инфраструктурой и ведут переговоры от имени своих пользователей. Технические задачи огромны — отнесение заслуг через тысячи шагов, безопасное исследование в открытых средах, надёжная многоагентная координация, — но так же велики и потенциальные выгоды.

Важность безопасности. С большей автономией приходит больший риск. Задачи безопасности и выравнивания, обсуждённые в Разделах 17.2 и 17.3, будут только усугубляться по мере того, как агенты становятся более мощными. Области нужны не только лучшие алгоритмы, но и лучшие институты: стандарты оценки, практики red teaming, регуляторные рамки и культура ответственного развития. Исследователи RL несут здесь особую ответственность, поскольку оптимизационное давление, оказываемое RL, — одно из наиболее сильных во всём машинном обучении.

Что может принести следующее десятилетие. Предсказывать будущее быстро движущейся области — занятие дурака, но несколько тенденций, по всей видимости, сохранятся. Во-первых, масштаб обучения RL продолжит расти, движимый экономикой дообучения базовых моделей. Во-вторых, граница между RL, обучением с учителем и самосупервизированным обучением продолжит размываться, как демонстрируют методы, такие как DPO (Глава 12) и Decision Transformer (Раздел 17.4). В-третьих, RL будет применяться ко всё более широкому спектру областей: от науки о материалах до климатической политики и образования. В-четвёртых, задачи безопасности и выравнивания вынудят сообщество разработать новые инструменты для оценки, надзора и управления.

История обучения с подкреплением далека от завершения. Если что, она только достигает своей наиболее интересной главы. Математические основы, заложенные в Частях I и II этой книги, алгоритмический инструментарий, разработанный в Части III, и передовые приложения, обозреваемые в Части IV, — всё это обеспечивает словарь, необходимый для внесения вклада. Открытые задачи, описанные в этой главе, — не повод для сожаления; они — возможности, которыми стоит воспользоваться. Мы надеемся, что эта книга дала вам инструменты, чтобы ими воспользоваться.

Приложение A

Математические основы

«Чудо о том, что язык математики так удивительно пригоден для формулировки законов физики, — это прекрасный дар, которого мы ни понять, ни заслужить не можем».

— Юджин Вигнер, 1960

Данное приложение содержит математические основы, предполагаемые на протяжении всей книги. Читатели с прочной базой бакалавриата в области математического анализа, линейной алгебры и теории вероятностей могут обращаться к нему как к справочнику при встрече с термином, используемым без определения. Читатели, впервые встречающие эти идеи, найдут здесь достаточно подробностей, чтобы следить за основным текстом, однако им рекомендуется дополнительно изучить специализированный учебник для более глубокого освоения.

A.1 Основы линейной алгебры

Векторы и матрицы

Вектор $x \in \mathbb{R}^n$ — это упорядоченный набор из n вещественных чисел. По умолчанию мы пишем столбцовые векторы; $x^\top$ обозначает строковый транспонированный. **Скалярное произведение** $x, y \in \mathbb{R}^n$ равно

$$\langle x, y \rangle = x^\top y = \sum_{i=1}^{n} x_i y_i.$$

Матрица $A \in \mathbb{R}^{m \times n}$ отображает $\mathbb{R}^n \to \mathbb{R}^m$ посредством $y = Ax$. **Транспонированная** $A^\top \in \mathbb{R}^{n \times m}$ удовлетворяет $(A^\top)_{ij} = A_{ji}$.

Определение A.1 (Нормы)**.** ℓ^p-норма вектора $x \in \mathbb{R}^n$ равна

$$\|x\|_p = \Big(\sum_{i=1}^{n} |x_i|^p \Big)^{1/p}, \qquad p \geq 1.$$

Частные случаи: $\|x\|_1 = \sum_i |x_i|$; $\|x\|_2 = \sqrt{\sum_i x_i^2}$ (евклидова норма, записываемая $\|x\|$ в тех случаях, когда нет неоднозначности); $\|x\|_\infty = \max_i |x_i|$. **Норма Фробениуса** матрицы равна $\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}$.

Собственные значения и спектральные свойства

Для квадратной матрицы $A \in \mathbb{R}^{n \times n}$ скаляр λ и ненулевой вектор v, удовлетворяющие $Av = \lambda v$, называются **собственным значением** и **собственным вектором**. Совокупность всех собственных значений называется **спектром** матрицы A.

Определение A.2 (Спектральный радиус)**.** Спектральный радиус матрицы A равен $\rho(A) = \max_i |\lambda_i|$, где $\lambda_1, \ldots, \lambda_n$ — собственные значения A.

Матрица A называется **положительно полуопределённой** (ПП) если $x^\top A x \geq 0$ для всех x, что равносильно тому, что все собственные значения неотрицательны. ПП-матрицы естественно возникают как ковариационные матрицы и гессианы в точках локального минимума.

Пример A.1 (Ковариационная матрица)**.** Пусть $X \in \mathbb{R}^{n \times d}$ — матрица данных со столбцами с нулевым средним. Эмпирическая ковариация $\Sigma = X^\top X / n$ является ПП. Её собственные векторы — это главные компоненты, используемые в PCA, а собственные значения измеряют дисперсию вдоль каждой компоненты.

Линейные системы и обратимость

Квадратная матрица A является **обратимой** (невырожденной) тогда и только тогда, когда $\det(A) \neq 0$, что равносильно тому, что все её собственные значения ненулевые. Если A обратима, единственное решение системы $Ax = b$ равно $x = A^{-1}b$.

В обучении с подкреплением системы вида $(I - \gamma P^\pi)V = R$ возникают из уравнений Беллмана (глава 4). Поскольку $\rho(\gamma P^\pi) = \gamma < 1$, матрица $I - \gamma P^\pi$ обратима, что гарантирует единственность функции ценности.

A.2 Математический анализ и оптимизация

Градиенты и гессианы

Пусть $f : \mathbb{R}^n \to \mathbb{R}$ дифференцируема. **Градиент** — это вектор частных производных:

$$\nabla_\theta f(\theta) = \Big(\frac{\partial f}{\partial \theta_1}, \dots, \frac{\partial f}{\partial \theta_n}\Big)^\top \in \mathbb{R}^n.$$

Градиент указывает в направлении наибольшего возрастания. **Гессиан** — это матрица вторых производных:

$$H(\theta) = \nabla^2 f(\theta), \qquad H_{ij} = \frac{\partial^2 f}{\partial \theta_i \partial \theta_j}.$$

Если f дважды непрерывно дифференцируема, то H симметрична.

Определение A.3 (Критическая точка)**.** θ^* является **критической точкой** f, если $\nabla f(\theta^*) = 0$. Это точка локального минимума, если $H(\theta^*)$ является ПП, и точка локального максимума, если $-H(\theta^*)$ является ПП.

Правило цепочки

Если $g : \mathbb{R}^m \to \mathbb{R}^n$ и $f : \mathbb{R}^n \to \mathbb{R}$, то правило цепочки даёт

$$\nabla_\theta (f \circ g)(\theta) = J_g(\theta)^\top \nabla_{g(\theta)} f,$$

где $J_g \in \mathbb{R}^{n \times m}$ — якобиан функции g. Автоматическое дифференцирование в системах глубокого обучения эффективно вычисляет это выражение посредством обратного распространения ошибки.

Градиентный спуск

Градиентный спуск итеративно выполняет

$$\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t),$$

где $\alpha > 0$ — **скорость обучения**. Обновление перемещает θ в направлении наибольшего убывания.

Стохастический градиентный спуск (SGD) заменяет истинный градиент зашумлённой оценкой $\hat{g}_t$, вычисленной на минибатче:

$$\theta_{t+1} = \theta_t - \alpha \hat{g}_t, \qquad \mathbb{E}[\hat{g}_t] = \nabla f(\theta_t).$$

Дисперсия $\hat{g}_t$ выступает как неявная регуляризация и может помогать выходить из острых локальных минимумов.

Adam (метод адаптивных моментов) поддерживает экспоненциальные скользящие средние градиента m_t и квадрата градиента v_t:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1)\hat{g}_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2)\hat{g}_t^2, \\ \theta_{t+1} &= \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}\,\hat{m}_t, \end{aligned}$$

где $\hat{m}_t = m_t/(1 - \beta_1^t)$ и $\hat{v}_t = v_t/(1 - \beta_2^t)$ — скорректированные оценки. Значения по умолчанию $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ хорошо работают в широком круге задач.

Выпуклость и сходимость

Определение A.4 (Выпуклость)**.** Функция $f : \mathbb{R}^n \to \mathbb{R}$ является **выпуклой**, если для всех x, y и $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

Она является **сильно выпуклой** с параметром $\mu > 0$, если $f(x) - \frac{\mu}{2}\|x\|^2$ выпукла.

Для выпуклой функции с L-липшицевым градиентом (т.е. $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$) градиентный спуск с размером шага $\alpha \leq 1/L$ сходится к глобальному минимуму. Скорость сходимости составляет $O(1/t)$ для выпуклых функций и $O(e^{-\mu t/L})$ для сильно выпуклых.

Пример A.2 (Минимизация квадратичной функции)**.** Пусть $f(\theta) = \frac{1}{2}\theta^\top A\theta - b^\top\theta$ при ПП матрице A. Тогда $\nabla f(\theta) = A\theta - b$, и обновление градиентного спуска принимает вид $\theta_{t+1} = \theta_t - \alpha(A\theta_t - b)$. Единственный минимум равен $\theta^* = A^{-1}b$. При $\alpha < 2/\lambda_{\max}(A)$ итераты сходятся геометрически.

A.3 Теория вероятностей

Случайные величины и распределения

Случайная величина X — это измеримая функция из вероятностного пространства Ω в $\mathbb{R}$. Для дискретной X **функция масс вероятности** (ФМВ) равна $p(x) = \mathbb{P}(X = x)$. Для непрерывной X **функция плотности вероятности** (ФПВ) p удовлетворяет $\mathbb{P}(a \leq X \leq b) = \int_a^b p(x)\,dx$.

Определение A.5 (Математическое ожидание и дисперсия)**.**

$$\mathbb{E}[X] = \sum_x x\,p(x) \quad \text{(дискретный случай)}, \qquad \mathbb{E}[X] = \int x\,p(x)\,dx \quad \text{(непрерывный случай)}.$$

$$\mathrm{Var}[X] = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Ковариация X и Y равна $\mathrm{Cov}[X, Y] = \mathbb{E}[(X - \mathbb{E}\,X)(Y - \mathbb{E}\,Y)]$.

Математическое ожидание линейно: $\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y]$ для любых констант a, b и случайных величин X, Y (независимо от их зависимости). Дисперсия квадратична: $\mathrm{Var}[aX] = a^2\,\mathrm{Var}[X]$, и для независимых X, Y выполняется $\mathrm{Var}[X + Y] = \mathrm{Var}[X] + \mathrm{Var}[Y]$.

Условная вероятность и теорема Байеса

Условная вероятность события A при условии B равна

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}, \quad \mathbb{P}(B) > 0.$$

Теорема A.1 (Теорема Байеса)**.**

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(B \mid A)\,\mathbb{P}(A)}{\mathbb{P}(B)}.$$

В непрерывном случае с априорным распределением $p(\theta)$ и правдоподобием $p(x|\theta)$:

$$p(\theta \mid x) = \frac{p(x \mid \theta)\,p(\theta)}{p(x)}.$$

Теорема Байеса лежит в основе байесовских стратегий исследования (апостериорное сэмплирование, глава 2) и моделирования вознаграждения в RLHF (глава 14).

Условное математическое ожидание и свойство башни

Условное математическое ожидание $\mathbb{E}[X \mid Y]$ — само по себе случайная величина — функция от Y. Его значение при $Y = y$ равно $\mathbb{E}[X \mid Y = y] = \int x\,p(x \mid y)\,dx$.

Теорема A.2 (Свойство башни / Закон полного математического ожидания)**.**

$$\mathbb{E}[X] = \mathbb{E}\big[\mathbb{E}[X \mid Y]\big].$$

Более общо, для σ-алгебр $\mathcal{F}_1 \subseteq \mathcal{F}_2$: $\mathbb{E}[\mathbb{E}[X|\mathcal{F}_2]|\mathcal{F}_1] = \mathbb{E}[X|\mathcal{F}_1]$.

Свойство башни используется в RL всякий раз, когда мы маргинализируем по будущим траекториям или вычисляем ожидаемый доход путём обусловливания на текущем состоянии.

Концентрационные неравенства

Теорема A.3 (Закон больших чисел)**.** *Пусть $X_1, X_2, \ldots$ — н.о.р. с математическим ожиданием μ. Тогда при $n \to \infty$:*

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i \xrightarrow{\text{п.н.}} \mu.$$

ЗБЧ обосновывает использование выборочных средних для оценки ожидаемого дохода в методах Монте-Карло (глава 5).

Теорема A.4 (Центральная предельная теорема)**.** *При тех же условиях, что и в Теореме A.3, и при $\sigma^2 = \mathrm{Var}[X_i] < \infty$:*

$$\sqrt{n}\,(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2).$$

ЦПТ даёт доверительные интервалы для оценок Монте-Карло и мотивирует гауссовскую модель шума в непрерывном управлении.

Пример A.3 (Оценка среднего дохода)**.** Предположим, что мы проводим n независимых эпизодов и записываем доходы $G^{(1)}, \ldots, G^{(n)}$. По ЗБЧ $\bar{G}_n \to V^\pi(s_0)$. По ЦПТ доверительный интервал уровня 95% приблизительно равен $\bar{G}_n \pm 1.96\,\hat{\sigma}/\sqrt{n}$, где $\hat{\sigma}^2$ — выборочная дисперсия доходов.

A.4 Методы выборки

Многие величины в RL — ожидаемые доходы, градиенты стратегии, KL-штрафы — не поддаются аналитическому вычислению, но могут быть *оценены* по выборкам. В этом разделе рассматриваются основные стратегии выборки.

Оценка методом Монте-Карло

Если математическое ожидание $\mu = \mathbb{E}_{X\sim p}[f(X)]$ трудно вычислить аналитически, **оценка Монте-Карло** определяется как

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} f(x^{(i)}), \qquad x^{(i)} \sim p.$$

По ЗБЧ $\hat{\mu}_n \to \mu$. Дисперсия оценки равна $\mathrm{Var}[\hat{\mu}_n] = \mathrm{Var}[f(X)]/n$, поэтому удвоение числа выборок вдвое уменьшает стандартную ошибку.

Методы Монте-Карло в RL (глава 5) используют полные доходы эпизодов как несмещённые (хотя и высокодисперсные) оценки функции ценности.

Выборка по значимости

Когда выборки из p недоступны, но доступны выборки из другого **предложения** q, **выборка по значимости** корректирует это с помощью отношения правдоподобий:

$$\mu = \mathbb{E}_{X\sim p}[f(X)] = \mathbb{E}_{X\sim q}\left[\frac{p(X)}{q(X)}\,f(X)\right].$$

Отношение $w(x) = p(x)/q(x)$ называется **весом значимости**. Оценка несмещена при условии $q(x) > 0$ там, где $p(x)f(x) \neq 0$, однако может иметь высокую (и даже бесконечную) дисперсию, если p и q сильно различаются.

Выборка по значимости применяется в обучении TD по внешней стратегии и занимает центральное место в целевой функции PPO (глава 9), где $q = \pi_{\theta_{\text{old}}}$ и $p = \pi_\theta$. Усечение отношения $\rho_t = \pi_\theta/\pi_{\theta_{\text{old}}}$ контролирует дисперсию.

Выборка с отклонением

Выборка с отклонением позволяет извлекать выборки из целевого распределения $p(x) \propto \tilde{p}(x)$ с помощью предложения $q(x)$ такого, что $\tilde{p}(x) \leq Mq(x)$ для всех x:

1. Извлечь $x \sim q$.
2. Принять x с вероятностью $\tilde{p}(x)/(Mq(x))$; в противном случае отклонить.

Вероятность принятия равна $1/M$, поэтому «плотное» предложение (малое M) критически важно для эффективности. Выборка с отклонением используется при анализе офлайновых моделей предпочтений (глава 14) и в некоторых схемах структурированного декодирования.

A.5 Теория информации

Теория информации количественно описывает неопределённость и стоимость передачи данных. В RL её концепции появляются при регуляризации вознаграждения, в задачах оптимизации стратегии и в анализе исследования.

Определение A.6 (Энтропия)**. Энтропия Шеннона** дискретного распределения p над алфавитом $\mathcal{V}$ равна

$$H(p) = -\sum_{x \in \mathcal{V}} p(x) \log p(x),$$

измеряемая в натах (основание e) или битах (основание 2). Энтропия максимальна для равномерного распределения и равна нулю для точечной меры.

Бонусы за энтропию $\mathcal{H}(\pi(\cdot|s))$ стимулируют исследование, штрафуя стратегии, вырождающиеся в детерминированные действия (глава 9).

Определение A.7 (Перекрёстная энтропия)**. Перекрёстная энтропия** между распределениями p и q равна

$$H(p, q) = -\sum_{x} p(x) \log q(x).$$

Она равна $H(p) + D_{\mathrm{KL}}(p \parallel q)$ и является стандартной целевой функцией при предобучении языковых моделей и дообучении с учителем.

Определение A.8 (KL-дивергенция)**. Дивергенция Кульбака-Лейблера** от q к p равна

$$D_{\mathrm{KL}}(p \parallel q) = \sum_{x} p(x) \log \frac{p(x)}{q(x)} \geq 0,$$

причём равенство выполняется тогда и только тогда, когда $p = q$ почти всюду. Заметим, что $D_{\mathrm{KL}}(p \parallel q) \neq D_{\mathrm{KL}}(q \parallel p)$ в общем случае; KL-дивергенция не является метрикой.

KL-штраф $\beta\, D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ в задачах RLHF (глава 14) и DPO (глава 12) штрафует за чрезмерное отклонение от эталонной стратегии, предотвращая взлом вознаграждения.

Определение A.9 (Взаимная информация)**. Взаимная информация** между случайными величинами X и Y равна

$$I(X; Y) = D_{\mathrm{KL}}(p(x, y) \parallel p(x)p(y)) = H(X) - H(X \mid Y) \geq 0.$$

Она измеряет уменьшение неопределённости об X после наблюдения Y.

Пример A.4 (RL с KL-регуляризацией)**.** Оптимальная стратегия для KL-регуляризованной задачи $\max_\pi \mathbb{E}_\pi[R] - \beta\, D_{\mathrm{KL}}(\pi \parallel \pi_{\mathrm{ref}})$ имеет замкнутую форму: $\pi^*(a|s) \propto \pi_{\mathrm{ref}}(a|s)\, e^{Q^*(s,a)/\beta}$, где Q^* — мягкая Q-функция. Это связано с выводом DPO в главе 12.

A.6 Операторы и сжимающие отображения

Доказательства сходимости динамического программирования и TD-обучения опираются на классический результат функционального анализа — теорему Банаха о неподвижной точке. В этом разделе развивается необходимая база и показывается, как оператор Беллмана вписывается в данную схему.

Метрические пространства и сжимающие отображения

Определение A.10 (Метрическое пространство)**. Метрическое пространство** (X, d) — это множество X, снабжённое функцией расстояния $d : X \times X \to [0, \infty)$, удовлетворяющей: (i) $d(x, y) = 0 \Leftrightarrow x = y$; (ii) $d(x, y) = d(y, x)$; (iii) $d(x, z) \leq d(x, y) + d(y, z)$ (неравенство треугольника). Метрическое пространство называется **полным**, если каждая последовательность Коши сходится в X.

Определение A.11 (Сжимающее отображение)**.** Оператор $T : X \to X$ на метрическом пространстве (X, d) является **сжимающим** с модулем $\kappa \in [0, 1)$, если

$$d(Tx, Ty) \leq \kappa\, d(x, y) \quad \text{для всех } x, y \in X.$$

Теорема A.5 (Теорема Банаха о неподвижной точке)**.** *Если T — сжимающее отображение на полном метрическом пространстве (X, d), то T имеет* единственную *неподвижную точку x^* (т.е. $Tx^* = x^*$), и для любой начальной точки $x_0 \in X$ итераты $x_{k+1} = Tx_k$ сходятся:*

$$d(x_k, x^*) \leq \frac{\kappa^k}{1-\kappa}\, d(x_1, x_0).$$

Сходимость геометрическая со скоростью κ.

Оператор Беллмана как сжимающее отображение

Обозначим через $\mathcal{B}(\mathcal{S})$ пространство ограниченных вещественнозначных функций на $\mathcal{S}$, снабжённое sup-нормой $\|V\|_\infty = \max_s |V(s)|$. Это полное метрическое пространство.

Определение A.12 (Операторы Беллмана)**.** Для фиксированной стратегии π **оператор оценки Беллмана** $\mathcal{T}^\pi$ отображает $V : \mathcal{S} \to \mathbb{R}$ в

$$(\mathcal{T}^\pi V)(s) = \sum_a \pi(a|s)\Big[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V(s')\Big].$$

Оператор оптимальности Беллмана $\mathcal{T}^*$ отображает V в

$$(\mathcal{T}^* V)(s) = \max_a \Big[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V(s')\Big].$$

Теорема A.6 (Сжимающее свойство Беллмана)**.** *Оба оператора $\mathcal{T}^\pi$ и $\mathcal{T}^*$ являются сжимающими на $(\mathcal{B}(\mathcal{S}), \|\cdot\|_\infty)$ с модулем γ.*

Доказательство. Докажем результат для $\mathcal{T}^*$; доказательство для $\mathcal{T}^\pi$ аналогично. Для любых $V, W \in \mathcal{B}(\mathcal{S})$ и любого состояния s:

$$\begin{aligned}
|(\mathcal{T}^* V)(s) - (\mathcal{T}^* W)(s)| &= \Big|\max_a [R(s,a) + \gamma (PV)(s,a)] - \max_a [R(s,a) + \gamma (PW)(s,a)]\Big| \\
&\leq \max_a \gamma \Big|\sum_{s'} P(s'|s,a)[V(s') - W(s')]\Big| \\
&\leq \gamma\, \|V - W\|_\infty,
\end{aligned}$$

где мы воспользовались неравенством $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$ и тем, что $P(\cdot|s,a)$ — распределение вероятностей. Взяв супремум по s, получаем $\|\mathcal{T}^* V - \mathcal{T}^* W\|_\infty \leq \gamma\, \|V - W\|_\infty$. Так как $\gamma < 1$, $\mathcal{T}^*$ является сжимающим. □

Следствие A.7 (Существование и единственность функций ценности)**.** *Для любого МДП с $\gamma < 1$ (или конечным горизонтом) функции ценности V^π и V^* существуют и являются единственными неподвижными точками операторов $\mathcal{T}^\pi$ и $\mathcal{T}^*$ соответственно. Начиная с любой ограниченной V_0, итераты $V_{k+1} = \mathcal{T}^\pi V_k$ (оценка стратегии) и $V_{k+1} = \mathcal{T}^* V_k$ (итерация ценности) сходятся геометрически:*

$$\|V_k - V^\pi\|_\infty \leq \gamma^k\, \|V_0 - V^\pi\|_\infty.$$

Этот результат обосновывает алгоритмы, разработанные в главах 4 и 6: каждый шаг оценки стратегии или итерации ценности уменьшает ошибку аппроксимации в γ раз.

Пример A.5 (Итерация ценности на двухсостоянии МДП)**.** Рассмотрим МДП с двумя состояниями $\mathcal{S} = \{s_1, s_2\}$, $\gamma = 0.9$ и оптимальными значениями $V^* = (10, 5)^\top$. Начиная с $V_0 = (0, 0)^\top$, после k итераций ошибка удовлетворяет $\|V_k - V^*\|_\infty \leq 0.9^k \cdot 10$. Для достижения ошибки ≤ 0.01 необходимо $k \geq \log(0.001)/\log(0.9) \approx 66$ итераций — точная гарантия сходимости, не зависящая от структуры вознаграждения.

Нерасширяющие операторы и приближённое ДП

При использовании аппроксимации функций (глава 7) проецированный оператор Беллмана $\Pi\mathcal{T}^\pi$ (где Π — ортогональная проекция на пространство аппроксимаций) в общем случае больше не является сжимающим. Теорема о неподвижной точке TD даёт более слабую, но всё же полезную гарантию: при линейной аппроксимации функций и определённых условиях выборки по собственной стратегии TD сходится к неподвижной точке, ошибка в которой ограничена кратным наилучшей достижимой ошибки аппроксимации.

Послесловие: будущее обучения с подкреплением

Если вы дочитали до этого места, значит, вы прошли путь от игрока у игрового автомата до алгоритмов, определяющих то, как передовые системы ИИ общаются с людьми. Этот путь оказывается удивительно прямым. Принцип оптимальности, сформулированный Ричардом Беллманом в 1950-х годах; уравнения временных разностей, разработанные Саттоном и Барто в 1980-х; теорема о градиенте стратегии, доказанная на рубеже тысячелетий — всё это не музейные экспонаты. Это живой механизм, работающий внутри систем, с которыми сотни миллионов людей взаимодействуют ежедневно. Редко когда разрыв между математической абстракцией и практической инженерией закрывался столь стремительно, и стоит остановиться, в конце этого пути, чтобы поразмышлять о том, куда движется область.

Обучение с подкреплением в центре ИИ. На протяжении большей части своей истории обучение с подкреплением занимало важный, но несколько периферийный угол в ландшафте машинного обучения. На практике доминировало обучение с учителем; RL было предметом красивой теории и впечатляющих демонстраций — Atari, Go, локомоция роботов — однако его влияние на развёрнутые системы оставалось ограниченным. Это кардинально изменилось.

Сегодня RL занимает центральное место по меньшей мере в четырёх наиболее активных направлениях исследований в области ИИ. В сфере *языка и рассуждения* выравнивание больших языковых моделей на основе обратной связи от людей и обучение моделей, способных последовательно рассуждать в рамках развёрнутых цепочек мыслей, оба зависят от оптимизации стратегий. В области *агентных систем* модели, которые планируют на несколько шагов вперёд, используют инструменты, просматривают веб и пишут и выполняют код, являются по существу RL-агентами, действующими в структурированных средах с отложенными вознаграждениями. В *мультимодальном ИИ* распространение обучения в стиле RLHF на зрение, аудио и видео — активно и стремительно развивающаяся область. В *робототехнике и воплощённом интеллекте* пайплайн sim-to-real, переносящий стратегии, обученные в симуляции, на физическое оборудование, остаётся одним из наиболее перспективных путей к созданию способных физических агентов. Во всех этих областях концептуальный словарь — именно тот, которым вы теперь владеете: состояния, действия, вознаграждения, стратегии, функции ценности и алгоритмы, их оптимизирующие.

Сближение двух традиций. Одним из наиболее поразительных интеллектуальных достижений прошедшего десятилетия стало открытие, что классическое RL и обучение больших языковых моделей не просто аналогичны, но глубоко едины. Задача дообучения языковой модели является марковским процессом принятия решений: состояние — это история разговора, действие — следующий токен, стратегия — языковая модель, а вознаграждение — сигнал — предоставленный людьми или полученный от модели — о качестве ответа.

Это осознание означает, что каждый результат данной книги применим в принципе к языковым моделям. Анализ смещения-дисперсии методов Монте-Карло в сравнении с методами временных разностей вновь возникает в дискуссиях о приписывании заслуг в длинных цепочках рассуждений. Теорема о градиенте стратегии лежит в основе оптимизации вознаграждения в стиле REINFORCE. Архитектура «актор-критик» является прямым предком пайплайнов RLHF на основе PPO, применяемых на практике. Уравнения Беллмана, записанные в эпоху исследования операций, теперь управляют обучением систем, которые сочиняют поэзию и доказывают теоремы.

Это сближение — не случайность. Оно отражает универсальность основы последовательного принятия решений и предполагает, что дальнейший прогресс в выравнивании языковых моделей будет частично связан с более глубоким погружением в теоретический инструментарий, накопленный RL за семьдесят лет.

Открытые вызовы. Было бы нечестно завершать на ноте чистого триумфа. Обучение с подкреплением, при всех недавних успехах, сталкивается с серьёзными нерешёнными проблемами, и интеллектуальная честность требует их назвать.

Выборочная эффективность остаётся фундаментальным ограничением. Люди научаются играть в большинство игр достаточно хорошо за несколько часов практики; лучшим RL-агентам требуются миллионы взаимодействий со средой. Преодоление этого разрыва — посредством более эффективного исследования, структурированных моделей мира или трансферного обучения — является одним из наиболее глубоких открытых вопросов в области.

Безопасность и робастность становятся всё более насущными. Взлом вознаграждения, когда агент находит непредусмотренные способы максимизировать некорректно заданный сигнал вознаграждения, — это не просто теоретический курьёз; он наблюдался в развёрнутых системах. Обеспечение надёжного поведения RL-агентов при изменении среды, несовершенстве модели вознаграждения или наличии противников — проблема как теоретической, так и практической важности.

Координация множества агентов вводит новый уровень сложности, с которым теория одного агента не справляется. Системы, состоящие из множества взаимодействующих агентов — будь то кооперирующие роботы или конкурирующие участники рынка — демонстрируют эмерджентную динамику, которая остаётся плохо изученной и трудно управляемой.

Масштабируемый надзор — пожалуй, наиболее самобытный современный вызов. По мере того как системы ИИ становятся более способными, люди-оценщики всё меньше способны надёжно судить о качестве их результатов. Как поддерживать значимый контроль со стороны человека над системой, рассуждения которой уже могут превосходить возможности оценщика по верификации? Конституциональный ИИ, дебаты и рекурсивное моделирование вознаграждения — ранние попытки ответить на этот вопрос, однако удовлетворительное решение остаётся недостижимым.

Спецификация вознаграждения лежит в основе всего вышесказанного. Проблема выражения человеческих ценностей, целей и предпочтений в виде функции вознаграждения, допускающей оптимизацию без нежелательных побочных эффектов, является по своей сути философской задачей в не меньшей мере, чем технической. Прогресс здесь потребует сотрудничества между сообществом RL, исследователями в области выравнивания, этиками и экспертами предметных областей.

Личное замечание. Написание учебника — это акт оптимизма. Он предполагает, что идеи, содержащиеся в нём, достойны передачи, что время читателя, потраченное на их изучение, потрачено хорошо, и что область станет лучше от того, что в ней появится больше людей, понимающих её глубоко. Мы искренне придерживаемся всех трёх этих убеждений.

Благодарим вас за время и внимание, вложенные в этот материал. Изучение обучения с подкреплением нетривиально: математика требовательна, интуиция временами контринтуитивна, а разрыв между теорией и работающей реализацией может смирять гордость. Если вы прошли этот путь, вы лучше большинства практиков подготовлены к тому, чтобы рассуждать вдумчиво о системах, которые будут определять облик наступающего десятилетия.

Мы надеемся, что вы используете эти знания не просто для создания более способных систем, но для создания лучших — систем, которые безопасны, интерпретируемы и подлинно полезны. Область нуждается в людях, которые серьёзно относятся и к возможностям, и к ответственности, и мы верим, что теперь вы готовы внести вклад в оба направления.

Заключительная мысль. В предисловии мы описали обучение с подкреплением как изучение того, как агент учится действовать правильно в мире, который он не вполне понимает, руководствуясь разреженным и порой вводящим в заблуждение сигналом вознаграждения. Это описание применимо с небольшими поправками к самому человеческому существованию. Возможно, именно поэтому эта концептуальная схема находит столь глубокий отклик и почему лучшие исследователи в области движимы чем-то большим, чем технический интерес.

Обучение с подкреплением в самых высоких своих амбициях — это теория о том, как разум может совершенствоваться через опыт. Если нам удастся всё сделать правильно — математику, инженерию и ценности, заложенные в сигналы вознаграждения, которые мы выбираем, — оно может оказаться одним из наиболее важных интеллектуальных вкладов нашей эпохи. Работа далека от завершения. Мы призываем вас продолжать там, где эта книга заканчивается.

Москва, 2026

Предметный указатель

μ-norm, 76
ε-greedy, 9
ε-optimal policy, 49
ε-soft policy, 48
ε-soft policy improvement theorem, 49
k-armed bandit, 7
10-armed testbed, 17

A2C, 106
A3C, 105
action space, 22
action value, 7
action-value estimate, 8
action-value function, 35, 47, 57
    Monte Carlo estimation, 47
actor, 101
actor-critic, 100
    two-timescale convergence, 112
Adam optimizer, 220
advantage function, 65, 82, 92, 100
    zero-mean property, 101
adversarial attack
    on RL, 214
agent, 5
agent–environment interface, 5
agentic RL, 187, 195
alignment, 213
Alpaca, 137
AlphaGo, 3
AlphaZero, 205, **205**
Anthropic, 183
applications, 4
asymptotically optimal, 15
asynchronous DP, 39
Atari, 74
AutoGen, 207
autoregressive language model, 133
autoregressive language modelling, 133

backup diagram, 27
Baird's counterexample, 80
    divergence, 80
Banach fixed-point theorem, 29, 33
base model, 136
baseline, 92
baseline theorem, 92
batch reinforcement learning, 214
batch TD(0), 57
Bayesian posterior, 12
behavior policy, 50
Bellman equation, 3, 26, 33, 54
    for Q_π, 27
    for V_π, 26
    worked example, 27
Bellman expectation equation, 33
Bellman operator, 29, 33, 56, 223
    contraction, 29
Bellman optimality equation, 28, 36
Bellman optimality operator, 37
benchmark contamination, 142
benchmarks, 142
best-of-N, 169
Beta-Bernoulli bandit, 12
bias–variance trade-off, 57
Blackjack, 46
    sample episode, 47
bootstrapping, 55
Bradley–Terry model, 121, 217

calculus, 220
catastrophic forgetting, 154
causal masking, 132
causality lemma, 89
cDPO (Conservative DPO), 152
centralised training with decentralised execution, **203**
chain-of-thought, 187, 189
chat template, 137
Chinchilla scaling laws, 134
Cliff Walking, 58
clinical trials, 16
Cohen's kappa, 175
CommNet, **206**
consistent algorithm, 14
Constitutional AI, 182
Constitutional AI (CAI), 140
constrained MDP, 214
contextual bandit, 15
continuing task, 2, 23
contraction, 223
contraction mapping, 33, 223
coverage assumption, 50
credit assignment
    agentic, 196
    in MARL, 208
critic, 101
cross-entropy loss
    language modelling, 133
CTDE, *см.* centralised training with decentralised execution
curse of dimensionality, 40, 74

DAPO, 198

data quality filtering, 140
datasets
    synthetic, 141
deadly triad, 68, 79, 217
debate (MARL), 207
Dec-POMDP, 203
Decision Transformer, 215
deep neural network, 80
DeepSeek, 183
DeepSeek R1, 191
delayed reward, 3
difference reward, 208
difficulty filtering, 191
direct alignment algorithm, 146, 147
Direct Preference Optimisation (DPO), 146
discount factor, 2, 22
discounted visitation measure, 90
distillation
    from teacher models, 139
    reasoning, 192
distribution shift, 3
    DPO, 152
    offline RL, 214
distributional robustness, 214
Double DQN, 82
Double Q-learning, 60
DPO, 3
    distribution shift, 162
DPO loss, 148
    gradient, 148
DQN, 3
Dreamer, 212
dueling network architecture, 82
dynamic programming, 3, 33

eigenvalue, 219
eligibility trace, 63
    replacing, 63
ellipsoidal exploration bonus, 16
Elo rating, 121
embodied RL, 215
emergent capabilities, 136
emergent language
    compositionality, 206
entropy, 223
environment, 5
episodic task, 2, 23
evaluation
    language models, 142
evaluative feedback, 2
every-visit Monte Carlo, 44
Evol-Instruct, 139
expected reward, 23
Expected SARSA, 59
experience replay, 80
    GRPO, 171
exploration
    absence in DPO, 153
exploration bonus, 10
exploration versus exploitation, 3
exploring starts, 48
exposure bias, 137

far-sighted agent, 24
feature map, 75
feed-forward block, 132
few-shot RL, 213
fictitious self-play, 205
finite MDP, 33
first-visit Monte Carlo, 44
Fisher information matrix, 95
fitted value iteration, 84
forgetting
    in self-play, 205
formal verification, 214
foundation model
    RL for, 216
fully cooperative game, 202
function approximation, 73
function calling
    format, 194

GAE, 66, *см.* Generalized Advantage Estimation
    language models, 178
general-sum game, 202
Generalised Advantage Estimation (GAE), 65
Generalized Advantage Estimation (GAE), 104
Generalized Policy Iteration, 33
Generalized Policy Iteration (GPI), 39
Go (game), 205
Goodhart's Law, 127
Goodhart's law, 213
GPI
    competing processes, 39
gradient, 220
gradient descent, 220
gradient-TD methods, 83
greedy action, 8
greedy policy, 36
gridworld, 34
Group Relative Policy Optimisation, *см.* GRPO
group sampling, 163
group-relative advantage, 163
GRPO, 3, 161, 197
    advantage, 163
    clipped objective, 163
    connection to REINFORCE, 166
    group size, 167
    history, 162
    KL penalty, 163
    motivation, 162
    notation, 163
    objective, 164
    practical use, 183
    training at scale, 170
    unbiasedness, 166
    variance reduction, 166
    vs RLOO, 166
GRPO (Group Relative Policy Optimisation), 156

helpfulness-harmlessness tradeoff, 182

Hessian, 220
history of RL, 3
Hoeffding's inequality, 10
human evaluation, 142
hypernetwork, 203

IGM condition, 203
imitation learning, 3
importance ratio, 106
importance sampling, 50, 222
importance sampling ratio, 50
in-place DP, 39
incremental update, 9
independent Q-learning, 208
inference-time scaling, 192
inference-time search, 190
information theory, 223
InstructGPT, 182
inter-annotator agreement, 175
intrinsic motivation
    social, 209
IPO (Identity Preference Optimisation), 150
iterative DPO, 155

Jack's Car Rental, 40
joint policy, 202

KL divergence, 14, 223
    explosion, 180
KL penalty
    implementation, 178
KL regularisation, 127
    optimal policy, 127
KTO (Kahneman-Tversky Optimisation), 151
KTO loss, 151

Lai–Robbins lower bound, 14
    Gaussian case, 15
λ-return, 62
language model
    as MDP, 134
language models
    state space, 74
large language model, 206
Law of Effect, 3
layer normalisation, 132
likelihood-ratio trick, 88
linear algebra, 219
linear TD(0)
    convergence, 75
linear value function approximation, 75
LinUCB, 16
Llama 2, 183
LLM-as-judge, 140, 142
logits, 133
LSTM, 206

majority voting, 193
MAPPO, **204**
Markov Decision Process, 21
Markov game, **201**
Markov property, 21
    joint reward-state, 22
maximisation bias, 59, 82
MCTS, 205
MDP
    definition, 22
mean-squared value error, 77
minimax theorem, 205
mixed precision, 179
mode collapse, 180
mode-seeking
    DPO, 153
model collapse, 141
model-based reinforcement learning, 212
model-free, 43
Monte Carlo, 222
Monte Carlo control
    exploring starts, 48
    off-policy, 52
    on-policy, 48
Monte Carlo methods, 43
Monte Carlo prediction, 43
multi-agent reinforcement learning, **201**
multi-armed bandit, 7
MuZero, 212
myopic agent, 24

n-step return, 61
Nash equilibrium, **202**
natural gradient, 95
natural policy gradient, 95
news recommendation, 16
non-stationarity
    in MARL, **208**
non-transitive strategies, 205

ODE method, 78
off-policy, 39, 43, 59
off-policy Monte Carlo, 50
offline MARL, 209
offline reinforcement learning, 214
offline-to-online RL, 215
on-policy, 39, 43, 58
    alignment, 154
one-step target, 54
online advertising, 16
online RL
    production systems, 153
OpenThoughts, 141
optimal policy, 28
    existence, 28
optimal value function, 28
optimality condition, 36
optimisation, 220
optimism in the face of uncertainty, 10
ORPO (Odds Ratio Preference Optimisation), 152
ORPO loss, 152
outcome reward model (ORM), 123
overthinking, 192

PAC-MDP, 216

pairwise accuracy, 126
parameter sharing, 204
perplexity, 134
persona-based generation, 140
personalised medicine, 16
pipeline parallelism, 181
policy, 2, 24
  deterministic, 24
  stochastic, 24
policy evaluation, 33
policy gradient, 87
  basic estimator, 89
  estimators comparison, 97
  objective, 88
  reward-to-go form, 89
policy gradient theorem, 88, 90
policy improvement, 35
policy improvement theorem, 35, 36
policy iteration, 36
  convergence, 36
policy ordering, 28
polynomial features, 75
POMDP, 22
population-based training, 205
positional encoding, 132
post-training pipeline, 137
potential function, 118
PPO, 107
  clipped objective, 107
  clipping cases, 108
  critic cost, 161
  reasoning, 197
  vs DPO online, 155
pre-training, 135
preference data, 120
  annotation formats, 175
  pitfalls, 176
preference data synthesis, 140
preference dataset, 120
  DPO requirements, 150
prioritised experience replay, 83
prioritized sweeping, 40
PRM800K, 124
probability matching, 12
probability theory, 221
process reward model, 193
process reward model (PRM), 124
projected Bellman equation, 76
projection operator
  μ-weighted, 76
prompt, 134
prospect theory, 151
pseudo-regret, 8
PUCT, 205
pure greedy, 8

Q(λ), 65
Q-learning, 3, 58
  convergence, 59
QMIX, **203**
  monotonicity, 203
Quiet-STaR, 192

random walk example, 56
real-time DP, 40
reasoning trace, 189
red-teaming, 182
reference policy, 110
referential game, 206
regret, 8
regret bounds
  deep RL, 216
regret decomposition, 8
REINFORCE, 90, 199
  convergence, 96
  with baseline, 93
  worked example, 91
REINFORCE Leave-One-Out, *см.* RLOO
reinforcement learning, 2
Reinforcement Learning from Human Feedback, *см.* RLHF
Reinforcement Learning with Verifiable Rewards, *см.* RLVR
rejection sampling, 169, 222
replay buffer, 80
representation error, 75
response, 134
return, 2, 23, 43
  recursive property, 24
  worked example, 24
reward hacking, 110, 119, 120, 147, 213
  symptoms, 179
reward model, 110
  architecture, 176
  cost of training, 146
  evaluation, 143, 177
  overfitting, 177
  retraining, 177
reward model loss, 122
reward overoptimisation, 127, 188
reward shaping
  in MARL, 209
  Ng-Harada-Russell theorem, 118
  potential function, 118
  shaping theorem, 118
reward-to-go, 89
RL theory, 216
RLAIF, 140
RLHF, 3, 110, 187
  four-model setup, 177
  generation phase, 181
  RLOO, 95
  statistical complexity, 217
  training instability, 180
RLOO, 93
  estimator, 93
  RLHF connection, 95
  unbiasedness, 94
  variance, 95
RLVR, 161, 187

definition, 167, 188
objective, 188
Robbins–Monro conditions, 56
robotics
state space, 74
robustness, 214

safe exploration, 214
safe reinforcement learning, 214
safety constraints
online RL, 155
sample average, 9
sample efficiency, 212
sampling methods, 222
SARSA, 57
convergence, 58
SARSA(λ), 65
scalable oversight, 213
scaling laws, 134
scientific discovery
RL for, 216
score function identity, 89
scratchpad, 190
self-attention, 131
Self-Instruct, 139
self-play, **205**
semi-gradient method, 78
semi-gradient TD(0), 75
sequential social dilemma, 206
sim-to-real gap, 215
sim-to-real transfer, 215
SMAC, 204
social dilemma, 203, 206
STaR, 192
state space, 22
step size, 9
stochastic game, 201
stopping criterion, 34
student MDP, 29
sub-optimality gap, 8
supervised fine-tuning (SFT), 136
supervised learning, 3
surrogate objective, 106
SWE-bench, 196
SWiRL, 195
sycophancy, 120
synchronous DP, 34
synthetic data, 138

target network, 81
target policy, 50
TarMAC, **206**
TD error, 55
as advantage estimate, 102
zero-mean property, 55
TD fixed point, 76
approximation error, 77
existence, 77
TD(λ), 3
TD(0), 54
convergence, 56
TD(lambda)
backward view, 64
forward view, 63
forward-backward equivalence, 64
TD-Gammon, 3
teacher forcing, 137
temperature, 133
tensor parallelism, 181
terminal state, 23
thinking tokens, 190
Thompson Sampling, 12
worked example, 13
tile coding, 75
Tit-for-Tat, 206
token budget, 190
token-level MDP, 134
tool use, 187, 194
trajectory, 5
transfer learning
in RL, 213
Transformer architecture, 131
transition kernel, 22
transition probability, 23
TRPO, 107
trust region, 107
monotone improvement bound, 107
Tülu, 137

UCB, 10
UCB1, 10
worked example, 10
UltraFeedback, 140
unsupervised learning, 3
Upper Confidence Bound, 10

value function
action-value, 25
state-value, 25
V-Q relationship, 25
worked example, 25
value iteration, 37
value specification, 213
verbosity bias, 120
verification function, 188
verifier, 167
vision-language-action model, 215
vocabulary, 133

WizardLM, 139
world model, 212

zero-sum game, 202

MOUNTAIN AI ▪ ТОМ I

ОБ ЭТОЙ КНИГЕ

Учебник, охватывающий обучение с подкреплением от первых принципов до современных методов выравнивания языковых моделей.

Книга предназначена для аспирантов, ML-инженеров и исследователей, ищущих единое изложение классической теории RL и современных техник постобучения после крупных языковых моделей.

В КНИГЕ РАЗОБРАНЫ

- Многорукие бандиты, MDP, уравнения Беллмана
- Динамическое программирование, Монте-Карло, TD-обучение
- Глубокий RL: DQN, REINFORCE, A2C, PPO, GAE
- Постобучение языковых моделей: SFT, RLHF, DPO, GRPO
- Мульти-агентные системы и фронтиры исследований

К. К. ПЧЕЛИН

Лекции по машинному обучению ▪ Москва, 2026

github.com/pyshka501/rl-textbook ▪ t.me/mountainai_info